

微网与外部电网电力调控策略

摘要

针对含光伏发电与储能装置的社区微网购电调度问题，考虑负荷、光伏出力及电价在不同决策时点下的不确定性，本文以实际结算成本最小为目标，以 10 min 为调度粒度，构建日前购电决策、日内滚动执行与跨日状态传递相结合的协同优化框架，并基于 2025 年数据开展因果回测。

针对问题一，在预测信息已知条件下，构建日前购电与储能协同调度的确定性线性规划模型。模型综合考虑供需平衡、储能充放电效率、SOC 边界、充放电功率及首末储能状态一致性等约束，采用 HiGHS 求解日前最优购电与储能计划。结果表明，购电费用为 33801.50 元，较无储能方案节省 14250.55 元，降幅为 29.66%；储能能够通过峰谷套利和提高光伏消纳水平降低购电成本。

针对问题二，在预测偏差引起的计划购电与高价紧急购电权衡条件下，构建基于 K-medoids 场景压缩、风险储备和滚动模型预测控制的日前与日内协同调度模型。日初利用历史信息生成计划购电额度，日内依据已观测误差滚动修正剩余时段的储能决策与紧急购电量，并以 8 个代表场景刻画负荷与光伏预测误差。对 2025 年 2 月至 12 月共 334 日进行因果回测，实际购电总成本为 15166538.46 元，其中计划购电成本为 11883687.24 元，紧急购电成本为 3282851.22 元。该模型实现了计划购电成本与供电缺口风险之间的协调。

针对问题三，在多时点光伏预测下既有购电承诺动态调整条件下，构建包含日初承诺、信息价值评估及剩余时段重优化的滚动购电调整模型。在 6:00、12:00 和 18:00 获取新预测后，模型锁定已执行时段，仅当更新预测带来的预期收益超过调整费用时，才修改未执行时段的购电计划。全年连续运行结果表明，价值触发调整策略的总成本为 16373508.75 元，较不调整策略降低 736059.50 元，降幅为 4.30%，表明该策略在本回测期内能够降低实际结算成本。

针对问题四，在实时波动电价条件下，基于由负荷、光伏和电价构成的三元联合残差场景，构建日前决策与日内滚动执行相结合的优化模型。日前阶段利用代表场景制定购电额度，日内阶段依据实际价格和已观测信息滚动更新剩余时段决策。结果表明，日前额度策略在 2 月至 12 月的实际结算成本为 15257873.75 元；多时点价值触发调整策略的同期成本为 15346418.14 元。由于两类策略的购电承诺、调整费用及紧急购电结算机制不同，比较时应在统一信息边界与结算区间下进行。

最后，对各模型进行核验，并评估模型优缺点。

关键词：社区微网；储能调度；线性规划；模型预测控制；因果回测；购电调整

一、问题重述

1.1 问题背景

微网由用电负荷、光伏发电单元、储能装置和外部电网组成。光伏出力与负荷需求在日内并不完全匹配，储能可通过充放电将电能转移至其他时段，外部电网则承担补充供电的作用。因此，需要结合电价、负荷、光伏预测与实际运行信息，统筹外网购电和储能调度，在保障负荷供电的前提下控制购电成本。随着负荷、光伏和电价随时间变化，日前计划还须与日内预测更新及实际运行相衔接。

1.2 问题一的要求

根据给定的日内电价、负荷预测功率和光伏预测功率，将一天按附件原始顺序划分为 144 个 10 分钟时段，制定日前外网购电与储能充放电计划。在满足各时段能量平衡、储能充放电功率限制、储电量上下界以及首末储电量一致等条件下，使全天计划购电费用最小。给出逐时段购电量，汇总指定时段购电量、全天购电量与购电费用，并列出六个四小时区间的充放电量及日初、日末储电量，按附件模板填报。

1.3 问题二的要求

在问题一的单日调度基础上，进一步考虑全年运行中负荷与光伏预测存在偏差的情形。每天 0:00 应依据当时可获得的信息制定当天普通购电计划；进入日内运行后，根据实际负荷、实际光伏出力和当前储能状态安排实际取电与储能充放电。当计划购电、光伏和储能仍不能满足负荷需求时，需按当时电价的 5 倍紧急购电。要求在保持储能状态跨日连续、满足设备运行约束且不利用未来实际信息的条件下，形成日前计划与日内执行相联动的购电策略，并按附件模板给出计划购电量、充放电量、日初与日末储电量以及紧急购电记录。

1.4 问题三的要求

每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率预报。要求先利用 0:00 的预报制定当日计划购电策略，再在后续预报时刻结合最新预报，对尚未执行时段的购电策略进行调整。计划购电量高于调整购电量的部分，按交易时刻电价的 50% 支付违约费用；调整购电量高于原计划的部分，超出量按交易时刻电价的 1.5 倍计费；总购电费用还应计入紧急购电费用。基于题给电价、负荷与光伏实际数据及多时点光伏预报，建立当天购电与储能调度模型，给出指定日期的计划、调整、充放电和紧急购电结果，按模板保存完整策略至 `result3.xlsx`，并分析是否需要利用后续预报时刻调整策略。

1.5 问题四的要求

进一步考虑外部电网电价在日内实时波动的情形。基于小区负载和光伏实际数据、光伏发电预报以及附件给出的分时电价，重新建立并计算问题二的日前计划与紧急购电策略，以及问题三的多时点预报更新与购电调整策略。论文应呈现波动电价下两类策略的相应计算结果，并严格沿用题目结果模板，分别将完整结果保存至 `result4-2.xlsx` 和 `result4-3.xlsx`。

二、模型假设

下列边界将题设条件与为求解所作的解释分开；模型性解释均以后文非前瞻回测和物理审计核验。“因果”只表示决策不使用决策时点后的信息，并非统计因果识别。

2.1 共同物理边界

- A1: 题设设备与时间。一天含 $T = 144$ 个 10 分钟时段， $\Delta t = 1/6\text{h}$ ；功率统一换算为能量 $L_t = \ell_t \Delta t$ 、 $P_t = \rho_t \Delta t$ 。储能母线侧往返效率为 90%，取 $\eta_c = \eta_d = \sqrt{0.9}$ ， $E_t \in [1200, 10800]$ kWh，充、放电单时段能量均不超过 $5000\Delta t$ kWh。
- A2: 状态与能量流。2025-01-01 0:00 的 SOC 为 6000 kWh，跨日满足 $E_{d+1,0} = E_{d,T}$ ，不强制日末复位。购电、紧急购电、光伏与放电在公共母线供能；负荷、充电与弃光取能，普通购电额度不能直接跨时段挪用。
- A3: 适用范围。题目未给出售电价、PCC 容量、需量费、退化费和辅助功耗，主模型不引入这些量，也不另设购电功率上限；这限定了结论的工程边界。
- A4: 求解与审计。主模型为 LP。在正电价、无售电收益且有损耗时，同时充放电被更低损耗解支配；逐时段审计 $c_t r_t = 0$ ，必要时以词典序次目标或 MILP 互斥复核。

2.2 各问信息、结算与输出边界

- Q1: 日初一次性使用题给负荷、光伏预测制定全天计划，日内不更新；结果仅是确定性单日基准，不能与 Q2–Q4 的非前瞻策略混同。
- Q2: 日初锁定 $q_{d,t}$ ，实际取用 $0 \leq x_{d,t} \leq q_{d,t}$ ，并按 $p_t q_{d,t} + 5p_t e_{d,t}$ 结算，未用额度仍计费。预测和联合残差仅取目标日前已结束记录；每个 (m, α, K) 候选均以“情景计划—锁定 q —10 分钟因果 MPC—实际结算”的连续 SOC 闭环评分。 $K = 8$ 是该候选集下的经验验证折中，而非全局最优情景数。1 月用于历史积累和 SOC 预热，题设工作簿及相应正文结果仅覆盖 2–12 月 334 日。
- Q3: 0:00、6:00、12:00、18:00 接收预报，后三次更新从 6:10、12:10、18:10 生效且不改写已执行段；最终承诺 g 只相对日初 g^0 结算一次， $\phi_t = p_t g + \frac{1}{2} p_t |g - g^0|$ 。日前负荷预测、光伏锚点插值和价值触发均只使用当时信息；终端价值不进入日账单，A 边界（年末 1200 kWh）为主口径，B 边界（6000 kWh）只作敏感性。
- Q4: 实际 $p_{d,t}$ 是交付时段价格而非日前锁价：Q4-2 按 $p_{d,t} q_{d,t} + 5p_{d,t} e_{d,t}$ 结算，Q4-3 的

调整费也取交付时段价。价格预测训练集严格早于决策时点，联合情景沿用同一历史日的 (r^L, r^P, r^p) 配对；仅在完成闭环校准、价格误差和物理/因果审计后采用正式结果。

三、符号说明

下文只列出四问共用的核心符号；各问特有的情景索引、校准块编号、策略代号与算法中间量在相应小节首次出现时定义。

符号	含义	单位
d, t	日期、时段索引	—
$p_{d,t}$	交付时段购电价	元/kWh
$L_{d,t}, P_{d,t}$	负荷、光伏电量	kWh
$\hat{L}_{d,t}, \hat{P}_{d,t}$	负荷、光伏预测	kWh
$E_{d,t}$	时段末储电量	kWh
$c_{d,t}, r_{d,t}$	充电量、放电量	kWh
$w_{d,t}$	弃光量	kWh
η_c, η_d	充、放电效率	—
$q_{d,t}$	日初锁定的普通购电额度	kWh
$x_{d,t}$	普通购电实际取用量	kWh
$e_{d,t}$	紧急购电量	kWh
$g_{d,t}^0, g_{d,t}$	原计划、有效普通承诺	kWh
$\phi_{d,t}(g^0, g)$	一次性调整结算费用	元
$V_{d+1}(\cdot)$	日末储能机会成本	元
C_d	第 d 日实际购电总成本	元

问题二至问题四的结果表只覆盖 2 月 1 日至 12 月 31 日，共 334 日。1 月用于积累历史数据和预热 SOC，不写入题设工作簿；问题一则对应单日 144 时段计划。文中成本均注明是输出区间还是 1–12 月连续运行口径。

四、问题分析

四问共用能量平衡、储能状态和结算台账，差别在于可用信息逐步增加：Q1 给出确定性日前基准；Q2 加入预测偏差和额度锁定；Q3 在新预报到达后调整未执行承诺；Q4 再将时变价格纳入信息集。递进关系和共同验收流程见图 1。

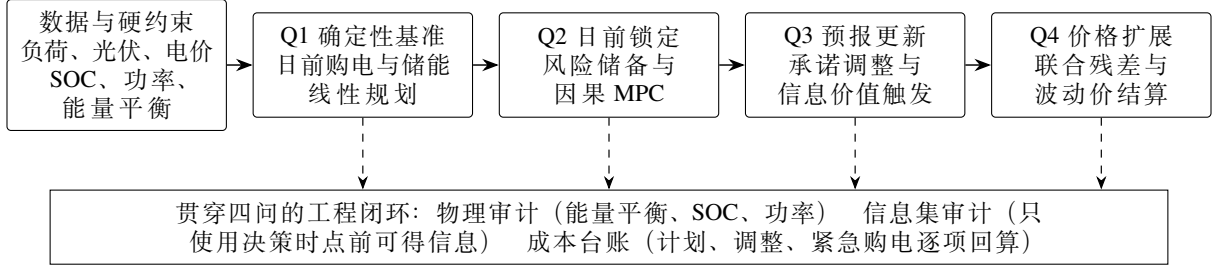


图 1 四个问题的递进建模路线与统一工程闭环

4.1 问题一的分析

本问的电价、负荷预测和光伏预测在日初均已给定，因此只需完成一个确定性日内优化。先将功率乘以 $\Delta t = 1/6 \text{ h}$ 换算为 10 min 时段能量；再以公共母线为计量边界，写出负荷、光伏、外网购电和储能的逐时平衡及含损耗的 SOC 递推。首末 SOC 相等，避免以透支日末储能压低当天成本。

目标函数和全部约束均为线性形式，且决策变量连续，故可建立确定性线性规划。储能的经济作用是将低价时段或光伏富余时段的能量转移至高价时段：只有当峰谷价差能够覆盖往返损耗时，跨时段套利才有意义。与需要离散启停决策的模型相比，本问无需引入二元变量，求解器可直接给出全局最优解和原始—对偶最优性证书。

4.2 问题 2 的分析

问题二的困难在于，日前必须锁定各时段的普通购电额度，而负荷和光伏偏差会在日内逐步揭示。若只按均值预测购电，正净负荷偏差可能迫使系统以 5 倍价格紧急购电；若机械地扩大计划额度，又会因未取用额度仍按计划价格结算而增加成本。因此，预测的评价不能只取负荷或光伏的 MAE，而应以实际计划费和紧急购电费之和衡量。

本文把一天分为“日前计划”和“日内执行”两个信息阶段。每日 0:00，仅由此前已经结束的日期构造负荷、光伏预测和残差池，并据此锁定唯一的普通购电额度 $q_{d,t}$ ；在每个 10 分钟时段结束后，实际负荷、光伏和 SOC 才进入信息集，控制器仅重算下一步储能动作。由此，历史残差可用于描述未来偏差的形态，却不能作为当天已经确定的未来路径直接执行。

为同时保留负荷与光伏误差的时序结构并控制求解规模，本文以完整历史残差日为样本，经 K-medoids 选取代表轨迹；K 只作为场景压缩规模，不被解释为真实概率分布的精确离散化。日前层以残差分位数形成必要的净负荷储备，下限只作用于相应时段的 $q_{d,t}$ ，而不人为放大全日购电量。日内则依据已观测残差前缀对代表轨迹重新加权，配合滚动 MPC 决定充放电和紧急购电。

这一结构的关键是“政策一致性”：预测结构和风险分位的选择，也必须经历同样的日前锁定与逐时段 MPC 过程，不能把整日真实负荷、光伏交给事后全视域调度后再评分。这样，少买计划电和避免 5 倍紧急购电由同一个货币目标自然权衡，结果可被解释为题设信息条件下的因果回测，而非预知未来的事后最优。

4.3 问题 3 的分析

问题三在问题一的日前购电与储能调度框架上引入了三次新的光伏预报。难点不在于每次都重新求解，而在于区分“已执行的物理状态”和“尚未执行的购电承诺”：后续预报只能改变未执行时段的普通购电量，且调整收益必须足以覆盖下调违约费或上调溢价。若忽略这一不可逆性，模型会把已发生时段当作可回溯决策，从而高估预报更新的价值。

据此，本文采用“日前承诺-信息更新-实时执行”的两层滚动结构。每日 0:00 先以当时发布的光伏预报和因果负荷预测制定基准承诺；在 6:00、12:00 和 18:00，以已观测的储电量为初始状态，仅对剩余时段比较“维持原承诺”和“允许调整”的未来成本。以二者的差额刻画新预报的信息价值，只有该价值超过求解器货币容差时才实施调整。这样既能直接回答“是否需要利用后续预报”，又避免为极小的数值差异频繁改变计划。

光伏预报是逐小时数据，而调度时段为 10 分钟。每次更新先用该时点已观测的实际光伏作为锚点，再与后续整点预报进行因果线性插值，从而保留预报曲线的日内变化，又不读取未来实际光伏。题目未单列负荷预报，故在每日 0:00 仅以前已结束的最近至多四个同星期日逐时段均值构造负荷预测；日内已执行时段由真实观测闭合，未来时段仍使用同一日前预测。该信息口径使计划、更新和实际执行可严格区分。

各次优化的目标、母线平衡和储电量递推均为线性关系，调整费的绝对值可线性化，因此主模型为线性规划。为防止末时段通过耗尽储能虚减当日账单，在目标中加入由下一日因果信息估计的储电量终端价值。求解后从购电承诺锁定、能量平衡、储电量边界、充放电互斥和调整费用五方面核验；最终再以不更新策略和三个单更新时点策略为对照，识别后续预报的总体价值及关键更新时点。

4.4 问题 4 的分析

问题四在问题二、三的已验收框架上引入附件 4 的交付时段时变电价。难点不再只是负荷与光伏偏差，同一物理缺口在不同时段还有不同价格权重：把价格当作 0:00 已知常数会错估“提前购电—储能转移—后续释放”的价值；假定 0:00 已知全天实际价同样不可执行。

因此，问题四的核心变化是因果价格信息，而不是重新放宽问题二、问题三已经建立的负荷与光伏非前瞻性边界。波动电价下的日前额度策略（Q4-2）在联合残差日前评估与逐十分钟 MPC 上，把历史价格残差与负荷、光伏残差同步配对；波动电价下的多时点调整策略（Q4-3）沿用全时点价值触发规则，把调整结算与紧急购电改为按交付时段实际价格计价。两条链共享同一价格预测模块，但分别对应题设的两类结果。

与问题二、问题三相同，1 月仍只作预测历史与 SOC 预热，题设输出区间为 2-12 月。两类策略均在相同设备边界、信息时序和输出区间下核算，避免把预热成本或不同价格机制下的差额误作策略优劣。

五、模型的建立与求解

5.1 问题一：确定性日前购电与储能优化

5.1.1 模型建立

题给储能充放电效率为 90%。本文按往返效率作对称等效拆分, 令 $\eta_c = \eta_d = \sqrt{0.9} \approx 0.9487$, 从而 $\eta_c \eta_d = 0.9$ 。往返效率为可用输出能量与输入能量之比 [1]; 实际大型储能系统的效率会随功率和运行状态变化 [2], 故固定效率是与题目数据粒度相匹配的近似。

以全天计划购电费用最小为目标:

$$\min C_1 = \sum_{t=1}^T p_t g_t. \quad (1)$$

逐时能量平衡和储电量递推分别为:

$$g_t + P_t + d_t = L_t + c_t + s_t, \quad t = 1, \dots, T, \quad (2)$$

$$E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, \dots, T. \quad (3)$$

储能容量、循环边界、充放电功率及非负约束为:

$$\begin{aligned} 1200 &\leq E_t \leq 10800, \quad t = 1, \dots, T, \\ E_0 &= E_T = 6000, \end{aligned} \quad (4)$$

$$\begin{aligned} 0 &\leq c_t \leq 5000\Delta t, \quad t = 1, \dots, T, \\ 0 &\leq d_t \leq 5000\Delta t, \quad t = 1, \dots, T, \end{aligned} \quad (5)$$

$$\begin{aligned} g_t &\geq 0, \quad t = 1, \dots, T, \\ 0 &\leq s_t \leq P_t, \quad t = 1, \dots, T. \end{aligned} \quad (6)$$

式(2)保证公共母线侧供需平衡, 式(3)把充电损耗和放电损耗计入储电量递推; 式(4)将储电量控制在额定容量的 10% ~ 90%, 并令日末状态回到日初水平; 式(5)由 $5000 \text{ kW} \times 1/6 \text{ h}$ 得到单时段充放电上限, 即 $833.333 \dots \text{ kWh}$ 。若在低价时段 i 充电并在高价时段 j 放电, 经济性条件为 $p_j \eta_c \eta_d > p_i$ 。

以逐时能量平衡和储能状态递推描述社区微网的能量调度, 是相关优化研究中的常见建模方式 [5]; 本文只采用其模型结构, 不移植文献中的设备或市场参数。图2说明各能量变量与公共母线计量边界的对应关系; 本问外网购电量记为 g_t 。

微网能量流与公共母线计量边界

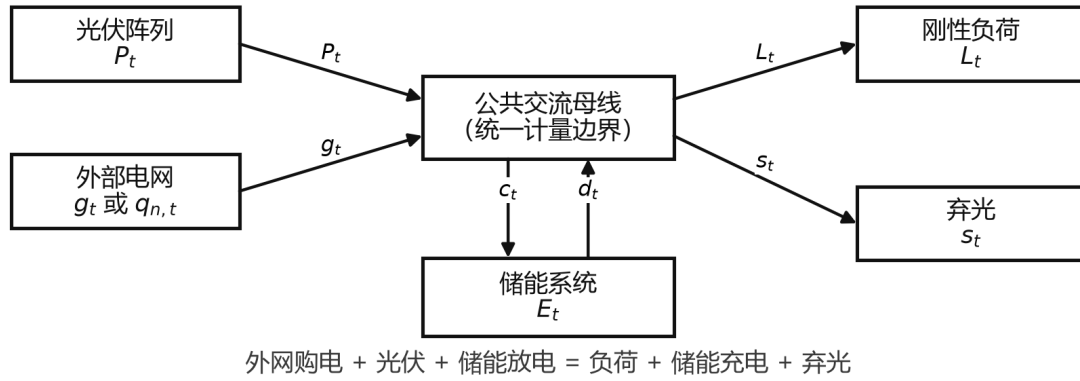


图2 微网能量流与公共母线计量边界

5.1.2 模型求解

模型的目标函数、能量平衡、储电量递推和变量边界均为线性关系，因此将 144 时段变量和约束组装为稀疏矩阵，调用 HiGHS 求解。HiGHS 提供高性能线性规划算法 [3]，求解接口可返回求解状态、原始残差和对偶信息。数值解中 $\max_t(c_t d_t) = 0$ ，说明本算例的连续 LP 解满足充放电互斥要求；其与显式互斥模型的对照结果在结果分析中给出。

HiGHS 稀疏线性规划算法

算法步骤

Step 1: 读取并校验数据

按附件原始行序读取 144 个时段，核查电价、负荷和光伏数据的完整性与量纲。

Step 2: 完成单位换算

将负荷功率和光伏功率乘以 $\Delta t = 1/6 \text{ h}$ ，得到各时段电量。

Step 3: 构造线性规划

建立 g, c, d, E, s 五类连续变量，写入目标函数、能量平衡、储电量递推和上下界。

Step 4: 调用 HiGHS 求解

传入目标系数向量、稀疏约束矩阵与变量边界，得到最优购电计划和储能轨迹。

Step 5: 进行可行性审计

检查平衡残差、储电量边界、首末储电量和同时充放电量；不通过时返回 Step 1 复核数据口径。

Step 6: 按题设表格回填

把 144 个购电量逐行写入“计划购电量”，按连续 24 个时段汇总六个四小时区间，写入“充放电量”，并按题目表 1、表 2 的格式展示正文结果。

5.1.3 成本下界与最优性核验

为给出可复核的成本界，先令储能不可用，则无储能基准成本为：

$$C_0 = \sum_{t=1}^T p_t \max\{L_t - P_t, 0\}. \quad (7)$$

再保留储电量递推、容量、功率与首末状态约束，放松“放电必须由本时段净负荷吸收”等限制，构造最大套利收益上界 A_{UB} 。令 $r_t = \max\{P_t - L_t, 0\}$ 为可免费用于充电的光伏富余， b_t 为充电所需的最少外购电量，求解：

$$\begin{aligned} A_{UB} = \max \quad & \sum_{t=1}^T p_t (d_t - b_t), \\ \text{s.t.} \quad & b_t \geq c_t - r_t, \quad b_t \geq 0, \\ & E_t = E_{t-1} + \eta_c c_t - d_t / \eta_d, \\ & 1200 \leq E_t \leq 10800, \quad E_0 = E_T = 6000, \\ & 0 \leq c_t, d_t \leq 5000 \Delta t. \end{aligned} \quad (8)$$

该放松模型允许将放电量全部按当期电价计为收益，因而 A_{UB} 不低于原模型真实套利收益，得到

$$C_{LB} = \max\{0, C_0 - A_{UB}\} \leq C_1^* \leq C_0. \quad (9)$$

A_{UB} 只需一次额外 LP 即可计算。对于本问正式 LP，HiGHS 返回 `optimal`，原始一对偶目标差可作为更紧的数值最优性证书；线性规划的对偶界与最优性关系参见文献 [4]。

5.1.4 求解结果与分析

求解得到全天计划购电量 57526.2435 kWh，购电费 33801.4955 元。无储能基准购电费为 48052.0466 元，储能使成本减少 14250.5510 元，降幅为 29.6565%。总充电量和总放电量分别为 19842.7105 kWh 和 17858.4394 kWh，弃光量为 0。指定时段购电结果列于表1，六个四小时区间的充放电结果及首末储电量列于表2，表头和排列沿用题目表 1、表 2；电量单位为 kWh，购电费单位为元。

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	486.4029	14:00-14:10	0.0000
16:00-16:10	394.9315	18:00-18:10	636.9826	20:00-20:10	0.0000
全天购电量	57526.2435	全天购电费	33801.4955		

图3显示，储能主要在低价时段和光伏出力较高的时段充电，在电价上升后释放能量；中午光伏提高时，外网购电明显下降。储电量始终位于 1200 ~ 10800 kWh，并于日末回到 6000 kWh，说明成本下降来自时段间能量转移，而非透支初始储能。最大购电功

表 2 储能设备在指定时间段的充放电电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4226.3109	0.0000	4:00-8:00	833.3333	6956.9814
8:00-12:00	4448.6339	1702.9970	12:00-16:00	5274.7881	91.1014
16:00-20:00	0.0000	6247.4915	20:00-24:00	5059.6443	2859.8681
0:00 储电量	6000.0000		24:00 储电量	6000.0000	

率为 8458.8273 kW；该值依赖“外网接口容量未给定、暂不设上限”的建模边界。

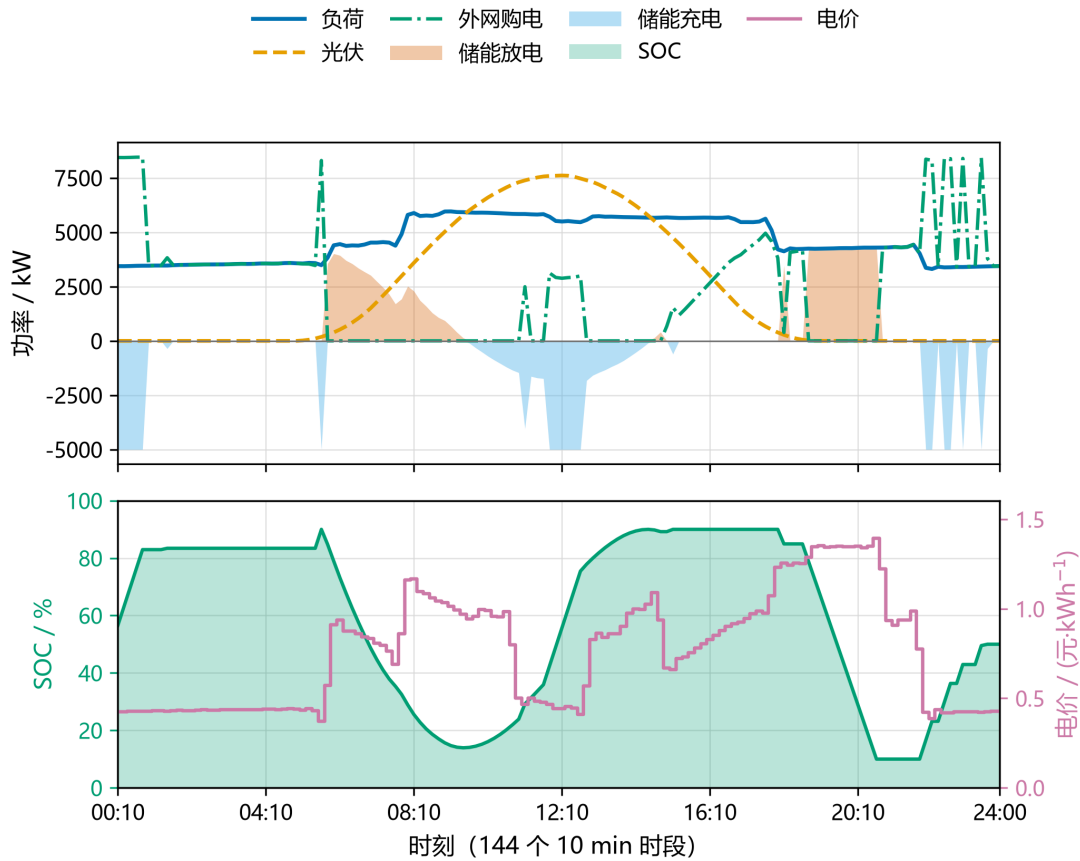


图 3 问题一最优调度与储能状态

5.1.5 可行性与稳健性分析

最优方案的最大能量平衡残差为 2.27×10^{-13} kWh，日末与日初储电量之差为 0，且 $\max_t(c_t d_t) = 0$ ；储电量、充放电功率和弃光约束均通过检验。纯成本模型与加入显式充放电互斥约束的对照模型所得成本一致，说明当前正电价数据下，连续 LP 已能得到物理可执行的调度。

进一步将储能容量分别下调、上调 10%，购电费为 34547.91 元和 33092.98 元；将充放电功率上限分别下调、上调 10%，购电费为 33827.76 元和 33780.88 元。容量变化造成的成本波动明显大于相同幅度的功率变化，表明本算例的主要限制是可跨时段转移的能量规模。固定效率未刻画功率、温度和老化的影响 [2]，且未计退化成本，故结果应理解为题给运行边界下的日前购电成本最优方案。

5.2 问题 2：预测偏差下的日前与日内协同调度

5.2.1 模型建立

设第 d 日有 $T = 144$ 个 10 分钟时段。以 $\mathcal{H}_d$ 表示严格早于 d 的最近至多四个同星期历史日，以 $\mathcal{R}_d$ 表示最近至多七个已结束日，日前预测定义为

$$\hat{L}_{d,t} = \frac{1}{|\mathcal{H}_d|} \sum_{j \in \mathcal{H}_d} L_{j,t}, \quad \hat{P}_{d,t} = \frac{1}{|\mathcal{R}_d|} \sum_{j \in \mathcal{R}_d} P_{j,t}. \quad (10)$$

由同一历史日产生的负荷与光伏残差分别记为 $r_{j,t}^L = L_{j,t} - \hat{L}_{j,t}$ 、 $r_{j,t}^P = P_{j,t} - \hat{P}_{j,t}$ 。在最近至多 28 个完整残差日中用 PAM/K-medoids 选取 $K = 8$ 条代表轨迹 [9]；第 k 条轨迹的权重为其所属簇的样本占比 π_k 。

令 $q_{d,t}$ 为日初锁定且按计划价格结算的普通购电额度， $x_{d,t}$ 为实际取用量， $e_{d,t}$ 为紧急购电量。对第 k 条残差轨迹，净负荷预测误差为 $r_{d,t,k}^N = r_{d,t,k}^L - r_{d,t,k}^P$ ；其正向 α 分位数给出风险储备

$$R_{d,t}(\alpha) = \max \{0, Q_{\alpha, \pi}(r_{d,t,1}^N, \dots, r_{d,t,K}^N)\}. \quad (11)$$

日前层只在唯一基准预测轨迹上生成 q ，并求解

$$\min \sum_{t=1}^T (p_t q_{d,t} + 5p_t e_{d,t}) + V_{d+1}(E_{d,T}), \quad (12)$$

$$\begin{aligned} \text{s.t.} \quad & q_{d,t} \geq R_{d,t}(\alpha), & t = 1, \dots, T, \\ & 0 \leq x_{d,t} \leq q_{d,t}, & t = 1, \dots, T, \\ & x_{d,t} + e_{d,t} + \hat{P}_{d,t} - w_{d,t} + r_{d,t} = \hat{L}_{d,t} + c_{d,t}, & t = 1, \dots, T, \end{aligned} \quad (13)$$

$$E_{d,t} = E_{d,t-1} + \eta_c c_{d,t} - r_{d,t} / \eta_d, \quad t = 1, \dots, T, \quad (14)$$

$$0 \leq c_{d,t}, r_{d,t} \leq 5000 \Delta t, \quad t = 1, \dots, T,$$

$$0 \leq w_{d,t} \leq \hat{P}_{d,t}, \quad 0 \leq e_{d,t}, \quad t = 1, \dots, T,$$

$$1200 \leq E_{d,t} \leq 10800, \quad \eta_c = \eta_d = \sqrt{0.9}.$$

式中 c, r, w, E 分别为公共母线侧充电、放电、弃光和时段末储电量。 $V_{d+1}(\cdot)$ 为按次日因果预测构造的 48 小时终端价值项，只衡量日末能量的机会成本，不提前计入下一日账单。

在日内执行时， $q_{d,t}$ 保持不变。到时段 t 为止已观测的残差前缀与每条 medoid 前缀比较，得到后验权重 $\tilde{\pi}_{k,t}$ ；未来预测取对应残差的加权均值，当前时段以实际 $L_{d,t}, P_{d,t}$ 闭合能量平衡。控制器仅执行滚动 LP 的第一步，下一时段再用更新后的 SOC 重算。这一非前瞻性限制使日前代表轨迹仅承担计划评估和预测修正功能，而不是可直接执行的完整调度方案。

候选预测结构 $m \in \{m_1, m_2, m_3\}$ 与 $\alpha \in \{0.60, 0.70, 0.80, 0.90\}$ 每 14 日在此前已结束的验证日上联合评估。每个候选都须连续传递自身 SOC，按“预测-锁定 q -逐时段

MPC-实际结算” 闭环计算平均成本；并列时依次偏好未用额度较少、更简单的预测结构和较小的 α 。因此参数选择与正式部署共享同一信息集和同一控制政策。

5.2.2 模型求解

式(12)–式(14)及其边界均为线性关系，日前和日内子问题均以线性规划求解。为防止数值退化造成无意义的同时充、放电，在经济目标最优值的货币容差内再最小化电池吞吐量；该词典序次目标不改变购电成本。

预测–计划–执行闭环算法

算法步骤

Step 1: 构造历史信息

在第 d 日 0:00，只读取 d 之前已经结束的负荷、光伏、价格和 SOC 记录，计算式(10)并更新完整残差池。

Step 2: 滚动选择参数

若到达新的 14 日校准块，则对所有 (m, α) 候选在此前验证窗口运行同一闭环政策，按实际成本选择候选；否则沿用当前已选参数。

Step 3: 生成日前额度

对残差池执行 K-medoids，按式(11)计算风险储备，在基准预测上求解日前 LP，锁定 $q_{d,1:T}$ 。

Step 4: 逐时段因果执行

每一时段仅读取当前实际负荷、光伏和上期 SOC；以已观测残差前缀更新未来预测，在剩余锁定额度下求解 MPC[8]，并只实施当前一步的 x, e, c, r, w 。

Step 5: 跨日传递与记账

将 $E_{d,T}$ 传递至下一日，按 $\sum_t (p_t q_{d,t} + 5p_t e_{d,t})$ 写入普通计划费、紧急费和总成本台账。

Step 6: 审计与输出

检查历史截止日、日内无未来信息、 $x \leq q$ 、能量平衡、SOC、功率及不同时充放电；随后按题设格式导出购电计划、充放电和紧急购电结果。

考虑到 1 月历史样本不足，1 月只用于预测历史与 SOC 预热；正式题设工作簿导出 2 月 1 日至 12 月 31 日。该处理必须和完整年度核算分开报告，不能将两种口径混称为“全年结果”。

5.2.3 结果分析

采用政策一致的 $K = 8$ 闭环方案，以 1 月预热后继承的 SOC 作为 2 月初状态。题设输出区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 日；该区间实际结算成本为 15 166 538.46 元，其中普通计划购电成本为 11 883 687.24 元，紧急购电成本为 3 282 851.22

元。工作簿汇总与正式逐日成本台账一致。

上述数值体现的是锁定额度的计划成本与实际缺口的紧急成本之和，而非仅统计实际取用的普通电量。因而，风险储备过高会直接增大计划成本，储备不足又会通过 5 倍价格的紧急购电体现出来；该成本结构正是模型选择预测结构和风险分位的依据。 $K = 8$ 是在既定候选集合和闭环验证协议下保留的情景压缩规模，本文不将其表述为全局最优场景数。

表 3 问题二年度成本分解（政策一致 $K = 8$ ；2-12 月为题设输出区间）

统计口径	天数	计划购电费/元	紧急购电费/元	总成本/元
1 月预热（不计入题设输出）	31	1463771.52	513796.81	1977568.32
2-12 月正式输出	334	11883687.24	3282851.22	15166538.46
2-12 月紧急购电量/kWh			924502.93	
2 月 1 日继承 SOC/kWh			8390.6808	

表 3 给出预热期与 2-12 月正式输出的分列汇总；表 7 报告 $K = 4, 8, 12$ 的闭环复核，综合成本与求解时间后保留 $K = 8$ 。题设格式结果以 2025 年 6 月 21 日为例展示如下。

表 4 问题二 2025-06-21 计划购电量及全天汇总（政策一致 $K = 8$ 主方案）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	2.9074	12:00-12:10	53.0583	14:00-14:10	97.0929
16:00-16:10	52.2398	18:00-18:10	83.9701	20:00-20:10	75.4671
全天购电量	39963.9723	全天购电费	21675.1444		

表 5 问题二 2025-06-21 充放电量及日初/日末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	630.4031	14.5428	4:00-8:00	832.1085	1609.7483
8:00-12:00	1934.3625	0.0000	12:00-16:00	8093.9768	0.0000
16:00-20:00	0.0000	6056.5180	20:00-24:00	7353.2930	3192.8477
0:00 储电量	1610.9749	24:00 储电量	8026.2588		

图 4 显示紧急购电成本在 6-7 月高于相邻月份，与夏季高负荷和预测偏差风险一致；图 5 表明日末 SOC 在输出区间内连续变化且未长期贴住下界；图 6 说明预测结构与风险分位按既定校准周期更新，而情景数 K 保持不变。

5.2.4 模型检验

预测、残差池和校准块的历史截止日均早于目标日；校准与正式运行均采用逐时段 MPC，未用整日真实路径事后重调度。逐时段核对能量平衡、SOC、功率、弃光、 $x \leq q$ 、跨日 SOC 与充放电互斥，并保留日成本台账、预测/策略审计和参数记录。

表 6 问题二 2025-06-21 紧急购电记录 (2-12 月输出区间内典型日)

购电时间段	购电量	备注
00:00-02:30	662.3288	紧急购电
当日合计	662.3288	kWh

表 7 问题二情景数 K 的闭环复核 (2-12 月输出区间; 正式主方案取 $K = 8$)

K	天数	总成本/元	紧急购电/kWh	耗时/s	结论
$K = 4$	334	15190820.26	922502.40	860.3	对照
$K = 8$	334	15166538.46	924502.93	859.0	主方案
$K = 12$	334	15156179.11	925322.33	971.8	对照

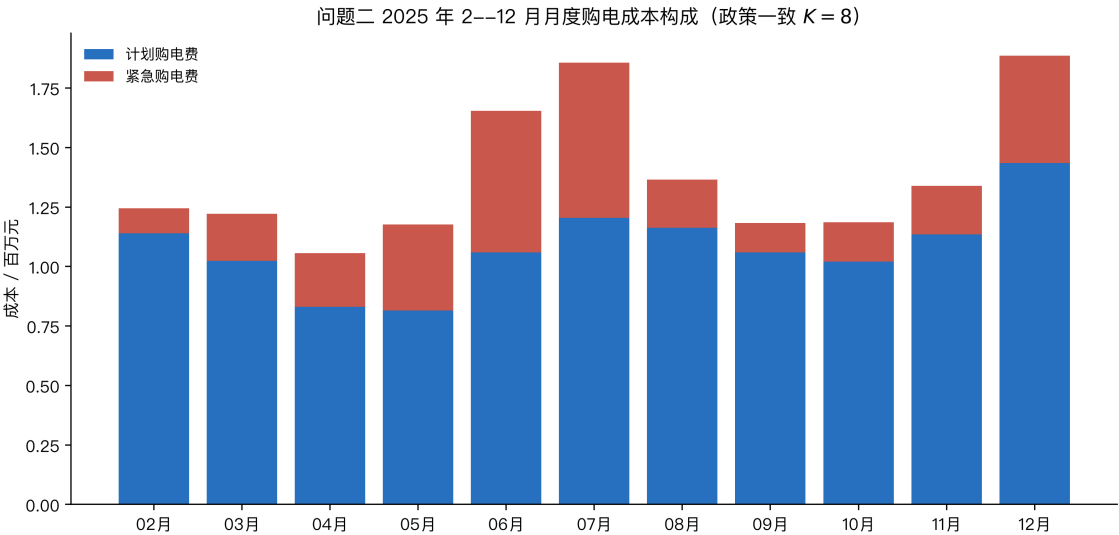


图 4 2025 年 2-12 月月度购电成本构成 (政策一致 $K = 8$; 数据来源: 正式逐日成本台账)

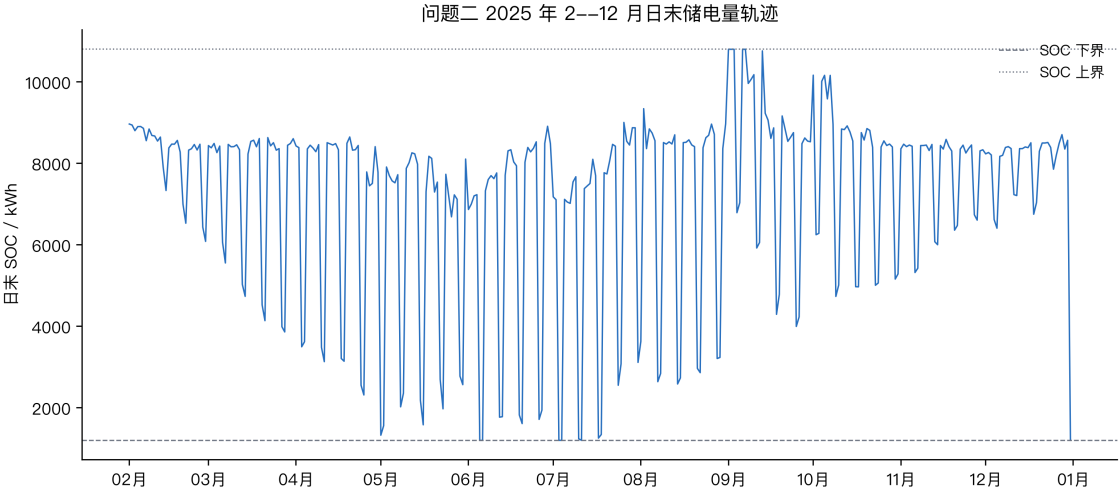


图 5 2025 年 2-12 月日末储电量轨迹; 虚线为 SOC 边界

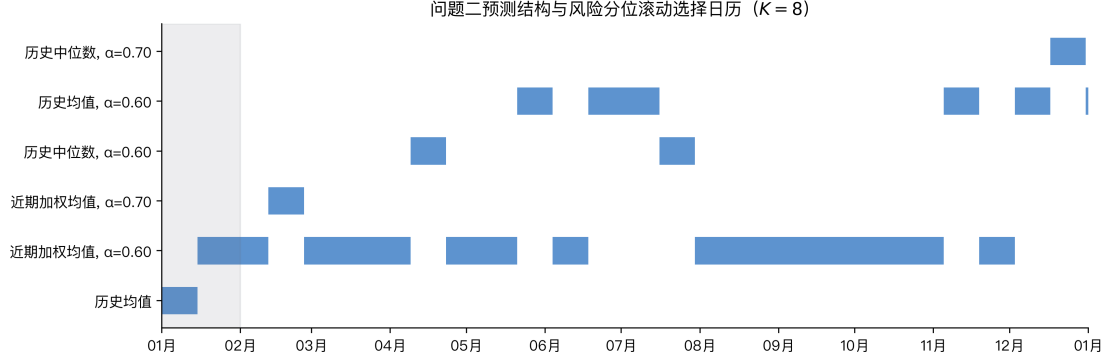


图6 预测结构与风险分位的滚动选择日历：1月为预热，2–12月为题设输出区间

K 复核只在同一闭环协议下比较 $K = 4, 8, 12$ 的实际成本、紧急购电和耗时，故支持的是本样本与候选集内的稳定折中，不是其他社区、年份或市场规则下的普适最优。

5.3 问题 3：多时点预报下的购电调整

5.3.1 模型建立

信息结构与预测映射。设一天含 $T = 144$ 个 10 分钟时段， $\Delta t = 1/6 \text{ h}$ 。预测更新时刻为 0:00、6:00、12:00 和 18:00，记其集合为 $\mathcal{U}$ 。以 m_τ 表示更新时刻 τ 后首个可调整时段，则

$$m_{0:00} = 1, \quad m_{6:00} = 37, \quad m_{12:00} = 73, \quad m_{18:00} = 109. \quad (15)$$

也就是说，6:00 的新预报只从 6:10 开始生效；此前已执行的购电、充放电和储电量均锁定。

原始负荷功率与光伏功率统一换算为时段能量：

$$L_{d,t} = \ell_{d,t} \Delta t, \quad P_{d,t} = \rho_{d,t} \Delta t. \quad (16)$$

其中 d 为日期， t 为时段。对在时刻 τ 发布的逐小时光伏预报，令 $z_{d,h}^\tau$ 为相邻的可用预报节点；以更新时刻刚观测到的光伏作为起始锚点，并在相邻节点间按时间权重 $\lambda_t \in [0, 1]$ 线性插值：

$$\hat{P}_{d,t}^\tau = (1 - \lambda_t) z_{d,h}^\tau + \lambda_t z_{d,h+1}^\tau. \quad (17)$$

式(17)中，更新时刻之前的时段直接保留已观测实际值，不参与未来预测插值。这样，任一更新时刻的预测路径只由已发布信息 and 已发生观测构成。

由于题目未提供独立负荷预报，令 $\mathcal{H}_d$ 为日期严格早于 d 的最近至多四个同星期历史日。日前负荷预测取为

$$\hat{L}_{d,t} = \frac{1}{|\mathcal{H}_d|} \sum_{j \in \mathcal{H}_d} L_{j,t}. \quad (18)$$

在日内更新时，已经执行的前缀用实际负荷和实际光伏更新状态；未执行时段仍使用式(18)与当前已发布的式(17)。这避免将当天未来实际负荷或光伏提前带入承诺决策。计划调整的结算模型。记 $g_{d,t}^0$ 为当天 0:00 制定的普通购电承诺， $g_{d,t}$ 为该时段到来前最

后一次有效调整后的承诺, $e_{d,t}$ 为实时紧急购电量。对同一时段, 结算只相对 $g_{d,t}^0$ 进行一次:

$$\phi_t(g_{d,t}^0, g_{d,t}) = p_t g_{d,t} + \frac{1}{2} p_t |g_{d,t} - g_{d,t}^0|. \quad (19)$$

当 $g_{d,t} \leq g_{d,t}^0$ 时, 式(19)等于实际采购电费加取消部分的 50% 违约金; 当 $g_{d,t} > g_{d,t}^0$ 时, 增量按 $1.5p_t$ 计费。为便于线性规划求解, 引入非负变量 $u_{d,t}, v_{d,t}$:

$$g_{d,t} - g_{d,t}^0 = u_{d,t} - v_{d,t}, \quad u_{d,t}, v_{d,t} \geq 0, \quad (20)$$

并将调整成本写为 $p_t g_{d,t} + \frac{1}{2} p_t (u_{d,t} + v_{d,t})$ 。因其系数为正, 最优解不会同时令 $u_{d,t}$ 和 $v_{d,t}$ 为正。

滚动购电与储能优化模型。在更新时刻 τ , 令剩余时段集合为 $\mathcal{T}_\tau = \{m_\tau, \dots, T\}$ 。以真实当前储电量 $E_{d,m_\tau-1}$ 为初始状态, 最小化剩余时域的结算费、紧急购电费和下一日储电量价值:

$$\min J_{d,\tau} = \sum_{t \in \mathcal{T}_\tau} \left[p_t g_{d,t} + \frac{1}{2} p_t (u_{d,t} + v_{d,t}) + 5p_t e_{d,t} \right] + V_{d+1}(E_{d,T}). \quad (21)$$

其中 $V_{d+1}(\cdot)$ 由下一日的因果负荷预测、光伏预测和同一储能约束求得。将不同参考储电量下的下一日 LP 对偶斜率记为 $(\bar{E}_q, \beta_q, \gamma_q)$, 采用分段线性下包络:

$$V_{d+1}(E_{d,T}) \geq \beta_q + \gamma_q (E_{d,T} - \bar{E}_q), \quad \forall q. \quad (22)$$

终端项只评估剩余储能的机会成本, 不重复计入下一日账单。

普通购电、紧急购电、光伏和储能放电共同供给母线; 负荷、储能充电与弃光消纳能量。故对 $t \in \mathcal{T}_\tau$ 有

$$x_{d,t} + e_{d,t} + \hat{P}_{d,t}^\tau - w_{d,t} + r_{d,t} = \hat{L}_{d,t} + c_{d,t}, \quad t \in \mathcal{T}_\tau, \quad (23)$$

$$E_{d,t} = E_{d,t-1} + \eta_c c_{d,t} - \frac{r_{d,t}}{\eta_d}, \quad t \in \mathcal{T}_\tau, \quad (24)$$

$$\begin{aligned} 0 &\leq x_{d,t} \leq g_{d,t}, & t \in \mathcal{T}_\tau, \\ 0 &\leq e_{d,t}, & t \in \mathcal{T}_\tau, \end{aligned} \quad (25)$$

$$\begin{aligned} 0 &\leq c_{d,t}, r_{d,t} \leq 5000\Delta t, & t \in \mathcal{T}_\tau, \\ 0 &\leq w_{d,t} \leq \hat{P}_{d,t}^\tau, & t \in \mathcal{T}_\tau, \end{aligned} \quad (26)$$

$$1200 \leq E_{d,t} \leq 10800, \quad \eta_c = \eta_d = \sqrt{0.9}. \quad (27)$$

其中 $x_{d,t}$ 是普通承诺中的实际取用量, 约束(25)确保普通购电不能超过已承诺额度; 不足部分只能由当前实际执行阶段的 $e_{d,t}$ 补足。已执行或已锁定时段满足

$$g_{d,t} = g_{d,t}^{\text{pre}}, \quad t < m_\tau, \quad (28)$$

其中 $g_{d,t}^{\text{pre}}$ 是本次更新前的承诺。式(28)将信息边界直接写入模型，而非在求解后再修正计划。

实际执行每次只实施当前 10 分钟动作：把式(23)中的预测量替换为当期实测 $L_{d,t}, P_{d,t}$ ，将得到的 $E_{d,t}$ 带入下一时段。未来时段始终保留当时可得的预测量，因此滚动求解不等同于事后已知完整实际路径的最优调度。

信息价值触发机制与对照策略。为判断某次新预报是否值得用于调整，分别在“锁定原承诺”和“允许调整”下求解式(21)：

$$J_{\tau}^{\text{fix}} = \min\{J_{d,\tau} : g_{d,t} = g_{d,t}^{\text{pre}}, t \in \mathcal{T}_{\tau}\}, \quad (29)$$

$$J_{\tau}^{\text{free}} = \min\{J_{d,\tau} : g_{d,t} \geq 0, t \in \mathcal{T}_{\tau}\}, \quad (30)$$

$$\text{Vol}_{\tau} = J_{\tau}^{\text{fix}} - J_{\tau}^{\text{free}}. \quad (31)$$

当 $\text{Vol}_{\tau} > \varepsilon$ 时实施新承诺，否则维持 g^{pre} ； $\varepsilon = 0.01$ 元仅用于屏蔽数值舍入引起的无意义微调。本文将日初计划不再调整的基准称为“不调整策略 (M0)”，将三个更新点均评估并按式(31)实施的方案称为“全时点价值触发策略 (M1/M6)”；另以 M_6, M_{12}, M_{18} 表示只开放相应单一更新点的消融策略。它们共同回答后续预报是否有价值，以及何时更新最有效。

5.3.2 模型求解

目标函数、调整费用线性化关系、母线平衡、储电量递推及边界约束均为线性关系，因此每个更新时刻的剩余时域问题均可写为稀疏线性规划。使用 HiGHS 求解时，先最小化式(21)；在目标值的货币容差内，再最小化充放电吞吐量，以消除数值上可能出现的等价循环。该次级目标不会改变经济目标值，但能使调度轨迹更易解释。算法不包含非线性迭代或复杂分支，故以步骤描述其信息流和锁定关系即可，无须另绘流程图。

四时点滚动购电调整算法

算法步骤

Step 1: 构造日初信息集

在每日 0:00 汇集全日电价、日初储电量、此前已结束日期的负荷历史与当期光伏预报；按式(18)生成日前负荷预测，并将逐小时光伏预报按式(17)映射至 10 分钟时段。滚动回测中，预测训练样本严格早于决策时点 [6]。

Step 2: 生成日初承诺

以预测供需、储能约束和终端价值求解日初 LP，得到 $g_{d,t}^0$ 及初始储能预案；该承诺成为当天所有调整结算的唯一基准。

Step 3: 实时执行一个时段

在每个 10 分钟时段，仅读取当前实际负荷、实际光伏和上期储电量；在承诺上限 $x_{d,t} \leq g_{d,t}$ 下求得充放电、弃光和必要的紧急购电，并更新储电量。

Step 4: 更新预测并锁定历史

到达 6:00、12:00 或 18:00 时，以新的已发布光伏预报重建剩余时段预测；将已执行时段的购电承诺、物理状态和费用设为常数，并从式(15)规定的下一时段开始优化。

Step 5: 计算信息价值

在锁定原剩余承诺和允许修改剩余承诺两种条件下分别求解 LP，计算 VoI_t 。若其大于 0.01 元，则用自由解替换未执行部分；否则保留原承诺。

Step 6: 继续滚动直至日末

重复 Step 3–Step 5，得到最终有效购电承诺、实际储能轨迹和紧急购电记录；跨日运行时，将日末储电量传递为下一日的日初状态。

Step 7: 核验并按题设输出

核验能量平衡、储电量递推、功率边界、锁定时段和同时充放电；将日初计划、最终调整计划、四小时充放电汇总和紧急购电量按题目模板写入相应工作表。

正式运行以 2025 年 1 月 1 日 6000 kWh 为初始 SOC，全年连续传递状态；主口径采用因果负荷预测、线性锚点光伏映射、相对日初计划的一次性调整结算和 48 小时终端价值。题设工作簿只导出 2–12 月，但策略优劣以全年连续 SOC 的实际成本比较，不以局部试算代替年度结论。

5.3.3 结果分析

主方案采用 A 边界（年末 SOC 为 1200 kWh）、因果负荷预测、线性锚点光伏映射和相对日初承诺的一次性调整结算。按照上一节的命名，主方案为全时点价值触发策略 (M1/M6)，对照方案为不调整策略 (M0)。以 2025 年 1 月 1 日 6000 kWh 为起点并全年连续运行时，主方案实际成本为 16 373 508.75 元，M0 成本为 17 109 568.25 元。因此，三次预测更新和价值触发调整使年度成本减少 736 059.50 元，降幅为 4.30%。

成本分项揭示了这 4.30% 的来源：相对 M0，M1/M6 的普通购电及调整结算成本由 13 335 989.57 元增至 13 602 473.20 元，即增加 266 483.62 元；紧急购电成本则由 3 773 578.68 元降至 2 771 035.55 元，即减少 1 002 543.13 元。二者相抵后净节省 736 059.50 元。这说明预报更新的经济价值主要来自减少 5 倍价格的紧急购电，而不是无成本地频繁修改承诺。

因此，后续预报只在其预期降本能够覆盖改约代价时才应触发调整。M1/M6 是同一套“评估信息价值—决定是否调整”的连续规则，不应被解释成两项可相加的独立收益。

在已测试的年末 SOC 边界中，将其由 1200 改为 6000 kWh 不改变策略排序，但使主策略成本增加 2 262.88 元。题设工作簿只含 2–12 月，共 334 日，其区间成本为 14 512 748.53 元；该值服务于输出核对，不能误称为全年成本。

表 8 汇总 A 边界下 M0 与 M1/M6 的全年比较，并分列题设工作簿输出区间成本。题设格式结果以 2025 年 6 月 21 日为例展示如下。

图 7 表明 M1/M6 在 A 边界下总成本最低；图 8 显示主方案在多数月份降低了紧急

表 8 问题三全年策略比较 (A 边界; 1-12 月连续运行口径)

策略	总成本/元	结算成本/元	紧急购电/元	紧急购电/kWh	调整次数
不调整 (M0)	17109568.25	13335989.57	3773578.68	1051774.78	0
全时点调整 (M1/M6)	16373508.75	13602473.20	2771035.55	742322.81	947
相对 M0 节省	736059.50 元 (4.30%)				
2-12 月输出区间成本	14512748.53 元 (334 日, 非全年口径)				

表 9 问题三 2025-06-21 计划购电量 g^0 (A 边界; 全时点价值触发策略)

时间段	计划购电量	时间段	计划购电量	时间段	计划购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	61.0032	18:00-18:10	0.0000	20:00-20:10	0.0000

表 10 问题三 2025-06-21 调整购电量 g^F (A 边界; 全时点价值触发策略)

时间段	调整购电量	时间段	调整购电量	时间段	调整购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	61.0032	18:00-18:10	0.0000	20:00-20:10	0.0000

表 11 问题三 2025-06-21 充放电量及日初/日末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	731.7612	0.0000	4:00-8:00	833.3333	1719.9540
8:00-12:00	411.5714	0.0000	12:00-16:00	9545.8433	0.0000
16:00-20:00	131.1779	6975.8261	20:00-24:00	7353.2930	2273.5396
0:00 储电量	1557.3324		24:00 储电量	8026.2587	

表 12 问题三 2025-06-21 紧急购电记录

购电时间段	购电量	备注
0:10-0:20	35.2009	紧急购电
0:20-0:30	57.0999	紧急购电
0:30-0:40	104.8654	紧急购电
0:40-0:50	32.7603	紧急购电
1:00-1:10	1.5729	紧急购电
1:10-1:20	6.1632	紧急购电
其余 34 条记录见随附正式工作簿		
当日合计	1455.0985	kWh

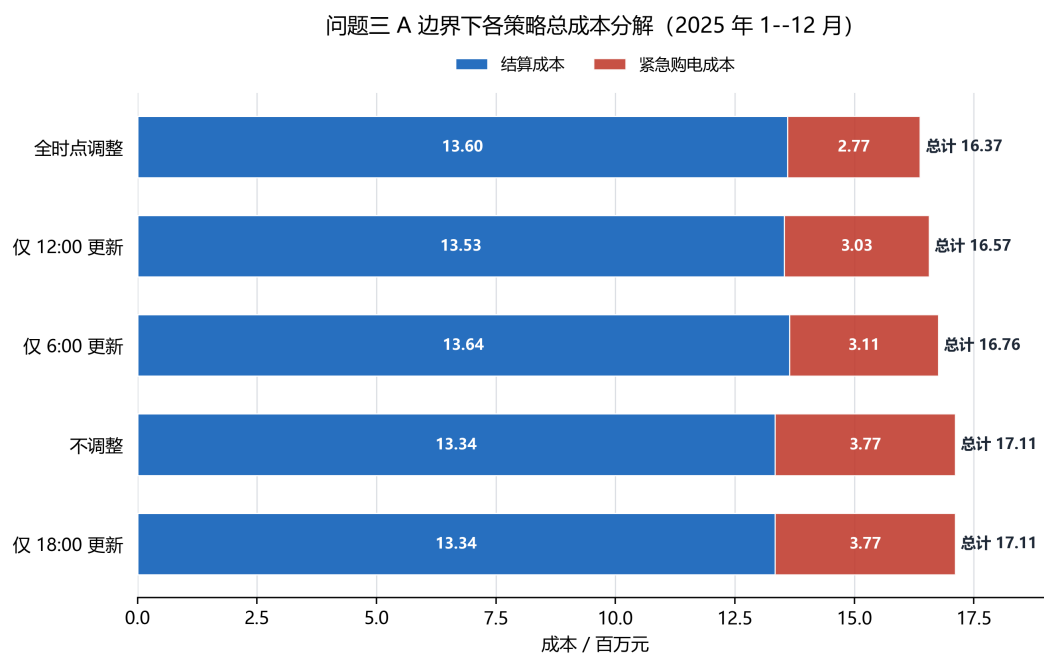


图 7 A 边界下各策略 2025 年 1-12 月总成本分解 (数据来源: 正式逐日成本台账)

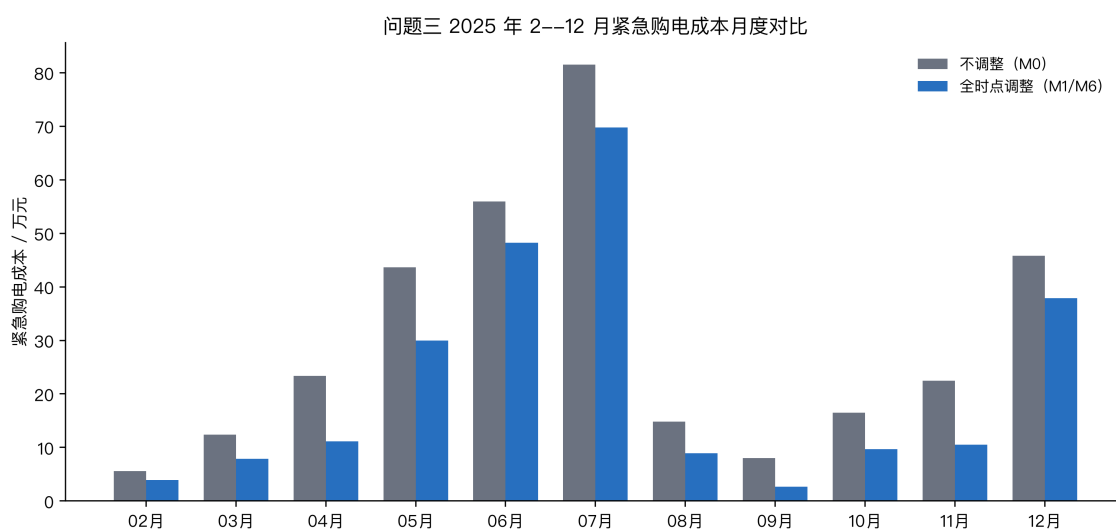


图 8 2025 年 2-12 月不调整策略 (M0) 与全时点价值触发策略 (M1/M6) 的紧急购电成本月度对比

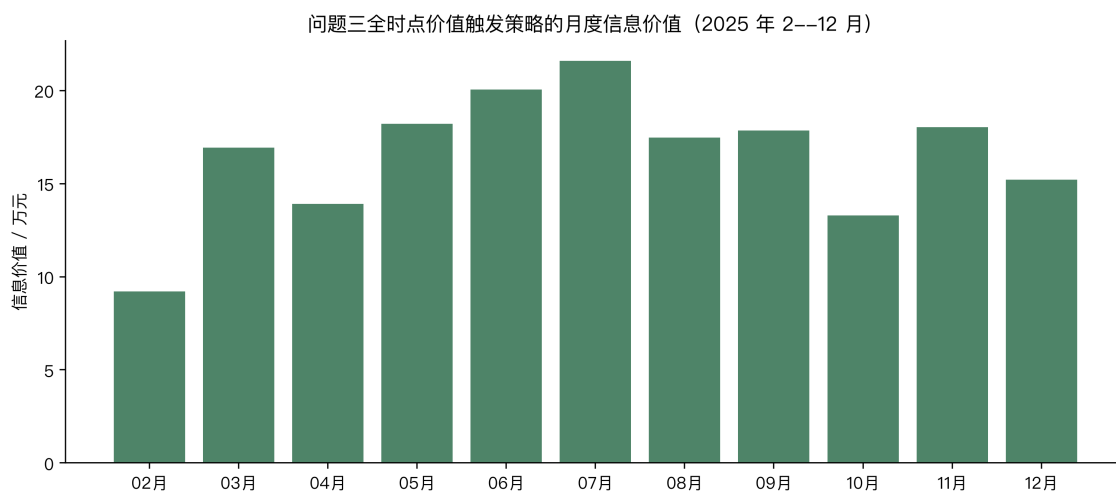


图 9 全时点价值触发策略 (M1/M6) 在 6:00、12:00、18:00 已实施调整的月度信息经济价值汇总 (2025 年 2-12 月)

购电支出；图 9 说明调整并非每日都发生，其价值在夏秋季更为集中。

5.3.4 模型检验

每次 0:00、6:00、12:00、18:00 决策只使用当时已发布预报、已执行状态和因果负荷预测；更新从 6:10、12:10、18:10 生效，最终结算始终相对 g^0 一次计算。对 365×144 个时段审计母线平衡、SOC 递推与 1200–10800 kWh 边界、5000 Δt 功率、额度、跨日连续及充放电互斥；输出可由日账单、紧急购电、弃光和 SOC 起止值逐日回算。

光伏映射和调整结算的替代读法仅完成两日回归检查，故只作局部口径验证；结论限于题设设备、2025 年数据与本文结算解释。

5.4 问题 4：波动电价下的调控策略

5.4.1 模型建立

交付价结算与信息集。Q4 不改变题给设备与效率，也不引入售电、退化或需求响应。普通购电仍锁定为时段额度：Q4-2 的 $q_{d,t}$ 在 0:00 确定，按 $p_{d,t}q_{d,t}$ 计费；Q4-3 的 $g_{d,t}^0$ 在 0:00 确定，后续仅在 6:00、12:00、18:00 评估是否调整未执行承诺，结算函数仍为

$$\phi_{d,t}(g^0, g^F) = p_{d,t}g^F + \frac{1}{2}p_{d,t}|g^F - g^0|, \quad (32)$$

紧急购电按 $5p_{d,t}e_{d,t}$ 计入。该口径把“计划市场锁价”明确排除在题设数据之外，避免引入附件未给出的合约变量。

为可审核地定义“实时”，本文采用保守端点约定：带标签 t 的实际价格、负荷与光伏只在该时段结束后进入信息集；因此 6:00 更新可使用截至 6:00 的记录，新计划从 6:10 起生效。任一决策必须是 $\mathcal{I}_{d,\tau}$ 的函数，其中 $\mathcal{I}_{d,\tau}$ 不含当天 τ 之后的附件 2/3/4 实际值。

非前瞻价格预测。日初构造三条候选价格路径：前一日同刻 $\hat{p}_{d,t}^{(1)} = p_{d-1,t}$ ；最近 $m \in \{2, 4\}$ 个同星期历史日的逐时段均值 $\hat{p}_{d,t}^{(m)}$ 。在日期严格早于 d 的已结束日上，以累计 MAE 在线选择候选 a_d ，得到基准 $\hat{p}_{d,t}^0$ ；历史不足时回退到唯一可得的前一日曲线。该规则固定，不得用全年误差事后挑选窗口 [6]。

在更新时点 τ ，用已结束前缀的平均基准残差

$$z_{d,\tau} = \frac{1}{\tau} \sum_{s=1}^{\tau} (p_{d,s} - \hat{p}_{d,s}^0) \quad (33)$$

与严格早于 d 的历史日拟合带截距 OLS，预测剩余价格

$$\hat{p}_{d,t|\tau} = \max \{0, \hat{p}_{d,t}^0 + \alpha_{\tau,t} + \beta_{\tau,t}z_{d,\tau}\}, \quad t > \tau. \quad (34)$$

样本不足或解释变量零方差时令 $\beta_{\tau,t} = 0$ ，只保留历史平均残差；截断为零仅体现价格非负，不能解释为售电收益。

负荷与光伏预测完全继承 Q2/Q3 主口径：负荷为最近至多四个同星期历史日均值；Q4-2 光伏沿用 Q2 因果基准与后验残差更新；Q4-3 在 0:00、6:00、12:00、18:00 使用附件 3 当次预报，并以更新时点已观测光伏为锚点做线性插值。

日前额度策略：三元联合残差与日前一日内分工。对历史日 $j < d$ ，在当日可得预测存档下构造三元残差

$$(r_{j,t}^L, r_{j,t}^P, r_{j,t}^p) = (L_{j,t} - \hat{L}_{j,t}, P_{j,t} - \hat{P}_{j,t}, p_{j,t} - \hat{p}_{j,t}^0). \quad (35)$$

在最近至多 28 个完整残差日上，以标准化欧氏距离做 PAM/K-medoids，取 $K = 8$ 条代表路径，概率为簇占比。第 k 条路径给出

$$L_{d,t}^{(k)} = \max(0, \hat{L}_{d,t} + r_{m_k,t}^L), \quad P_{d,t}^{(k)} = \max(0, \hat{P}_{d,t} + r_{m_k,t}^P), \quad p_{d,t}^{(k)} = \max(0, \hat{p}_{d,t}^0 + r_{m_k,t}^p). \quad (36)$$

同一历史日的三条残差必须配对，不允许独立洗牌 [10]。

日初在全部代表路径上选择共享的 $q_{d,t}$ ，目标为

$$\min \sum_{k=1}^8 \pi_{d,k} \sum_{t=1}^T (p_{d,t}^{(k)} q_{d,t} + 5p_{d,t}^{(k)} e_{d,t}) + \sum_{k=1}^8 \pi_{d,k} V_{d+1}^{(k)}(E_{d,T}^{(k)}), \quad (37)$$

令 $\mathcal{T}_\tau$ 为时点 τ 后的未执行时段集合， $a_{d,t} = q_{d,t}$ 表示 Q4-2 的锁定额度， $a_{d,t} = g_{d,t}$ 表示 Q4-3 的当前有效承诺。两条链在任一未执行时段均使用同一组母线与设备约束：

$$x_{d,t} + e_{d,t} + \tilde{P}_{d,t} - w_{d,t} + r_{d,t} = \tilde{L}_{d,t} + c_{d,t}, \quad t \in \mathcal{T}_\tau, \quad (38)$$

$$E_{d,t} = E_{d,t-1} + \eta_c c_{d,t} - r_{d,t} / \eta_d, \quad t \in \mathcal{T}_\tau, \quad (39)$$

$$0 \leq x_{d,t} \leq a_{d,t}, \quad 0 \leq e_{d,t}, \quad t \in \mathcal{T}_\tau,$$

$$0 \leq c_{d,t}, r_{d,t} \leq 5000\Delta t, \quad t \in \mathcal{T}_\tau,$$

$$0 \leq w_{d,t} \leq \tilde{P}_{d,t}, \quad t \in \mathcal{T}_\tau,$$

$$1200 \leq E_{d,t} \leq 10800, \quad \eta_c = \eta_d = \sqrt{0.9}. \quad (40)$$

其中 $(\tilde{L}, \tilde{P})$ 为当前信息集下的预测，执行当前段时替换为实测值。Q4-2 另施加 Q2 的滚动风险分位储备下界；Q4-3 在每次更新时把 $t < m_\tau$ 的承诺固定为更新前值。该优化只用于计划筛选：路径内储能轨迹评估共同 q 的可行性，不是可直接下发的控制指令。

对已锁定的 q ，日内每个十分钟末观测实际 (L, P, p) 与 SOC，按 Q2 规则更新后验权重，对 $t > \tau$ 用式(36)的条件均值修正剩余预测，在 q 不变的前提下滚动求解并只执行第一步。实现成本为

$$C_d^{4+2} = \sum_t (p_{d,t} q_{d,t} + 5p_{d,t} e_{d,t}). \quad (41)$$

只有式(41)用于结论，日前情景目标不得替代实际账单。

多时点调整策略：沿用全时点价值触发规则。多时点调整策略不重新选择问题三的策

略，而是沿用全时点价值触发策略 (M1/M6)。0:00 以 $(\hat{L}, \hat{P}^0, \hat{p}^0)$ 求 g^0 与储能预案；在 $\tau \in \{6:00, 12:00, 18:00\}$ ，对未执行时段分别计算固定承诺与自由调整的两个剩余时域最优值 $J_\tau^{\text{fix}}, J_\tau^{\text{free}}$ ，令 $VoI_\tau = J_\tau^{\text{fix}} - J_\tau^{\text{free}}$ 。仅当 VoI_τ 超过求解器货币容差时才采用自由解。预测阶段使用 $\hat{p}_{d,t|\tau}$ 与式(32)的预测结算函数；实现阶段按实际 $p_{d,t}$ 重算

$$C_d^{4-3} = \sum_t [\phi_{d,t}(g_{d,t}^0, g_{d,t}^F) + 5p_{d,t}e_{d,t}]. \quad (42)$$

方法边界。本文不把 CVaR 风险厌恶、深度网络价格预测或 0:00 已知全天实际价格的全信息基准纳入主方案；它们最多作为解释边界或敏感性。Q4 的表述边界是“三元联合残差日前评估 + 非前瞻 MPC 执行”，而不是全信息多阶段随机最优控制。

5.4.2 模型求解

问题四与问题二、三采用相同的“日前线性规划-日内滚动执行-逐日审计”求解框架。新增的价格预测只提供各决策时点可用的价格路径，不改变 LP 的变量类型、约束处理方式或逐十分钟的执行逻辑。日前额度和多时点调整两条策略均先最小化经济目标；在货币容差内，再最小化电池吞吐量以消除等价循环。

算法步骤

Step 1: 构造因果信息

在第 d 日 0:00，仅读取此前已结束日的负荷、光伏、价格和 SOC 记录；以滚动 MAE 选择日初价格基准 $\hat{p}^0$ ，并生成负荷、光伏与价格的联合残差池。

Step 2: 生成 Q4-2 日前额度

对联合残差执行 K-medoids，保留 8 条同日配对的代表路径；在 Q2 已冻结的 (m, α) 日历和风险储备下，求解式(37)，锁定 $q_{d,1:T}$ 。

Step 3: 执行 Q4-2 日内 MPC

每一时段仅读取当前实际 (L, P, p) 与上期 SOC；根据已观测前缀更新后验权重和剩余预测，在锁定额度下求解 LP，并只实施当前一步。已锁定额度和已发生状态不回滚。

Step 4: 生成 Q4-3 日初承诺

以 $\hat{L}$ 、 $\hat{P}^0$ 和 $\hat{p}^0$ 求解日初 LP，得到 $g_{d,t}^0$ 与储能预案；该承诺成为当天全部调整结算的唯一基准。

Step 5: 更新 Q4-3 未执行承诺

到达 6:00、12:00、18:00 时，用已发布预报和已观测价格前缀修正剩余预测；分别求解锁定原承诺与允许调整的 LP，仅当 VoI_τ 超过货币容差时替换未执行部分，并继续逐十分钟执行。

Step 6: 审计并导出结果

核验信息截止点、联合残差配对、能量平衡、SOC、功率、额度、调整锁定和不同时充放电；随后按题设模板导出正式工作簿。每个 α 候选及 $K = 4, 8, 12$ 均在同一闭环

下复核实际成本、紧急购电量与耗时。

主模型保持 LP；情景数固定为 8，日内 MPC 的视域为剩余时段加 48 小时终端价值。价格维度加入后，仍须在 Q2 使用的求解时间预算内完成同一验证日的闭环评分，否则该候选不得进入正式部署。

5.4.3 结果分析

日前额度策略 (Q4-2)。日前额度策略在问题二政策一致框架上引入附件 4 的交付时段实际价格，结算按 $p_{d,t}q_{d,t} + 5p_{d,t}e_{d,t}$ 计入。风险分位 α 的校准采用与部署相同的闭环协议：每个候选均在“联合情景日前计划—锁定 q —逐十分钟非前瞻 MPC—连续传递 SOC”链条上评分； $\alpha \in \{0.60, 0.70, 0.80, 0.90\}$ 共 25 个校准窗口，最终滚动选中 $\{0.60, 0.70\}$ 。情景数取 $K = 8$ ，并在相同协议下复核 $K = 4, 8, 12$ 。

1 月仍只作预测历史与 SOC 预热。题设输出区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 日；该区间实际结算成本为 15 257 873.75 元，其中按锁定额度计费的计划部分为 13 213 373.44 元，紧急购电部分为 2 044 500.32 元。若报告 1–12 月完整核算，全年成本为 17 681 029.10 元，其中 1 月预热成本为 2 423 155.34 元；2 月 1 日继承 SOC 为 5541.1011 kWh。

与问题二固定价主方案 (2–12 月 15 166 538.46 元) 相比，同一输出区间因交付价权重变化增加 91 335.29 元 (约 0.60%)。该差异来自波动价下“提前锁定额度”的沉没计划成本与紧急购电风险的再权衡，而非改变问题二的因果预测与 MPC 结构。

表 13 问题四日前额度策略 (Q4-2) 成本分解 (交付时段实际价；2–12 月为题设输出区间)

统计口径	天数	计划购电费/元	紧急购电费/元	总成本/元
1 月预热 (不计入题设输出)	31	1937214.20	485941.14	2423155.34
2–12 月正式输出	334	13213373.44	2044500.32	15257873.75
2–12 月紧急购电量/kWh			541155.39	
2 月 1 日继承 SOC/kWh			5541.1011	

表 13 分列预热期与正式输出；表 14 给出 K 复核；表 15 汇总逐月价格预测误差。题设格式结果以 2025 年 6 月 21 日为例展示如下。

表 14 问题四日前额度策略 (Q4-2) 情景数 K 的闭环复核

K	窗口数	验证总成本/元	验证紧急费/元	平均耗时/s	角色
$K = 4$	25	16315671.52	2215476.92	4.8	低场景数对照
$K = 8$	25	16234219.00	2067551.15	5.6	主方案 (稳定折中)
$K = 12$	25	16194910.76	1985536.60	6.7	高场景数对照

在同一闭环协议下， $K = 8$ 相比 $K = 12$ 的验证成本仅高 0.243%，平均耗时由 6.7 s 降至 5.6 s (约 16.4%)；故在本文采用的 1% 成本容差下，取 $K = 8$ 为在线可用的稳定折中，而非成本最优。

表 15 问题四因果价格预测误差 (2025 年 1-12 月; 单位: 元/kWh)

月份	日初 MAE	日初 RMSE	6:00 更新 MAE	18:00 更新 MAE	日内剩余 MAE
2025-01	0.0572	0.0891	0.0521	0.0414	0.0486
2025-02	0.0506	0.0752	0.0482	0.0480	0.0444
2025-03	0.0390	0.0507	0.0433	0.0332	0.0378
2025-04	0.0381	0.0491	0.0432	0.0328	0.0382
2025-05	0.0395	0.0508	0.0446	0.0346	0.0395
2025-06	0.0502	0.0673	0.0480	0.0314	0.0405
2025-07	0.0552	0.0791	0.0570	0.0587	0.0527
2025-08	0.0418	0.0545	0.0457	0.0347	0.0399
2025-09	0.0585	0.0863	0.0529	0.0635	0.0531
2025-10	0.0392	0.0506	0.0462	0.0360	0.0392
2025-11	0.0355	0.0458	0.0415	0.0326	0.0359
2025-12	0.0496	0.0769	0.0449	0.0403	0.0416
全年	0.0461	0.0663	0.0473	0.0405	0.0426

表 16 问题四日前额度策略 (Q4-2) 2025-06-21 计划购电量及全天汇总 ($K = 8$)

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	43.8536	12:00-12:10	70.8954	14:00-14:10	118.3038
16:00-16:10	230.7738	18:00-18:10	394.2621	20:00-20:10	72.5862
全天购电量	43870.6688	全天购电费	21871.3099		

图 10 显示夏秋季紧急购电占比上升, 与 Q2 类似的高负荷时段一致。价格预测误差须按各自剩余时域解读: 全年日初、6:00 和 18:00 的 MAE 分别为 0.0461、0.0473、0.0405 元/kWh; 因预测时长与覆盖时段不同, 不能据此直接推断更新时点之间的精度单调改善。图 11 仅说明已观测价格前缀可形成非前瞻的剩余时段预测, 不意味着模型能够预测全部价格尖峰; 调度层仍须依靠滚动重算、SOC 余量和紧急购电吸收误差。

多时点调整策略 (Q4-3)。多时点调整策略沿用问题三的全时点价值触发规则 (M1/M6), 将调整费与紧急购电改为按交付时段实际价格结算; 策略结构和更新时点保持不变, 不在看到全年真实价格后重新挑选策略。题设输出区间为 2025 年 2 月 1 日至 12 月 31 日, 共 334 日; 该区间实际结算成本为 15 346 418.14 元, 其中普通购电 12 447 748.75 元、调整费 321 297.70 元、紧急购电 2 577 371.68 元。若报告 1-12 月完整核算, 全年

表 17 问题四日前额度策略 (Q4-2) 2025-06-21 充放电量及日初/日末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	1250.7382	3786.6509	4:00-8:00	1635.7116	1406.9017
8:00-12:00	1139.4605	0.0000	12:00-16:00	8363.6620	0.0000
16:00-20:00	576.5484	5348.5695	20:00-24:00	7193.2999	2990.5975
0:00 储电量	4060.8313		24:00 储电量	8920.9978	

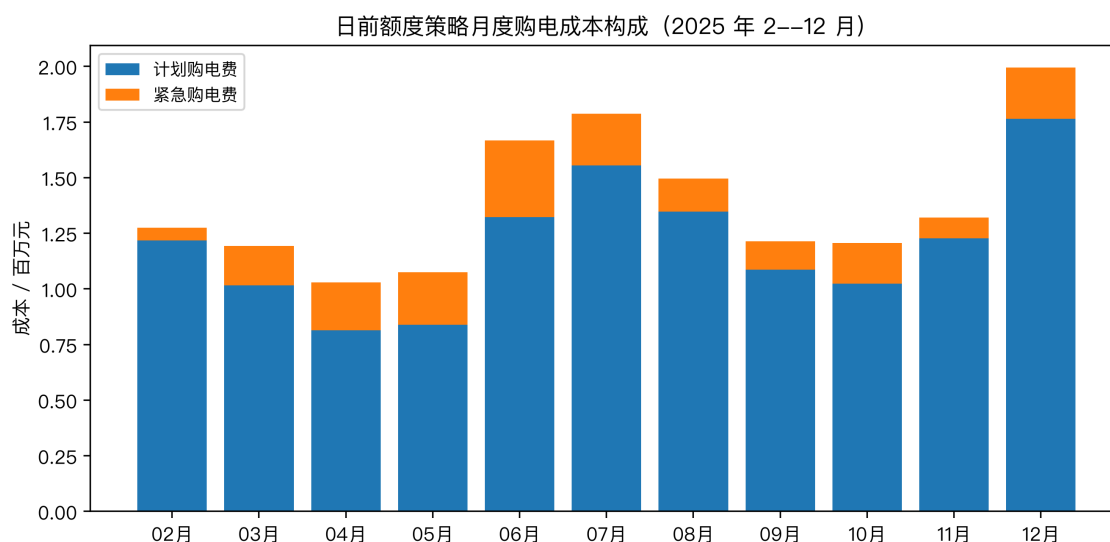


图 10 日前额度策略 (Q4-2) 2025 年 2-12 月月度购电成本构成 (数据来源: 正式逐日成本台账)

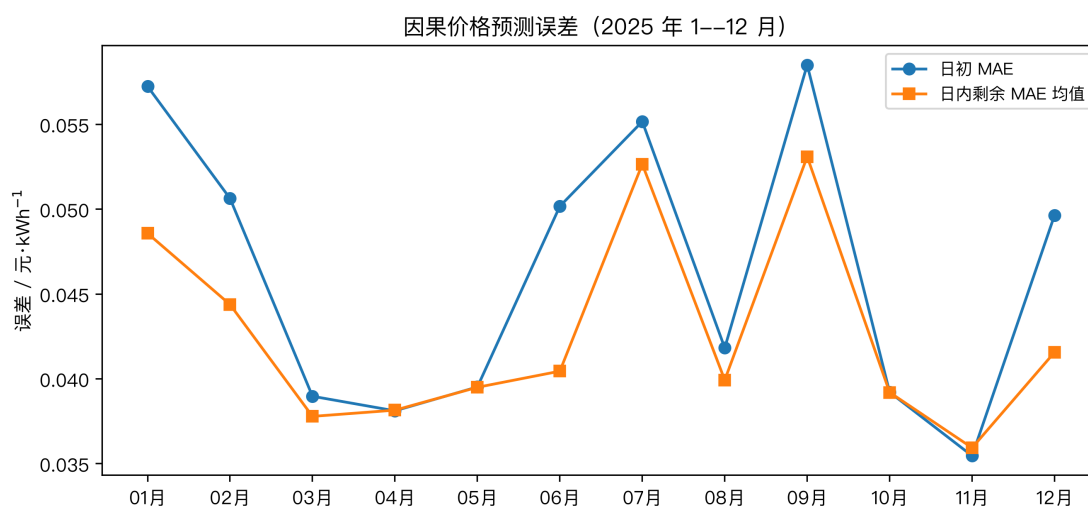


图 11 日前额度策略 (Q4-2) 逐月价格预测误差 (2025 年 1-12 月; 数据来源: 正式价格预测审计)

成本为 17489808.24 元, 其中 1 月预热成本为 2143390.10 元; 2 月 1 日继承 SOC 为 7036.1914kWh。全年共实施 1004 次有效调整, 调整费占全年成本约 2.03%。

与问题三固定价主方案 (1-12 月 16373508.75 元) 相比, 多时点调整策略在同一策略、不同价格机制下增加 1116299.49 元 (约 6.82%); 该差异只反映波动价权重变化, 不作为策略优劣结论。在 2-12 月输出区间, 该策略较问题三同区间 (14512748.53 元) 高 833669.61 元 (5.74%); 较日前额度策略 (15257873.75 元) 高 88544.39 元 (0.58%)。后一个差异只反映本算例中两条题设策略在波动价结算下的成本规模不同; 由于普通承诺、调整权与紧急购电机制并不相同, 不据此判定两类策略优劣或分离价格机制的单独作用。

表 18 分列预热期与正式输出; 表 19 对照 Q3 与 Q4-3 的价格机制差异。题设格式结果以 2025 年 6 月 21 日为例展示如下。

图 12 显示调整费与紧急购电在夏秋季占比上升; 图 13 表明交付价机制下总成本高于问题三固定价口径, 与两问结算定义一致。

表 18 问题四多时点调整策略 (Q4-3) 成本分解 (交付时段实际价; 2-12 月为题设输出区间)

统计口径	天数	普通购电/元	调整费/元	紧急购电/元	总成本/元
1 月预热 (不计入题设输出)	31	1650859.29	34374.75	458156.06	2143390.10
2-12 月正式输出	334	12447748.75	321297.70	2577371.68	15346418.14
2-12 月紧急购电量/kWh			651103.38		
2-12 月调整次数			935		
2 月 1 日继承 SOC/kWh			7036.1914		

表 19 多时点调整策略在固定价与交付价机制下的成本对照

方案	价格机制	1-12 月/元	2-12 月输出/元	紧急购电/元	调整次数
问题三	固定价附件 1	16373508.75	14512748.53	2771035.55	947
问题四调整策略	交付时段实际价	17489808.24	15346418.14	3035527.74	1004
1-12 月机制差	1116299.49 元 (6.82%); 反映价格波动权重, 非策略重选收益				
2-12 月输出区间差	833669.61 元 (5.74%)				
问题四调整费占全年	355672.45 元 (2.03%)				

5.4.4 模型检验

价格预测训练样本严格早于决策时点, 未来真实价格、负荷或光伏扰动不得改变已锁定的 q 、 g^0 与已执行轨迹; 价格日志可逐日复算候选 MAE 与选择结果。代表情景的 (L, P, p) 来自同一历史日, 保留相关性而不将其解释为因果机制。两类策略逐时段审计信息时序、 $x \leq q$ 、母线平衡、SOC、功率、弃光、跨日连续和充放电互斥; Q4-3 另核查 g^F 只影响更新后时段, 且始终相对 g^0 结算。

365 日物理审计全部通过, 跨日 SOC 最大衔接误差为 0, 两个代表性日期的信息集扰动抽查通过; 成本按式(41)、(42) 与正式日台账逐日核对。每个 α 候选均按“联合情景计划—锁定额度—10 分钟 MPC—连续 SOC” 闭环评分; $K = 4, 8, 12$ 的同协议复核支持保留 $K = 8$ (具体成本与耗时见表14)。正式工作簿完整覆盖 2-12 月 334 日; 与 Q3 固定价结果仅用于说明价格机制影响, 不作优劣排序。

六、模型的评价、部署闭环与推广

6.1 关键稳健性证据

表 24 汇总三项直接影响结论的已有对照: Q1 表明日内成本对可转移能量规模更敏感; Q3 的年末边界对照排除了以年末透支改变策略排序的可能; Q4 给出了场景规模、成本精度和在线求解时间之间的取舍。

三项检验覆盖设备尺度、跨期边界和算法规模; 结论只针对题给数据、候选集合与

表 20 问题四多时点调整策略 (Q4-3) 2025-06-21 计划购电量 g^0 (交付时段实际价)

时间段	计划购电量	时间段	计划购电量	时间段	计划购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	61.0032	18:00-18:10	385.2115	20:00-20:10	0.0000

表 21 问题四多时点调整策略（Q4-3）2025-06-21 调整购电量 g^F （交付时段实际价）

时间段	调整购电量	时间段	调整购电量	时间段	调整购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	61.0032	18:00-18:10	385.2115	20:00-20:10	0.0000

表 22 问题四多时点调整策略（Q4-3）2025-06-21 充放电量及日初/日末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	1666.6667	1433.2545	4:00-8:00	1867.5943	1719.9540
8:00-12:00	411.5714	0.0000	12:00-16:00	9545.8433	0.0000
16:00-20:00	131.1779	6606.8464	20:00-24:00	8786.5376	3181.0295
0:00 储电量	1200.0000		24:00 储电量	8818.3144	

表 23 问题四多时点调整策略（Q4-3）2025-06-21 紧急购电记录

购电时间段	购电量	备注
0:10-0:20	35.2009	紧急购电
0:20-0:30	57.0999	紧急购电
0:30-0:40	104.8654	紧急购电
0:40-0:50	32.7603	紧急购电
1:00-1:10	1.5729	紧急购电
1:10-1:20	6.1632	紧急购电
其余 29 条记录见随附正式工作簿		
当日合计	1361.4638	kWh

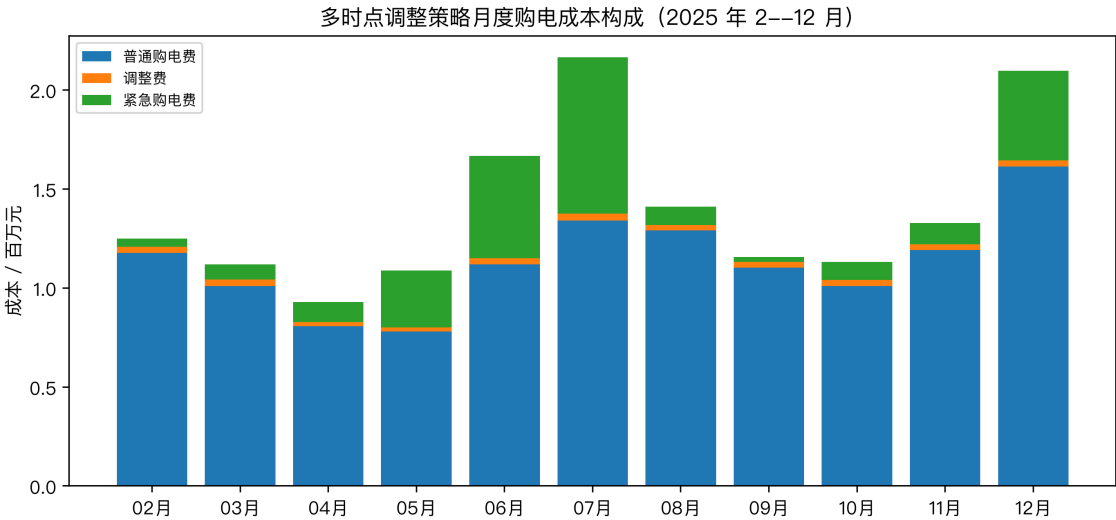


图 12 多时点调整策略（Q4-3）2025 年 2--12 月月度购电成本构成（数据来源：正式逐日成本台账）

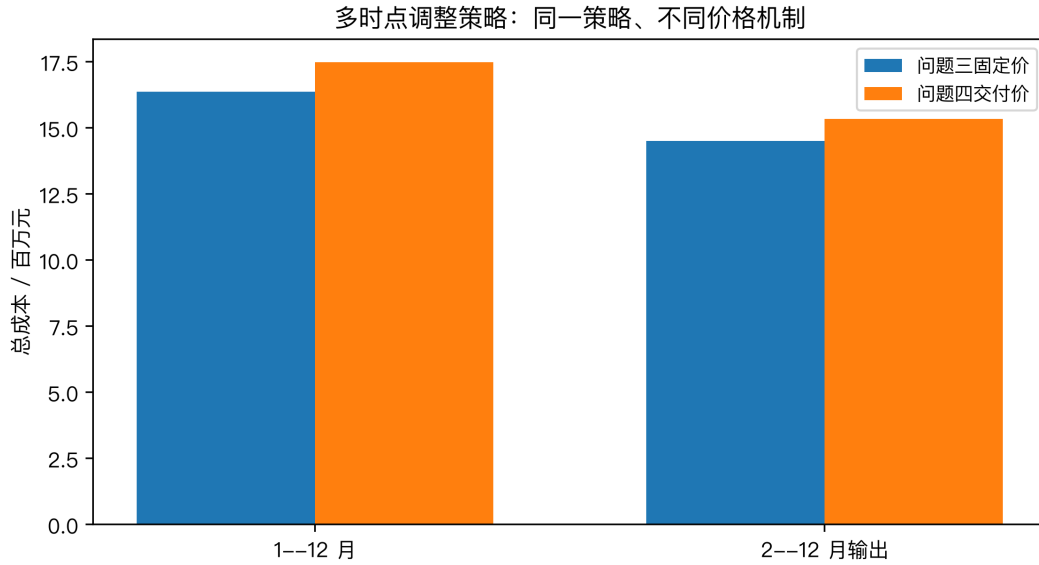


图 13 多时点调整策略在固定价与交付价机制下的成本对照（分列 1-12 月完整核算与 2-12 月输出区间）

表 24 关键稳健性证据汇总（均为已有试验结果）

问题	核验设置	已有结果	支持的结论
Q1	容量与功率上限分别变动 $\pm 10\%$	容量变动后成本为 34547.91/33092.98 元；功率变动后为 33827.76/33780.88 元	同幅度变化下，容量对成本的影响大于充放电功率上限
Q3	年末 SOC 由 1200 调整为 6000 kWh	主策略成本增加 2262.88 元，五种策略的相对排序保持不变	在已测试的两种年末边界下，策略排序保持不变
Q4	在同一闭环协议下比较 $K = 8$ 与 $K = 12$	$K = 8$ 成本高 39308.25 元 (0.24%)，平均耗时由 6.7 s 降至 5.6 s，约缩短 16.3%	在既定 1% 成本容差内， $K = 8$ 以较小成本差换取更快求解

当前结算口径，不外推到所有微网场景。

6.2 面向部署的运行闭环

若将本文策略实现为调度原型，运行链路可按以下三个环节闭合：

- D1：日初计划。每日 0:00 接入截至当时的负荷、光伏、价格、预报和 BMS-SOC，生成并锁定普通购电额度或基准承诺。
- D2：日内控制与安全门。每 10 分钟以最新已观测状态重建剩余时域 LP，只执行下一步；6:00、12:00、18:00 仅可调整未执行承诺，且预测收益须覆盖改约成本。EMS 校验能量平衡、额度与弃光，BMS 校验 SOC、功率与互斥，任一硬约束不满足即拒绝下发。
- D3：降级与留痕。求解超时、数据缺失或通信中断时冻结已锁定承诺，储能转入 BMS 限幅安全保持，缺口由紧急购电补足；保存预测版本、信息截止点、承

诺、SOC、求解状态和成本分项，确保费用与信息边界均可逐时复核。

预测、优化、执行和结算分别留痕，因此可以区分预测偏差与求解故障，也能核对控制动作是否越过信息边界。

6.3 工程边界与后续扩展

题目未给出公共耦合点（PCC）允许交换容量，主模型也未计电池循环退化，因此结果仅可视为经济调度原型，不能直接作为已完成并网校核或寿命权衡的现场方案。部署前应由并网协议给出 $G_{\max}$ ，在每时段加入 $0 \leq g_t \leq G_{\max} \Delta t$ ，并重新核验 Q1–Q4 的可行性、紧急购电和成本；必要时还应加入变压器容量、线路潮流和备用约束。若获得单位吞吐退化成本 c_{deg} ，可在目标中加入 $\sum_t c_{\text{deg}}(c_t + r_t)$ ；需求响应、余电上网和多能源耦合也须在取得对应设备与市场参数后另行建模。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码运行与调试辅助、语言润色和文字整理，核心建模与分析由参赛队独立完成并核验，详细使用情况见支撑材料。

参考文献

- [1] National Renewable Energy Laboratory. 2019 Electricity ATB: Battery Storage[EB/OL]. <https://atb-archive.nrel.gov/electricity/2019/index.html?t=st>.
- [2] GRIMALDI A, MINUTO F D, PEROL A, et al. Ageing and energy performance analysis of a utility-scale lithium-ion battery for power grid applications through a data-driven empirical modelling approach[J]. Journal of Energy Storage, 2023, 65: 107232. DOI: 10.1016/j.est.2023.107232.
- [3] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.
- [4] BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004.
- [5] PINTO E S, SERRA L M, LÁZARO A. Energy communities approach applied to optimize polygeneration systems in residential buildings: Case study in Zaragoza, Spain[J]. Sustainable Cities and Society, 2022, 82: 103885. DOI: 10.1016/j.scs.2022.103885.
- [6] TASHMAN L J. Out-of-sample tests of forecasting accuracy: an analysis and review[J]. International Journal of Forecasting, 2000, 16(4): 437–450. DOI: 10.1016/S0169-2070(00)00065-0.
- [7] SHAPIRO A, DENTCHEVA D, RUSZCZYNSKI A. Lectures on Stochastic Programming: Modeling and Theory[M]. 3rd ed. Philadelphia: SIAM, 2021. DOI: 10.1137/1.9781611976595.
- [8] PARISIO A, RIKOS E, GLIELMO L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813–1827. DOI: 10.1109/TCST.2013.2295737.
- [9] PINSON P, MADSEN H, NIELSEN H A, et al. From probabilistic forecasts to statistical scenarios of short-term wind power production[J]. Wind Energy, 2009, 12(1): 51–62. DOI: 10.1002/we.284.
- [10] DURANTE F, GIANFREDA A, RAVAZZOLO F, et al. A multivariate dependence analysis for electricity prices, demand and renewable energy sources[J]. Information Sciences, 2022, 590: 74–89. DOI: 10.1016/j.ins.2022.01.003.

附录 A 问题一完整计划购电量

下表按题目附件模板既有行序列出 144 个时段的计划购电量，单位为 kWh。时段标签保留原样，完整精度结果见随附的正式结果工作簿。

时间段	购电量
0:10-0:20	1406.6411
0:20-0:30	1406.3299
0:30-0:40	1407.3838
0:40-0:50	1409.0739
0:50-1:00	1409.8046
1:00-1:10	576.6400
1:10-1:20	577.8597
1:20-1:30	577.9939
1:30-1:40	637.4723
1:40-1:50	580.4154
1:50-2:00	582.5140
2:00-2:10	583.0740
2:10-2:20	584.2049
2:20-2:30	585.4398
2:30-2:40	586.1447
2:40-2:50	586.3208
2:50-3:00	586.4253
3:00-3:10	586.8496
3:10-3:20	587.8210
3:20-3:30	589.5476
3:30-3:40	590.8135
3:40-3:50	590.2131
3:50-4:00	591.0663
4:00-4:10	593.3678
4:10-4:20	593.6597
4:20-4:30	592.7864
4:30-4:40	593.3379
4:40-4:50	596.6758
4:50-5:00	597.7803
5:00-5:10	595.1625
5:10-5:20	590.6065
5:20-5:30	589.4852

时间段	购电量
5:30-5:40	578.4554
5:40-5:50	1383.6598
5:50-6:00	0.0000
6:00-6:10	0.0000
6:10-6:20	0.0000
6:20-6:30	0.0000
6:30-6:40	0.0000
6:40-6:50	0.0000
6:50-7:00	0.0000
7:00-7:10	0.0000
7:10-7:20	0.0000
7:20-7:30	0.0000
7:30-7:40	0.0000
7:40-7:50	0.0000
7:50-8:00	0.0000
8:00-8:10	0.0000
8:10-8:20	0.0000
8:20-8:30	0.0000
8:30-8:40	0.0000
8:40-8:50	0.0000
8:50-9:00	0.0000
9:00-9:10	0.0000
9:10-9:20	0.0000
9:20-9:30	0.0000
9:30-9:40	0.0000
9:40-9:50	0.0000
9:50-10:00	0.0000
10:00-10:10	0.0000
10:10-10:20	0.0000
10:20-10:30	0.0000
10:30-10:40	0.0000
10:40-10:50	0.0000
10:50-11:00	0.0000
11:00-11:10	0.0000
11:10-11:20	415.1177
11:20-11:30	0.0000

时间段	购电量
11:30-11:40	0.0000
11:40-11:50	0.0000
11:50-12:00	520.1245
12:00-12:10	486.4029
12:10-12:20	480.4124
12:20-12:30	485.3000
12:30-12:40	488.6070
12:40-12:50	498.2707
12:50-13:00	0.0000
13:00-13:10	0.0000
13:10-13:20	0.0000
13:20-13:30	0.0000
13:30-13:40	0.0000
13:40-13:50	0.0000
13:50-14:00	0.0000
14:00-14:10	0.0000
14:10-14:20	0.0000
14:20-14:30	0.0000
14:30-14:40	0.0000
14:40-14:50	0.0000
14:50-15:00	0.0000
15:00-15:10	112.0899
15:10-15:20	255.4856
15:20-15:30	197.5235
15:30-15:40	244.8140
15:40-15:50	295.5346
15:50-16:00	345.7775
16:00-16:10	394.9315
16:10-16:20	445.4317
16:20-16:30	495.6966
16:30-16:40	547.1191
16:40-16:50	599.9854
16:50-17:00	635.4297
17:00-17:10	660.8307
17:10-17:20	702.0369
17:20-17:30	738.0658

时间段	购电量
17:30-17:40	775.9447
17:40-17:50	826.0833
17:50-18:00	762.4228
18:00-18:10	636.9826
18:10-18:20	64.5344
18:20-18:30	678.8305
18:30-18:40	686.2685
18:40-18:50	696.5044
18:50-19:00	0.0000
19:00-19:10	0.0000
19:10-19:20	0.0000
19:20-19:30	0.0000
19:30-19:40	0.0000
19:40-19:50	0.0000
19:50-20:00	0.0000
20:00-20:10	0.0000
20:10-20:20	0.0000
20:20-20:30	0.0000
20:30-20:40	0.0000
20:40-20:50	0.0000
20:50-21:00	715.8689
21:00-21:10	717.0051
21:10-21:20	720.1603
21:20-21:30	717.0549
21:30-21:40	720.4460
21:40-21:50	738.1347
21:50-22:00	671.6936
22:00-22:10	1393.7903
22:10-22:20	1384.8989
22:20-22:30	568.8260
22:30-22:40	1396.9886
22:40-22:50	1397.7599
22:50-23:00	566.7474
23:00-23:10	1399.8738
23:10-23:20	567.8302
23:20-23:30	569.3586

时间段	购电量
23:30-23:40	1403.6635
23:40-23:50	631.2596
23:50-0:00+1	573.1768
0:00+1-0:10+1	574.1210

附录 B 支撑材料文件清单

支撑材料压缩包不包含题设原始附件；运行时按 `README.txt` 将题设附件置于约定路径。其余文件均为复算正文结果所需的程序、依赖说明、正式结果工作簿和 AI 工具使用详情。

目录或文件	内容与用途
<code>README.txt</code> 与 <code>requirements.txt</code>	运行环境、题设附件路径和复算步骤。
<code>q1/-q4/ scripts/ results/result1.xlsx- result4-3.xlsx</code>	四问正式计算所需的 Python 源程序与结果导出程序。 论文表格导出与数字对账脚本。 五份按题设模板导出的正式结果工作簿。
<code>results/reconciliation.md</code>	正文数字与正式结果工作簿的对账记录。
<code>AI_Tool_Usage_Details.pdf</code>	AI 工具使用的具体说明。

附录 C 建模程序代码

下列代码列出正式计算所需的核心源文件，并与支撑材料压缩包中的 q1/-q4/ 目录保持一致；压缩包保留原目录结构，便于按 README.txt 运行。

```
1
2 # ===== q1/__init__.py =====
3 from q1.config import DELTA_H, ETA_C, ETA_D, P_MAX_KW, T
4
5 C_MAX = P_MAX_KW * DELTA_H
6 D_MAX = P_MAX_KW * DELTA_H
7
8 __all__ = ["C_MAX", "D_MAX", "DELTA_H", "ETA_C", "ETA_D", "T"]
9
10 # ===== q1/config.py =====
11 """Q1 default constants. Runtime models take ModelParams; do not mutate these for cases."""
12
13 from __future__ import annotations
14
15 import os
16 from pathlib import Path
17
18 ROOT = Path(__file__).resolve().parents[1]
19
20 T = 144
21 DELTA_H = 1.0 / 6.0
22 NOMINAL_CAPACITY_KWH = 12000.0
23 P_MAX_KW = 5000.0
24 E_MAX_KWH = 10800.0
25 E_MIN_KWH = 1200.0
26 EO_KWH = 6000.0
27 ETA_RT = 0.90
28 ETA_C = ETA_RT**0.5
29 ETA_D = ETA_RT**0.5
30
31 BALANCE_TOL = 1e-6
32 SOC_END_TOL = 1e-6
33 SIMUL_CD_TOL = 1e-4
34 EPS_C = 1e-6
35 S_BOUND_TOL = 1e-8
36
37 SOLVER = "HIGHS"
38 OFFICIAL_MODEL = "M1"
39 EXPECTED_M1_COST = 33801.495542
40 EXPECTED_EFF_B_COST = 35126.948589
41 COST_REPRO_TOL = 1e-5
42
43 # Table 2: pending captain sign-off. PCC AC-bus side, not battery-internal.
44 TABLE2_ENERGY_SIDE = "pcc_ac_bus"
45 TABLE2_CHARGE_VAR = "c_t"
46 TABLE2_DISCHARGE_VAR = "d_t"
47
48 ROW_ORDER_POLICY = "attachment_raw_order"
49
50 DEFAULT_ATTACH_DIR = Path(
51     os.environ.get("CUMCM_C_ATTACH_DIR", ROOT / "data" / "raw" / "附件")
52 )
53 DEFAULT_ATTACH1 = Path(os.environ.get("CUMCM_C_ATTACH1", DEFAULT_ATTACH_DIR / "附件1.xlsx"))
54 DEFAULT_RESULT1_TEMPLATE = Path(
55     os.environ.get("CUMCM_C_RESULT1_TEMPLATE", DEFAULT_ATTACH_DIR / "附件5" / "result1.xlsx")
56 )
57
58 OUTPUT_DIR = ROOT / "output"
59 FIG_DIR = ROOT / "fig"
60
61 FOUR_HOUR_BLOCKS = [
```

```

62     ("0:00-4:00", 0, 24),
63     ("4:00-8:00", 24, 48),
64     ("8:00-12:00", 48, 72),
65     ("12:00-16:00", 72, 96),
66     ("16:00-20:00", 96, 120),
67     ("20:00-24:00", 120, 144),
68 ]
69
70 PURCHASE_SHEET = "计划购电量"
71 CHARGE_SHEET = "充放电量"
72 PURCHASE_HEADERS = ("时间段", "购电量")
73 CHARGE_HEADERS = ("时间段", "充电量", "放电量", "时刻", "储电量")
74 ALLOWED_PURCHASE_CELLS = {(r, 2) for r in range(2, 2 + T)}
75 ALLOWED_CHARGE_CELLS = {(r, 2) for r in range(2, 8)} | {(r, 3) for r in range(2, 8)} | {(2, 5), (3, 5)}
76
77 # ===== q1/export_results.py =====
78 from __future__ import annotations
79
80 from pathlib import Path
81 import shutil
82
83 import matplotlib.pyplot as plt
84 import numpy as np
85 import pandas as pd
86 from openpyxl import load_workbook
87
88 from q1.config import (
89     ALLOWED_CHARGE_CELLS,
90     ALLOWED_PURCHASE_CELLS,
91     CHARGE_SHEET,
92     DELTA_H,
93     EPS_C,
94     ETA_RT,
95     FIG_DIR,
96     FOUR_HOUR_BLOCKS,
97     OFFICIAL_MODEL,
98     PURCHASE_SHEET,
99     ROW_ORDER_POLICY,
100     SIMUL_CD_TOL,
101     T,
102     TABLE2_CHARGE_VAR,
103     TABLE2_DISCHARGE_VAR,
104     TABLE2_ENERGY_SIDE,
105 )
106 from q1.optimize import SolveResult
107
108
109 def _cjk_rc() -> None:
110     plt.rcParams["font.sans-serif"] = ["PingFang SC", "Heiti SC", "Songti SC", "Arial Unicode MS", "DejaVu Sans"]
111     plt.rcParams["axes.unicode_minus"] = False
112
113
114 def write_plan_csv(path: Path, labels: list[str], data: pd.DataFrame, res: SolveResult) -> None:
115     balance = res.g + data["pv_kwh"].to_numpy() + res.d - data["load_kwh"].to_numpy() - res.c - res.s
116     plan = pd.DataFrame(
117         {
118             "time": labels,
119             "price": data["price"],
120             "load_kwh": data["load_kwh"],
121             "pv_kwh": data["pv_kwh"],
122             "grid_purchase_kwh": res.g,
123             "charge_kwh": res.c,
124             "discharge_kwh": res.d,
125             "soc_kwh": res.E[1:],
126             "curtailment_kwh": res.s,
127             "balance_residual_kwh": balance,
128         }
129     )

```

```

130     path.parent.mkdir(parents=True, exist_ok=True)
131     plan.to_csv(path, index=False)
132
133
134 def write_summary_csv(path: Path, res: SolveResult) -> None:
135     p = res.params
136     rows = [
137         ("official_model", OFFICIAL_MODEL, "1", "正式方案; M2-M4 仅对照"),
138         ("grid_purchase_kwh", res.grid_purchase, "kWh", "全天计划购电量合计"),
139         ("purchase_cost_yuan", res.purchase_cost, "yuan", "全天购电费"),
140         ("peak_grid_kw", res.peak_grid_kw, "kW", "购电功率峰值  $\max(g_t)/\Delta$ "),
141         ("curtailment_kwh", res.curtailment, "kWh", "全天弃光量"),
142         ("throughput_kwh", res.throughput, "kWh", "充放电吞吐量  $Q=\sum(c+d)$ "),
143         ("soc_min_kwh", res.soc_min, "kWh", "SOC 最小值含 EO"),
144         ("soc_max_kwh", res.soc_max, "kWh", "SOC 最大值含 EO"),
145         ("EO_kwh", res.E[0], "kWh", "0:00 储电量"),
146         ("E144_kwh", res.E[-1], "kWh", "24:00 储电量"),
147         ("max_balance_residual_kwh", res.max_balance_residual, "kWh", "能量平衡残差绝对值最大"),
148         ("max_simultaneous_charge_discharge", res.max_cd, "kWh^2", " $\max(c_t * d_t)$ "),
149         ("max_s_minus_pv_kwh", res.max_s_minus_pv, "kWh", " $\max(s_t - P_t)$ , 应 $\leq 0$ "),
150         ("eta_rt", ETA_RT, "1", "主方案往返效率"),
151         ("eta_c", p.eta_c, "1", "充电效率"),
152         ("eta_d", p.eta_d, "1", "放电效率"),
153         ("c_max_kwh", p.c_max_kwh, "kWh", "每时段充电上限  $\text{power\_limit\_kw} * \Delta$ "),
154         ("d_max_kwh", p.d_max_kwh, "kWh", "每时段放电上限"),
155         ("power_limit_kw", p.power_limit_kw, "kW", "充放电功率上限"),
156         ("grid_limit_kw", "" if p.grid_limit_kw is None else p.grid_limit_kw, "kW", "主方案无上限"),
157         ("eps_C", EPS_C, "yuan", "词典序成本容差 (仅对照)"),
158         ("table2_energy_side", TABLE2_ENERGY_SIDE, "1", "表2计量侧, 待队长最终签收"),
159         ("table2_charge_var", TABLE2_CHARGE_VAR, "1", "充电量汇总变量"),
160         ("table2_discharge_var", TABLE2_DISCHARGE_VAR, "1", "放电量汇总变量"),
161         ("row_order_policy", ROW_ORDER_POLICY, "1", "附件原始行序口径 (队长确认)"),
162     ]
163     pd.DataFrame(rows, columns=["metric", "value", "unit", "meaning"]).to_csv(path, index=False)
164
165
166 def write_comparison_csv(path: Path, results: list[SolveResult]) -> None:
167     rows = []
168     for r in results:
169         rows.append(
170             {
171                 "case": r.name,
172                 "solver": "HIGHS",
173                 "purchase_cost_yuan": r.purchase_cost,
174                 "grid_purchase_kwh": r.grid_purchase,
175                 "peak_grid_kw": r.peak_grid_kw,
176                 "curtailment_kwh": r.curtailment,
177                 "throughput_kwh": r.throughput,
178                 "max_simultaneous_charge_discharge": r.max_cd,
179                 "soc_min_kwh": r.soc_min,
180                 "soc_max_kwh": r.soc_max,
181                 "max_balance_residual_kwh": r.max_balance_residual,
182                 "max_s_minus_pv_kwh": r.max_s_minus_pv,
183                 "solve_seconds": r.solve_seconds,
184                 "pass": r.passed,
185             }
186         )
187     pd.DataFrame(rows).to_csv(path, index=False)
188
189
190 def write_baseline_csv(
191     path: Path,
192     g_base: np.ndarray,
193     s_base: np.ndarray,
194     price: np.ndarray,
195     m1: SolveResult,
196 ) -> None:
197     g_sum = float(np.sum(g_base))

```

```

198     s_sum = float(np.sum(s_base))
199     cost = float(price @ g_base)
200     save = cost - m1.purchase_cost
201     ratio = save / cost if cost else np.nan
202     rows = [
203         ("baseline_grid_purchase_kwh", g_sum, "kWh", "无储能无售电:  $g=\max(L-P,0)$ "),
204         ("baseline_purchase_cost_yuan", cost, "yuan", "基准购电费"),
205         ("baseline_curtailment_kwh", s_sum, "kWh", "基准弃光  $s=\max(P-L,0)$ "),
206         ("m1_grid_purchase_kwh", m1.grid_purchase, "kWh", "正式 M1 总购电量"),
207         ("m1_purchase_cost_yuan", m1.purchase_cost, "yuan", "正式 M1 购电费"),
208         ("absolute_cost_saving_yuan", save, "yuan", "基准费用减 M1 费用"),
209         ("cost_saving_ratio", ratio, "1", "节省比例 (相对基准)"),
210     ]
211     pd.DataFrame(rows, columns=["metric", "value", "unit", "meaning"]).to_csv(path, index=False)
212
213
214 def write_sensitivity_csv(path: Path, rows: list[dict]) -> None:
215     cols = [
216         "case",
217         "eta_c",
218         "eta_d",
219         "nominal_capacity_kwh",
220         "soc_min_kwh",
221         "soc_max_kwh",
222         "power_limit_kw",
223         "grid_limit_kw",
224         "purchase_cost_yuan",
225         "grid_purchase_kwh",
226         "peak_grid_kw",
227         "curtailment_kwh",
228         "throughput_kwh",
229         "soc_observed_min_kwh",
230         "soc_observed_max_kwh",
231         "pass",
232     ]
233     pd.DataFrame(rows)[cols].to_csv(path, index=False)
234
235
236 def sensitivity_row(case: str, res: SolveResult) -> dict:
237     p = res.params
238     return {
239         "case": case,
240         "eta_c": p.eta_c,
241         "eta_d": p.eta_d,
242         "nominal_capacity_kwh": p.nominal_capacity_kwh,
243         "soc_min_kwh": p.e_min_kwh,
244         "soc_max_kwh": p.e_max_kwh,
245         "power_limit_kw": p.power_limit_kw,
246         "grid_limit_kw": "" if p.grid_limit_kw is None else p.grid_limit_kw,
247         "purchase_cost_yuan": res.purchase_cost,
248         "grid_purchase_kwh": res.grid_purchase,
249         "peak_grid_kw": res.peak_grid_kw,
250         "curtailment_kwh": res.curtailment,
251         "throughput_kwh": res.throughput,
252         "soc_observed_min_kwh": res.soc_min,
253         "soc_observed_max_kwh": res.soc_max,
254         "pass": res.passed,
255     }
256
257
258 def write_model_selection_md(path: Path, results: dict[str, SolveResult], official: str) -> None:
259     m1, m2, m3, m4 = results["M1"], results["M2"], results["M3"], results["M4"]
260     lines = [
261         "# Q1 模型形式对照与选择",
262         "",
263         f"- 正式方案为 **{official}** (纯成本最优 LP)。M2-M4 仅作数值对照, 不得覆盖 `result1.xlsx` / 正式图。",
264         "- M2 是在成本最优容差内再最小化吞吐量的数值择优, **不能**据此声称降低循环寿命损耗或延长电池寿命。",
265         f"- 词典序成本容差 `eps_C` = {EPS_C} 元 (仅用于 M2/M4 对照)。",

```



```

266     f"- 效率口径 A (正式) : eta_rt = {ETA_RT}, eta_c = eta_d = sqrt(0.9) approx {m1.params.eta_c:.12f}。",
267     "",
268     "## 结果摘要",
269     "",
270     "| case | cost | G | peak kW | S | Q | max c*d | SOC min/max | residual | seconds | pass |",
271     "| ---|---|---|---|---|---|---|---|---|---|---|",
272 ]
273 for key in ("M1", "M2", "M3", "M4"):
274     r = results[key]
275     lines.append(
276         f"| {key} | {r.purchase_cost:.6f} | {r.grid_purchase:.6f} | {r.peak_grid_kw:.6f} | "
277         f"{r.curtailment:.6f} | {r.throughput:.6f} | {r.max_cd:.3e} | "
278         f"{r.soc_min:.4f}/{r.soc_max:.4f} | {r.max_balance_residual:.3e} | "
279         f"{r.solve_seconds:.3f} | {r.passed} | "
280     )
281     lines += [
282         "",
283         "## 约束检查",
284         "",
285         f"- M1 同时充放电 max(c d) = {m1.max_cd:.6e}; 阈值 {SIMUL_CD_TOL}。",
286         f"- M1 与 M3 成本差 = {abs(m1.purchase_cost - m3.purchase_cost):.6e}。",
287         f"- M2 相对 M1 成本增量 = {m2.purchase_cost - m1.purchase_cost:.6e} (应 <= eps_C)。",
288         f"- M2 相对 M1 吞吐量变化 ΔQ = {m2.throughput - m1.throughput:.6f}。",
289         f"- M2 与 M4 成本差 = {abs(m2.purchase_cost - m4.purchase_cost):.6e}, ΔQ = {m2.throughput - m4.throughput:.6f}。",
290         f"- M1 max(s-P) = {m1.max_s_minus_pv:.6e}。",
291         "",
292         "## 推荐",
293         "",
294         "- 正式采用 M1: 目标仅为购电费, 与题面一致。",
295     ]
296     if m1.passed and m3.passed and abs(m1.purchase_cost - m3.purchase_cost) <= EPS_C and m1.max_cd <= SIMUL_CD_TOL:
297         lines.append("- M1 与 M3 成本相同且同时充放电仅为数值残差, 正式模型不需要二元互斥变量。")
298     if m2.passed and m2.purchase_cost <= m1.purchase_cost + EPS_C + 1e-9:
299         lines.append("- M2 可在对照表中作为同等费用下的低吞吐量解, 但不作为正式答卷。")
300     lines += ["", "## 求解状态", ""]
301     for key, r in results.items():
302         lines.append(f"- {key}: status={r.status}; notes={r.notes or 'ok'}")
303     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
304
305
306 def write_validation_md(
307     path: Path,
308     data: pd.DataFrame,
309     labels: list[str],
310     official: SolveResult,
311     attach_path: Path,
312     template_path: Path,
313     price_min: float,
314     gmax_min_kw: float,
315 ) -> None:
316     mapping_rows = []
317     for i in [0, 1, 2, T // 2 - 1, T - 3, T - 2, T - 1]:
318         mapping_rows.append(
319             f"| {i + 1} | {data['attach_time'].iloc[i]} | {labels[i]} | "
320         )
321     p = official.params
322     lines = [
323         "## Q1 验收",
324         "",
325         f"- 附件1: `{attach_path}`",
326         f"- 模板: `{template_path}`",
327         f"- 正式方案: {official.name} ({OFFICIAL_MODEL}), solver=HIGHS, status={official.status}",
328         f"- **附件原始行序口径 (队长确认) **: 不按时间排序, 不把末行旋转到首行。",
329         f"- 行序策略代码标记: `{ROW_ORDER_POLICY}`",
330         "",
331         "## 时间标签映射",
332         "",
333         "内部模型用整数索引 t=1..144。SOC 独立使用 E_0,...,E_144, 其中 E_0=E_144=6000 kWh 对应 0:00 与 24:00。",

```

```

334     "导出时按官方模板**既有行序**填写第 t 个购电结果，不新增 `0:00-0:10`，不改标签。",
335     "",
336     "规则：内部索引 t → 附件1第 t 条数据 → 模板第 t 条时间标签。",
337     "",
338     "| t | 附件1时间 | 模板时间 |",
339     "|---:|---:|---:",
340     *mapping_rows,
341     "",
342     "附件1首条为 `00:10:00`，模板首条为 `0:10-0:20`；本题按行序一一对应。",
343     "四小时充放电汇总按内部索引每 24 个时段分块：t=1..24, 25..48, ..., 121..144。",
344     "",
345     "## 输入检查",
346     "",
347     f"- 价格最小值 = {price_min:.6g} 元/kWh。当前附件无负价；若出现负价必须报错，不得沿用“无需同时充放电”的结论。",
348     "",
349     "## 表 2 计量侧（待队长最终签收）",
350     "",
351     f"- 集中定义：`TABLE2_ENERGY_SIDE={TABLE2_ENERGY_SIDE}`。",
352     f"- 充电量 = `{TABLE2_CHARGE_VAR}`（公共母线侧充入储能）。",
353     f"- 放电量 = `{TABLE2_DISCHARGE_VAR}`（公共母线侧储能放电）。",
354     "- 未换算电池内部能量；签收前不得在多处隐式改写。",
355     "",
356     "## 数值验收",
357     "",
358     f"- 时段数：{T}",
359     f"- 功率已乘 Δ={DELTA_H} h 转为 kWh",
360     f"- max |能量平衡残差| = {official.max_balance_residual:.6e} kWh（阈值 1e-6）",
361     f"- max c,d = {official.c.max():.6f}, {official.d.max():.6f} kWh（上限 {p.c_max_kwh:.9f}）",
362     f"- SOC min/max = {official.soc_min:.6f} / {official.soc_max:.6f} kWh（[{p.e_min_kwh}, {p.e_max_kwh}]）",
363     f"- E0 = {official.E[0]:.12f}, E144 = {official.E[-1]:.12f}, |E144-E0|={abs(official.E[-1]-p.e0_kwh):.6e}",
364     f"- max(c_t d_t) = {official.max_cd:.6e}（阈值 1e-4）",
365     f"- max(s_t-P_t) = {official.max_s_minus_pv:.6e}",
366     f"- 峰值购电功率 = {official.peak_grid_kw:.6f} kW（主方案无 PCC 上限）",
367     f"- PCC 敏感性：最小可行峰值约 {gmax_min_kw:.6f} kW，该值**不是**题给参数。",
368     f"- 验收通过：{official.passed}",
369     "",
370 ]
371 if not official.passed:
372     lines.append(f"**未通过：** {official.notes}")
373 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
374
375
376 def table2_block_energy(res: SolveResult) -> tuple[np.ndarray, np.ndarray]:
377     """PCC AC-bus side: charge = c_t, discharge = d_t. Single definition for Table 2."""
378     charge = np.array([float(np.sum(res.c[a:b])) for _n, a, b in FOUR_HOUR_BLOCKS])
379     discharge = np.array([float(np.sum(res.d[a:b])) for _n, a, b in FOUR_HOUR_BLOCKS])
380     return charge, discharge
381
382
383 def write_result1_xlsx(template: Path, dest: Path, res: SolveResult) -> None:
384     dest.parent.mkdir(parents=True, exist_ok=True)
385     shutil.copyfile(template, dest)
386     wb = load_workbook(dest)
387     ws_g = wb[PURCHASE_SHEET]
388     for i in range(T):
389         header = ws_g.cell(row=i + 2, column=1).value
390         if header is None:
391             raise RuntimeError(f"template row {i+2} missing time label")
392         ws_g.cell(row=i + 2, column=2).value = float(res.g[i])
393     ws_cd = wb[CHARGE_SHEET]
394     charge, discharge = table2_block_energy(res)
395     for row, val_c, val_d in zip(range(2, 8), charge, discharge):
396         ws_cd.cell(row=row, column=2).value = val_c
397         ws_cd.cell(row=row, column=3).value = val_d
398     ws_cd.cell(row=2, column=5).value = float(res.E[0])
399     ws_cd.cell(row=3, column=5).value = float(res.E[-1])
400     wb.save(dest)
401     verify_xlsx_only_allowed_cells(template, dest)

```

```

402
403
404 def verify_xlsx_only_allowed_cells(template: Path, dest: Path) -> None:
405     src = load_workbook(template, data_only=False)
406     out = load_workbook(dest, data_only=False)
407     for name in src.sheetnames:
408         ws_s, ws_o = src[name], out[name]
409         for row in ws_s.iter_rows(min_row=1, max_row=ws_s.max_row, max_col=ws_s.max_column):
410             for cell in row:
411                 key = (cell.row, cell.column)
412                 if name == PURCHASE_SHEET and key in ALLOWED_PURCHASE_CELLS:
413                     continue
414                 if name == CHARGE_SHEET and key in ALLOWED_CHARGE_CELLS:
415                     continue
416                 if ws_o.cell(cell.row, cell.column).value != cell.value:
417                     raise RuntimeError(
418                         f"modified non-fillable cell {name}!{cell.coordinate}: "
419                         f"{cell.value!r} -> {ws_o.cell(cell.row, cell.column).value!r}"
420                     )
421
422
423 def write_figures(data: pd.DataFrame, res: SolveResult) -> None:
424     _cjk_rc()
425     FIG_DIR.mkdir(parents=True, exist_ok=True)
426     hours = (np.arange(T) + 1) * DELTA_H
427
428     fig, ax = plt.subplots(figsize=(10, 4.2))
429     ax.plot(hours, data["load_kwh"], label="负荷 (kWh/10min)", lw=1.2)
430     ax.plot(hours, data["pv_kwh"], label="光伏 (kWh/10min)", lw=1.2)
431     ax.plot(hours, res.g, label="购电 (kWh)", lw=1.2)
432     ax.plot(hours, res.c, label="充电 (kWh)", lw=1.0)
433     ax.plot(hours, res.d, label="放电 (kWh)", lw=1.0)
434     ax.set_xlabel("时刻 (h)")
435     ax.set_ylabel("能量 (kWh / 时段)")
436     ax.set_xlim(0, 24)
437     ax.grid(True, alpha=0.3)
438     ax.legend(ncol=3, fontsize=8)
439     fig.tight_layout()
440     fig.savefig(FIG_DIR / "fig1_dispatch.png", dpi=200)
441     plt.close(fig)
442
443     fig, ax1 = plt.subplots(figsize=(10, 4.2))
444     ax1.plot(np.concatenate([[0.0], hours]), res.E, color="#1D4ED8", label="SOC (kWh)")
445     ax1.set_xlabel("时刻 (h)")
446     ax1.set_ylabel("SOC (kWh)")
447     ax1.set_xlim(0, 24)
448     ax1.set_ylim(1000, 11000)
449     ax2 = ax1.twinx()
450     ax2.step(hours, data["price"], where="pre", color="#B45309", label="电价 (元/kWh)", alpha=0.85)
451     ax2.set_ylabel("电价 (元/kWh)")
452     h1, l1 = ax1.get_legend_handles_labels()
453     h2, l2 = ax2.get_legend_handles_labels()
454     ax1.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=8)
455     ax1.grid(True, alpha=0.3)
456     fig.tight_layout()
457     fig.savefig(FIG_DIR / "fig1_soc_price.png", dpi=200)
458     plt.close(fig)
459
460 # ===== q1/load_data.py =====
461 from __future__ import annotations
462
463 from datetime import datetime, time
464 from pathlib import Path
465
466 import numpy as np
467 import pandas as pd
468 from openpyxl import load_workbook
469

```

```

470 from q1.config import (
471     CHARGE_HEADERS,
472     CHARGE_SHEET,
473     DEFAULT_ATTACH1,
474     DEFAULT_RESULT1_TEMPLATE,
475     DELTA_H,
476     FOUR_HOUR_BLOCKS,
477     PURCHASE_HEADERS,
478     PURCHASE_SHEET,
479     T,
480 )
481
482
483 def normalize_attach_time(value) -> str:
484     if isinstance(value, datetime):
485         return value.strftime("%H:%M:%S")
486     if isinstance(value, time):
487         return value.strftime("%H:%M:%S")
488     return str(value).strip()
489
490
491 def load_attachment1(path: Path | None = None) -> pd.DataFrame:
492     """Preserve Excel raw row order. Do not sort or rotate."""
493     path = Path(path or DEFAULT_ATTACH1)
494     if not path.exists():
495         raise FileNotFoundError(
496             f"附件1 not found: {path}. Set CUMCM_C_ATTACH1 to the xlsx path."
497         )
498     df = pd.read_excel(path, sheet_name="Sheet1")
499     expected = ["时间", "电价", "小区负载", "光伏发电预测功率"]
500     if list(df.columns) != expected:
501         raise ValueError(f"Unexpected 附件1 columns: {list(df.columns)}")
502     if len(df) != T:
503         raise ValueError(f"附件1 must have {T} rows, got {len(df)}")
504
505     times = df["时间"].map(normalize_attach_time)
506     price = pd.to_numeric(df["电价"], errors="coerce").to_numpy(dtype=float)
507     load_kw = pd.to_numeric(df["小区负载"], errors="coerce").to_numpy(dtype=float)
508     pv_kw = pd.to_numeric(df["光伏发电预测功率"], errors="coerce").to_numpy(dtype=float)
509
510     if not np.isfinite(price).all() or not np.isfinite(load_kw).all() or not np.isfinite(pv_kw).all():
511         raise ValueError("附件1 contains non-finite numeric values")
512     if (load_kw < 0).any() or (pv_kw < 0).any():
513         raise ValueError("附件1 has negative load or PV")
514     if (price < 0).any():
515         raise ValueError(
516             "附件1 has negative prices; simultaneous charge/discharge logic must be revisited"
517         )
518
519     if times.iloc[0] != "00:10:00":
520         raise ValueError(f"First attach time must be 00:10:00, got {times.iloc[0]!r}")
521     if times.iloc[-1] != "0:00+1":
522         raise ValueError(f"Last attach time must be 0:00+1, got {times.iloc[-1]!r}")
523
524     out = pd.DataFrame(
525         {
526             "t": np.arange(1, T + 1),
527             "attach_time": times.to_numpy(),
528             "price": price,
529             "load_kw": load_kw,
530             "pv_kw": pv_kw,
531         }
532     )
533     out["load_kwh"] = out["load_kw"] * DELTA_H
534     out["pv_kwh"] = out["pv_kw"] * DELTA_H
535     out.attrs["row_order_policy"] = "attachment_raw_order"
536     return out
537

```

```

538
539 def inspect_template(path: Path | None = None) -> tuple[list[str], list[str]]:
540     path = Path(path or DEFAULT_RESULT1_TEMPLATE)
541     if not path.exists():
542         raise FileNotFoundError(
543             f"result1 template not found: {path}. Set CUMCM_C_RESULT1_TEMPLATE."
544         )
545     wb = load_workbook(path)
546     if wb.sheetnames != [PURCHASE_SHEET, CHARGE_SHEET]:
547         raise ValueError(f"Unexpected sheets: {wb.sheetnames}")
548     ws_g = wb[PURCHASE_SHEET]
549     headers_g = (ws_g.cell(1, 1).value, ws_g.cell(1, 2).value)
550     if headers_g != PURCHASE_HEADERS:
551         raise ValueError(f"计划购电量表头不符: {headers_g}")
552     labels = [ws_g.cell(row=i, column=1).value for i in range(2, 2 + T)]
553     if any(x is None or str(x).strip() == "" for x in labels):
554         raise ValueError("result1 template has empty time labels")
555     labels = [str(x) for x in labels]
556     if labels[0] != "0:10-0:20" or labels[-1] != "0:00+1-0:10+1":
557         raise ValueError(f"Unexpected purchase labels: {labels[0]!r} .. {labels[-1]!r}")
558
559     ws_cd = wb[CHARGE_SHEET]
560     headers_cd = tuple(ws_cd.cell(1, c).value for c in range(1, 6))
561     if headers_cd != CHARGE_HEADERS:
562         raise ValueError(f"充放电量表头不符: {headers_cd}")
563     block_labels = [str(ws_cd.cell(row=i, column=1).value) for i in range(2, 8)]
564     expected_blocks = [name for name, _a, _b in FOUR_HOUR_BLOCKS]
565     if block_labels != expected_blocks:
566         raise ValueError(f"四小时区间标签不符: {block_labels}")
567     return labels, block_labels
568
569
570 def load_template_labels(path: Path | None = None) -> list[str]:
571     labels, _blocks = inspect_template(path)
572     return labels
573
574 # ===== q1/optimize.py =====
575 from __future__ import annotations
576
577 from dataclasses import dataclass
578 import time
579
580 import cvxpy as cp
581 import numpy as np
582
583 from q1.config import (
584     BALANCE_TOL,
585     DELTA_H,
586     EO_KWH,
587     E_MAX_KWH,
588     E_MIN_KWH,
589     EPS_C,
590     ETA_C,
591     ETA_D,
592     NOMINAL_CAPACITY_KWH,
593     P_MAX_KW,
594     S_BOUND_TOL,
595     SIMUL_CD_TOL,
596     SOC_END_TOL,
597     SOLVER,
598     T,
599 )
600
601
602 @dataclass
603 class ModelParams:
604     eta_c: float = ETA_C
605     eta_d: float = ETA_D

```

```

606     e_min_kwh: float = E_MIN_KWH
607     e_max_kwh: float = E_MAX_KWH
608     e0_kwh: float = EO_KWH
609     power_limit_kw: float = P_MAX_KW
610     grid_limit_kw: float | None = None
611     nominal_capacity_kwh: float = NOMINAL_CAPACITY_KWH
612
613     @property
614     def c_max_kwh(self) -> float:
615         return self.power_limit_kw * DELTA_H
616
617     @property
618     def d_max_kwh(self) -> float:
619         return self.power_limit_kw * DELTA_H
620
621     @property
622     def g_max_kwh(self) -> float | None:
623         if self.grid_limit_kw is None:
624             return None
625         return self.grid_limit_kw * DELTA_H
626
627
628 def default_params() -> ModelParams:
629     return ModelParams()
630
631
632 def soc_window(nominal_capacity_kwh: float) -> tuple[float, float]:
633     return 0.10 * nominal_capacity_kwh, 0.90 * nominal_capacity_kwh
634
635
636 @dataclass
637 class SolveResult:
638     name: str
639     exclusive: bool
640     lexico: bool
641     status: str
642     g: np.ndarray
643     c: np.ndarray
644     d: np.ndarray
645     s: np.ndarray
646     E: np.ndarray
647     purchase_cost: float
648     grid_purchase: float
649     curtailment: float
650     throughput: float
651     max_cd: float
652     soc_min: float
653     soc_max: float
654     max_balance_residual: float
655     peak_grid_kw: float
656     max_s_minus_pv: float
657     solve_seconds: float
658     passed: bool
659     params: ModelParams
660     notes: str = ""
661
662
663 def _build_problem(
664     price: np.ndarray,
665     load: np.ndarray,
666     pv: np.ndarray,
667     exclusive: bool,
668     cost_cap: float | None,
669     params: ModelParams,
670     peak_var: bool,
671 ):
672     g = cp.Variable(T, nonneg=True)
673     c = cp.Variable(T, nonneg=True)

```

```

674 d = cp.Variable(T, nonneg=True)
675 s = cp.Variable(T, nonneg=True)
676 E = cp.Variable(T + 1)
677 c_max = params.c_max_kwh
678 d_max = params.d_max_kwh
679 cons = [
680     E[0] == params.e0_kwh,
681     E[T] == params.e0_kwh,
682     E[1:] >= params.e_min_kwh,
683     E[1:] <= params.e_max_kwh,
684     c <= c_max,
685     d <= d_max,
686     s <= pv,
687     g + pv + d == load + c + s,
688     E[1:] == E[:-1] + params.eta_c * c - d / params.eta_d,
689 ]
690 if params.g_max_kwh is not None:
691     cons.append(g <= params.g_max_kwh)
692 z = None
693 if exclusive:
694     z = cp.Variable(T, boolean=True)
695     cons += [c <= c_max * z, d <= d_max * (1 - z)]
696 peak = None
697 if peak_var:
698     peak = cp.Variable(nonneg=True)
699     cons.append(g <= peak)
700     objective = cp.Minimize(peak)
701 elif cost_cap is not None:
702     cons.append(price @ g <= cost_cap)
703     objective = cp.Minimize(cp.sum(c + d))
704 else:
705     objective = cp.Minimize(price @ g)
706 return cp.Problem(objective, cons), g, c, d, s, E, z, peak
707
708
709 def _metrics(price, load, pv, g, c, d, s, E, params: ModelParams) -> dict:
710     balance = g + pv + d - load - c - s
711     return {
712         "purchase_cost": float(price @ g),
713         "grid_purchase": float(np.sum(g)),
714         "curtailment": float(np.sum(s)),
715         "throughput": float(np.sum(c + d)),
716         "max_cd": float(np.max(c + d)),
717         "soc_min": float(np.min(E)),
718         "soc_max": float(np.max(E)),
719         "max_balance_residual": float(np.max(np.abs(balance))),
720         "e_end_gap": float(abs(E[-1] - params.e0_kwh)),
721         "c_over": float(np.max(c - params.c_max_kwh)),
722         "d_over": float(np.max(d - params.d_max_kwh)),
723         "peak_grid_kw": float(np.max(g) / DELTA_H),
724         "max_s_minus_pv": float(np.max(s - pv)),
725         "g_over": float(np.max(g - (params.g_max_kwh if params.g_max_kwh is not None else np.inf))),
726     }
727
728
729 def _pass(m: dict, params: ModelParams) -> tuple[bool, str]:
730     reasons = []
731     if m["max_balance_residual"] >= BALANCE_TOL:
732         reasons.append(f"balance {m['max_balance_residual']}")
733     if m["e_end_gap"] >= SOC_END_TOL:
734         reasons.append(f"E144 {m['e_end_gap']}")
735     if m["soc_min"] < params.e_min_kwh - SOC_END_TOL:
736         reasons.append(f"soc_min {m['soc_min']}")
737     if m["soc_max"] > params.e_max_kwh + SOC_END_TOL:
738         reasons.append(f"soc_max {m['soc_max']}")
739     if m["c_over"] > BALANCE_TOL or m["d_over"] > BALANCE_TOL:
740         reasons.append("power limit")
741     if m["max_cd"] > SIMUL_CD_TOL:

```

```

742         reasons.append(f"max c*d {m['max_cd']}")
743     if m["max_s_minus_pv"] > S_BOUND_TOL:
744         reasons.append(f"s>P {m['max_s_minus_pv']}")
745     if m["g_over"] > BALANCE_TOL:
746         reasons.append("grid limit")
747     return (len(reasons) == 0, "; ".join(reasons))
748
749
750 def solve_stage(
751     name: str,
752     price: np.ndarray,
753     load: np.ndarray,
754     pv: np.ndarray,
755     exclusive: bool = False,
756     cost_cap: float | None = None,
757     params: ModelParams | None = None,
758 ) -> SolveResult:
759     params = params or default_params()
760     prob, g, c, d, s, E, _z, _peak = _build_problem(
761         price, load, pv, exclusive, cost_cap, params, peak_var=False
762     )
763     t0 = time.perf_counter()
764     prob.solve(solver=SOLVER, verbose=False)
765     elapsed = time.perf_counter() - t0
766     if g.value is None:
767         return SolveResult(
768             name=name,
769             exclusive=exclusive,
770             lexico=cost_cap is not None,
771             status=str(prob.status),
772             g=np.zeros(T),
773             c=np.zeros(T),
774             d=np.zeros(T),
775             s=np.zeros(T),
776             E=np.full(T + 1, np.nan),
777             purchase_cost=np.nan,
778             grid_purchase=np.nan,
779             curtailment=np.nan,
780             throughput=np.nan,
781             max_cd=np.nan,
782             soc_min=np.nan,
783             soc_max=np.nan,
784             max_balance_residual=np.nan,
785             peak_grid_kw=np.nan,
786             max_s_minus_pv=np.nan,
787             solve_seconds=elapsed,
788             passed=False,
789             params=params,
790             notes=f"solver status={prob.status}",
791         )
792     gv = np.asarray(g.value, dtype=float).ravel()
793     cv = np.asarray(c.value, dtype=float).ravel()
794     dv = np.asarray(d.value, dtype=float).ravel()
795     sv = np.asarray(s.value, dtype=float).ravel()
796     Ev = np.asarray(E.value, dtype=float).ravel()
797     m = _metrics(price, load, pv, gv, cv, dv, sv, Ev, params)
798     ok, why = _pass(m, params)
799     return SolveResult(
800         name=name,
801         exclusive=exclusive,
802         lexico=cost_cap is not None,
803         status=str(prob.status),
804         g=gv,
805         c=cv,
806         d=dv,
807         s=sv,
808         E=Ev,
809         purchase_cost=m["purchase_cost"],

```



```

810     grid_purchase=m["grid_purchase"],
811     curtailment=m["curtailment"],
812     throughput=m["throughput"],
813     max_cd=m["max_cd"],
814     soc_min=m["soc_min"],
815     soc_max=m["soc_max"],
816     max_balance_residual=m["max_balance_residual"],
817     peak_grid_kw=m["peak_grid_kw"],
818     max_s_minus_pv=m["max_s_minus_pv"],
819     solve_seconds=elapsed,
820     passed=ok,
821     params=params,
822     notes=why,
823 )
824
825
826 def solve_lexico(
827     name: str,
828     price,
829     load,
830     pv,
831     exclusive: bool,
832     params: ModelParams | None = None,
833 ) -> tuple[SolveResult, SolveResult]:
834     params = params or default_params()
835     s1 = solve_stage(name + "_stage1", price, load, pv, exclusive, None, params)
836     if not np.isfinite(s1.purchase_cost):
837         return s1, s1
838     cap = s1.purchase_cost + EPS_C
839     s2 = solve_stage(name, price, load, pv, exclusive, cap, params)
840     s2.notes = (s2.notes + f"; C*={s1.purchase_cost:.12g}; eps_C={EPS_C}").strip("; ")
841     if s2.purchase_cost > s1.purchase_cost + EPS_C + 1e-8:
842         s2.passed = False
843         s2.notes += "; lexico cost exceeded C*+eps"
844     return s1, s2
845
846
847 def min_feasible_peak_kw(price, load, pv, params: ModelParams | None = None) -> float:
848     params = params or default_params()
849     prob, g, _c, _d, _s, _E, _z, peak = _build_problem(
850         price, load, pv, exclusive=False, cost_cap=None, params=params, peak_var=True
851     )
852     prob.solve(solver=SOLVER, verbose=False)
853     if peak is None or peak.value is None:
854         raise RuntimeError(f"min peak LP failed: {prob.status}")
855     return float(peak.value) / DELTA_H
856
857 # ===== q1/run_q1.py =====
858 #!/usr/bin/env python3
859 """C 题 Q1: 按 docs/handoff/cursor-c-q1-second-revision.md 求解并导出。"""
860
861 from __future__ import annotations
862
863 import sys
864 from pathlib import Path
865
866 import numpy as np
867
868 ROOT = Path(__file__).resolve().parents[1]
869 if str(ROOT) not in sys.path:
870     sys.path.insert(0, str(ROOT))
871
872 from q1.config import ( # noqa: E402
873     COST_REPRO_TOL,
874     DEFAULT_ATTACH1,
875     DEFAULT_RESULT1_TEMPLATE,
876     EPS_C,
877     EXPECTED_EFF_B_COST,

```

```

878     EXPECTED_M1_COST,
879     OFFICIAL_MODEL,
880     OUTPUT_DIR,
881     SIMUL_CD_TOL,
882 )
883 from q1.export_results import ( # noqa: E402
884     sensitivity_row,
885     write_baseline_csv,
886     write_comparison_csv,
887     write_figures,
888     write_model_selection_md,
889     write_plan_csv,
890     write_result1_xlsx,
891     write_sensitivity_csv,
892     write_summary_csv,
893     write_validation_md,
894 )
895 from q1.load_data import inspect_template, load_attachment1 # noqa: E402
896 from q1.optimize import ( # noqa: E402
897     ModelParams,
898     SolveResult,
899     default_params,
900     min_feasible_peak_kw,
901     soc_window,
902     solve_lexico,
903     solve_stage,
904 )
905
906 N_PCC_POINTS = 5
907
908
909 def _require(res: SolveResult, context: str) -> None:
910     if not res.passed:
911         raise SystemExit(f"{context} failed: status={res.status}; {res.notes}")
912     if res.max_cd > SIMUL_CD_TOL:
913         raise SystemExit(
914             f"{context}: max(c_t d_t)={res.max_cd:.6e} exceeds {SIMUL_CD_TOL}; stop per handoff."
915         )
916
917
918 def no_storage_baseline(load: np.ndarray, pv: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
919     g = np.maximum(load - pv, 0.0)
920     s = np.maximum(pv - load, 0.0)
921     return g, s
922
923
924 def main() -> None:
925     attach = DEFAULT_ATTACH1
926     template = DEFAULT_RESULT1_TEMPLATE
927     data = load_attachment1(attach)
928     labels, _blocks = inspect_template(template)
929     price = data["price"].to_numpy()
930     load = data["load_kwh"].to_numpy()
931     pv = data["pv_kwh"].to_numpy()
932     params = default_params()
933     price_min = float(np.min(price))
934
935     m1 = solve_stage("M1", price, load, pv, exclusive=False, cost_cap=None, params=params)
936     _m1s, m2 = solve_lexico("M2", price, load, pv, exclusive=False, params=params)
937     m3 = solve_stage("M3", price, load, pv, exclusive=True, cost_cap=None, params=params)
938     _m3s, m4 = solve_lexico("M4", price, load, pv, exclusive=True, params=params)
939
940     results = {"M1": m1, "M2": m2, "M3": m3, "M4": m4}
941     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
942     write_comparison_csv(OUTPUT_DIR / "q1_model_comparison.csv", [m1, m2, m3, m4])
943     write_model_selection_md(OUTPUT_DIR / "q1_model_selection.md", results, official=OFFICIAL_MODEL)
944
945     _require(m1, "M1")

```

```

946     _require(m3, "M3")
947     if abs(m1.purchase_cost - m3.purchase_cost) > EPS_C:
948         raise SystemExit(
949             f"M1 vs M3 cost mismatch: {m1.purchase_cost} vs {m3.purchase_cost}"
950         )
951     if abs(m1.purchase_cost - EXPECTED_M1_COST) > COST_REPRO_TOL:
952         raise SystemExit(
953             f"M1 cost {m1.purchase_cost:.12f} differs from confirmed {EXPECTED_M1_COST} "
954             f"by more than {COST_REPRO_TOL} yuan; stop."
955         )
956     if m1.max_s_minus_pv > 1e-8:
957         raise SystemExit(f"s_t > P_t: {m1.max_s_minus_pv}")
958
959     official = m1
960     write_plan_csv(OUTPUT_DIR / "q1_plan.csv", labels, data, official)
961     write_summary_csv(OUTPUT_DIR / "q1_summary.csv", official)
962     write_result1_xlsx(template, OUTPUT_DIR / "result1.xlsx", official)
963     write_figures(data, official)
964
965     g_base, s_base = no_storage_baseline(load, pv)
966     write_baseline_csv(OUTPUT_DIR / "q1_baseline.csv", g_base, s_base, price, official)
967
968     sens_rows: list[dict] = []
969     sens_rows.append(sensitivity_row("eff_A", m1))
970
971     p_b = ModelParams(eta_c=0.9, eta_d=0.9)
972     m_b = solve_stage("eff_B", price, load, pv, params=p_b)
973     _require(m_b, "eff_B")
974     if abs(m_b.purchase_cost - EXPECTED_EFF_B_COST) > 1e-3:
975         raise SystemExit(
976             f"eff_B cost {m_b.purchase_cost} != expected {EXPECTED_EFF_B_COST}; "
977             "check that eta_c/eta_d are passed into the model."
978         )
979     sens_rows.append(sensitivity_row("eff_B", m_b))
980
981     for case, nom in (("cap_minus10", 10800.0), ("cap_plus10", 13200.0)):
982         e_min, e_max = soc_window(nom)
983         p = ModelParams(
984             e_min_kwh=e_min,
985             e_max_kwh=e_max,
986             e0_kwh=6000.0,
987             nominal_capacity_kwh=nom,
988         )
989         r = solve_stage(case, price, load, pv, params=p)
990         _require(r, case)
991         sens_rows.append(sensitivity_row(case, r))
992
993     for case, pkw in (("power_minus10", 4500.0), ("power_plus10", 5500.0)):
994         p = ModelParams(power_limit_kw=pkw)
995         r = solve_stage(case, price, load, pv, params=p)
996         _require(r, case)
997         sens_rows.append(sensitivity_row(case, r))
998
999     gmax_min_kw = min_feasible_peak_kw(price, load, pv, params)
1000     peak_nat = m1.peak_grid_kw
1001     if gmax_min_kw > peak_nat + 1e-6:
1002         raise SystemExit(f"min peak {gmax_min_kw} kW exceeds unconstrained M1 peak {peak_nat}")
1003     grid_kw_grid = np.linspace(gmax_min_kw, peak_nat, N_PCC_POINTS)
1004     for i, gkw in enumerate(grid_kw_grid):
1005         p = ModelParams(grid_limit_kw=float(gkw))
1006         r = solve_stage(f"pcc_{i}", price, load, pv, params=p)
1007         _require(r, f"pcc_{i}")
1008         sens_rows.append(sensitivity_row(f"pcc_gmax_{gkw:.4f}kw", r))
1009
1010     write_sensitivity_csv(OUTPUT_DIR / "q1_sensitivity.csv", sens_rows)
1011     write_validation_md(
1012         OUTPUT_DIR / "q1_validation.md",
1013         data,

```

```

1014     labels,
1015     official,
1016     attach,
1017     template,
1018     price_min=price_min,
1019     gmax_min_kw=gmax_min_kw,
1020 )
1021
1022     print(f"Q1 official = {OFFICIAL_MODEL}")
1023     print(f"    cost={official.purchase_cost:.6f} yuan")
1024     print(f"    G={official.grid_purchase:.6f} kWh")
1025     print(f"    peak={official.peak_grid_kw:.6f} kW")
1026     print(f"    S={official.curtailment:.6f} kWh")
1027     print(f"    Q={official.throughput:.6f} kWh")
1028     print(f"    max(c d)={official.max_cd:.3e}")
1029     print(f"    residual={official.max_balance_residual:.3e}")
1030     print(f"    price_min={price_min:.6g}")
1031     print(f"    Gmax_min(analysis)={gmax_min_kw:.6f} kW")
1032     print("wrote output/q1*.csv, output/q1*.md, output/result1.xlsx, fig/fig1*.png")
1033
1034
1035 if __name__ == "__main__":
1036     main()
1037
1038 # ===== q2/__init__.py =====
1039 """C-problem Q2 pilot implementation."""
1040
1041
1042 # ===== q2/config.py =====
1043 """Central configuration for the Q2 pilot only."""
1044
1045 from __future__ import annotations
1046
1047 import os
1048 from pathlib import Path
1049
1050 import numpy as np
1051
1052 ROOT = Path(__file__).resolve().parents[1]
1053 ATTACH_DIR = Path(os.environ.get("CUMCM_C_ATTACH_DIR", ROOT / "data" / "raw" / "附件"))
1054 ATTACH1 = ATTACH_DIR / "附件1.xlsx"
1055 ATTACH2 = ATTACH_DIR / "附件2.xlsx"
1056 OUTPUT_DIR = ROOT / "output" / "q2_pilot"
1057 FULL_OUTPUT_DIR = ROOT / "output" / "q2_full_linked"
1058 FULL_K8_RISK_OUTPUT_DIR = ROOT / "output" / "q2_full_k8_risk"
1059 POLICY_CONSISTENT_OUTPUT_DIR = ROOT / "output" / "q2_policy_consistent"
1060 FIG_DIR = ROOT / "fig" / "q2_pilot"
1061 FIG_POLICY_CONSISTENT_DIR = ROOT / "fig" / "q2_policy_consistent"
1062 FIG_Q2_FINAL_DIR = ROOT / "fig" / "q2_final"
1063 SIGNED_OFF_RESULT2 = ROOT / "output" / "result2.xlsx"
1064 SIGNED_OFF_RESULT2_SHA256 = (
1065     "70a9785c9e2bba66f7694878e4587ad15c57dc090ec6798d07ba6780b4780ef7"
1066 )
1067 RESULT2_TEMPLATE = ATTACH_DIR / "附件5" / "result2.xlsx"
1068 CANDIDATE_RESULT2 = POLICY_CONSISTENT_OUTPUT_DIR / "result2.xlsx"
1069 RESULT2_SIGNED_OFF_BACKUP = POLICY_CONSISTENT_OUTPUT_DIR / "result2_signed_off_backup.xlsx"
1070
1071 T = 144
1072 DELTA_H = 1.0 / 6.0
1073 ETA_C = float(np.sqrt(0.9))
1074 ETA_D = float(np.sqrt(0.9))
1075 E_MIN_KWH = 1200.0
1076 E_MAX_KWH = 10800.0
1077 E_INITIAL_KWH = 6000.0
1078 POWER_LIMIT_KW = 5000.0
1079 POWER_LIMIT_KWH = POWER_LIMIT_KW * DELTA_H
1080
1081 # Signed-off contract semantics; keep all production Q2 assumptions here.

```

```

1082 CONTRACT_TAKE_MODE = "x_le_q"
1083 NORMAL_COST_BASIS = "planned_q"
1084 EMERGENCY_PRICE_MULTIPLIER = 5.0
1085
1086 LOAD_HISTORY_SAME_WEEKDAY = 4
1087 PV_HISTORY_DAYS = 7
1088 RESIDUAL_POOL_DAYS = 28
1089 K_CANDIDATES = (2, 4, 6, 8, 10, 12)
1090 K_VALIDATION_DAYS = 14
1091 K_RECALIBRATION_DAYS = 14
1092 # Fixed from the R1--R4 timing audit: a validation-day score must be obtained
1093 # within this budget before its K is eligible under the one-standard-error rule.
1094 T_MAX_SECONDS: float | None = 0.20
1095 FIXED_SCENARIO_K = 8
1096 K_SENSITIVITY_CANDIDATES = (4, 8, 12)
1097 RISK_ALPHA_CANDIDATES = (0.60, 0.70, 0.80, 0.90)
1098 OFFICIAL_OUTPUT_START = "2025-02-01"
1099 RISK_CALIBRATION_DAYS = 14
1100 MPC_COST_TOL = 1e-7
1101 NEXT_DAY_VALUE_GAP_TOL_YUAN = 1.0
1102 NEXT_DAY_VALUE_MAX_SAMPLES = 25
1103 NUMERIC_TOL = 1e-6
1104 SIMULTANEOUS_CD_TOL = 1e-4
1105 SOLVER = "HIGHS"
1106 PILOT_DATES = ("2025-02-01", "2025-06-21")
1107
1108 # ===== q2/data.py =====
1109 from __future__ import annotations
1110
1111 import json
1112 from dataclasses import dataclass
1113 from datetime import datetime, time
1114 from pathlib import Path
1115
1116 import numpy as np
1117 import pandas as pd
1118
1119 from q2.config import ATTACH1, ATTACH2, DELTA_H, T
1120
1121
1122 def normalize_time(value: object) -> str:
1123     if isinstance(value, (datetime, time)):
1124         return value.strftime("%H:%M")
1125     return str(value).strip()
1126
1127
1128 @dataclass(frozen=True)
1129 class Q2Data:
1130     dates: pd.DatetimeIndex
1131     time_labels: tuple[str, ...]
1132     price: np.ndarray
1133     fallback_load: np.ndarray
1134     fallback_pv: np.ndarray
1135     load: np.ndarray
1136     pv: np.ndarray
1137
1138
1139 def _finite_nonnegative(values: np.ndarray, name: str) -> None:
1140     if not np.isfinite(values).all():
1141         raise ValueError(f"{name} contains missing or non-finite values")
1142     if np.any(values < 0):
1143         raise ValueError(f"{name} contains negative values")
1144
1145
1146 def load_q2_data(attach1: Path = ATTACH1, attach2: Path = ATTACH2) -> Q2Data:
1147     if not attach1.exists() or not attach2.exists():
1148         raise FileNotFoundError(f"Required attachments not found: {attach1}, {attach2}")
1149

```

```

1150 day = pd.read_excel(attach1, sheet_name="Sheet1")
1151 expected = ["时间", "电价", "小区负载", "光伏发电预测功率"]
1152 if list(day.columns) != expected or len(day) != T:
1153     raise ValueError("附件1 must have the four expected columns and 144 raw-order rows")
1154 price = pd.to_numeric(day["电价"], errors="coerce").to_numpy(float)
1155 fallback_load = pd.to_numeric(day["小区负载"], errors="coerce").to_numpy(float) * DELTA_H
1156 fallback_pv = pd.to_numeric(day["光伏发电预测功率"], errors="coerce").to_numpy(float) * DELTA_H
1157 _finite_nonnegative(price, "附件1 price")
1158 _finite_nonnegative(fallback_load, "附件1 load forecast")
1159 _finite_nonnegative(fallback_pv, "附件1 PV forecast")
1160 if np.any(price <= 0):
1161     raise ValueError("Q2 pilot requires strictly positive prices")
1162
1163 sheets = pd.read_excel(attach2, sheet_name=None, header=None)
1164 if list(sheets) != ["小区负载", "光伏发电实际功率"]:
1165     raise ValueError(f"Unexpected 附件2 sheets: {list(sheets)}")
1166 load_raw = sheets["小区负载"]
1167 pv_raw = sheets["光伏发电实际功率"]
1168 if load_raw.shape != (366, 145) or pv_raw.shape != (366, 145):
1169     raise ValueError(f"附件2 must be 366x145; got {load_raw.shape}, {pv_raw.shape}")
1170 if load_raw.iloc[0, 0] != "日期\\时间" or pv_raw.iloc[0, 0] != "日期\\时间":
1171     raise ValueError("附件2 first header must be 日期\\时间")
1172
1173 time_labels = tuple(normalize_time(x) for x in load_raw.iloc[0, 1:])
1174 pv_labels = tuple(normalize_time(x) for x in pv_raw.iloc[0, 1:])
1175 if time_labels != pv_labels or len(time_labels) != T:
1176     raise ValueError("附件2 load/PV time labels differ")
1177 if time_labels[0] != "00:10" or time_labels[-1] != "0:00+1":
1178     raise ValueError(f"Unexpected raw time order: {time_labels[0]} .. {time_labels[-1]}")
1179
1180 dates = pd.DatetimeIndex(pd.to_datetime(load_raw.iloc[1:, 0], errors="raise")).normalize()
1181 pv_dates = pd.DatetimeIndex(pd.to_datetime(pv_raw.iloc[1:, 0], errors="raise")).normalize()
1182 expected_dates = pd.date_range("2025-01-01", "2025-12-31", freq="D")
1183 if not dates.equals(expected_dates) or not pv_dates.equals(expected_dates):
1184     raise ValueError("附件2 dates must be continuous 2025-01-01 through 2025-12-31")
1185 load = load_raw.iloc[1:, 1:].apply(pd.to_numeric, errors="coerce").to_numpy(float) * DELTA_H
1186 pv = pv_raw.iloc[1:, 1:].apply(pd.to_numeric, errors="coerce").to_numpy(float) * DELTA_H
1187 _finite_nonnegative(load, "附件2 load")
1188 _finite_nonnegative(pv, "附件2 PV")
1189 return Q2Data(dates, time_labels, price, fallback_load, fallback_pv, load, pv)
1190
1191
1192 def write_input_audit(data: Q2Data, path: Path) -> None:
1193     daily = []
1194     for i, date in enumerate(data.dates):
1195         load = float(data.load[i].sum())
1196         pv = float(data.pv[i].sum())
1197         daily.append(
1198             {
1199                 "date": date.strftime("%Y-%m-%d"),
1200                 "load_kwh": load,
1201                 "pv_kwh": pv,
1202                 "net_load_kwh": load - pv,
1203             }
1204         )
1205     audit = {
1206         "shape": {"days": len(data.dates), "periods_per_day": T},
1207         "date_start": data.dates[0].strftime("%Y-%m-%d"),
1208         "date_end": data.dates[-1].strftime("%Y-%m-%d"),
1209         "dates_continuous": True,
1210         "missing_count": int(np.isnan(data.load).sum() + np.isnan(data.pv).sum()),
1211         "negative_load_count": int(np.sum(data.load < 0)),
1212         "negative_pv_count": int(np.sum(data.pv < 0)),
1213         "price_min_yuan_per_kwh": float(data.price.min()),
1214         "unit_conversion": "attachment powers multiplied by 1/6 h to obtain kWh",
1215         "time_mapping": (
1216             "attachment raw column t maps to internal index t=0..143 and template column t; "
1217             "no sorting, rotation, or invented 0:00-0:10 period"
1218         )
1219     }

```

```

1218         ),
1219         "first_time_label": data.time_labels[0],
1220         "last_time_label": data.time_labels[-1],
1221         "daily_energy": daily,
1222     }
1223     path.parent.mkdir(parents=True, exist_ok=True)
1224     path.write_text(json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
1225
1226
1227 # ===== q2/export_result2.py =====
1228 """Export a candidate result2.xlsx for the policy-consistent Q2 path.
1229
1230 Writes only under output/q2_policy_consistent/. Never overwrites the signed-off
1231 output/result2.xlsx.
1232 """
1233
1234 from __future__ import annotations
1235
1236 import hashlib
1237 import json
1238 import shutil
1239 from copy import copy
1240 from datetime import datetime
1241 from pathlib import Path
1242
1243 import numpy as np
1244 import pandas as pd
1245 from openpyxl import load_workbook
1246 from openpyxl.worksheet.worksheet import Worksheet
1247
1248 from q2.config import (
1249     CANDIDATE_RESULT2,
1250     E_MAX_KWH,
1251     E_MIN_KWH,
1252     EMERGENCY_PRICE_MULTIPLIER,
1253     NUMERIC_TOL,
1254     OFFICIAL_OUTPUT_START,
1255     POLICY_CONSISTENT_OUTPUT_DIR,
1256     POWER_LIMIT_KWH,
1257     RESULT2_SIGNED_OFF_BACKUP,
1258     RESULT2_TEMPLATE,
1259     ROOT,
1260     SIGNED_OFF_RESULT2,
1261     SIGNED_OFF_RESULT2_SHA256,
1262     SIMULTANEOUS_CD_TOL,
1263     T,
1264 )
1265 from q2.data import Q2Data
1266 from q2.pilot import planned_q_hash
1267
1268 PURCHASE_SHEET = "计划购电量"
1269 CHARGE_SHEET = "充放电量"
1270 EMERGENCY_SHEET = "紧急购电量"
1271 OUTPUT_END = "2025-12-31"
1272 N_OUTPUT_DAYS = 334
1273 BLOCKS_PER_DAY = 6
1274 LAST_PERIOD_COL = 1 + T
1275 Q_SUM_COL = 2 + T
1276 COST_COL = 3 + T
1277 FOUR_HOUR_BLOCKS = (
1278     ("0:00-4:00", 0, 24),
1279     ("4:00-8:00", 24, 48),
1280     ("8:00-12:00", 48, 72),
1281     ("12:00-16:00", 72, 96),
1282     ("16:00-20:00", 96, 120),
1283     ("20:00-24:00", 120, 144),
1284 )
1285

```

```

1286
1287 def file_sha256(path: Path) -> str:
1288     digest = hashlib.sha256()
1289     with path.open("rb") as stream:
1290         for block in iter(lambda: stream.read(1024 * 1024), b''):
1291             digest.update(block)
1292     return digest.hexdigest()
1293
1294
1295 def official_output_dates() -> pd.DatetimeIndex:
1296     return pd.date_range(OFFICIAL_OUTPUT_START, OUTPUT_END, freq="D")
1297
1298
1299 def minutes_to_clock(minutes: int) -> str:
1300     if minutes == 24 * 60:
1301         return "24:00"
1302     hours, mins = divmod(int(minutes), 60)
1303     return f"{hours:02d}:{mins:02d}"
1304
1305
1306 def format_emergency_interval(t0: int, t1: int) -> str:
1307     if not (0 <= t0 <= t1 < T):
1308         raise ValueError(f"invalid emergency interval [{t0}, {t1}]")
1309     return f"{minutes_to_clock(t0 * 10)}-{minutes_to_clock((t1 + 1) * 10)}"
1310
1311
1312 def emergency_segments(
1313     emergency: np.ndarray,
1314     tol: float = 0.0,
1315 ) -> list[tuple[int, int, float]]:
1316     values = np.asarray(emergency, dtype=float)
1317     if values.shape != (T,):
1318         raise ValueError(f"emergency vector must have length {T}")
1319     segments: list[tuple[int, int, float]] = []
1320     t = 0
1321     while t < T:
1322         if values[t] > tol:
1323             t0 = t
1324             total = 0.0
1325             while t < T and values[t] > tol:
1326                 total += float(values[t])
1327                 t += 1
1328             segments.append((t0, t - 1, total))
1329         else:
1330             t += 1
1331     return segments
1332
1333
1334 def block_energy(values: np.ndarray) -> np.ndarray:
1335     values = np.asarray(values, dtype=float)
1336     return np.array([float(np.sum(values[start:end])) for _name, start, end in FOUR_HOUR_BLOCKS])
1337
1338
1339 def assert_isolated_candidate_path(dest: Path) -> None:
1340     dest_resolved = dest.resolve()
1341     signed = SIGNED_OFF_RESULT2.resolve()
1342     root = POLICY_CONSISTENT_OUTPUT_DIR.resolve()
1343     if dest_resolved == signed:
1344         raise RuntimeError("refusing to overwrite signed-off output/result2.xlsx")
1345     if dest_resolved.parent != root:
1346         raise RuntimeError("candidate result2.xlsx must live under output/q2_policy_consistent/")
1347
1348
1349 def preserve_signed_off_backup() -> str:
1350     if not SIGNED_OFF_RESULT2.exists():
1351         raise FileNotFoundError(f"missing signed-off {SIGNED_OFF_RESULT2}")
1352     signed_hash = file_sha256(SIGNED_OFF_RESULT2)
1353     if signed_hash != SIGNED_OFF_RESULT2_SHA256:

```



```

1354         raise RuntimeError(
1355             "signed-off output/result2.xlsx hash mismatch; refusing to copy a mutated file"
1356         )
1357     RESULT2_SIGNED_OFF_BACKUP.parent.mkdir(parents=True, exist_ok=True)
1358     if RESULT2_SIGNED_OFF_BACKUP.exists():
1359         backup_hash = file_sha256(RESULT2_SIGNED_OFF_BACKUP)
1360         if backup_hash != signed_hash:
1361             raise RuntimeError("existing result2 backup does not match the signed-off file")
1362     else:
1363         shutil.copy2(SIGNED_OFF_RESULT2, RESULT2_SIGNED_OFF_BACKUP)
1364         backup_hash = file_sha256(RESULT2_SIGNED_OFF_BACKUP)
1365         if backup_hash != signed_hash:
1366             raise RuntimeError("failed to copy a bit-identical signed-off result2 backup")
1367     return signed_hash
1368
1369
1370 def _copy_style(source, dest) -> None:
1371     if source.has_style:
1372         dest.font = copy(source.font)
1373         dest.border = copy(source.border)
1374         dest.fill = copy(source.fill)
1375         dest.number_format = source.number_format
1376         dest.alignment = copy(source.alignment)
1377         dest.protection = copy(source.protection)
1378
1379
1380 def _set_number(cell, value: float | None) -> None:
1381     cell.value = None if value is None else float(value)
1382     cell.number_format = "General"
1383
1384
1385 def _sheet_date(value: object) -> pd.Timestamp:
1386     return pd.Timestamp(value).normalize()
1387
1388
1389 def _load_day_dispatch(dispatch_dir: Path, date_text: str) -> pd.DataFrame:
1390     path = dispatch_dir / f"dispatch_{date_text}.csv"
1391     if not path.exists():
1392         raise FileNotFoundError(f"missing dispatch {path}")
1393     frame = pd.read_csv(path)
1394     if len(frame) != T:
1395         raise AssertionError(f"{date_text} dispatch has {len(frame)} rows, expected {T}")
1396     if str(frame["time"].iloc[0]) != "00:10" or str(frame["time"].iloc[-1]) != "0:00+1":
1397         raise AssertionError(f"{date_text} dispatch is not in raw attachment time order")
1398     return frame
1399
1400
1401 def write_candidate_result2(
1402     dest: Path,
1403     dispatch_dir: Path,
1404     daily: pd.DataFrame,
1405     data: Q2Data,
1406     template: Path = RESULT2_TEMPLATE,
1407 ) -> None:
1408     assert_isolated_candidate_path(dest)
1409     if not template.exists():
1410         raise FileNotFoundError(f"missing result2 template {template}")
1411     dates = official_output_dates()
1412     ledger = daily.copy()
1413     ledger["date"] = pd.to_datetime(ledger["date"]).dt.strftime("%Y-%m-%d")
1414     ledger = ledger.loc[ledger["date"].isin(dates.strftime("%Y-%m-%d"))].sort_values("date")
1415     if list(ledger["date"]) != list(dates.strftime("%Y-%m-%d")):
1416         raise AssertionError("Feb-Dec ledger dates do not match the 334 official output days")
1417
1418     dest.parent.mkdir(parents=True, exist_ok=True)
1419     shutil.copyfile(template, dest)
1420     workbook = load_workbook(dest)
1421     purchase = workbook[PURCHASE_SHEET]

```

```

1422 charge = workbook[CHARGE_SHEET]
1423 emergency = workbook[EMERGENCY_SHEET]
1424 if purchase.max_row != 1 + N_OUTPUT_DAYS:
1425     raise AssertionError("purchase template does not contain 334 dated rows")
1426 if purchase.cell(1, Q_SUM_COL).value != "全天购电量" or purchase.cell(1, COST_COL).value != "全天购电费":
1427     raise AssertionError("purchase template totals headers mismatch")
1428 if str(purchase.cell(1, 2).value) != "0:10-0:20":
1429     raise AssertionError("purchase template is not in raw 144-period order")
1430 if str(purchase.cell(1, LAST_PERIOD_COL).value) != "0:00-0:10+1":
1431     raise AssertionError("purchase template last period header mismatch")
1432
1433 charge_style_rows = [2 + offset for offset in range(BLOCKS_PER_DAY)]
1434 emergency_styles = [emergency.cell(2, column) for column in range(1, 4)]
1435 emergency_row = 2
1436 for day_index, date in enumerate(dates):
1437     date_text = date.strftime("%Y-%m-%d")
1438     row = ledger.loc[ledger["date"] == date_text].iloc[0]
1439     frame = _load_day_dispatch(dispatch_dir, date_text)
1440     q = frame["planned_q_kwh"].to_numpy(dtype=float)
1441     planned_cost = float(data.price @ q)
1442     purchase_row = 2 + day_index
1443     sheet_date = _sheet_date(purchase.cell(purchase_row, 1).value)
1444     if sheet_date != date:
1445         raise AssertionError(f"purchase date mismatch at row {purchase_row}: {sheet_date} vs {date}")
1446     for period in range(T):
1447         _set_number(purchase.cell(purchase_row, 2 + period), float(q[period]))
1448         _set_number(purchase.cell(purchase_row, Q_SUM_COL), float(np.sum(q)))
1449         _set_number(purchase.cell(purchase_row, COST_COL), planned_cost)
1450
1451     charge_base = 2 + day_index * BLOCKS_PER_DAY
1452     charge_kwh = block_energy(frame["charge_kwh"].to_numpy(dtype=float))
1453     discharge_kwh = block_energy(frame["discharge_kwh"].to_numpy(dtype=float))
1454     for offset, (name, _start, _end) in enumerate(FOUR_HOUR_BLOCKS):
1455         excel_row = charge_base + offset
1456         style_source_row = charge_style_rows[offset]
1457         for column in range(1, 7):
1458             _copy_style(charge.cell(style_source_row, column), charge.cell(excel_row, column))
1459             charge.cell(excel_row, 1).value = purchase.cell(purchase_row, 1).value if offset == 0 else None
1460             charge.cell(excel_row, 2).value = name
1461             _set_number(charge.cell(excel_row, 3), float(charge_kwh[offset]))
1462             _set_number(charge.cell(excel_row, 4), float(discharge_kwh[offset]))
1463             if offset == 0:
1464                 charge.cell(excel_row, 5).value = "00:00"
1465                 _set_number(charge.cell(excel_row, 6), float(row["soc_start_kwh"]))
1466             elif offset == 1:
1467                 charge.cell(excel_row, 5).value = "24:00"
1468                 _set_number(charge.cell(excel_row, 6), float(row["soc_end_kwh"]))
1469             else:
1470                 charge.cell(excel_row, 5).value = None
1471                 _set_number(charge.cell(excel_row, 6), None)
1472
1473     segments = emergency_segments(frame["emergency_kwh"].to_numpy(dtype=float))
1474     if not segments:
1475         for column in range(1, 4):
1476             _copy_style(emergency_styles[column - 1], emergency.cell(emergency_row, column))
1477             emergency.cell(emergency_row, 1).value = purchase.cell(purchase_row, 1).value
1478             emergency.cell(emergency_row, 2).value = None
1479             _set_number(emergency.cell(emergency_row, 3), None)
1480             emergency_row += 1
1481     else:
1482         for seg_index, (t0, t1, amount) in enumerate(segments):
1483             for column in range(1, 4):
1484                 _copy_style(emergency_styles[column - 1], emergency.cell(emergency_row, column))
1485                 emergency.cell(emergency_row, 1).value = (
1486                     purchase.cell(purchase_row, 1).value if seg_index == 0 else None
1487                 )
1488                 emergency.cell(emergency_row, 2).value = format_emergency_interval(t0, t1)
1489                 _set_number(emergency.cell(emergency_row, 3), float(amount))

```

```

1490         emergency_row += 1
1491
1492     leftover_start = emergency_row
1493     leftover_end = max(emergency.max_row, leftover_start - 1)
1494     for row_index in range(leftover_start, leftover_end + 1):
1495         for column in range(1, 4):
1496             emergency.cell(row_index, column).value = None
1497
1498     workbook.save(dest)
1499
1500
1501 def _cell_float(sheet: Worksheet, row: int, column: int) -> float:
1502     value = sheet.cell(row, column).value
1503     if value in (None, ""):
1504         return 0.0
1505     if isinstance(value, datetime):
1506         raise TypeError(f"{sheet.title}!{sheet.cell(row, column).coordinate} stored a date instead of a number")
1507     return float(value)
1508
1509
1510 def audit_candidate_result2(
1511     dest: Path,
1512     dispatch_dir: Path,
1513     daily: pd.DataFrame,
1514     data: Q2Data,
1515     inherited_soc_kwh: float,
1516 ) -> dict:
1517     assert_isolated_candidate_path(dest)
1518     workbook = load_workbook(dest, data_only=False)
1519     purchase = workbook[PURCHASE_SHEET]
1520     charge = workbook[CHARGE_SHEET]
1521     emergency = workbook[EMERGENCY_SHEET]
1522     dates = official_output_dates()
1523     ledger = daily.copy()
1524     ledger["date"] = pd.to_datetime(ledger["date"]).dt.strftime("%Y-%m-%d")
1525     ledger = ledger.loc[ledger["date"].isin(dates.strftime("%Y-%m-%d"))].sort_values("date")
1526     if len(ledger) != N_OUTPUT_DAYS:
1527         raise AssertionError("ledger does not cover 334 official output days")
1528
1529     rows: list[dict] = []
1530     emergency_row = 2
1531     previous_soc_end: float | None = None
1532     max_q_abs = 0.0
1533     max_cost_abs = 0.0
1534     max_emergency_abs = 0.0
1535     max_charge_abs = 0.0
1536     max_soc_abs = 0.0
1537     max_balance = 0.0
1538     max_soc_gap = 0.0
1539     dated_emergency_rows = 0
1540
1541     for day_index, date in enumerate(dates):
1542         date_text = date.strftime("%Y-%m-%d")
1543         ledger_row = ledger.loc[ledger["date"] == date_text].iloc[0]
1544         frame = _load_day_dispatch(dispatch_dir, date_text)
1545         q = frame["planned_q_kwh"].to_numpy(dtype=float)
1546         x = frame["actual_x_kwh"].to_numpy(dtype=float)
1547         emergency_kwh = frame["emergency_kwh"].to_numpy(dtype=float)
1548         charge_kwh = frame["charge_kwh"].to_numpy(dtype=float)
1549         discharge_kwh = frame["discharge_kwh"].to_numpy(dtype=float)
1550         purchase_row = 2 + day_index
1551         sheet_date = _sheet_date(purchase.cell(purchase_row, 1).value)
1552         if sheet_date != date:
1553             raise AssertionError(f"xlsx purchase date mismatch: {sheet_date} vs {date}")
1554         xlsx_q = np.array([_cell_float(purchase, purchase_row, 2 + period) for period in range(T)])
1555         q_abs = float(np.max(np.abs(xlsx_q - q)))
1556         xlsx_q_sum = _cell_float(purchase, purchase_row, Q_SUM_COL)
1557         xlsx_cost = _cell_float(purchase, purchase_row, COST_COL)

```

```

1558 planned_cost = float(data.price @ q)
1559 cost_abs = abs(xlsx_cost - planned_cost)
1560 q_sum_abs = abs(xlsx_q_sum - float(np.sum(q)))
1561
1562 charge_base = 2 + day_index * BLOCKS_PER_DAY
1563 expected_charge = block_energy(charge_kwh)
1564 expected_discharge = block_energy(discharge_kwh)
1565 charge_abs = 0.0
1566 for offset, (name, _start, _end) in enumerate(FOUR_HOUR_BLOCKS):
1567     excel_row = charge_base + offset
1568     date_cell = charge.cell(excel_row, 1).value
1569     if offset == 0:
1570         if _sheet_date(date_cell) != date:
1571             raise AssertionError(f"charge date mismatch on {date_text}")
1572         elif date_cell not in (None, ""):
1573             raise AssertionError(f"charge date repeated on {date_text} block {offset}")
1574         if charge.cell(excel_row, 2).value != name:
1575             raise AssertionError(f"charge block label mismatch on {date_text}")
1576         charge_abs = max(
1577             charge_abs,
1578             abs(_cell_float(charge, excel_row, 3) - expected_charge[offset]),
1579             abs(_cell_float(charge, excel_row, 4) - expected_discharge[offset]),
1580         )
1581     start_clock = charge.cell(charge_base, 5).value
1582     end_clock = charge.cell(charge_base + 1, 5).value
1583     if start_clock not in ("00:00", datetime.strptime("00:00", "%H:%M").time()):
1584         raise AssertionError(f"{date_text} 00:00 SOC clock mismatch")
1585     if end_clock != "24:00":
1586         raise AssertionError(f"{date_text} 24:00 SOC clock mismatch")
1587     xlsx_soc_start = _cell_float(charge, charge_base, 6)
1588     xlsx_soc_end = _cell_float(charge, charge_base + 1, 6)
1589     soc_start_abs = abs(xlsx_soc_start - float(ledger_row["soc_start_kwh"]))
1590     soc_end_abs = abs(xlsx_soc_end - float(ledger_row["soc_end_kwh"]))
1591     last_dispatch_soc = float(frame["soc_kwh"].iloc[-1])
1592     soc_dispatch_abs = abs(xlsx_soc_end - last_dispatch_soc)
1593     if previous_soc_end is not None:
1594         max_soc_gap = max(max_soc_gap, abs(xlsx_soc_start - previous_soc_end))
1595     previous_soc_end = xlsx_soc_end
1596
1597 date_cell = emergency.cell(emergency_row, 1).value
1598 if date_cell in (None, ""):
1599     raise AssertionError(f"missing dated emergency row for {date_text}")
1600 if _sheet_date(date_cell) != date:
1601     raise AssertionError(f"emergency date mismatch: {date_cell} vs {date_text}")
1602 dated_emergency_rows += 1
1603 xlsx_emergency = 0.0
1604 first = True
1605 while emergency_row <= emergency.max_row:
1606     current_date = emergency.cell(emergency_row, 1).value
1607     interval = emergency.cell(emergency_row, 2).value
1608     amount = emergency.cell(emergency_row, 3).value
1609     if not first and current_date not in (None, ""):
1610         break
1611     if interval in (None, "") and amount in (None, ""):
1612         if first:
1613             emergency_row += 1
1614         break
1615     xlsx_emergency += float(amount)
1616     first = False
1617     emergency_row += 1
1618 if float(emergency_kwh.min()) < -NUMERIC_TOL:
1619     raise AssertionError(f"{date_text} has negative emergency energy")
1620 e_abs = abs(xlsx_emergency - float(np.sum(np.clip(emergency_kwh, 0.0, None))))
1621 e_ledger_abs = abs(xlsx_emergency - float(ledger_row["emergency_kwh"]))
1622 e_recon_abs = e_ledger_abs
1623
1624 residual = (
1625     frame["actual_x_kwh"]

```

```

1626         + frame["emergency_kwh"]
1627         + frame["pv_kwh"]
1628         - frame["curtailment_kwh"]
1629         + frame["discharge_kwh"]
1630         - frame["load_kwh"]
1631         - frame["charge_kwh"]
1632     )
1633     day_index_data = int(data.dates.get_loc(date))
1634     balance = float(residual.abs().max())
1635     x_minus_q = float(np.max(x - q))
1636     cd = float((charge_kwh * discharge_kwh).max())
1637     if (
1638         planned_q_hash(q) != ledger_row["planned_q_sha256"]
1639         or abs(planned_cost - float(ledger_row["planned_cost_yuan"])) > 1e-6
1640         or abs(float(np.sum(q)) - float(ledger_row["planned_q_kwh"])) > 1e-6
1641     ):
1642         raise AssertionError(f"{date_text} dispatch does not match the annual ledger")
1643     if not np.allclose(frame["load_kwh"].to_numpy(), data.load[day_index_data], atol=1e-12):
1644         raise AssertionError(f"{date_text} load mismatch versus attachment")
1645     emergency_cost = float(EMERGENCY_PRICE_MULTIPLIER * data.price @ emergency_kwh)
1646     if abs(emergency_cost - float(ledger_row["emergency_cost_yuan"])) > 1e-4:
1647         raise AssertionError(f"{date_text} emergency cost mismatch versus ledger")
1648
1649     rows.append(
1650         {
1651             "date": date_text,
1652             "q_max_abs_kwh": q_abs,
1653             "q_sum_abs_kwh": q_sum_abs,
1654             "planned_cost_abs_yuan": cost_abs,
1655             "charge_block_abs_kwh": charge_abs,
1656             "soc_start_abs_kwh": soc_start_abs,
1657             "soc_end_abs_kwh": soc_end_abs,
1658             "soc_vs_dispatch_abs_kwh": soc_dispatch_abs,
1659             "emergency_abs_kwh": e_abs,
1660             "emergency_vs_ledger_abs_kwh": e_ledger_abs,
1661             "emergency_recon_abs_kwh": e_recon_abs,
1662             "max_balance_residual_kwh": balance,
1663             "max_x_minus_q_kwh": x_minus_q,
1664             "max_simultaneous_cd_kwh2": cd,
1665             "charge_power_ok": float(charge_kwh.max()) <= POWER_LIMIT_KWH + NUMERIC_TOL,
1666             "discharge_power_ok": float(discharge_kwh.max()) <= POWER_LIMIT_KWH + NUMERIC_TOL,
1667             "soc_bounds_ok": (
1668                 float(frame["soc_kwh"].min()) >= E_MIN_KWH - NUMERIC_TOL
1669                 and float(frame["soc_kwh"].max()) <= E_MAX_KWH + NUMERIC_TOL
1670             ),
1671         }
1672     )
1673     max_q_abs = max(max_q_abs, q_abs, q_sum_abs)
1674     max_cost_abs = max(max_cost_abs, cost_abs)
1675     max_emergency_abs = max(max_emergency_abs, e_recon_abs)
1676     max_charge_abs = max(max_charge_abs, charge_abs)
1677     max_soc_abs = max(max_soc_abs, soc_start_abs, soc_end_abs, soc_dispatch_abs)
1678     max_balance = max(max_balance, balance)
1679
1680     if dated_emergency_rows != N_OUTPUT_DAYS:
1681         raise AssertionError("emergency sheet does not have one dated row per official day")
1682     feb1_soc = float(ledger.iloc[0]["soc_start_kwh"])
1683     if abs(feb1_soc - inherited_soc_kwh) > NUMERIC_TOL:
1684         raise AssertionError("Feb 1 xlsx/ledger SOC does not inherit January warmup")
1685
1686     xlsx_planned_q = sum(_cell_float(purchase, row, Q_SUM_COL) for row in range(2, 2 + N_OUTPUT_DAYS))
1687     xlsx_planned_cost = sum(_cell_float(purchase, row, COST_COL) for row in range(2, 2 + N_OUTPUT_DAYS))
1688     xlsx_emergency = 0.0
1689     for row in range(2, emergency.max_row + 1):
1690         amount = emergency.cell(row, 3).value
1691         if amount not in (None, ""):
1692             xlsx_emergency += float(amount)
1693

```

```

1694 checks = {
1695     "n_days": N_OUTPUT_DAYS,
1696     "n_periods": T,
1697     "units": {"energy": "kWh", "cost": "yuan"},
1698     "purchase_sheet_rows": 1 + N_OUTPUT_DAYS,
1699     "charge_sheet_rows": 1 + N_OUTPUT_DAYS * BLOCKS_PER_DAY,
1700     "dated_emergency_rows": dated_emergency_rows,
1701     "febi_inherited_soc_kwh": inherited_soc_kwh,
1702     "febi_xlsx_soc_kwh": float(ledger.iloc[0]["soc_start_kwh"]),
1703     "max_q_abs_kwh": max_q_abs,
1704     "max_planned_cost_abs_yuan": max_cost_abs,
1705     "max_charge_block_abs_kwh": max_charge_abs,
1706     "max_soc_abs_kwh": max_soc_abs,
1707     "max_soc_continuity_gap_kwh": max_soc_gap,
1708     "max_emergency_recon_abs_kwh": max_emergency_abs,
1709     "max_balance_residual_kwh": max_balance,
1710     "xlsx_planned_q_kwh": xlsx_planned_q,
1711     "ledger_planned_q_kwh": float(ledger["planned_q_kwh"].sum()),
1712     "xlsx_planned_cost_yuan": xlsx_planned_cost,
1713     "ledger_planned_cost_yuan": float(ledger["planned_cost_yuan"].sum()),
1714     "xlsx_emergency_kwh": xlsx_emergency,
1715     "ledger_emergency_kwh": float(ledger["emergency_kwh"].sum()),
1716     "ledger_total_cost_yuan": float(ledger["total_cost_yuan"].sum()),
1717     "pass": (
1718         max_q_abs < 1e-8
1719         and max_cost_abs < 1e-6
1720         and max_charge_abs < 1e-8
1721         and max_soc_abs < 1e-8
1722         and max_soc_gap < NUMERIC_TOL
1723         and max_emergency_abs < 1e-8
1724         and max_balance < NUMERIC_TOL
1725         and abs(xlsx_planned_q - float(ledger["planned_q_kwh"].sum())) < 1e-6
1726         and abs(xlsx_planned_cost - float(ledger["planned_cost_yuan"].sum())) < 1e-4
1727         and abs(xlsx_emergency - float(ledger["emergency_kwh"].sum())) < 1e-4
1728         and all(row["charge_power_ok"] and row["discharge_power_ok"] and row["soc_bounds_ok"] for row in rows)
1729         and all(row["max_x_minus_q_kwh"] < NUMERIC_TOL for row in rows)
1730         and all(row["max_simultaneous_cd_kwh2"] <= SIMULTANEOUS_CD_TOL for row in rows)
1731         and dest.resolve() != SIGNED_OFF_RESULT2.resolve()
1732         and file_sha256(SIGNED_OFF_RESULT2) == SIGNED_OFF_RESULT2_SHA256
1733     ),
1734 }
1735 return {"summary": checks, "daily": rows}
1736
1737
1738 def write_result2_audit(audit: dict, output: Path) -> None:
1739     output.mkdir(parents=True, exist_ok=True)
1740     pd.DataFrame(audit["daily"]).to_csv(output / "result2_cell_audit.csv", index=False)
1741     payload = {
1742         "candidate_result2": str(CANDIDATE_RESULT2.relative_to(ROOT)),
1743         "signed_off_result2_unchanged": True,
1744         "signed_off_sha256": SIGNED_OFF_RESULT2_SHA256,
1745         "backup": str(RESULT2_SIGNED_OFF_BACKUP.relative_to(ROOT)),
1746         **audit["summary"],
1747     }
1748     (output / "result2_cell_audit.json").write_text(
1749         json.dumps(payload, ensure_ascii=False, indent=2) + "\n",
1750         encoding="utf-8",
1751     )
1752
1753 # ===== q2/forecast.py =====
1754 from __future__ import annotations
1755
1756 from dataclasses import dataclass
1757 from pathlib import Path
1758
1759 import numpy as np
1760 import pandas as pd
1761

```

```

1762 from q2.config import LOAD_HISTORY_SAME_WEEKDAY, PV_HISTORY_DAYS, T
1763 from q2.data import Q2Data
1764
1765
1766 @dataclass(frozen=True)
1767 class ForecastArchive:
1768     load_hat: np.ndarray
1769     pv_hat: np.ndarray
1770     load_residual: np.ndarray
1771     pv_residual: np.ndarray
1772     load_sources: tuple[tuple[int, ...], ...]
1773     pv_sources: tuple[tuple[int, ...], ...]
1774
1775
1776 def forecast_as_of(
1777     data: Q2Data, target_index: int, history_end_exclusive: int
1778 ) -> tuple[np.ndarray, np.ndarray, tuple[int, ...], tuple[int, ...]]:
1779     """Forecast target using actual observations with index < history_end_exclusive."""
1780     if not 0 <= history_end_exclusive <= target_index:
1781         raise ValueError("forecast cutoff must not exceed target date")
1782     target = data.dates[target_index]
1783     same_weekday = [
1784         j
1785         for j in range(history_end_exclusive)
1786         if data.dates[j].weekday() == target.weekday()
1787     ]
1788     load_sources = tuple(same_weekday[-LOAD_HISTORY_SAME_WEEKDAY:])
1789     pv_sources = tuple(
1790         range(max(0, history_end_exclusive - PV_HISTORY_DAYS), history_end_exclusive)
1791     )
1792     load_hat = (
1793         data.load[list(load_sources)].mean(axis=0)
1794         if load_sources
1795         else data.fallback_load.copy()
1796     )
1797     pv_hat = (
1798         data.pv[list(pv_sources)].mean(axis=0)
1799         if pv_sources
1800         else data.fallback_pv.copy()
1801     )
1802     return load_hat, pv_hat, load_sources, pv_sources
1803
1804
1805 def build_forecast_archive(data: Q2Data) -> ForecastArchive:
1806     n = len(data.dates)
1807     load_hat = np.empty_like(data.load)
1808     pv_hat = np.empty_like(data.pv)
1809     load_sources: list[tuple[int, ...]] = []
1810     pv_sources: list[tuple[int, ...]] = []
1811     for i in range(n):
1812         load_hat[i], pv_hat[i], ls, ps = forecast_as_of(data, i, i)
1813         load_sources.append(ls)
1814         pv_sources.append(ps)
1815     return ForecastArchive(
1816         load_hat=load_hat,
1817         pv_hat=pv_hat,
1818         load_residual=data.load - load_hat,
1819         pv_residual=data.pv - pv_hat,
1820         load_sources=tuple(load_sources),
1821         pv_sources=tuple(pv_sources),
1822     )
1823
1824
1825 def assert_no_forecast_leakage(archive: ForecastArchive) -> None:
1826     for i, (load_sources, pv_sources) in enumerate(
1827         zip(archive.load_sources, archive.pv_sources)
1828     ):
1829         if any(j >= i for j in load_sources + pv_sources):

```

```

1830         raise AssertionError(f"forecast leakage at day index {i}")
1831
1832
1833 def write_forecast_archive(
1834     data: Q2Data, archive: ForecastArchive, csv_path: Path, audit_path: Path
1835 ) -> None:
1836     frames = []
1837     for i, date in enumerate(data.dates):
1838         load_source_dates = ";".join(
1839             data.dates[j].strftime("%Y-%m-%d") for j in archive.load_sources[i]
1840         )
1841         pv_source_dates = ";".join(
1842             data.dates[j].strftime("%Y-%m-%d") for j in archive.pv_sources[i]
1843         )
1844         frames.append(
1845             pd.DataFrame(
1846                 {
1847                     "date": date.strftime("%Y-%m-%d"),
1848                     "information_cutoff": (
1849                         data.dates[i - 1].strftime("%Y-%m-%d") if i else "attachment1_fallback"
1850                     ),
1851                     "load_source_dates": load_source_dates or "attachment1_fallback",
1852                     "pv_source_dates": pv_source_dates or "attachment1_fallback",
1853                     "period": np.arange(T),
1854                     "time": data.time_labels,
1855                     "load_hat_kwh": archive.load_hat[i],
1856                     "pv_hat_kwh": archive.pv_hat[i],
1857                     "actual_load_kwh": data.load[i],
1858                     "actual_pv_kwh": data.pv[i],
1859                     "load_residual_kwh": archive.load_residual[i],
1860                     "pv_residual_kwh": archive.pv_residual[i],
1861                 }
1862             )
1863         )
1864     csv_path.parent.mkdir(parents=True, exist_ok=True)
1865     pd.concat(frames, ignore_index=True).to_csv(csv_path, index=False)
1866
1867     lines = [
1868         "## Q2 预测无泄漏审计",
1869         "",
1870         "- 负荷: 日期严格早于目标日的最近至多 4 个同星期历史日。",
1871         "- 光伏: 日期严格早于目标日的最近至多 7 个历史日。",
1872         "- 若没有可用历史日, 使用附件 1 固定预测曲线; 不读取目标日实际值。",
1873         "",
1874     ]
1875     for target in ("2025-02-01", "2025-06-21"):
1876         i = int(data.dates.get_loc(pd.Timestamp(target)))
1877         load_dates = [data.dates[j].strftime("%Y-%m-%d") for j in archive.load_sources[i]]
1878         pv_dates = [data.dates[j].strftime("%Y-%m-%d") for j in archive.pv_sources[i]]
1879         valid = all(j < i for j in archive.load_sources[i] + archive.pv_sources[i])
1880         lines += [
1881             f"### {target}",
1882             "",
1883             f"- 负荷样本日期: {' '.join(load_dates) or '无 (附件1回退)'}",
1884             f"- 光伏样本日期: {' '.join(pv_dates) or '无 (附件1回退)'}",
1885             f"- 所有样本日期严格早于目标日期: {valid}",
1886             "",
1887         ]
1888     audit_path.write_text("\n".join(lines), encoding="utf-8")
1889
1890
1891 # ===== q2/optimization.py =====
1892 from __future__ import annotations
1893
1894 from dataclasses import dataclass
1895 from time import perf_counter
1896
1897 import cvxpy as cp

```



```

1898 import numpy as np
1899
1900 from q2.config import (
1901     CONTRACT_TAKE_MODE,
1902     EMERGENCY_PRICE_MULTIPLIER,
1903     E_MAX_KWH,
1904     E_MIN_KWH,
1905     ETA_C,
1906     ETA_D,
1907     MPC_COST_TOL,
1908     NORMAL_COST_BASIS,
1909     POWER_LIMIT_KWH,
1910     SOLVER,
1911 )
1912
1913
1914 @dataclass(frozen=True)
1915 class DispatchResult:
1916     x: np.ndarray
1917     emergency: np.ndarray
1918     charge: np.ndarray
1919     discharge: np.ndarray
1920     curtailment: np.ndarray
1921     soc: np.ndarray
1922     emergency_cost: float
1923     terminal_value: float
1924     solve_seconds: float
1925     status: str
1926
1927     @property
1928     def max_cd(self) -> float:
1929         return float(np.max(self.charge * self.discharge))
1930
1931
1932 @dataclass(frozen=True)
1933 class PlanResult:
1934     q: np.ndarray
1935     planned_cost: float
1936     expected_emergency_cost: float
1937     expected_terminal_value: float
1938     initial_soc_marginal: float
1939     solve_seconds: float
1940     status: str
1941
1942
1943 ValueCut = tuple[float, float, float] # reference SOC, value, subgradient
1944
1945
1946 def evaluate_value_cuts(soc: float, cuts: tuple[ValueCut, ...] | None) -> float:
1947     if not cuts:
1948         return 0.0
1949     return max(
1950         0.0,
1951         max(
1952             value + slope * (soc - reference)
1953             for reference, value, slope in cuts
1954         ),
1955     )
1956
1957
1958 def _terminal_value_expression(
1959     terminal_soc: cp.Expression, cuts: tuple[ValueCut, ...] | None
1960 ) -> tuple[cp.Variable | None, list[cp.Constraint]]:
1961     if not cuts:
1962         return None, []
1963     value = cp.Variable(nonneg=True, name="terminal_value")
1964     constraints = [
1965         value >= cut_value + slope * (terminal_soc - reference_soc)

```

```

1966         for reference_soc, cut_value, slope in cuts
1967     ]
1968     return value, constraints
1969
1970
1971 def _contract_constraints(x: cp.Variable, q: cp.Variable | np.ndarray) -> list[cp.Constraint]:
1972     if CONTRACT_TAKE_MODE != "x_le_q":
1973         raise NotImplementedError(f"Unsupported pilot contract mode: {CONTRACT_TAKE_MODE}")
1974     return [x <= q]
1975
1976
1977 def _planned_normal_cost(price: np.ndarray, q: cp.Expression) -> cp.Expression:
1978     if NORMAL_COST_BASIS != "planned_q":
1979         raise NotImplementedError(f"Unsupported normal-cost basis: {NORMAL_COST_BASIS}")
1980     return price @ q
1981
1982
1983 def _physical_constraints(
1984     load: np.ndarray,
1985     pv: np.ndarray,
1986     initial_soc: float,
1987     normal: cp.Variable,
1988     emergency: cp.Variable,
1989     charge: cp.Variable,
1990     discharge: cp.Variable,
1991     curtailment: cp.Variable,
1992     soc: cp.Variable,
1993 ) -> list[cp.Constraint]:
1994     n = len(load)
1995     return [
1996         soc[0] == initial_soc,
1997         soc[1:] == soc[:-1] + ETA_C * charge - discharge / ETA_D,
1998         soc[1:] >= E_MIN_KWH,
1999         soc[1:] <= E_MAX_KWH,
2000         charge <= POWER_LIMIT_KWH,
2001         discharge <= POWER_LIMIT_KWH,
2002         curtailment <= pv,
2003         normal + emergency + pv - curtailment + discharge == load + charge,
2004     ]
2005
2006
2007 def solve_perfect_information_day(
2008     price: np.ndarray, load: np.ndarray, pv: np.ndarray, initial_soc: float
2009 ) -> DispatchResult:
2010     n = len(load)
2011     g, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
2012     E = cp.Variable(n + 1)
2013     constraints = _physical_constraints(load, pv, initial_soc, g, e, c, d, w, E)
2014     objective = cp.Minimize(price @ g + EMERGENCY_PRICE_MULTIPLIER * price @ e)
2015     problem = cp.Problem(objective, constraints)
2016     started = perf_counter()
2017     problem.solve(solver=SOLVER, verbose=False)
2018     elapsed = perf_counter() - started
2019     if g.value is None:
2020         raise RuntimeError(f"perfect-information LP failed: {problem.status}")
2021     return DispatchResult(
2022         x=np.asarray(g.value).ravel(),
2023         emergency=np.asarray(e.value).ravel(),
2024         charge=np.asarray(c.value).ravel(),
2025         discharge=np.asarray(d.value).ravel(),
2026         curtailment=np.asarray(w.value).ravel(),
2027         soc=np.asarray(E.value).ravel(),
2028         emergency_cost=float(EMERGENCY_PRICE_MULTIPLIER * price @ e.value),
2029         terminal_value=0.0,
2030         solve_seconds=elapsed,
2031         status=str(problem.status),
2032     )
2033

```

```

2034
2035 def solve_stochastic_plan(
2036     price: np.ndarray,
2037     scenario_load: np.ndarray,
2038     scenario_pv: np.ndarray,
2039     probabilities: np.ndarray,
2040     initial_soc: float,
2041     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
2042     q_floor: np.ndarray | None = None,
2043 ) -> PlanResult:
2044     k, n = scenario_load.shape
2045     q = cp.Variable(n, nonneg=True)
2046     expected_emergency = 0
2047     expected_terminal_value: cp.Expression | float = 0.0
2048     constraints: list[cp.Constraint] = []
2049     if q_floor is not None:
2050         floor = np.asarray(q_floor, dtype=float).ravel()
2051         if floor.shape != (n,) or np.any(floor < -1e-9):
2052             raise ValueError("q_floor must be a nonnegative vector matching the horizon")
2053         constraints.append(q >= floor)
2054     initial_constraints: list[cp.Constraint] = []
2055     for omega in range(k):
2056         x, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
2057         E = cp.Variable(n + 1)
2058         constraints += _contract_constraints(x, q)
2059         physical = _physical_constraints(
2060             scenario_load[omega],
2061             scenario_pv[omega],
2062             initial_soc,
2063             x,
2064             e,
2065             c,
2066             d,
2067             w,
2068             E,
2069         )
2070         constraints += physical
2071         initial_constraints.append(physical[0])
2072         terminal_value, terminal_constraints = _terminal_value_expression(
2073             E[-1], terminal_value_cuts
2074         )
2075         constraints += terminal_constraints
2076         if terminal_value is not None:
2077             expected_terminal_value += probabilities[omega] * terminal_value
2078             expected_emergency += (
2079                 probabilities[omega] * EMERGENCY_PRICE_MULTIPLIER * price @ e
2080             )
2081     planned_cost = _planned_normal_cost(price, q)
2082     problem = cp.Problem(
2083         cp.Minimize(planned_cost + expected_emergency + expected_terminal_value),
2084         constraints,
2085     )
2086     started = perf_counter()
2087     problem.solve(solver=SOLVER, verbose=False)
2088     elapsed = perf_counter() - started
2089     if q.value is None:
2090         raise RuntimeError(f"stochastic plan LP failed: {problem.status}")
2091     qv = np.asarray(q.value).ravel()
2092     marginal = -float(
2093         sum(float(np.asarray(constraint.dual_value)) for constraint in initial_constraints)
2094     )
2095     terminal_value_result = (
2096         float(expected_terminal_value.value)
2097         if isinstance(expected_terminal_value, cp.Expression)
2098         else float(expected_terminal_value)
2099     )
2100     return PlanResult(
2101         q=qv,

```

```

2102         planned_cost=float(price @ qv),
2103         expected_emergency_cost=float(problem.value - price @ qv - terminal_value_result),
2104         expected_terminal_value=terminal_value_result,
2105         initial_soc_marginal=marginal,
2106         solve_seconds=elapsed,
2107         status=str(problem.status),
2108     )
2109
2110
2111 @dataclass(frozen=True)
2112 class BaselinePlanResult(PlanResult):
2113     variable_shapes: dict[str, tuple[int, ...]]
2114     has_scenario_specific_battery: bool
2115
2116
2117 def solve_baseline_plan(
2118     price: np.ndarray,
2119     load_hat: np.ndarray,
2120     pv_hat: np.ndarray,
2121     initial_soc: float,
2122     q_floor: np.ndarray | None = None,
2123     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
2124 ) -> BaselinePlanResult:
2125     """Day-ahead LP on a single forecast path; q is locked and billed at p@q."""
2126     n = len(load_hat)
2127     if load_hat.shape != (n,) or pv_hat.shape != (n,) or price.shape != (n,):
2128         raise ValueError("baseline plan requires a single forecast trajectory")
2129     q = cp.Variable(n, nonneg=True, name="q")
2130     x = cp.Variable(n, nonneg=True, name="x")
2131     emergency = cp.Variable(n, nonneg=True, name="emergency")
2132     charge = cp.Variable(n, nonneg=True, name="charge")
2133     discharge = cp.Variable(n, nonneg=True, name="discharge")
2134     curtailment = cp.Variable(n, nonneg=True, name="curtailment")
2135     soc = cp.Variable(n + 1, name="soc")
2136     constraints = _contract_constraints(x, q)
2137     if q_floor is not None:
2138         floor = np.asarray(q_floor, dtype=float).ravel()
2139         if floor.shape != (n,) or np.any(floor < -1e-9):
2140             raise ValueError("q_floor must be a nonnegative vector matching the horizon")
2141         constraints.append(q >= floor)
2142     physical = _physical_constraints(
2143         load_hat, pv_hat, initial_soc, x, emergency, charge, discharge, curtailment, soc
2144     )
2145     constraints += physical
2146     terminal_value, terminal_constraints = _terminal_value_expression(
2147         soc[-1], terminal_value_cuts
2148     )
2149     constraints += terminal_constraints
2150     planned_cost = _planned_normal_cost(price, q)
2151     expected_emergency = EMERGENCY_PRICE_MULTIPLIER * price @ emergency
2152     objective = planned_cost + expected_emergency
2153     if terminal_value is not None:
2154         objective = objective + terminal_value
2155     problem = cp.Problem(cp.Minimize(objective), constraints)
2156     started = perf_counter()
2157     problem.solve(solver=SOLVER, verbose=False)
2158     elapsed = perf_counter() - started
2159     if q.value is None:
2160         raise RuntimeError(f"baseline plan LP failed: {problem.status}")
2161     qv = np.asarray(q.value).ravel()
2162     shapes = {
2163         str(variable.name()): tuple(int(dim) for dim in variable.shape)
2164         for variable in problem.variables()
2165     }
2166     has_scenario_battery = any(
2167         name in {"charge", "discharge", "soc"} and len(shape) == 2
2168         for name, shape in shapes.items()
2169     )

```

```

2170     terminal_value_result = (
2171         float(terminal_value.value) if terminal_value is not None else 0.0
2172     )
2173     return BaselinePlanResult(
2174         q=qv,
2175         planned_cost=float(price @ qv),
2176         expected_emergency_cost=float(EMERGENCY_PRICE_MULTIPLIER * price @ emergency.value),
2177         expected_terminal_value=terminal_value_result,
2178         initial_soc_marginal=-float(np.asarray(physical[0].dual_value)),
2179         solve_seconds=elapsed,
2180         status=str(problem.status),
2181         variable_shapes=shapes,
2182         has_scenario_specific_battery=has_scenario_battery,
2183     )
2184
2185
2186 def solve_fixed_plan_dispatch(
2187     price: np.ndarray,
2188     q: np.ndarray,
2189     load: np.ndarray,
2190     pv: np.ndarray,
2191     initial_soc: float,
2192     throughput_tiebreak: bool = True,
2193     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
2194 ) -> DispatchResult:
2195     n = len(load)
2196     x, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
2197     E = cp.Variable(n + 1)
2198     constraints = _contract_constraints(x, q)
2199     constraints += _physical_constraints(load, pv, initial_soc, x, e, c, d, w, E)
2200     emergency_cost = EMERGENCY_PRICE_MULTIPLIER * price @ e
2201     terminal_value, terminal_constraints = _terminal_value_expression(
2202         E[-1], terminal_value_cuts
2203     )
2204     constraints += terminal_constraints
2205     primary = emergency_cost + (terminal_value if terminal_value is not None else 0.0)
2206     first = cp.Problem(cp.Minimize(primary), constraints)
2207     started = perf_counter()
2208     first.solve(solver=SOLVER, verbose=False)
2209     if e.value is None:
2210         raise RuntimeError(f"fixed-plan execution LP failed: {first.status}")
2211     primary_opt = float(primary.value)
2212     status = str(first.status)
2213     if throughput_tiebreak:
2214         second = cp.Problem(
2215             cp.Minimize(cp.sum(c + d)),
2216             constraints + [primary <= primary_opt + MPC_COST_TOL],
2217         )
2218         second.solve(solver=SOLVER, verbose=False)
2219         if x.value is None:
2220             raise RuntimeError(f"fixed-plan tiebreak LP failed: {second.status}")
2221         status = str(second.status)
2222     elapsed = perf_counter() - started
2223     return DispatchResult(
2224         x=np.asarray(x.value).ravel(),
2225         emergency=np.asarray(e.value).ravel(),
2226         charge=np.asarray(c.value).ravel(),
2227         discharge=np.asarray(d.value).ravel(),
2228         curtailment=np.asarray(w.value).ravel(),
2229         soc=np.asarray(E.value).ravel(),
2230         emergency_cost=float(EMERGENCY_PRICE_MULTIPLIER * price @ e.value),
2231         terminal_value=evaluate_value_cuts(float(E.value[-1]), terminal_value_cuts),
2232         solve_seconds=elapsed,
2233         status=status,
2234     )
2235
2236
2237 def dispatch_balance_residual(

```

```

2238     load: np.ndarray, pv: np.ndarray, result: DispatchResult
2239 ) -> np.ndarray:
2240     return (
2241         result.x
2242         + result.emergency
2243         + pv
2244         - result.curtailment
2245         + result.discharge
2246         - load
2247         - result.charge
2248     )
2249
2250 # ===== q2/pilot.py =====
2251 from __future__ import annotations
2252
2253 import json
2254 import hashlib
2255 from dataclasses import dataclass
2256 from math import sqrt
2257 from pathlib import Path
2258
2259 import matplotlib.pyplot as plt
2260 import numpy as np
2261 import pandas as pd
2262
2263 from q2.config import (
2264     EMERGENCY_PRICE_MULTIPLIER,
2265     E_INITIAL_KWH,
2266     E_MAX_KWH,
2267     E_MIN_KWH,
2268     FIG_DIR,
2269     K_CANDIDATES,
2270     K_VALIDATION_DAYS,
2271     RISK_ALPHA_CANDIDATES,
2272     RISK_CALIBRATION_DAYS,
2273     NEXT_DAY_VALUE_GAP_TOL_YUAN,
2274     NEXT_DAY_VALUE_MAX_SAMPLES,
2275     NUMERIC_TOL,
2276     OUTPUT_DIR,
2277     PILOT_DATES,
2278     POWER_LIMIT_KWH,
2279     SIMULTANEOUS_CD_TOL,
2280     T,
2281     T_MAX_SECONDS,
2282 )
2283 from q2.data import Q2Data
2284 from q2.forecast import ForecastArchive, forecast_as_of
2285 from q2.optimization import (
2286     ValueCut,
2287     dispatch_balance_residual,
2288     evaluate_value_cuts,
2289     solve_fixed_plan_dispatch,
2290     solve_perfect_information_day,
2291     solve_stochastic_plan,
2292 )
2293 from q2.scenarios import (
2294     ScenarioSet,
2295     build_scenarios,
2296     posterior_weights,
2297     scenario_trajectories,
2298     write_scenarios_csv,
2299 )
2300
2301 @dataclass(frozen=True)
2302 class KChoice:
2303     target_index: int
2304     selected_k: int

```

```

2306     scenarios: ScenarioSet
2307     rows: tuple[dict, ...]
2308
2309
2310 @dataclass(frozen=True)
2311 class FreezeCalendar:
2312     daily_k: np.ndarray
2313     choices: tuple[KChoice | None, ...]
2314     rows: tuple[dict, ...]
2315
2316
2317 @dataclass(frozen=True)
2318 class RiskChoice:
2319     target_index: int
2320     selected_alpha: float
2321     rows: tuple[dict, ...]
2322
2323
2324 def planned_q_hash(q: np.ndarray) -> str:
2325     # Decimal canonicalization survives CSV round-trips while detecting any
2326     # operationally meaningful mutation of the locked plan.
2327     canonical = ",".join(f"{float(value):.9f}" for value in np.asarray(q).ravel())
2328     return hashlib.sha256(canonical.encode("ascii")).hexdigest()
2329
2330
2331 def _weighted_quantile(values: np.ndarray, weights: np.ndarray, alpha: float) -> float:
2332     order = np.argsort(values)
2333     cumulative = np.cumsum(weights[order])
2334     return float(values[order][np.searchsorted(cumulative, alpha, side="left")])
2335
2336
2337 def risk_quantile_floor(
2338     scenario_load: np.ndarray,
2339     scenario_pv: np.ndarray,
2340     probabilities: np.ndarray,
2341     base_load: np.ndarray,
2342     base_pv: np.ndarray,
2343     alpha: float | None,
2344 ) -> np.ndarray:
2345     """Nonnegative high-net-load residual reserve for the locked normal plan."""
2346     if alpha is None:
2347         return np.zeros(scenario_load.shape[1])
2348     if not 0.0 < alpha < 1.0:
2349         raise ValueError("risk alpha must lie strictly between 0 and 1")
2350     residual_net = (
2351         scenario_load - scenario_pv - (base_load[None, :] - base_pv[None, :])
2352     )
2353     return np.maximum(
2354         np.array(
2355             [
2356                 _weighted_quantile(residual_net[:, t], probabilities, alpha)
2357                 for t in range(residual_net.shape[1])
2358             ]
2359         ),
2360         0.0,
2361     )
2362
2363
2364 def scenario_inputs_as_of(
2365     data: Q2Data,
2366     archive: ForecastArchive,
2367     target_index: int,
2368     history_end_exclusive: int,
2369     k: int,
2370 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, ScenarioSet | None, dict]:
2371     load_hat, pv_hat, load_sources, pv_sources = forecast_as_of(
2372         data, target_index, history_end_exclusive
2373     )

```

```

2374 if history_end_exclusive == 0:
2375     loads, pvs = load_hat[None, :], pv_hat[None, :]
2376     probabilities = np.ones(1)
2377     scenarios = None
2378 else:
2379     scenarios = build_scenarios(
2380         target_index,
2381         data,
2382         archive,
2383         min(k, history_end_exclusive),
2384         history_end_exclusive=history_end_exclusive,
2385     )
2386     loads, pvs = scenario_trajectories(
2387         scenarios, archive, base_load=load_hat, base_pv=pv_hat
2388     )
2389     probabilities = scenarios.probabilities
2390     audit = {
2391         "target_date": data.dates[target_index].strftime("%Y-%m-%d"),
2392         "history_cutoff_date": (
2393             data.dates[history_end_exclusive - 1].strftime("%Y-%m-%d")
2394             if history_end_exclusive
2395             else "attachment1_fallback"
2396         ),
2397         "load_source_dates": ";".join(
2398             data.dates[j].strftime("%Y-%m-%d") for j in load_sources
2399         )
2400         or "attachment1_fallback",
2401         "pv_source_dates": ";".join(
2402             data.dates[j].strftime("%Y-%m-%d") for j in pv_sources
2403         )
2404         or "attachment1_fallback",
2405         "residual_pool_end_date": (
2406             data.dates[scenarios.pool_indices[-1]].strftime("%Y-%m-%d")
2407             if scenarios is not None
2408             else "none"
2409         ),
2410         "scenario_k": len(probabilities),
2411     }
2412     return loads, pvs, probabilities, scenarios, audit
2413
2414
2415 def build_next_day_value_cuts(
2416     data: Q2Data,
2417     archive: ForecastArchive,
2418     current_index: int,
2419     k: int,
2420     risk_alpha: float | None = None,
2421 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
2422     if current_index + 1 >= len(data.dates):
2423         return (), []
2424     # At day d 0:00, only indices < d are available. Day d actual data is excluded.
2425     loads, pvs, probabilities, _scenarios, source_audit = scenario_inputs_as_of(
2426         data,
2427         archive,
2428         target_index=current_index + 1,
2429         history_end_exclusive=current_index,
2430         k=k,
2431     )
2432     base_load, base_pv, _load_sources, _pv_sources = forecast_as_of(
2433         data, current_index + 1, current_index
2434     )
2435     q_floor = risk_quantile_floor(
2436         loads, pvs, probabilities, base_load, base_pv, risk_alpha
2437     )
2438     solved: dict[float, ValueCut] = {}
2439
2440     def solve_at(soc: float) -> None:
2441         result = solve_stochastic_plan(

```



```

2442     data.price, loads, pvs, probabilities, float(soc), q_floor=q_floor
2443 )
2444 value = result.planned_cost + result.expected_emergency_cost
2445 solved[float(soc)] = (
2446     float(soc),
2447     float(value),
2448     float(result.initial_soc_marginal),
2449 )
2450
2451 def certified_max_gap(cuts: tuple[ValueCut, ...]) -> tuple[float, float]:
2452     points = sorted(solved)
2453     best_gap, best_soc = -np.inf, points[0]
2454     lower_lines = [(0.0, 0.0)] + [
2455         (slope, value - slope * reference)
2456         for reference, value, slope in cuts
2457     ]
2458     for left, right in zip(points[:-1], points[1:]):
2459         left_value, right_value = solved[left][1], solved[right][1]
2460         upper_slope = (right_value - left_value) / (right - left)
2461         upper_intercept = left_value - upper_slope * left
2462         candidates = [left, right]
2463         for a in range(len(lower_lines)):
2464             for b in range(a + 1, len(lower_lines)):
2465                 slope_a, intercept_a = lower_lines[a]
2466                 slope_b, intercept_b = lower_lines[b]
2467                 if abs(slope_a - slope_b) <= 1e-14:
2468                     continue
2469                 cross = (intercept_b - intercept_a) / (slope_a - slope_b)
2470                 if left < cross < right:
2471                     candidates.append(float(cross))
2472         for soc in candidates:
2473             upper = upper_slope * soc + upper_intercept
2474             gap = upper - evaluate_value_cuts(soc, cuts)
2475             if gap > best_gap:
2476                 best_gap, best_soc = float(gap), float(soc)
2477     return max(best_gap, 0.0), best_soc
2478
2479 solve_at(E_MIN_KWH)
2480 solve_at(E_MAX_KWH)
2481 while True:
2482     cuts = tuple(solved[soc] for soc in sorted(solved))
2483     slopes = np.array([cut[2] for cut in cuts])
2484     if np.any(np.diff(slopes) < -1e-6):
2485         raise AssertionError("next-day LP value subgradients are not monotone")
2486     gap, refine_soc = certified_max_gap(cuts)
2487     if gap <= NEXT_DAY_VALUE_GAP_TOL_YUAN:
2488         break
2489     if len(cuts) >= NEXT_DAY_VALUE_MAX_SAMPLES:
2490         raise AssertionError(
2491             f"next-day value gap {gap:.6f} exceeds tolerance after {len(cuts)} samples"
2492         )
2493     if min(abs(refine_soc - soc) for soc in solved) < 1e-7:
2494         raise AssertionError("next-day value refinement stalled")
2495     solve_at(refine_soc)
2496
2497 rows = [
2498     {
2499         **source_audit,
2500         "soc_sample_kwh": reference,
2501         "virtual_next_day_cost_yuan": value,
2502         "value_subgradient_yuan_per_kwh": slope,
2503         "certified_max_gap_yuan": gap,
2504         "gap_tolerance_yuan": NEXT_DAY_VALUE_GAP_TOL_YUAN,
2505         "bundle_sample_count": len(cuts),
2506         "risk_alpha": risk_alpha,
2507         "risk_q_floor_kwh": float(q_floor.sum()),
2508     }
2509     for reference, value, slope in cuts

```

```

2510 ]
2511 for reference_soc, reference_value, _slope in cuts:
2512     if evaluate_value_cuts(reference_soc, cuts) > reference_value + 1e-4:
2513         raise AssertionError("next-day value supporting cuts violate sampled LP values")
2514 return tuple(cuts), rows
2515
2516
2517 def run_perfect_information(data: Q2Data, path: Path) -> np.ndarray:
2518     rows = []
2519     day_start_soc = np.empty(len(data.dates))
2520     soc = E_INITIAL_KWH
2521     for i, date in enumerate(data.dates):
2522         day_start_soc[i] = soc
2523         result = solve_perfect_information_day(data.price, data.load[i], data.pv[i], soc)
2524         residual = dispatch_balance_residual(data.load[i], data.pv[i], result)
2525         normal_cost = float(data.price @ result.x)
2526         rows.append(
2527             {
2528                 "date": date.strftime("%Y-%m-%d"),
2529                 "normal_purchase_cost_yuan": normal_cost,
2530                 "emergency_cost_yuan": result.emergency_cost,
2531                 "total_cost_yuan": normal_cost + result.emergency_cost,
2532                 "normal_purchase_kwh": float(result.x.sum()),
2533                 "emergency_kwh": float(result.emergency.sum()),
2534                 "curtailment_kwh": float(result.curtailment.sum()),
2535                 "soc_start_kwh": float(result.soc[0]),
2536                 "soc_end_kwh": float(result.soc[-1]),
2537                 "soc_min_kwh": float(result.soc.min()),
2538                 "soc_max_kwh": float(result.soc.max()),
2539                 "max_balance_residual_kwh": float(np.max(np.abs(residual))),
2540                 "max_simultaneous_cd_kwh2": result.max_cd,
2541                 "solve_seconds": result.solve_seconds,
2542                 "pass": bool(
2543                     np.max(np.abs(residual)) < NUMERIC_TOL
2544                     and result.emergency.sum() < NUMERIC_TOL
2545                     and result.soc.min() >= E_MIN_KWH - NUMERIC_TOL
2546                     and result.soc.max() <= E_MAX_KWH + NUMERIC_TOL
2547                 ),
2548             }
2549         )
2550     soc = float(result.soc[-1])
2551     frame = pd.DataFrame(rows)
2552     if not np.allclose(
2553         frame["soc_start_kwh"].to_numpy()[1:],
2554         frame["soc_end_kwh"].to_numpy()[:-1],
2555         atol=NUMERIC_TOL,
2556     ):
2557         raise AssertionError("perfect-information SOC is not continuous across days")
2558     frame.to_csv(path, index=False)
2559     return day_start_soc
2560
2561
2562 def _validation_score(
2563     data: Q2Data,
2564     archive: ForecastArchive,
2565     day_start_soc: np.ndarray,
2566     validation_index: int,
2567     k: int,
2568     risk_alpha: float | None = None,
2569 ) -> tuple[float, float, float]:
2570     scenarios = build_scenarios(validation_index, data, archive, k)
2571     loads, pvs = scenario_trajectories(scenarios, archive)
2572     q_floor = risk_quantile_floor(
2573         loads,
2574         pvs,
2575         scenarios.proBABILITIES,
2576         archive.load_hat[validation_index],
2577         archive.pv_hat[validation_index],

```

```

2578         risk_alpha,
2579     )
2580     plan = solve_stochastic_plan(
2581         data.price,
2582         loads,
2583         pvs,
2584         scenarios.proBABILITIES,
2585         day_start_soc[validation_index],
2586         q_floor=q_floor,
2587     )
2588     execution = solve_fixed_plan_dispatch(
2589         data.price,
2590         plan.q,
2591         data.load[validation_index],
2592         data.pv[validation_index],
2593         day_start_soc[validation_index],
2594     )
2595     actual_cost = plan.planned_cost + execution.emergency_cost
2596     return actual_cost, float(execution.emergency.sum()), plan.solve_seconds + execution.solve_seconds
2597
2598
2599 def select_risk_alpha(
2600     data: Q2Data,
2601     archive: ForecastArchive,
2602     policy_day_start_soc: np.ndarray,
2603     target_index: int,
2604     k: int,
2605 ) -> RiskChoice:
2606     """Choose the risk reserve by strictly prior 14-day realised-cost scores."""
2607     validation = np.arange(
2608         max(1, target_index - RISK_CALIBRATION_DAYS), target_index
2609     )
2610     records: list[dict] = []
2611     for alpha in RISK_ALPHA_CANDIDATES:
2612         costs, emergency, times = [], [], []
2613         for i in validation:
2614             cost, e, seconds = _validation_score(
2615                 data,
2616                 archive,
2617                 policy_day_start_soc,
2618                 int(i),
2619                 k,
2620                 risk_alpha=float(alpha),
2621             )
2622             costs.append(cost)
2623             emergency.append(e)
2624             times.append(seconds)
2625         records.append(
2626             {
2627                 "risk_alpha": float(alpha),
2628                 "validation_days": len(validation),
2629                 "mean_validation_cost_yuan": float(np.mean(costs)),
2630                 "standard_error_yuan": float(
2631                     np.std(costs, ddof=1) / sqrt(len(costs)) if len(costs) > 1 else 0.0
2632                 ),
2633                 "emergency_purchase_kwh": float(np.sum(emergency)),
2634                 "mean_solve_seconds": float(np.mean(times)),
2635                 "within_time_limit": bool(
2636                     T_MAX_SECONDS is None or np.mean(times) <= T_MAX_SECONDS
2637                 ),
2638                 "selected": False,
2639             }
2640         )
2641     eligible = [row for row in records if row["within_time_limit"]]
2642     selected = min(
2643         eligible or records,
2644         key=lambda row: (row["mean_validation_cost_yuan"], row["risk_alpha"]),
2645     )

```

```

2646     selected["selected"] = True
2647     return RiskChoice(target_index, float(selected["risk_alpha"]), tuple(records))
2648
2649
2650 def select_dynamic_k(
2651     data: Q2Data,
2652     archive: ForecastArchive,
2653     day_start_soc: np.ndarray,
2654     target_index: int,
2655 ) -> KChoice:
2656     validation = np.arange(max(1, target_index - K_VALIDATION_DAYS), target_index)
2657     min_pool = min(min(28, i) for i in validation)
2658     candidates = [k for k in K_CANDIDATES if k <= min_pool]
2659     if not candidates:
2660         candidates = [1]
2661     records: list[dict] = []
2662     for k in candidates:
2663         costs, emergency, times = [], [], []
2664         for i in validation:
2665             cost, e, seconds = _validation_score(data, archive, day_start_soc, int(i), k)
2666             costs.append(cost)
2667             emergency.append(e)
2668             times.append(seconds)
2669         records.append(
2670             {
2671                 "calibration_date": data.dates[target_index].strftime("%Y-%m-%d"),
2672                 "candidate_k": k,
2673                 "validation_days": len(validation),
2674                 "mean_validation_cost_yuan": float(np.mean(costs)),
2675                 "standard_error_yuan": float(
2676                     np.std(costs, ddof=1) / sqrt(len(costs)) if len(costs) > 1 else 0.0
2677                 ),
2678                 "emergency_purchase_kwh": float(np.sum(emergency)),
2679                 "mean_solve_seconds": float(np.mean(times)),
2680                 "within_time_limit": bool(
2681                     T_MAX_SECONDS is None or np.mean(times) <= T_MAX_SECONDS
2682                 ),
2683                 "selected": False,
2684             }
2685         )
2686     best = min(records, key=lambda x: x["mean_validation_cost_yuan"])
2687     threshold = best["mean_validation_cost_yuan"] + best["standard_error_yuan"]
2688     eligible = [
2689         row
2690         for row in records
2691         if row["mean_validation_cost_yuan"] <= threshold and row["within_time_limit"]
2692     ]
2693     selected = min(eligible, key=lambda x: x["candidate_k"]) if eligible else best
2694     selected["selected"] = True
2695     scenarios = build_scenarios(target_index, data, archive, int(selected["candidate_k"]))
2696     return KChoice(target_index, int(selected["candidate_k"]), scenarios, tuple(records))
2697
2698
2699 def build_k_freeze_calendar(
2700     data: Q2Data,
2701     archive: ForecastArchive,
2702     day_start_soc: np.ndarray,
2703 ) -> FreezeCalendar:
2704     daily_k = np.ones(len(data.dates), dtype=int)
2705     choices: list[KChoice | None] = []
2706     calendar_rows: list[dict] = []
2707     for calibration in range(0, len(data.dates), 14):
2708         effective_end = min(calibration + 13, len(data.dates) - 1)
2709         if calibration < 2:
2710             selected_k = 1
2711             choice = None
2712             candidate_rows = [
2713

```

```

2714         "candidate_k": 1,
2715         "validation_days": 0,
2716         "mean_validation_cost_yuan": np.nan,
2717         "standard_error_yuan": np.nan,
2718         "emergency_purchase_kwh": np.nan,
2719         "mean_solve_seconds": np.nan,
2720         "within_time_limit": True,
2721         "selected": True,
2722     }
2723 ]
2724 fallback = "no_complete_prior_residual_pool"
2725 else:
2726     choice = select_dynamic_k(
2727         data, archive, day_start_soc, calibration
2728     )
2729     selected_k = choice.selected_k
2730     candidate_rows = list(choice.rows)
2731     fallback = "K=1_early_pool_fallback" if selected_k == 1 else ""
2732     daily_k[calibration : effective_end + 1] = selected_k
2733     choices.append(choice)
2734     for candidate in candidate_rows:
2735         calendar_rows.append(
2736             {
2737                 "calibration_date": data.dates[calibration].strftime("%Y-%m-%d"),
2738                 "history_cutoff_date": (
2739                     data.dates[calibration - 1].strftime("%Y-%m-%d")
2740                     if calibration
2741                     else "none"
2742                 ),
2743                 "candidate_k": int(candidate["candidate_k"]),
2744                 "selected_k": selected_k,
2745                 "effective_start_date": data.dates[calibration].strftime("%Y-%m-%d"),
2746                 "effective_end_date": data.dates[effective_end].strftime("%Y-%m-%d"),
2747                 "mean_validation_cost_yuan": candidate["mean_validation_cost_yuan"],
2748                 "standard_error_yuan": candidate["standard_error_yuan"],
2749                 "validation_days": candidate["validation_days"],
2750                 "emergency_purchase_kwh": candidate[
2751                     "emergency_purchase_kwh"
2752                 ],
2753                 "mean_solve_seconds": candidate["mean_solve_seconds"],
2754                 "within_time_limit": candidate["within_time_limit"],
2755                 "candidate_selected": bool(candidate["selected"]),
2756                 "selected": bool(candidate["selected"]),
2757                 "fallback_reason": fallback,
2758             }
2759         )
2760     return FreezeCalendar(daily_k, tuple(choices), tuple(calendar_rows))
2761
2762
2763 def write_k_freeze_calendar(path: Path, calendar: FreezeCalendar) -> None:
2764     pd.DataFrame(calendar.rows).to_csv(path, index=False)
2765
2766
2767 def write_k_audit(path: Path, choices: list[KChoice], data: Q2Data) -> None:
2768     lines = [
2769         "## Q2 动态 K 试算审计",
2770         "",
2771         "- 每个试算日前只使用此前 14 个验证日；每个验证日的情景池只含更早日期。",
2772         "- 验证成本以锁定计划 q 在验证日实际轨迹上的固定计划再调度计算；实际轨迹只用于已结束验证日评分。",
2773         "- 采用一标准误规则选取满足阈值的最小 K。",
2774         "- `T_MAX_SECONDS` 尚未由队长给定，本试算只记录耗时，不据此剔除 K。",
2775         "- 初期池不足时，自动删除大于可用完整残差日数量的候选 K；若不足 2 日则回退 K=1。",
2776         "- 完整冻结日历见 `k_freeze_calendar.csv`；两个目标日使用其所在周期已冻结的 K。",
2777         "",
2778     ]
2779     for choice in choices:
2780         pool = choice.scenarios.pool_indices
2781         lines += [

```

```

2782         f"## {data.dates[choice.target_index].strftime('%Y-%m-%d')}",
2783         "",
2784         f"- 选中 K: {choice.selected_k}",
2785         f"- 情景候选池: {data.dates[pool[0]].strftime('%Y-%m-%d')} 至 "
2786         f"{data.dates[pool[-1]].strftime('%Y-%m-%d')}, 共 {len(pool)} 日。",
2787         f"- 压力测试日: {data.dates[choice.scenarios.stress_index].strftime('%Y-%m-%d')} (不额外加入期望模型)。",
2788         "",
2789     ]
2790     path.write_text("\n".join(lines), encoding="utf-8")
2791
2792
2793 def run_posterior_mpc(
2794     data: Q2Data,
2795     archive: ForecastArchive,
2796     choice: KChoice | None,
2797     initial_soc: float,
2798     path: Path | None,
2799     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
2800     variant: str = "with_48h_value",
2801     risk_alpha: float | None = None,
2802 ) -> dict:
2803     if choice is None:
2804         i = 0
2805         scenarios = None
2806         scenario_load = archive.load_hat[i][None, :]
2807         scenario_pv = archive.pv_hat[i][None, :]
2808         probabilities = np.ones(1)
2809         selected_k = 1
2810         q_floor = np.zeros(T)
2811     else:
2812         i = choice.target_index
2813         scenarios = choice.scenarios
2814         scenario_load, scenario_pv = scenario_trajectories(scenarios, archive)
2815         probabilities = scenarios.probabilities
2816         selected_k = choice.selected_k
2817         q_floor = risk_quantile_floor(
2818             scenario_load,
2819             scenario_pv,
2820             probabilities,
2821             archive.load_hat[i],
2822             archive.pv_hat[i],
2823             risk_alpha,
2824         )
2825     plan = solve_stochastic_plan(
2826         data.price,
2827         scenario_load,
2828         scenario_pv,
2829         probabilities,
2830         initial_soc,
2831         terminal_value_cuts=terminal_value_cuts,
2832         q_floor=q_floor,
2833     )
2834     locked_q = plan.q.copy()
2835     locked_q_sha256 = planned_q_hash(locked_q)
2836     rows = []
2837     soc = initial_soc
2838     max_cd = 0.0
2839     for t in range(T):
2840         if scenarios is None:
2841             weights = np.ones(1)
2842             bandwidth = np.nan
2843             load_horizon = archive.load_hat[i, t:].copy()
2844             pv_horizon = archive.pv_hat[i, t:].copy()
2845         else:
2846             observed_lr = archive.load_residual[i, : t + 1]
2847             observed_pr = archive.pv_residual[i, : t + 1]
2848             weights, bandwidth = posterior_weights(
2849                 scenarios, archive, observed_lr, observed_pr

```

```

2850     )
2851     load_horizon = np.maximum(
2852         0.0,
2853         archive.load_hat[i, t:]
2854         + weights @ archive.load_residual[scenarios.medoid_indices, t:],
2855     )
2856     pv_horizon = np.maximum(
2857         0.0,
2858         archive.pv_hat[i, t:]
2859         + weights @ archive.pv_residual[scenarios.medoid_indices, t:],
2860     )
2861     load_horizon[0] = data.load[i, t]
2862     pv_horizon[0] = data.pv[i, t]
2863     result = solve_fixed_plan_dispatch(
2864         data.price[t:],
2865         locked_q[t:],
2866         load_horizon,
2867         pv_horizon,
2868         soc,
2869         terminal_value_cuts=terminal_value_cuts,
2870     )
2871     residual = (
2872         result.x[0]
2873         + result.emergency[0]
2874         + data.pv[i, t]
2875         - result.curtailment[0]
2876         + result.discharge[0]
2877         - data.load[i, t]
2878         - result.charge[0]
2879     )
2880     entropy = float(-np.sum(weights * np.log(np.maximum(weights, 1e-300))))
2881     rows.append(
2882         {
2883             "date": data.dates[i].strftime("%Y-%m-%d"),
2884             "period": t,
2885             "time": data.time_labels[t],
2886             "planned_q_kwh": locked_q[t],
2887             "actual_x_kwh": result.x[0],
2888             "load_kwh": data.load[i, t],
2889             "pv_kwh": data.pv[i, t],
2890             "charge_kwh": result.charge[0],
2891             "discharge_kwh": result.discharge[0],
2892             "curtailment_kwh": result.curtailment[0],
2893             "emergency_kwh": result.emergency[0],
2894             "soc_kwh": result.soc[1],
2895             "posterior_entropy": entropy,
2896             "posterior_bandwidth": bandwidth,
2897             "solve_seconds": result.solve_seconds,
2898             "virtual_next_day_value_yuan": result.terminal_value,
2899             "balance_residual_kwh": residual,
2900         }
2901     )
2902     max_cd = max(max_cd, float(result.charge[0] * result.discharge[0]))
2903     soc = float(result.soc[1])
2904     if not np.array_equal(plan.q, locked_q):
2905         raise AssertionError("locked day-ahead q changed during MPC")
2906     if planned_q_hash(locked_q) != locked_q_sha256:
2907         raise AssertionError("locked day-ahead q hash changed during MPC")
2908     frame = pd.DataFrame(rows)
2909     if path is not None:
2910         frame.to_csv(path, index=False)
2911     return {
2912         "date": data.dates[i].strftime("%Y-%m-%d"),
2913         "variant": variant,
2914         "selected_k": selected_k,
2915         "risk_alpha": risk_alpha,
2916         "risk_q_floor_kwh": float(q_floor.sum()),
2917         "planned_cost_yuan": plan.planned_cost,

```

```

2918     "emergency_cost_yuan": float(
2919         np.sum(
2920             EMERGENCY_PRICE_MULTIPLIER
2921             * data.price
2922             * frame["emergency_kwh"].to_numpy()
2923         )
2924     ),
2925     "total_cost_yuan": float(
2926         plan.planned_cost
2927         + np.sum(
2928             EMERGENCY_PRICE_MULTIPLIER
2929             * data.price
2930             * frame["emergency_kwh"].to_numpy()
2931         )
2932     ),
2933     "planned_q_kwh": float(locked_q.sum()),
2934     "planned_q_sha256": locked_q.sha256,
2935     "day_ahead_virtual_next_day_value_yuan": plan.expected_terminal_value,
2936     "actual_x_kwh": float(frame["actual_x_kwh"].sum()),
2937     "unused_plan_kwh": float((locked_q - frame["actual_x_kwh"].to_numpy()).sum()),
2938     "emergency_kwh": float(frame["emergency_kwh"].sum()),
2939     "curtailment_kwh": float(frame["curtailment_kwh"].sum()),
2940     "soc_start_kwh": initial_soc,
2941     "soc_end_kwh": soc,
2942     "max_balance_residual_kwh": float(frame["balance_residual_kwh"].abs().max()),
2943     "max_simultaneous_cd_kwh2": max_cd,
2944     "max_x_minus_q_kwh": float(
2945         np.max(frame["actual_x_kwh"].to_numpy() - locked_q)
2946     ),
2947     "mean_mpc_solve_seconds": float(frame["solve_seconds"].mean()),
2948     "total_mpc_solve_seconds": float(frame["solve_seconds"].sum()),
2949     "terminal_virtual_next_day_value_yuan": float(
2950         frame["virtual_next_day_value_yuan"].iloc[-1]
2951     ),
2952     "estimated_334_day_runtime_hours": float(
2953         frame["solve_seconds"].mean() * T * 334 / 3600.0
2954     ),
2955     "pass": bool(
2956         frame["balance_residual_kwh"].abs().max() < NUMERIC_TOL
2957         and np.max(frame["actual_x_kwh"].to_numpy() - locked_q) < NUMERIC_TOL
2958         and frame["soc_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL
2959         and frame["soc_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL
2960         and max_cd <= SIMULTANEOUS_CD_TOL
2961     ),
2962 }
2963
2964
2965 def run_linked_warmup(
2966     data: Q2Data,
2967     archive: ForecastArchive,
2968     calendar: FreezeCalendar,
2969     path: Path,
2970 ) -> tuple[pd.DataFrame, list[dict]]:
2971     rows = []
2972     soc = E_INITIAL_KWH
2973     value_audits: list[dict] = []
2974     for i in range(31):
2975         k = int(calendar.daily_k[i])
2976         if i == 0:
2977             choice = None
2978         else:
2979             scenarios = build_scenarios(i, data, archive, k)
2980             choice = KChoice(i, k, scenarios, ())
2981         cuts, cut_rows = build_next_day_value_cuts(data, archive, i, k)
2982         value_audits.extend({"decision_date": data.dates[i].strftime("%Y-%m-%d"), **row} for row in cut_rows)
2983         start_soc = soc
2984         result = run_posterior_mpc(
2985             data,

```



```

2986         archive,
2987         choice,
2988         start_soc,
2989         path=None,
2990         terminal_value_cuts=cuts,
2991         variant="linked_warmup_48h",
2992     )
2993     soc = float(result["soc_end_kwh"])
2994     forecast_fallback = (
2995         len(archive.load_sources[i]) == 0 or len(archive.pv_sources[i]) == 0
2996     )
2997     rows.append(
2998         {
2999             "date": data.dates[i].strftime("%Y-%m-%d"),
3000             "soc_start_kwh": start_soc,
3001             "soc_end_kwh": soc,
3002             "planned_cost_yuan": result["planned_cost_yuan"],
3003             "emergency_cost_yuan": result["emergency_cost_yuan"],
3004             "total_cost_yuan": result["total_cost_yuan"],
3005             "selected_k": k,
3006             "forecast_fallback": forecast_fallback,
3007             "k_fallback": k == 1,
3008             "max_balance_residual_kwh": result["max_balance_residual_kwh"],
3009             "max_simultaneous_cd_kwh2": result[
3010                 "max_simultaneous_cd_kwh2"
3011             ],
3012             "pass": result["pass"],
3013         }
3014     )
3015     frame = pd.DataFrame(rows)
3016     continuity = np.abs(
3017         frame["soc_start_kwh"].to_numpy()[1:]
3018         - frame["soc_end_kwh"].to_numpy()[:-1]
3019     )
3020     if np.max(continuity) >= NUMERIC_TOL or not frame["pass"].all():
3021         raise AssertionError("linked January warmup validation failed")
3022     frame.to_csv(path, index=False)
3023     return frame, value_audits
3024
3025
3026 def run_full_linked_policy(
3027     data: Q2Data,
3028     archive: ForecastArchive,
3029     output_dir: Path,
3030 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
3031     """Run the deployable policy in one causal state path for all 365 days.
3032
3033     At every 14-day boundary, only states and realised trajectories from prior
3034     days have been written to ``policy_day_start_soc``. Thus the K score,
3035     frozen K, day-ahead plan and realised MPC actions share one SOC history.
3036     """
3037     output_dir.mkdir(parents=True, exist_ok=True)
3038     dispatch_dir = output_dir / "dispatch_daily"
3039     dispatch_dir.mkdir(parents=True, exist_ok=True)
3040     policy_day_start_soc = np.full(len(data.dates), np.nan)
3041     daily_k = np.ones(len(data.dates), dtype=int)
3042     daily_rows: list[dict] = []
3043     calendar_rows: list[dict] = []
3044     value_audits: list[dict] = []
3045     soc = E_INITIAL_KWH
3046
3047     for calibration in range(0, len(data.dates), 14):
3048         effective_end = min(calibration + 13, len(data.dates) - 1)
3049         if calibration < 2:
3050             selected_k = 1
3051             candidate_rows = [
3052                 {
3053                     "candidate_k": 1,

```

```

3054         "validation_days": 0,
3055         "mean_validation_cost_yuan": np.nan,
3056         "standard_error_yuan": np.nan,
3057         "emergency_purchase_kwh": np.nan,
3058         "mean_solve_seconds": np.nan,
3059         "within_time_limit": True,
3060         "selected": True,
3061     }
3062 ]
3063 fallback = "no_complete_prior_residual_pool"
3064 else:
3065     choice = select_dynamic_k(data, archive, policy_day_start_soc, calibration)
3066     selected_k = choice.selected_k
3067     candidate_rows = list(choice.rows)
3068     fallback = "K=1_early_pool_fallback" if selected_k == 1 else ""
3069     daily_k[calibration : effective_end + 1] = selected_k
3070     for candidate in candidate_rows:
3071         calendar_rows.append(
3072             {
3073                 "calibration_date": data.dates[calibration].strftime("%Y-%m-%d"),
3074                 "history_cutoff_date": (
3075                     data.dates[calibration - 1].strftime("%Y-%m-%d")
3076                     if calibration else "none"
3077                 ),
3078                 "candidate_k": int(candidate["candidate_k"]),
3079                 "selected_k": selected_k,
3080                 "effective_start_date": data.dates[calibration].strftime("%Y-%m-%d"),
3081                 "effective_end_date": data.dates[effective_end].strftime("%Y-%m-%d"),
3082                 "mean_validation_cost_yuan": candidate["mean_validation_cost_yuan"],
3083                 "standard_error_yuan": candidate["standard_error_yuan"],
3084                 "validation_days": candidate["validation_days"],
3085                 "emergency_purchase_kwh": candidate["emergency_purchase_kwh"],
3086                 "mean_solve_seconds": candidate["mean_solve_seconds"],
3087                 "within_time_limit": candidate["within_time_limit"],
3088                 "candidate_selected": bool(candidate["selected"]),
3089                 "selected": bool(candidate["selected"]),
3090                 "fallback_reason": fallback,
3091             }
3092         )
3093
3094     for i in range(calibration, effective_end + 1):
3095         policy_day_start_soc[i] = soc
3096         k = int(daily_k[i])
3097         if i == 0:
3098             day_choice = None
3099         else:
3100             day_choice = KChoice(i, k, build_scenarios(i, data, archive, k), ())
3101         cuts, cut_rows = build_next_day_value_cuts(data, archive, i, k)
3102         value_audits.extend(
3103             {"decision_date": data.dates[i].strftime("%Y-%m-%d"), **row}
3104             for row in cut_rows
3105         )
3106         path = dispatch_dir / f"dispatch_{data.dates[i].strftime('%Y-%m-%d')}.csv"
3107         result = run_posterior_mpc(
3108             data,
3109             archive,
3110             day_choice,
3111             soc,
3112             path=path,
3113             terminal_value_cuts=cuts,
3114             variant="full_linked_48h",
3115         )
3116         if not result["pass"]:
3117             raise AssertionError(f"full linked policy failed on {result['date']}")
3118         daily_rows.append(result)
3119         soc = float(result["soc_end_kwh"])
3120
3121 daily = pd.DataFrame(daily_rows)

```

```

3122     calendar = pd.DataFrame(calendar_rows)
3123     value_audit = pd.DataFrame(value_audits)
3124     if len(daily) != len(data.dates) or not daily["pass"].all():
3125         raise AssertionError("full linked policy does not cover every day")
3126     continuity = np.abs(
3127         daily["soc_start_kwh"].to_numpy()[1:]
3128         - daily["soc_end_kwh"].to_numpy()[:-1]
3129     )
3130     if np.max(continuity) >= NUMERIC_TOL:
3131         raise AssertionError("full linked policy SOC is not continuous")
3132     daily.to_csv(output_dir / "daily_summary.csv", index=False)
3133     calendar.to_csv(output_dir / "k_freeze_calendar.csv", index=False)
3134     value_audit.to_csv(output_dir / "next_day_value_audit.csv", index=False)
3135     return daily, calendar, value_audit
3136
3137
3138 def write_pilot_figures(data: Q2Data, summaries: list[dict]) -> None:
3139     FIG_DIR.mkdir(parents=True, exist_ok=True)
3140     plt.rcParams["axes.unicode_minus"] = False
3141     for summary in summaries:
3142         date = summary["date"]
3143         frame = pd.read_csv(OUTPUT_DIR / f"q2_pilot_dispatch_{date}.csv")
3144         hours = (np.arange(T) + 1) / 6.0
3145         fig, axes = plt.subplots(2, 1, figsize=(10, 6.5), sharex=True)
3146         axes[0].plot(hours, frame["load_kwh"], label="Load")
3147         axes[0].plot(hours, frame["pv_kwh"], label="PV")
3148         axes[0].plot(hours, frame["planned_q_kwh"], label="Planned q")
3149         axes[0].plot(hours, frame["emergency_kwh"], label="Emergency")
3150         axes[0].legend(ncol=4, fontsize=8)
3151         axes[0].set_ylabel("kWh / period")
3152         axes[0].grid(alpha=0.25)
3153         axes[1].plot(hours, frame["soc_kwh"], label="SOC", color="#1d4ed8")
3154         axes[1].set_ylabel("kWh")
3155         axes[1].set_xlabel("Hour")
3156         axes[1].grid(alpha=0.25)
3157         fig.suptitle(f"Q2 pilot MPC: {date}")
3158         fig.tight_layout()
3159         fig.savefig(FIG_DIR / f"q2_pilot_{date}.png", dpi=180)
3160         plt.close(fig)
3161
3162
3163 def write_validation_report(
3164     path_json: Path,
3165     path_md: Path,
3166     perfect_path: Path,
3167     summaries: list[dict],
3168 ) -> None:
3169     perfect = pd.read_csv(perfect_path)
3170     warmup = pd.read_csv(OUTPUT_DIR / "linked_warmup_daily.csv")
3171     calendar = pd.read_csv(OUTPUT_DIR / "k_freeze_calendar.csv")
3172     value_comparison = pd.read_csv(OUTPUT_DIR / "next_day_value_comparison.csv")
3173     warmup_gap = float(
3174         np.max(
3175             np.abs(
3176                 warmup["soc_start_kwh"].to_numpy()[1:]
3177                 - warmup["soc_end_kwh"].to_numpy()[:-1]
3178             )
3179         )
3180     )
3181     calendar_periods = calendar[
3182         ["effective_start_date", "effective_end_date", "selected_k"]
3183     ].drop_duplicates()
3184     report = {
3185         "status": (
3186             "pass"
3187             if (
3188                 perfect["pass"].all()
3189                 and all(x["pass"] for x in summaries)

```

```

3190         and warmup["pass"].all()
3191         and warmup_gap < NUMERIC_TOL
3192         and len(calendar_periods) == int(np.ceil(365 / 14))
3193         and not value_comparison["virtual_next_day_value_billed_today"].any()
3194     )
3195     else "fail"
3196 ),
3197 "perfect_information": {
3198     "days": len(perfect),
3199     "all_days_pass": bool(perfect["pass"].all()),
3200     "max_balance_residual_kwh": float(perfect["max_balance_residual_kwh"].max()),
3201     "emergency_purchase_kwh": float(perfect["emergency_kwh"].sum()),
3202     "max_cross_day_soc_gap_kwh": float(
3203         np.max(
3204             np.abs(
3205                 perfect["soc_start_kwh"].to_numpy()[1:]
3206                 - perfect["soc_end_kwh"].to_numpy()[:-1]
3207             )
3208         )
3209     ),
3210     "max_simultaneous_cd_kwh2": float(
3211         perfect["max_simultaneous_cd_kwh2"].max()
3212     ),
3213 },
3214 "pilot_days": summaries,
3215 "linked_january_warmup": {
3216     "days": len(warmup),
3217     "soc_continuity_max_gap_kwh": warmup_gap,
3218     "february_1_initial_soc_kwh": float(warmup["soc_end_kwh"].iloc[-1]),
3219     "all_days_pass": bool(warmup["pass"].all()),
3220 },
3221 "next_day_value_comparison": value_comparison.to_dict(orient="records"),
3222 "k_freeze_calendar": {
3223     "calibration_periods": len(calendar_periods),
3224     "first_effective_date": calendar_periods["effective_start_date"].iloc[0],
3225     "last_effective_date": calendar_periods["effective_end_date"].iloc[-1],
3226     "all_periods_at_most_14_days": bool(
3227         all(
3228             (
3229                 pd.Timestamp(row.effective_end_date)
3230                 - pd.Timestamp(row.effective_start_date)
3231             ).days
3232             <= 13
3233             for row in calendar_periods.itertuples()
3234         )
3235     ),
3236 },
3237 "contract_audit": {
3238     "normal_cost_uses": "sum(price_t * planned_q_t)",
3239     "emergency_cost_uses": "sum(5 * price_t * emergency_t)",
3240     "actual_normal_purchase_constraint": "0 <= x_t <= q_t",
3241     "q_locked_during_day": True,
3242 },
3243 "output_guard": "No result2.xlsx is created or modified by the pilot.",
3244 "historical_pilot_notes": [
3245     "x<=q contract semantics were signed off for the production Q2 model",
3246     (
3247         "P1 is a sequential daily perfect-information comparator, not a proven "
3248         "global lower bound against a controller with a different horizon"
3249     ),
3250     "T_max was later fixed at 0.20 seconds per validation day for production K filtering",
3251     "June 21 remains an isolated pilot initialized from the comparator path",
3252     "the next-day value is an adaptively certified LP cut approximation signed off for Q2",
3253 ],
3254 }
3255 path_json.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
3256 lines = [
3257     "# Q2 分阶段试算验证",

```

```

3258     "",
3259     f"- 总体状态: **{report['status']}**",
3260     f"- 逐日完美信息比较器 365 日全部通过: {report['perfect_information']['all_days_pass']}",
3261     f"- 完美信息紧急购电: {report['perfect_information']['emergency_purchase_kwh']:.6e} kWh",
3262     f"- 完美信息最大能量平衡残差: {report['perfect_information']['max_balance_residual_kwh']:.6e} kWh",
3263     f"- 完美信息跨日 SOC 最大断点: {report['perfect_information']['max_cross_day_soc_gap_kwh']:.6e} kWh",
3264     f"- 联动 1 月预热全部通过: {report['linked_january_warmup']['all_days_pass']}; "
3265     f"跨日 SOC 最大断点 {warmup_gap:.3e} kWh; 2 月 1 日初值 "
3266     f"{report['linked_january_warmup']['february_1_initial_soc_kwh']:.6f} kWh。",
3267     f"- K 冻结日历覆盖 {report['k_freeze_calendar']['first_effective_date']} 至 "
3268     f"{report['k_freeze_calendar']['last_effective_date']}, 共 "
3269     f"{report['k_freeze_calendar']['calibration_periods']} 个周期。",
3270     "",
3271     "## 两个 MPC 试算日",
3272     "",
3273 ]
3274 for item in summaries:
3275     lines += [
3276         f"### {item['date']}",
3277         "",
3278         f"- K={item['selected_k']}, 计划成本 {item['planned_cost_yuan']:.6f} 元, "
3279         f"紧急成本 {item['emergency_cost_yuan']:.6f} 元, 总成本 {item['total_cost_yuan']:.6f} 元。",
3280         f"- 次日虚拟价值仅进入决策目标, 不计入上述当天实际成本; 日初使用 "
3281         f"{item['soc_start_kwh']:.6f} kWh。",
3282         f"- 紧急购电 {item['emergency_kwh']:.6f} kWh, 弃光 {item['curtailment_kwh']:.6f} kWh。",
3283         f"- SOC {item['soc_start_kwh']:.6f} + {item['soc_end_kwh']:.6f} kWh。",
3284         f"- 最大平衡残差 {item['max_balance_residual_kwh']:.3e} kWh; "
3285         f"max(c*d)={item['max_simultaneous_cd_kwh2']:.3e} kWh²; 通过={item['pass']}".",
3286         f"- 每步平均 {item['mean_mpc_solve_seconds']:.4f} s; 外推 334 日约 "
3287         f"{item['estimated_334_day_runtime_hours']:.2f} h。",
3288         "",
3289     ]
3290     lines += ["## 48 小时价值对比", ""]
3291     for item in value_comparison.to_dict(orient="records"):
3292         lines += [
3293             f"- {item['date']}: 当天有/无价值成本差 {item['cost_difference_yuan']:.6f} 元; "
3294             f"两日实现成本差 {item['realized_two_day_cost_difference_yuan']:.6f} 元; "
3295             f"日末 SOC 差 {item['soc_end_difference_kwh']:.6f} kWh; "
3296             f"紧急购电差 {item['emergency_difference_kwh']:.6f} kWh。",
3297         ]
3298     lines += [""]
3299     lines += [
3300         "## 历史试算边界 (已由生产模型处理)",
3301         "",
3302         "- `x<q` 合同语义已由队长签收; 生产模型按计划购电量 q 计费。",
3303         "- P1 与主方案视域不同, 因此当前只能称“逐日完美信息比较器”, 不能据此强制认定全局下界。",
3304         "- 生产模型已固定 `T_max=0.20` 秒/验证日, 并采用 K=8 与滚动风险分位。",
3305         f"- 次日价值采用自适应 LP 对偶支撑切面, 弦线一切面最大认证误差不超过 "
3306         f"{NEXT_DAY_VALUE_GAP_TOL_YUAN:.2f} 元; 该口径已在生产 Q2 中签收。",
3307         "- 6 月 21 日仍为孤立试算, 其日初 SOC 暂取逐日比较器路径; 2 月 1 日已改用联动 1 月预热未值。",
3308         "- 未生成或修改 `result2.xlsx`。",
3309     ]
3310     path_md.write_text("\n".join(lines) + "\n", encoding="utf-8")
3311
3312 # ===== q2/plot_final_figures.py =====
3313 #!/usr/bin/env python3
3314 """Render Q2 final figures F1-F4 from signed-off result artifacts only."""
3315
3316 from __future__ import annotations
3317
3318 import hashlib
3319 import json
3320 import sys
3321 from pathlib import Path
3322
3323 import matplotlib.dates as mdates
3324 import matplotlib.pyplot as plt
3325 from matplotlib import font_manager

```

```

3326 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
3327 from matplotlib.ticker import FuncFormatter
3328 import numpy as np
3329 import pandas as pd
3330
3331 ROOT = Path(__file__).resolve().parents[1]
3332 if str(ROOT) not in sys.path:
3333     sys.path.insert(0, str(ROOT))
3334
3335 K8_DAILY = ROOT / "output/q2_full_k8_risk/daily_summary.csv"
3336 K8_DISPATCH_DIR = ROOT / "output/q2_full_k8_risk/dispatch_daily"
3337 RISK_CALIBRATION = ROOT / "output/q2_full_k8_risk/risk_calibration.csv"
3338 DYNAMIC_DAILY = ROOT / "output/q2_full_linked/daily_summary.csv"
3339 RESULT2 = ROOT / "output/result2.xlsx"
3340 FIG_DIR = ROOT / "fig/q2_final"
3341
3342 START = pd.Timestamp("2025-02-01")
3343 END = pd.Timestamp("2025-12-31")
3344 EXPECTED_K8_COST = 14_456_670.55
3345 EXPECTED_K8_EMERGENCY_KWH = 534_490.42
3346 EXPECTED_COST_CHANGE = -50_168.90
3347 EXPECTED_EMERGENCY_CHANGE_KWH = -120_809.11
3348
3349 BLUE = "#276BBF"
3350 RED = "#C0392B"
3351 GRAY = "#6E7280"
3352 BLACK = "#1F2933"
3353 LIGHT_BLUE = "#E8F1FA"
3354 LIGHT_RED = "#F8E7E5"
3355 LIGHT_GRAY = "#F3F4F6"
3356
3357
3358 def file_hash(path: Path) -> str:
3359     digest = hashlib.sha256()
3360     with path.open("rb") as stream:
3361         for block in iter(lambda: stream.read(1024 * 1024), b""):
3362             digest.update(block)
3363     return digest.hexdigest()
3364
3365
3366 def configure_style() -> str:
3367     available = {font.name for font in font_manager.fontManager.ttflist}
3368     candidates = (
3369         "PingFang SC",
3370         "Hiragino Sans GB",
3371         "Heiti SC",
3372         "Songti SC",
3373         "Arial Unicode MS",
3374         "Noto Sans CJK SC",
3375     )
3376     font = next((name for name in candidates if name in available), None)
3377     if font is None:
3378         raise RuntimeError("No usable Chinese font found")
3379     plt.rcParams.update(
3380         {
3381             "font.family": "sans-serif",
3382             "font.sans-serif": [font],
3383             "axes.unicode_minus": False,
3384             "figure.facecolor": "white",
3385             "axes.facecolor": "white",
3386             "font.size": 10,
3387             "axes.titlesize": 12,
3388             "axes.labelsize": 10,
3389             "legend.fontsize": 9,
3390             "axes.titleweight": "medium",
3391             "pdf.fonttype": 42,
3392         }
3393     )

```

```

3394     return font
3395
3396
3397 def save_figure(fig: plt.Figure, stem: str) -> None:
3398     FIG_DIR.mkdir(parents=True, exist_ok=True)
3399     fig.savefig(FIG_DIR / f"{stem}.pdf", bbox_inches="tight")
3400     fig.savefig(FIG_DIR / f"{stem}.png", dpi=320, bbox_inches="tight")
3401     plt.close(fig)
3402
3403
3404 def draw_box(
3405     ax: plt.Axes,
3406     x: float,
3407     y: float,
3408     width: float,
3409     height: float,
3410     text: str,
3411     facecolor: str,
3412 ) -> None:
3413     patch = FancyBboxPatch(
3414         (x, y),
3415         width,
3416         height,
3417         boxstyle="round,pad=0.012,rounding_size=0.02",
3418         linewidth=1.15,
3419         edgecolor=BLACK,
3420         facecolor=facecolor,
3421     )
3422     ax.add_patch(patch)
3423     ax.text(x + width / 2, y + height / 2, text, ha="center", va="center", linespacing=1.38)
3424
3425
3426 def style_axes(ax: plt.Axes, hide_right: bool = True) -> None:
3427     ax.spines["top"].set_visible(False)
3428     if hide_right:
3429         ax.spines["right"].set_visible(False)
3430     for side in ("left", "bottom"):
3431         ax.spines[side].set_color(GRAY)
3432         ax.spines[side].set_linewidth(0.8)
3433     ax.tick_params(color=GRAY, length=3.5)
3434
3435
3436 def draw_arrow(
3437     ax: plt.Axes,
3438     start: tuple[float, float],
3439     end: tuple[float, float],
3440     label: str,
3441     label_y: float,
3442 ) -> None:
3443     ax.add_patch(
3444         FancyArrowPatch(
3445             start,
3446             end,
3447             arrowstyle="->",
3448             mutation_scale=12,
3449             linewidth=1.25,
3450             color=BLACK,
3451             shrinkA=2,
3452             shrinkB=2,
3453         )
3454     )
3455     ax.text(
3456         (start[0] + end[0]) / 2,
3457         label_y,
3458         label,
3459         ha="center",
3460         va="bottom",
3461         fontsize=8.8,

```

```

3462         color=BLACK,
3463         bbox={ "boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.95},
3464     )
3465
3466
3467 def plot_f1() -> None:
3468     fig, ax = plt.subplots(figsize=(15.6, 6.5))
3469     ax.set_xlim(0, 1)
3470     ax.set_ylim(0, 1)
3471     ax.axis("off")
3472     y, width, height = 0.48, 0.148, 0.30
3473     gap = 0.055
3474     xs = [0.020 + index * (width + gap) for index in range(5)]
3475     texts = [
3476         "历史负荷—光伏\n配对残差\n( 仅已结束日 )",
3477         "日前预测\n8 个联合情景\n风险分位购电下限",
3478         "0:00 锁定\n计划购电量\n$q_{d,t}$",
3479         "每 10 分钟观测实际值\n更新 $x,c,d,e$\n仅执行当前一步",
3480         "实际日末 SOC\n$E_{d,144}$\n传至下一日",
3481     ]
3482     faces = [LIGHT_GRAY, LIGHT_BLUE, LIGHT_BLUE, LIGHT_RED, LIGHT_GRAY]
3483     for x, text, face in zip(xs, texts, faces):
3484         draw_box(ax, x, y, width, height, text, face)
3485     labels = [
3486         "构造因果预测",
3487         "0:00 决策",
3488         "当天 $q$ 不可修改",
3489         "仅传递实际 SOC",
3490     ]
3491     for index in range(4):
3492         draw_arrow(
3493             ax,
3494             (xs[index] + width, y + height / 2),
3495             (xs[index + 1], y + height / 2),
3496             labels[index],
3497             y + height + 0.035,
3498         )
3499     mid_y = 0.30
3500     ax.plot(
3501         [xs[4] + width / 2, xs[4] + width / 2, xs[1] + width / 2, xs[1] + width / 2],
3502         [y, mid_y, mid_y, y],
3503         color=BLUE,
3504         linestyle="--",
3505         linewidth=1.25,
3506         solid_capstyle="round",
3507     )
3508     ax.annotate(
3509         "",
3510         xy=(xs[1] + width / 2, y),
3511         xytext=(xs[1] + width / 2, mid_y + 0.01),
3512         arrowprops={"arrowstyle": "-|>", "color": BLUE, "lw": 1.25},
3513     )
3514     ax.text(
3515         (xs[1] + xs[4] + width) / 2,
3516         mid_y - 0.055,
3517         "次日 0:00 重新预测并锁定自己的购电计划; 不提前锁定次日 $q$",
3518         ha="center",
3519         color=BLUE,
3520         fontsize=9.2,
3521     )
3522     ax.text(
3523         0.5,
3524         0.09,
3525         "日内仅修正储能动作与实际取用; 紧急购电 $e$ 只补当期缺口",
3526         ha="center",
3527         color=RED,
3528         fontsize=9.2,
3529         bbox={

```



```

3530         "boxstyle": "round,pad=0.32",
3531         "facecolor": LIGHT_RED,
3532         "edgecolor": RED,
3533         "linewidth": 0.6,
3534     },
3535 )
3536 ax.set_title("Q2 日前计划—日内执行—跨日 SOC 闭环机制", pad=10)
3537 save_figure(fig, "fig_q2_closed_loop")
3538
3539
3540 def output_interval(frame: pd.DataFrame) -> pd.DataFrame:
3541     dates = pd.to_datetime(frame["date"])
3542     selected = frame.loc[dates.between(START, END)].copy()
3543     if len(selected) != 334:
3544         raise AssertionError(f"Expected 334 output days, got {len(selected)}")
3545     return selected
3546
3547
3548 def plot_f2(k8: pd.DataFrame, dynamic: pd.DataFrame) -> dict:
3549     frames = [output_interval(dynamic), output_interval(k8)]
3550     names = ["原动态 K", "固定 K=8 + 风险分位"]
3551     planned = np.array([frame["planned_cost_yuan"].sum() for frame in frames])
3552     emergency_cost = np.array([frame["emergency_cost_yuan"].sum() for frame in frames])
3553     total = np.array([frame["total_cost_yuan"].sum() for frame in frames])
3554     emergency_kwh = np.array([frame["emergency_kwh"].sum() for frame in frames])
3555     if not np.allclose(total, planned + emergency_cost, atol=0.01):
3556         raise AssertionError("F2 cost components do not sum to total")
3557     cost_change = float(total[1] - total[0])
3558     emergency_change = float(emergency_kwh[1] - emergency_kwh[0])
3559     checks = (
3560         abs(total[1] - EXPECTED_K8_COST) < 0.01,
3561         abs(emergency_kwh[1] - EXPECTED_K8_EMERGENCY_KWH) < 0.01,
3562         abs(cost_change - EXPECTED_COST_CHANGE) < 0.01,
3563         abs(emergency_change - EXPECTED_EMERGENCY_CHANGE_KWH) < 0.01,
3564     )
3565     if not all(checks):
3566         raise AssertionError(
3567             f"F2 signed-off values mismatch: {total[1]}, {emergency_kwh[1]}, "
3568             f"{cost_change}, {emergency_change}"
3569         )
3570
3571     fig, ax = plt.subplots(figsize=(9.6, 6.9))
3572     x = np.arange(2)
3573     bar_width = 0.48
3574     ax.bar(
3575         x,
3576         planned,
3577         width=bar_width,
3578         color=BLUE,
3579         edgecolor=BLACK,
3580         linewidth=0.7,
3581         label="计划购电费",
3582         zorder=3,
3583     )
3584     ax.bar(
3585         x,
3586         emergency_cost,
3587         width=bar_width,
3588         bottom=planned,
3589         color="white",
3590         edgecolor=RED,
3591         linewidth=1.15,
3592         hatch="///",
3593         label="紧急购电费",
3594         zorder=3,
3595     )
3596     for index, (plan, emergency, value) in enumerate(zip(planned, emergency_cost, total)):
3597         ax.text(

```

```

3598         x[index],
3599         plan * 0.52,
3600         f"{plan / 1e6:.2f} 百万",
3601         ha="center",
3602         va="center",
3603         color="white",
3604         fontsize=9.5,
3605         zorder=4,
3606     )
3607     ax.text(
3608         x[index],
3609         plan + emergency * 0.55,
3610         f"{emergency / 1e6:.2f} 百万",
3611         ha="center",
3612         va="center",
3613         color=RED,
3614         fontsize=8.6,
3615         zorder=4,
3616         bbox={
3617             "boxstyle": "round,pad=0.12",
3618             "facecolor": "white",
3619             "edgecolor": "none",
3620             "alpha": 0.9,
3621         },
3622     )
3623     ax.text(
3624         x[index],
3625         value + total.max() * 0.018,
3626         f"合计 {value:,.2f} 元",
3627         ha="center",
3628         va="bottom",
3629         fontsize=9.6,
3630         zorder=4,
3631         clip_on=False,
3632     )
3633     ax.set_xticks(x, names)
3634     ax.set_ylabel("2—12 月购电成本 (元)")
3635     ax.set_title("Q2 两套日前风险处理策略的成本构成对照", pad=18)
3636     ax.yaxis.set_major_formatter(FuncFormatter(lambda value, _pos: f"{value / 1e6:.1f} 百万"))
3637     ax.set_ylim(0, total.max() * 1.12)
3638     ax.set_xlim(-0.55, 1.55)
3639     ax.grid(axis="y", linestyle=":", color=GRAY, alpha=0.35, zorder=0)
3640     ax.set_axisbelow(True)
3641     style_axes(ax)
3642     ax.legend(
3643         loc="lower center",
3644         bbox_to_anchor=(0.5, 1.01),
3645         ncol=2,
3646         frameon=False,
3647         borderaxespad=0.0,
3648     )
3649     fig.subplots_adjust(bottom=0.22, top=0.86)
3650     fig.text(
3651         0.5,
3652         0.085,
3653         f"K=8 相对原动态 K: 总成本变化 {cost_change:,.2f} 元; ",
3654         f"紧急购电量变化 {emergency_change:,.2f} kWh",
3655         ha="center",
3656         fontsize=9.5,
3657         bbox={"boxstyle": "round,pad=0.38", "facecolor": LIGHT_GRAY, "edgecolor": GRAY},
3658     )
3659     fig.text(
3660         0.5,
3661         0.02,
3662         "注: 仅比较两套日前风险处理策略; 不表示储能净收益或投资成本节省。",
3663         ha="center",
3664         fontsize=9,
3665         color=GRAY,

```

```

3666     )
3667     save_figure(fig, "fig_q2_cost_comparison")
3668     return {
3669         "k8_planned_cost_yuan": float(planned[1]),
3670         "k8_emergency_cost_yuan": float(emergency_cost[1]),
3671         "k8_total_cost_yuan": float(total[1]),
3672         "k8_emergency_kwh": float(emergency_kwh[1]),
3673         "dynamic_total_cost_yuan": float(total[0]),
3674         "dynamic_emergency_kwh": float(emergency_kwh[0]),
3675         "cost_change_yuan": cost_change,
3676         "emergency_change_kwh": emergency_change,
3677     }
3678
3679
3680 def recover_tariff(k8: pd.DataFrame) -> tuple[np.ndarray, dict]:
3681     """Recover the fixed 144-slot tariff from signed-off accounting identities."""
3682     matrix_rows: list[np.ndarray] = []
3683     targets: list[float] = []
3684     for row in k8.itertuples():
3685         dispatch = pd.read_csv(K8_DISPATCH_DIR / f"dispatch_{row.date}.csv")
3686         matrix_rows.extend(
3687             [
3688                 dispatch["planned_q_kwh"].to_numpy(float),
3689                 dispatch["emergency_kwh"].to_numpy(float),
3690             ]
3691         )
3692         targets.extend([float(row.planned_cost_yuan), float(row.emergency_cost_yuan) / 5.0])
3693     matrix = np.asarray(matrix_rows)
3694     target = np.asarray(targets)
3695     price, _residuals, rank, _singular = np.linalg.lstsq(matrix, target, rcond=None)
3696     max_residual = float(np.max(np.abs(matrix @ price - target)))
3697     if rank != 144 or max_residual >= 0.01 or not np.isfinite(price).all():
3698         raise AssertionError(
3699             f"Unable to recover 144-slot tariff: rank={rank}, residual={max_residual}"
3700         )
3701     return price, {
3702         "method": (
3703             "least-squares recovery from planned_cost=p@q and "
3704             "emergency_cost=5*p@e over 365 signed-off days"
3705         ),
3706         "matrix_rows": len(matrix),
3707         "rank": int(rank),
3708         "max_accounting_residual_yuan": max_residual,
3709         "price_min_yuan_per_kwh": float(price.min()),
3710         "price_max_yuan_per_kwh": float(price.max()),
3711     }
3712
3713
3714 def plot_f3(k8: pd.DataFrame, price: np.ndarray) -> dict:
3715     output = output_interval(k8)
3716     selected = output.loc[output["emergency_cost_yuan"].idxmax()]
3717     date = pd.Timestamp(selected["date"])
3718     path = K8_DISPATCH_DIR / f"dispatch_{date:%Y-%m-%d}.csv"
3719     frame = pd.read_csv(path)
3720     required = {
3721         "period",
3722         "time",
3723         "planned_q_kwh",
3724         "actual_x_kwh",
3725         "load_kwh",
3726         "pv_kwh",
3727         "charge_kwh",
3728         "discharge_kwh",
3729         "emergency_kwh",
3730         "soc_kwh",
3731     }
3732     if len(frame) != 144 or not required.issubset(frame.columns):
3733         raise AssertionError("F3 dispatch schema/row count mismatch")

```

```

3734 if not np.array_equal(frame["period"].to_numpy(), np.arange(144)):
3735     raise AssertionError("F3 dispatch period order changed")
3736 if str(frame["time"].iloc[0]) != "00:10" or str(frame["time"].iloc[-1]) != "0:00+1":
3737     raise AssertionError("F3 raw attachment time order mismatch")
3738
3739 hours = (frame["period"].to_numpy() + 1) / 6.0
3740 net_load = frame["load_kwh"].to_numpy() - frame["pv_kwh"].to_numpy()
3741 battery_net = frame["discharge_kwh"].to_numpy() - frame["charge_kwh"].to_numpy()
3742 if len(price) != 144:
3743     raise AssertionError("F3 recovered tariff must contain 144 raw-order values")
3744
3745 fig, axes = plt.subplots(
3746     3, 1, figsize=(12.6, 11.0), sharex=True, gridspec_kw={"hspace": 0.16}
3747 )
3748 legend_box = {
3749     "frameon": True,
3750     "framealpha": 0.96,
3751     "edgecolor": "#E5E7EB",
3752     "fancybox": False,
3753     "borderpad": 0.45,
3754     "handlelength": 2.2,
3755 }
3756
3757 ax = axes[0]
3758 ax.plot(hours, net_load, color=BLACK, linewidth=1.35, label="实际净负荷 $L-P$")
3759 ax.plot(hours, frame["planned_q_kwh"], color=BLUE, linewidth=1.25, label="锁定计划 $q$")
3760 ax.plot(
3761     hours,
3762     frame["actual_x_kwh"],
3763     color=GRAY,
3764     linestyle="--",
3765     linewidth=1.15,
3766     label="实际普通取电 $x$",
3767 )
3768 ax.plot(
3769     hours,
3770     frame["emergency_kwh"],
3771     color=RED,
3772     linestyle="-.",
3773     linewidth=1.45,
3774     label="紧急购电 $e$",
3775 )
3776 ax.fill_between(hours, 0, frame["emergency_kwh"], color=RED, alpha=0.12, linewidth=0)
3777 ymax = max(net_load.max(), frame["planned_q_kwh"].max(), frame["emergency_kwh"].max())
3778 ax.set_ylim(-0.10 * ymax, ymax * 1.22)
3779 ax.set_ylabel("时段电量 (kWh) ")
3780 ax.legend(loc="upper left", ncol=2, **legend_box)
3781 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
3782 ax.set_axisbelow(True)
3783 style_axes(ax)
3784
3785 ax = axes[1]
3786 ax.axhline(0, color=GRAY, linewidth=0.8)
3787 ax.plot(hours, battery_net, color=BLACK, linewidth=1.2, label="电池净动作 $d-c$")
3788 ax.fill_between(hours, 0, battery_net, color=BLACK, alpha=0.10, linewidth=0)
3789 ax.set_ylabel("$d-c$ (kWh) ")
3790 abs_max = np.max(np.abs(battery_net))
3791 ax.set_ylim(-abs_max * 1.28, abs_max * 1.28)
3792 ax_price = ax.twinx()
3793 ax_price.step(
3794     hours,
3795     price,
3796     where="mid",
3797     color=RED,
3798     linestyle="--",
3799     linewidth=1.15,
3800     label="分时电价",
3801 )

```

```

3802 ax_price.set_ylabel("电价 (元/kWh)", color=RED)
3803 ax_price.tick_params(axis="y", colors=RED)
3804 ax_price.set_ylim(0.28, price.max() * 1.22)
3805 ax_price.spines["top"].set_visible(False)
3806 ax_price.spines["right"].set_color(RED)
3807 handles1, labels1 = ax.get_legend_handles_labels()
3808 handles2, labels2 = ax_price.get_legend_handles_labels()
3809 ax.legend(
3810     handles1 + handles2,
3811     labels1 + labels2,
3812     loc="lower left",
3813     **legend_box,
3814 )
3815 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
3816 ax.set_axisbelow(True)
3817 style_axes(ax, hide_right=False)
3818
3819 ax = axes[2]
3820 ax.plot(hours, frame["soc_kwh"], color=BLUE, linewidth=1.5, label="SOC")
3821 ax.axhline(1200, color=GRAY, linestyle="--", linewidth=1.0, label="SOC 边界 1200 / 10800 kWh")
3822 ax.axhline(10800, color=GRAY, linestyle="--", linewidth=1.0)
3823 ax.set_ylabel("SOC (kWh)")
3824 ax.set_xlabel("时刻")
3825 ax.set_ylim(200, 12800)
3826 ax.legend(loc="center left", **legend_box)
3827 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
3828 ax.set_axisbelow(True)
3829 style_axes(ax)
3830 axes[-1].set_xlim(0, 24)
3831 axes[-1].set_xticks(np.arange(0, 25, 2))
3832 axes[-1].set_xticklabels([f"{hour:02d}:00" for hour in range(0, 25, 2)])
3833 fig.suptitle(
3834     f"Q2 日 前一日内联动轨迹: {date:%Y-%m-%d} (紧急费用最高日)",
3835     y=0.98,
3836 )
3837 fig.subplots_adjust(top=0.945, bottom=0.055, left=0.08, right=0.91, hspace=0.20)
3838 save_figure(fig, "fig_q2_representative_day")
3839 return {
3840     "selected_date": date.strftime("%Y-%m-%d"),
3841     "selection_metric": "maximum emergency_cost_yuan over 2025-02-01..2025-12-31",
3842     "emergency_cost_yuan": float(selected["emergency_cost_yuan"]),
3843     "emergency_kwh": float(selected["emergency_kwh"]),
3844     "dispatch_rows": len(frame),
3845     "first_time": str(frame["time"].iloc[0]),
3846     "last_time": str(frame["time"].iloc[-1]),
3847     "source": str(path.relative_to(ROOT)),
3848 }
3849
3850
3851 def plot_f4(calibration: pd.DataFrame) -> dict:
3852     if "selected" not in calibration:
3853         raise AssertionError("F4 calibration file lacks selected flag")
3854     selected = calibration.loc[calibration["selected"].astype(bool)].copy()
3855     selected["calibration_date"] = pd.to_datetime(selected["calibration_date"])
3856     selected["effective_start_date"] = pd.to_datetime(selected["effective_start_date"])
3857     selected["effective_end_date"] = pd.to_datetime(selected["effective_end_date"])
3858     selected = selected.sort_values("effective_start_date").reset_index(drop=True)
3859     if len(selected) != 27:
3860         raise AssertionError(f"F4 expected 27 selected blocks, got {len(selected)}")
3861     if selected["effective_start_date"].iloc[0] != pd.Timestamp("2025-01-01"):
3862         raise AssertionError("F4 calibration calendar start mismatch")
3863     if selected["effective_end_date"].iloc[-1] != pd.Timestamp("2025-12-31"):
3864         raise AssertionError("F4 calibration calendar end mismatch")
3865     for previous, current in zip(selected.itertuples(), selected.iloc[1:].itertuples()):
3866         if previous.effective_end_date + pd.Timedelta(days=1) != current.effective_start_date:
3867             raise AssertionError("F4 calibration blocks are not contiguous")
3868     early = selected["effective_start_date"] < pd.Timestamp("2025-01-29")
3869     if not (

```

```

3870         (selected.loc[early, "fixed_k"] == 1).all()
3871         and selected.loc[early, "risk_alpha"].isna().all()
3872         and (selected.loc[~early, "fixed_k"] == 8).all()
3873     ):
3874         raise AssertionError("F4 K=1/K=8 split mismatch")
3875     candidates = calibration.loc[calibration["fixed_k"] == 8].groupby("calibration_date")[
3876         "risk_alpha"
3877     ].apply(lambda values: set(np.round(values.dropna(), 2)))
3878     if not all(values == {0.6, 0.7, 0.8, 0.9} for values in candidates):
3879         raise AssertionError("F4 alpha candidate set mismatch")
3880     causal = selected.loc[selected["history_cutoff_date"] != "none"].apply(
3881         lambda row: pd.Timestamp(row["history_cutoff_date"]) < row["calibration_date"],
3882         axis=1,
3883     )
3884     if not causal.all():
3885         raise AssertionError("F4 history cutoff is not causal")
3886
3887     fig, ax = plt.subplots(figsize=(13.4, 6.2))
3888     y_map = {None: 0, 0.6: 1, 0.7: 2, 0.8: 3, 0.9: 4}
3889     ytick_labels = ["K=1 / 无风险储备", "alpha = 0.60", "alpha = 0.70", "alpha = 0.80", "alpha = 0.90"]
3890     year_start = mdates.date2num(pd.Timestamp("2025-01-01").to_pydatetime())
3891     year_end = mdates.date2num(pd.Timestamp("2026-01-01").to_pydatetime())
3892     for y in range(5):
3893         ax.barh(
3894             y,
3895             year_end - year_start,
3896             left=year_start,
3897             height=0.62,
3898             color=LIGHT_GRAY,
3899             edgecolor="none",
3900             zorder=0,
3901         )
3902     ax.axvspan(
3903         pd.Timestamp("2025-01-01"),
3904         pd.Timestamp("2025-01-29"),
3905         color=GRAY,
3906         alpha=0.10,
3907         zorder=1,
3908     )
3909     for row in selected.itertuples():
3910         start = mdates.date2num(pd.Timestamp(row.effective_start_date).to_pydatetime())
3911         end = mdates.date2num(
3912             (pd.Timestamp(row.effective_end_date) + pd.Timedelta(days=1)).to_pydatetime()
3913         )
3914         duration = int((row.effective_end_date - row.effective_start_date).days + 1)
3915         if row.fixed_k == 1:
3916             y = y_map[None]
3917             ax.barh(
3918                 y,
3919                 end - start,
3920                 left=start,
3921                 height=0.62,
3922                 color="white",
3923                 edgecolor=BLACK,
3924                 linewidth=0.8,
3925                 hatch="///",
3926                 zorder=3,
3927             )
3928         else:
3929             alpha = round(float(row.risk_alpha), 2)
3930             y = y_map[alpha]
3931             ax.barh(
3932                 y,
3933                 end - start,
3934                 left=start,
3935                 height=0.62,
3936                 color=BLUE,
3937                 edgecolor=BLUE,

```

```

3938         linewidth=0.8,
3939         zorder=3,
3940     )
3941     if alpha == 0.7:
3942         ax.text(
3943             (start + end) / 2,
3944             y,
3945             "alpha=0.70",
3946             ha="center",
3947             va="center",
3948             fontsize=8.2,
3949             color="white",
3950             zorder=4,
3951         )
3952     elif duration == 1:
3953         ax.text(
3954             start - 1.2,
3955             y,
3956             "12/31",
3957             ha="right",
3958             va="center",
3959             fontsize=8.2,
3960             color=BLACK,
3961             zorder=5,
3962             bbox={
3963                 "boxstyle": "round,pad=0.12",
3964                 "facecolor": "white",
3965                 "edgecolor": "none",
3966                 "alpha": 0.92,
3967             },
3968         )
3969     ax.text(
3970         mdates.date2num(pd.Timestamp("2025-01-15").to_pydatetime()),
3971         0,
3972         "1/1-1/28",
3973         ha="center",
3974         va="center",
3975         fontsize=8.2,
3976         color=BLACK,
3977         zorder=4,
3978         bbox={
3979             "boxstyle": "round,pad=0.12",
3980             "facecolor": "white",
3981             "edgecolor": "none",
3982             "alpha": 0.88,
3983         },
3984     )
3985     for unused_y in (3, 4):
3986         ax.text(
3987             mdates.date2num(pd.Timestamp("2025-07-01").to_pydatetime()),
3988             unused_y,
3989             "候选未选用",
3990             ha="center",
3991             va="center",
3992             fontsize=8.4,
3993             color=GRAY,
3994             zorder=2,
3995         )
3996     ax.set_yticks(list(range(5)), ytick_labels)
3997     ax.set_ylim(-0.55, 4.75)
3998     ax.set_ylabel("政策档位")
3999     ax.set_xlabel("风险分位生效区间 (每 14 日一块)")
4000     ax.set_title("Q2 滚动风险分位的因果校准时间线", pad=8)
4001     ax.xaxis.set_major_locator(mdates.MonthLocator())
4002     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
4003     ax.set_xlim(pd.Timestamp("2025-01-01"), pd.Timestamp("2026-01-04"))
4004     ax.grid(axis="x", linestyle=":", color=GRAY, alpha=0.32, zorder=0)
4005     style_axes(ax)

```

```

4006 fig.subplots_adjust(bottom=0.24)
4007 fig.autofmt_xdate(rotation=30, ha="right")
4008 fig.text(
4009     0.5,
4010     0.04,
4011     "K=8 阶段候选集 alphas{0.60, 0.70, 0.80, 0.90}; 每次仅使用此前已结束日的结果选择。"
4012     "12/17-12/30 为 alpha=0.70, 12/31 回落至 alpha=0.60。",
4013     ha="center",
4014     fontsize=9,
4015     bbox={"boxstyle": "round,pad=0.35", "facecolor": LIGHT_GRAY, "edgecolor": GRAY},
4016 )
4017 save_figure(fig, "fig_q2_risk_calibration")
4018 return {
4019     "selected_blocks": len(selected),
4020     "k1_no_reserve_blocks": int(early.sum()),
4021     "k8_blocks": int((~early).sum()),
4022     "alpha_060_days": int(
4023         sum(
4024             (row.effective_end_date - row.effective_start_date).days + 1
4025             for row in selected.itertuples()
4026             if row.risk_alpha == 0.6
4027         )
4028     ),
4029     "alpha_070_days": int(
4030         sum(
4031             (row.effective_end_date - row.effective_start_date).days + 1
4032             for row in selected.itertuples()
4033             if row.risk_alpha == 0.7
4034         )
4035     ),
4036     "calendar_start": selected["effective_start_date"].iloc[0].strftime("%Y-%m-%d"),
4037     "calendar_end": selected["effective_end_date"].iloc[-1].strftime("%Y-%m-%d"),
4038     "causal_history_cutoffs": bool(causal.all()),
4039 }
4040
4041
4042 def write_readme(
4043     font: str,
4044     f2: dict,
4045     f3: dict,
4046     f4: dict,
4047     tariff_audit: dict,
4048     hashes: dict[str, str],
4049 ) -> None:
4050     lines = [
4051         "# Q2 正式交付图",
4052         "",
4053         "运行命令:",
4054         "",
4055         "```bash",
4056         "MPLCONFIGDIR=\"$PWD/.mplconfig\" .venv/bin/python q2/plot_final_figures.py",
4057         "```",
4058         "",
4059         f"- 中文字体: {font}",
4060         "- 所有 PNG 均以 320 dpi 导出, 同时提供矢量 PDF。",
4061         "- 原始时段顺序保持 `00:10` 至 `0:00+1`, 未旋转或补造时段。",
4062         "",
4063         "## F1 `fig_q2_closed_loop`,
4064         "",
4065         "- 数据源: 模型机制与已签收固定口径; 图形由 matplotlib patches 矢量绘制。",
4066         "- 校验: 明确当天 q 锁定、日内可修正变量、紧急购电作用和实际 SOC 跨日传递。",
4067         "",
4068         "## F2 `fig_q2_cost_comparison`,
4069         "",
4070         "- 数据源: `output/q2_full_k8_risk/daily_summary.csv` 与 `",
4071         "`output/q2_full_linked/daily_summary.csv`。",
4072         "- 汇总范围: 2025-02-01 至 2025-12-31, 共 334 日。",
4073         f"- K=8: 计划费 {f2['k8_planned_cost_yuan']:, .2f} 元, 紧急费 "

```



```

4074     f"{f2['k8_emergency_cost_yuan']:, .2f} 元, 总成本 {f2['k8_total_cost_yuan']:, .2f} 元, "
4075     f"紧急购电 {f2['k8_emergency_kwh']:, .2f} kWh.",
4076     f"- 相对原动态 K: 总成本变化 {f2['cost_change_yuan']:, .2f} 元, "
4077     f"紧急购电量变化 {f2['emergency_change_kwh']:, .2f} kWh.",
4078     "- 核验: 与任务单签收值差异均小于 0.01 元 / 0.01 kWh.",
4079     "",
4080     "## F3 `fig_q2_representative_day`",
4081     "",
4082     "- 数据源: K=8 日汇总及对应逐时调度 CSV.",
4083     f"- 可复现选择: 2—12 月 `emergency_cost_yuan` 最大日为 {f3['selected_date']}.",
4084     f"- 源文件: `{f3['source']}`; 共 {f3['dispatch_rows']} 时段, "
4085     f"{f3['first_time']} 至 {f3['last_time']}.",
4086     "- 分时电价由允许读取的 365 日锁定计划、紧急购电和日成本按 "
4087     "`planned_cost=p0q`、`emergency_cost=5*p0e` 联立恢复.",
4088     f"- 电价恢复矩阵秩 {tariff_audit['rank']}; 最大成本回代残差 "
4089     f"{tariff_audit['max_accounting_residual_yuan']:.3e} 元.",
4090     "- 核验: 净负荷、q、x、e、d-c、价格、SOC 均直接读取或逐项确定性计算.",
4091     "",
4092     "## F4 `fig_q2_risk_calibration`",
4093     "",
4094     "- 数据源: `output/q2_full_k8_risk/risk_calibration.csv`. ",
4095     f"- 共 {f4['selected_blocks']} 个生效块: 前 {f4['k1_no_reserve_blocks']} 块为 "
4096     f"K=1 / 无风险储备, 随后 {f4['k8_blocks']} 块为 K=8.",
4097     f"- 日数核验: alpha=0.60 为 {f4['alpha_060_days']} 日, alpha=0.70 为 "
4098     f"{f4['alpha_070_days']} 日; 区间连续且历史截止日均早于校准日.",
4099     "",
4100     "## 只读保护",
4101     "",
4102     "- 绘图脚本对所有源 CSV 与 `output/result2.xlsx` 执行前后 SHA-256 校验.",
4103     f"- `output/result2.xlsx` SHA-256: "
4104     f"{hashes.get(str(RESULT2.relative_to(ROOT)), '文件不存在')}.".
4105 ]
4106 (FIG_DIR / "README.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
4107
4108
4109 def main() -> None:
4110     dispatch_paths = sorted(K8_DISPATCH_DIR.glob("dispatch_*.csv"))
4111     if len(dispatch_paths) != 365:
4112         raise AssertionError(f"Expected 365 K=8 dispatch files, got {len(dispatch_paths)}")
4113     source_paths = [K8_DAILY, DYNAMIC_DAILY, RISK_CALIBRATION, *dispatch_paths]
4114     if RESULT2.exists():
4115         source_paths.append(RESULT2)
4116     for path in source_paths:
4117         if not path.exists():
4118             raise FileNotFoundError(path)
4119     hashes_before = {
4120         str(path.relative_to(ROOT)): file_hash(path) for path in source_paths
4121     }
4122     font = configure_style()
4123     k8 = pd.read_csv(K8_DAILY)
4124     dynamic = pd.read_csv(DYNAMIC_DAILY)
4125     calibration = pd.read_csv(RISK_CALIBRATION)
4126     price, tariff_audit = recover_tariff(k8)
4127
4128     plot_f1()
4129     f2 = plot_f2(k8, dynamic)
4130     f3 = plot_f3(k8, price)
4131     f4 = plot_f4(calibration)
4132
4133     hashes_after = {
4134         str(path.relative_to(ROOT)): file_hash(path) for path in source_paths
4135     }
4136     if hashes_before != hashes_after:
4137         raise AssertionError("A source result file changed during plotting")
4138     audit = {
4139         "status": "PASS",
4140         "font": font,
4141         "png_dpi": 320,

```

```

4142     "f2": f2,
4143     "f3": f3,
4144     "tariff_recovery": tariff_audit,
4145     "f4": f4,
4146     "source_sha256": hashes_after,
4147     "result2_unchanged": hashes_before.get(str(RESULT2.relative_to(ROOT)))
4148     == hashes_after.get(str(RESULT2.relative_to(ROOT))),
4149 }
4150 FIG_DIR.mkdir(parents=True, exist_ok=True)
4151 (FIG_DIR / "figure_audit.json").write_text(
4152     json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
4153 )
4154 write_readme(font, f2, f3, f4, tariff_audit, hashes_after)
4155 print(json.dumps(audit, ensure_ascii=False, indent=2))
4156
4157
4158 if __name__ == "__main__":
4159     main()
4160
4161
4162 # ===== q2/plot_q2_policy_consistent_r3.py =====
4163 #!/usr/bin/env python3
4164 """C2-R3 comparison figures versus the signed-off old K=8 Q2 path.
4165
4166 Writes only to fig/q2_policy_consistent/; never touches fig/q2_final/.
4167 """
4168
4169 from __future__ import annotations
4170
4171 import sys
4172 from pathlib import Path
4173
4174 import matplotlib.dates as mdates
4175 import matplotlib.pyplot as plt
4176 from matplotlib import font_manager
4177 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
4178 from matplotlib.ticker import FuncFormatter
4179 import numpy as np
4180 import pandas as pd
4181
4182 ROOT = Path(__file__).resolve().parents[1]
4183 if str(ROOT) not in sys.path:
4184     sys.path.insert(0, str(ROOT))
4185
4186 from q2.config import FIG_POLICY_CONSISTENT_DIR, OFFICIAL_OUTPUT_START, T
4187
4188
4189 BLUE = "#276FBF"
4190 RED = "#C0392B"
4191 GRAY = "#6B7280"
4192 BLACK = "#1F2933"
4193 GREEN = "#2F6F4E"
4194 LIGHT_GRAY = "#F3F4F6"
4195 LIGHT_BLUE = "#E8F1FA"
4196 LIGHT_RED = "#F8E7E5"
4197
4198
4199 def configure_style() -> str:
4200     available = {font.name for font in font_manager.fontManager.ttflist}
4201     candidates = (
4202         "PingFang SC",
4203         "Hiragino Sans GB",
4204         "Heiti SC",
4205         "Songti SC",
4206         "Arial Unicode MS",
4207         "Noto Sans CJK SC",
4208     )
4209     font = next((name for name in candidates if name in available), None)

```

```

4210     if font is None:
4211         raise RuntimeError("No usable Chinese font found")
4212     plt.rcParams.update(
4213         {
4214             "font.family": "sans-serif",
4215             "font.sans-serif": [font],
4216             "axes.unicode_minus": False,
4217             "figure.facecolor": "white",
4218             "axes.facecolor": "white",
4219             "font.size": 10,
4220             "axes.titlesize": 12,
4221             "axes.labelsize": 10,
4222             "legend.fontsize": 9,
4223             "axes.titleweight": "medium",
4224             "pdf.fonttype": 42,
4225         }
4226     )
4227     return font
4228
4229
4230 def save_figure(fig: plt.Figure, stem: str, fig_dir: Path) -> None:
4231     fig_dir.mkdir(parents=True, exist_ok=True)
4232     fig.savefig(fig_dir / f"{stem}.pdf", bbox_inches="tight")
4233     fig.savefig(fig_dir / f"{stem}.png", dpi=320, bbox_inches="tight")
4234     plt.close(fig)
4235
4236
4237 def style_axes(ax: plt.Axes) -> None:
4238     ax.spines["top"].set_visible(False)
4239     ax.spines["right"].set_visible(False)
4240     for side in ("left", "bottom"):
4241         ax.spines[side].set_color(GRAY)
4242         ax.spines[side].set_linewidth(0.8)
4243     ax.tick_params(color=GRAY, length=3.5)
4244     ax.grid(axis="y", linestyle=":", color=GRAY, alpha=0.35, zorder=0)
4245     ax.set_axisbelow(True)
4246
4247
4248 def official_interval(frame: pd.DataFrame) -> pd.DataFrame:
4249     out = frame.copy()
4250     out["date"] = pd.to_datetime(out["date"])
4251     start = pd.Timestamp(OFFICIAL_OUTPUT_START)
4252     end = pd.Timestamp("2025-12-31")
4253     return (
4254         out.loc[(out["date"] >= start) & (out["date"] <= end)]
4255         .sort_values("date")
4256         .reset_index(drop=True)
4257     )
4258
4259
4260 def style_time_axis(ax: plt.Axes) -> None:
4261     ax.set_xlim(pd.Timestamp(OFFICIAL_OUTPUT_START), pd.Timestamp("2025-12-31"))
4262     ax.xaxis.set_major_locator(mdates.MonthLocator())
4263     ax.xaxis.set_major_formatter(mdates.DateFormatter("%m月"))
4264
4265
4266 def plot_r3_comparison(
4267     redesign: pd.DataFrame,
4268     old_k8: pd.DataFrame,
4269     fig_dir: Path | None = None,
4270 ) -> dict:
4271     configure_style()
4272     target = Path(fig_dir) if fig_dir is not None else FIG_POLICY_CONSISTENT_DIR
4273     new = official_interval(redesign)
4274     old = official_interval(old_k8)
4275     if len(new) != 334 or len(old) != 334:
4276         raise AssertionError("official comparison window must be 334 days")
4277     if not new["date"].equals(old["date"]):

```

```

4278         raise AssertionError("old and new official dates do not align")
4279
4280     names = ["旧全路径近似对照 (K=8) ", "政策一致主方案 (预注册 K=8) "]
4281     frames = [old, new]
4282     planned = np.array([frame["planned_cost_yuan"].sum() for frame in frames])
4283     emergency_cost = np.array([frame["emergency_cost_yuan"].sum() for frame in frames])
4284     total = planned + emergency_cost
4285     emergency_kwh = np.array([frame["emergency_kwh"].sum() for frame in frames])
4286     curtailment = np.array([frame["curtailment_kwh"].sum() for frame in frames])
4287     if not np.allclose(total, [frame["total_cost_yuan"].sum() for frame in frames], atol=0.05):
4288         raise AssertionError("cost components do not match total_cost_yuan")
4289
4290     fig, ax = plt.subplots(figsize=(9.6, 6.9))
4291     x = np.arange(2)
4292     ax.bar(x, planned, width=0.48, color=BLUE, edgecolor=BLACK, linewidth=0.7, label="计划购电费", zorder=3)
4293     ax.bar(
4294         x,
4295         emergency_cost,
4296         width=0.48,
4297         bottom=planned,
4298         color="white",
4299         edgecolor=RED,
4300         linewidth=1.15,
4301         hatch="///",
4302         label="紧急购电费",
4303         zorder=3,
4304     )
4305     for index, (plan, emergency, value) in enumerate(zip(planned, emergency_cost, total)):
4306         ax.text(
4307             x[index],
4308             plan * 0.52,
4309             f"{plan / 1e6:.2f} 百万",
4310             ha="center",
4311             va="center",
4312             color="white",
4313             fontsize=9.5,
4314             zorder=4,
4315         )
4316         ax.text(
4317             x[index],
4318             plan + emergency * 0.55,
4319             f"{emergency / 1e6:.2f} 百万",
4320             ha="center",
4321             va="center",
4322             color=RED,
4323             fontsize=8.6,
4324             zorder=4,
4325             bbox={"boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.9},
4326         )
4327         ax.text(
4328             x[index],
4329             value + total.max() * 0.018,
4330             f"合计 {value:.2f} 元",
4331             ha="center",
4332             va="bottom",
4333             fontsize=9.6,
4334             zorder=4,
4335             clip_on=False,
4336         )
4337     ax.set_xticks(x, names)
4338     ax.set_ylabel("2—12 月购电成本 (元) ")
4339     ax.set_title("政策一致主方案与旧全路径近似对照的成本构成", pad=18)
4340     ax.yaxis.set_major_formatter(FuncFormatter(lambda value, _pos: f"{value / 1e6:.1f} 百万"))
4341     ax.set_ylim(0, total.max() * 1.12)
4342     ax.set_xlim(-0.55, 1.55)
4343     style_axes(ax)
4344     ax.legend(loc="lower center", bbox_to_anchor=(0.5, 1.01), ncol=2, frameon=False, borderaxespad=0.0)
4345     fig.subplots_adjust(bottom=0.22, top=0.86)

```

```

4346 fig.text(
4347     0.5,
4348     0.085,
4349     f"政策一致主方案相对旧全路径对照：总成本变化 {total[1] - total[0]:.2f} 元；"
4350     f"紧急购电量变化 {emergency_kwh[1] - emergency_kwh[0]:.2f} kWh",
4351     ha="center",
4352     fontsize=9.5,
4353     bbox={"boxstyle": "round,pad=0.38", "facecolor": LIGHT_GRAY, "edgecolor": GRAY},
4354 )
4355 fig.text(
4356     0.5,
4357     0.02,
4358     "注：K=8 为预注册且对 K=4/12 稳定的简洁主方案，并非成本最优；"
4359     "旧全路径方案仅作参考信息更强的近似对照。1 月仅作参考。",
4360     ha="center",
4361     fontsize=9,
4362     color=GRAY,
4363 )
4364 save_figure(fig, "fig_q2_pc_cost_comparison", target)
4365
4366 fig, axes = plt.subplots(2, 1, figsize=(10.2, 7.4), sharex=True)
4367 series = [
4368     (axes[0], "emergency_kwh", "紧急购电量 (kWh)", RED, LIGHT_RED),
4369     (axes[1], "curtailment_kwh", "弃光量 (kWh)", GREEN, LIGHT_BLUE),
4370 ]
4371 for ax, column, ylabel, color, fill in series:
4372     ax.fill_between(old["date"], old[column], color=fill, alpha=0.9, zorder=1)
4373     ax.plot(old["date"], old[column], color=GRAY, linewidth=1.05, label="旧全路径对照", zorder=2)
4374     ax.plot(new["date"], new[column], color=color, linewidth=1.25, label="政策一致主方案", zorder=3)
4375     ax.set_ylabel(ylabel)
4376     style_axes(ax)
4377     ax.legend(frameon=False, loc="upper right")
4378 axes[0].set_title("2—12 月紧急购电与弃光逐日对照")
4379 style_time_axis(axes[1])
4380 fig.tight_layout()
4381 save_figure(fig, "fig_q2_pc_emergency_curtailment", target)
4382
4383 fig, ax = plt.subplots(figsize=(10.2, 5.4))
4384 ax.plot(old["date"], old["soc_end_kwh"], color=GRAY, linewidth=1.1, label="旧全路径对照")
4385 ax.plot(new["date"], new["soc_end_kwh"], color=BLUE, linewidth=1.3, label="政策一致主方案")
4386 ax.set_ylabel("日末 SOC (kWh)")
4387 ax.set_title("2—12 月储能日末 SOC 对照")
4388 style_time_axis(ax)
4389 style_axes(ax)
4390 ax.legend(frameon=False)
4391 fig.tight_layout()
4392 save_figure(fig, "fig_q2_pc_soc", target)
4393
4394 fig, ax = plt.subplots(figsize=(10.2, 5.4))
4395 ax.plot(
4396     old["date"],
4397     old["total_cost_yuan"].cumsum(),
4398     color=GRAY,
4399     linewidth=1.2,
4400     label="旧全路径对照",
4401 )
4402 ax.plot(
4403     new["date"],
4404     new["total_cost_yuan"].cumsum(),
4405     color=BLUE,
4406     linewidth=1.4,
4407     label="政策一致主方案",
4408 )
4409 ax.set_ylabel("累计购电成本 (元)")
4410 ax.set_title("2—12 月累计实际购电成本对照")
4411 ax.yaxis.set_major_formatter(FuncFormatter(lambda value, _pos: f"{value / 1e6:.1f} 百万"))
4412 style_time_axis(ax)
4413 style_axes(ax)

```

```

4414     ax.legend(frameon=False)
4415     fig.tight_layout()
4416     save_figure(fig, "fig_q2_pc_cumulative_cost", target)
4417
4418     return {
4419         "old_feb_dec_total_cost_yuan": float(total[0]),
4420         "new_feb_dec_total_cost_yuan": float(total[1]),
4421         "cost_change_yuan": float(total[1] - total[0]),
4422         "old_feb_dec_emergency_kwh": float(emergency_kwh[0]),
4423         "new_feb_dec_emergency_kwh": float(emergency_kwh[1]),
4424         "emergency_change_kwh": float(emergency_kwh[1] - emergency_kwh[0]),
4425         "old_feb_dec_curtailement_kwh": float(curtailement[0]),
4426         "new_feb_dec_curtailement_kwh": float(curtailement[1]),
4427         "figure_dir": str(target),
4428         "stems": [
4429             "fig_q2_pc_cost_comparison",
4430             "fig_q2_pc_emergency_curtailement",
4431             "fig_q2_pc_soc",
4432             "fig_q2_pc_cumulative_cost",
4433         ],
4434     }
4435
4436
4437 def _draw_box(ax, x, y, width, height, text, facecolor) -> None:
4438     ax.add_patch(
4439         FancyBboxPatch(
4440             (x, y),
4441             width,
4442             height,
4443             boxstyle="round,pad=0.012,rounding_size=0.02",
4444             linewidth=1.15,
4445             edgecolor=BLACK,
4446             facecolor=facecolor,
4447         )
4448     )
4449     ax.text(x + width / 2, y + height / 2, text, ha="center", va="center", linespacing=1.38)
4450
4451
4452 def _draw_arrow(ax, start, end, label, label_y) -> None:
4453     ax.add_patch(
4454         FancyArrowPatch(
4455             start,
4456             end,
4457             arrowstyle="->",
4458             mutation_scale=12,
4459             linewidth=1.25,
4460             color=BLACK,
4461             shrinkA=2,
4462             shrinkB=2,
4463         )
4464     )
4465     ax.text(
4466         (start[0] + end[0]) / 2,
4467         label_y,
4468         label,
4469         ha="center",
4470         va="bottom",
4471         fontsize=8.8,
4472         bbox={"boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.95},
4473     )
4474
4475
4476 def plot_pc_closed_loop(fig_dir: Path) -> dict:
4477     fig, ax = plt.subplots(figsize=(15.6, 6.5))
4478     ax.set_xlim(0, 1)
4479     ax.set_ylim(0, 1)
4480     ax.axis("off")
4481     y, width, height = 0.48, 0.148, 0.30

```

```

4482 gap = 0.055
4483 xs = [0.020 + index * (width + gap) for index in range(5)]
4484 texts = [
4485     "已结束日负荷—光伏\n构造  $m$  在  $\{m_1, m_2, m_3\}$ ",
4486     "基准预测日前 LP\n $\alpha$  风险储备",
4487     "0:00 锁定\n计划购电量  $q_{d,t}$ ",
4488     "每 10 分钟残差匹配\n更新  $x, c, d, e$  仅执行当前一步",
4489     "实际日末 SOC  $E_{d,144}$  传至下一日",
4490 ]
4491 faces = [LIGHT_GRAY, LIGHT_BLUE, LIGHT_BLUE, LIGHT_RED, LIGHT_GRAY]
4492 for x, text, face in zip(xs, texts, faces):
4493     _draw_box(ax, x, y, width, height, text, face)
4494 labels = [
4495     "冻结日历选用  $(m, \alpha)$ ",
4496     "0:00 决策",
4497     "当天  $q$  不可修改",
4498     "仅传递实际 SOC",
4499 ]
4500 for index in range(4):
4501     _draw_arrow(
4502         ax,
4503         (xs[index] + width, y + height / 2),
4504         (xs[index + 1], y + height / 2),
4505         labels[index],
4506         y + height + 0.035,
4507     )
4508 mid_y = 0.30
4509 ax.plot(
4510     [xs[4] + width / 2, xs[4] + width / 2, xs[1] + width / 2, xs[1] + width / 2],
4511     [y, mid_y, mid_y, y],
4512     color=BLUE,
4513     linestyle="--",
4514     linewidth=1.25,
4515     solid_capstyle="round",
4516 )
4517 ax.annotate(
4518     "",
4519     xy=(xs[1] + width / 2, y),
4520     xytext=(xs[1] + width / 2, mid_y + 0.01),
4521     arrowprops={"arrowstyle": "-|>", "color": BLUE, "lw": 1.25},
4522 )
4523 ax.text(
4524     (xs[1] + xs[4] + width) / 2,
4525     mid_y - 0.055,
4526     "次日价值割近似次日基准预测成本；不提前锁定次日  $q$ ，不计入真实账单",
4527     ha="center",
4528     color=BLUE,
4529     fontsize=9.2,
4530 )
4531 ax.text(
4532     0.5,
4533     0.09,
4534     "K=8 为预注册残差压缩规模，且对 K=4/12 稳定；并非成本最优。日内仅修正储能与实际取用。",
4535     ha="center",
4536     color=RED,
4537     fontsize=9.2,
4538     bbox={"boxstyle": "round,pad=0.32", "facecolor": LIGHT_RED, "edgecolor": RED, "linewidth": 0.6},
4539 )
4540 ax.set_title("Q2 政策一致：基准日前计划—残差 MPC—跨日 SOC 闭环", pad=10)
4541 save_figure(fig, "fig_q2_pc_closed_loop", fig_dir)
4542 return {"stem": "fig_q2_pc_closed_loop"}
4543
4544
4545 def plot_pc_representative_day(
4546     daily: pd.DataFrame,
4547     dispatch_dir: Path,
4548     price: np.ndarray,
4549     fig_dir: Path,

```

```

4550 ) -> dict:
4551     output = official_interval(daily)
4552     selected = output.loc[output["emergency_cost_yuan"].idxmax()]
4553     date = pd.Timestamp(selected["date"])
4554     path = dispatch_dir / f"dispatch_{date:%Y-%m-%d}.csv"
4555     frame = pd.read_csv(path)
4556     required = {
4557         "period",
4558         "time",
4559         "planned_q_kwh",
4560         "actual_x_kwh",
4561         "load_kwh",
4562         "pv_kwh",
4563         "charge_kwh",
4564         "discharge_kwh",
4565         "emergency_kwh",
4566         "soc_kwh",
4567     }
4568     if len(frame) != T or not required.issubset(frame.columns):
4569         raise AssertionError("representative-day dispatch schema/row count mismatch")
4570     if not np.array_equal(frame["period"].to_numpy(), np.arange(T)):
4571         raise AssertionError("representative-day period order changed")
4572     if str(frame["time"].iloc[0]) != "00:10" or str(frame["time"].iloc[-1]) != "0:00+1":
4573         raise AssertionError("representative-day raw attachment time order mismatch")
4574     if len(price) != T:
4575         raise AssertionError("representative-day tariff must contain 144 values")
4576
4577     hours = (frame["period"].to_numpy() + 1) / 6.0
4578     net_load = frame["load_kwh"].to_numpy() - frame["pv_kwh"].to_numpy()
4579     battery_net = frame["discharge_kwh"].to_numpy() - frame["charge_kwh"].to_numpy()
4580     fig, axes = plt.subplots(3, 1, figsize=(12.6, 11.0), sharex=True, gridspec_kw={"hspace": 0.16})
4581     legend_box = {
4582         "frameon": True,
4583         "framealpha": 0.96,
4584         "edgecolor": "#E5E7EB",
4585         "fancybox": False,
4586         "borderpad": 0.45,
4587         "handlelength": 2.2,
4588     }
4589     ax = axes[0]
4590     ax.plot(hours, net_load, color=BLACK, linewidth=1.35, label="实际净负荷 $L-P$")
4591     ax.plot(hours, frame["planned_q_kwh"], color=BLUE, linewidth=1.25, label="锁定计划 $q$")
4592     ax.plot(hours, frame["actual_x_kwh"], color=GRAY, linestyle="--", linewidth=1.15, label="实际普通取电 $x$")
4593     ax.plot(hours, frame["emergency_kwh"], color=RED, linestyle="-. ", linewidth=1.45, label="紧急购电 $e$")
4594     ax.fill_between(hours, 0, frame["emergency_kwh"], color=RED, alpha=0.12, linewidth=0)
4595     ymax = max(net_load.max(), frame["planned_q_kwh"].max(), frame["emergency_kwh"].max())
4596     ax.set_ylim(-0.10 * ymax, ymax * 1.22)
4597     ax.set_ylabel("时段电量 (kWh) ")
4598     ax.legend(loc="upper left", ncol=2, **legend_box)
4599     ax.grid(linestyle=":", color=GRAY, alpha=0.32)
4600     ax.set_axisbelow(True)
4601     style_axes(ax)
4602
4603     ax = axes[1]
4604     ax.axhline(0, color=GRAY, linewidth=0.8)
4605     ax.plot(hours, battery_net, color=BLACK, linewidth=1.2, label="电池净动作 $d-c$")
4606     ax.fill_between(hours, 0, battery_net, color=BLACK, alpha=0.10, linewidth=0)
4607     ax.set_ylabel("$d-c$ (kWh) ")
4608     abs_max = np.max(np.abs(battery_net))
4609     ax.set_ylim(-abs_max * 1.28, abs_max * 1.28)
4610     ax_price = ax.twinx()
4611     ax_price.step(hours, price, where="mid", color=RED, linestyle="--", linewidth=1.15, label="分时电价")
4612     ax_price.set_ylabel("电价 (元/kWh) ", color=RED)
4613     ax_price.tick_params(axis="y", colors=RED)
4614     ax_price.set_ylim(0.28, price.max() * 1.22)
4615     ax_price.spines["top"].set_visible(False)
4616     ax_price.spines["right"].set_color(RED)
4617     handles1, labels1 = ax.get_legend_handles_labels()

```



```

4618 handles2, labels2 = ax_price.get_legend_handles_labels()
4619 ax.legend(handles1 + handles2, labels1 + labels2, loc="lower left", **legend_box)
4620 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
4621 ax.set_axisbelow(True)
4622 ax.spines["top"].set_visible(False)
4623
4624 ax = axes[2]
4625 ax.plot(hours, frame["soc_kwh"], color=BLUE, linewidth=1.5, label="SOC")
4626 ax.axhline(1200, color=GRAY, linestyle="--", linewidth=1.0, label="SOC 边界 1200 / 10800 kWh")
4627 ax.axhline(10800, color=GRAY, linestyle="--", linewidth=1.0)
4628 ax.set_ylabel("SOC (kWh) ")
4629 ax.set_xlabel("时刻")
4630 ax.set_ylim(200, 12800)
4631 ax.legend(loc="center left", **legend_box)
4632 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
4633 ax.set_axisbelow(True)
4634 style_axes(ax)
4635 axes[-1].set_xlim(0, 24)
4636 axes[-1].set_xticks(np.arange(0, 25, 2))
4637 axes[-1].set_xticklabels([f"{hour:02d}:00" for hour in range(0, 25, 2)])
4638 fig.suptitle(f"Q2 政策一致轨迹: {date:%Y-%m-%d} (2—12 月紧急费用最高日)", y=0.98)
4639 fig.subplots_adjust(top=0.945, bottom=0.055, left=0.08, right=0.91, hspace=0.20)
4640 save_figure(fig, "fig_q2_pc_representative_day", fig_dir)
4641 return {
4642     "selected_date": date.strftime("%Y-%m-%d"),
4643     "selection_metric": "maximum emergency_cost_yuan over 2025-02-01..2025-12-31",
4644     "not_typical_day": True,
4645     "emergency_cost_yuan": float(selected["emergency_cost_yuan"]),
4646     "emergency_kwh": float(selected["emergency_kwh"]),
4647     "dispatch_rows": len(frame),
4648     "source": str(path),
4649 }
4650
4651
4652 def plot_pc_forecast_calendar(calibration: pd.DataFrame, fig_dir: Path) -> dict:
4653     if "selected" not in calibration:
4654         raise AssertionError("frozen calendar source lacks selected flag")
4655     selected = calibration.loc[calibration["selected"].astype(bool)].copy()
4656     selected["effective_start_date"] = pd.to_datetime(selected["effective_start_date"])
4657     selected["effective_end_date"] = pd.to_datetime(selected["effective_end_date"])
4658     selected = selected.sort_values("effective_start_date").reset_index(drop=True)
4659     if selected["effective_start_date"].iloc[0] != pd.Timestamp("2025-01-01"):
4660         raise AssertionError("frozen calendar start mismatch")
4661     if selected["effective_end_date"].iloc[-1] != pd.Timestamp("2025-12-31"):
4662         raise AssertionError("frozen calendar end mismatch")
4663     for previous, current in zip(selected.itertuples(), selected.iloc[1:].itertuples()):
4664         if previous.effective_end_date + pd.Timedelta(days=1) != current.effective_start_date:
4665             raise AssertionError("frozen calendar blocks are not contiguous")
4666
4667     def row_key(row) -> tuple:
4668         fallback = str(getattr(row, "fallback_reason", "")) or ""
4669         if fallback not in ("", "nan"):
4670             return ("fallback", None, None)
4671         alpha = None if pd.isna(row.risk_alpha) else round(float(row.risk_alpha), 2)
4672         return ("policy", str(row.forecast_mode), alpha)
4673
4674     keys: list[tuple] = []
4675     for row in selected.itertuples():
4676         key = row_key(row)
4677         if key not in keys:
4678             keys.append(key)
4679     keys = sorted(
4680         keys,
4681         key=lambda item: (0 if item[0] == "fallback" else 1, str(item[1] or ""), item[2] or 0.0),
4682     )
4683     y_map = {key: index for index, key in enumerate(keys)}
4684     labels = []
4685     for kind, mode, alpha in keys:

```

```

4686         if kind == "fallback":
4687             labels.append("回退 K=1 / 无风险储备")
4688         else:
4689             labels.append(f"{mode}, alpha={alpha:.2f}")
4690
4691     mode_color = {"m1": BLUE, "m2": GREEN, "m3": RED, None: GRAY}
4692     fig, ax = plt.subplots(figsize=(13.4, 6.4))
4693     year_start = mdates.date2num(pd.Timestamp("2025-01-01").to_pydatetime())
4694     year_end = mdates.date2num(pd.Timestamp("2026-01-01").to_pydatetime())
4695     for y in range(len(keys)):
4696         ax.barh(y, year_end - year_start, left=year_start, height=0.62, color=LIGHT_GRAY, edgecolor="none", zorder=0)
4697     ax.axvspan(pd.Timestamp("2025-01-01"), pd.Timestamp("2025-02-01"), color=GRAY, alpha=0.10, zorder=1)
4698     for row in selected.itertuples():
4699         start = mdates.date2num(pd.Timestamp(row.effective_start_date).to_pydatetime())
4700         end = mdates.date2num((pd.Timestamp(row.effective_end_date) + pd.Timedelta(days=1)).to_pydatetime())
4701         key = row_key(row)
4702         color = GRAY if key[0] == "fallback" else mode_color.get(key[1], BLUE)
4703         ax.barh(y_map[key], end - start, left=start, height=0.62, color=color, edgecolor=BLACK, linewidth=0.4, zorder=3)
4704     ax.set_yticks(range(len(keys)), labels)
4705     ax.set_xlim(pd.Timestamp("2025-01-01"), pd.Timestamp("2026-01-01"))
4706     ax.xaxis.set_major_locator(mdates.MonthLocator())
4707     ax.xaxis.set_major_formatter(mdates.DateFormatter("%m月"))
4708     ax.set_title("C2-R2 冻结的预测结构与风险分位日历 (正式主方案沿用)")
4709     ax.spines["top"].set_visible(False)
4710     ax.spines["right"].set_visible(False)
4711     fig.text(
4712         0.5,
4713         0.02,
4714         "1 月为预热; 2—12 月正式输出继承该日历。K=8 保持预注册, 不随块重选。",
4715         ha="center",
4716         fontsize=9,
4717         color=GRAY,
4718     )
4719     fig.subplots_adjust(bottom=0.12)
4720     save_figure(fig, "fig_q2_pc_forecast_calendar", fig_dir)
4721     return {
4722         "stem": "fig_q2_pc_forecast_calendar",
4723         "n_blocks": int(len(selected)),
4724         "n_rows": int(len(keys)),
4725     }
4726
4727
4728 def plot_r4_final_figures(
4729     redesign: pd.DataFrame,
4730     old_k8: pd.DataFrame,
4731     calibration: pd.DataFrame,
4732     dispatch_dir: Path,
4733     price: np.ndarray,
4734     fig_dir: Path | None = None,
4735 ) -> dict:
4736     configure_style()
4737     target = Path(fig_dir) if fig_dir is not None else FIG_POLICY_CONSISTENT_DIR
4738     comparison = plot_r3_comparison(redesign, old_k8, fig_dir=target)
4739     closed = plot_pc_closed_loop(target)
4740     representative = plot_pc_representative_day(redesign, dispatch_dir, price, target)
4741     calendar = plot_pc_forecast_calendar(calibration, target)
4742     stems = list(comparison["stems"]) + [
4743         closed["stem"],
4744         "fig_q2_pc_representative_day",
4745         calendar["stem"],
4746     ]
4747     return {
4748         **comparison,
4749         "closed_loop": closed,
4750         "representative_day": representative,
4751         "forecast_calendar": calendar,
4752         "stems": stems,
4753         "figure_dir": str(target),

```

```

4754     }
4755
4756
4757 if __name__ == "__main__":
4758     from q2.config import FULL_K8_RISK_OUTPUT_DIR, POLICY_CONSISTENT_OUTPUT_DIR
4759
4760     plot_r3_comparison(
4761         pd.read_csv(POLICY_CONSISTENT_OUTPUT_DIR / "q2_redesign_daily_summary.csv"),
4762         pd.read_csv(FULL_K8_RISK_OUTPUT_DIR / "daily_summary.csv"),
4763     )
4764
4765 # ===== q2/policy_consistent.py =====
4766 """Policy-consistent Q2 redesign: baseline forecast LP + K=8 reserve + residual MPC.
4767
4768 This module is independent of the signed-off stochastic day-ahead production path.
4769 K-medoids enter only through the risk reserve R(alpha) and intra-day residual weights.
4770 """
4771
4772 from __future__ import annotations
4773
4774 from concurrent.futures import ThreadPoolExecutor
4775 from dataclasses import dataclass
4776 from enum import Enum
4777 from math import sqrt
4778 import json
4779 import os
4780 from pathlib import Path
4781 from threading import Lock
4782
4783 import numpy as np
4784 import pandas as pd
4785
4786 from q2.config import (
4787     EMERGENCY_PRICE_MULTIPLIER,
4788     E_INITIAL_KWH,
4789     E_MAX_KWH,
4790     E_MIN_KWH,
4791     FIXED_SCENARIO_K,
4792     K_RECALIBRATION_DAYS,
4793     LOAD_HISTORY_SAME_WEEKDAY,
4794     NEXT_DAY_VALUE_GAP_TOL_YUAN,
4795     NEXT_DAY_VALUE_MAX_SAMPLES,
4796     NUMERIC_TOL,
4797     OFFICIAL_OUTPUT_START,
4798     POWER_LIMIT_KWH,
4799     PV_HISTORY_DAYS,
4800     RISK_ALPHA_CANDIDATES,
4801     RISK_CALIBRATION_DAYS,
4802     SIMULTANEOUS_CD_TOL,
4803     T,
4804 )
4805 from q2.data import Q2Data
4806 from q2.forecast import ForecastArchive
4807 from q2.optimization import (
4808     BaselinePlanResult,
4809     ValueCut,
4810     evaluate_value_cuts,
4811     solve_baseline_plan,
4812     solve_fixed_plan_dispatch,
4813 )
4814 from q2.pilot import planned_q_hash, risk_quantile_floor
4815 from q2.scenarios import (
4816     ScenarioSet,
4817     build_scenarios,
4818     posterior_weights,
4819 )
4820
4821

```

```

4822 class ForecastMode(str, Enum):
4823     M1 = "m1"
4824     M2 = "m2"
4825     M3 = "m3"
4826
4827
4828 FORECAST_MODES: tuple[ForecastMode, ...] = (
4829     ForecastMode.M1,
4830     ForecastMode.M2,
4831     ForecastMode.M3,
4832 )
4833 R1_DEFAULT_MODE = ForecastMode.M1
4834 R1_DEFAULT_ALPHA = 0.60
4835 ValueCutCache = dict[
4836     tuple[int, str, float | None, int],
4837     tuple[tuple[ValueCut, ...], list[dict]],
4838 ]
4839 PERIOD_JANUARY_WARMUP = "january_warmup"
4840 PERIOD_FEB_DEC_OUTPUT = "feb_dec_output"
4841
4842
4843 @dataclass(frozen=True)
4844 class FrozenDayPolicy:
4845     date: str
4846     forecast_mode: ForecastMode
4847     risk_alpha: float | None
4848     fallback_reason: str
4849     calibration_date: str
4850     effective_start_date: str
4851     effective_end_date: str
4852
4853     def effective_k(self, requested_k: int) -> int:
4854         if self.risk_alpha is None:
4855             return 1
4856         return int(requested_k)
4857
4858
4859 @dataclass(frozen=True)
4860 class DayForecast:
4861     mode: ForecastMode
4862     load_hat: np.ndarray
4863     pv_hat: np.ndarray
4864     load_sources: tuple[int, ...]
4865     pv_sources: tuple[int, ...]
4866     load_fallback: bool
4867     pv_fallback: bool
4868
4869
4870 def _aggregate_rows(rows: np.ndarray, mode: ForecastMode) -> np.ndarray:
4871     if rows.ndim != 2 or rows.shape[0] == 0:
4872         raise ValueError("forecast aggregation requires at least one history row")
4873     if mode is ForecastMode.M1:
4874         return rows.mean(axis=0)
4875     if mode is ForecastMode.M2:
4876         return np.median(rows, axis=0)
4877     if mode is ForecastMode.M3:
4878         weights = np.arange(1, rows.shape[0] + 1, dtype=float)
4879         weights /= weights.sum()
4880         return weights @ rows
4881     raise ValueError(f"unknown forecast mode: {mode}")
4882
4883
4884 def forecast_as_of_mode(
4885     data: Q2Data,
4886     target_index: int,
4887     history_end_exclusive: int,
4888     mode: ForecastMode,
4889 ) -> DayForecast:

```

```

4890     if not 0 <= history_end_exclusive <= target_index:
4891         raise ValueError("forecast cutoff must not exceed target date")
4892     target = data.dates[target_index]
4893     same_weekday = [
4894         j
4895         for j in range(history_end_exclusive)
4896         if data.dates[j].weekday() == target.weekday()
4897     ]
4898     load_sources = tuple(same_weekday[-LOAD_HISTORY_SAME_WEEKDAY:])
4899     pv_sources = tuple(
4900         range(max(0, history_end_exclusive - PV_HISTORY_DAYS), history_end_exclusive)
4901     )
4902     if load_sources:
4903         load_hat = _aggregate_rows(data.load[list(load_sources)], mode)
4904         load_fallback = False
4905     else:
4906         load_hat = data.fallback_load.copy()
4907         load_fallback = True
4908     if pv_sources:
4909         pv_hat = _aggregate_rows(data.pv[list(pv_sources)], mode)
4910         pv_fallback = False
4911     else:
4912         pv_hat = data.fallback_pv.copy()
4913         pv_fallback = True
4914     return DayForecast(
4915         mode=mode,
4916         load_hat=np.maximum(0.0, load_hat),
4917         pv_hat=np.maximum(0.0, pv_hat),
4918         load_sources=load_sources,
4919         pv_sources=pv_sources,
4920         load_fallback=load_fallback,
4921         pv_fallback=pv_fallback,
4922     )
4923
4924
4925 def build_forecast_archive_mode(data: Q2Data, mode: ForecastMode) -> ForecastArchive:
4926     n = len(data.dates)
4927     load_hat = np.empty_like(data.load)
4928     pv_hat = np.empty_like(data.pv)
4929     load_sources: list[tuple[int, ...]] = []
4930     pv_sources: list[tuple[int, ...]] = []
4931     for i in range(n):
4932         forecast = forecast_as_of_mode(data, i, i, mode)
4933         load_hat[i] = forecast.load_hat
4934         pv_hat[i] = forecast.pv_hat
4935         load_sources.append(forecast.load_sources)
4936         pv_sources.append(forecast.pv_sources)
4937     return ForecastArchive(
4938         load_hat=load_hat,
4939         pv_hat=pv_hat,
4940         load_residual=data.load - load_hat,
4941         pv_residual=data.pv - pv_hat,
4942         load_sources=tuple(load_sources),
4943         pv_sources=tuple(pv_sources),
4944     )
4945
4946
4947 def assert_history_strictly_prior(
4948     indices: tuple[int, ...] | np.ndarray, decision_index: int, label: str
4949 ) -> None:
4950     values = np.asarray(indices, dtype=int).ravel()
4951     if values.size and np.any(values >= decision_index):
4952         raise AssertionError(f"{label} leaks into decision index {decision_index}")
4953
4954
4955 def k8_scenarios(
4956     target_index: int,
4957     data: Q2Data,

```

```

4958     archive: ForecastArchive,
4959     history_end_exclusive: int | None = None,
4960     k: int = FIXED_SCENARIO_K,
4961 ) -> ScenarioSet | None:
4962     history_end = target_index if history_end_exclusive is None else history_end_exclusive
4963     if history_end <= 0:
4964         return None
4965     scenarios = build_scenarios(
4966         target_index,
4967         data,
4968         archive,
4969         min(k, history_end),
4970         history_end_exclusive=history_end,
4971     )
4972     assert_history_strictly_prior(
4973         scenarios.pool_indices, target_index, "residual pool"
4974     )
4975     assert_history_strictly_prior(
4976         scenarios.medoid_indices, target_index, "medoid dates"
4977     )
4978     return scenarios
4979
4980
4981 def risk_reserve_from_scenarios(
4982     scenarios: ScenarioSet | None,
4983     archive: ForecastArchive,
4984     load_hat: np.ndarray,
4985     pv_hat: np.ndarray,
4986     alpha: float | None,
4987 ) -> np.ndarray:
4988     if scenarios is None or alpha is None:
4989         return np.zeros(T)
4990     loads = np.maximum(
4991         0.0, load_hat[None, :] + archive.load_residual[scenarios.medoid_indices]
4992     )
4993     pvs = np.maximum(
4994         0.0, pv_hat[None, :] + archive.pv_residual[scenarios.medoid_indices]
4995     )
4996     return risk_quantile_floor(
4997         loads, pvs, scenarios.proBABILITIES, load_hat, pv_hat, alpha
4998     )
4999
5000
5001 def build_baseline_value_cuts(
5002     data: Q2Data,
5003     archive: ForecastArchive,
5004     current_index: int,
5005     mode: ForecastMode,
5006     k: int = FIXED_SCENARIO_K,
5007     risk_alpha: float | None = R1_DEFAULT_ALPHA,
5008 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
5009     if current_index + 1 >= len(data.dates):
5010         return (), []
5011     nxt = current_index + 1
5012     forecast = forecast_as_of_mode(data, nxt, current_index, mode)
5013     assert_history_strictly_prior(forecast.load_sources, nxt, "next-day load forecast")
5014     assert_history_strictly_prior(forecast.pv_sources, nxt, "next-day PV forecast")
5015     scenarios = k8_scenarios(nxt, data, archive, current_index, k=k)
5016     q_floor = risk_reserve_from_scenarios(
5017         scenarios, archive, forecast.load_hat, forecast.pv_hat, risk_alpha
5018     )
5019     solved: dict[float, ValueCut] = {}
5020
5021     def solve_at(soc: float) -> None:
5022         result = solve_baseline_plan(
5023             data.price, forecast.load_hat, forecast.pv_hat, float(soc), q_floor=q_floor
5024         )
5025         if result.has_scenario_specific_battery:

```

```

5026         raise AssertionError("next-day baseline LP created scenario battery variables")
5027     value = result.planned_cost + result.expected_emergency_cost
5028     solved[float(soc)] = (
5029         float(soc),
5030         float(value),
5031         float(result.initial_soc_marginal),
5032     )
5033
5034 def certified_max_gap(cuts: tuple[ValueCut, ...]) -> tuple[float, float]:
5035     points = sorted(solved)
5036     best_gap, best_soc = -np.inf, points[0]
5037     lower_lines = [(0.0, 0.0)] + [
5038         (slope, value - slope * reference) for reference, value, slope in cuts
5039     ]
5040     for left, right in zip(points[:-1], points[1:]):
5041         left_value, right_value = solved[left][1], solved[right][1]
5042         upper_slope = (right_value - left_value) / (right - left)
5043         upper_intercept = left_value - upper_slope * left
5044         candidates = [left, right]
5045         for a in range(len(lower_lines)):
5046             for b in range(a + 1, len(lower_lines)):
5047                 slope_a, intercept_a = lower_lines[a]
5048                 slope_b, intercept_b = lower_lines[b]
5049                 if abs(slope_a - slope_b) <= 1e-14:
5050                     continue
5051                 cross = (intercept_b - intercept_a) / (slope_a - slope_b)
5052                 if left < cross < right:
5053                     candidates.append(float(cross))
5054         for soc in candidates:
5055             upper = upper_slope * soc + upper_intercept
5056             gap = upper - evaluate_value_cuts(soc, cuts)
5057             if gap > best_gap:
5058                 best_gap, best_soc = float(gap), float(soc)
5059     return max(best_gap, 0.0), best_soc
5060
5061 solve_at(E_MIN_KWH)
5062 solve_at(E_MAX_KWH)
5063 gap = 0.0
5064 while True:
5065     cuts = tuple(solved[soc] for soc in sorted(solved))
5066     slopes = np.array([cut[2] for cut in cuts])
5067     if np.any(np.diff(slopes) < -1e-6):
5068         raise AssertionError("baseline next-day value subgradients are not monotone")
5069     gap, refine_soc = certified_max_gap(cuts)
5070     if gap <= NEXT_DAY_VALUE_GAP_TOL_YUAN:
5071         break
5072     if len(cuts) >= NEXT_DAY_VALUE_MAX_SAMPLES:
5073         raise AssertionError(
5074             f"baseline next-day value gap {gap:.6f} exceeds tolerance after {len(cuts)} samples"
5075         )
5076     if min(abs(refine_soc - soc) for soc in solved) < 1e-7:
5077         raise AssertionError("baseline next-day value refinement stalled")
5078     solve_at(refine_soc)
5079
5080 rows = [
5081     {
5082         "target_date": data.dates[nxt].strftime("%Y-%m-%d"),
5083         "history_cutoff_date": (
5084             data.dates[current_index - 1].strftime("%Y-%m-%d")
5085             if current_index
5086             else "attachment1_fallback"
5087         ),
5088         "forecast_mode": mode.value,
5089         "risk_alpha": risk_alpha,
5090         "risk_q_floor_kwh": float(q_floor.sum()),
5091         "soc_sample_kwh": reference,
5092         "virtual_next_day_cost_yuan": value,
5093         "value_subgradient_yuan_per_kwh": slope,

```

```

5094         "certified_max_gap_yuan": gap,
5095         "gap_tolerance_yuan": NEXT_DAY_VALUE_GAP_TOL_YUAN,
5096         "bundle_sample_count": len(cuts),
5097     }
5098     for reference, value, slope in cuts
5099 ]
5100 return cuts, rows
5101
5102
5103 def _horizon_forecasts(
5104     data: Q2Data,
5105     archive: ForecastArchive,
5106     scenarios: ScenarioSet | None,
5107     load_hat: np.ndarray,
5108     pv_hat: np.ndarray,
5109     day_index: int,
5110     period: int,
5111 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, float]:
5112     remaining = slice(period, T)
5113     if scenarios is None:
5114         weights = np.ones(1)
5115         bandwidth = float("nan")
5116         load_horizon = load_hat[remaining].copy()
5117         pv_horizon = pv_hat[remaining].copy()
5118     else:
5119         observed_lr = data.load[day_index, : period + 1] - load_hat[: period + 1]
5120         observed_pr = data.pv[day_index, : period + 1] - pv_hat[: period + 1]
5121         weights, bandwidth = posterior_weights(
5122             scenarios, archive, observed_lr, observed_pr
5123         )
5124         load_horizon = np.maximum(
5125             0.0,
5126             load_hat[remaining]
5127             + weights @ archive.load_residual[scenarios.medoid_indices, remaining],
5128         )
5129         pv_horizon = np.maximum(
5130             0.0,
5131             pv_hat[remaining]
5132             + weights @ archive.pv_residual[scenarios.medoid_indices, remaining],
5133         )
5134     load_horizon[0] = data.load[day_index, period]
5135     pv_horizon[0] = data.pv[day_index, period]
5136     return load_horizon, pv_horizon, weights, bandwidth
5137
5138
5139 def run_residual_mpc(
5140     data: Q2Data,
5141     archive: ForecastArchive,
5142     scenarios: ScenarioSet | None,
5143     load_hat: np.ndarray,
5144     pv_hat: np.ndarray,
5145     locked_q: np.ndarray,
5146     initial_soc: float,
5147     day_index: int,
5148     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
5149     path: Path | None = None,
5150     mpc_periods: int | None = None,
5151 ) -> tuple[dict, pd.DataFrame]:
5152     locked_q = np.asarray(locked_q, dtype=float).ravel()
5153     locked_hash = planned_q_hash(locked_q)
5154     rows = []
5155     soc = float(initial_soc)
5156     max_cd = 0.0
5157     n_steps = T if mpc_periods is None else int(mpc_periods)
5158     if not 1 <= n_steps <= T:
5159         raise ValueError("mpc_periods must lie in 1..T")
5160     for t in range(n_steps):
5161         load_horizon, pv_horizon, weights, bandwidth = _horizon_forecasts(

```



```

5162     data, archive, scenarios, load_hat, pv_hat, day_index, t
5163 )
5164 result = solve_fixed_plan_dispatch(
5165     data.price[t:],
5166     locked_q[t:],
5167     load_horizon,
5168     pv_horizon,
5169     soc,
5170     terminal_value_cuts=terminal_value_cuts,
5171 )
5172 residual = (
5173     result.x[0]
5174     + result.emergency[0]
5175     + data.pv[day_index, t]
5176     - result.curtailment[0]
5177     + result.discharge[0]
5178     - data.load[day_index, t]
5179     - result.charge[0]
5180 )
5181 entropy = float(-np.sum(weights * np.log(np.maximum(weights, 1e-300))))
5182 rows.append(
5183     {
5184         "date": data.dates[day_index].strftime("%Y-%m-%d"),
5185         "period": t,
5186         "time": data.time_labels[t],
5187         "planned_q_kwh": locked_q[t],
5188         "actual_x_kwh": result.x[0],
5189         "load_kwh": data.load[day_index, t],
5190         "pv_kwh": data.pv[day_index, t],
5191         "charge_kwh": result.charge[0],
5192         "discharge_kwh": result.discharge[0],
5193         "curtailment_kwh": result.curtailment[0],
5194         "emergency_kwh": result.emergency[0],
5195         "soc_kwh": result.soc[1],
5196         "posterior_entropy": entropy,
5197         "posterior_bandwidth": bandwidth,
5198         "solve_seconds": result.solve_seconds,
5199         "virtual_next_day_value_yuan": result.terminal_value,
5200         "balance_residual_kwh": residual,
5201     }
5202 )
5203 max_cd = max(max_cd, float(result.charge[0] * result.discharge[0]))
5204 soc = float(result.soc[1])
5205 if planned_q_hash(locked_q) != locked_hash:
5206     raise AssertionError("locked day-ahead q changed during MPC")
5207 frame = pd.DataFrame(rows)
5208 if path is not None:
5209     path.parent.mkdir(parents=True, exist_ok=True)
5210     frame.to_csv(path, index=False)
5211 x = frame["actual_x_kwh"].to_numpy()
5212 emergency = frame["emergency_kwh"].to_numpy()
5213 executed_q = locked_q[:n_steps]
5214 executed_price = data.price[:n_steps]
5215 planned_cost = float(executed_price @ executed_q)
5216 emergency_cost = float(EMERGENCY_PRICE_MULTIPLIER * executed_price @ emergency)
5217 physical_ok = bool(
5218     frame["balance_residual_kwh"].abs().max() < NUMERIC_TOL
5219     and np.max(x - executed_q) < NUMERIC_TOL
5220     and frame["soc_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL
5221     and frame["soc_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL
5222     and max_cd <= SIMULTANEOUS_CD_TOL
5223     and np.max(frame["charge_kwh"].to_numpy()) <= POWER_LIMIT_KWH + NUMERIC_TOL
5224     and np.max(frame["discharge_kwh"].to_numpy()) <= POWER_LIMIT_KWH + NUMERIC_TOL
5225 )
5226 summary = {
5227     "date": data.dates[day_index].strftime("%Y-%m-%d"),
5228     "planned_cost_yuan": planned_cost,
5229     "emergency_cost_yuan": emergency_cost,

```

```

5230         "total_cost_yuan": planned_cost + emergency_cost,
5231         "planned_q_kwh": float(executed_q.sum()),
5232         "planned_q_sha256": locked_hash,
5233         "actual_x_kwh": float(x.sum()),
5234         "unused_plan_kwh": float((executed_q - x).sum()),
5235         "emergency_kwh": float(emergency.sum()),
5236         "curtailment_kwh": float(frame["curtailment_kwh"].sum()),
5237         "soc_start_kwh": float(initial_soc),
5238         "soc_end_kwh": soc,
5239         "max_balance_residual_kwh": float(frame["balance_residual_kwh"].abs().max()),
5240         "max_simultaneous_cd_kwh2": max_cd,
5241         "max_x_minus_q_kwh": float(np.max(x - executed_q)),
5242         "mean_mpc_solve_seconds": float(frame["solve_seconds"].mean()),
5243         "total_mpc_solve_seconds": float(frame["solve_seconds"].sum()),
5244         "pass": physical_ok,
5245         "used_full_day_actual_lp": False,
5246         "future_actuals_in_optimizer": False,
5247     }
5248     return summary, frame
5249
5250
5251 def plan_closed_loop_day(
5252     data: Q2Data,
5253     archive: ForecastArchive,
5254     day_index: int,
5255     initial_soc: float,
5256     mode: ForecastMode = R1_DEFAULT_MODE,
5257     risk_alpha: float | None = R1_DEFAULT_ALPHA,
5258     k: int = FIXED_SCENARIO_K,
5259     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
5260     dispatch_path: Path | None = None,
5261     mpc_periods: int | None = None,
5262 ) -> tuple[dict, pd.DataFrame, BaselinePlanResult, np.ndarray, ScenarioSet | None]:
5263     forecast = forecast_as_of_mode(data, day_index, day_index, mode)
5264     assert_history_strictly_prior(forecast.load_sources, day_index, "load forecast")
5265     assert_history_strictly_prior(forecast.pv_sources, day_index, "PV forecast")
5266     scenarios = k8_scenarios(day_index, data, archive, day_index, k=k)
5267     q_floor = risk_reserve_from_scenarios(
5268         scenarios, archive, forecast.load_hat, forecast.pv_hat, risk_alpha
5269     )
5270     plan = solve_baseline_plan(
5271         data.price,
5272         forecast.load_hat,
5273         forecast.pv_hat,
5274         initial_soc,
5275         q_floor=q_floor,
5276         terminal_value_cuts=terminal_value_cuts,
5277     )
5278     if plan.has_scenario_specific_battery:
5279         raise AssertionError("day-ahead baseline LP created scenario battery variables")
5280     for name, shape in plan.variable_shapes.items():
5281         if name in {"charge", "discharge", "soc"} and shape not in {(T,), (T + 1,)}:
5282             raise AssertionError(f"unexpected {name} shape {shape}")
5283     summary, frame = run_residual_mpc(
5284         data,
5285         archive,
5286         scenarios,
5287         forecast.load_hat,
5288         forecast.pv_hat,
5289         plan.q,
5290         initial_soc,
5291         day_index,
5292         terminal_value_cuts=terminal_value_cuts,
5293         path=dispatch_path,
5294         mpc_periods=mpc_periods,
5295     )
5296     summary.update(
5297         {

```

```

5298         "forecast_mode": mode.value,
5299         "risk_alpha": risk_alpha,
5300         "scenario_k": 0 if scenarios is None else int(len(scenarios.proBABILITIES)),
5301         "risk_q_floor_kwh": float(q_floor.sum()),
5302         "load_fallback": forecast.load_fallback,
5303         "pv_fallback": forecast.pv_fallback,
5304         "load_source_dates": ";".join(
5305             data.dates[j].strftime("%Y-%m-%d") for j in forecast.load_sources
5306         )
5307         or "attachment1_fallback",
5308         "pv_source_dates": ";".join(
5309             data.dates[j].strftime("%Y-%m-%d") for j in forecast.pv_sources
5310         )
5311         or "attachment1_fallback",
5312         "residual_pool_end_date": (
5313             data.dates[scenarios.pool_indices[-1]].strftime("%Y-%m-%d")
5314             if scenarios is not None
5315             else "none"
5316         ),
5317         "medoid_dates": (
5318             ";".join(
5319                 data.dates[j].strftime("%Y-%m-%d") for j in scenarios.medoid_indices
5320             )
5321             if scenarios is not None
5322             else "none"
5323         ),
5324         "day_ahead_variable_shapes": {
5325             name: list(shape) for name, shape in plan.variable_shapes.items()
5326         },
5327         "has_scenario_specific_battery": plan.has_scenario_specific_battery,
5328         "day_ahead_solve_seconds": plan.solve_seconds,
5329         "day_ahead_expected_emergency_cost_yuan": plan.expected_emergency_cost,
5330         "day_ahead_virtual_next_day_value_yuan": plan.expected_terminal_value,
5331     }
5332 )
5333 return summary, frame, plan, q_floor, scenarios
5334
5335
5336 def forecast_mae(actual: np.ndarray, hat: np.ndarray) -> float:
5337     return float(np.mean(np.abs(actual - hat)))
5338
5339
5340 def target_quantile_coverage(
5341     realised_net: np.ndarray,
5342     scenario_net: np.ndarray,
5343     probabilities: np.ndarray,
5344     alpha: float,
5345 ) -> float:
5346     hits = []
5347     for t in range(len(realised_net)):
5348         order = np.argsort(scenario_net[:, t])
5349         cumulative = np.cumsum(probabilities[order])
5350         threshold = scenario_net[order][np.searchsorted(cumulative, alpha, side="left")]
5351         hits.append(realised_net[t] <= threshold + 1e-12)
5352     return float(np.mean(hits))
5353
5354
5355 def mode_diagnostics(
5356     data: Q2Data,
5357     day_index: int,
5358     archives: dict[ForecastMode, ForecastArchive],
5359     alpha: float,
5360     k: int = FIXED_SCENARIO_K,
5361 ) -> list[dict]:
5362     rows = []
5363     for mode, archive in archives.items():
5364         forecast = forecast_as_of_mode(data, day_index, day_index, mode)
5365         scenarios = k8_scenarios(day_index, data, archive, day_index, k=k)

```

```

5366         load_mae = forecast_mae(data.load[day_index], forecast.load_hat)
5367         pv_mae = forecast_mae(data.pv[day_index], forecast.pv_hat)
5368         net_mae = forecast_mae(
5369             data.load[day_index] - data.pv[day_index],
5370             forecast.load_hat - forecast.pv_hat,
5371         )
5372         coverage = float("nan")
5373         if scenarios is not None:
5374             scenario_net = (
5375                 archive.load_residual[scenarios.medoid_indices]
5376                 - archive.pv_residual[scenarios.medoid_indices]
5377             )
5378             realised_net = (
5379                 data.load[day_index]
5380                 - data.pv[day_index]
5381                 - (forecast.load_hat - forecast.pv_hat)
5382             )
5383             coverage = target_quantile_coverage(
5384                 realised_net, scenario_net, scenarios.probabilities, alpha
5385             )
5386         rows.append(
5387             {
5388                 "date": data.dates[day_index].strftime("%Y-%m-%d"),
5389                 "forecast_mode": mode.value,
5390                 "load_mae_kwh": load_mae,
5391                 "pv_mae_kwh": pv_mae,
5392                 "net_load_mae_kwh": net_mae,
5393                 "risk_alpha": alpha,
5394                 "quantile_coverage": coverage,
5395                 "selection_role": "diagnostic_only",
5396             }
5397         )
5398         return rows
5399
5400
5401 def one_se_tiebreak(
5402     rows: list[dict],
5403 ) -> dict:
5404     """Select by mean actual cost, then unused plan, then m1, then smaller alpha."""
5405     best = min(rows, key=lambda row: row["mean_actual_cost_yuan"])
5406     threshold = best["mean_actual_cost_yuan"] + best["standard_error_yuan"]
5407     eligible = [row for row in rows if row["mean_actual_cost_yuan"] <= threshold]
5408     mode_rank = {ForecastMode.M1.value: 0, ForecastMode.M2.value: 1, ForecastMode.M3.value: 2}
5409     return min(
5410         eligible,
5411         key=lambda row: (
5412             row["mean_unused_plan_kwh"],
5413             mode_rank[row["forecast_mode"]],
5414             row["risk_alpha"],
5415         ),
5416     )
5417
5418
5419 def value_cut_cache_key(
5420     day_index: int,
5421     mode: ForecastMode,
5422     risk_alpha: float | None,
5423     k: int,
5424 ) -> tuple[int, str, float | None, int]:
5425     return (int(day_index), mode.value, None if risk_alpha is None else float(risk_alpha), int(k))
5426
5427
5428 def cached_baseline_value_cuts(
5429     cache: ValueCutCache,
5430     data: Q2Data,
5431     archive: ForecastArchive,
5432     day_index: int,
5433     mode: ForecastMode,

```

```

5434     k: int,
5435     risk_alpha: float | None,
5436     lock: Lock | None = None,
5437 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
5438     """Reuse next-day baseline value cuts for the same (day, m, alpha, K)."""
5439     key = value_cut_cache_key(day_index, mode, risk_alpha, k)
5440     if lock is None:
5441         hit = cache.get(key)
5442         if hit is not None:
5443             return hit
5444         built = build_baseline_value_cuts(
5445             data, archive, day_index, mode, k=k, risk_alpha=risk_alpha
5446         )
5447         cache[key] = built
5448         return built
5449     with lock:
5450         hit = cache.get(key)
5451         if hit is not None:
5452             return hit
5453     built = build_baseline_value_cuts(
5454         data, archive, day_index, mode, k=k, risk_alpha=risk_alpha
5455     )
5456     with lock:
5457         existing = cache.get(key)
5458         if existing is not None:
5459             return existing
5460         cache[key] = built
5461         return built
5462
5463
5464 def score_one_candidate(
5465     data: Q2Data,
5466     archive: ForecastArchive,
5467     mode_value: str,
5468     alpha: float,
5469     validation_indices: tuple[int, ...],
5470     window_start_soc: float,
5471     k: int = FIXED_SCENARIO_K,
5472     include_value_cuts: bool = True,
5473     cut_cache: ValueCutCache | None = None,
5474     cut_cache_lock: Lock | None = None,
5475 ) -> dict:
5476     """Run one (m, alpha) on a finished-day window with that candidate's own SOC path."""
5477     mode = ForecastMode(mode_value)
5478     costs: list[float] = []
5479     unused: list[float] = []
5480     planned: list[float] = []
5481     emergency: list[float] = []
5482     times: list[float] = []
5483     soc_path = [float(window_start_soc)]
5484     soc = float(window_start_soc)
5485     used_full_day_actual_lp = False
5486     future_actuals_in_optimizer = False
5487     cache = {} if cut_cache is None else cut_cache
5488     for i in validation_indices:
5489         cuts: tuple[ValueCut, ...] | None = None
5490         if include_value_cuts:
5491             cuts, _cut_rows = cached_baseline_value_cuts(
5492                 cache,
5493                 data,
5494                 archive,
5495                 int(i),
5496                 mode,
5497                 k,
5498                 float(alpha),
5499                 lock=cut_cache_lock,
5500             )
5501         summary, _frame, _plan, _floor, _scenarios = plan_closed_loop_day(

```

```

5502         data,
5503         archive,
5504         int(i),
5505         soc,
5506         mode=mode,
5507         risk_alpha=float(alpha),
5508         k=k,
5509         terminal_value_cuts=cuts,
5510     )
5511     used_full_day_actual_lp = used_full_day_actual_lp or bool(
5512         summary["used_full_day_actual_lp"]
5513     )
5514     future_actuals_in_optimizer = future_actuals_in_optimizer or bool(
5515         summary["future_actuals_in_optimizer"]
5516     )
5517     costs.append(summary["total_cost_yuan"])
5518     unused.append(summary["unused_plan_kwh"])
5519     planned.append(summary["planned_cost_yuan"])
5520     emergency.append(summary["emergency_cost_yuan"])
5521     times.append(summary["day_ahead_solve_seconds"] + summary["total_mpc_solve_seconds"])
5522     soc = float(summary["soc_end_kwh"])
5523     soc_path.append(soc)
5524     return {
5525         "forecast_mode": mode.value,
5526         "risk_alpha": float(alpha),
5527         "validation_days": int(len(validation_indices)),
5528         "window_start_soc_kwh": float(window_start_soc),
5529         "candidate_soc_end_kwh": soc,
5530         "candidate_soc_path_kwh": ";".join(f"{value:.6f}" for value in soc_path),
5531         "mean_actual_cost_yuan": float(np.mean(costs)),
5532         "standard_error_yuan": float(
5533             np.std(costs, ddof=1) / sqrt(len(costs)) if len(costs) > 1 else 0.0
5534         ),
5535         "mean_planned_cost_yuan": float(np.mean(planned)),
5536         "mean_emergency_cost_yuan": float(np.mean(emergency)),
5537         "mean_unused_plan_kwh": float(np.mean(unused)),
5538         "mean_solve_seconds": float(np.mean(times)),
5539         "used_full_day_actual_lp": used_full_day_actual_lp,
5540         "future_actuals_in_optimizer": future_actuals_in_optimizer,
5541         "selection_criterion": "mean_actual_cost_sum_pq_plus_5pe",
5542         "include_value_cuts": bool(include_value_cuts),
5543     }
5544
5545
5546 def _score_candidate_job(payload: dict) -> dict:
5547     return score_one_candidate(
5548         payload["data"],
5549         payload["archive"],
5550         payload["mode_value"],
5551         payload["alpha"],
5552         payload["validation_indices"],
5553         payload["window_start_soc"],
5554         payload["k"],
5555         include_value_cuts=payload.get("include_value_cuts", True),
5556         cut_cache=payload.get("cut_cache"),
5557         cut_cache_lock=payload.get("cut_cache_lock"),
5558     )
5559
5560
5561 def closed_loop_candidate_window(
5562     data: Q2Data,
5563     archives: dict[ForecastMode, ForecastArchive],
5564     validation_indices: np.ndarray,
5565     window_start_soc: float,
5566     k: int = FIXED_SCENARIO_K,
5567     workers: int | None = None,
5568     include_value_cuts: bool = True,
5569     cut_cache: ValueCutCache | None = None,

```

```

5570     cut_cache_lock: Lock | None = None,
5571 ) -> list[dict]:
5572     """Score all 12 (m, alpha) pairs by the same residual-MPC policy."""
5573     indices = tuple(int(i) for i in np.asarray(validation_indices).ravel())
5574     cache = {} if cut_cache is None else cut_cache
5575     jobs = [
5576         {
5577             "data": data,
5578             "archive": archives[mode],
5579             "mode_value": mode.value,
5580             "alpha": float(alpha),
5581             "validation_indices": indices,
5582             "window_start_soc": float(window_start_soc),
5583             "k": k,
5584             "include_value_cuts": include_value_cuts,
5585             "cut_cache": cache,
5586             "cut_cache_lock": cut_cache_lock,
5587         }
5588         for mode in FORECAST_MODES
5589         for alpha in RISK_ALPHA_CANDIDATES
5590     ]
5591     n_workers = int(os.environ.get("Q2_R2_WORKERS", workers if workers is not None else 8))
5592     n_workers = max(1, min(n_workers, len(jobs)))
5593     if n_workers == 1:
5594         records = [_score_candidate_job(job) for job in jobs]
5595     else:
5596         with ThreadPoolExecutor(max_workers=n_workers) as pool:
5597             records = list(pool.map(_score_candidate_job, jobs))
5598     if len(records) != len(FORECAST_MODES) * len(RISK_ALPHA_CANDIDATES):
5599         raise AssertionError("closed-loop calibration must evaluate 12 (m, alpha) pairs")
5600     selected = one_se_tiebreak(records)
5601     threshold = selected["mean_actual_cost_yuan"] + selected["standard_error_yuan"]
5602     for row in records:
5603         row["selected"] = (
5604             row["forecast_mode"] == selected["forecast_mode"]
5605             and row["risk_alpha"] == selected["risk_alpha"]
5606         )
5607         row["one_se_threshold_yuan"] = threshold
5608     return records
5609
5610 def _validation_diagnostics(
5611     data: Q2Data,
5612     archives: dict[ForecastMode, ForecastArchive],
5613     validation_indices: np.ndarray,
5614     k: int,
5615 ) -> dict[str, dict[str, float]]:
5616     by_mode: dict[str, dict[str, float]] = {}
5617     for mode, archive in archives.items():
5618         load_mae = []
5619         pv_mae = []
5620         net_mae = []
5621         coverage = []
5622         for i in validation_indices:
5623             rows = mode_diagnostics(data, int(i), {mode: archive}, RISK_ALPHA_CANDIDATES[0], k=k)
5624             load_mae.append(rows[0]["load_mae_kwh"])
5625             pv_mae.append(rows[0]["pv_mae_kwh"])
5626             net_mae.append(rows[0]["net_load_mae_kwh"])
5627             coverage.append(rows[0]["quantile_coverage"])
5628         coverage_arr = np.asarray(coverage, dtype=float)
5629         finite = coverage_arr[np.isfinite(coverage_arr)]
5630         by_mode[mode.value] = {
5631             "load_mae_kwh": float(np.mean(load_mae)),
5632             "pv_mae_kwh": float(np.mean(pv_mae)),
5633             "net_load_mae_kwh": float(np.mean(net_mae)),
5634             "quantile_coverage": float(np.mean(finite)) if finite.size else float("nan"),
5635         }
5636     }
5637     return by_mode

```

```

5638
5639
5640 def run_rolling_closed_loop_calibration(
5641     data: Q2Data,
5642     archives: dict[ForecastMode, ForecastArchive],
5643     k: int = FIXED_SCENARIO_K,
5644     workers: int | None = None,
5645     include_value_cuts: bool = True,
5646 ) -> tuple[list[dict], list[dict], list[dict], dict]:
5647     """Walk 14-day blocks, score 12 candidates, then deploy the winner to carry SOC."""
5648     n = len(data.dates)
5649     policy_start_soc = np.full(n, np.nan)
5650     soc = float(E_INITIAL_KWH)
5651     calibration_rows: list[dict] = []
5652     diagnostic_rows: list[dict] = []
5653     deployed_rows: list[dict] = []
5654     block = K_RECALIBRATION_DAYS
5655     window = RISK_CALIBRATION_DAYS
5656     cut_cache: ValueCutCache = {}
5657     cut_lock = Lock()
5658
5659     for calibration in range(0, n, block):
5660         effective_end = min(calibration + block - 1, n - 1)
5661         validation = np.arange(max(0, calibration - window), calibration)
5662         history_cutoff = (
5663             data.dates[calibration - 1].strftime("%Y-%m-%d")
5664             if calibration
5665             else "attachment1_fallback"
5666         )
5667         block_meta = {
5668             "calibration_date": data.dates[calibration].strftime("%Y-%m-%d"),
5669             "history_cutoff_date": history_cutoff,
5670             "effective_start_date": data.dates[calibration].strftime("%Y-%m-%d"),
5671             "effective_end_date": data.dates[effective_end].strftime("%Y-%m-%d"),
5672         }
5673         if len(validation) < window:
5674             selected_mode = ForecastMode.M1
5675             selected_alpha: float | None = None
5676             fallback = "insufficient_prior_14_day_window"
5677             calibration_rows.append(
5678                 {
5679                     **block_meta,
5680                     "forecast_mode": selected_mode.value,
5681                     "risk_alpha": selected_alpha,
5682                     "validation_days": int(len(validation)),
5683                     "validation_start_date": "none",
5684                     "validation_end_date": "none",
5685                     "window_start_soc_kwh": float(soc),
5686                     "candidate_soc_end_kwh": float("nan"),
5687                     "candidate_soc_path_kwh": "",
5688                     "mean_actual_cost_yuan": float("nan"),
5689                     "standard_error_yuan": float("nan"),
5690                     "mean_planned_cost_yuan": float("nan"),
5691                     "mean_emergency_cost_yuan": float("nan"),
5692                     "mean_unused_plan_kwh": float("nan"),
5693                     "mean_solve_seconds": float("nan"),
5694                     "used_full_day_actual_lp": False,
5695                     "future_actuals_in_optimizer": False,
5696                     "selection_criterion": "mean_actual_cost_sum_pq_plus_5pe",
5697                     "one_se_threshold_yuan": float("nan"),
5698                     "selected": True,
5699                     "fallback_reason": fallback,
5700                     "include_value_cuts": bool(include_value_cuts),
5701                 }
5702             )
5703             active_k = 1
5704         else:
5705             window_start_soc = float(policy_start_soc[int(validation[0])])

```



```

5706     if not np.isfinite(window_start_soc):
5707         raise AssertionError("deployed policy SOC missing at validation window start")
5708     records = closed_loop_candidate_window(
5709         data,
5710         archives,
5711         validation,
5712         window_start_soc,
5713         k=k,
5714         workers=workers,
5715         include_value_cuts=include_value_cuts,
5716         cut_cache=cut_cache,
5717         cut_cache_lock=cut_lock,
5718     )
5719     mae_by_mode = _validation_diagnostics(data, archives, validation, k=k)
5720     selected_row = next(row for row in records if row["selected"])
5721     selected_mode = ForecastMode(selected_row["forecast_mode"])
5722     selected_alpha = float(selected_row["risk_alpha"])
5723     fallback = ""
5724     active_k = k
5725     for row in records:
5726         calibration_rows.append(
5727             {
5728                 **block_meta,
5729                 **row,
5730                 "validation_start_date": data.dates[int(validation[0])].strftime(
5731                     "%Y-%m-%d"
5732                 ),
5733                 "validation_end_date": data.dates[int(validation[-1])].strftime(
5734                     "%Y-%m-%d"
5735                 ),
5736                 "fallback_reason": fallback,
5737             }
5738         )
5739     diagnostic = mae_by_mode[row["forecast_mode"]]
5740     diagnostic_rows.append(
5741         {
5742             **block_meta,
5743             "forecast_mode": row["forecast_mode"],
5744             "risk_alpha": row["risk_alpha"],
5745             "load_mae_kwh": diagnostic["load_mae_kwh"],
5746             "pv_mae_kwh": diagnostic["pv_mae_kwh"],
5747             "net_load_mae_kwh": diagnostic["net_load_mae_kwh"],
5748             "quantile_coverage": diagnostic["quantile_coverage"],
5749             "mean_planned_cost_yuan": row["mean_planned_cost_yuan"],
5750             "mean_emergency_cost_yuan": row["mean_emergency_cost_yuan"],
5751             "mean_unused_plan_kwh": row["mean_unused_plan_kwh"],
5752             "mean_actual_cost_yuan": row["mean_actual_cost_yuan"],
5753             "selected": row["selected"],
5754             "selection_role": "diagnostic_only",
5755         }
5756     )
5757     print(
5758         f"C2-R2 {block_meta['calibration_date']}: "
5759         f"selected {selected_mode.value} alpha={selected_alpha:.2f} "
5760         f"cost={selected_row['mean_actual_cost_yuan']:.2f}",
5761         flush=True,
5762     )
5763
5764     for i in range(calibration, effective_end + 1):
5765         policy_start_soc[i] = soc
5766         cuts: tuple[ValueCut, ...] | None = None
5767         if include_value_cuts:
5768             cuts, _cut_rows = cached_baseline_value_cuts(
5769                 cut_cache,
5770                 data,
5771                 archives[selected_mode],
5772                 i,
5773                 selected_mode,

```

```

5774         active_k,
5775         selected_alpha,
5776         lock=cut_lock,
5777     )
5778     summary, _frame, _plan, _floor, _scenarios = plan_closed_loop_day(
5779         data,
5780         archives[selected_mode],
5781         i,
5782         soc,
5783         mode=selected_mode,
5784         risk_alpha=selected_alpha,
5785         k=active_k,
5786         terminal_value_cuts=cuts,
5787     )
5788     if not summary["pass"]:
5789         raise AssertionError(f"deployed policy failed on {summary['date']}")
5790     deployed_rows.append(
5791         {
5792             "date": summary["date"],
5793             "forecast_mode": selected_mode.value,
5794             "risk_alpha": selected_alpha,
5795             "scenario_k": active_k if selected_alpha is not None else 1,
5796             "soc_start_kwh": summary["soc_start_kwh"],
5797             "soc_end_kwh": summary["soc_end_kwh"],
5798             "planned_cost_yuan": summary["planned_cost_yuan"],
5799             "emergency_cost_yuan": summary["emergency_cost_yuan"],
5800             "total_cost_yuan": summary["total_cost_yuan"],
5801             "unused_plan_kwh": summary["unused_plan_kwh"],
5802             "emergency_kwh": summary["emergency_kwh"],
5803             "used_full_day_actual_lp": summary["used_full_day_actual_lp"],
5804             "future_actuals_in_optimizer": summary["future_actuals_in_optimizer"],
5805             "pass": summary["pass"],
5806             "fallback_reason": fallback,
5807             "include_value_cuts": bool(include_value_cuts),
5808         }
5809     )
5810     soc = float(summary["soc_end_kwh"])
5811     return (
5812         calibration_rows,
5813         diagnostic_rows,
5814         deployed_rows,
5815         {
5816             "include_value_cuts": bool(include_value_cuts),
5817             "calibration_include_value_cuts": bool(include_value_cuts),
5818             "deployment_include_value_cuts": bool(include_value_cuts),
5819             "value_cut_cache_entries": int(len(cut_cache)),
5820         },
5821     )
5822
5823
5824 def output_period(date: pd.Timestamp | str) -> str:
5825     stamp = pd.Timestamp(date).normalize()
5826     if stamp < pd.Timestamp(OFFICIAL_OUTPUT_START):
5827         return PERIOD_JANUARY_WARMUP
5828     return PERIOD_FEB_DEC_OUTPUT
5829
5830
5831 def selected_calendar_for_dates(
5832     calibration: pd.DataFrame,
5833     dates: pd.DatetimeIndex,
5834 ) -> list[FrozenDayPolicy]:
5835     """Map each calendar day to the unique R2-selected (m, alpha) block."""
5836     selected = calibration.loc[calibration["selected"].astype(bool)].copy()
5837     if selected.empty:
5838         raise AssertionError("closed-loop calibration has no selected rows")
5839     selected["effective_start_date"] = pd.to_datetime(selected["effective_start_date"])
5840     selected["effective_end_date"] = pd.to_datetime(selected["effective_end_date"])
5841     policies: list[FrozenDayPolicy] = []

```

```

5842     for date in pd.DatetimeIndex(dates):
5843         match = selected.loc[
5844             (selected["effective_start_date"] <= date)
5845             & (selected["effective_end_date"] >= date)
5846         ]
5847         if len(match) != 1:
5848             raise AssertionError(f"expected one selected block for {date.date()}, got {len(match)}")
5849         row = match.iloc[0]
5850         alpha_raw = row["risk_alpha"]
5851         alpha = None if pd.isna(alpha_raw) else float(alpha_raw)
5852         policies.append(
5853             FrozenDayPolicy(
5854                 date=date.strftime("%Y-%m-%d"),
5855                 forecast_mode=ForecastMode(str(row["forecast_mode"])),
5856                 risk_alpha=alpha,
5857                 fallback_reason=str(row["fallback_reason"] or ""),
5858                 calibration_date=pd.Timestamp(row["calibration_date"]).strftime("%Y-%m-%d"),
5859                 effective_start_date=pd.Timestamp(row["effective_start_date"]).strftime(
5860                     "%Y-%m-%d"
5861                 ),
5862                 effective_end_date=pd.Timestamp(row["effective_end_date"]).strftime("%Y-%m-%d"),
5863             )
5864         )
5865     return policies
5866
5867
5868 def _forecast_audit_row(summary: dict) -> dict:
5869     return {
5870         "date": summary["date"],
5871         "forecast_mode": summary["forecast_mode"],
5872         "risk_alpha": summary["risk_alpha"],
5873         "scenario_k": summary["scenario_k"],
5874         "load_source_dates": summary["load_source_dates"],
5875         "pv_source_dates": summary["pv_source_dates"],
5876         "residual_pool_end_date": summary["residual_pool_end_date"],
5877         "medoid_dates": summary["medoid_dates"],
5878         "load_fallback": summary["load_fallback"],
5879         "pv_fallback": summary["pv_fallback"],
5880         "risk_q_floor_kwh": summary["risk_q_floor_kwh"],
5881         "has_scenario_specific_battery": summary["has_scenario_specific_battery"],
5882         "day_ahead_variable_shapes": json.dumps(
5883             summary["day_ahead_variable_shapes"], ensure_ascii=False
5884         )
5885         if not isinstance(summary["day_ahead_variable_shapes"], str)
5886         else summary["day_ahead_variable_shapes"],
5887     }
5888
5889
5890 def _physical_audit_row(summary: dict) -> dict:
5891     return {
5892         "date": summary["date"],
5893         "period": summary["period"],
5894         "pass": summary["pass"],
5895         "planned_q_sha256": summary["planned_q_sha256"],
5896         "max_balance_residual_kwh": summary["max_balance_residual_kwh"],
5897         "max_x_minus_q_kwh": summary["max_x_minus_q_kwh"],
5898         "max_simultaneous_cd_kwh2": summary["max_simultaneous_cd_kwh2"],
5899         "soc_start_kwh": summary["soc_start_kwh"],
5900         "soc_end_kwh": summary["soc_end_kwh"],
5901         "unused_plan_kwh": summary["unused_plan_kwh"],
5902         "emergency_kwh": summary["emergency_kwh"],
5903         "curtailment_kwh": summary["curtailment_kwh"],
5904         "used_full_day_actual_lp": summary["used_full_day_actual_lp"],
5905         "future_actuals_in_optimizer": summary["future_actuals_in_optimizer"],
5906     }
5907
5908
5909 def _daily_summary_row(summary: dict) -> dict:

```

```

5910     shapes = summary["day_ahead_variable_shapes"]
5911     if isinstance(shapes, dict):
5912         shapes = json.dumps(shapes, ensure_ascii=False)
5913     row = {key: value for key, value in summary.items() if key != "day_ahead_variable_shapes"}
5914     row["day_ahead_variable_shapes"] = shapes
5915     return row
5916
5917
5918 def aggregate_cost_row(
5919     frame: pd.DataFrame,
5920     *,
5921     label: str,
5922     extra: dict | None = None,
5923 ) -> dict:
5924     if frame.empty:
5925         raise AssertionError(f"cannot aggregate empty ledger for {label}")
5926     row = {
5927         "label": label,
5928         "n_days": int(len(frame)),
5929         "start_date": str(frame["date"].iloc[0]),
5930         "end_date": str(frame["date"].iloc[-1]),
5931         "planned_cost_yuan": float(frame["planned_cost_yuan"].sum()),
5932         "emergency_cost_yuan": float(frame["emergency_cost_yuan"].sum()),
5933         "total_cost_yuan": float(frame["total_cost_yuan"].sum()),
5934         "planned_q_kwh": float(frame["planned_q_kwh"].sum()),
5935         "actual_x_kwh": float(frame["actual_x_kwh"].sum()),
5936         "unused_plan_kwh": float(frame["unused_plan_kwh"].sum()),
5937         "emergency_kwh": float(frame["emergency_kwh"].sum()),
5938         "curtailment_kwh": float(frame["curtailment_kwh"].sum()),
5939         "soc_start_kwh": float(frame["soc_start_kwh"].iloc[0]),
5940         "soc_end_kwh": float(frame["soc_end_kwh"].iloc[-1]),
5941         "max_balance_residual_kwh": float(frame["max_balance_residual_kwh"].max()),
5942         "all_days_pass": bool(frame["pass"].all()),
5943         "used_full_day_actual_lp": bool(frame["used_full_day_actual_lp"].any()),
5944         "future_actuals_in_optimizer": bool(frame["future_actuals_in_optimizer"].any()),
5945     }
5946     if extra:
5947         row.update(extra)
5948     return row
5949
5950
5951 def k_sensitivity_conclusion(rows: list[dict]) -> str:
5952     """Keep K=8 when cost and emergency remain stable versus K=4 and K=12."""
5953     by_k = {int(row["scenario_k"]): row for row in rows if int(row["scenario_k"]) in {4, 8, 12}}
5954     if set(by_k) != {4, 8, 12}:
5955         raise AssertionError("K sensitivity must report K=4, 8 and 12")
5956     costs = np.array([by_k[k]["total_cost_yuan"] for k in (4, 8, 12)], dtype=float)
5957     emergencies = np.array([by_k[k]["emergency_kwh"] for k in (4, 8, 12)], dtype=float)
5958     cost_span = float(np.max(np.abs(costs - costs[1])) / max(abs(costs[1]), 1.0))
5959     emergency_span = float(
5960         np.max(np.abs(emergencies - emergencies[1])) / max(abs(emergencies[1]), 1.0)
5961     )
5962     if cost_span <= 0.05 and emergency_span <= 0.10:
5963         return "retain_k8_stable"
5964     return "review_k_reselection"
5965
5966
5967 def run_frozen_policy_path(
5968     data: Q2Data,
5969     archives: dict[ForecastMode, ForecastArchive],
5970     calendar: list[FrozenDayPolicy],
5971     *,
5972     start_index: int,
5973     end_index: int,
5974     initial_soc: float,
5975     requested_k: int,
5976     soc_source: str,
5977     include_value_cuts: bool = True,

```

```

5978     dispatch_dir: Path | None = None,
5979     cut_cache: ValueCutCache | None = None,
5980     progress_label: str = "",
5981 ) -> tuple[list[dict], list[dict], list[dict], list[dict]]:
5982     """Execute a frozen (m, alpha) calendar with residual MPC; never uses full-day actual LP."""
5983     if not 0 <= start_index <= end_index < len(data.dates):
5984         raise ValueError("frozen path indices out of range")
5985     if len(calendar) != len(data.dates):
5986         raise AssertionError("frozen calendar must cover every 2025 day")
5987     cache: ValueCutCache = {} if cut_cache is None else cut_cache
5988     daily_rows: list[dict] = []
5989     forecast_rows: list[dict] = []
5990     physical_rows: list[dict] = []
5991     value_rows: list[dict] = []
5992     soc = float(initial_soc)
5993     for i in range(start_index, end_index + 1):
5994         policy = calendar[i]
5995         if policy.date != data.dates[i].strftime("%Y-%m-%d"):
5996             raise AssertionError(f"calendar date mismatch at {i}: {policy.date}")
5997         archive = archives[policy.forecast_mode]
5998         active_k = policy.effective_k(requested_k)
5999         cuts: tuple[ValueCut, ...] | None = None
6000         if include_value_cuts:
6001             cuts, cut_rows = cached_baseline_value_cuts(
6002                 cache,
6003                 data,
6004                 archive,
6005                 i,
6006                 policy.forecast_mode,
6007                 active_k,
6008                 policy.risk_alpha,
6009             )
6010             value_rows.extend({"decision_date": policy.date, **row} for row in cut_rows)
6011         dispatch_path = (
6012             dispatch_dir / f"dispatch_{policy.date}.csv" if dispatch_dir is not None else None
6013         )
6014         summary, _frame, _plan, _floor, _scenarios = plan_closed_loop_day(
6015             data,
6016             archive,
6017             i,
6018             soc,
6019             mode=policy.forecast_mode,
6020             risk_alpha=policy.risk_alpha,
6021             k=active_k,
6022             terminal_value_cuts=cuts,
6023             dispatch_path=dispatch_path,
6024         )
6025         if not summary["pass"]:
6026             raise AssertionError(f"physical validation failed on {summary['date']}")
6027         summary.update(
6028             {
6029                 "period": output_period(policy.date),
6030                 "soc_source": soc_source,
6031                 "requested_k": int(requested_k),
6032                 "calibration_date": policy.calibration_date,
6033                 "fallback_reason": policy.fallback_reason,
6034                 "include_value_cuts": bool(include_value_cuts),
6035             }
6036         )
6037         daily_rows.append(_daily_summary_row(summary))
6038         forecast_rows.append(_forecast_audit_row(summary))
6039         physical_rows.append(_physical_audit_row(summary))
6040         soc = float(summary["soc_end_kwh"])
6041         if progress_label and (i == start_index or (i + 1) % 14 == 0 or i == end_index):
6042             print(
6043                 f"{progress_label} {policy.date}: "
6044                 f"{policy.forecast_mode.value} alpha={policy.risk_alpha} K={active_k} "
6045                 f"cost={summary['total_cost_yuan']:.2f}",

```

```

6046         flush=True,
6047     )
6048     return daily_rows, forecast_rows, physical_rows, value_rows
6049
6050 # ===== q2/run_q2_comparators.py =====
6051 #!/usr/bin/env python3
6052 """Build Q2 annual comparison baselines on the completed linked-policy path."""
6053
6054 from __future__ import annotations
6055
6056 import hashlib
6057 import json
6058 import sys
6059 from pathlib import Path
6060
6061 import numpy as np
6062 import pandas as pd
6063
6064 ROOT = Path(__file__).resolve().parents[1]
6065 if str(ROOT) not in sys.path:
6066     sys.path.insert(0, str(ROOT))
6067
6068 from q2.config import EMERGENCY_PRICE_MULTIPLIER, FULL_OUTPUT_DIR # noqa: E402
6069 from q2.data import load_q2_data # noqa: E402
6070 from q2.forecast import build_forecast_archive # noqa: E402
6071 from q2.pilot import planned_q_hash, run_perfect_information # noqa: E402
6072 from q2.scenarios import build_scenarios, scenario_trajectories # noqa: E402
6073
6074
6075 def weighted_quantile(values: np.ndarray, weights: np.ndarray, probability: float) -> float:
6076     order = np.argsort(values)
6077     sorted_values = values[order]
6078     cumulative = np.cumsum(weights[order])
6079     return float(sorted_values[np.searchsorted(cumulative, probability, side="left")])
6080
6081
6082 def run_no_storage_baseline(data, archive, linked: pd.DataFrame) -> pd.DataFrame:
6083     rows: list[dict] = []
6084     for i, date in enumerate(data.dates):
6085         k = int(linked.loc[i, "selected_k"])
6086         if i == 0:
6087             scenario_net = np.maximum(archive.load_hat[i] - archive.pv_hat[i], 0.0)[None, :]
6088             probability = np.ones(1)
6089         else:
6090             scenarios = build_scenarios(i, data, archive, k)
6091             loads, pvs = scenario_trajectories(scenarios, archive)
6092             scenario_net = np.maximum(loads - pvs, 0.0)
6093             probability = scenarios.proBABILITIES
6094         q = np.array(
6095             [weighted_quantile(scenario_net[:, t], probability, 0.8) for t in range(len(data.price))]
6096         )
6097         actual_net = np.maximum(data.load[i] - data.pv[i], 0.0)
6098         x = np.minimum(q, actual_net)
6099         emergency = np.maximum(actual_net - q, 0.0)
6100         curtailment = np.maximum(data.pv[i] - data.load[i], 0.0)
6101         residual = x + emergency + data.pv[i] - curtailment - data.load[i]
6102         plan_cost = float(data.price @ q)
6103         emergency_cost = float(EMERGENCY_PRICE_MULTIPLIER * data.price @ emergency)
6104         rows.append(
6105             {
6106                 "date": date.strftime("%Y-%m-%d"),
6107                 "selected_k": k,
6108                 "planned_cost_yuan": plan_cost,
6109                 "emergency_cost_yuan": emergency_cost,
6110                 "total_cost_yuan": plan_cost + emergency_cost,
6111                 "planned_q_kwh": float(q.sum()),
6112                 "planned_q_sha256": planned_q_hash(q),
6113                 "actual_x_kwh": float(x.sum()),

```

```

6114         "unused_plan_kwh": float((q - x).sum()),
6115         "emergency_kwh": float(emergency.sum()),
6116         "curtailment_kwh": float(curtailment.sum()),
6117         "max_balance_residual_kwh": float(np.max(np.abs(residual))),
6118         "max_x_minus_q_kwh": float(np.max(x - q)),
6119         "pass": bool(np.max(np.abs(residual)) < 1e-6 and np.max(x - q) < 1e-6),
6120     }
6121 )
6122 return pd.DataFrame(rows)
6123
6124
6125 def main() -> None:
6126     linked_path = FULL_OUTPUT_DIR / "daily_summary.csv"
6127     if not linked_path.exists():
6128         raise FileNotFoundError("Run q2/run_q2_full.py before comparison baselines.")
6129     data = load_q2_data()
6130     archive = build_forecast_archive(data)
6131     linked = pd.read_csv(linked_path)
6132     no_storage = run_no_storage_baseline(data, archive, linked)
6133     if not no_storage["pass"].all():
6134         raise AssertionError("no-storage baseline physical validation failed")
6135     no_storage.to_csv(FULL_OUTPUT_DIR / "no_storage_daily.csv", index=False)
6136
6137     perfect_path = FULL_OUTPUT_DIR / "perfect_information_daily.csv"
6138     run_perfect_information(data, perfect_path)
6139     perfect = pd.read_csv(perfect_path)
6140     if abs(float(perfect["soc_end_kwh"].iloc[-1]) - float(linked["soc_end_kwh"].iloc[-1])) >= 1e-6:
6141         raise AssertionError("linked and perfect-information terminal SOC differ")
6142     comparison = {
6143         "main_linked_total_cost_yuan": float(linked["total_cost_yuan"].sum()),
6144         "no_storage_total_cost_yuan": float(no_storage["total_cost_yuan"].sum()),
6145         "perfect_information_total_cost_yuan": float(perfect["total_cost_yuan"].sum()),
6146         "linked_minus_no_storage_yuan": float(linked["total_cost_yuan"].sum() - no_storage["total_cost_yuan"].sum()),
6147         "linked_minus_perfect_information_yuan": float(linked["total_cost_yuan"].sum() - perfect["total_cost_yuan"].sum()),
6148         "main_start_soc_kwh": float(linked["soc_start_kwh"].iloc[0]),
6149         "main_terminal_soc_kwh": float(linked["soc_end_kwh"].iloc[-1]),
6150         "perfect_terminal_soc_kwh": float(perfect["soc_end_kwh"].iloc[-1]),
6151         "comparison_note": "The no-storage baseline is not SOC-comparable because it has no initial stored energy; use it only as
an operational benchmark, not as a pure storage-value claim.",
6152     }
6153     (FULL_OUTPUT_DIR / "comparison.json").write_text(
6154         json.dumps(comparison, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6155     )
6156     print(json.dumps(comparison, ensure_ascii=False, indent=2))
6157
6158
6159 if __name__ == "__main__":
6160     main()
6161
6162 # ===== q2/run_q2_full.py =====
6163 #!/usr/bin/env python3
6164 """Run the Q2 annual causal linked-policy calculation; no result2.xlsx export."""
6165
6166 from __future__ import annotations
6167
6168 import json
6169 import sys
6170 from pathlib import Path
6171 from time import perf_counter
6172
6173 import numpy as np
6174
6175 ROOT = Path(__file__).resolve().parents[1]
6176 if str(ROOT) not in sys.path:
6177     sys.path.insert(0, str(ROOT))
6178
6179 from q2.config import FULL_OUTPUT_DIR, NUMERIC_TOL # noqa: E402
6180 from q2.data import load_q2_data, write_input_audit # noqa: E402

```

```

6181 from q2.forecast import assert_no_forecast_leakage, build_forecast_archive, write_forecast_archive # noqa: E402
6182 from q2.pilot import run_full_linked_policy # noqa: E402
6183
6184
6185 def main() -> None:
6186     started = perf_counter()
6187     forbidden = ROOT / "output" / "result2.xlsx"
6188     forbidden_before = forbidden.stat().st_mtime_ns if forbidden.exists() else None
6189     data = load_q2_data()
6190     FULL_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
6191     write_input_audit(data, FULL_OUTPUT_DIR / "input_audit.json")
6192     archive = build_forecast_archive(data)
6193     assert_no_forecast_leakage(archive)
6194     write_forecast_archive(
6195         data,
6196         archive,
6197         FULL_OUTPUT_DIR / "forecast_archive.csv",
6198         FULL_OUTPUT_DIR / "forecast_audit.md",
6199     )
6200     daily, calendar, value_audit = run_full_linked_policy(data, archive, FULL_OUTPUT_DIR)
6201     soc_gap = float(
6202         np.max(
6203             np.abs(
6204                 daily["soc_start_kwh"].to_numpy()[1:]
6205                 - daily["soc_end_kwh"].to_numpy()[:-1]
6206             )
6207         )
6208     )
6209     report = {
6210         "status": "PASS",
6211         "days": int(len(daily)),
6212         "total_cost_yuan": float(daily["total_cost_yuan"].sum()),
6213         "planned_cost_yuan": float(daily["planned_cost_yuan"].sum()),
6214         "emergency_cost_yuan": float(daily["emergency_cost_yuan"].sum()),
6215         "planned_q_kwh": float(daily["planned_q_kwh"].sum()),
6216         "actual_x_kwh": float(daily["actual_x_kwh"].sum()),
6217         "unused_plan_kwh": float(daily["unused_plan_kwh"].sum()),
6218         "emergency_kwh": float(daily["emergency_kwh"].sum()),
6219         "curtailment_kwh": float(daily["curtailment_kwh"].sum()),
6220         "soc_start_kwh": float(daily["soc_start_kwh"].iloc[0]),
6221         "soc_end_kwh": float(daily["soc_end_kwh"].iloc[-1]),
6222         "max_balance_residual_kwh": float(daily["max_balance_residual_kwh"].max()),
6223         "max_simultaneous_cd_kwh2": float(daily["max_simultaneous_cd_kwh2"].max()),
6224         "max_x_minus_q_kwh": float(daily["max_x_minus_q_kwh"].max()),
6225         "max_cross_day_soc_gap_kwh": soc_gap,
6226         "calendar_periods": int(calendar["calibration_date"].nunique()),
6227         "all_within_time_limit": bool(calendar["within_time_limit"].all()),
6228         "max_value_cut_gap_yuan": float(value_audit["certified_max_gap_yuan"].max()),
6229         "elapsed_seconds": perf_counter() - started,
6230     }
6231     if (
6232         report["max_balance_residual_kwh"] >= NUMERIC_TOL
6233         or report["max_cross_day_soc_gap_kwh"] >= NUMERIC_TOL
6234         or report["max_x_minus_q_kwh"] >= NUMERIC_TOL
6235     ):
6236         raise AssertionError("annual physical validation failed")
6237     (FULL_OUTPUT_DIR / "validation.json").write_text(
6238         json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6239     )
6240     forbidden_after = forbidden.stat().st_mtime_ns if forbidden.exists() else None
6241     if forbidden_after != forbidden_before:
6242         raise RuntimeError("Forbidden output/result2.xlsx was created or modified")
6243     print(json.dumps(report, ensure_ascii=False, indent=2))
6244
6245
6246 if __name__ == "__main__":
6247     main()
6248

```



```

6249 # ===== q2/run_q2_k8_risk.py =====
6250 #!/usr/bin/env python3
6251 """Annual Q2 re-run: fixed K=8 plus causal rolling risk-quantile reserve."""
6252
6253 from __future__ import annotations
6254
6255 import json
6256 import sys
6257 from pathlib import Path
6258 from time import perf_counter
6259
6260 import numpy as np
6261 import pandas as pd
6262
6263 ROOT = Path(__file__).resolve().parents[1]
6264 if str(ROOT) not in sys.path:
6265     sys.path.insert(0, str(ROOT))
6266
6267 from q2.config import ( # noqa: E402
6268     E_INITIAL_KWH,
6269     FIXED_SCENARIO_K,
6270     FULL_K8_RISK_OUTPUT_DIR,
6271     NUMERIC_TOL,
6272 )
6273 from q2.data import load_q2_data, write_input_audit # noqa: E402
6274 from q2.forecast import assert_no_forecast_leakage, build_forecast_archive, write_forecast_archive # noqa: E402
6275 from q2.pilot import KChoice, build_next_day_value_cuts, run_posterior_mpc, select_risk_alpha # noqa: E402
6276 from q2.scenarios import build_scenarios # noqa: E402
6277
6278
6279 def main() -> None:
6280     started = perf_counter()
6281     output = FULL_K8_RISK_OUTPUT_DIR
6282     output.mkdir(parents=True, exist_ok=True)
6283     dispatch_dir = output / "dispatch_daily"
6284     dispatch_dir.mkdir(parents=True, exist_ok=True)
6285     forbidden = ROOT / "output" / "result2.xlsx"
6286     before = forbidden.stat().st_mtime_ns if forbidden.exists() else None
6287
6288     data = load_q2_data()
6289     archive = build_forecast_archive(data)
6290     assert_no_forecast_leakage(archive)
6291     write_input_audit(data, output / "input_audit.json")
6292     write_forecast_archive(data, archive, output / "forecast_archive.csv", output / "forecast_audit.md")
6293
6294     policy_start_soc = np.full(len(data.dates), np.nan)
6295     daily_rows: list[dict] = []
6296     calibration_rows: list[dict] = []
6297     value_rows: list[dict] = []
6298     soc = E_INITIAL_KWH
6299
6300     for calibration in range(0, len(data.dates), 14):
6301         effective_end = min(calibration + 13, len(data.dates) - 1)
6302         if calibration < 28:
6303             # A K=8 distribution and an empirical risk quantile both need the
6304             # complete 28-day residual pool. Before then, use the same causal
6305             # K=1/no-reserve warmup rather than inventing an unsupported tail.
6306             alpha = None
6307             active_k = 1
6308             candidates = [
6309                 {
6310                     "risk_alpha": alpha,
6311                     "validation_days": 0,
6312                     "mean_validation_cost_yuan": np.nan,
6313                     "standard_error_yuan": np.nan,
6314                     "emergency_purchase_kwh": np.nan,
6315                     "mean_solve_seconds": np.nan,
6316                     "within_time_limit": True,

```

```

6317         "selected": True,
6318         "fallback_reason": "K=1_no_risk_reserve_before_28_day_pool",
6319     }
6320 ]
6321 else:
6322     risk_choice = select_risk_alpha(
6323         data, archive, policy_start_soc, calibration, FIXED_SCENARIO_K
6324     )
6325     alpha = risk_choice.selected_alpha
6326     active_k = FIXED_SCENARIO_K
6327     candidates = [
6328         {**row, "fallback_reason": ""} for row in risk_choice.rows
6329     ]
6330 for candidate in candidates:
6331     calibration_rows.append(
6332         {
6333             "calibration_date": data.dates[calibration].strftime("%Y-%m-%d"),
6334             "history_cutoff_date": (
6335                 data.dates[calibration - 1].strftime("%Y-%m-%d")
6336                 if calibration else "none"
6337             ),
6338             "fixed_k": active_k,
6339             "effective_start_date": data.dates[calibration].strftime("%Y-%m-%d"),
6340             "effective_end_date": data.dates[effective_end].strftime("%Y-%m-%d"),
6341             **candidate,
6342         }
6343     )
6344 for i in range(calibration, effective_end + 1):
6345     policy_start_soc[i] = soc
6346     if i == 0:
6347         choice = None
6348     else:
6349         choice = KChoice(
6350             i,
6351             active_k,
6352             build_scenarios(i, data, archive, active_k),
6353             (),
6354         )
6355     cuts, cut_audit = build_next_day_value_cuts(
6356         data, archive, i, active_k, risk_alpha=alpha
6357     )
6358     value_rows.extend(
6359         {"decision_date": data.dates[i].strftime("%Y-%m-%d"), **row}
6360         for row in cut_audit
6361     )
6362     result = run_posterior_mpc(
6363         data,
6364         archive,
6365         choice,
6366         soc,
6367         dispatch_dir / f"dispatch_{data.dates[i].strftime('%Y-%m-%d')}.csv",
6368         terminal_value_cuts=cuts,
6369         variant="fixed_k8_rolling_risk_quantile",
6370         risk_alpha=alpha,
6371     )
6372     if not result["pass"]:
6373         raise AssertionError(f"physical validation failed on {result['date']}")
6374     result["effective_scenario_count"] = (
6375         1 if choice is None else len(choice.scenarios.probabilities)
6376     )
6377     daily_rows.append(result)
6378     soc = float(result["soc_end_kwh"])
6379
6380 daily = pd.DataFrame(daily_rows)
6381 calibration_frame = pd.DataFrame(calibration_rows)
6382 value_audit = pd.DataFrame(value_rows)
6383 continuity = float(
6384     np.max(

```

```

6385         np.abs(
6386             daily["soc_start_kwh"].to_numpy()[1:]
6387             - daily["soc_end_kwh"].to_numpy()[:-1]
6388         )
6389     )
6390 )
6391 if (
6392     len(daily) != len(data.dates)
6393     or not daily["pass"].all()
6394     or continuity >= NUMERIC_TOL
6395     or not calibration_frame["within_time_limit"].all()
6396 ):
6397     raise AssertionError("annual K=8 risk-quantile validation failed")
6398 daily.to_csv(output / "daily_summary.csv", index=False)
6399 calibration_frame.to_csv(output / "risk_calibration.csv", index=False)
6400 value_audit.to_csv(output / "next_day_value_audit.csv", index=False)
6401 report = {
6402     "status": "PASS",
6403     "policy": "fixed K=8 plus 14-day causal rolling risk-quantile reserve",
6404     "days": len(daily),
6405     "total_cost_yuan": float(daily["total_cost_yuan"].sum()),
6406     "planned_cost_yuan": float(daily["planned_cost_yuan"].sum()),
6407     "emergency_cost_yuan": float(daily["emergency_cost_yuan"].sum()),
6408     "emergency_kwh": float(daily["emergency_kwh"].sum()),
6409     "unused_plan_kwh": float(daily["unused_plan_kwh"].sum()),
6410     "curtailment_kwh": float(daily["curtailment_kwh"].sum()),
6411     "max_balance_residual_kwh": float(daily["max_balance_residual_kwh"].max()),
6412     "max_simultaneous_cd_kwh2": float(daily["max_simultaneous_cd_kwh2"].max()),
6413     "max_x_minus_q_kwh": float(daily["max_x_minus_q_kwh"].max()),
6414     "max_cross_day_soc_gap_kwh": continuity,
6415     "selected_alpha_periods": {
6416         str(alpha): int(count)
6417         for alpha, count in daily["risk_alpha"].fillna("no_reserve").groupby(
6418             daily["risk_alpha"].fillna("no_reserve")
6419         ).size().items()
6420     },
6421     "max_value_cut_gap_yuan": float(value_audit["certified_max_gap_yuan"].max()),
6422     "elapsed_seconds": perf_counter() - started,
6423 }
6424 (output / "validation.json").write_text(
6425     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6426 )
6427 after = forbidden.stat().st_mtime_ns if forbidden.exists() else None
6428 if after != before:
6429     raise RuntimeError("Forbidden output/result2.xlsx was created or modified")
6430 print(json.dumps(report, ensure_ascii=False, indent=2))
6431
6432
6433 if __name__ == "__main__":
6434     main()
6435
6436 # ===== q2/run_q2_pilot.py =====
6437 #!/usr/bin/env python3
6438 """Run Q2 P0-P4 pilot outputs; never creates result2.xlsx."""
6439
6440 from __future__ import annotations
6441
6442 import sys
6443 from pathlib import Path
6444 from time import perf_counter
6445
6446 import pandas as pd
6447
6448 ROOT = Path(__file__).resolve().parents[1]
6449 if str(ROOT) not in sys.path:
6450     sys.path.insert(0, str(ROOT))
6451
6452 from q2.config import OUTPUT_DIR, PILOT_DATES # noqa: E402

```

```

6453 from q2.data import load_q2_data, write_input_audit # noqa: E402
6454 from q2.forecast import ( # noqa: E402
6455     assert_no_forecast_leakage,
6456     build_forecast_archive,
6457     write_forecast_archive,
6458 )
6459 from q2.pilot import ( # noqa: E402
6460     KChoice,
6461     build_k_freeze_calendar,
6462     build_next_day_value_cuts,
6463     run_linked_warmup,
6464     run_perfect_information,
6465     run_posterior_mpc,
6466     write_k_freeze_calendar,
6467     write_k_audit,
6468     write_pilot_figures,
6469     write_validation_report,
6470 )
6471 from q2.scenarios import build_scenarios, write_scenarios_csv # noqa: E402
6472
6473
6474 def main() -> None:
6475     started = perf_counter()
6476     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
6477     forbidden = ROOT / "output" / "result2.xlsx"
6478     forbidden_before = forbidden.stat().st_mtime_ns if forbidden.exists() else None
6479
6480     data = load_q2_data()
6481     write_input_audit(data, OUTPUT_DIR / "input_audit.json")
6482     print("P0 complete: input audit")
6483
6484     perfect_path = OUTPUT_DIR / "perfect_information_daily.csv"
6485     day_start_soc = run_perfect_information(data, perfect_path)
6486     print("P1 complete: 365-day sequential perfect-information comparator")
6487
6488     archive = build_forecast_archive(data)
6489     assert_no_forecast_leakage(archive)
6490     write_forecast_archive(
6491         data,
6492         archive,
6493         OUTPUT_DIR / "forecast_archive.csv",
6494         OUTPUT_DIR / "forecast_audit.md",
6495     )
6496     print("P2 complete: causal forecast archive")
6497
6498     calendar = build_k_freeze_calendar(data, archive, day_start_soc)
6499     write_k_freeze_calendar(OUTPUT_DIR / "k_freeze_calendar.csv", calendar)
6500     print("R4 complete: 14-day K freeze calendar")
6501
6502     warmup, warmup_value_audit = run_linked_warmup(
6503         data, archive, calendar, OUTPUT_DIR / "linked_warmup_daily.csv"
6504     )
6505     print("R2 complete: January linked-policy warmup")
6506
6507     choices = []
6508     scenario_rows = []
6509     for date_text in PILOT_DATES:
6510         i = int(data.dates.get_loc(pd.Timestamp(date_text)))
6511         k = int(calendar.daily_k[i])
6512         choice = KChoice(i, k, build_scenarios(i, data, archive, k), ())
6513         choices.append(choice)
6514         calibration_index = (i // 14) * 14
6515         scenario_rows.extend(
6516             row
6517             for row in calendar.rows
6518             if row["calibration_date"]
6519             == data.dates[calibration_index].strftime("%Y-%m-%d")
6520         )

```

```

6521     write_scenarios_csv(
6522         OUTPUT_DIR / f"scenarios_{date_text}.csv", data, choice.scenarios
6523     )
6524     print(f"P3 {date_text}: selected K={choice.selected_k}")
6525     pd.DataFrame(scenario_rows).to_csv(OUTPUT_DIR / "scenario_selection.csv", index=False)
6526     write_k_audit(OUTPUT_DIR / "k_selection_validation.md", choices, data)
6527
6528     summaries = []
6529     value_comparisons = []
6530     all_value_audits = list(warmup_value_audit)
6531     for choice in choices:
6532         date_text = data.dates[choice.target_index].strftime("%Y-%m-%d")
6533         if date_text == "2025-02-01":
6534             initial_soc = float(warmup["soc_end_kwh"].iloc[-1])
6535         else:
6536             initial_soc = float(day_start_soc[choice.target_index])
6537         cuts, cut_rows = build_next_day_value_cuts(
6538             data, archive, choice.target_index, choice.selected_k
6539         )
6540         all_value_audits.extend(
6541             {"decision_date": date_text, **row} for row in cut_rows
6542         )
6543         without_value = run_posterior_mpc(
6544             data,
6545             archive,
6546             choice,
6547             initial_soc,
6548             OUTPUT_DIR / f"q2_pilot_dispatch_{date_text}_no48h.csv",
6549             terminal_value_cuts=None,
6550             variant="without_48h_value",
6551         )
6552         summary = run_posterior_mpc(
6553             data,
6554             archive,
6555             choice,
6556             initial_soc,
6557             OUTPUT_DIR / f"q2_pilot_dispatch_{date_text}.csv",
6558             terminal_value_cuts=cuts,
6559             variant="with_48h_value",
6560         )
6561         summaries.append(summary)
6562         next_index = choice.target_index + 1
6563         next_k = int(calendar.daily_k[next_index])
6564         next_choice = KChoice(
6565             next_index,
6566             next_k,
6567             build_scenarios(next_index, data, archive, next_k),
6568             (),
6569         )
6570         next_cuts, next_cut_rows = build_next_day_value_cuts(
6571             data, archive, next_index, next_k
6572         )
6573         all_value_audits.extend(
6574             {
6575                 "decision_date": data.dates[next_index].strftime("%Y-%m-%d"),
6576                 **row,
6577             }
6578             for row in next_cut_rows
6579         )
6580         next_after_without = run_posterior_mpc(
6581             data,
6582             archive,
6583             next_choice,
6584             float(without_value["soc_end_kwh"]),
6585             path=None,
6586             terminal_value_cuts=next_cuts,
6587             variant="next_day_after_without_48h_today",
6588         )

```

```

6589     next_after_with = run_posterior_mpc(
6590         data,
6591         archive,
6592         next_choice,
6593         float(summary["soc_end_kwh"]),
6594         path=None,
6595         terminal_value_cuts=next_cuts,
6596         variant="next_day_after_with_48h_today",
6597     )
6598     value_comparisons.append(
6599         {
6600             "date": date_text,
6601             "initial_soc_kwh": initial_soc,
6602             "cost_without_48h_yuan": without_value["total_cost_yuan"],
6603             "cost_with_48h_yuan": summary["total_cost_yuan"],
6604             "cost_difference_yuan": (
6605                 summary["total_cost_yuan"] - without_value["total_cost_yuan"]
6606             ),
6607             "emergency_without_48h_kwh": without_value["emergency_kwh"],
6608             "emergency_with_48h_kwh": summary["emergency_kwh"],
6609             "emergency_difference_kwh": (
6610                 summary["emergency_kwh"] - without_value["emergency_kwh"]
6611             ),
6612             "soc_end_without_48h_kwh": without_value["soc_end_kwh"],
6613             "soc_end_with_48h_kwh": summary["soc_end_kwh"],
6614             "soc_end_difference_kwh": (
6615                 summary["soc_end_kwh"] - without_value["soc_end_kwh"]
6616             ),
6617             "virtual_next_day_value_billed_today": False,
6618             "next_day_cost_after_without_48h_yuan": next_after_without[
6619                 "total_cost_yuan"
6620             ],
6621             "next_day_cost_after_with_48h_yuan": next_after_with[
6622                 "total_cost_yuan"
6623             ],
6624             "realized_two_day_cost_without_48h_today_yuan": (
6625                 without_value["total_cost_yuan"]
6626                 + next_after_without["total_cost_yuan"]
6627             ),
6628             "realized_two_day_cost_with_48h_today_yuan": (
6629                 summary["total_cost_yuan"] + next_after_with["total_cost_yuan"]
6630             ),
6631             "realized_two_day_cost_difference_yuan": (
6632                 summary["total_cost_yuan"]
6633                 + next_after_with["total_cost_yuan"]
6634                 - without_value["total_cost_yuan"]
6635                 - next_after_without["total_cost_yuan"]
6636             ),
6637             "next_day_plan_recomputed_and_billed_once": True,
6638         }
6639     )
6640     print(f"P4 {date_text}: pass={summary['pass']}")
6641     pd.DataFrame(summaries).to_csv(OUTPUT_DIR / "pilot_day_summary.csv", index=False)
6642     pd.DataFrame(value_comparisons).to_csv(
6643         OUTPUT_DIR / "next_day_value_comparison.csv", index=False
6644     )
6645     pd.DataFrame(all_value_audits).to_csv(
6646         OUTPUT_DIR / "next_day_value_audit.csv", index=False
6647     )
6648     write_pilot_figures(data, summaries)
6649     write_validation_report(
6650         OUTPUT_DIR / "validation.json",
6651         OUTPUT_DIR / "validation.md",
6652         perfect_path,
6653         summaries,
6654     )
6655
6656     forbidden_after = forbidden.stat().st_mtime_ns if forbidden.exists() else None

```

```

6657     if forbidden_after != forbidden_before:
6658         raise RuntimeError("Forbidden output/result2.xlsx was created or modified")
6659     print(f"Q2 P0-P4 pilot complete in {perf_counter() - started:.2f} s")
6660     print(f"Outputs: {OUTPUT_DIR}")
6661
6662
6663 if __name__ == "__main__":
6664     main()
6665
6666
6667 # ===== q2/run_q2_policy_consistent.py =====
6668 #!/usr/bin/env python3
6669 """C2-R1: two-day policy-consistent closed loop. Never writes result2.xlsx."""
6670
6671 from __future__ import annotations
6672
6673 import hashlib
6674 import json
6675 import sys
6676 from pathlib import Path
6677 from time import perf_counter
6678
6679 import pandas as pd
6680
6681 ROOT = Path(__file__).resolve().parents[1]
6682 if str(ROOT) not in sys.path:
6683     sys.path.insert(0, str(ROOT))
6684
6685 from q2.config import ( # noqa: E402
6686     E_INITIAL_KWH,
6687     FIXED_SCENARIO_K,
6688     FULL_K8_RISK_OUTPUT_DIR,
6689     PILOT_DATES,
6690     POLICY_CONSISTENT_OUTPUT_DIR,
6691 )
6692 from q2.data import load_q2_data, write_input_audit # noqa: E402
6693 from q2.policy_consistent import ( # noqa: E402
6694     FORECAST_MODES,
6695     R1_DEFAULT_ALPHA,
6696     R1_DEFAULT_MODE,
6697     build_baseline_value_cuts,
6698     build_forecast_archive_mode,
6699     mode_diagnostics,
6700     plan_closed_loop_day,
6701 )
6702
6703
6704 def file_hash(path: Path) -> str | None:
6705     if not path.exists() or not path.is_file():
6706         return None
6707     digest = hashlib.sha256()
6708     with path.open("rb") as stream:
6709         for block in iter(lambda: stream.read(1024 * 1024), b""):
6710             digest.update(block)
6711     return digest.hexdigest()
6712
6713
6714 def snapshot_protected() -> dict[str, str | None]:
6715     protected = [ROOT / "output" / "result2.xlsx"]
6716     if FULL_K8_RISK_OUTPUT_DIR.exists():
6717         protected.extend(sorted(FULL_K8_RISK_OUTPUT_DIR.rglob("*")))
6718     return {str(path.relative_to(ROOT)): file_hash(path) for path in protected if path.is_file()}
6719
6720
6721 def main() -> None:
6722     started = perf_counter()
6723     output = POLICY_CONSISTENT_OUTPUT_DIR
6724     output.mkdir(parents=True, exist_ok=True)

```

```

6725 dispatch_dir = output / "dispatch_daily"
6726 dispatch_dir.mkdir(parents=True, exist_ok=True)
6727 before = snapshot_protected()
6728
6729 data = load_q2_data()
6730 write_input_audit(data, output / "input_audit.json")
6731 archives = {mode: build_forecast_archive_mode(data, mode) for mode in FORECAST_MODES}
6732 archive = archives[R1_DEFAULT_MODE]
6733
6734 forecast_rows: list[dict] = []
6735 diagnostic_rows: list[dict] = []
6736 summaries: list[dict] = []
6737 value_rows: list[dict] = []
6738 physical_rows: list[dict] = []
6739
6740 for date_text in PILOT_DATES:
6741     i = int(data.dates.get_loc(pd.Timestamp(date_text)))
6742     diagnostic_rows.extend(
6743         mode_diagnostics(data, i, archives, R1_DEFAULT_ALPHA, k=FIXED_SCENARIO_K)
6744     )
6745     cuts, cut_rows = build_baseline_value_cuts(
6746         data,
6747         archive,
6748         i,
6749         R1_DEFAULT_MODE,
6750         k=FIXED_SCENARIO_K,
6751         risk_alpha=R1_DEFAULT_ALPHA,
6752     )
6753     value_rows.extend({"decision_date": date_text, **row} for row in cut_rows)
6754     summary, _frame, plan, q_floor, scenarios = plan_closed_loop_day(
6755         data,
6756         archive,
6757         i,
6758         E_INITIAL_KWH,
6759         mode=R1_DEFAULT_MODE,
6760         risk_alpha=R1_DEFAULT_ALPHA,
6761         k=FIXED_SCENARIO_K,
6762         terminal_value_cuts=cuts,
6763         dispatch_path=dispatch_dir / f"dispatch_{date_text}.csv",
6764     )
6765     if not summary["pass"]:
6766         raise AssertionError(f"physical validation failed on {date_text}")
6767     summary["soc_source"] = "R1_isolated_E_INITIAL_6000"
6768     summary["stage"] = "C2-R1"
6769     summaries.append(summary)
6770     forecast_rows.append(
6771         {
6772             "date": date_text,
6773             "forecast_mode": summary["forecast_mode"],
6774             "risk_alpha": summary["risk_alpha"],
6775             "scenario_k": summary["scenario_k"],
6776             "load_source_dates": summary["load_source_dates"],
6777             "pv_source_dates": summary["pv_source_dates"],
6778             "residual_pool_end_date": summary["residual_pool_end_date"],
6779             "medoid_dates": summary["medoid_dates"],
6780             "load_fallback": summary["load_fallback"],
6781             "pv_fallback": summary["pv_fallback"],
6782             "risk_q_floor_kwh": summary["risk_q_floor_kwh"],
6783             "has_scenario_specific_battery": summary["has_scenario_specific_battery"],
6784             "day_ahead_variable_shapes": json.dumps(
6785                 summary["day_ahead_variable_shapes"], ensure_ascii=False
6786             ),
6787         }
6788     )
6789     physical_rows.append(
6790         {
6791             "date": date_text,
6792             "pass": summary["pass"],

```



```

6793         "planned_q_sha256": summary["planned_q_sha256"],
6794         "max_balance_residual_kwh": summary["max_balance_residual_kwh"],
6795         "max_x_minus_q_kwh": summary["max_x_minus_q_kwh"],
6796         "max_simultaneous_cd_kwh2": summary["max_simultaneous_cd_kwh2"],
6797         "soc_start_kwh": summary["soc_start_kwh"],
6798         "soc_end_kwh": summary["soc_end_kwh"],
6799         "unused_plan_kwh": summary["unused_plan_kwh"],
6800         "emergency_kwh": summary["emergency_kwh"],
6801         "curtailment_kwh": summary["curtailment_kwh"],
6802         "used_full_day_actual_lp": summary["used_full_day_actual_lp"],
6803         "future_actuals_in_optimizer": summary["future_actuals_in_optimizer"],
6804     }
6805 )
6806 _ = plan, q_floor, scenarios
6807
6808 pd.DataFrame(forecast_rows).to_csv(output / "forecast_policy_audit.csv", index=False)
6809 pd.DataFrame(diagnostic_rows).to_csv(output / "forecast_diagnostics.csv", index=False)
6810 pd.DataFrame(summaries).drop(columns=["day_ahead_variable_shapes"]).to_csv(
6811     output / "r1_day_summary.csv", index=False
6812 )
6813 pd.DataFrame(value_rows).to_csv(output / "next_day_value_audit.csv", index=False)
6814 pd.DataFrame(physical_rows).to_csv(output / "physical_audit.csv", index=False)
6815
6816 consistency = {
6817     "stage": "C2-R1",
6818     "policy": (
6819         "single baseline-forecast day-ahead LP plus K=8 risk reserve "
6820         "and residual-matching 10-minute MPC"
6821     ),
6822     "forecast_mode_main_path": R1_DEFAULT_MODE.value,
6823     "risk_alpha_main_path": R1_DEFAULT_ALPHA,
6824     "scenario_k": FIXED_SCENARIO_K,
6825     "k_used_for": ["risk_reserve_R", "intra_day_residual_weights"],
6826     "k_not_used_for": [
6827         "scenario_specific_charge",
6828         "scenario_specific_discharge",
6829         "scenario_specific_soc",
6830         "full_path_scenario_storage_recourse",
6831     ],
6832     "calibration_closed_loop": False,
6833     "annual_run": False,
6834     "candidate_result2_xlsx": False,
6835     "validation_uses_full_day_actual_lp": False,
6836     "formal_execution_uses_full_day_actual_lp": False,
6837     "forecast_history_strictly_prior": True,
6838     "residual_pool_strictly_prior": True,
6839     "current_period_actuals_only": True,
6840     "soc_source_r1": "isolated E_INITIAL=6000 kWh; January warmup is deferred to C2-R3",
6841     "dates": list(PILOT_DATES),
6842     "elapsed_seconds": perf_counter() - started,
6843 }
6844 (output / "policy_consistency_audit.json").write_text(
6845     json.dumps(consistency, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6846 )
6847 report = {
6848     "status": "PASS" if all(row["pass"] for row in summaries) else "FAIL",
6849     "stage": "C2-R1",
6850     "days": summaries,
6851     "protected_outputs_unchanged": snapshot_protected() == before,
6852     "candidate_result2_xlsx": False,
6853 }
6854 (output / "validation.json").write_text(
6855     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6856 )
6857 lines = [
6858     "# Q2 政策一致重设计 · C2-R1",
6859     "",
6860     f"- 状态: **{report['status']}**",

```

```

6861         "- 本阶段只交付独立模块、单元测试和 2025-02-01、2025-06-21 两日闭环试运行。",
6862         "- 日前 LP 只在单一基准预测轨迹上锁定 q, 并施加  $q>R(\alpha)$ 。",
6863         "- K=8 只用于风险储备 R 与日内残差匹配权重, 不创建情景专属充放电/SOC 变量。",
6864         "- 日内只把当期实际负荷/光伏写入当前时段; 未来时段使用残差匹配点预测。",
6865         "- 未运行 14 日联合校准、未运行全年、未导出候选 result2.xlsx。",
6866         "- 两日初值均记录为孤立试算的 6000 kWh; 1 月预热与 2 月初 SOC 敏感性留到 C2-R3。",
6867         f"- 主路径预注册: m={R1_DEFAULT_MODE.value}, alpha={R1_DEFAULT_ALPHA}, K={FIXED_SCENARIO_K}。",
6868         "- MAE/覆盖率只写入 `forecast_diagnostics.csv`, 不作为选择准则。",
6869         "- `history_cutoff_date` 记录最后可用历史日, 例如 2 月 1 日决策记为 1 月 31 日。",
6870         "",
6871     ]
6872     for item in summaries:
6873         lines += [
6874             f"## {item['date']}",
6875             "",
6876             f"- 计划费 {item['planned_cost_yuan']:.6f} 元, 紧急费 {item['emergency_cost_yuan']:.6f} 元, "
6877             f"实际总成本 {item['total_cost_yuan']:.6f} 元。",
6878             f"- 未用计划电 {item['unused_plan_kwh']:.6f} kWh, 紧急购电 {item['emergency_kwh']:.6f} kWh。",
6879             f"- SOC {item['soc_start_kwh']:.6f} → {item['soc_end_kwh']:.6f} kWh。",
6880             f"- 最大平衡残差 {item['max_balance_residual_kwh']:.3e} kWh; 通过={item['pass']}".
6881             "",
6882         ]
6883     (output / "validation.md").write_text("\n".join(lines), encoding="utf-8")
6884
6885     after = snapshot_protected()
6886     if after != before:
6887         raise RuntimeError("Protected old Q2 outputs were modified")
6888     print(json.dumps({'status': report["status"], "output": str(output)}, ensure_ascii=False, indent=2))
6889
6890
6891 if __name__ == "__main__":
6892     main()
6893
6894 # ===== q2/run_q2_policy_consistent_r2.py =====
6895 #!/usr/bin/env python3
6896 """C2-R2: 12-candidate closed-loop rolling calibration. Never writes result2.xlsx."""
6897
6898 from __future__ import annotations
6899
6900 import hashlib
6901 import json
6902 import sys
6903 from pathlib import Path
6904 from time import perf_counter
6905
6906 import pandas as pd
6907
6908 ROOT = Path(__file__).resolve().parents[1]
6909 if str(ROOT) not in sys.path:
6910     sys.path.insert(0, str(ROOT))
6911
6912 from q2.config import ( # noqa: E402
6913     FIXED_SCENARIO_K,
6914     FULL_K8_RISK_OUTPUT_DIR,
6915     POLICY_CONSISTENT_OUTPUT_DIR,
6916     RISK_ALPHA_CANDIDATES,
6917 )
6918 from q2.data import load_q2_data # noqa: E402
6919 from q2.policy_consistent import ( # noqa: E402
6920     FORECAST_MODES,
6921     build_forecast_archive_mode,
6922     run_rolling_closed_loop_calibration,
6923 )
6924
6925
6926 def file_hash(path: Path) -> str | None:
6927     if not path.exists() or not path.is_file():
6928         return None

```

```

6929     digest = hashlib.sha256()
6930     with path.open("rb") as stream:
6931         for block in iter(lambda: stream.read(1024 * 1024), b""):
6932             digest.update(block)
6933     return digest.hexdigest()
6934
6935
6936 def snapshot_protected() -> dict[str, str | None]:
6937     protected = [ROOT / "output" / "result2.xlsx"]
6938     if FULL_K8_RISK_OUTPUT_DIR.exists():
6939         protected.extend(sorted(FULL_K8_RISK_OUTPUT_DIR.rglob("*")))
6940     return {str(path.relative_to(ROOT)): file_hash(path) for path in protected if path.is_file()}
6941
6942
6943 def main() -> None:
6944     started = perf_counter()
6945     output = POLICY_CONSISTENT_OUTPUT_DIR
6946     output.mkdir(parents=True, exist_ok=True)
6947     before = snapshot_protected()
6948
6949     data = load_q2_data()
6950     archives = {mode: build_forecast_archive_mode(data, mode) for mode in FORECAST_MODES}
6951     calibration_rows, diagnostic_rows, deployed_rows, cut_stats = run_rolling_closed_loop_calibration(
6952         data, archives, k=FIXED_SCENARIO_K, include_value_cuts=True
6953     )
6954     calibration = pd.DataFrame(calibration_rows)
6955     diagnostics = pd.DataFrame(diagnostic_rows)
6956     deployed = pd.DataFrame(deployed_rows)
6957     calibration.to_csv(output / "closed_loop_calibration.csv", index=False)
6958     diagnostics.to_csv(output / "forecast_diagnostics.csv", index=False)
6959     deployed.to_csv(output / "r2_deployed_daily.csv", index=False)
6960
6961     scored = calibration.loc[calibration["fallback_reason"].fillna("") == ""]
6962     selected = calibration.loc[calibration["selected"].astype(bool)]
6963     n_candidates = len(FORECAST_MODES) * len(RISK_ALPHA_CANDIDATES)
6964     scored_groups = scored.groupby("calibration_date").size() if len(scored) else pd.Series(dtype=int)
6965     consistency = {
6966         "stage": "C2-R2",
6967         "policy": (
6968             "single baseline-forecast day-ahead LP plus K=8 risk reserve "
6969             "and residual-matching 10-minute MPC"
6970         ),
6971         "n_forecast_modes": len(FORECAST_MODES),
6972         "risk_alpha_candidates": list(RISK_ALPHA_CANDIDATES),
6973         "n_candidates_per_block": n_candidates,
6974         "selection_criterion": "mean_actual_cost_sum_pq_plus_5pe",
6975         "tiebreak": "one_se_then_unused_plan_then_m1_then_smaller_alpha",
6976         "mae_is_diagnostic_only": True,
6977         "mae_used_for_selection": False,
6978         "calibration_closed_loop": True,
6979         "candidate_window_carries_own_soc": True,
6980         "window_start_soc_from_deployed_policy": True,
6981         "validation_uses_full_day_actual_lp": False,
6982         "formal_execution_uses_full_day_actual_lp": False,
6983         "calibration_and_execution_exclude_future_actuals": True,
6984         "include_value_cuts": True,
6985         "calibration_include_value_cuts": True,
6986         "deployment_include_value_cuts": True,
6987         "include_value_cuts_consistent": True,
6988         "value_cut_cache_entries": int(cut_stats["value_cut_cache_entries"]),
6989         "annual_run": False,
6990         "candidate_result2.xlsx": False,
6991         "k_used_for": ["risk_reserve_R", "intra_day_residual_weights"],
6992         "fallback_blocks": int((calibration["fallback_reason"].fillna("") != "").sum()),
6993         "scored_blocks": int(scored["calibration_date"].nunique()) if len(scored) else 0,
6994         "selected_blocks": int(len(selected)),
6995         "elapsed_seconds": perf_counter() - started,
6996     }

```

```

6997 (output / "policy_consistency_audit.json").write_text(
6998     json.dumps(consistency, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
6999 )
7000 status = (
7001     deployed["pass"].all()
7002     and not deployed["used_full_day_actual_lp"].any()
7003     and not deployed["future_actuals_in_optimizer"].any()
7004     and diagnostics["selection_role"].eq("diagnostic_only").all()
7005     and (scored_groups == n_candidates).all()
7006     and selected.groupby("calibration_date").size().eq(1).all()
7007     and calibration["include_value_cuts"].all()
7008     and deployed["include_value_cuts"].all()
7009 )
7010 report = {
7011     "status": "PASS" if bool(status) else "FAIL",
7012     "stage": "C2-R2",
7013     "n_candidates_per_block": n_candidates,
7014     "scored_blocks": consistency["scored_blocks"],
7015     "fallback_blocks": consistency["fallback_blocks"],
7016     "selected_pairs": selected[["calibration_date", "forecast_mode", "risk_alpha"]].to_dict(
7017         orient="records"
7018     ),
7019     "deployed_days": int(len(deployed)),
7020     "protected_outputs_unchanged": snapshot_protected() == before,
7021     "candidate_result2_xlsx": False,
7022     "mae_is_diagnostic_only": True,
7023     "include_value_cuts": True,
7024     "include_value_cuts_consistent": True,
7025 }
7026 (output / "validation.json").write_text(
7027     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7028 )
7029 lines = [
7030     "# Q2 政策一致重设计 · C2-R2",
7031     "",
7032     f"- 状态: **{report['status']}**",
7033     f"- 每个可评分校准块评估 {n_candidates} 组 ",
7034     f"min({m1,m2,m3}) × alpha({list(RISK_ALPHA_CANDIDATES)})",
7035     "- 主选择准则是验证窗口内闭环实际成本均值  $\sum(pq+5pe)$ ;",
7036     "- 一倍标准误内再按未用计划电、m1、较小 alpha 打破平局。",
7037     "- MAE / 净负荷 MAE / 覆盖率只写入 `forecast_diagnostics.csv`, 不参与选择。",
7038     "- 每个候选在 14 日窗口内连续传递自己的 SOC; 窗口初值取已部署主政策记录的 SOC。",
7039     "- 校准评分与获选策略部署均纳入次日基准预测价值割; 同一日、同一 (m, alpha, K) 的割缓存复用。",
7040     "- 校准与部署均未把验证日未来实际值输入优化问题, 也未调用全视域真实路径 LP。",
7041     "- 未导出候选 result2.xlsx; 全年台账、K 敏感性与 2 月初 SOC 敏感性留到 C2-R3。",
7042     f"- 回退块 {consistency['fallback_blocks']} 个, 评分块 {consistency['scored_blocks']} 个。",
7043     "",
7044     "## 各块选定 (m, alpha)",
7045     "",
7046 ]
7047 for row in selected.itertuples():
7048     lines.append(
7049         f"- {row.calibration_date} 至 {row.effective_end_date}: "
7050         f"{row.forecast_mode}, alpha={row.risk_alpha}; "
7051         f"回退={row.fallback_reason or '无'}"
7052     )
7053 (output / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
7054
7055 after = snapshot_protected()
7056 if after != before:
7057     raise RuntimeError("Protected old Q2 outputs were modified")
7058 if not status:
7059     raise AssertionError("C2-R2 validation failed")
7060 print(json.dumps({"status": report["status"], "output": str(output)}, ensure_ascii=False, indent=2))
7061
7062
7063 if __name__ == "__main__":
7064     main()

```

```

7065
7066 # ===== q2/run_q2_policy_consistent_r3.py =====
7067 #!/usr/bin/env python3
7068 """C2-R3: official year path, January/Feb-Dec split, SOC and K sensitivities.
7069
7070 Never writes result2.xlsx. Frozen (m, alpha) calendar comes from C2-R2.
7071 """
7072
7073 from __future__ import annotations
7074
7075 import hashlib
7076 import json
7077 import shutil
7078 import sys
7079 from pathlib import Path
7080 from time import perf_counter
7081
7082 import pandas as pd
7083
7084 ROOT = Path(__file__).resolve().parents[1]
7085 if str(ROOT) not in sys.path:
7086     sys.path.insert(0, str(ROOT))
7087
7088 from q2.config import ( # noqa: E402
7089     E_INITIAL_KWH,
7090     FIG_POLICY_CONSISTENT_DIR,
7091     FIXED_SCENARIO_K,
7092     FULL_K8_RISK_OUTPUT_DIR,
7093     K_SENSITIVITY_CANDIDATES,
7094     OFFICIAL_OUTPUT_START,
7095     POLICY_CONSISTENT_OUTPUT_DIR,
7096 )
7097 from q2.data import load_q2_data # noqa: E402
7098 from q2.plot_q2_policy_consistent_r3 import plot_r3_comparison # noqa: E402
7099 from q2.policy_consistent import ( # noqa: E402
7100     FORECAST_MODES,
7101     PERIOD_FEB_DEC_OUTPUT,
7102     PERIOD_JANUARY_WARMUP,
7103     aggregate_cost_row,
7104     build_forecast_archive_mode,
7105     k_sensitivity_conclusion,
7106     output_period,
7107     run_frozen_policy_path,
7108     selected_calendar_for_dates,
7109 )
7110
7111
7112 R1_COPIES = (
7113     ("forecast_policy_audit.csv", "r1_forecast_policy_audit.csv"),
7114     ("physical_audit.csv", "r1_physical_audit.csv"),
7115     ("next_day_value_audit.csv", "r1_next_day_value_audit.csv"),
7116 )
7117
7118
7119 def file_hash(path: Path) -> str | None:
7120     if not path.exists() or not path.is_file():
7121         return None
7122     digest = hashlib.sha256()
7123     with path.open("rb") as stream:
7124         for block in iter(lambda: stream.read(1024 * 1024), b""):
7125             digest.update(block)
7126     return digest.hexdigest()
7127
7128
7129 def snapshot_protected() -> dict[str, str | None]:
7130     protected = [ROOT / "output" / "result2.xlsx"]
7131     if FULL_K8_RISK_OUTPUT_DIR.exists():
7132         protected.extend(sorted(FULL_K8_RISK_OUTPUT_DIR.rglob("*")))

```

```

7133     return {str(path.relative_to(ROOT)): file_hash(path) for path in protected if path.is_file()}
7134
7135
7136 def preserve_r1_artifacts(output: Path) -> None:
7137     for source_name, dest_name in R1_COPIES:
7138         source = output / source_name
7139         dest = output / dest_name
7140         if source.exists() and not dest.exists():
7141             shutil.copy2(source, dest)
7142     r1_dispatch = output / "r1_dispatch_daily"
7143     dispatch = output / "dispatch_daily"
7144     if dispatch.exists() and not r1_dispatch.exists():
7145         r1_dispatch.mkdir(parents=True, exist_ok=True)
7146         for path in dispatch.glob("dispatch_*.csv"):
7147             shutil.copy2(path, r1_dispatch / path.name)
7148
7149
7150 def split_period(daily: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
7151     january = daily.loc[daily["period"] == PERIOD_JANUARY_WARMUP].copy()
7152     feb_dec = daily.loc[daily["period"] == PERIOD_FEB_DEC_OUTPUT].copy()
7153     return january, feb_dec
7154
7155
7156 def main() -> None:
7157     started = perf_counter()
7158     output = POLICY_CONSISTENT_OUTPUT_DIR
7159     output.mkdir(parents=True, exist_ok=True)
7160     dispatch_dir = output / "dispatch_daily"
7161     dispatch_dir.mkdir(parents=True, exist_ok=True)
7162     before = snapshot_protected()
7163     preserve_r1_artifacts(output)
7164
7165     calibration_path = output / "closed_loop_calibration.csv"
7166     if not calibration_path.exists():
7167         raise FileNotFoundError("C2-R3 requires closed_loop_calibration.csv from C2-R2")
7168
7169     data = load_q2_data()
7170     archives = {mode: build_forecast_archive_mode(data, mode) for mode in FORECAST_MODES}
7171     calendar = selected_calendar_for_dates(
7172         pd.read_csv(calibration_path), data.dates
7173     )
7174     feb_index = int(data.dates.get_loc(pd.Timestamp(OFFICIAL_OUTPUT_START)))
7175     year_end = len(data.dates) - 1
7176     cut_cache: dict = {}
7177
7178     print("C2-R3 main path K=8 with next-day value cuts", flush=True)
7179     main_daily, main_forecast, main_physical, main_value = run_frozen_policy_path(
7180         data,
7181         archives,
7182         calendar,
7183         start_index=0,
7184         end_index=year_end,
7185         initial_soc=E_INITIAL_KWH,
7186         requested_k=FIXED_SCENARIO_K,
7187         soc_source="january_1_E_INITIAL_6000_then_inherited",
7188         include_value_cuts=True,
7189         dispatch_dir=dispatch_dir,
7190         cut_cache=cut_cache,
7191         progress_label="C2-R3-main",
7192     )
7193     daily = pd.DataFrame(main_daily)
7194     if list(daily["date"]) != [stamp.strftime("%Y-%m-%d") for stamp in data.dates]:
7195         raise AssertionError("main path does not cover the full year")
7196     if abs(float(daily["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) > 1e-8:
7197         raise AssertionError("main path must start at 6000 kWh on 2025-01-01")
7198     january, feb_dec = split_period(daily)
7199     if len(january) != 31 or len(feb_dec) != 334:
7200         raise AssertionError("January warmup / Feb-Dec split is not 31 + 334")

```

```

7201 inherited_feb_soc = float(january["soc_end_kwh"].iloc[-1])
7202 if abs(float(feb_dec["soc_start_kwh"].iloc[0]) - inherited_feb_soc) > 1e-8:
7203     raise AssertionError("Feb 1 SOC must inherit January 31 end SOC")
7204 main_elapsed = perf_counter() - started
7205
7206 print(
7207     f"C2-R3 Feb 1 SOC sensitivity: inherited {inherited_feb_soc:.6f} vs registered {E_INITIAL_KWH:.1f}",
7208     flush=True,
7209 )
7210 alt_daily_rows, _alt_forecast, _alt_physical, _alt_value = run_frozen_policy_path(
7211     data,
7212     archives,
7213     calendar,
7214     start_index=feb_index,
7215     end_index=year_end,
7216     initial_soc=E_INITIAL_KWH,
7217     requested_k=FIXED_SCENARIO_K,
7218     soc_source="registered_boundary_E_INITIAL_6000",
7219     include_value_cuts=True,
7220     dispatch_dir=None,
7221     cut_cache=cut_cache,
7222     progress_label="C2-R3-feb-soc",
7223 )
7224 alt_feb_dec = pd.DataFrame(alt_daily_rows)
7225 if list(alt_feb_dec["date"]) != list(feb_dec["date"]):
7226     raise AssertionError("Feb SOC sensitivity dates differ from the main Feb-Dec path")
7227 if abs(float(alt_feb_dec["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) > 1e-8:
7228     raise AssertionError("registered Feb 1 boundary is not 6000 kWh")
7229
7230 k_daily: dict[int, pd.DataFrame] = {FIXED_SCENARIO_K: daily}
7231 k_elapsed: dict[int, float] = {FIXED_SCENARIO_K: main_elapsed}
7232 for k in K_SENSITIVITY_CANDIDATES:
7233     if k == FIXED_SCENARIO_K:
7234         continue
7235     path_started = perf_counter()
7236     print(f"C2-R3 K sensitivity path K={k}", flush=True)
7237     rows, _forecast, _physical, _value = run_frozen_policy_path(
7238         data,
7239         archives,
7240         calendar,
7241         start_index=0,
7242         end_index=year_end,
7243         initial_soc=E_INITIAL_KWH,
7244         requested_k=k,
7245         soc_source="january_1_E_INITIAL_6000_then_inherited",
7246         include_value_cuts=True,
7247         dispatch_dir=None,
7248         cut_cache=cut_cache,
7249         progress_label=f"C2-R3-K{k}",
7250     )
7251     k_daily[k] = pd.DataFrame(rows)
7252     k_elapsed[k] = perf_counter() - path_started
7253
7254 k_rows = []
7255 for k in K_SENSITIVITY_CANDIDATES:
7256     frame = k_daily[k]
7257     _, k_feb_dec = split_period(frame)
7258     k_rows.append(
7259         aggregate_cost_row(
7260             k_feb_dec,
7261             label=f"feb_dec_k{k}",
7262             extra={
7263                 "scenario_k": int(k),
7264                 "period": PERIOD_FEB_DEC_OUTPUT,
7265                 "calendar_frozen_from_r2": True,
7266                 "include_value_cuts": True,
7267                 "elapsed_seconds": float(k_elapsed[k]),
7268                 "mean_effective_k": float(k_feb_dec["scenario_k"].mean()),

```

```

7269         },
7270     )
7271 )
7272 conclusion = k_sensitivity_conclusion(k_rows)
7273 for row in k_rows:
7274     row["conclusion"] = conclusion
7275
7276 cost_rows = [
7277     aggregate_cost_row(
7278         january,
7279         label="january_warmup_main",
7280         extra={
7281             "period": PERIOD_JANUARY_WARMUP,
7282             "scenario_k": FIXED_SCENARIO_K,
7283             "soc_source": "january_1_E_INITIAL_6000_then_inherited",
7284         },
7285     ),
7286     aggregate_cost_row(
7287         feb_dec,
7288         label="feb_dec_inherited_warmup_soc",
7289         extra={
7290             "period": PERIOD_FEB_DEC_OUTPUT,
7291             "scenario_k": FIXED_SCENARIO_K,
7292             "soc_source": "inherited_january_warmup",
7293             "feb1_soc_start_kwh": inherited_feb_soc,
7294         },
7295     ),
7296     aggregate_cost_row(
7297         alt_feb_dec,
7298         label="feb_dec_registered_boundary_6000",
7299         extra={
7300             "period": PERIOD_FEB_DEC_OUTPUT,
7301             "scenario_k": FIXED_SCENARIO_K,
7302             "soc_source": "registered_boundary_E_INITIAL_6000",
7303             "feb1_soc_start_kwh": E_INITIAL_KWH,
7304             "only_registered_boundary_changed": True,
7305         },
7306     ),
7307     aggregate_cost_row(
7308         daily,
7309         label="full_year_including_warmup",
7310         extra={
7311             "period": "full_year",
7312             "scenario_k": FIXED_SCENARIO_K,
7313         },
7314     ),
7315 ]
7316
7317 soc_sensitivity = pd.DataFrame(
7318     [
7319         {
7320             "soc_source": "inherited_january_warmup",
7321             "feb1_soc_start_kwh": inherited_feb_soc,
7322             "dec31_soc_end_kwh": float(feb_dec["soc_end_kwh"].iloc[-1]),
7323             "n_days": int(len(feb_dec)),
7324             "start_date": str(feb_dec["date"].iloc[0]),
7325             "end_date": str(feb_dec["date"].iloc[-1]),
7326             "forecast_modes": ";".join(sorted(feb_dec["forecast_mode"].unique())),
7327             "requested_k": FIXED_SCENARIO_K,
7328             "planned_cost_yuan": float(feb_dec["planned_cost_yuan"].sum()),
7329             "emergency_cost_yuan": float(feb_dec["emergency_cost_yuan"].sum()),
7330             "total_cost_yuan": float(feb_dec["total_cost_yuan"].sum()),
7331             "emergency_kwh": float(feb_dec["emergency_kwh"].sum()),
7332             "unused_plan_kwh": float(feb_dec["unused_plan_kwh"].sum()),
7333             "curtailment_kwh": float(feb_dec["curtailment_kwh"].sum()),
7334             "only_registered_boundary_changed": False,
7335             "main_path": True,
7336         },

```



```

7337         {
7338             "soc_source": "registered_boundary_E_INITIAL_6000",
7339             "feb1_soc_start_kwh": E_INITIAL_KWH,
7340             "dec31_soc_end_kwh": float(alt_feb_dec["soc_end_kwh"].iloc[-1]),
7341             "n_days": int(len(alt_feb_dec)),
7342             "start_date": str(alt_feb_dec["date"].iloc[0]),
7343             "end_date": str(alt_feb_dec["date"].iloc[-1]),
7344             "forecast_modes": ";".join(sorted(alt_feb_dec["forecast_mode"].unique())),
7345             "requested_k": FIXED_SCENARIO_K,
7346             "planned_cost_yuan": float(alt_feb_dec["planned_cost_yuan"].sum()),
7347             "emergency_cost_yuan": float(alt_feb_dec["emergency_cost_yuan"].sum()),
7348             "total_cost_yuan": float(alt_feb_dec["total_cost_yuan"].sum()),
7349             "emergency_kwh": float(alt_feb_dec["emergency_kwh"].sum()),
7350             "unused_plan_kwh": float(alt_feb_dec["unused_plan_kwh"].sum()),
7351             "curtailment_kwh": float(alt_feb_dec["curtailment_kwh"].sum()),
7352             "only_registered_boundary_changed": True,
7353             "main_path": False,
7354         },
7355     ]
7356 )
7357
7358 daily.to_csv(output / "q2_redesign_daily_summary.csv", index=False)
7359 january.to_csv(output / "january_warmup_summary.csv", index=False)
7360 feb_dec.to_csv(output / "feb_dec_daily_summary.csv", index=False)
7361 alt_feb_dec.to_csv(output / "feb_soc_sensitivity_daily.csv", index=False)
7362 soc_sensitivity.to_csv(output / "feb_soc_sensitivity.csv", index=False)
7363 pd.DataFrame(k_rows).to_csv(output / "k_sensitivity.csv", index=False)
7364 pd.DataFrame(cost_rows).to_csv(output / "cost_breakdown.csv", index=False)
7365 pd.DataFrame(main_forecast).to_csv(output / "forecast_policy_audit.csv", index=False)
7366 pd.DataFrame(main_physical).to_csv(output / "physical_audit.csv", index=False)
7367 pd.DataFrame(main_value).to_csv(output / "next_day_value_audit.csv", index=False)
7368 pd.DataFrame(
7369     [
7370         {
7371             "date": policy.date,
7372             "forecast_mode": policy.forecast_mode.value,
7373             "risk_alpha": policy.risk_alpha,
7374             "fallback_reason": policy.fallback_reason,
7375             "calibration_date": policy.calibration_date,
7376             "effective_start_date": policy.effective_start_date,
7377             "effective_end_date": policy.effective_end_date,
7378             "period": output_period(policy.date),
7379         }
7380         for policy in calendar
7381     ]
7382 ).to_csv(output / "frozen_policy_calendar.csv", index=False)
7383
7384 figures = plot_r3_comparison(
7385     daily,
7386     pd.read_csv(FULL_K8_RISK_OUTPUT_DIR / "daily_summary.csv"),
7387     FIG_POLICY_CONSISTENT_DIR,
7388 )
7389
7390 after = snapshot_protected()
7391 protected_ok = after == before
7392 status = bool(
7393     daily["pass"].all()
7394     and not daily["used_full_day_actual_lp"].any()
7395     and not daily["future_actuals_in_optimizer"].any()
7396     and not alt_feb_dec["used_full_day_actual_lp"].any()
7397     and protected_ok
7398     and conclusion
7399 )
7400 consistency = {
7401     "stage": "C2-R3",
7402     "policy": (
7403         "single baseline-forecast day-ahead LP plus K=8 risk reserve "
7404         "and residual-matching 10-minute MPC"

```

```

7405     ),
7406     "k_used_for": ["risk_reserve_R", "intra_day_residual_weights"],
7407     "k_not_used_for": [
7408         "scenario_specific_charge",
7409         "scenario_specific_discharge",
7410         "scenario_specific_soc",
7411         "full_path_scenario_storage_recourse",
7412     ],
7413     "calendar_frozen_from_r2": True,
7414     "calibration_closed_loop": True,
7415     "mae_is_diagnostic_only": True,
7416     "mae_used_for_selection": False,
7417     "include_value_cuts": True,
7418     "calibration_include_value_cuts": True,
7419     "deployment_include_value_cuts": True,
7420     "annual_run_include_value_cuts": True,
7421     "include_value_cuts_consistent": True,
7422     "main_path_initial_soc_kwh": E_INITIAL_KWH,
7423     "main_path_initial_rule_unchanged": True,
7424     "january_warmup_days": int(len(january)),
7425     "feb_dec_days": int(len(feb_dec)),
7426     "february_1_inherited_soc_kwh": inherited_feb_soc,
7427     "february_1_sensitivity_soc_kwh": E_INITIAL_KWH,
7428     "k_sensitivity_candidates": list(K_SENSITIVITY_CANDIDATES),
7429     "k_sensitivity_conclusion": conclusion,
7430     "validation_uses_full_day_actual_lp": False,
7431     "formal_execution_uses_full_day_actual_lp": False,
7432     "calibration_and_execution_exclude_future_actuals": True,
7433     "forecast_history_strictly_prior": True,
7434     "residual_pool_strictly_prior": True,
7435     "current_period_actuals_only": True,
7436     "annual_run": True,
7437     "candidate_result2_xlsx": False,
7438     "elapsed_seconds": perf_counter() - started,
7439 }
7440 (output / "policy_consistency_audit.json").write_text(
7441     json.dumps(consistency, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7442 )
7443 report = {
7444     "status": "PASS" if status else "FAIL",
7445     "stage": "C2-R3",
7446     "january_warmup": cost_rows[0],
7447     "feb_dec_inherited": cost_rows[1],
7448     "feb_dec_soc_sensitivity": cost_rows[2],
7449     "k_sensitivity_conclusion": conclusion,
7450     "figures": figures,
7451     "protected_outputs_unchanged": protected_ok,
7452     "candidate_result2_xlsx": False,
7453     "annual_run": True,
7454     "include_value_cuts": True,
7455     "include_value_cuts_consistent": True,
7456 }
7457 (output / "validation.json").write_text(
7458     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7459 )
7460 (output / "run_metadata.json").write_text(
7461     json.dumps(
7462         {
7463             "stage": "C2-R3",
7464             "elapsed_seconds": consistency["elapsed_seconds"],
7465             "n_cut_cache_entries": len(cut_cache),
7466             "main_path_days": int(len(daily)),
7467             "dispatch_dir": str(dispatch_dir.relative_to(ROOT)),
7468             "figure_dir": str(FIG_POLICY_CONSISTENT_DIR.relative_to(ROOT)),
7469         },
7470         ensure_ascii=False,
7471         indent=2,
7472     )

```

```

7473         + "\n",
7474         encoding="utf-8",
7475     )
7476     lines = [
7477         "# Q2 政策一致重设计 · C2-R3",
7478         "",
7479         f"- 状态: **{report['status']}**",
7480         "- 全年主路径从 2025-01-01 的 6000 kWh 起步, 冻结 C2-R2 选定的 (m, alpha) 日历, "
7481         "并加入次日基准预测价值割。",
7482         "- 校准评分、获选部署与全年正式运行的 `include_value_cuts=true` 一致。",
7483         "- 1 月预热与 2—12 月正式输出分栏; 主路径 2 月 1 日 SOC 继承 1 月 31 日末值, "
7484         "不修改年初值规则。",
7485         f"- 1 月预热实际成本 {cost_rows[0]['total_cost_yuan']:.2f} 元; "
7486         f"回退预测日 {int(january['load_fallback'].sum() + january['pv_fallback'].sum())} 条"
7487         " (负荷/光伏回退计数)。",
7488         f"- 2 月 1 日继承 SOC {inherited_feb_soc:.6f} kWh; "
7489         f"2—12 月正式成本 {cost_rows[1]['total_cost_yuan']:.2f} 元, "
7490         f"紧急购电 {cost_rows[1]['emergency_kwh']:.2f} kWh。",
7491         f"- 2 月初登记边界敏感性 (6000 kWh) 2—12 月成本 "
7492         f"{cost_rows[2]['total_cost_yuan']:.2f} 元。",
7493         f"- Kin{{4,8,12}} 闭环敏感性结论: `{conclusion}`。",
7494         "- 未导出候选 result2.xlsx; 正式附件留到 C2-R4。",
7495         f"- 对照图写入 `{FIG_POLICY_CONSISTENT_DIR.relative_to(ROOT)}/`。",
7496         "",
7497         "## 1 月预热",
7498         "",
7499         f"- 日数 {len(january)}, 初值 {float(january['soc_start_kwh'].iloc[0]):.6f} kWh, "
7500         f"2 月 1 日初值 {inherited_feb_soc:.6f} kWh。",
7501         f"- 负荷附件1回退 {int(january['load_fallback'].sum())} 日, "
7502         f"光伏附件1回退 {int(january['pv_fallback'].sum())} 日。",
7503         "",
7504         "## K 敏感性 (2—12 月, 冻结日历)",
7505         "",
7506     ]
7507     for row in k_rows:
7508         lines.append(
7509             f"- K={row['scenario_k']}: 总成本 {row['total_cost_yuan']:.2f} 元, "
7510             f"紧急购电 {row['emergency_kwh']:.2f} kWh。"
7511         )
7512     (output / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
7513
7514     if after != before:
7515         raise RuntimeError("Protected old Q2 outputs were modified")
7516     if not status:
7517         raise AssertionError("C2-R3 validation failed")
7518     print(json.dumps({"status": report["status"], "output": str(output)}, ensure_ascii=False, indent=2))
7519
7520
7521 if __name__ == "__main__":
7522     main()
7523
7524 # ===== q2/run_q2_policy_consistent_r4.py =====
7525 #!/usr/bin/env python3
7526 """C2-R4: candidate result2.xlsx, final Q2 figures, and cell-by-cell audit.
7527
7528 Never overwrites output/result2.xlsx. Writes only under output/q2_policy_consistent/
7529 and fig/q2_policy_consistent/.
7530 """
7531
7532 from __future__ import annotations
7533
7534 import hashlib
7535 import json
7536 import sys
7537 from pathlib import Path
7538 from time import perf_counter
7539
7540 import pandas as pd

```

```

7541
7542 ROOT = Path(__file__).resolve().parents[1]
7543 if str(ROOT) not in sys.path:
7544     sys.path.insert(0, str(ROOT))
7545
7546 from q2.config import ( # noqa: E402
7547     CANDIDATE_RESULT2,
7548     E_INITIAL_KWH,
7549     FIG_POLICY_CONSISTENT_DIR,
7550     FIG_Q2_FINAL_DIR,
7551     FIXED_SCENARIO_K,
7552     FULL_K8_RISK_OUTPUT_DIR,
7553     NUMERIC_TOL,
7554     OFFICIAL_OUTPUT_START,
7555     POLICY_CONSISTENT_OUTPUT_DIR,
7556     RESULT2_SIGNED_OFF_BACKUP,
7557     SIGNED_OFF_RESULT2,
7558     SIGNED_OFF_RESULT2_SHA256,
7559 )
7560 from q2.data import load_q2_data # noqa: E402
7561 from q2.export_result2 import ( # noqa: E402
7562     N_OUTPUT_DAYS,
7563     audit_candidate_result2,
7564     file_sha256,
7565     preserve_signed_off_backup,
7566     write_candidate_result2,
7567     write_result2_audit,
7568 )
7569 from q2.plot_q2_policy_consistent_r3 import plot_r4_final_figures # noqa: E402
7570 from q2.policy_consistent import PERIOD_FEB_DEC_OUTPUT, PERIOD_JANUARY_WARMUP # noqa: E402
7571
7572
7573 def snapshot_protected() -> dict[str, str | None]:
7574     protected = [SIGNED_OFF_RESULT2]
7575     if FULL_K8_RISK_OUTPUT_DIR.exists():
7576         protected.extend(sorted(FULL_K8_RISK_OUTPUT_DIR.rglob("*")))
7577     if FIG_Q2_FINAL_DIR.exists():
7578         protected.extend(sorted(FIG_Q2_FINAL_DIR.rglob("*")))
7579     hashes: dict[str, str | None] = {}
7580     for path in protected:
7581         if path.is_file():
7582             digest = hashlib.sha256()
7583             with path.open("rb") as stream:
7584                 for block in iter(lambda: stream.read(1024 * 1024), b""):
7585                     digest.update(block)
7586             hashes[str(path.relative_to(ROOT))] = digest.hexdigest()
7587     return hashes
7588
7589
7590 def require_locked_main_scheme(daily: pd.DataFrame, january: pd.DataFrame, feb_dec: pd.DataFrame) -> float:
7591     if int(feb_dec["scenario_k"].nunique()) != 1 or int(feb_dec["scenario_k"].iloc[0]) != FIXED_SCENARIO_K:
7592         raise AssertionError("official Feb-Dec path is not K=8")
7593     if not bool(feb_dec["include_value_cuts"].all()) or not bool(january["include_value_cuts"].all()):
7594         raise AssertionError("official year path must keep next-day value cuts")
7595     if abs(float(daily["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) > NUMERIC_TOL:
7596         raise AssertionError("year path does not start at 6000 kWh on 2025-01-01")
7597     jan_dates = pd.to_datetime(january["date"])
7598     feb_dates = pd.to_datetime(feb_dec["date"])
7599     if jan_dates.iloc[0] != pd.Timestamp("2025-01-01") or jan_dates.iloc[-1] != pd.Timestamp("2025-01-31"):
7600         raise AssertionError("January warmup window mismatch")
7601     if feb_dates.iloc[0] != pd.Timestamp(OFFICIAL_OUTPUT_START) or feb_dates.iloc[-1] != pd.Timestamp("2025-12-31"):
7602         raise AssertionError("official output window mismatch")
7603     if len(january) != 31 or len(feb_dec) != N_OUTPUT_DAYS:
7604         raise AssertionError("January/Feb-Dec day counts mismatch")
7605     inherited = float(january["soc_end_kwh"].iloc[-1])
7606     if abs(float(feb_dec["soc_start_kwh"].iloc[0]) - inherited) > NUMERIC_TOL:
7607         raise AssertionError("Feb 1 does not inherit January 31 SOC")
7608     if (january["period"] != PERIOD_JANUARY_WARMUP).any() or (feb_dec["period"] != PERIOD_FEB_DEC_OUTPUT).any():

```

```

7609         raise AssertionError("period labels mismatch")
7610     return inherited
7611
7612
7613 def main() -> None:
7614     started = perf_counter()
7615     output = POLICY_CONSISTENT_OUTPUT_DIR
7616     dispatch_dir = output / "dispatch_daily"
7617     before = snapshot_protected()
7618     signed_hash = preserve_signed_off_backup()
7619     data = load_q2_data()
7620     daily = pd.read_csv(output / "q2_redesign_daily_summary.csv")
7621     january = pd.read_csv(output / "january_warmup_summary.csv")
7622     feb_dec = pd.read_csv(output / "feb_dec_daily_summary.csv")
7623     calibration = pd.read_csv(output / "closed_loop_calibration.csv")
7624     k_table = pd.read_csv(output / "k_sensitivity.csv")
7625     inherited = require_locked_main_scheme(daily, january, feb_dec)
7626     conclusion = str(k_table["conclusion"].iloc[0])
7627     if conclusion != "retain_k8_stable":
7628         raise AssertionError("K=8 is not retained as the stable pre-registered scheme")
7629     if not dispatch_dir.exists():
7630         raise FileNotFoundError("C2-R4 requires the C2-R3 dispatch_daily archive")
7631
7632     print("C2-R4 writing candidate result2.xlsx under output/q2_policy_consistent/", flush=True)
7633     write_candidate_result2(CANDIDATE_RESULT2, dispatch_dir, feb_dec, data)
7634     audit = audit_candidate_result2(CANDIDATE_RESULT2, dispatch_dir, feb_dec, data, inherited)
7635     write_result2_audit(audit, output)
7636     if not audit["summary"]["pass"]:
7637         raise AssertionError("candidate result2 cell audit failed")
7638
7639     figures = plot_r4_final_figures(
7640         daily,
7641         pd.read_csv(FULL_K8_RISK_OUTPUT_DIR / "daily_summary.csv"),
7642         calibration,
7643         dispatch_dir,
7644         data.price,
7645         fig_dir=FIG_POLICY_CONSISTENT_DIR,
7646     )
7647
7648     after = snapshot_protected()
7649     protected_ok = after == before and signed_hash == SIGNED_OFF_RESULT2_SHA256
7650     if file_sha256(SIGNED_OFF_RESULT2) != SIGNED_OFF_RESULT2_SHA256:
7651         raise RuntimeError("signed-off output/result2.xlsx changed during C2-R4")
7652     if file_sha256(RESET2_SIGNED_OFF_BACKUP) != SIGNED_OFF_RESULT2_SHA256:
7653         raise RuntimeError("signed-off result2 backup is not bit-identical")
7654     if CANDIDATE_RESULT2.resolve() == SIGNED_OFF_RESULT2.resolve():
7655         raise RuntimeError("candidate path collapsed onto the signed-off file")
7656
7657     previous = json.loads((output / "policy_consistency_audit.json").read_text(encoding="utf-8"))
7658     previous.update(
7659         {
7660             "stage": "C2-R4",
7661             "candidate_result2_excel": True,
7662             "candidate_result2_path": str(CANDIDATE_RESULT2.relative_to(ROOT)),
7663             "signed_off_result2_path": str(SIGNED_OFF_RESULT2.relative_to(ROOT)),
7664             "signed_off_result2_sha256": SIGNED_OFF_RESULT2_SHA256,
7665             "signed_off_result2_unchanged": protected_ok,
7666             "k8_paper_language": "预注册且对 K=4/12 稳定的简洁主方案",
7667             "k8_not_cost_optimal": True,
7668             "old_full_path_role": "信息更强的近似对照",
7669             "r4_elapsed_seconds": perf_counter() - started,
7670         }
7671     )
7672     (output / "policy_consistency_audit.json").write_text(
7673         json.dumps(previous, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7674     )
7675
7676     previous_report = json.loads((output / "validation.json").read_text(encoding="utf-8"))

```

```

7677 report = {
7678     **previous_report,
7679     "status": "PASS" if audit["summary"]["pass"] and protected_ok else "FAIL",
7680     "stage": "C2-R4",
7681     "candidate_result2_xlsx": True,
7682     "candidate_result2_path": str(CANDIDATE_RESULT2.relative_to(ROOT)),
7683     "signed_off_result2_unchanged": protected_ok,
7684     "signed_off_result2_sha256": SIGNED_OFF_RESULT2.SHA256,
7685     "result2_cell_audit_pass": bool(audit["summary"]["pass"]),
7686     "result2_max_q_abs_kwh": audit["summary"]["max_q_abs_kwh"],
7687     "result2_max_planned_cost_abs_yuan": audit["summary"]["max_planned_cost_abs_yuan"],
7688     "result2_max_emergency_recon_abs_kwh": audit["summary"]["max_emergency_recon_abs_kwh"],
7689     "result2_max_soc_abs_kwh": audit["summary"]["max_soc_abs_kwh"],
7690     "figures": figures,
7691     "k8_paper_language": "预注册且对 K=4/12 稳定的简洁主方案",
7692     "k8_not_cost_optimal": True,
7693     "old_full_path_role": "信息更强的近似对照",
7694     "protected_outputs_unchanged": protected_ok,
7695 }
7696 (output / "validation.json").write_text(
7697     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7698 )
7699 metadata = json.loads((output / "run_metadata.json").read_text(encoding="utf-8"))
7700 metadata.update(
7701     {
7702         "stage": "C2-R4",
7703         "candidate_result2": str(CANDIDATE_RESULT2.relative_to(ROOT)),
7704         "signed_off_backup": str(RESULT2_SIGNED_OFF_BACKUP.relative_to(ROOT)),
7705         "r4_elapsed_seconds": previous["r4_elapsed_seconds"],
7706         "figure_dir": str(FIG_POLICY_CONSISTENT_DIR.relative_to(ROOT)),
7707     }
7708 )
7709 (output / "run_metadata.json").write_text(
7710     json.dumps(metadata, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
7711 )
7712
7713 jan_cost = float(january["total_cost_yuan"].sum())
7714 feb_cost = float(feb_dec["total_cost_yuan"].sum())
7715 feb_emergency = float(feb_dec["emergency_kwh"].sum())
7716 lines = [
7717     "# Q2 政策一致重设计 · C2-R4",
7718     "",
7719     f"- 状态: **{report['status']}**",
7720     "- 正式主方案固定为: K=8; C2-R2 冻结的 (m, alpha) 日历; 校准、部署与全年路径均启用次日价值制; "
7721     "全年从 2025-01-01 的 6000 kWh 起步, 1 月仅预热, 2—12 月正式输出继承 1 月末 SOC。",
7722     "- **K=8 表述为「预注册且对 K=4/12 稳定的简洁主方案」, 不得称为成本最优。**",
7723     "- **旧全路径 K=8 风险方案仅作信息更强的近似对照, 不是正式答卷。**",
7724     f"- 1 月预热实际成本 {jan_cost:.2f} 元; 2 月 1 日继承 SOC {inherited:.6f} kWh。",
7725     f"- 2—12 月正式成本 {feb_cost:.2f} 元, 紧急购电 {feb_emergency:.2f} kWh。",
7726     f"- Kin{{4,8,12}} 闭环敏感性结论: `{conclusion}`。",
7727     f"- 候选附件写入 `{CANDIDATE_RESULT2.relative_to(ROOT)}`; "
7728     f"- 旧签收文件 `{SIGNED_OFF_RESULT2.relative_to(ROOT)}` 未覆盖, "
7729     f"- 可回退副本 `{RESULT2_SIGNED_OFF_BACKUP.relative_to(ROOT)}`。",
7730     f"- 旧 result2.xlsx SHA-256: `{SIGNED_OFF_RESULT2.SHA256}`。",
7731     f"- 逐格对账: `result2_cell_audit.json` / `result2_cell_audit.csv`; pass={audit['summary']['pass']}",
7732     f"- 最终图写入 `{FIG_POLICY_CONSISTENT_DIR.relative_to(ROOT)}`/, 未改写 `fig/q2_final`。",
7733     "",
7734     "## 候选附件对账",
7735     "",
7736     f"- 日期: 2 月 1 日至 12 月 31 日, 共 {N_OUTPUT_DAYS} 日。",
7737     "- 每行 144 时段, 按附件原始行序写入, 不旋转。",
7738     "- 计划购电量/全天购电费与台账 `planned_q_kwh` / `planned_cost_yuan` 一致; 单位 kWh / 元。",
7739     "- 紧急购电量按连续正量合并时段, 日合计与台账 `emergency_kwh` 一致。",
7740     "- 充放电量按 4 小时块汇总; 00:00 / 24:00 储电量与日初/日末 SOC 一致, 且跨日连续。",
7741     "- 功率上限、能量平衡、x<=q 与全年物理审计一致。",
7742     "",
7743     "## 论文口径",
7744     "",

```

```

7745         "- 可写：基于历史残差风险储备的日前购电计划，配合逐时段残差匹配 MPC。",
7746         "- 不可写：K=8 成本最优、严格多阶段随机最优控制、旧全路径方案为正式主方案。",
7747     ]
7748     (output / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
7749
7750     if after != before:
7751         raise RuntimeError("Protected old Q2 outputs were modified")
7752     if not audit["summary"]["pass"]:
7753         raise AssertionError("C2-R4 validation failed")
7754     print(
7755         json.dumps(
7756             {
7757                 "status": report["status"],
7758                 "candidate": str(CANDIDATE_RESULT2.relative_to(ROOT)),
7759                 "signed_off_unchanged": protected_ok,
7760             },
7761             ensure_ascii=False,
7762             indent=2,
7763         )
7764     )
7765
7766
7767 if __name__ == "__main__":
7768     main()
7769
7770 # ===== q2/scenarios.py =====
7771 from __future__ import annotations
7772
7773 from dataclasses import dataclass
7774 from pathlib import Path
7775
7776 import numpy as np
7777 import pandas as pd
7778
7779 from q2.config import RESIDUAL_POOL_DAYS
7780 from q2.data import Q2Data
7781 from q2.forecast import ForecastArchive
7782
7783
7784 @dataclass(frozen=True)
7785 class ScenarioSet:
7786     target_index: int
7787     history_end_exclusive: int
7788     pool_indices: np.ndarray
7789     medoid_indices: np.ndarray
7790     cluster_labels: np.ndarray
7791     probabilities: np.ndarray
7792     scale_load: float
7793     scale_pv: float
7794     stress_index: int
7795
7796
7797 def robust_scale(values: np.ndarray) -> float:
7798     flat = np.asarray(values, dtype=float).ravel()
7799     med = np.median(flat)
7800     mad = 1.4826 * np.median(np.abs(flat - med))
7801     if mad > 1e-12:
7802         return float(mad)
7803     std = np.std(flat)
7804     return float(std if std > 1e-12 else 1.0)
7805
7806
7807 def residual_distance_matrix(
7808     load_residual: np.ndarray, pv_residual: np.ndarray, price: np.ndarray
7809 ) -> tuple[np.ndarray, float, float]:
7810     sl = robust_scale(load_residual)
7811     sp = robust_scale(pv_residual)
7812     weight = price / np.mean(price)

```

```

7813     dl = (load_residual[:, None, :] - load_residual[None, :, :]) / s1
7814     dp = (pv_residual[:, None, :] - pv_residual[None, :, :]) / sp
7815     d2 = np.mean(weight[None, None, :] * (dl * dl + dp * dp), axis=2)
7816     return np.sqrt(np.maximum(d2, 0.0)), s1, sp
7817
7818
7819 def pam(distance: np.ndarray, k: int) -> tuple[np.ndarray, np.ndarray]:
7820     """Small deterministic PAM implementation over a <=28-day pool."""
7821     n = distance.shape[0]
7822     if not 1 <= k <= n:
7823         raise ValueError(f"k={k} is invalid for pool size {n}")
7824     medoids = [int(np.argmin(distance.sum(axis=1)))]
7825     while len(medoids) < k:
7826         nearest = distance[:, medoids].min(axis=1)
7827         nearest[medoids] = -np.inf
7828         medoids.append(int(np.argmax(nearest)))
7829     medoids = np.array(sorted(medoids), dtype=int)
7830
7831     def cost(ms: np.ndarray) -> float:
7832         return float(distance[:, ms].min(axis=1).sum())
7833
7834     current = cost(medoids)
7835     while True:
7836         best_cost, best = current, medoids
7837         outside = np.setdiff1d(np.arange(n), medoids)
7838         for pos in range(k):
7839             for candidate in outside:
7840                 trial = medoids.copy()
7841                 trial[pos] = candidate
7842                 trial.sort()
7843                 trial_cost = cost(trial)
7844                 if trial_cost < best_cost - 1e-12:
7845                     best_cost, best = trial_cost, trial
7846         if best_cost >= current - 1e-12:
7847             break
7848         medoids, current = best, best_cost
7849     labels = np.argmin(distance[:, medoids], axis=1)
7850     return medoids, labels
7851
7852
7853 def build_scenarios(
7854     target_index: int,
7855     data: Q2Data,
7856     archive: ForecastArchive,
7857     k: int,
7858     history_end_exclusive: int | None = None,
7859 ) -> ScenarioSet:
7860     history_end = target_index if history_end_exclusive is None else history_end_exclusive
7861     if history_end <= 0 or history_end > target_index:
7862         raise ValueError("No historical residual exists for the first date")
7863     pool = np.arange(max(0, history_end - RESIDUAL_POOL_DAYS), history_end)
7864     lr = archive.load_residual[pool]
7865     pr = archive.pv_residual[pool]
7866     distance, s1, sp = residual_distance_matrix(lr, pr, data.price)
7867     local_medoids, labels = pam(distance, min(k, len(pool)))
7868     counts = np.bincount(labels, minlength=len(local_medoids))
7869     probabilities = counts.astype(float) / len(pool)
7870     if not np.isclose(probabilities.sum(), 1.0, atol=1e-12):
7871         raise AssertionError("scenario probabilities do not sum to one")
7872     high = data.price >= np.quantile(data.price, 0.75)
7873     net_positive = np.maximum(lr - pr, 0.0)
7874     stress_local = int(np.argmax((net_positive[:, high] * data.price[high]).sum(axis=1)))
7875     return ScenarioSet(
7876         target_index=target_index,
7877         history_end_exclusive=history_end,
7878         pool_indices=pool,
7879         medoid_indices=pool[local_medoids],
7880         cluster_labels=labels,

```



```

7881         probabilities=probabilities,
7882         scale_load=sl,
7883         scale_pv=sp,
7884         stress_index=int(pool[stress_local]),
7885     )
7886
7887
7888 def scenario_trajectories(
7889     scenarios: ScenarioSet,
7890     archive: ForecastArchive,
7891     base_load: np.ndarray | None = None,
7892     base_pv: np.ndarray | None = None,
7893 ) -> tuple[np.ndarray, np.ndarray]:
7894     i = scenarios.target_index
7895     m = scenarios.medoid_indices
7896     load_hat = archive.load_hat[i] if base_load is None else base_load
7897     pv_hat = archive.pv_hat[i] if base_pv is None else base_pv
7898     load = np.maximum(0.0, load_hat[None, :] + archive.load_residual[m])
7899     pv = np.maximum(0.0, pv_hat[None, :] + archive.pv_residual[m])
7900     return load, pv
7901
7902
7903 def write_scenarios_csv(
7904     path: Path, data: Q2Data, scenarios: ScenarioSet
7905 ) -> None:
7906     rows = []
7907     for cluster, medoid in enumerate(scenarios.medoid_indices):
7908         rows.append(
7909             {
7910                 "calibration_date": data.dates[scenarios.target_index].strftime("%Y-%m-%d"),
7911                 "medoid_date": data.dates[medoid].strftime("%Y-%m-%d"),
7912                 "cluster_size": int(np.sum(scenarios.cluster_labels == cluster)),
7913                 "probability": scenarios.probabilities[cluster],
7914                 "stress_test_date": data.dates[scenarios.stress_index].strftime("%Y-%m-%d"),
7915                 "medoid_is_stress_day": bool(medoid == scenarios.stress_index),
7916                 "pool_start": data.dates[scenarios.pool_indices[0]].strftime("%Y-%m-%d"),
7917                 "pool_end": data.dates[scenarios.pool_indices[-1]].strftime("%Y-%m-%d"),
7918             }
7919         )
7920     pd.DataFrame(rows).to_csv(path, index=False)
7921
7922
7923 def posterior_weights(
7924     scenarios: ScenarioSet,
7925     archive: ForecastArchive,
7926     observed_load_residual_prefix: np.ndarray,
7927     observed_pv_residual_prefix: np.ndarray,
7928 ) -> tuple[np.ndarray, float]:
7929     m = scenarios.medoid_indices
7930     t = len(observed_load_residual_prefix)
7931     lr = archive.load_residual[m, :t]
7932     pr = archive.pv_residual[m, :t]
7933     delta = np.mean(
7934         ((lr - observed_load_residual_prefix) / scenarios.scale_load) ** 2
7935         + ((pr - observed_pv_residual_prefix) / scenarios.scale_pv) ** 2,
7936         axis=1,
7937     )
7938     if len(m) > 1:
7939         pair = []
7940         for a in range(len(m)):
7941             for b in range(a + 1, len(m)):
7942                 pair.append(
7943                     np.mean(
7944                         ((lr[a] - lr[b]) / scenarios.scale_load) ** 2
7945                         + ((pr[a] - pr[b]) / scenarios.scale_pv) ** 2
7946                     )
7947                 )
7948     bandwidth = max(float(np.sqrt(np.median(pair))), 1e-6)

```

```

7949     else:
7950         bandwidth = 1.0
7951         logw = np.log(np.maximum(scenarios.proBABILITIES, 1e-300)) - delta / (
7952             2.0 * bandwidth * bandwidth
7953         )
7954         logw -= np.max(logw)
7955         weights = np.exp(logw)
7956         weights /= weights.sum()
7957         return weights, bandwidth
7958
7959
7960 # ===== q2/validate_q2_pilot.py =====
7961 #!/usr/bin/env python3
7962 """Independent checks over generated Q2 pilot artifacts."""
7963
7964 from __future__ import annotations
7965
7966 import json
7967 import sys
7968 from pathlib import Path
7969
7970 import numpy as np
7971 import pandas as pd
7972
7973 ROOT = Path(__file__).resolve().parents[1]
7974 if str(ROOT) not in sys.path:
7975     sys.path.insert(0, str(ROOT))
7976
7977 from q2.config import ( # noqa: E402
7978     EMERGENCY_PRICE_MULTIPLIER,
7979     E_MAX_KWH,
7980     E_MIN_KWH,
7981     NUMERIC_TOL,
7982     NEXT_DAY_VALUE_GAP_TOL_YUAN,
7983     OUTPUT_DIR,
7984     PILOT_DATES,
7985     POWER_LIMIT_KWH,
7986     SIMULTANEOUS_CD_TOL,
7987     T,
7988 )
7989 from q2.data import load_q2_data # noqa: E402
7990 from q2.pilot import planned_q_hash # noqa: E402
7991
7992
7993 def require(condition: bool, message: str) -> None:
7994     if not condition:
7995         raise AssertionError(message)
7996
7997
7998 def main() -> None:
7999     data = load_q2_data()
8000     audit = json.loads((OUTPUT_DIR / "input_audit.json").read_text(encoding="utf-8"))
8001     require(audit["shape"] == {"days": 365, "periods_per_day": 144}, "P0 shape")
8002     require(audit["missing_count"] == 0, "P0 missing data")
8003
8004     perfect = pd.read_csv(OUTPUT_DIR / "perfect_information_daily.csv")
8005     require(len(perfect) == 365 and perfect["pass"].all(), "P1 daily status")
8006     require(float(perfect["emergency_kwh"].sum()) < NUMERIC_TOL, "P1 emergency")
8007     require(
8008         np.max(
8009             np.abs(
8010                 perfect["soc_start_kwh"].to_numpy()[1:]
8011                 - perfect["soc_end_kwh"].to_numpy()[:-1]
8012             )
8013         )
8014         < NUMERIC_TOL,
8015         "P1 cross-day SOC",
8016     )

```

```

8017
8018 forecast = pd.read_csv(OUTPUT_DIR / "forecast_archive.csv")
8019 require(len(forecast) == 365 * T, "P2 archive rows")
8020 require(forecast[["load_hat_kwh", "pv_hat_kwh"]].notna().all().all(), "P2 finite")
8021 for row in forecast.groupby("date", sort=False).first().reset_index().itertuples():
8022     for field in ("load_source_dates", "pv_source_dates"):
8023         sources = str(getattr(row, field)).split(";")
8024         if sources != ["attachment1_fallback"]:
8025             require(
8026                 all(pd.Timestamp(source) < pd.Timestamp(row.date) for source in sources),
8027                 f"P2 future source in {field} for {row.date}",
8028             )
8029
8030 selected = pd.read_csv(OUTPUT_DIR / "scenario_selection.csv")
8031 require(selected.groupby("calibration_date")["selected"].sum().eq(1).all(), "P3 K choice")
8032 calendar = pd.read_csv(OUTPUT_DIR / "k_freeze_calendar.csv")
8033 periods = calendar[
8034     ["calibration_date", "effective_start_date", "effective_end_date", "selected_k"]
8035 ].drop_duplicates()
8036 require(len(periods) == int(np.ceil(365 / 14)), "R4 calibration count")
8037 require(periods["effective_start_date"].iloc[0] == "2025-01-01", "R4 start")
8038 require(periods["effective_end_date"].iloc[-1] == "2025-12-31", "R4 end")
8039 for left, right in zip(periods.itertuples(), periods.iloc[1:].itertuples()):
8040     require(
8041         pd.Timestamp(left.effective_end_date) + pd.Timedelta(days=1)
8042         == pd.Timestamp(right.effective_start_date),
8043         "R4 calendar gap",
8044     )
8045     require(
8046         calendar.groupby("calibration_date")["candidate_selected"].sum().eq(1).all(),
8047         "R4 selected candidate count",
8048     )
8049
8050 warmup = pd.read_csv(OUTPUT_DIR / "linked_warmup_daily.csv")
8051 require(len(warmup) == 31 and warmup["pass"].all(), "R2 warmup status")
8052 require(abs(float(warmup["soc_start_kwh"].iloc[0]) - 6000.0) < NUMERIC_TOL, "R2 initial SOC")
8053 require(
8054     float(
8055         np.max(
8056             np.abs(
8057                 warmup["soc_start_kwh"].to_numpy()[1:]
8058                 - warmup["soc_end_kwh"].to_numpy()[:-1]
8059             )
8060         )
8061     )
8062     < NUMERIC_TOL,
8063     "R2 SOC continuity",
8064 )
8065
8066 value_audit = pd.read_csv(OUTPUT_DIR / "next_day_value_audit.csv")
8067 require(not value_audit.empty, "R3 value audit")
8068 require(
8069     float(value_audit["certified_max_gap_yuan"].max())
8070     <= NEXT_DAY_VALUE_GAP_TOL_YUAN + 1e-6,
8071     "R3 value approximation certificate",
8072 )
8073 for _date, bundle in value_audit.groupby("decision_date"):
8074     ordered = bundle.sort_values("soc_sample_kwh")
8075     require(
8076         np.all(
8077             np.diff(ordered["value_subgradient_yuan_per_kwh"].to_numpy())
8078             >= -1e-6
8079         ),
8080         "R3 nonmonotone value subgradients",
8081     )
8082     require(
8083         (
8084             pd.to_datetime(value_audit["target_date"])

```

```

8085         == pd.to_datetime(value_audit["decision_date"]) + pd.Timedelta(days=1)
8086     ).all(),
8087     "R3 next-day target",
8088 )
8089 for row in value_audit.itertuples():
8090     if row.history_cutoff_date != "attachment1_fallback":
8091         require(
8092             pd.Timestamp(row.history_cutoff_date) < pd.Timestamp(row.decision_date),
8093             "R3 history cutoff leakage",
8094         )
8095     if row.residual_pool_end_date != "none":
8096         require(
8097             pd.Timestamp(row.residual_pool_end_date) < pd.Timestamp(row.decision_date),
8098             "R3 residual pool leakage",
8099         )
8100
8101 value_comparison = pd.read_csv(OUTPUT_DIR / "next_day_value_comparison.csv")
8102 require(
8103     not value_comparison["virtual_next_day_value_billed_today"].any(),
8104     "R3 virtual value double billing",
8105 )
8106 require(
8107     value_comparison["next_day_plan_recomputed_and_billed_once"].all(),
8108     "R3 next-day plan billing",
8109 )
8110 require(
8111     np.allclose(
8112         value_comparison["realized_two_day_cost_without_48h_today_yuan"],
8113         value_comparison["cost_without_48h_yuan"]
8114         + value_comparison["next_day_cost_after_without_48h_yuan"],
8115         atol=1e-6,
8116     ),
8117     "R3 two-day accounting without value",
8118 )
8119 require(
8120     np.allclose(
8121         value_comparison["realized_two_day_cost_with_48h_today_yuan"],
8122         value_comparison["cost_with_48h_yuan"]
8123         + value_comparison["next_day_cost_after_with_48h_yuan"],
8124         atol=1e-6,
8125     ),
8126     "R3 two-day accounting with value",
8127 )
8128 for date_text in PILOT_DATES:
8129     scenarios = pd.read_csv(OUTPUT_DIR / f"scenarios_{date_text}.csv")
8130     require(abs(float(scenarios["probability"]).sum()) - 1.0) < 1e-12, "P3 probability")
8131     require((pd.to_datetime(scenarios["medoid_date"]) < pd.Timestamp(date_text)).all(), "P3 future medoid")
8132
8133     dispatch = pd.read_csv(OUTPUT_DIR / f"q2_pilot_dispatch_{date_text}.csv")
8134     summary = pd.read_csv(OUTPUT_DIR / "pilot_day_summary.csv")
8135     summary = summary.loc[summary["date"] == date_text].iloc[0]
8136     if date_text == "2025-02-01":
8137         require(
8138             abs(summary["soc_start_kwh"] - warmup["soc_end_kwh"].iloc[-1])
8139             < NUMERIC_TOL,
8140             "R2 February initial SOC trace",
8141         )
8142     require(len(dispatch) == T, "P4 row count")
8143     require(float(dispatch["balance_residual_kwh"].abs().max()) < NUMERIC_TOL, "P4 balance")
8144     require(float((dispatch["actual_x_kwh"] - dispatch["planned_q_kwh"]).max()) < NUMERIC_TOL, "P4 x<=q")
8145     require(float(dispatch["charge_kwh"].max()) <= POWER_LIMIT_KWH + NUMERIC_TOL, "P4 charge bound")
8146     require(float(dispatch["discharge_kwh"].max()) <= POWER_LIMIT_KWH + NUMERIC_TOL, "P4 discharge bound")
8147     require(float((dispatch["curtailment_kwh"] - dispatch["pv_kwh"]).max()) < NUMERIC_TOL, "P4 curtailment")
8148     require(float(dispatch["soc_kwh"].min()) >= E_MIN_KWH - NUMERIC_TOL, "P4 SOC min")
8149     require(float(dispatch["soc_kwh"].max()) <= E_MAX_KWH + NUMERIC_TOL, "P4 SOC max")
8150     require(
8151         float((dispatch["charge_kwh"] * dispatch["discharge_kwh"]).max())
8152         <= SIMULTANEOUS_CD_TOL,

```

```

8153         "P4 simultaneous c/d",
8154     )
8155     planned_cost = float(data.price @ dispatch["planned_q_kwh"].to_numpy())
8156     emergency_cost = float(
8157         EMERGENCY_PRICE_MULTIPLIER
8158         * data.price
8159         @ dispatch["emergency_kwh"].to_numpy()
8160     )
8161     require(abs(planned_cost - summary["planned_cost_yuan"]) < 1e-6, "P4 q cost")
8162     require(abs(emergency_cost - summary["emergency_cost_yuan"]) < 1e-6, "P4 e cost")
8163     require(
8164         abs(
8165             summary["total_cost_yuan"]
8166             - summary["planned_cost_yuan"]
8167             - summary["emergency_cost_yuan"]
8168         )
8169         < 1e-6,
8170         "R3 virtual value billed in actual cost",
8171     )
8172     require(
8173         planned_q_hash(dispatch["planned_q_kwh"].to_numpy())
8174         == summary["planned_q_sha256"],
8175         "P4 locked q hash",
8176     )
8177     no48 = pd.read_csv(
8178         OUTPUT_DIR / f"q2_pilot_dispatch_{date_text}_no48h.csv"
8179     )
8180     require(len(no48) == T, "R3 no-48h row count")
8181     require(
8182         float(no48["balance_residual_kwh"].abs().max()) < NUMERIC_TOL,
8183         "R3 no-48h balance",
8184     )
8185     require(
8186         float((no48["actual_x_kwh"] - no48["planned_q_kwh"]).max())
8187         < NUMERIC_TOL,
8188         "R3 no-48h x<=q",
8189     )
8190
8191     require(not (ROOT / "output" / "result2.xlsx").exists(), "result2.xlsx forbidden")
8192     print("Q2 pilot independent validation: PASS")
8193
8194
8195 if __name__ == "__main__":
8196     main()
8197
8198
8199 # ===== q2/validate_q2_policy_consistent.py =====
8200 #!/usr/bin/env python3
8201 """Independent checks over C2-R1 / R2 / R3 / R4 policy-consistent artifacts."""
8202
8203 from __future__ import annotations
8204
8205 import json
8206 import sys
8207 from pathlib import Path
8208
8209 import numpy as np
8210 import pandas as pd
8211
8212 ROOT = Path(__file__).resolve().parents[1]
8213 if str(ROOT) not in sys.path:
8214     sys.path.insert(0, str(ROOT))
8215
8216 from q2.config import ( # noqa: E402
8217     CANDIDATE_RESULT2,
8218     E_INITIAL_KWH,
8219     E_MAX_KWH,
8220     E_MIN_KWH,

```

```

8221     FIG_POLICY_CONSISTENT_DIR,
8222     FIG_Q2_FINAL_DIR,
8223     FIXED_SCENARIO_K,
8224     K_SENSITIVITY_CANDIDATES,
8225     NUMERIC_TOL,
8226     OFFICIAL_OUTPUT_START,
8227     PILOT_DATES,
8228     POLICY_CONSISTENT_OUTPUT_DIR,
8229     POWER_LIMIT_KWH,
8230     RESULT2_SIGNED_OFF_BACKUP,
8231     SIGNED_OFF_RESULT2,
8232     SIGNED_OFF_RESULT2_SHA256,
8233     SIMULTANEOUS_CD_TOL,
8234     T,
8235 )
8236 from q2.data import load_q2_data # noqa: E402
8237 from q2.export_result2 import ( # noqa: E402
8238     N_OUTPUT_DAYS,
8239     audit_candidate_result2,
8240     file_sha256,
8241 )
8242 from q2.pilot import planned_q_hash # noqa: E402
8243 from q2.policy_consistent import PERIOD_FEB_DEC_OUTPUT, PERIOD_JANUARY_WARMUP # noqa: E402
8244
8245
8246 def require(condition: bool, message: str) -> None:
8247     if not condition:
8248         raise AssertionError(message)
8249
8250
8251 def assert_no_forecast_leakage(forecast: pd.DataFrame) -> None:
8252     for row in forecast.itertuples():
8253         target = pd.Timestamp(row.date)
8254         for field in ("load_source_dates", "pv_source_dates"):
8255             sources = str(getattr(row, field)).split(";")
8256             if sources != ["attachment1_fallback"]:
8257                 require(
8258                     all(pd.Timestamp(source) < target for source in sources),
8259                     f"forecast leakage in {field} on {row.date}",
8260                 )
8261             if str(row.residual_pool_end_date) != "none":
8262                 require(
8263                     pd.Timestamp(row.residual_pool_end_date) < target,
8264                     f"residual pool leakage on {row.date}",
8265                 )
8266             if str(row.medoid_dates) != "none":
8267                 require(
8268                     all(pd.Timestamp(item) < target for item in str(row.medoid_dates).split(";")),
8269                     f"medoid leakage on {row.date}",
8270                 )
8271             shapes = json.loads(row.day_ahead_variable_shapes)
8272             for name in ("charge", "discharge", "soc"):
8273                 require(name in shapes, f"missing {name}")
8274                 require(len(shapes[name]) == 1, f"{name} must be 1-D, got {shapes[name]}")
8275             require(not bool(row.has_scenario_specific_battery), "scenario battery flag")
8276
8277
8278 def assert_dispatch_physics(path: Path, data, physical_row: pd.Series, date_text: str) -> None:
8279     i = int(data.dates.get_loc(pd.Timestamp(date_text)))
8280     frame = pd.read_csv(path)
8281     require(len(frame) == T, f"{date_text} dispatch length")
8282     q = frame["planned_q_kwh"].to_numpy()
8283     x = frame["actual_x_kwh"].to_numpy()
8284     require(planned_q_hash(q) == physical_row["planned_q_sha256"], "q hash")
8285     require(np.max(x - q) < NUMERIC_TOL, f"{date_text} x<=q")
8286     residual = (
8287         frame["actual_x_kwh"]
8288         + frame["emergency_kwh"]

```

```

8289         + frame["pv_kwh"]
8290         - frame["curtailment_kwh"]
8291         + frame["discharge_kwh"]
8292         - frame["load_kwh"]
8293         - frame["charge_kwh"]
8294     )
8295     require(float(residual.abs().max()) < NUMERIC_TOL, f"{date_text} balance")
8296     require(frame["soc_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL, f"{date_text} SOC min")
8297     require(frame["soc_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL, f"{date_text} SOC max")
8298     require(
8299         float((frame["charge_kwh"] * frame["discharge_kwh"]).max()) <= SIMULTANEOUS_CD_TOL,
8300         f"{date_text} simultaneous charge/discharge",
8301     )
8302     require(frame["charge_kwh"].max() <= POWER_LIMIT_KWH + NUMERIC_TOL, f"{date_text} charge limit")
8303     require(
8304         frame["discharge_kwh"].max() <= POWER_LIMIT_KWH + NUMERIC_TOL,
8305         f"{date_text} discharge limit",
8306     )
8307     require(np.allclose(frame["load_kwh"].to_numpy(), data.load[i], atol=1e-12), "dispatch load")
8308
8309
8310 def validate_r1(output: Path, data, consistency: dict, report: dict) -> None:
8311     r1_forecast = output / "r1_forecast_policy_audit.csv"
8312     if r1_forecast.exists():
8313         forecast = pd.read_csv(r1_forecast)
8314         physical = pd.read_csv(output / "r1_physical_audit.csv")
8315         value_audit = pd.read_csv(output / "r1_next_day_value_audit.csv")
8316         dispatch_dir = output / "r1_dispatch_daily"
8317     else:
8318         forecast = pd.read_csv(output / "forecast_policy_audit.csv")
8319         physical = pd.read_csv(output / "physical_audit.csv")
8320         value_audit = pd.read_csv(output / "next_day_value_audit.csv")
8321         dispatch_dir = output / "dispatch_daily"
8322         require(consistency["annual_run"] is False, "R1 must not run the year")
8323
8324     require(list(forecast["date"]) == list(PILOT_DATES), "R1 dates")
8325     require(report["status"] == "PASS", "validation status")
8326     if not r1_forecast.exists():
8327         require(report["candidate_result2.xlsx"] is False, "no candidate result2")
8328     require(consistency["validation_uses_full_day_actual_lp"] is False, "no full-day actual LP")
8329     require(consistency["k_used_for"] == ["risk_reserve_R", "intra_day_residual_weights"], "K uses")
8330     assert_no_forecast_leakage(forecast)
8331     require(physical["pass"].all(), "physical pass")
8332     require(not physical["used_full_day_actual_lp"].any(), "artifact used full-day LP")
8333     require(set(value_audit["decision_date"]) == set(PILOT_DATES), "value-cut decision dates")
8334     for row in value_audit.itertuples():
8335         decision = pd.Timestamp(row.decision_date)
8336         cutoff = pd.Timestamp(row.history_cutoff_date)
8337         target = pd.Timestamp(row.target_date)
8338         require(cutoff == decision - pd.Timedelta(days=1), f"cutoff is last history day for {row.decision_date}")
8339         require(cutoff < decision, f"cutoff precedes decision {row.decision_date}")
8340         require(target == decision + pd.Timedelta(days=1), f"target is next day for {row.decision_date}")
8341     for date_text in PILOT_DATES:
8342         physical_row = physical.loc[physical["date"] == date_text].iloc[0]
8343         assert_dispatch_physics(
8344             dispatch_dir / f"dispatch_{date_text}.csv",
8345             data,
8346             physical_row,
8347             date_text,
8348         )
8349     print("C2-R1 artifact validation passed")
8350
8351
8352 def validate_r2(
8353     output: Path,
8354     data,
8355     diagnostics: pd.DataFrame,
8356     consistency: dict,

```

```

8357     report: dict,
8358 ) -> None:
8359     from q2.config import RISK_ALPHA_CANDIDATES
8360     from q2.policy_consistent import FORECAST_MODES
8361
8362     calibration = pd.read_csv(output / "closed_loop_calibration.csv")
8363     deployed = pd.read_csv(output / "r2_deployed_daily.csv")
8364     n_candidates = len(FORECAST_MODES) * len(RISK_ALPHA_CANDIDATES)
8365     stage = consistency["stage"]
8366     require(n_candidates == 12, "12 (m, alpha) pairs")
8367     if stage in ("C2-R3", "C2-R4"):
8368         require(consistency["annual_run"] is True, "R3 annual run flag")
8369     elif stage == "C2-R2":
8370         require(consistency["annual_run"] is False, "R3 annual run not started")
8371     else:
8372         raise AssertionError(f"unexpected stage {stage}")
8373     require(consistency["calibration_closed_loop"] is True, "closed-loop calibration")
8374     require(consistency["mae_is_diagnostic_only"] is True, "MAE diagnostic flag")
8375     require(consistency["mae_used_for_selection"] is False, "MAE not used to select")
8376     require(consistency["validation_uses_full_day_actual_lp"] is False, "R2 no full-day LP")
8377     require(consistency["formal_execution_uses_full_day_actual_lp"] is False, "deploy no full-day LP")
8378     require(consistency["calibration_and_execution_exclude_future_actuals"] is True, "no future actuals")
8379     require(consistency["calibration_include_value_cuts"] is True, "calibration value cuts")
8380     require(consistency["deployment_include_value_cuts"] is True, "deployment value cuts")
8381     require(consistency["include_value_cuts"] is True, "include_value_cuts flag")
8382     require(consistency["include_value_cuts_consistent"] is True, "value-cut consistency")
8383     if stage != "C2-R4":
8384         require(report["candidate_result2.xlsx"] is False, "no candidate result2")
8385     require(diagnostics["selection_role"].eq("diagnostic_only").all(), "R2 MAE diagnostic")
8386     require(len(deployed) == len(data.dates), "deployed path covers the year for SOC")
8387     require(deployed["pass"].all(), "deployed physical pass")
8388     require(not deployed["used_full_day_actual_lp"].any(), "deployed used full-day LP")
8389     require(not deployed["future_actuals_in_optimizer"].any(), "deployed future actuals")
8390     require(deployed["include_value_cuts"].all(), "deployed value cuts")
8391     require(calibration["include_value_cuts"].all(), "calibration value cuts column")
8392     require(
8393         np.max(np.abs(deployed["soc_start_kwh"].to_numpy()[1:] - deployed["soc_end_kwh"].to_numpy()[:-1]))
8394         < NUMERIC_TOL,
8395         "deployed SOC continuity",
8396     )
8397
8398     scored = calibration.loc[calibration["fallback_reason"].fillna("") == ""]
8399     require((scored.groupby("calibration_date").size() == n_candidates).all(), "12 candidates per scored block")
8400     require(
8401         calibration.loc[calibration["selected"].astype(bool)].groupby("calibration_date").size().eq(1).all(),
8402         "one selected pair per block",
8403     )
8404     require(not scored["used_full_day_actual_lp"].any(), "candidate used full-day LP")
8405     require(not scored["future_actuals_in_optimizer"].any(), "candidate future actuals")
8406     for _, block in scored.groupby("calibration_date"):
8407         best = block["mean_actual_cost_yuan"].min()
8408         selected_row = block.loc[block["selected"].astype(bool)].iloc[0]
8409         threshold = float(selected_row["one_se_threshold_yuan"])
8410         require(float(selected_row["mean_actual_cost_yuan"]) <= threshold + 1e-8, "selected within 1-SE")
8411         require(best <= float(selected_row["mean_actual_cost_yuan"]) + 1e-8, "best cost not worse than selected")
8412         require(
8413             pd.Timestamp(selected_row["history_cutoff_date"])
8414             == pd.Timestamp(selected_row["calibration_date"]) - pd.Timedelta(days=1),
8415             "calibration cutoff is last history day",
8416         )
8417         window_start = pd.Timestamp(selected_row["validation_start_date"])
8418         deployed_start = deployed.loc[deployed["date"] == window_start.strftime("%Y-%m-%d"), "soc_start_kwh"]
8419         require(len(deployed_start) == 1, "window start date in deployed path")
8420         require(
8421             abs(float(selected_row["window_start_soc_kwh"]) - float(deployed_start.iloc[0])) < 1e-6,
8422             "window SOC comes from deployed policy",
8423         )
8424     print("C2-R2 artifact validation passed")

```



```

8425
8426
8427 def validate_r3(output: Path, data, consistency: dict, report: dict) -> None:
8428     daily = pd.read_csv(output / "q2_redesign_daily_summary.csv")
8429     january = pd.read_csv(output / "january_warmup_summary.csv")
8430     feb_dec = pd.read_csv(output / "feb_dec_daily_summary.csv")
8431     forecast = pd.read_csv(output / "forecast_policy_audit.csv")
8432     physical = pd.read_csv(output / "physical_audit.csv")
8433     value_audit = pd.read_csv(output / "next_day_value_audit.csv")
8434     k_table = pd.read_csv(output / "k_sensitivity.csv")
8435     soc_table = pd.read_csv(output / "feb_soc_sensitivity.csv")
8436     alt = pd.read_csv(output / "feb_soc_sensitivity_daily.csv")
8437     calendar = pd.read_csv(output / "frozen_policy_calendar.csv")
8438
8439     require(consistency["stage"] in ("C2-R3", "C2-R4"), "R3/R4 stage")
8440     require(consistency["annual_run"] is True, "annual run")
8441     if consistency["stage"] == "C2-R3":
8442         require(report["candidate_result2_xlsx"] is False, "no candidate result2")
8443         require(consistency["main_path_initial_rule_unchanged"] is True, "initial rule frozen")
8444         require(consistency["calibration_include_value_cuts"] is True, "R3 calibration value cuts")
8445         require(consistency["deployment_include_value_cuts"] is True, "R3 deployment value cuts")
8446         require(consistency["annual_run_include_value_cuts"] is True, "R3 annual value cuts")
8447         require(consistency["include_value_cuts_consistent"] is True, "R3 value-cut consistency")
8448         require(daily["include_value_cuts"].all(), "annual daily value cuts")
8449         require(len(daily) == len(data.dates), "full-year daily summary")
8450         require(list(daily["date"]) == [stamp.strftime("%Y-%m-%d") for stamp in data.dates], "date order")
8451         require(abs(float(daily["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) < NUMERIC_TOL, "Jan 1 SOC 6000")
8452         require(len(january) == 31, "January warmup days")
8453         require(len(feb_dec) == 334, "Feb-Dec days")
8454         require((january["period"] == PERIOD_JANUARY_WARMUP).all(), "January period label")
8455         require((feb_dec["period"] == PERIOD_FEB_DEC_OUTPUT).all(), "Feb-Dec period label")
8456         require(january["date"].iloc[0] == "2025-01-01", "January start")
8457         require(january["date"].iloc[-1] == "2025-01-31", "January end")
8458         require(feb_dec["date"].iloc[0] == OFFICIAL_OUTPUT_START, "Feb-Dec start")
8459         require(feb_dec["date"].iloc[-1] == "2025-12-31", "Feb-Dec end")
8460         require(
8461             np.max(np.abs(daily["soc_start_kwh"].to_numpy()[1:] - daily["soc_end_kwh"].to_numpy()[:-1]))
8462             < NUMERIC_TOL,
8463             "main-path SOC continuity",
8464         )
8465         inherited = float(january["soc_end_kwh"].iloc[-1])
8466         require(abs(float(feb_dec["soc_start_kwh"].iloc[0]) - inherited) < NUMERIC_TOL, "Feb 1 inherits warmup")
8467         require(daily["pass"].all(), "annual physical pass")
8468         require(not daily["used_full_day_actual_lp"].any(), "annual full-day LP")
8469         require(not daily["future_actuals_in_optimizer"].any(), "annual future actuals")
8470         require(len(forecast) == len(data.dates), "annual forecast audit")
8471         assert_no_forecast_leakage(forecast)
8472         require(physical["pass"].all(), "physical audit pass")
8473         require((physical["period"] == daily["period"]).all(), "physical period labels")
8474
8475     decisions = set(pd.to_datetime(value_audit["decision_date"]).dt.strftime("%Y-%m-%d"))
8476     require("2025-12-31" not in decisions, "no next-day cuts on Dec 31")
8477     require(len(decisions) == 364, "value cuts on 364 decision days")
8478     for row in value_audit.itertuples():
8479         decision = pd.Timestamp(row.decision_date)
8480         cutoff_raw = row.history_cutoff_date
8481         target = pd.Timestamp(row.target_date)
8482         if str(cutoff_raw) == "attachment1_fallback":
8483             require(decision == pd.Timestamp("2025-01-01"), "fallback cutoff only on Jan 1")
8484         else:
8485             require(
8486                 pd.Timestamp(cutoff_raw) == decision - pd.Timedelta(days=1),
8487                 f"cutoff is last history day for {row.decision_date}",
8488             )
8489             require(target == decision + pd.Timedelta(days=1), f"target is next day for {row.decision_date}")
8490
8491     require(set(k_table["scenario_k"].astype(int)) == set(K_SENSITIVITY_CANDIDATES), "K=4,8,12")
8492     require((k_table["period"] == PERIOD_FEB_DEC_OUTPUT).all(), "K table is Feb-Dec")

```

```

8493     require(bool(k_table["calendar_frozen_from_r2"].all()), "K uses frozen calendar")
8494     k8 = k_table.loc[k_table["scenario_k"].astype(int) == FIXED_SCENARIO_K].iloc[0]
8495     require(abs(float(k8["total_cost_yuan"]) - float(feb_dec["total_cost_yuan"].sum())) < 1e-4, "K=8 matches main")
8496
8497     require(len(soc_table) == 2, "two Feb SOC rows")
8498     inherited_row = soc_table.loc[soc_table["soc_source"] == "inherited_january_warmup"].iloc[0]
8499     boundary_row = soc_table.loc[soc_table["soc_source"] == "registered_boundary_E_INITIAL_6000"].iloc[0]
8500     require(abs(float(inherited_row["feb1_soc_start_kwh"]) - inherited) < NUMERIC_TOL, "inherited SOC table")
8501     require(abs(float(boundary_row["feb1_soc_start_kwh"]) - E_INITIAL_KWH) < NUMERIC_TOL, "boundary SOC table")
8502     require(bool(boundary_row["only_registered_boundary_changed"]), "boundary-only flag")
8503     require(list(alt["date"]) == list(feb_dec["date"]), "SOC sensitivity dates")
8504     require(list(alt["forecast_mode"]) == list(feb_dec["forecast_mode"]), "SOC sensitivity forecast mode")
8505     require(
8506         np.allclose(
8507             alt["risk_alpha"].to_numpy(dtype=float),
8508             feb_dec["risk_alpha"].to_numpy(dtype=float),
8509             equal_nan=True,
8510         ),
8511         "SOC sensitivity alpha",
8512     )
8513     require(abs(float(alt["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) < NUMERIC_TOL, "alt Feb 1 SOC")
8514     require(not np.isclose(float(alt["soc_start_kwh"].iloc[0]), inherited), "alt SOC differs from inherited")
8515
8516     require(len(calendar) == len(data.dates), "frozen calendar coverage")
8517     require(calendar["forecast_mode"].isin(["m1", "m2", "m3"]).all(), "calendar modes")
8518
8519     for date_text in ("2025-01-01", OFFICIAL_OUTPUT_START, "2025-06-21", "2025-12-31"):
8520         physical_row = physical.loc[physical["date"] == date_text].iloc[0]
8521         assert_dispatch_physics(
8522             output / "dispatch_daily" / f"dispatch_{date_text}.csv",
8523             data,
8524             physical_row,
8525             date_text,
8526         )
8527
8528     for stem in (
8529         "fig_q2_pc_cost_comparison",
8530         "fig_q2_pc_emergency_curtailement",
8531         "fig_q2_pc_soc",
8532         "fig_q2_pc_cumulative_cost",
8533     ):
8534         require((FIG_POLICY_CONSISTENT_DIR / f"{stem}.pdf").exists(), f"missing {stem}.pdf")
8535         require((FIG_POLICY_CONSISTENT_DIR / f"{stem}.png").exists(), f"missing {stem}.png")
8536     print("C2-R3 artifact validation passed")
8537
8538
8539 def validate_r4(output: Path, data, consistency: dict, report: dict) -> None:
8540     require(consistency["stage"] == "C2-R4", "R4 stage")
8541     require(report["stage"] == "C2-R4", "R4 report stage")
8542     require(report["status"] == "PASS", "R4 status")
8543     require(report["candidate_result2_xlsx"] is True, "candidate result2 flag")
8544     require(consistency["candidate_result2_xlsx"] is True, "audit candidate flag")
8545     require(consistency["k8_not_cost_optimal"] is True, "K=8 not cost-optimal")
8546     require(consistency["k8_paper_language"] == "预注册且对 K=4/12 稳定的简洁主方案", "K=8 paper language")
8547     require(consistency["old_full_path_role"] == "信息更强的近似对照", "old path role")
8548     require(CANDIDATE_RESULT2.exists(), "candidate workbook missing")
8549     require(CANDIDATE_RESULT2.resolve() != SIGNED_OFF_RESULT2.resolve(), "candidate overwrote signed-off file")
8550     require(CANDIDATE_RESULT2.parent.resolve() == output.resolve(), "candidate not in policy-consistent dir")
8551     require(file_sha256(SIGNED_OFF_RESULT2) == SIGNED_OFF_RESULT2_SHA256, "signed-off result2 hash")
8552     require(RESULT2_SIGNED_OFF_BACKUP.exists(), "signed-off backup missing")
8553     require(file_sha256(RESULT2_SIGNED_OFF_BACKUP) == SIGNED_OFF_RESULT2_SHA256, "backup hash")
8554     require(FIG_Q2_FINAL_DIR.resolve() != FIG_POLICY_CONSISTENT_DIR.resolve(), "figure dirs collapsed")
8555
8556     january = pd.read_csv(output / "january_warmup_summary.csv")
8557     feb_dec = pd.read_csv(output / "feb_dec_daily_summary.csv")
8558     inherited = float(january["soc_end_kwh"].iloc[-1])
8559     audit = audit_candidate_result2(
8560         CANDIDATE_RESULT2,

```

```

8561         output / "dispatch_daily",
8562         feb_dec,
8563         data,
8564         inherited,
8565     )
8566     require(audit["summary"]["pass"] is True, "cell audit pass")
8567     require(audit["summary"]["n_days"] == N_OUTPUT_DAYS, "334 official days")
8568     require(audit["summary"]["n_periods"] == T, "144 periods")
8569     require(report["result2_cell_audit_pass"] is True, "report cell audit flag")
8570     require(
8571         abs(float(feb_dec["soc_start_kwh"].iloc[0]) - inherited) < NUMERIC_TOL,
8572         "R4 Feb 1 inherited SOC",
8573     )
8574     for stem in (
8575         "fig_q2_pc_closed_loop",
8576         "fig_q2_pc_representative_day",
8577         "fig_q2_pc_forecast_calendar",
8578         "fig_q2_pc_cost_comparison",
8579     ):
8580         require((FIG_POLICY_CONSISTENT_DIR / f"{stem}.pdf").exists(), f"missing {stem}.pdf")
8581         require((FIG_POLICY_CONSISTENT_DIR / f"{stem}.png").exists(), f"missing {stem}.png")
8582     print("C2-R4 artifact validation passed")
8583
8584
8585 def main() -> None:
8586     output = POLICY_CONSISTENT_OUTPUT_DIR
8587     data = load_q2_data()
8588     diagnostics = pd.read_csv(output / "forecast_diagnostics.csv")
8589     consistency = json.loads((output / "policy_consistency_audit.json").read_text(encoding="utf-8"))
8590     report = json.loads((output / "validation.json").read_text(encoding="utf-8"))
8591     require(diagnostics["selection_role"].eq("diagnostic_only").all(), "MAE must stay diagnostic")
8592
8593     if (output / "r1_forecast_policy_audit.csv").exists() or list(
8594         pd.read_csv(output / "forecast_policy_audit.csv")["date"]
8595     ) == list(PILOT_DATES):
8596         validate_r1(output, data, consistency, report)
8597     if (output / "closed_loop_calibration.csv").exists():
8598         validate_r2(output, data, diagnostics, consistency, report)
8599     if consistency.get("stage") in ("C2-R3", "C2-R4") and (output / "q2_redesign_daily_summary.csv").exists():
8600         validate_r3(output, data, consistency, report)
8601     if consistency.get("stage") == "C2-R4":
8602         validate_r4(output, data, consistency, report)
8603
8604
8605 if __name__ == "__main__":
8606     main()
8607
8608 # ===== q3/_init_.py =====
8609 """C-problem Q3 rolling purchase-adjustment pilot."""
8610
8611 # ===== q3/config.py =====
8612 """Central configuration for the Q3 two-day rolling-adjustment pilot."""
8613
8614 from __future__ import annotations
8615
8616 import os
8617 from pathlib import Path
8618
8619 import numpy as np
8620
8621 ROOT = Path(__file__).resolve().parents[1]
8622
8623
8624 def resolve_attach_dir() -> Path:
8625     candidates = []
8626     env = os.environ.get("CUMCM_C_ATTACH_DIR")
8627     if env:
8628         candidates.append(Path(env))

```

```

8629     candidates.extend(
8630         [
8631             ROOT / "data" / "raw" / "附件",
8632         ]
8633     )
8634     for path in candidates:
8635         if (path / "附件1.xlsx").exists() and (path / "附件3.xlsx").exists():
8636             return path
8637     return candidates[0]
8638
8639
8640 ATTACH_DIR = resolve_attach_dir()
8641 ATTACH1 = ATTACH_DIR / "附件1.xlsx"
8642 ATTACH2 = ATTACH_DIR / "附件2.xlsx"
8643 ATTACH3 = ATTACH_DIR / "附件3.xlsx"
8644 ATTACH4 = ATTACH_DIR / "附件4.xlsx"
8645 RESULT3_TEMPLATE = ATTACH_DIR / "附件5" / "result3.xlsx"
8646 OUTPUT_DIR = ROOT / "output" / "q3_pilot"
8647 FIG_DIR = ROOT / "fig" / "q3_pilot"
8648 RESULT3_XLSX = ROOT / "output" / "result3.xlsx"
8649 RESULT3_EXPORT_AUDIT = ROOT / "output" / "result3_export_audit.json"
8650
8651 T = 144
8652 DELTA_H = 1.0 / 6.0
8653 ETA_C = float(np.sqrt(0.9))
8654 ETA_D = float(np.sqrt(0.9))
8655 E_MIN_KWH = 1200.0
8656 E_MAX_KWH = 10800.0
8657 E_INITIAL_KWH = 6000.0
8658 POWER_LIMIT_KW = 5000.0
8659 POWER_LIMIT_KWH = POWER_LIMIT_KW * DELTA_H
8660
8661 EMERGENCY_PRICE_MULTIPLIER = 5.0
8662 ADJUST_ABS_COEFF = 0.5
8663 VOI_EPS_YUAN = 0.01
8664 LOAD_INFORMATION_MAIN = "causal_load_main"
8665 LOAD_INFORMATION_PROXY = "actual_load_proxy"
8666 LOAD_INFORMATION_CASES = (LOAD_INFORMATION_MAIN, LOAD_INFORMATION_PROXY)
8667 LOAD_TREATMENT = LOAD_INFORMATION_MAIN
8668 WARMUP_CSV_MAIN = "q3_warmup_daily_causal_load_main.csv"
8669 LOAD_HISTORY_SAME_WEEKDAY = 4
8670 NEXT_DAY_PV_HISTORY_DAYS = 7
8671 NEXT_DAY_VALUE_GAP_TOL_YUAN = 1.0
8672 NEXT_DAY_VALUE_MAX_SAMPLES = 25
8673 MPC_COST_TOL = 1e-7
8674 NUMERIC_TOL = 1e-6
8675 SIMULTANEOUS_CD_TOL = 1e-4
8676 SOLVER = "HIGHS"
8677 PILOT_DATES = ("2025-02-01", "2025-06-21")
8678
8679 PV_MAPPING_LINEAR = "linear_anchor_main"
8680 PV_MAPPING_STEP = "step_hourly_sensitivity"
8681 PV_MAPPING_MODES = (PV_MAPPING_LINEAR, PV_MAPPING_STEP)
8682
8683 SETTLEMENT_MAIN = "anchor_final_main"
8684 SETTLEMENT_ALT = "adjacent_literal_sensitivity"
8685 SETTLEMENT_MODES = (SETTLEMENT_MAIN, SETTLEMENT_ALT)
8686
8687 SENSITIVITY_OUTPUT_DIR = ROOT / "output" / "q3_sensitivity"
8688 SENSITIVITY_FIG_DIR = ROOT / "fig" / "q3_sensitivity"
8689
8690 YEAR_N_DAYS = 365
8691 YEAR_END_SOC_A_KWH = 1200.0
8692 YEAR_END_SOC_B_KWH = 6000.0
8693 YEAR_END_BOUNDARY_A = "A_q2_aligned"
8694 YEAR_END_BOUNDARY_B = "B_energy_neutral"
8695 YEAR_END_BOUNDARIES = (
8696     (YEAR_END_BOUNDARY_A, YEAR_END_SOC_A_KWH),

```

```

8697     (YEAR_END_BOUNDARY_B, YEAR_END_SOC_B_KWH),
8698 )
8699 TERMINAL_SOC_PILOT_START = "2025-12-01"
8700 TERMINAL_SOC_PILOT_END = "2025-12-31"
8701 TERMINAL_SOC_OUTPUT_DIR = ROOT / "output" / "q3_terminal_soc_pilot"
8702 TERMINAL_SOC_FIG_DIR = ROOT / "fig" / "q3_terminal_soc_pilot"
8703 ANNUAL_OUTPUT_DIR = ROOT / "output" / "q3_full_annual"
8704 ANNUAL_FIG_DIR = ROOT / "fig" / "q3_full_annual"
8705 ANNUAL_START = "2025-01-01"
8706 ANNUAL_END = "2025-12-31"
8707 RESULT3_EXPORT_START = "2025-02-01"
8708 RESULT3_EXPORT_END = "2025-12-31"
8709 RESULT3_EXPORT_N_DAYS = 334
8710 RESULT3_OFFICIAL_STRATEGY = "M1_M6"
8711 RESULT3_ANNUAL_COST_YUAN = 16373508.75
8712 FOUR_HOUR_BLOCKS = (
8713     ("0:00-4:00", 0, 24),
8714     ("4:00-8:00", 24, 48),
8715     ("8:00-12:00", 48, 72),
8716     ("12:00-16:00", 72, 96),
8717     ("16:00-20:00", 96, 120),
8718     ("20:00-24:00", 120, 144),
8719 )
8720 REQUIRED_TRAJECTORY_COLS = (
8721     "date",
8722     "period_index",
8723     "time_label",
8724     "g0_kwh",
8725     "g_final_kwh",
8726     "x_kwh",
8727     "emergency_kwh",
8728     "charge_kwh",
8729     "discharge_kwh",
8730     "curtailment_kwh",
8731     "soc_start_kwh",
8732     "soc_end_kwh",
8733     "actual_load_kwh",
8734     "actual_pv_kwh",
8735     "locked_or_mutable",
8736     "last_update_time",
8737 )
8738
8739
8740 def year_end_tag(year_end_soc_kwh: float | None) -> str:
8741     if year_end_soc_kwh is None:
8742         return ""
8743     return f"ye{int(round(float(year_end_soc_kwh)))}"
8744
8745
8746 def year_end_boundary_label(year_end_soc_kwh: float | None) -> str | None:
8747     if year_end_soc_kwh is None:
8748         return None
8749     value = float(year_end_soc_kwh)
8750     if abs(value - YEAR_END_SOC_A_KWH) <= 1e-9:
8751         return YEAR_END_BOUNDARY_A
8752     if abs(value - YEAR_END_SOC_B_KWH) <= 1e-9:
8753         return YEAR_END_BOUNDARY_B
8754     return f"custom_{value:.0f}"
8755
8756
8757 def today_year_end_soc(
8758     day_index: int, n_days: int, year_end_soc_kwh: float | None
8759 ) -> float | None:
8760     """Hard E_144 target for remaining-horizon LPs on the calendar year's last day."""
8761     if year_end_soc_kwh is None:
8762         return None
8763     if int(day_index) == int(n_days) - 1:
8764         return float(year_end_soc_kwh)

```

```

8765     return None
8766
8767
8768 def next_day_year_end_soc(
8769     day_index: int, n_days: int, year_end_soc_kwh: float | None
8770 ) -> float | None:
8771     """Hard E_144 target for the virtual next-day LP on Dec 30 (48h cost-to-go)."""
8772     if year_end_soc_kwh is None:
8773         return None
8774     if int(day_index) == int(n_days) - 2:
8775         return float(year_end_soc_kwh)
8776     return None
8777
8778
8779 def make_run_id(
8780     date: str,
8781     strategy: str,
8782     load_information_case: str,
8783     pv_mapping_mode: str,
8784     settlement_mode: str,
8785     with_terminal_value: bool,
8786     year_end_soc_kwh: float | None = None,
8787 ) -> str:
8788     tv = "48h" if with_terminal_value else "no48h"
8789     run_id = (
8790         f"{date}__{strategy}__{load_information_case}__"
8791         f"{pv_mapping_mode}__{settlement_mode}__{tv}"
8792     )
8793     tag = year_end_tag(year_end_soc_kwh)
8794     if tag:
8795         run_id += f"__{tag}"
8796     return run_id
8797
8798
8799 def dispatch_stem(
8800     date: str,
8801     strategy: str,
8802     load_information_case: str,
8803     pv_mapping_mode: str,
8804     settlement_mode: str,
8805     with_terminal_value: bool,
8806     year_end_soc_kwh: float | None = None,
8807 ) -> str:
8808     suffix = f"{strategy}_{load_information_case}_{pv_mapping_mode}_{settlement_mode}"
8809     if not with_terminal_value:
8810         suffix += "_no48h"
8811     tag = year_end_tag(year_end_soc_kwh)
8812     if tag:
8813         suffix += f"_{tag}"
8814     return f"q3_dispatch_{date}_{suffix}"
8815
8816 # Update clock hours and the first 10-minute index that may change.
8817 # Index t corresponds to the period ending at (t+1)*10 minutes after 0:00.
8818 UPDATE_SPECS = (
8819     (0, 0),    # 0:00 plan; first mutable period is 00:10 (index 0)
8820     (6, 36),   # 6:00 update; first mutable period is 06:10
8821     (12, 72),  # 12:00 update; first mutable period is 12:10
8822     (18, 108), # 18:00 update; first mutable period is 18:10
8823 )
8824 UPDATE_HOURS = tuple(hour for hour, _index in UPDATE_SPECS)
8825 HOUR_TO_FIRST_MUTABLE = {hour: index for hour, index in UPDATE_SPECS}
8826
8827 STRATEGIES = ("M0", "M1_M6", "M6_only", "M12_only", "M18_only")
8828 STRATEGY_ALLOWED_UPDATES = {
8829     "M0": (),
8830     "M1_M6": (6, 12, 18),
8831     "M6_only": (6,),
8832     "M12_only": (12,),

```

```

8833     "M18_only": (18,),
8834 }
8835
8836 # ===== q3/data.py =====
8837 from __future__ import annotations
8838
8839 import json
8840 from dataclasses import dataclass
8841 from datetime import datetime, time
8842 from pathlib import Path
8843
8844 import numpy as np
8845 import pandas as pd
8846
8847 from q3.config import ATTACH1, ATTACH2, ATTACH3, DELTA_H, PILOT_DATES, T, UPDATE_HOURS
8848
8849
8850 def normalize_time(value: object) -> str:
8851     if isinstance(value, datetime):
8852         return value.strftime("%H:%M")
8853     if isinstance(value, time):
8854         return value.strftime("%H:%M")
8855     text = str(value).strip()
8856     if text in {"0:00+1", "24:00"}:
8857         return text
8858     if len(text) == 4 and text[1] == ":":
8859         return "0" + text
8860     return text
8861
8862
8863 def normalize_clock_hour(value: object) -> int:
8864     text = normalize_time(value).replace(":", "")
8865     if text in {"0:00", "00:00"}:
8866         return 0
8867     hour, minute = text.split(":")[:2]
8868     if int(minute) != 0:
8869         raise ValueError(f"Attachment 3 issue time is not on the hour: {value!r}")
8870     return int(hour)
8871
8872
8873 def _finite_nonnegative(values: np.ndarray, name: str) -> None:
8874     if not np.isfinite(values).all():
8875         raise ValueError(f"{name} contains missing or non-finite values")
8876     if np.any(values < 0):
8877         raise ValueError(f"{name} contains negative values")
8878
8879
8880 @dataclass(frozen=True)
8881 class Q3Data:
8882     dates: pd.DatetimeIndex
8883     time_labels: tuple[str, ...]
8884     price: np.ndarray
8885     fallback_load: np.ndarray
8886     fallback_pv: np.ndarray
8887     load: np.ndarray
8888     pv: np.ndarray
8889     # issue_hour -> (n_days, 24) hourly PV forecast in kW
8890     hourly_forecast_kw: dict[int, np.ndarray]
8891
8892     def date_index(self, date: str) -> int:
8893         return int(self.dates.get_loc(pd.Timestamp(date)))
8894
8895
8896 def load_q3_data(
8897     attach1: Path = ATTACH1,
8898     attach2: Path = ATTACH2,
8899     attach3: Path = ATTACH3,
8900 ) -> Q3Data:

```

```

8901 missing = [str(path) for path in (attach1, attach2, attach3) if not path.exists()]
8902 if missing:
8903     raise FileNotFoundError(
8904         "Required attachments not found: "
8905         + ", ".join(missing)
8906         + ". Set CUMCM_C_ATTACH_DIR to the folder containing 附件1/2/3.xlsx."
8907     )
8908
8909 day = pd.read_excel(attach1, sheet_name="Sheet1")
8910 expected = ["时间", "电价", "小区负载", "光伏发电预测功率"]
8911 if list(day.columns) != expected or len(day) != T:
8912     raise ValueError("附件1 must have the four expected columns and 144 row-order rows")
8913 price = pd.to_numeric(day["电价"], errors="coerce").to_numpy(float)
8914 fallback_load = pd.to_numeric(day["小区负载"], errors="coerce").to_numpy(float) * DELTA_H
8915 fallback_pv = pd.to_numeric(day["光伏发电预测功率"], errors="coerce").to_numpy(float) * DELTA_H
8916 _finite_nonnegative(price, "附件1 price")
8917 _finite_nonnegative(fallback_load, "附件1 load forecast")
8918 _finite_nonnegative(fallback_pv, "附件1 PV forecast")
8919 if np.any(price <= 0):
8920     raise ValueError("Q3 requires strictly positive prices")
8921
8922 sheets = pd.read_excel(attach2, sheet_name=None, header=None)
8923 if list(sheets) != ["小区负载", "光伏发电实际功率"]:
8924     raise ValueError(f"Unexpected 附件2 sheets: {list(sheets)}")
8925 load_raw = sheets["小区负载"]
8926 pv_raw = sheets["光伏发电实际功率"]
8927 if load_raw.shape != (366, 145) or pv_raw.shape != (366, 145):
8928     raise ValueError(f"附件2 must be 366x145; got {load_raw.shape}, {pv_raw.shape}")
8929 if load_raw.iloc[0, 0] != "日期\\时间" or pv_raw.iloc[0, 0] != "日期\\时间":
8930     raise ValueError("附件2 first header must be 日期\\时间")
8931
8932 time_labels = tuple(normalize_time(x) for x in load_raw.iloc[0, 1:])
8933 pv_labels = tuple(normalize_time(x) for x in pv_raw.iloc[0, 1:])
8934 if time_labels != pv_labels or len(time_labels) != T:
8935     raise ValueError("附件2 load/PV time labels differ")
8936 if time_labels[0] != "00:10" or time_labels[-1] != "0:00+1":
8937     raise ValueError(f"Unexpected raw time order: {time_labels[0]} .. {time_labels[-1]}")
8938
8939 dates = pd.DatetimeIndex(pd.to_datetime(load_raw.iloc[1:, 0], errors="raise")).normalize()
8940 pv_dates = pd.DatetimeIndex(pd.to_datetime(pv_raw.iloc[1:, 0], errors="raise")).normalize()
8941 expected_dates = pd.date_range("2025-01-01", "2025-12-31", freq="D")
8942 if not dates.equals(expected_dates) or not pv_dates.equals(expected_dates):
8943     raise ValueError("附件2 dates must be continuous 2025-01-01 through 2025-12-31")
8944 load = load_raw.iloc[1:, 1:].apply(pd.to_numeric, errors="coerce").to_numpy(float) * DELTA_H
8945 pv = pv_raw.iloc[1:, 1:].apply(pd.to_numeric, errors="coerce").to_numpy(float) * DELTA_H
8946 _finite_nonnegative(load, "附件2 load")
8947 _finite_nonnegative(pv, "附件2 PV")
8948
8949 forecast = pd.read_excel(attach3, sheet_name=0)
8950 expected_forecast_cols = ["日期", "预报时刻"] + [f"预报{h}小时" for h in range(1, 25)]
8951 if list(forecast.columns) != expected_forecast_cols:
8952     raise ValueError(f"Unexpected 附件3 columns: {list(forecast.columns)}")
8953 if len(forecast) != 4 * len(dates):
8954     raise ValueError(f"附件3 must have 4 rows per day; got {len(forecast)}")
8955 forecast = forecast.copy()
8956 forecast["日期"] = forecast["日期"].ffill()
8957 forecast["date"] = pd.to_datetime(forecast["日期"], errors="raise").dt.normalize()
8958 forecast["issue_hour"] = forecast["预报时刻"].map(normalize_clock_hour)
8959 hourly_values = (
8960     forecast[[f"预报{h}小时" for h in range(1, 25)]]
8961     .apply(pd.to_numeric, errors="coerce")
8962     .to_numpy(float)
8963 )
8964 _finite_nonnegative(hourly_values, "附件3 PV forecast")
8965 if set(forecast["issue_hour"].unique()) != set(UPDATE_HOURS):
8966     raise ValueError(f"Unexpected 附件3 issue hours: {sorted(forecast['issue_hour'].unique())}")
8967 forecast_dates = pd.DatetimeIndex(forecast["date"].drop_duplicates()).normalize()
8968 if not forecast_dates.equals(dates):

```



```

8969         raise ValueError("附件3 dates do not match 附件2")
8970
8971     hourly_forecast_kw = {}
8972     for hour in UPDATE_HOURS:
8973         block = forecast.loc[forecast["issue_hour"] == hour].sort_values("date")
8974         if len(block) != len(dates):
8975             raise ValueError(f"附件3 is missing {hour:02d}:00 rows")
8976         if not pd.DatetimeIndex(block["date"]).equals(dates):
8977             raise ValueError(f"附件3 {hour:02d}:00 dates are not aligned")
8978         hourly_forecast_kw[hour] = block[[f"预报{h}小时" for h in range(1, 25)]].to_numpy(float)
8979
8980     return Q3Data(
8981         dates=dates,
8982         time_labels=time_labels,
8983         price=price,
8984         fallback_load=fallback_load,
8985         fallback_pv=fallback_pv,
8986         load=load,
8987         pv=pv,
8988         hourly_forecast_kw=hourly_forecast_kw,
8989     )
8990
8991
8992 def period_end_minutes(index: int) -> int:
8993     """Minutes after 0:00 at the end of the 10-minute period with internal index."""
8994     return 10 * (index + 1)
8995
8996
8997 def write_input_audit(data: Q3Data, path: Path) -> None:
8998     attach3_rows = []
8999     for date in PILOT_DATES:
9000         i = data.date_index(date)
9001         for hour in UPDATE_HOURS:
9002             hourly = data.hourly_forecast_kw[hour][i]
9003             attach3_rows.append(
9004                 {
9005                     "date": date,
9006                     "issue_hour": hour,
9007                     "n_hourly_nodes": 24,
9008                     "first_hour_kw": float(hourly[0]),
9009                     "hour12_kw": float(hourly[11]),
9010                     "last_hour_kw": float(hourly[23]),
9011                     "sum_kw": float(hourly.sum()),
9012                     "zero_count": int(np.sum(hourly == 0.0)),
9013                 }
9014             )
9015     audit = {
9016         "shape": {
9017             "days": len(data.dates),
9018             "periods_per_day": T,
9019             "attach3_rows": 4 * len(data.dates),
9020         },
9021         "date_start": data.dates[0].strftime("%Y-%m-%d"),
9022         "date_end": data.dates[-1].strftime("%Y-%m-%d"),
9023         "dates_continuous": True,
9024         "missing_count": int(
9025             np.isnan(data.load).sum()
9026             + np.isnan(data.pv).sum()
9027             + sum(np.isnan(arr).sum() for arr in data.hourly_forecast_kw.values())
9028         ),
9029         "negative_load_count": int(np.sum(data.load < 0)),
9030         "negative_pv_count": int(np.sum(data.pv < 0)),
9031         "negative_forecast_count": int(
9032             sum(np.sum(arr < 0) for arr in data.hourly_forecast_kw.values())
9033         ),
9034         "price_min_yuan_per_kwh": float(data.price.min()),
9035         "unit_conversion": "attachment powers multiplied by 1/6 h to obtain kWh",
9036         "time_mapping": (

```

```

9037         "attachment raw column t maps to internal index t=0.143 and template column t; "
9038         "no sorting, rotation, or invented 0:00-0:10 period"
9039     ),
9040     "first_time_label": data.time_labels[0],
9041     "last_time_label": data.time_labels[-1],
9042     "attach3_issue_hours": list(UPDATE_HOURS),
9043     "attach3_complete_four_rows_per_day": True,
9044     "forecast_horizon": (
9045         "each 附件3 row is a 24-hour-ahead on-the-hour forecast in kW; "
9046         "hour k after issue hour H is clock time H+k, wrapping into the next day"
9047     ),
9048     "update_index_map": {
9049         "0:00": {"first_mutable_index": 0, "label": data.time_labels[0]},
9050         "6:00": {
9051             "last_locked_index": 35,
9052             "last_locked_label": data.time_labels[35],
9053             "first_mutable_index": 36,
9054             "first_mutable_label": data.time_labels[36],
9055         },
9056         "12:00": {
9057             "last_locked_index": 71,
9058             "last_locked_label": data.time_labels[71],
9059             "first_mutable_index": 72,
9060             "first_mutable_label": data.time_labels[72],
9061         },
9062         "18:00": {
9063             "last_locked_index": 107,
9064             "last_locked_label": data.time_labels[107],
9065             "first_mutable_index": 108,
9066             "first_mutable_label": data.time_labels[108],
9067         },
9068     },
9069     "pilot_attach3_spotcheck": attach3_rows,
9070     "daily_energy_pilot": [
9071         {
9072             "date": date,
9073             "load_kwh": float(data.load[data.date_index(date)].sum()),
9074             "pv_kwh": float(data.pv[data.date_index(date)].sum()),
9075             "net_load_kwh": float(
9076                 data.load[data.date_index(date)].sum()
9077                 - data.pv[data.date_index(date)].sum()
9078             ),
9079         }
9080         for date in PILOT_DATES
9081     ],
9082 }
9083 path.parent.mkdir(parents=True, exist_ok=True)
9084 path.write_text(json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
9085
9086 # ===== q3/export_result3.py =====
9087 """Export the official Q3 workbook from a single A + M1_M6 10-minute trajectory.
9088
9089 January is solved only for SOC continuity. The contest workbook covers
9090 2025-02-01 through 2025-12-31. Does not reconstruct intervals from daily
9091 summaries and does not rerun the other annual paths.
9092 """
9093
9094 from __future__ import annotations
9095
9096 import json
9097 import shutil
9098 import subprocess
9099 from datetime import datetime, time
9100 from pathlib import Path
9101
9102 import numpy as np
9103 import pandas as pd
9104 from openpyxl import load_workbook

```

```

9105 from openpyxl.styles import Alignment, Font
9106 from openpyxl.utils import get_column_letter
9107
9108 from q3.config import (
9109     ANNUAL_END,
9110     ANNUAL_OUTPUT_DIR,
9111     ANNUAL_START,
9112     FOUR_HOUR_BLOCKS,
9113     HOUR_TO_FIRST_MUTABLE,
9114     REQUIRED_TRAJECTORY_COLS,
9115     RESULT3_ANNUAL_COST_YUAN,
9116     RESULT3_EXPORT_AUDIT,
9117     RESULT3_EXPORT_END,
9118     RESULT3_EXPORT_N_DAYS,
9119     RESULT3_EXPORT_START,
9120     RESULT3_OFFICIAL_STRATEGY,
9121     RESULT3_TEMPLATE,
9122     RESULT3_XLSX,
9123     ROOT,
9124     T,
9125     YEAR_END_BOUNDARY_A,
9126     YEAR_END_SOC_A_KWH,
9127     dispatch_stem,
9128 )
9129 from q3.pilot import DayRun
9130
9131 PURCHASE_SHEET = "计划购电量"
9132 ADJUST_SHEET = "调整购电量"
9133 CHARGE_SHEET = "充放电电量"
9134 EMERGENCY_SHEET = "紧急购电量"
9135 NOTES_SHEET = "说明"
9136 N_TIME_COLS = T
9137 COL_FIRST_TIME = 2
9138 COL_DAY_QTY = 146
9139 COL_DAY_FEE = 147
9140 COST_MATCH_TOL_YUAN = 5e-3
9141 ENERGY_MATCH_TOL_KWH = 1e-3
9142 SOC_MATCH_TOL_KWH = 1e-3
9143 EMERGENCY_EPS_KWH = 1e-8
9144
9145 TRAJECTORY_WRITE_COLS = list(REQUIRED_TRAJECTORY_COLS) + [
9146     "price",
9147     "phi_yuan",
9148     "emergency_cost_yuan",
9149     "run_id",
9150     "year_end_boundary",
9151     "year_end_soc_kwh",
9152     "strategy",
9153     "balance_residual_kwh",
9154 ]
9155
9156
9157 def git_hash() -> str:
9158     try:
9159         return subprocess.check_output(
9160             ["git", "rev-parse", "HEAD"], cwd=ROOT, text=True
9161         ).strip()
9162     except (OSError, subprocess.CalledProcessError):
9163         return "unknown"
9164
9165
9166 def export_dates() -> list[str]:
9167     return [
9168         ts.strftime("%Y-%m-%d")
9169         for ts in pd.date_range(RESULT3_EXPORT_START, RESULT3_EXPORT_END, freq="D")
9170     ]
9171
9172

```

```

9173 def annual_calendar() -> list[str]:
9174     return [
9175         ts.strftime("%Y-%m-%d")
9176         for ts in pd.date_range(ANNUAL_START, ANNUAL_END, freq="D")
9177     ]
9178
9179
9180 def main_dispatch_stem(date: str) -> str:
9181     return dispatch_stem(
9182         date,
9183         RESULT3_OFFICIAL_STRATEGY,
9184         "causal_load_main",
9185         "linear_anchor_main",
9186         "anchor_final_main",
9187         True,
9188         YEAR_END_SOC_A_KWH,
9189     )
9190
9191
9192 def dispatch_path(date: str, dispatch_dir: Path | None = None) -> Path:
9193     folder = dispatch_dir or (ANNUAL_OUTPUT_DIR / "dispatch_daily")
9194     return folder / f"{main_dispatch_stem(date)}.csv"
9195
9196
9197 def _update_hour(last_update_time: object) -> int:
9198     text = str(last_update_time).strip()
9199     hour = int(text.split(":")[0])
9200     if hour not in HOUR_TO_FIRST_MUTABLE:
9201         raise ValueError(f"unknown last_update_time {last_update_time!r}")
9202     return hour
9203
9204
9205 def locked_or_mutable_label(period_index: int, last_update_time: object) -> str:
9206     first = HOUR_TO_FIRST_MUTABLE[_update_hour(last_update_time)]
9207     return "locked" if int(period_index) < first else "mutable"
9208
9209
9210 def enrich_dispatch(run: DayRun) -> pd.DataFrame:
9211     raw = run.dispatch.copy()
9212     n = len(raw)
9213     if n != T:
9214         raise ValueError(f"{run.date} dispatch has {n} rows, expected {T}")
9215     soc_end = raw["soc_kwh"].to_numpy(float)
9216     soc_start = np.empty(n, dtype=float)
9217     soc_start[0] = float(run.summary["soc_start_kwh"])
9218     soc_start[1:] = soc_end[:-1]
9219     periods = raw["period"].to_numpy(int) if "period" in raw.columns else np.arange(n)
9220     last_update = raw["last_update_time"].astype(str)
9221     frame = pd.DataFrame(
9222         {
9223             "date": run.date,
9224             "period_index": periods,
9225             "time_label": raw["time"].astype(str),
9226             "g0_kwh": raw["planned_g0_kwh"].to_numpy(float),
9227             "g_final_kwh": raw["final_g_kwh"].to_numpy(float),
9228             "x_kwh": raw["normal_x_kwh"].to_numpy(float),
9229             "emergency_kwh": raw["emergency_kwh"].to_numpy(float),
9230             "charge_kwh": raw["charge_kwh"].to_numpy(float),
9231             "discharge_kwh": raw["discharge_kwh"].to_numpy(float),
9232             "curtailment_kwh": raw["curtailment_kwh"].to_numpy(float),
9233             "soc_start_kwh": soc_start,
9234             "soc_end_kwh": soc_end,
9235             "actual_load_kwh": raw["load_kwh"].to_numpy(float),
9236             "actual_pv_kwh": raw["actual_pv_kwh"].to_numpy(float),
9237             "locked_or_mutable": [
9238                 locked_or_mutable_label(int(t), u) for t, u in zip(periods, last_update)
9239             ],
9240             "last_update_time": last_update,

```

```

9241         "price": raw["price"].to_numpy(float),
9242         "phi_yuan": raw["phi_yuan"].to_numpy(float),
9243         "emergency_cost_yuan": raw["emergency_cost_yuan"].to_numpy(float),
9244         "run_id": run.run_id,
9245         "year_end_boundary": run.summary.get("year_end_boundary", YEAR_END_BOUNDARY_A),
9246         "year_end_soc_kwh": YEAR_END_SOC_A_KWH,
9247         "strategy": run.strategy,
9248         "balance_residual_kwh": raw["balance_residual_kwh"].to_numpy(float)
9249         if "balance_residual_kwh" in raw.columns
9250         else np.zeros(n),
9251     }
9252 )
9253 return frame[TRAJECTORY_WRITE_COLS]
9254
9255
9256 def has_required_trajectory(frame: pd.DataFrame) -> bool:
9257     return all(col in frame.columns for col in REQUIRED_TRAJECTORY_COLS) and len(frame) == T
9258
9259
9260 def write_daily_trajectory(frame: pd.DataFrame, path: Path) -> None:
9261     path.parent.mkdir(parents=True, exist_ok=True)
9262     missing = [col for col in REQUIRED_TRAJECTORY_COLS if col not in frame.columns]
9263     if missing:
9264         raise ValueError(f"{path} missing trajectory columns: {missing}")
9265     if len(frame) != T:
9266         raise ValueError(f"{path} has {len(frame)} rows, expected {T}")
9267     frame.to_csv(path, index=False)
9268
9269
9270 def load_daily_trajectory(path: Path) -> pd.DataFrame:
9271     frame = pd.read_csv(path)
9272     if not has_required_trajectory(frame):
9273         raise ValueError(f"{path} is not a streamed 10-minute export trajectory")
9274     return frame.sort_values("period_index").reset_index(drop=True)
9275
9276
9277 def summarize_trajectory(frame: pd.DataFrame) -> dict:
9278     return {
9279         "date": str(frame["date"].iloc[0]),
9280         "strategy": RESULT3_OFFICIAL_STRATEGY,
9281         "year_end_boundary": YEAR_END_BOUNDARY_A,
9282         "year_end_soc_kwh": YEAR_END_SOC_A_KWH,
9283         "total_cost_yuan": float(frame["phi_yuan"].sum() + frame["emergency_cost_yuan"].sum()),
9284         "settlement_cost_yuan": float(frame["phi_yuan"].sum()),
9285         "emergency_cost_yuan": float(frame["emergency_cost_yuan"].sum()),
9286         "emergency_kwh": float(frame["emergency_kwh"].sum()),
9287         "curtailment_kwh": float(frame["curtailment_kwh"].sum()),
9288         "charge_kwh": float(frame["charge_kwh"].sum()),
9289         "discharge_kwh": float(frame["discharge_kwh"].sum()),
9290         "soc_start_kwh": float(frame["soc_start_kwh"].iloc[0]),
9291         "soc_end_kwh": float(frame["soc_end_kwh"].iloc[-1]),
9292         "g0_kwh": float(frame["g0_kwh"].sum()),
9293         "g_final_kwh": float(frame["g_final_kwh"].sum()),
9294         "planned_energy_cost_yuan": float((frame["price"] * frame["g0_kwh"]).sum()),
9295         "max_balance_residual_kwh": float(frame["balance_residual_kwh"].abs().max())
9296         if "balance_residual_kwh" in frame.columns
9297         else 0.0,
9298     }
9299
9300
9301 def _excel_date(value: object) -> datetime:
9302     ts = pd.Timestamp(value)
9303     return datetime(ts.year, ts.month, ts.day)
9304
9305
9306 def _time_headers(ws) -> list[str]:
9307     headers = [str(ws.cell(1, COL_FIRST_TIME + t).value) for t in range(N_TIME_COLS)]
9308     if headers[0] != "0:10-0:20" or headers[-1] != "0:00-0:10+1":

```

```

9309         raise ValueError(f"unexpected result3 time headers: {headers[0]!r} .. {headers[-1]!r}")
9310     return headers
9311
9312
9313 def _fill_energy_grid(ws, by_date: dict[str, pd.DataFrame], value_col: str, fee_col: str) -> None:
9314     dates = export_dates()
9315     if ws.max_row != 1 + RESULT3_EXPORT_N_DAYS:
9316         raise ValueError(f"{ws.title} expected {1 + RESULT3_EXPORT_N_DAYS} rows, got {ws.max_row}")
9317     for offset, date in enumerate(dates):
9318         row = 2 + offset
9319         cell_date = _excel_date(ws.cell(row, 1).value)
9320         if cell_date.strftime("%Y-%m-%d") != date:
9321             raise ValueError(f"{ws.title} row {row} date {cell_date} != {date}")
9322         frame = by_date[date].sort_values("period_index")
9323         if len(frame) != T:
9324             raise ValueError(f"{date} {value_col} has {len(frame)} periods")
9325         values = frame[value_col].to_numpy(float)
9326         for t, val in enumerate(values):
9327             if not np.isfinite(val):
9328                 raise ValueError(f"{ws.title} {date} t={t} is not finite")
9329             ws.cell(row, COL_FIRST_TIME + t).value = float(val)
9330         ws.cell(row, COL_DAY_QTY).value = float(np.sum(values))
9331         ws.cell(row, COL_DAY_FEE).value = float(frame[fee_col].sum())
9332         ws.cell(row, 1).value = cell_date
9333         ws.cell(row, 1).number_format = "yyyy-mm-dd"
9334
9335
9336 def _rewrite_charge_sheet(ws, by_date: dict[str, pd.DataFrame]) -> None:
9337     header = [ws.cell(1, c).value for c in range(1, 7)]
9338     if header != ["日期", "时间段", "充电量", "放电量", "时刻", "储电量"]:
9339         raise ValueError(f"unexpected 充放电量 header: {header}")
9340     if ws.max_row > 1:
9341         ws.delete_rows(2, ws.max_row - 1)
9342     dates = export_dates()
9343     row = 2
9344     for date in dates:
9345         frame = by_date[date].sort_values("period_index")
9346         charge = frame["charge_kwh"].to_numpy(float)
9347         discharge = frame["discharge_kwh"].to_numpy(float)
9348         soc_start = float(frame["soc_start_kwh"].iloc[0])
9349         soc_end = float(frame["soc_end_kwh"].iloc[-1])
9350         for b, (label, start, end) in enumerate(FOUR_HOUR_BLOCKS):
9351             ws.cell(row, 1).value = _excel_date(date) if b == 0 else None
9352             if b == 0:
9353                 ws.cell(row, 1).number_format = "yyyy-mm-dd"
9354             ws.cell(row, 2).value = label
9355             ws.cell(row, 3).value = float(np.sum(charge[start:end]))
9356             ws.cell(row, 4).value = float(np.sum(discharge[start:end]))
9357             if b == 0:
9358                 ws.cell(row, 5).value = time(0, 0)
9359                 ws.cell(row, 5).number_format = "h:mm"
9360                 ws.cell(row, 6).value = soc_start
9361             elif b == 1:
9362                 ws.cell(row, 5).value = "24:00"
9363                 ws.cell(row, 6).value = soc_end
9364             else:
9365                 ws.cell(row, 5).value = None
9366                 ws.cell(row, 6).value = None
9367             row += 1
9368         expected = 1 + RESULT3_EXPORT_N_DAYS * len(FOUR_HOUR_BLOCKS)
9369         if ws.max_row != expected:
9370             raise ValueError(f"充放电量 expected {expected} rows, got {ws.max_row}")
9371
9372
9373 def _rewrite_emergency_sheet(ws, by_date: dict[str, pd.DataFrame], interval_headers: list[str]) -> int:
9374     header = [ws.cell(1, c).value for c in range(1, 4)]
9375     if header != ["日期", "购电时间段", "购电量"]:
9376         raise ValueError(f"unexpected 紧急购电量 header: {header}")

```

```

9377     if ws.max_row > 1:
9378         ws.delete_rows(2, ws.max_row - 1)
9379     n_rows = 0
9380     row = 2
9381     for date in export_dates():
9382         frame = by_date[date].sort_values("period_index")
9383         emergency = frame["emergency_kwh"].to_numpy(float)
9384         for t, value in enumerate(emergency):
9385             if value <= EMERGENCY_EPS_KWH:
9386                 continue
9387             ws.cell(row, 1).value = _excel_date(date)
9388             ws.cell(row, 1).number_format = "yyyy-mm-dd"
9389             ws.cell(row, 2).value = interval_headers[t]
9390             ws.cell(row, 3).value = float(value)
9391             n_rows += 1
9392             row += 1
9393     return n_rows
9394
9395
9396 def _write_notes_sheet(wb, source_commit: str, n_emergency: int, export_cost: float) -> None:
9397     if NOTES_SHEET in wb.sheetnames:
9398         del wb[NOTES_SHEET]
9399     ws = wb.create_sheet(NOTES_SHEET)
9400     lines = [
9401         "C 题第 3 问正式导出 (唯一主路径)",
9402         "主策略: M1_M6 (0:00 制定 g0; 6:00/12:00/18:00 评估是否调整未执行承诺)",
9403         "年末 SOC: A / A_q2_aligned / 1200 kWh",
9404         "负荷信息: causal_load_main; 光伏映射: linear_anchor_main; 调整结算: anchor_final_main",
9405         "初始 SOC: 2025-01-01 00:00 = 6000 kWh, 跨日连续 E_{d+1,0}=E_{d,144}",
9406         "输出区间: 2025-02-01 至 2025-12-31 (334 天)。1 月只用于预热、预测历史和 SOC 连续, 不写入本表。",
9407         "单位: 购电/充放电/紧急电量为 kWh; 全天购电费为元。",
9408         "计划购电量: 日初承诺 g0, 时间列按附件 00:10 ... 0:00+1 原顺序, 不旋转。全天购电量为行合计; 全天购电费为附件1电价与 g0 之积 (不含紧急购电惩罚)。",
9409         "调整购电量: 最终有效承诺 gF; 未调整时段等于 g0, 无空值。全天购电费为实际结算 phi(g0,gF)+5 p e。",
9410         "充放电电量: 按执行轨迹的充电量 c、放电量 r 分 6 个四小时段合计; 每日第一行储电量为 0:00 SOC, 第二行为 24:00 SOC。SOC 是存量, 不可跨行或跨日求和。",
9411         f"紧急购电量: 仅列出 e>0 的时段 (本文件 {n_emergency} 行)。不得把可由电池平衡的普通缺口列入。",
9412         f"数据来源 commit: {source_commit}",
9413         "本文件不含 B 路径、oracle、中间试验或其它策略。",
9414         f"2-12 月输出区间实际结算合计 {export_cost:.2f} 元; 不可用该数字替代 1-12 月年度主结论 {RESULT3_ANNUAL_COST_YUAN:.2f} 元。",
9415     ]
9416     font = Font(name="宋体", size=11)
9417     for i, line in enumerate(lines, start=1):
9418         cell = ws.cell(i, 1)
9419         cell.value = line
9420         cell.font = font
9421         cell.alignment = Alignment(wrap_text=True, vertical="top")
9422         ws.row_dimensions[i].height = 22
9423     ws.column_dimensions["A"].width = 118
9424
9425
9426 def write_result3_xlsx(
9427     by_date: dict[str, pd.DataFrame],
9428     dest: Path = RESULT3_XLSX,
9429     template: Path = RESULT3_TEMPLATE,
9430     source_commit: str | None = None,
9431 ) -> dict:
9432     dates = export_dates()
9433     missing = [d for d in dates if d not in by_date]
9434     if missing:
9435         raise FileNotFoundError(f"missing export-window trajectories: {missing[:5]}")
9436     if not template.exists():
9437         raise FileNotFoundError(f"result3 template not found: {template}")
9438     dest.parent.mkdir(parents=True, exist_ok=True)
9439     shutil.copyfile(template, dest)
9440     wb = load_workbook(dest)
9441     for name in (PURCHASE_SHEET, ADJUST_SHEET, CHARGE_SHEET, EMERGENCY_SHEET):
9442         if name not in wb.sheetnames:

```

```

9443         raise ValueError(f"template missing sheet {name}")
9444     plan = wb[PURCHASE_SHEET]
9445     adjust = wb[ADJUST_SHEET]
9446     interval_headers = _time_headers(plan)
9447     if _time_headers(adjust) != interval_headers:
9448         raise ValueError("计划/调整 time headers differ")
9449     fee_plan = {}
9450     fee_adjust = {}
9451     grids = {}
9452     for date in dates:
9453         frame = by_date[date].copy()
9454         frame["plan_fee_yuan"] = frame["price"] * frame["g0_kwh"]
9455         frame["actual_fee_yuan"] = frame["phi_yuan"] + frame["emergency_cost_yuan"]
9456         grids[date] = frame
9457         fee_plan[date] = float(frame["plan_fee_yuan"].sum())
9458         fee_adjust[date] = float(frame["actual_fee_yuan"].sum())
9459     _fill_energy_grid(plan, grids, "g0_kwh", "plan_fee_yuan")
9460     _fill_energy_grid(adjust, grids, "g_final_kwh", "actual_fee_yuan")
9461     _rewrite_charge_sheet(wb[CHARGE_SHEET], grids)
9462     n_emergency = _rewrite_emergency_sheet(wb[EMERGENCY_SHEET], grids, interval_headers)
9463     export_cost = float(sum(fee_adjust.values()))
9464     _write_notes_sheet(wb, source_commit or git_hash(), n_emergency, export_cost)
9465     for sheet in (PURCHASE_SHEET, ADJUST_SHEET):
9466         ws = wb[sheet]
9467         ws.cell(1, COL_DAY_QTY).value = "全天购电量"
9468         ws.cell(1, COL_DAY_FEE).value = "全天购电费"
9469         ws.column_dimensions[get_column_letter(COL_DAY_QTY)].width = 14
9470         ws.column_dimensions[get_column_letter(COL_DAY_FEE)].width = 14
9471     wb.save(dest)
9472     return {
9473         "path": str(dest),
9474         "n_export_days": len(dates),
9475         "n_emergency_rows": n_emergency,
9476         "export_window_cost_yuan": export_cost,
9477         "interval_headers_first": interval_headers[0],
9478         "interval_headers_last": interval_headers[-1],
9479         "planned_window_cost_yuan": float(sum(fee_plan.values()))},
9480     }
9481
9482
9483 def _blank_or_ref(value: object) -> bool:
9484     if value is None:
9485         return True
9486     text = str(value)
9487     return "#REF!" in text or text.strip() == ""
9488
9489
9490 def inspect_result3_workbook(path: Path) -> dict:
9491     wb = load_workbook(path, data_only=False)
9492     plan = wb[PURCHASE_SHEET]
9493     adjust = wb[ADJUST_SHEET]
9494     charge = wb[CHARGE_SHEET]
9495     emergency = wb[EMERGENCY_SHEET]
9496     anomalies: list[str] = []
9497     if NOTES_SHEET not in wb.sheetnames:
9498         anomalies.append("missing 说明 sheet")
9499     dates = export_dates()
9500     if plan.max_row != 1 + RESULT3_EXPORT_N_DAYS or adjust.max_row != 1 + RESULT3_EXPORT_N_DAYS:
9501         anomalies.append("purchase sheets do not have 334 date rows")
9502     n_blank_adjust = 0
9503     n_ref = 0
9504     plan_dates = []
9505     for offset, date in enumerate(dates):
9506         row = 2 + offset
9507         for sheet in (plan, adjust):
9508             raw = sheet.cell(row, 1).value
9509             got = pd.Timestamp(raw).strftime("%Y-%m-%d") if raw is not None else None
9510             if got != date:

```



```

9511         anomalies.append(f"{sheet.title} row {row} date {got} != {date}")
9512     for col in range(COL_FIRST_TIME, COL_DAY_FEE + 1):
9513         val = sheet.cell(row, col).value
9514         if val is not None and "#REF!" in str(val):
9515             n_ref += 1
9516         if sheet is adjust and col < COL_DAY_QTY and _blank_or_ref(val):
9517             n_blank_adjust += 1
9518         if sheet is plan and col < COL_DAY_QTY and _blank_or_ref(val):
9519             anomalies.append(f"{sheet.title} {date} col {col} blank")
9520         break
9521     plan_dates.append(date)
9522 if n_blank_adjust:
9523     anomalies.append(f"调整购电量 has {n_blank_adjust} blank interval cells")
9524 if n_ref:
9525     anomalies.append(f"workbook contains {n_ref} #REF! cells")
9526 expected_charge_rows = 1 + RESULT3_EXPORT_N_DAYS * len(FOUR_HOUR_BLOCKS)
9527 if charge.max_row != expected_charge_rows:
9528     anomalies.append(f"充放电量 rows {charge.max_row} != {expected_charge_rows}")
9529 jan_leaks = [
9530     pd.Timestamp(plan.cell(r, 1).value).month == 1
9531     for r in range(2, plan.max_row + 1)
9532     if plan.cell(r, 1).value is not None
9533 ]
9534 if any(jan_leaks):
9535     anomalies.append("January dates leaked into 计划购电量")
9536 return {
9537     "n_plan_dates": len(plan_dates),
9538     "n_adjust_dates": adjust.max_row - 1,
9539     "n_charge_rows": charge.max_row - 1,
9540     "n_emergency_rows": max(emergency.max_row - 1, 0),
9541     "has_notes_sheet": NOTES_SHEET in wb.sheetnames,
9542     "n_anomalies": len(anomalies),
9543     "anomalies": anomalies,
9544     "first_date": plan_dates[0] if plan_dates else None,
9545     "last_date": plan_dates[-1] if plan_dates else None,
9546     "n_time_columns": N_TIME_COLS,
9547 }
9548
9549
9550 def compare_to_annual_summary(
9551     rerun_daily: pd.DataFrame,
9552     summary_path: Path,
9553 ) -> dict:
9554     if not summary_path.exists():
9555         return {"compared": False, "reason": f"missing {summary_path}"}
9556     archive = pd.read_csv(summary_path)
9557     archive = archive[
9558         (archive["strategy"] == RESULT3_OFFICIAL_STRATEGY)
9559         & (archive["year_end_boundary"] == YEAR_END_BOUNDARY_A)
9560     ].copy()
9561     archive["date"] = pd.to_datetime(archive["date"]).dt.strftime("%Y-%m-%d")
9562     rerun = rerun_daily.copy()
9563     rerun["date"] = pd.to_datetime(rerun["date"]).dt.strftime("%Y-%m-%d")
9564     merged = rerun.merge(archive, on="date", suffixes=("_rerun", "_archive"), how="left")
9565     missing = merged[merged["total_cost_yuan_archive"].isna()]["date"].tolist()
9566     diffs = []
9567     fields = [
9568         ("total_cost_yuan", COST_MATCH_TOL_YUAN),
9569         ("emergency_kwh", ENERGY_MATCH_TOL_KWH),
9570         ("curtailment_kwh", ENERGY_MATCH_TOL_KWH),
9571         ("soc_start_kwh", SOC_MATCH_TOL_KWH),
9572         ("soc_end_kwh", SOC_MATCH_TOL_KWH),
9573     ]
9574     max_abs = {name: 0.0 for name, _tol in fields}
9575     n_fail = 0
9576     for _, row in merged.iterrows():
9577         if pd.isna(row.get("total_cost_yuan_archive")):
9578             continue

```

```

9579         for name, tol in fields:
9580             delta = abs(float(row[f"{name}_rerun"]) - float(row[f"{name}_archive"]))
9581             max_abs[name] = max(max_abs[name], delta)
9582             if delta > tol:
9583                 n_fail += 1
9584                 diffs.append(
9585                     {
9586                         "date": row["date"],
9587                         "field": name,
9588                         "rerun": float(row[f"{name}_rerun"]),
9589                         "archive": float(row[f"{name}_archive"]),
9590                         "abs_diff": delta,
9591                     }
9592                 )
9593         return {
9594             "compared": True,
9595             "n_rerun_days": int(len(rerun)),
9596             "n_archive_days": int(len(archive)),
9597             "n_unmatched_dates": missing,
9598             "max_abs_diff": max_abs,
9599             "n_field_failures": n_fail,
9600             "failures": diffs[:50],
9601             "all_match": n_fail == 0 and not missing,
9602             "tolerance_yuan": COST_MATCH_TOL_YUAN,
9603             "tolerance_kwh": ENERGY_MATCH_TOL_KWH,
9604         }
9605
9606
9607 def write_export_audit(
9608     payload: dict,
9609     path: Path = RESULT3_EXPORT_AUDIT,
9610 ) -> Path:
9611     path.parent.mkdir(parents=True, exist_ok=True)
9612     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n", encoding="utf-8")
9613     return path
9614
9615 # ===== q3/forecast.py =====
9616 from __future__ import annotations
9617
9618 from dataclasses import dataclass
9619 from pathlib import Path
9620
9621 import numpy as np
9622 import pandas as pd
9623
9624 from q3.config import (
9625     DELTA_H,
9626     HOUR_TO_FIRST_MUTABLE,
9627     LOAD_HISTORY_SAME_WEEKDAY,
9628     LOAD_INFORMATION_MAIN,
9629     LOAD_INFORMATION_PROXY,
9630     NEXT_DAY_PV_HISTORY_DAYS,
9631     PILOT_DATES,
9632     PV_MAPPING_LINEAR,
9633     PV_MAPPING_STEP,
9634     T,
9635     UPDATE_HOURS,
9636 )
9637 from q3.data import Q3Data, period_end_minutes
9638
9639
9640 def previous_end_pv_kw(data: Q3Data, day_index: int) -> float:
9641     """Actual PV at today's 0:00, i.e. the previous day's 24:00 observation."""
9642     if day_index <= 0:
9643         return 0.0
9644     return float(data.pv[day_index - 1, -1] / DELTA_H)
9645
9646

```

```

9647 def actual_kw(data: Q3Data, day_index: int, index: int) -> float:
9648     return float(data.pv[day_index, index] / DELTA_H)
9649
9650
9651 def forecast_knots_minutes_kw(
9652     issue_hour: int, hourly_kw: np.ndarray, anchor_kw: float
9653 ) -> tuple[np.ndarray, np.ndarray]:
9654     """On-the-hour knots for one 24-hour-ahead forecast, plus the causal actual anchor."""
9655     hourly_kw = np.asarray(hourly_kw, dtype=float).ravel()
9656     if hourly_kw.shape != (24,):
9657         raise ValueError("hourly forecast must have 24 nodes")
9658     minutes = [issue_hour * 60]
9659     values = [float(anchor_kw)]
9660     for step, value in enumerate(hourly_kw, start=1):
9661         minutes.append(issue_hour * 60 + 60 * step)
9662         values.append(float(value))
9663     return np.asarray(minutes, dtype=float), np.asarray(values, dtype=float)
9664
9665
9666 def interpolate_series(query_minutes: np.ndarray, knot_minutes: np.ndarray, knot_values: np.ndarray) -> np.ndarray:
9667     order = np.argsort(knot_minutes)
9668     return np.interp(query_minutes, knot_minutes[order], knot_values[order])
9669
9670
9671 def interpolate_weights(
9672     query_minute: float, knot_minutes: np.ndarray, knot_values: np.ndarray
9673 ) -> dict:
9674     order = np.argsort(knot_minutes)
9675     minutes = knot_minutes[order]
9676     values = knot_values[order]
9677     if query_minute <= minutes[0]:
9678         return {
9679             "left_node_minute": float(minutes[0]),
9680             "right_node_minute": float(minutes[0]),
9681             "left_weight": 1.0,
9682             "right_weight": 0.0,
9683             "left_kw": float(values[0]),
9684             "right_kw": float(values[0]),
9685         }
9686     if query_minute >= minutes[-1]:
9687         return {
9688             "left_node_minute": float(minutes[-1]),
9689             "right_node_minute": float(minutes[-1]),
9690             "left_weight": 1.0,
9691             "right_weight": 0.0,
9692             "left_kw": float(values[-1]),
9693             "right_kw": float(values[-1]),
9694         }
9695     right = int(np.searchsorted(minutes, query_minute, side="left"))
9696     left = right - 1
9697     span = minutes[right] - minutes[left]
9698     right_weight = (query_minute - minutes[left]) / span
9699     left_weight = 1.0 - right_weight
9700     return {
9701         "left_node_minute": float(minutes[left]),
9702         "right_node_minute": float(minutes[right]),
9703         "left_weight": float(left_weight),
9704         "right_weight": float(right_weight),
9705         "left_kw": float(values[left]),
9706         "right_kw": float(values[right]),
9707     }
9708
9709
9710 @dataclass(frozen=True)
9711 class MappedForecast:
9712     today_kwh: np.ndarray
9713     next_day_kwh: np.ndarray
9714     knot_minutes: np.ndarray

```

```

9715     knot_kw: np.ndarray
9716     issue_hour: int
9717     first_mutable_index: int
9718     pv_mapping_mode: str
9719     today_source_k: np.ndarray
9720     next_source_k: np.ndarray
9721     today_covered: np.ndarray
9722     next_covered: np.ndarray
9723
9724
9725 def forecast_hour_k(end_minutes: float, issue_hour: int) -> int:
9726     """1-based 预报k小时 index for a period ending at end_minutes after today's 0:00."""
9727     delta_min = float(end_minutes) - 60.0 * issue_hour
9728     if delta_min <= 1e-9:
9729         return 0
9730     return int(np.ceil(delta_min / 60.0 - 1e-12))
9731
9732
9733 def step_hourly_energy(hourly_kw: np.ndarray, issue_hour: int) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
9734     """Piecewise-constant map: 预报 k 小时 fills (H+k-1, H+k]."""
9735     hourly_kw = np.asarray(hourly_kw, dtype=float).ravel()
9736     today = np.zeros(T)
9737     nxt = np.zeros(T)
9738     today_k = np.zeros(T, dtype=int)
9739     next_k = np.zeros(T, dtype=int)
9740     for t in range(T):
9741         end_today = float(period_end_minutes(t))
9742         k_today = forecast_hour_k(end_today, issue_hour)
9743         today_k[t] = k_today if 1 <= k_today <= 24 else 0
9744         if 1 <= k_today <= 24:
9745             today[t] = hourly_kw[k_today - 1] * DELTA_H
9746         k_next = forecast_hour_k(end_today + 24 * 60, issue_hour)
9747         next_k[t] = k_next if 1 <= k_next <= 24 else 0
9748         if 1 <= k_next <= 24:
9749             nxt[t] = hourly_kw[k_next - 1] * DELTA_H
9750     return today, nxt, today_k, next_k
9751
9752
9753 def map_issue_forecast(
9754     data: Q3Data,
9755     day_index: int,
9756     issue_hour: int,
9757     pv_mapping_mode: str = PV_MAPPING_LINEAR,
9758 ) -> MappedForecast:
9759     """Map one 附件3 row onto today/next-day 10-minute energy using only causal anchors."""
9760     hourly_kw = data.hourly_forecast_kw[issue_hour][day_index]
9761     first_mutable = HOUR_TO_FIRST_MUTABLE[issue_hour]
9762     if pv_mapping_mode == PV_MAPPING_STEP:
9763         today_kwh, next_kwh, today_k, next_k = step_hourly_energy(hourly_kw, issue_hour)
9764         knot_minutes = issue_hour * 60 + 60 * np.arange(1, 25, dtype=float)
9765         knot_kw = np.asarray(hourly_kw, dtype=float).ravel()
9766         today_covered = today_k > 0
9767         next_covered = next_k > 0
9768     elif pv_mapping_mode == PV_MAPPING_LINEAR:
9769         if issue_hour == 0:
9770             anchor_kw = previous_end_pv_kw(data, day_index)
9771         else:
9772             anchor_index = 6 * issue_hour - 1
9773             anchor_kw = actual_kw(data, day_index, anchor_index)
9774         knot_minutes, knot_kw = forecast_knots_minutes_kw(issue_hour, hourly_kw, anchor_kw)
9775         today_minutes = np.array([period_end_minutes(t) for t in range(T)], dtype=float)
9776         next_minutes = today_minutes + 24 * 60
9777         today_kwh = interpolate_series(today_minutes, knot_minutes, knot_kw) * DELTA_H
9778         next_kwh = interpolate_series(next_minutes, knot_minutes, knot_kw) * DELTA_H
9779         last_knot = float(knot_minutes[-1])
9780         today_covered = today_minutes <= last_knot + 1e-9
9781         next_covered = next_minutes <= last_knot + 1e-9
9782         today_k = np.array([forecast_hour_k(m, issue_hour) for m in today_minutes], dtype=int)

```

```

9783     next_k = np.array([forecast_hour_k(m, issue_hour) for m in next_minutes], dtype=int)
9784     today_k = np.where((today_k >= 1) & (today_k <= 24), today_k, 0)
9785     next_k = np.where((next_k >= 1) & (next_k <= 24), next_k, 0)
9786 else:
9787     raise ValueError(f"unknown pv_mapping_mode: {pv_mapping_mode}")
9788
9789 # Periods already observed at the issue time are filled with actuals for the
9790 # mapping table, but they are never used as future interpolation anchors.
9791 if first_mutable > 0:
9792     today_kwh[:first_mutable] = data.pv[day_index, :first_mutable]
9793     today_k[:first_mutable] = 0
9794     today_covered[:first_mutable] = False
9795 today_kwh = np.maximum(today_kwh, 0.0)
9796 next_kwh = np.maximum(next_kwh, 0.0)
9797 return MappedForecast(
9798     today_kwh=today_kwh,
9799     next_day_kwh=next_kwh,
9800     knot_minutes=knot_minutes,
9801     knot_kw=knot_kw,
9802     issue_hour=issue_hour,
9803     first_mutable_index=first_mutable,
9804     pv_mapping_mode=pv_mapping_mode,
9805     today_source_k=today_k,
9806     next_source_k=next_k,
9807     today_covered=today_covered,
9808     next_covered=next_covered,
9809 )
9810
9811
9812 def causal_load_sources(
9813     data: Q3Data, target_index: int, history_end_exclusive: int
9814 ) -> tuple[np.ndarray, tuple[int, ...], str]:
9815     """Q2-style day-ahead load forecast: last <=4 same-weekday complete days."""
9816     if history_end_exclusive > target_index:
9817         raise ValueError("load forecast cutoff must not exceed the target date")
9818     if history_end_exclusive <= 0:
9819         return data.fallback_load.copy(), (), "attachment1_fallback"
9820     target_weekday = data.dates[target_index].weekday()
9821     sources = tuple(
9822         j
9823         for j in range(history_end_exclusive)
9824         if data.dates[j].weekday() == target_weekday
9825     )[-LOAD_HISTORY_SAME_WEEKDAY:]
9826     if not sources:
9827         return data.fallback_load.copy(), (), "attachment1_fallback"
9828     label = ";".join(data.dates[j].strftime("%Y-%m-%d") for j in sources)
9829     return data.load[list(sources)].mean(axis=0), sources, label
9830
9831
9832 def causal_load_forecast(
9833     data: Q3Data, target_index: int, history_end_exclusive: int
9834 ) -> np.ndarray:
9835     """Load forecast for target_index using only complete days before the cutoff."""
9836     hat, _sources, _label = causal_load_sources(data, target_index, history_end_exclusive)
9837     return hat
9838
9839
9840 def planning_load_curve(
9841     data: Q3Data, day_index: int, load_information_case: str
9842 ) -> np.ndarray:
9843     """Future-load vector used at 0:00 and later updates.
9844
9845     causal_load_main: day-ahead hat formed at 0:00; never reread today's actuals.
9846     actual_load_proxy: full Attachment 2 path, idealized information only.
9847     """
9848     if load_information_case == LOAD_INFORMATION_MAIN:
9849         return causal_load_forecast(data, day_index, day_index)
9850     if load_information_case == LOAD_INFORMATION_PROXY:

```

```

9851         return data.load[day_index].copy()
9852         raise ValueError(f"unknown load_information_case: {load_information_case}")
9853
9854
9855 def execution_load_horizon(
9856     plan_load: np.ndarray, actual_load: np.ndarray, t: int
9857 ) -> np.ndarray:
9858     """Current 10-minute step uses actual load; later steps keep the 0:00 hat."""
9859     horizon = np.asarray(plan_load[t:], dtype=float).copy()
9860     horizon[0] = float(np.asarray(actual_load, dtype=float)[t])
9861     return horizon
9862
9863
9864 def load_forecast_audit_rows(data: Q3Data, day_index: int) -> list[dict]:
9865     """Day-ahead causal load vs Attachment 2 actuals; future periods are not executed yet."""
9866     date = data.dates[day_index].strftime("%Y-%m-%d")
9867     hat, _sources, label = causal_load_sources(data, day_index, day_index)
9868     actual = data.load[day_index]
9869     rows: list[dict] = []
9870     for t in range(T):
9871         forecast = float(hat[t])
9872         observed = float(actual[t])
9873         rows.append(
9874             {
9875                 "date": date,
9876                 "period": t,
9877                 "time": data.time_labels[t],
9878                 "forecast_load_kwh": forecast,
9879                 "actual_load_kwh": observed,
9880                 "source_dates": label,
9881                 "is_current_execution_period": False,
9882                 "abs_error_kwh": abs(forecast - observed),
9883                 "load_information_case": LOAD_INFORMATION_MAIN,
9884                 "snapshot": "day_ahead_planning",
9885             }
9886         )
9887     return rows
9888
9889
9890 def causal_next_day_pv_base(
9891     data: Q3Data,
9892     day_index: int,
9893     pv_mapping_mode: str = PV_MAPPING_LINEAR,
9894 ) -> np.ndarray:
9895     """Causal stand-in for the next day's 0:00 PV forecast, in kWh."""
9896     start = max(0, day_index - NEXT_DAY_PV_HISTORY_DAYS)
9897     if start >= day_index:
9898         return data.fallback_pv.copy()
9899     mapped = np.vstack(
9900         [
9901             map_issue_forecast(data, j, 0, pv_mapping_mode).today_kwh
9902             for j in range(start, day_index)
9903         ]
9904     )
9905     return mapped.mean(axis=0)
9906
9907
9908 def next_day_pv_forecast(
9909     data: Q3Data,
9910     day_index: int,
9911     issue_hour: int,
9912     pv_mapping_mode: str = PV_MAPPING_LINEAR,
9913 ) -> np.ndarray:
9914     """Next-day PV used only in the terminal-value / 48h module."""
9915     if day_index + 1 >= len(data.dates):
9916         return np.zeros(T)
9917     mapped = map_issue_forecast(data, day_index, issue_hour, pv_mapping_mode)
9918     overlay = mapped.next_day_kwh.copy()

```

```

9919     base = causal_next_day_pv_base(data, day_index + 1, pv_mapping_mode)
9920     overlay[-mapped.next_covered] = base[-mapped.next_covered]
9921     return np.maximum(overlay, 0.0)
9922
9923
9924 def _source_interval_label(issue_hour: int, k: int) -> str:
9925     if k <= 0:
9926         return ""
9927     start = issue_hour + k - 1
9928     end = issue_hour + k
9929     def clock(h: int) -> str:
9930         day = ""
9931         if h >= 24:
9932             h -= 24
9933             day = "+1"
9934         return f"{h:02d}:00{day}"
9935     return f"({clock(start)},{clock(end)})"
9936
9937
9938 def mapping_rows_for_day(
9939     data: Q3Data,
9940     day_index: int,
9941     pv_mapping_mode: str = PV_MAPPING_LINEAR,
9942 ) -> list[dict]:
9943     date = data.dates[day_index].strftime("%Y-%m-%d")
9944     rows: list[dict] = []
9945     for hour in UPDATE_HOURS:
9946         mapped = map_issue_forecast(data, day_index, hour, pv_mapping_mode)
9947         for t in range(T):
9948             query = float(period_end_minutes(t))
9949             executed = t < mapped.first_mutable_index
9950             k = int(mapped.today_source_k[t])
9951             if pv_mapping_mode == PV_MAPPING_LINEAR and not executed:
9952                 weights = interpolate_weights(query, mapped.knot_minutes, mapped.knot_kw)
9953             elif executed:
9954                 weights = {
9955                     "left_node_minute": query,
9956                     "right_node_minute": query,
9957                     "left_weight": 1.0,
9958                     "right_weight": 0.0,
9959                     "left_kw": float(data.pv[day_index, t] / DELTA_H),
9960                     "right_kw": float(data.pv[day_index, t] / DELTA_H),
9961                 }
9962             else:
9963                 left_min = (hour + max(k, 1) - 1) * 60
9964                 right_min = (hour + max(k, 1)) * 60
9965                 weights = {
9966                     "left_node_minute": float(left_min),
9967                     "right_node_minute": float(right_min),
9968                     "left_weight": 1.0 if k else 0.0,
9969                     "right_weight": 0.0,
9970                     "left_kw": float(mapped.today_kwh[t] / DELTA_H) if k else 0.0,
9971                     "right_kw": float(mapped.today_kwh[t] / DELTA_H) if k else 0.0,
9972                 }
9973             forecast_kwh = (
9974                 float(data.pv[day_index, t]) if executed else float(mapped.today_kwh[t])
9975             )
9976             source = "actual_executed" if executed else (
9977                 "step_hourly" if pv_mapping_mode == PV_MAPPING_STEP else "causal_hourly_interpolation"
9978             )
9979             rows.append(
9980                 {
9981                     "date": date,
9982                     "update_time": f"{hour:02d}:00",
9983                     "period": t,
9984                     "time": data.time_labels[t],
9985                     "forecast_kwh": forecast_kwh,
9986                     "actual_pv_kwh": float(data.pv[day_index, t]),

```

```

9987         "pv_mapping_mode": pv_mapping_mode,
9988         "source_forecast_hour": k,
9989         "source_interval": "" if executed else _source_interval_label(hour, k),
9990         "left_node_minute": weights["left_node_minute"],
9991         "right_node_minute": weights["right_node_minute"],
9992         "left_weight": weights["left_weight"],
9993         "right_weight": weights["right_weight"],
9994         "is_executed_actual": bool(executed),
9995         "source": source,
9996     }
9997 )
9998 return rows
9999
10000
10001 def write_forecast_mapping(data: Q3Data, path, pv_mapping_mode: str = PV_MAPPING_LINEAR) -> pd.DataFrame:
10002     frames = []
10003     for date in PILOT_DATES:
10004         frames.extend(mapping_rows_for_day(data, data.date_index(date), pv_mapping_mode))
10005     frame = pd.DataFrame(frames)
10006     path.parent.mkdir(parents=True, exist_ok=True)
10007     frame.to_csv(path, index=False)
10008     return frame
10009
10010
10011 def write_forecast_mapping_modes(data: Q3Data, path, modes: tuple[str, ...] | None = None) -> pd.DataFrame:
10012     modes = modes or (PV_MAPPING_LINEAR, PV_MAPPING_STEP)
10013     rows: list[dict] = []
10014     for mode in modes:
10015         for date in PILOT_DATES:
10016             rows.extend(mapping_rows_for_day(data, data.date_index(date), mode))
10017     frame = pd.DataFrame(rows)
10018     path = Path(path)
10019     path.parent.mkdir(parents=True, exist_ok=True)
10020     frame.to_csv(path, index=False)
10021     return frame
10022
10023 # ===== q3/full_annual.py =====
10024 """Official 365-day continuous-SOC runs for year-end boundaries A and B.
10025
10026 Each boundary is an independent annual campaign: every strategy starts at
10027 6000 kWh on 2025-01-01 and carries its own SOC. Does not write result3.xlsx
10028 or overwrite q3_pilot / q3_sensitivity / q3_terminal_soc_pilot.
10029 """
10030
10031 from __future__ import annotations
10032
10033 import json
10034 from collections.abc import Iterable
10035 from time import perf_counter
10036
10037 import matplotlib.pyplot as plt
10038 import numpy as np
10039 import pandas as pd
10040
10041 from q3.config import (
10042     ANNUAL_END,
10043     ANNUAL_FIG_DIR,
10044     ANNUAL_OUTPUT_DIR,
10045     ANNUAL_START,
10046     E_INITIAL_KWH,
10047     E_MAX_KWH,
10048     E_MIN_KWH,
10049     LOAD_INFORMATION_MAIN,
10050     PV_MAPPING_LINEAR,
10051     SETTLEMENT_MAIN,
10052     SIMULTANEOUS_CD_TOL,
10053     STRATEGIES,
10054     YEAR_END_BOUNDARIES,

```



```

10055     YEAR_END_BOUNDARY_A,
10056     YEAR_END_BOUNDARY_B,
10057     YEAR_END_SOC_A_KWH,
10058     YEAR_END_SOC_B_KWH,
10059     YEAR_N_DAYS,
10060     dispatch_stem,
10061     make_run_id,
10062 )
10063 from q3.data import Q3Data, write_input_audit
10064 from q3.forecast import causal_load_sources
10065 from q3.pilot import (
10066     NAVY,
10067     ORANGE,
10068     PURPLE,
10069     TEAL,
10070     ValueCutCache,
10071     run_day,
10072 )
10073 from q3.terminal_soc_pilot import DISPATCH_COLS, boundary_delta
10074
10075 plt.rcParams.update(
10076     {
10077         "font.sans-serif": ["Noto Sans CJK SC", "DejaVu Sans"],
10078         "axes.unicode_minus": False,
10079         "figure.dpi": 140,
10080     }
10081 )
10082
10083 STRATEGY_COLORS = {
10084     "M0": NAVY,
10085     "M1_M6": TEAL,
10086     "M6_only": ORANGE,
10087     "M12_only": PURPLE,
10088     "M18_only": "#9c6644",
10089 }
10090
10091 DAILY_CSV = "q3_annual_daily_summary.csv"
10092 UPDATE_CSV = "q3_update_log.csv"
10093 LEDGER_RAW_NAME = "period_ledger.csv"
10094
10095
10096 def parse_boundaries(name: str | None) -> tuple[tuple[str, float], ...]:
10097     text = (name or "both").strip().lower()
10098     if text in {"both", "all", "ab", "a,b"}:
10099         return YEAR_END_BOUNDARIES
10100     if text in {"a", "a_q2_aligned", "1200"}:
10101         return ((YEAR_END_BOUNDARY_A, YEAR_END_SOC_A_KWH),)
10102     if text in {"b", "b_energy_neutral", "6000"}:
10103         return ((YEAR_END_BOUNDARY_B, YEAR_END_SOC_B_KWH),)
10104     raise ValueError(f"unknown year-end boundary {name!r}; use A, B, or both")
10105
10106
10107 def annual_dates(data: Q3Data) -> list[str]:
10108     dates = [ts.strftime("%Y-%m-%d") for ts in data.dates]
10109     if len(dates) != YEAR_N_DAYS:
10110         raise ValueError(f"expected {YEAR_N_DAYS} dates, got {len(dates)}")
10111     if dates[0] != ANNUAL_START or dates[-1] != ANNUAL_END:
10112         raise ValueError(f"expected {ANNUAL_START}..{ANNUAL_END}, got {dates[0]}..{dates[-1]}")
10113     return dates
10114
10115
10116 def write_annual_forecast_audit(data: Q3Data, path) -> pd.DataFrame:
10117     rows = []
10118     for i, ts in enumerate(data.dates):
10119         _hat, sources, label = causal_load_sources(data, i, i)
10120         max_source = int(max(sources)) if sources else -1
10121         rows.append(
10122             {

```

```

10123         "date": ts.strftime("%Y-%m-%d"),
10124         "day_index": i,
10125         "load_information_case": LOAD_INFORMATION_MAIN,
10126         "pv_mapping_mode": PV_MAPPING_LINEAR,
10127         "n_sources": len(sources),
10128         "source_dates": label,
10129         "used_attachment1_fallback": label == "attachment1_fallback",
10130         "history_end_exclusive_index": i,
10131         "max_source_index": max_source,
10132         "source_cutoff_ok": bool(max_source < i),
10133     }
10134 )
10135 frame = pd.DataFrame(rows)
10136 path.parent.mkdir(parents=True, exist_ok=True)
10137 frame.to_csv(path, index=False)
10138 return frame
10139
10140
10141 def _append_csv(path, frame: pd.DataFrame) -> None:
10142     path.parent.mkdir(parents=True, exist_ok=True)
10143     header = not path.exists() or path.stat().st_size == 0
10144     frame.to_csv(path, mode="a", header=header, index=False)
10145
10146
10147 def strategy_totals(daily: pd.DataFrame) -> pd.DataFrame:
10148     ordered = daily.sort_values(["year_end_boundary", "strategy", "date"])
10149     rows = []
10150     for (strategy, boundary), block in ordered.groupby(["strategy", "year_end_boundary"], sort=False):
10151         rows.append(
10152             {
10153                 "strategy": strategy,
10154                 "year_end_boundary": boundary,
10155                 "year_end_soc_kwh": float(block["year_end_soc_kwh"].iloc[0]),
10156                 "load_information_case": LOAD_INFORMATION_MAIN,
10157                 "pv_mapping_mode": PV_MAPPING_LINEAR,
10158                 "settlement_mode": SETTLEMENT_MAIN,
10159                 "with_terminal_value": True,
10160                 "n_days": int(block["date"].nunique()),
10161                 "total_cost_yuan": float(block["total_cost_yuan"].sum()),
10162                 "settlement_cost_yuan": float(block["settlement_cost_yuan"].sum()),
10163                 "emergency_cost_yuan": float(block["emergency_cost_yuan"].sum()),
10164                 "emergency_kwh": float(block["emergency_kwh"].sum()),
10165                 "curtailment_kwh": float(block["curtailment_kwh"].sum()),
10166                 "charge_kwh": float(block["charge_kwh"].sum()),
10167                 "discharge_kwh": float(block["discharge_kwh"].sum()),
10168                 "adjustment_count": int(block["adjustment_count"].sum()),
10169                 "up_adjust_kwh": float(block["up_adjust_kwh"].sum()),
10170                 "down_adjust_kwh": float(block["down_adjust_kwh"].sum()),
10171                 "soc_start_jan01_kwh": float(block["soc_start_kwh"].iloc[0]),
10172                 "soc_end_dec31_kwh": float(block["soc_end_kwh"].iloc[-1]),
10173                 "max_balance_residual_kwh": float(block["max_balance_residual_kwh"].max()),
10174                 "max_soc_residual_kwh": float(block["max_soc_residual_kwh"].max()),
10175                 "max_simultaneous_cd_kwh2": float(block["max_simultaneous_cd_kwh2"].max()),
10176                 "locked_period_violations": int(block["locked_period_violations"].sum()),
10177                 "runtime_seconds": float(block["runtime_seconds"].sum()),
10178             }
10179         )
10180     totals = pd.DataFrame(rows)
10181     totals["cost_rank_within_boundary"] = totals.groupby("year_end_boundary")["total_cost_yuan"].rank(
10182         method="min"
10183     )
10184     return totals
10185
10186
10187 def continuity_audit(daily: pd.DataFrame) -> dict:
10188     items = []
10189     all_pass = True
10190     ordered = daily.sort_values(["year_end_boundary", "strategy", "date"])

```

```

10191 for (strategy, boundary), block in ordered.groupby(["strategy", "year_end_boundary"], sort=False):
10192     starts = block["soc_start_kwh"].to_numpy()
10193     ends = block["soc_end_kwh"].to_numpy()
10194     dates = block["date"].to_numpy()
10195     max_gap = float(np.max(np.abs(starts[1:] - ends[:-1]))) if len(block) > 1 else 0.0
10196     start_ok = abs(float(starts[0]) - E_INITIAL_KWH) <= 1e-5
10197     end_target = float(block["year_end_soc_kwh"].iloc[0])
10198     end_ok = abs(float(ends[-1]) - end_target) <= 1e-5
10199     n_ok = (
10200         len(block) == YEAR_N_DAYS
10201         and str(dates[0]) == ANNUAL_START
10202         and str(dates[-1]) == ANNUAL_END
10203     )
10204     rec = {
10205         "strategy": strategy,
10206         "year_end_boundary": boundary,
10207         "n_days": int(len(block)),
10208         "start_soc_kwh": float(starts[0]),
10209         "end_soc_kwh": float(ends[-1]),
10210         "year_end_target_kwh": end_target,
10211         "max_cross_day_soc_gap_kwh": max_gap,
10212         "pass": bool(start_ok and end_ok and n_ok and max_gap <= 1e-5),
10213     }
10214     all_pass = all_pass and rec["pass"]
10215     items.append(rec)
10216 return {"all_pass": all_pass, "paths": items}
10217
10218
10219 def plot_annual_figures(daily: pd.DataFrame, totals: pd.DataFrame) -> None:
10220     fig_dir = ANNUAL_FIG_DIR
10221     fig_dir.mkdir(parents=True, exist_ok=True)
10222     present = [name for name, _target in YEAR_END_BOUNDARIES if name in set(totals["year_end_boundary"])]
10223     colors = {YEAR_END_BOUNDARY_A: NAVY, YEAR_END_BOUNDARY_B: TEAL}
10224     x = np.arange(len(STRATEGIES))
10225     width = 0.36 if len(present) > 1 else 0.55
10226     fig, ax = plt.subplots(figsize=(8.6, 4.6))
10227     bottoms = {name: np.zeros(len(STRATEGIES)) for name in present}
10228     for component, alpha, hatch in (
10229         ("settlement_cost_yuan", 1.0, None),
10230         ("emergency_cost_yuan", 0.55, "/"),
10231     ):
10232         for i, boundary in enumerate(present):
10233             subset = totals[totals["year_end_boundary"] == boundary].set_index("strategy").loc[list(STRATEGIES)]
10234             offset = (i - 0.5) * width if len(present) > 1 else 0.0
10235             ax.bar(
10236                 x + offset,
10237                 subset[component],
10238                 width=width,
10239                 bottom=bottoms[boundary],
10240                 color=colors[boundary],
10241                 alpha=alpha,
10242                 hatch=hatch,
10243                 label=f"{boundary} / {component.split('_')[0]}",
10244             )
10245             bottoms[boundary] = bottoms[boundary] + subset[component].to_numpy()
10246     ax.set_xticks(x)
10247     ax.set_xticklabels(STRATEGIES)
10248     ax.set_ylabel("annual purchase cost (yuan)")
10249     ax.set_title("Q3 annual main path: year-end A=1200 vs B=6000 (settlement + emergency)")
10250     ax.legend(frameon=False, fontsize=8, ncol=2)
10251     ax.spines["top"].set_visible(False)
10252     ax.spines["right"].set_visible(False)
10253     fig.tight_layout()
10254     fig.savefig(fig_dir / "fig3_annual_strategy_cost_decomposition.png")
10255     plt.close(fig)
10256
10257     ordered = daily.sort_values("date")
10258     dates = list(dict.fromkeys(ordered["date"]))

```

```

10259 x_dates = np.arange(len(dates))
10260 xtick = list(range(0, len(dates), 20))
10261 fig, axes = plt.subplots(len(present), 1, figsize=(10.2, 3.6 * len(present)), sharex=True)
10262 if len(present) == 1:
10263     axes = [axes]
10264 for ax, boundary in zip(axes, present):
10265     for strategy in STRATEGIES:
10266         block = daily[
10267             (daily["year_end_boundary"] == boundary) & (daily["strategy"] == strategy)
10268         ].sort_values("date")
10269         ax.plot(x_dates, block["soc_end_kwh"], color=STRATEGY_COLORS[strategy], lw=1.15, label=strategy)
10270 ax.axhline(E_MIN_KWH, color=ORANGE, ls="--", lw=0.8)
10271 ax.axhline(E_MAX_KWH, color=ORANGE, ls="--", lw=0.8)
10272 ax.axhline(E_INITIAL_KWH, color="#888888", ls=":", lw=0.8)
10273 ax.set_ylabel("end-of-day SOC (kWh)")
10274 ax.set_title(boundary)
10275 ax.legend(frameon=False, ncol=5, fontsize=8)
10276 ax.spines["top"].set_visible(False)
10277 ax.spines["right"].set_visible(False)
10278 ax.set_xticks(xtick)
10279 ax.set_xticklabels([dates[i][5:] for i in xtick], rotation=60)
10280 axes[-1].set_xlabel("date")
10281 fig.suptitle("Annual continuous SOC paths (main mapping/settlement)", fontsize=12)
10282 fig.tight_layout()
10283 fig.savefig(fig_dir / "fig3_annual_soc_paths.png")
10284 plt.close(fig)
10285
10286 month = pd.to_datetime(daily["date"]).dt.to_period("M").astype(str)
10287 work = daily.copy()
10288 work["month"] = month
10289 monthly = (
10290     work.groupby(["month", "strategy", "year_end_boundary"], as_index=False)[
10291         ["emergency_cost_yuan", "curtailment_kwh"]
10292     ].sum()
10293 )
10294 months = list(dict.fromkeys(monthly["month"]))
10295 xm = np.arange(len(months))
10296 fig, axes = plt.subplots(2, 1, figsize=(10.2, 7.0), sharex=True)
10297 for ax, (column, ylabel) in zip(
10298     axes,
10299     (("emergency_cost_yuan", "emergency cost (yuan)"), ("curtailment_kwh", "curtailment (kWh)")),
10300 ):
10301     for strategy in STRATEGIES:
10302         styles = {YEAR_END_BOUNDARY_A: "-", YEAR_END_BOUNDARY_B: "--"}
10303         for boundary in present:
10304             block = monthly[
10305                 (monthly["year_end_boundary"] == boundary) & (monthly["strategy"] == strategy)
10306             ].set_index("month").reindex(months)
10307             ax.plot(
10308                 xm,
10309                 block[column],
10310                 color=STRATEGY_COLORS[strategy],
10311                 ls=styles.get(boundary, "-"),
10312                 lw=1.2,
10313                 label=f"{strategy} / {boundary.split('_')[0]}",
10314             )
10315             ax.set_ylabel(ylabel)
10316             ax.spines["top"].set_visible(False)
10317             ax.spines["right"].set_visible(False)
10318             ax.set_xticks(xm)
10319             ax.set_xticklabels(months, rotation=45)
10320 axes[0].legend(frameon=False, ncol=2, fontsize=7)
10321 fig.suptitle("Monthly emergency cost and curtailment (main path; dashed = B)", fontsize=12)
10322 fig.tight_layout()
10323 fig.savefig(fig_dir / "fig3_annual_monthly_emergency_curtailment.png")
10324 plt.close(fig)
10325 monthly.to_csv(ANNUAL_OUTPUT_DIR / "q3_annual_monthly.csv", index=False)
10326

```

```

10327
10328 def _physical_from_daily(daily: pd.DataFrame) -> dict:
10329     recs = []
10330     all_pass = True
10331     for _, row in daily.iterrows():
10332         ok = bool(
10333             float(row["max_balance_residual_kwh"]) < 1e-5
10334             and float(row["max_soc_residual_kwh"]) < 1e-5
10335             and float(row["max_simultaneous_cd_kwh2"]) <= SIMULTANEOUS_CD_TOL
10336             and int(row["locked_period_violations"]) == 0
10337         )
10338         all_pass = all_pass and ok
10339         if not ok:
10340             recs.append(
10341                 {
10342                     "run_id": row["run_id"],
10343                     "date": row["date"],
10344                     "strategy": row["strategy"],
10345                     "year_end_boundary": row["year_end_boundary"],
10346                     "max_balance_residual_kwh": float(row["max_balance_residual_kwh"]),
10347                     "max_soc_residual_kwh": float(row["max_soc_residual_kwh"]),
10348                     "max_simultaneous_cd_kwh2": float(row["max_simultaneous_cd_kwh2"]),
10349                     "locked_period_violations": int(row["locked_period_violations"]),
10350                     "pass": False,
10351                 }
10352             )
10353     return {"all_pass": all_pass, "n_failed_days": len(recs), "failures": recs}
10354
10355
10356 def run_q3_full_annual(
10357     data: Q3Data,
10358     boundaries: Iterable[tuple[str, float]] | None = None,
10359 ) -> dict:
10360     out = ANNUAL_OUTPUT_DIR
10361     out.mkdir(parents=True, exist_ok=True)
10362     dispatch_dir = out / "dispatch_daily"
10363     dispatch_dir.mkdir(parents=True, exist_ok=True)
10364     ledger_raw = out / "ledger_raw"
10365     ledger_raw.mkdir(parents=True, exist_ok=True)
10366     ANNUAL_FIG_DIR.mkdir(parents=True, exist_ok=True)
10367     write_input_audit(data, out / "input_audit.json")
10368     dates = annual_dates(data)
10369     write_annual_forecast_audit(data, out / "q3_annual_forecast_audit.csv")
10370
10371     selected = tuple(boundaries) if boundaries is not None else YEAR_END_BOUNDARIES
10372     daily_path = out / DAILY_CSV
10373     update_path = out / UPDATE_CSV
10374     existing = pd.read_csv(daily_path) if daily_path.exists() else pd.DataFrame()
10375     completed = set(existing["run_id"].astype(str)) if len(existing) else set()
10376     cache: ValueCutCache = {}
10377     started = perf_counter()
10378     n_expected = len(selected) * len(STRATEGIES) * len(dates)
10379     done = 0
10380
10381     for boundary, target in selected:
10382         print(f"==== year-end {boundary} target={target:.0f} kWh =====", flush=True)
10383         ledger_path = ledger_raw / f"{boundary}_{LEDGER_RAW_NAME}"
10384         for strategy in STRATEGIES:
10385             soc = E_INITIAL_KWH
10386             for date in dates:
10387                 run_id = make_run_id(
10388                     date,
10389                     strategy,
10390                     LOAD_INFORMATION_MAIN,
10391                     PV_MAPPING_LINEAR,
10392                     SETTLEMENT_MAIN,
10393                     True,
10394                     target,

```

```

10395         )
10396         done += 1
10397         if run_id in completed:
10398             row = existing[existing["run_id"] == run_id].iloc[-1]
10399             soc = float(row["soc_end_kwh"])
10400             print(f"[{done}/{n_expected}] skip {run_id}", flush=True)
10401             continue
10402         day_index = data.date_index(date)
10403         run = run_day(
10404             data,
10405             day_index,
10406             strategy,
10407             soc,
10408             with_terminal_value=True,
10409             load_information_case=LOAD_INFORMATION_MAIN,
10410             pv_mapping_mode=PV_MAPPING_LINEAR,
10411             settlement_mode=SETTLEMENT_MAIN,
10412             value_cut_cache=cache,
10413             year_end_soc_kwh=target,
10414         )
10415         soc = float(run.summary["soc_end_kwh"])
10416         stem = dispatch_stem(
10417             run.date,
10418             run.strategy,
10419             run.load_information_case,
10420             run.pv_mapping_mode,
10421             run.settlement_mode,
10422             run.with_terminal_value,
10423             run.year_end_soc_kwh,
10424         )
10425         run.dispatch[DISPATCH_COLS].to_csv(dispatch_dir / f"{stem}.csv", index=False)
10426         _append_csv(daily_path, pd.DataFrame([run.summary]))
10427         _append_csv(update_path, run.update_log)
10428         period = run.ledger[run.ledger["row_role"] == "period_total"]
10429         _append_csv(ledger_path, period)
10430         completed.add(run_id)
10431         print(
10432             f"[{done}/{n_expected}] {run.run_id} "
10433             f"cost={run.summary['total_cost_yuan']:.2f} "
10434             f"soc_end={soc:.4f}",
10435             flush=True,
10436         )
10437
10438     daily = pd.read_csv(daily_path)
10439     daily = daily.drop_duplicates("run_id", keep="last")
10440     daily = daily.sort_values(["year_end_boundary", "strategy", "date"])
10441     daily.to_csv(daily_path, index=False)
10442     totals = strategy_totals(daily)
10443     totals.to_csv(out / "q3_annual_strategy_comparison.csv", index=False)
10444     if set(daily["year_end_boundary"]) >= {YEAR_END_BOUNDARY_A, YEAR_END_BOUNDARY_B}:
10445         delta = boundary_delta(totals)
10446         delta.to_csv(out / "q3_annual_boundary_delta.csv", index=False)
10447     else:
10448         delta = pd.DataFrame()
10449     value_rows = []
10450     for key in sorted(cache):
10451         value_rows.extend(cache[key][1])
10452     if value_rows:
10453         pd.DataFrame(value_rows).to_csv(out / "q3_next_day_value_audit.csv", index=False)
10454
10455     plot_annual_figures(daily, totals)
10456     phys = _physical_from_daily(daily)
10457     cont = continuity_audit(daily)
10458     audit = {
10459         "all_pass": bool(phys.get("all_pass") and cont.get("all_pass")),
10460         "physical": phys,
10461         "continuity": cont,
10462         "n_day_runs": int(len(daily)),

```

```

10463         "n_paths": int(daily.groupby(["strategy", "year_end_boundary"]).ngroups),
10464         "start_date": ANNUAL_START,
10465         "end_date": ANNUAL_END,
10466         "jan01_soc_kwh": E_INITIAL_KWH,
10467         "pv_mapping_mode": PV_MAPPING_LINEAR,
10468         "settlement_mode": SETTLEMENT_MAIN,
10469         "load_information_case": LOAD_INFORMATION_MAIN,
10470         "boundaries_run": [name for name, _target in selected],
10471         "note": (
10472             "A and B are independent 365-day campaigns. "
10473             "This archive does not choose a single official year-end boundary "
10474             "and does not write result3.xlsx. Mapping/settlement annual "
10475             "sensitivities are not included in this run."
10476         ),
10477         "elapsed_seconds_this_process": float(perf_counter() - started),
10478     }
10479     (out / "q3_annual_physical_audit.json").write_text(
10480         json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
10481     )
10482     return {
10483         "n_runs": int(len(daily)),
10484         "daily": daily,
10485         "totals": totals,
10486         "delta": delta,
10487         "audit": audit,
10488         "output_dir": str(out),
10489     }
10490
10491 # ===== q3/optimization.py =====
10492 from __future__ import annotations
10493
10494 from dataclasses import dataclass
10495 from time import perf_counter
10496
10497 import cvxpy as cp
10498 import numpy as np
10499
10500 from q3.config import (
10501     ADJUST_ABS_COEFF,
10502     EMERGENCY_PRICE_MULTIPLIER,
10503     E_MAX_KWH,
10504     E_MIN_KWH,
10505     ETA_C,
10506     ETA_D,
10507     MPC_COST_TOL,
10508     POWER_LIMIT_KWH,
10509     SETTLEMENT_ALT,
10510     SETTLEMENT_MAIN,
10511     SIMULTANEOUS_CD_TOL,
10512     SOLVER,
10513 )
10514
10515 ValueCut = tuple[float, float, float] # reference SOC, value, subgradient
10516
10517
10518 def settlement_cost(price: np.ndarray, g0: np.ndarray, g_final: np.ndarray) -> np.ndarray:
10519     """Main  $\phi_t = p g^F + 0.5 p |g^F - g^0|$ . Vectorised and used in tests and audits."""
10520     price = np.asarray(price, dtype=float).ravel()
10521     g0 = np.asarray(g0, dtype=float).ravel()
10522     g_final = np.asarray(g_final, dtype=float).ravel()
10523     return price * g_final + ADJUST_ABS_COEFF * price * np.abs(g_final - g0)
10524
10525
10526 def adjacent_adjustment_cost(price: np.ndarray, g_prev: np.ndarray, g_new: np.ndarray) -> np.ndarray:
10527     """1.5 p (up)_+ + 0.5 p (down)_+ between two adjacent commitment versions."""
10528     price = np.asarray(price, dtype=float).ravel()
10529     g_prev = np.asarray(g_prev, dtype=float).ravel()

```

```

10531     g_new = np.asarray(g_new, dtype=float).ravel()
10532     up = np.maximum(g_new - g_prev, 0.0)
10533     down = np.maximum(g_prev - g_new, 0.0)
10534     return 1.5 * price * up + 0.5 * price * down
10535
10536
10537 def realized_settlement(
10538     price: np.ndarray,
10539     versions: list[np.ndarray] | tuple[np.ndarray, ...],
10540     settlement_mode: str,
10541 ) -> np.ndarray:
10542     """Period-wise ordinary+adjustment bill for a full version path (no emergency)."""
10543     price = np.asarray(price, dtype=float).ravel()
10544     path = [np.asarray(v, dtype=float).ravel() for v in versions]
10545     if not path:
10546         raise ValueError("need at least the 0:00 plan")
10547     if settlement_mode == SETTLEMENT_MAIN:
10548         return settlement_cost(price, path[0], path[-1])
10549     if settlement_mode == SETTLEMENT_ALT:
10550         phi = price * path[0]
10551         for prev, cur in zip(path[:-1], path[1:]):
10552             phi = phi + adjacent_adjustment_cost(price, prev, cur)
10553         return phi
10554     raise ValueError(f"unknown settlement_mode: {settlement_mode}")
10555
10556
10557 def decompose_settlement(
10558     price: np.ndarray,
10559     versions: list[np.ndarray] | tuple[np.ndarray, ...],
10560     settlement_mode: str,
10561 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
10562     """Return ordinary, up-fee, down-fee arrays that sum to realized_settlement."""
10563     price = np.asarray(price, dtype=float).ravel()
10564     path = [np.asarray(v, dtype=float).ravel() for v in versions]
10565     if settlement_mode == SETTLEMENT_MAIN:
10566         g0, gf = path[0], path[-1]
10567         up = np.maximum(gf - g0, 0.0)
10568         down = np.maximum(g0 - gf, 0.0)
10569         return price * gf, 0.5 * price * up, 0.5 * price * down
10570     if settlement_mode == SETTLEMENT_ALT:
10571         ordinary = price * path[0]
10572         up_fee = np.zeros_like(price)
10573         down_fee = np.zeros_like(price)
10574         for prev, cur in zip(path[:-1], path[1:]):
10575             up = np.maximum(cur - prev, 0.0)
10576             down = np.maximum(prev - cur, 0.0)
10577             up_fee = up_fee + 1.5 * price * up
10578             down_fee = down_fee + 0.5 * price * down
10579         return ordinary, up_fee, down_fee
10580     raise ValueError(f"unknown settlement_mode: {settlement_mode}")
10581
10582
10583 def evaluate_value_cuts(soc: float, cuts: tuple[ValueCut, ...] | None) -> float:
10584     if not cuts:
10585         return 0.0
10586     return max(
10587         0.0,
10588         max(value + slope * (soc - reference) for reference, value, slope in cuts),
10589     )
10590
10591
10592 def _terminal_value_expression(
10593     terminal_soc: cp.Expression, cuts: tuple[ValueCut, ...] | None
10594 ) -> tuple[cp.Variable | None, list[cp.Constraint]]:
10595     if not cuts:
10596         return None, []
10597     value = cp.Variable(nonneg=True)
10598     constraints = [

```



```

10599         value >= cut_value + slope * (terminal_soc - reference_soc)
10600         for reference_soc, cut_value, slope in cuts
10601     ]
10602     return value, constraints
10603
10604
10605 def _physical_constraints(
10606     load: np.ndarray,
10607     pv: np.ndarray,
10608     initial_soc: float,
10609     normal: cp.Variable,
10610     emergency: cp.Variable,
10611     charge: cp.Variable,
10612     discharge: cp.Variable,
10613     curtailment: cp.Variable,
10614     soc: cp.Variable,
10615 ) -> list[cp.Constraint]:
10616     return [
10617         soc[0] == initial_soc,
10618         soc[1:] == soc[:-1] + ETA_C * charge - discharge / ETA_D,
10619         soc[1:] >= E_MIN_KWH,
10620         soc[1:] <= E_MAX_KWH,
10621         charge <= POWER_LIMIT_KWH,
10622         discharge <= POWER_LIMIT_KWH,
10623         curtailment <= pv,
10624         normal + emergency + pv - curtailment + discharge == load + charge,
10625     ]
10626
10627
10628 @dataclass(frozen=True)
10629 class HorizonResult:
10630     g: np.ndarray
10631     x: np.ndarray
10632     emergency: np.ndarray
10633     charge: np.ndarray
10634     discharge: np.ndarray
10635     curtailment: np.ndarray
10636     soc: np.ndarray
10637     objective: float
10638     settlement_cost: float
10639     emergency_cost: float
10640     terminal_value: float
10641     initial_soc_marginal: float
10642     solve_seconds: float
10643     status: str
10644
10645     @property
10646     def max_cd(self) -> float:
10647         return float(np.max(self.charge * self.discharge))
10648
10649
10650 def solve_horizon(
10651     price: np.ndarray,
10652     load: np.ndarray,
10653     pv: np.ndarray,
10654     initial_soc: float,
10655     g0: np.ndarray | None = None,
10656     g_fixed: np.ndarray | None = None,
10657     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
10658     throughput_tiebreak: bool = True,
10659     bill_as_day_ahead: bool = False,
10660     settlement_mode: str = SETTLEMENT_MAIN,
10661     g_pre: np.ndarray | None = None,
10662     sunk_settlement: float = 0.0,
10663     terminal_soc: float | None = None,
10664 ) -> HorizonResult:
10665     """Solve a remaining-horizon LP.
10666

```

```

10667     If bill_as_day_ahead is True, ordinary energy is billed as p@g with no
10668     adjustment (used for 0:00 and the virtual next day). Otherwise:
10669     - anchor_final_main:  $\phi(g_0, g) = p \cdot g + 0.5 \cdot p \cdot |g - g_0|$ 
10670     - adjacent_literal_sensitivity: sunk  $p \cdot g_0$  + prior adjacent fees, plus
10671        $1.5 \cdot p \cdot (g - g_{pre})_+ + 0.5 \cdot p \cdot (g_{pre} - g)_+$  for this update only.
10672
10673     If terminal_soc is set, the last SOC node of this remaining horizon
10674     (end of the current calendar day) is constrained to that value.
10675     """
10676     n = len(load)
10677     price = np.asarray(price, dtype=float).ravel()
10678     load = np.asarray(load, dtype=float).ravel()
10679     pv = np.asarray(pv, dtype=float).ravel()
10680     g = cp.Variable(n, nonneg=True)
10681     x, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
10682     E = cp.Variable(n + 1)
10683     constraints = _physical_constraints(load, pv, initial_soc, x, e, c, d, w, E)
10684     constraints.append(x <= g)
10685     if terminal_soc is not None:
10686         constraints.append(E[-1] == float(terminal_soc))
10687
10688     if g_fixed is not None:
10689         fixed = np.asarray(g_fixed, dtype=float).ravel()
10690         if fixed.shape != (n,):
10691             raise ValueError("g_fixed must match the remaining horizon")
10692         constraints.append(g == fixed)
10693
10694     emergency_cost = EMERGENCY_PRICE_MULTIPLIER * price @ e
10695     terminal_value, terminal_constraints = _terminal_value_expression(E[-1], terminal_value_cuts)
10696     constraints += terminal_constraints
10697     terminal_term: cp.Expression | float = terminal_value if terminal_value is not None else 0.0
10698
10699     if bill_as_day_ahead:
10700         settlement = price @ g
10701     elif settlement_mode == SETTLEMENT_MAIN:
10702         if g0 is None:
10703             raise ValueError("g0 is required unless bill_as_day_ahead=True")
10704         g0 = np.asarray(g0, dtype=float).ravel()
10705         if g0.shape != (n,):
10706             raise ValueError("g0 must match the remaining horizon")
10707         up = cp.Variable(n, nonneg=True)
10708         down = cp.Variable(n, nonneg=True)
10709         constraints.append(g - g0 == up - down)
10710         settlement = price @ g + ADJUST_ABS_COEFF * price @ (up + down)
10711     elif settlement_mode == SETTLEMENT_ALT:
10712         if g0 is None or g_pre is None:
10713             raise ValueError("adjacent settlement requires g0 and g_pre")
10714         g0 = np.asarray(g0, dtype=float).ravel()
10715         g_pre_arr = np.asarray(g_pre, dtype=float).ravel()
10716         if g0.shape != (n,) or g_pre_arr.shape != (n,):
10717             raise ValueError("g0 and g_pre must match the remaining horizon")
10718         up = cp.Variable(n, nonneg=True)
10719         down = cp.Variable(n, nonneg=True)
10720         constraints.append(g - g_pre_arr == up - down)
10721         settlement = float(sunk_settlement) + 1.5 * price @ up + 0.5 * price @ down
10722     else:
10723         raise ValueError(f"unknown settlement_mode: {settlement_mode}")
10724
10725     primary = settlement + emergency_cost + terminal_term
10726     problem = cp.Problem(cp.Minimize(primary), constraints)
10727     started = perf_counter()
10728     problem.solve(solver=SOLVER, verbose=False)
10729     if g.value is None:
10730         extra = ""
10731         if terminal_soc is not None:
10732             extra = (
10733                 f" (terminal_soc={float(terminal_soc):.6f} kWh, "
10734                 f"n={n}, initial_soc={float(initial_soc):.6f} kWh)"

```

```

10735         )
10736         raise RuntimeError(f"Q3 horizon LP failed: {problem.status}{extra}")
10737     status = str(problem.status)
10738     primary_opt = float(primary.value)
10739     if throughput_tiebreak:
10740         second = cp.Problem(
10741             cp.Minimize(cp.sum(c + d)),
10742             constraints + [primary <= primary_opt + MPC_COST_TOL],
10743         )
10744         second.solve(solver=SOLVER, verbose=False)
10745         if g.value is None:
10746             raise RuntimeError(f"Q3 throughput tiebreak failed: {second.status}")
10747         status = str(second.status)
10748     elapsed = perf_counter() - started
10749
10750     gv = np.asarray(g.value).ravel()
10751     xv = np.asarray(x.value).ravel()
10752     ev = np.asarray(e.value).ravel()
10753     cv = np.asarray(c.value).ravel()
10754     dv = np.asarray(d.value).ravel()
10755     wv = np.asarray(w.value).ravel()
10756     Ev = np.asarray(E.value).ravel()
10757     if bill_as_day_ahead:
10758         settle = float(price @ gv)
10759     elif settlement_mode == SETTLEMENT_MAIN:
10760         settle = float(np.sum(settlement_cost(price, g0, gv)))
10761     else:
10762         settle = float(sunk_settlement) + float(np.sum(adjacent_adjustment_cost(price, g_pre_arr, gv)))
10763     terminal = evaluate_value_cuts(float(Ev[-1]), terminal_value_cuts)
10764     dual = float(np.asarray(constraints[0].dual_value))
10765     return HorizonResult(
10766         g=gv,
10767         x=xv,
10768         emergency=ev,
10769         charge=cv,
10770         discharge=dv,
10771         curtailment=wv,
10772         soc=Ev,
10773         objective=float(primary_opt),
10774         settlement_cost=settle,
10775         emergency_cost=float(EMERGENCY_PRICE_MULTIPLIER * price @ ev),
10776         terminal_value=terminal,
10777         initial_soc_marginal=-dual,
10778         solve_seconds=elapsed,
10779         status=status,
10780     )
10781
10782
10783 def dispatch_balance_residual(load: np.ndarray, pv: np.ndarray, result: HorizonResult) -> np.ndarray:
10784     return (
10785         result.x
10786         + result.emergency
10787         + pv
10788         - result.curtailment
10789         + result.discharge
10790         - load
10791         - result.charge
10792     )
10793
10794
10795 def assert_physical(result: HorizonResult, load: np.ndarray, pv: np.ndarray, prefix: str = "") -> None:
10796     residual = dispatch_balance_residual(load, pv, result)
10797     soc_residual = result.soc[1:] - (
10798         result.soc[:-1] + ETA_C * result.charge - result.discharge / ETA_D
10799     )
10800     if np.max(np.abs(residual)) > 1e-5:
10801         raise AssertionError(f"{prefix}energy balance residual {np.max(np.abs(residual)):.3e}")
10802     if np.max(np.abs(soc_residual)) > 1e-5:

```

```

10803         raise AssertionError(f"{prefix}SOC residual {np.max(np.abs(soc_residual)):.3e}")
10804     if result.soc.min() < E_MIN_KWH - 1e-6 or result.soc.max() > E_MAX_KWH + 1e-6:
10805         raise AssertionError(f"{prefix}SOC bound violated")
10806     if np.any(result.charge < -1e-8) or np.any(result.discharge < -1e-8):
10807         raise AssertionError(f"{prefix}negative charge/discharge")
10808     if np.any(result.curtailment < -1e-8) or np.any(result.curtailment - pv > 1e-6):
10809         raise AssertionError(f"{prefix}curtailment bound violated")
10810     if np.any(result.x - result.g > 1e-6):
10811         raise AssertionError(f"{prefix}x exceeds commitment g")
10812     if result.max_cd > SIMULTANEOUS_CD_TOL:
10813         raise AssertionError(
10814             f"{prefix}simultaneous charge/discharge product {result.max_cd:.3e} "
10815             "exceeds tolerance; stop rather than ignore"
10816         )
10817
10818     # ===== q3/pilot.py =====
10819     from __future__ import annotations
10820
10821     import json
10822     from dataclasses import dataclass, field
10823     from math import inf
10824     from pathlib import Path
10825     from time import perf_counter
10826
10827     import matplotlib.pyplot as plt
10828     import numpy as np
10829     import pandas as pd
10830
10831     from q3.config import (
10832         E_INITIAL_KWH,
10833         E_MAX_KWH,
10834         E_MIN_KWH,
10835         EMERGENCY_PRICE_MULTIPLIER,
10836         ETA_C,
10837         ETA_D,
10838         FIG_DIR,
10839         HOUR_TO_FIRST_MUTABLE,
10840         LOAD_INFORMATION_CASES,
10841         LOAD_INFORMATION_MAIN,
10842         LOAD_INFORMATION_PROXY,
10843         NEXT_DAY_VALUE_GAP_TOL_YUAN,
10844         NEXT_DAY_VALUE_MAX_SAMPLES,
10845         NUMERIC_TOL,
10846         OUTPUT_DIR,
10847         PILOT_DATES,
10848         PV_MAPPING_LINEAR,
10849         SETTLEMENT_ALT,
10850         SETTLEMENT_MAIN,
10851         SIMULTANEOUS_CD_TOL,
10852         STRATEGY_ALLOWED_UPDATES,
10853         STRATEGIES,
10854         T,
10855         VOI_EPS_YUAN,
10856         WARMUP_CSV_MAIN,
10857         make_run_id,
10858         next_day_year_end_soc,
10859         today_year_end_soc,
10860         year_end_boundary_label,
10861     )
10862     from q3.data import Q3Data
10863     from q3.forecast import (
10864         causal_load_forecast,
10865         causal_load_sources,
10866         execution_load_horizon,
10867         load_forecast_audit_rows,
10868         map_issue_forecast,
10869         next_day_pv_forecast,
10870         planning_load_curve,

```

```

10871     write_forecast_mapping,
10872 )
10873 from q3.optimization import (
10874     ValueCut,
10875     assert_physical,
10876     decompose_settlement,
10877     evaluate_value_cuts,
10878     realized_settlement,
10879     settlement_cost,
10880     solve_horizon,
10881 )
10882
10883 plt.rcParams.update(
10884     {
10885         "font.sans-serif": ["Noto Sans CJK SC", "DejaVu Sans"],
10886         "axes.unicode_minus": False,
10887         "figure.dpi": 140,
10888     }
10889 )
10890
10891 NAVY = "#1f4e79"
10892 TEAL = "#2a9d8f"
10893 ORANGE = "#e09f3e"
10894 PURPLE = "#6d597a"
10895
10896
10897 ValueCutCache = dict[tuple[int, int, str, str], tuple[tuple[ValueCut, ...], list[dict]]]
10898
10899
10900 @dataclass
10901 class DayRun:
10902     date: str
10903     strategy: str
10904     load_information_case: str
10905     pv_mapping_mode: str
10906     settlement_mode: str
10907     with_terminal_value: bool
10908     run_id: str
10909     dispatch: pd.DataFrame
10910     update_log: pd.DataFrame
10911     ledger: pd.DataFrame
10912     summary: dict
10913     next_day_value_rows: list[dict] = field(default_factory=list)
10914     year_end_soc_kwh: float | None = None
10915
10916
10917 def dispatch_output_stem(date: str, strategy: str, load_information_case: str, with_terminal_value: bool) -> str:
10918     suffix = strategy + "_" + load_information_case
10919     if not with_terminal_value:
10920         suffix += "_no48h"
10921     return f"q3_dispatch_{date}_{suffix}"
10922
10923
10924 def build_next_day_value_cuts(
10925     data: Q3Data,
10926     current_index: int,
10927     issue_hour: int,
10928     pv_mapping_mode: str = PV_MAPPING_LINEAR,
10929     next_day_terminal_soc: float | None = None,
10930 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
10931     if current_index + 1 >= len(data.dates):
10932         return (), []
10933     load = causal_load_forecast(
10934         data, target_index=current_index + 1, history_end_exclusive=current_index
10935     )
10936     pv = next_day_pv_forecast(data, current_index, issue_hour, pv_mapping_mode)
10937     solved: dict[float, ValueCut] = {}
10938

```

```

10939 def solve_at(soc: float) -> None:
10940     result = solve_horizon(
10941         data.price,
10942         load,
10943         pv,
10944         float(soc),
10945         bill_as_day_ahead=True,
10946         throughput_tiebreak=False,
10947         terminal_soc=next_day_terminal_soc,
10948     )
10949     value = result.settlement_cost + result.emergency_cost
10950     solved[float(soc)] = (float(soc), float(value), float(result.initial_soc_marginal))
10951
10952 def certified_max_gap(cuts: tuple[ValueCut, ...]) -> tuple[float, float]:
10953     points = sorted(solved)
10954     best_gap, best_soc = -np.inf, points[0]
10955     lower_lines = [(0.0, 0.0)] + [
10956         (slope, value - slope * reference) for reference, value, slope in cuts
10957     ]
10958     for left, right in zip(points[:-1], points[1:]):
10959         left_value, right_value = solved[left][1], solved[right][1]
10960         upper_slope = (right_value - left_value) / (right - left)
10961         upper_intercept = left_value - upper_slope * left
10962         candidates = [left, right]
10963         for a, (slope_a, intercept_a) in enumerate(lower_lines):
10964             for slope_b, intercept_b in lower_lines[a + 1 :]:
10965                 if abs(slope_a - slope_b) <= 1e-14:
10966                     continue
10967                 cross = (intercept_b - intercept_a) / (slope_a - slope_b)
10968                 if left < cross < right:
10969                     candidates.append(float(cross))
10970         for soc in candidates:
10971             upper = upper_slope * soc + upper_intercept
10972             gap = upper - evaluate_value_cuts(soc, cuts)
10973             if gap > best_gap:
10974                 best_gap, best_soc = float(gap), float(soc)
10975     return max(best_gap, 0.0), best_soc
10976
10977 solve_at(E_MIN_KWH)
10978 solve_at(E_MAX_KWH)
10979 gap = inf
10980 while True:
10981     cuts = tuple(solved[soc] for soc in sorted(solved))
10982     slopes = np.array([cut[2] for cut in cuts])
10983     if np.any(np.diff(slopes) < -1e-6):
10984         raise AssertionError("Q3 next-day value subgradients are not monotone")
10985     gap, refine_soc = certified_max_gap(cuts)
10986     if gap <= NEXT_DAY_VALUE_GAP_TOL_YUAN:
10987         break
10988     if len(cuts) >= NEXT_DAY_VALUE_MAX_SAMPLES:
10989         raise AssertionError(
10990             f"Q3 next-day value gap {gap:.6f} exceeds tolerance after {len(cuts)} samples"
10991         )
10992     if min(abs(refine_soc - soc) for soc in solved) < 1e-7:
10993         raise AssertionError("Q3 next-day value refinement stalled")
10994     solve_at(refine_soc)
10995
10996 target = data.dates[current_index + 1].strftime("%Y-%m-%d")
10997 rows = [
10998     {
10999         "current_date": data.dates[current_index].strftime("%Y-%m-%d"),
11000         "target_date": target,
11001         "issue_hour": issue_hour,
11002         "pv_mapping_mode": pv_mapping_mode,
11003         "next_day_terminal_soc_kwh": (
11004             None if next_day_terminal_soc is None else float(next_day_terminal_soc)
11005         ),
11006         "soc_sample_kwh": reference,

```

```

11007         "virtual_next_day_cost_yuan": value,
11008         "value_subgradient_yuan_per_kwh": slope,
11009         "certified_max_gap_yuan": gap,
11010         "bundle_sample_count": len(cuts),
11011         "information": (
11012             f"Q3 {pv_mapping_mode} overlay + causal prior-day load/PV; no future actual PV"
11013         ),
11014     }
11015     for reference, value, slope in cuts
11016 ]
11017 return cuts, rows
11018
11019
11020 def _index_to_update_hour(index: int) -> int | None:
11021     inverse = {first: hour for hour, first in HOUR_TO_FIRST_MUTABLE.items() if hour != 0}
11022     return inverse.get(index)
11023
11024
11025 def _run_key_fields(
11026     date: str,
11027     strategy: str,
11028     load_information_case: str,
11029     pv_mapping_mode: str,
11030     settlement_mode: str,
11031     with_terminal_value: bool,
11032     run_id: str,
11033     year_end_soc_kwh: float | None = None,
11034 ) -> dict:
11035     return {
11036         "date": date,
11037         "strategy": strategy,
11038         "load_information_case": load_information_case,
11039         "pv_mapping_mode": pv_mapping_mode,
11040         "settlement_mode": settlement_mode,
11041         "with_terminal_value": with_terminal_value,
11042         "year_end_soc_kwh": None if year_end_soc_kwh is None else float(year_end_soc_kwh),
11043         "year_end_boundary": year_end_boundary_label(year_end_soc_kwh),
11044         "run_id": run_id,
11045     }
11046
11047
11048 def _settlement_ledger_rows(
11049     date: str,
11050     strategy: str,
11051     load_information_case: str,
11052     pv_mapping_mode: str,
11053     settlement_mode: str,
11054     with_terminal_value: bool,
11055     run_id: str,
11056     time_labels: tuple[str, ...],
11057     price: np.ndarray,
11058     versions: list[np.ndarray],
11059     version_times: list[str],
11060     dispatch: pd.DataFrame,
11061     year_end_soc_kwh: float | None = None,
11062 ) -> list[dict]:
11063     keys = _run_key_fields(
11064         date,
11065         strategy,
11066         load_information_case,
11067         pv_mapping_mode,
11068         settlement_mode,
11069         with_terminal_value,
11070         run_id,
11071         year_end_soc_kwh,
11072     )
11073     ordinary, up_fee, down_fee = decompose_settlement(price, versions, settlement_mode)
11074     emergency = dispatch["emergency_cost_yuan"].to_numpy()

```

```

11075 emergency_kwh = dispatch["emergency_kwh"].to_numpy()
11076 g0 = versions[0]
11077 gf = versions[-1]
11078 rows: list[dict] = []
11079 for t in range(T):
11080     p = float(price[t])
11081     rows.append(
11082         {
11083             **keys,
11084             "row_role": "period_total",
11085             "period": t,
11086             "time": time_labels[t],
11087             "plan_version_time": str(dispatch["last_update_time"].iloc[t]),
11088             "g_before_kwh": float(g0[t]),
11089             "g_after_kwh": float(gf[t]),
11090             "up_adjust_kwh": float(max(gf[t] - g0[t], 0.0)),
11091             "down_adjust_kwh": float(max(g0[t] - gf[t], 0.0)),
11092             "ordinary_yuan": float(ordinary[t]),
11093             "up_fee_yuan": float(up_fee[t]),
11094             "down_fee_yuan": float(down_fee[t]),
11095             "emergency_kwh": float(emergency_kwh[t]),
11096             "emergency_yuan": float(emergency[t]),
11097             "total_yuan": float(ordinary[t] + up_fee[t] + down_fee[t] + emergency[t]),
11098         }
11099     )
11100 if settlement_mode == SETTLEMENT_ALT:
11101     rows.append(
11102         {
11103             **keys,
11104             "row_role": "event",
11105             "period": t,
11106             "time": time_labels[t],
11107             "plan_version_time": "00:00",
11108             "g_before_kwh": 0.0,
11109             "g_after_kwh": float(g0[t]),
11110             "up_adjust_kwh": float(g0[t]),
11111             "down_adjust_kwh": 0.0,
11112             "ordinary_yuan": p * float(g0[t]),
11113             "up_fee_yuan": 0.0,
11114             "down_fee_yuan": 0.0,
11115             "emergency_kwh": 0.0,
11116             "emergency_yuan": 0.0,
11117             "total_yuan": p * float(g0[t]),
11118         }
11119     )
11120 for prev, cur, ts in zip(versions[:-1], versions[1:], version_times[1:]):
11121     up = float(max(cur[t] - prev[t], 0.0))
11122     down = float(max(prev[t] - cur[t], 0.0))
11123     rows.append(
11124         {
11125             **keys,
11126             "row_role": "event",
11127             "period": t,
11128             "time": time_labels[t],
11129             "plan_version_time": ts,
11130             "g_before_kwh": float(prev[t]),
11131             "g_after_kwh": float(cur[t]),
11132             "up_adjust_kwh": up,
11133             "down_adjust_kwh": down,
11134             "ordinary_yuan": 0.0,
11135             "up_fee_yuan": 1.5 * p * up,
11136             "down_fee_yuan": 0.5 * p * down,
11137             "emergency_kwh": 0.0,
11138             "emergency_yuan": 0.0,
11139             "total_yuan": 1.5 * p * up + 0.5 * p * down,
11140         }
11141     )
11142 else:

```



```

11143         up = float(max(gf[t] - g0[t], 0.0))
11144         down = float(max(g0[t] - gf[t], 0.0))
11145         rows.append(
11146             {
11147                 **keys,
11148                 "row_role": "event",
11149                 "period": t,
11150                 "time": time_labels[t],
11151                 "plan_version_time": str(dispatch["last_update_time"].iloc[t]),
11152                 "g_before_kwh": float(g0[t]),
11153                 "g_after_kwh": float(gf[t]),
11154                 "up_adjust_kwh": up,
11155                 "down_adjust_kwh": down,
11156                 "ordinary_yuan": p * float(gf[t]),
11157                 "up_fee_yuan": 0.5 * p * up,
11158                 "down_fee_yuan": 0.5 * p * down,
11159                 "emergency_kwh": 0.0,
11160                 "emergency_yuan": 0.0,
11161                 "total_yuan": p * float(gf[t]) + 0.5 * p * (up + down),
11162             }
11163         )
11164         rows.append(
11165             {
11166                 **keys,
11167                 "row_role": "event",
11168                 "period": t,
11169                 "time": time_labels[t],
11170                 "plan_version_time": "execution",
11171                 "g_before_kwh": float(gf[t]),
11172                 "g_after_kwh": float(gf[t]),
11173                 "up_adjust_kwh": 0.0,
11174                 "down_adjust_kwh": 0.0,
11175                 "ordinary_yuan": 0.0,
11176                 "up_fee_yuan": 0.0,
11177                 "down_fee_yuan": 0.0,
11178                 "emergency_kwh": float(emergency_kwh[t]),
11179                 "emergency_yuan": float(emergency[t]),
11180                 "total_yuan": float(emergency[t]),
11181             }
11182         )
11183         return rows
11184
11185
11186     def _value_cut_cache_key(
11187         day_index: int,
11188         issue_hour: int,
11189         pv_mapping_mode: str,
11190         next_day_terminal_soc: float | None = None,
11191     ) -> tuple[int, int, str, str]:
11192         term = "" if next_day_terminal_soc is None else f"{float(next_day_terminal_soc):.6f}"
11193         return (int(day_index), int(issue_hour), str(pv_mapping_mode), term)
11194
11195
11196     def _next_day_value_cuts(
11197         data: Q3Data,
11198         day_index: int,
11199         issue_hour: int,
11200         cache: ValueCutCache | None,
11201         pv_mapping_mode: str = PV_MAPPING_LINEAR,
11202         next_day_terminal_soc: float | None = None,
11203     ) -> tuple[tuple[ValueCut, ...], list[dict]]:
11204         key = _value_cut_cache_key(day_index, issue_hour, pv_mapping_mode, next_day_terminal_soc)
11205         if cache is not None and key in cache:
11206             return cache[key]
11207         cuts, rows = build_next_day_value_cuts(
11208             data, day_index, issue_hour, pv_mapping_mode, next_day_terminal_soc
11209         )
11210         if cache is not None:

```

```

11211         cache[key] = (cuts, rows)
11212     return cuts, rows
11213
11214
11215 def run_day(
11216     data: Q3Data,
11217     day_index: int,
11218     strategy: str,
11219     initial_soc: float,
11220     with_terminal_value: bool = True,
11221     load_information_case: str = LOAD_INFORMATION_MAIN,
11222     pv_mapping_mode: str = PV_MAPPING_LINEAR,
11223     settlement_mode: str = SETTLEMENT_MAIN,
11224     value_cut_cache: ValueCutCache | None = None,
11225     year_end_soc_kwh: float | None = None,
11226 ) -> DayRun:
11227     date = data.dates[day_index].strftime("%Y-%m-%d")
11228     run_id = make_run_id(
11229         date,
11230         strategy,
11231         load_information_case,
11232         pv_mapping_mode,
11233         settlement_mode,
11234         with_terminal_value,
11235         year_end_soc_kwh,
11236     )
11237     n_days = len(data.dates)
11238     today_terminal = today_year_end_soc(day_index, n_days, year_end_soc_kwh)
11239     next_terminal = next_day_year_end_soc(day_index, n_days, year_end_soc_kwh)
11240     allowed = STRATEGY_ALLOWED_UPDATES[strategy]
11241     price = data.price
11242     actual_load = data.load[day_index]
11243     plan_load = planning_load_curve(data, day_index, load_information_case)
11244     actual_pv = data.pv[day_index]
11245     started = perf_counter()
11246
11247     mapped0 = map_issue_forecast(data, day_index, 0, pv_mapping_mode)
11248     current_forecast = mapped0.today_kwh.copy()
11249     cuts: tuple[ValueCut, ...] = ()
11250     value_rows: list[dict] = []
11251     if with_terminal_value:
11252         cuts, rows = _next_day_value_cuts(
11253             data, day_index, 0, value_cut_cache, pv_mapping_mode, next_terminal
11254         )
11255         value_rows.extend(rows)
11256
11257     midnight = solve_horizon(
11258         price,
11259         plan_load,
11260         current_forecast,
11261         initial_soc,
11262         bill_as_day_ahead=True,
11263         terminal_value_cuts=cuts or None,
11264         terminal_soc=today_terminal,
11265     )
11266     assert_physical(midnight, plan_load, current_forecast, prefix=f"{date} 0:00 plan ")
11267     g0 = midnight.g.copy()
11268     g = g0.copy()
11269     versions = [g0.copy()]
11270     version_times = ["00:00"]
11271     last_update = "00:00"
11272
11273     def remaining_kwargs(t: int, g_pre_rem: np.ndarray) -> dict:
11274         kwargs: dict = {
11275             "settlement_mode": settlement_mode,
11276             "terminal_value_cuts": cuts or None,
11277             "g0": g0[t:],
11278             "terminal_soc": today_terminal,

```

```

11279     }
11280     if settlement_mode == SETTLEMENT_ALT:
11281         kwargs["g_pre"] = g_pre_rem
11282         kwargs["sunk_settlement"] = float(
11283             np.sum(
11284                 realized_settlement(
11285                     price[t:], [v[t:] for v in versions], SETTLEMENT_ALT
11286                 )
11287             )
11288         )
11289     return kwargs
11290
11291 update_rows = [
11292     {
11293         "date": date,
11294         "update_time": "00:00",
11295         "first_mutable_index": 0,
11296         "voi_yuan": np.nan,
11297         "implemented": True,
11298         "l1_change_kwh": float(np.abs(g0).sum()),
11299         "up_adjust_kwh": float(g0.sum()),
11300         "down_adjust_kwh": 0.0,
11301         "j_fix_yuan": np.nan,
11302         "j_free_yuan": float(midnight.objective),
11303         "soc_before_kwh": float(initial_soc),
11304         "soc_after_plan_kwh": float(midnight.soc[-1]),
11305         "solver_status": midnight.status,
11306         "solve_seconds": midnight.solve_seconds,
11307     }
11308 ]
11309
11310 dispatch_rows = []
11311 soc = float(initial_soc)
11312 max_cd = 0.0
11313 max_balance = 0.0
11314 max_soc_residual = 0.0
11315 locked_violations = 0
11316 g_before_update = g.copy()
11317
11318 for t in range(T):
11319     hour = _index_to_update_hour(t)
11320     if hour is not None and hour in allowed:
11321         g_before_update = g.copy()
11322         mapped = map_issue_forecast(data, day_index, hour, pv_mapping_mode)
11323         current_forecast = mapped.today_kwh.copy()
11324         if with_terminal_value:
11325             cuts, rows = _next_day_value_cuts(
11326                 data, day_index, hour, value_cut_cache, pv_mapping_mode, next_terminal
11327             )
11328             value_rows.extend(rows)
11329         pv_plan = current_forecast[t:]
11330         load_plan = plan_load[t:]
11331         g_pre = g[t:].copy()
11332         settle_kw = remaining_kwargs(t, g_pre)
11333         j_fix = solve_horizon(
11334             price[t:],
11335             load_plan,
11336             pv_plan,
11337             soc,
11338             g_fixed=g_pre,
11339             **settle_kw,
11340         )
11341         j_free = solve_horizon(
11342             price[t:],
11343             load_plan,
11344             pv_plan,
11345             soc,
11346             **settle_kw,

```

```

11347     )
11348     voi = float(j_fix.objective - j_free.objective)
11349     implemented = bool(voi > VOI_EPS_YUAN)
11350     if implemented:
11351         g[t:] = j_free.g
11352         versions.append(g.copy())
11353         version_times.append(f"{hour:02d}:00")
11354     delta = g[t:] - g_pre
11355     update_rows.append(
11356         {
11357             "date": date,
11358             "update_time": f"{hour:02d}:00",
11359             "first_mutable_index": t,
11360             "voi_yuan": voi,
11361             "implemented": implemented,
11362             "l1_change_kwh": float(np.abs(delta).sum()),
11363             "up_adjust_kwh": float(np.maximum(delta, 0.0).sum()),
11364             "down_adjust_kwh": float(np.maximum(-delta, 0.0).sum()),
11365             "j_fix_yuan": float(j_fix.objective),
11366             "j_free_yuan": float(j_free.objective),
11367             "soc_before_kwh": float(soc),
11368             "soc_after_plan_kwh": float((j_free if implemented else j_fix).soc[-1]),
11369             "solver_status": j_free.status,
11370             "solve_seconds": j_fix.solve_seconds + j_free.solve_seconds,
11371         }
11372     )
11373     last_update = f"{hour:02d}:00"
11374     if np.max(np.abs(g[t:] - g_before_update[:t])) > NUMERIC_TOL:
11375         locked_violations += 1
11376
11377     pv_horizon = current_forecast[t:].copy()
11378     pv_horizon[0] = actual_pv[t]
11379     load_horizon = execution_load_horizon(plan_load, actual_load, t)
11380     step = solve_horizon(
11381         price[t:],
11382         load_horizon,
11383         pv_horizon,
11384         soc,
11385         g_fixed=g[t:],
11386         **remaining_kwargs(t, g[t:]),
11387     )
11388     residual = (
11389         step.x[0]
11390         + step.emergency[0]
11391         + actual_pv[t]
11392         - step.curtailment[0]
11393         + step.discharge[0]
11394         - actual_load[t]
11395         - step.charge[0]
11396     )
11397     soc_next = float(step.soc[1])
11398     soc_residual = soc_next - (
11399         soc + ETA_C * step.charge[0] - step.discharge[0] / ETA_D
11400     )
11401     max_cd = max(max_cd, float(step.charge[0] * step.discharge[0]))
11402     max_balance = max(max_balance, abs(float(residual)))
11403     max_soc_residual = max(max_soc_residual, abs(float(soc_residual)))
11404     dispatch_rows.append(
11405         {
11406             "time": data.time_labels[t],
11407             "period": t,
11408             "planned_g0_kwh": float(g0[t]),
11409             "final_g_kwh": float(g[t]),
11410             "normal_x_kwh": float(step.x[0]),
11411             "load_kwh": float(actual_load[t]),
11412             "forecast_load_kwh": float(plan_load[t]),
11413             "actual_pv_kwh": float(actual_pv[t]),
11414             "forecast_pv_kwh": float(current_forecast[t]),

```

```

11415         "charge_kwh": float(step.charge[0]),
11416         "discharge_kwh": float(step.discharge[0]),
11417         "curtailment_kwh": float(step.curtailment[0]),
11418         "emergency_kwh": float(step.emergency[0]),
11419         "soc_kwh": soc_next,
11420         "balance_residual_kwh": float(residual),
11421         "last_update_time": last_update,
11422         "price": float(price[t]),
11423     }
11424 )
11425 if abs(g[t] - step.g[0]) > 1e-6:
11426     locked_violations += 1
11427 soc = soc_next
11428 if max_cd > SIMULTANEOUS_CD_TOL:
11429     raise AssertionError(
11430         f"{date} {strategy} t={t}: simultaneous charge/discharge {max_cd:.3e}"
11431     )
11432
11433 dispatch = pd.DataFrame(dispatch_rows)
11434 phi = realized_settlement(price, versions, settlement_mode)
11435 dispatch["phi_yuan"] = phi
11436 dispatch["emergency_cost_yuan"] = (
11437     EMERGENCY_PRICE_MULTIPLIER * dispatch["price"] * dispatch["emergency_kwh"]
11438 )
11439 dispatch["run_id"] = run_id
11440 dispatch["pv_mapping_mode"] = pv_mapping_mode
11441 dispatch["settlement_mode"] = settlement_mode
11442 dispatch["year_end_soc_kwh"] = None if year_end_soc_kwh is None else float(year_end_soc_kwh)
11443 dispatch["year_end_boundary"] = year_end_boundary_label(year_end_soc_kwh)
11444 ledger = pd.DataFrame(
11445     _settlement_ledger_rows(
11446         date=date,
11447         strategy=strategy,
11448         load_information_case=load_information_case,
11449         pv_mapping_mode=pv_mapping_mode,
11450         settlement_mode=settlement_mode,
11451         with_terminal_value=with_terminal_value,
11452         run_id=run_id,
11453         time_labels=data.time_labels,
11454         price=price,
11455         versions=versions,
11456         version_times=version_times,
11457         dispatch=dispatch,
11458         year_end_soc_kwh=year_end_soc_kwh,
11459     )
11460 )
11461 update_log = pd.DataFrame(update_rows)
11462 update_log.insert(1, "strategy", strategy)
11463 update_log.insert(2, "load_information_case", load_information_case)
11464 update_log.insert(3, "pv_mapping_mode", pv_mapping_mode)
11465 update_log.insert(4, "settlement_mode", settlement_mode)
11466 update_log.insert(5, "with_terminal_value", with_terminal_value)
11467 update_log.insert(6, "year_end_soc_kwh", None if year_end_soc_kwh is None else float(year_end_soc_kwh))
11468 update_log.insert(7, "year_end_boundary", year_end_boundary_label(year_end_soc_kwh))
11469 update_log.insert(8, "run_id", run_id)
11470 n_adjust = int(((update_log["update_time"] != "00:00") & update_log["implemented"]).sum())
11471 summary = {
11472     "date": date,
11473     "strategy": strategy,
11474     "load_information_case": load_information_case,
11475     "pv_mapping_mode": pv_mapping_mode,
11476     "settlement_mode": settlement_mode,
11477     "with_terminal_value": with_terminal_value,
11478     "year_end_soc_kwh": None if year_end_soc_kwh is None else float(year_end_soc_kwh),
11479     "year_end_boundary": year_end_boundary_label(year_end_soc_kwh),
11480     "run_id": run_id,
11481     "total_cost_yuan": float(dispatch["phi_yuan"].sum() + dispatch["emergency_cost_yuan"].sum()),
11482     "settlement_cost_yuan": float(dispatch["phi_yuan"].sum()),

```

```

11483     "emergency_cost_yuan": float(dispatch["emergency_cost_yuan"].sum()),
11484     "curtailment_kwh": float(dispatch["curtailment_kwh"].sum()),
11485     "emergency_kwh": float(dispatch["emergency_kwh"].sum()),
11486     "charge_kwh": float(dispatch["charge_kwh"].sum()),
11487     "discharge_kwh": float(dispatch["discharge_kwh"].sum()),
11488     "soc_start_kwh": float(initial_soc),
11489     "soc_end_kwh": float(dispatch["soc_kwh"].iloc[-1]),
11490     "adjustment_count": n_adjust,
11491     "up_adjust_kwh": float(
11492         np.maximum(
11493             dispatch["final_g_kwh"].to_numpy() - dispatch["planned_g0_kwh"].to_numpy(),
11494             0.0,
11495         ).sum()
11496     ),
11497     "down_adjust_kwh": float(
11498         np.maximum(
11499             dispatch["planned_g0_kwh"].to_numpy() - dispatch["final_g_kwh"].to_numpy(),
11500             0.0,
11501         ).sum()
11502     ),
11503     "max_balance_residual_kwh": float(max_balance),
11504     "max_soc_residual_kwh": float(max_soc_residual),
11505     "max_simultaneous_cd_kwh2": float(max_cd),
11506     "locked_period_violations": int(locked_violations),
11507     "runtime_seconds": float(perf_counter() - started),
11508 }
11509 return DayRun(
11510     date,
11511     strategy,
11512     load_information_case,
11513     pv_mapping_mode,
11514     settlement_mode,
11515     with_terminal_value,
11516     run_id,
11517     dispatch,
11518     update_log,
11519     ledger,
11520     summary,
11521     value_rows,
11522     year_end_soc_kwh,
11523 )
11524
11525
11526 def warmup_soc_to(
11527     data: Q3Data, target_index: int, path: Path
11528 ) -> tuple[np.ndarray, pd.DataFrame]:
11529     """Propagate M0 from 2025-01-01 so the first pilot day has a causal SOC."""
11530     starts = np.full(target_index + 1, E_INITIAL_KWH)
11531     rows = []
11532     soc = E_INITIAL_KWH
11533     for i in range(target_index):
11534         starts[i] = soc
11535         result = run_day(
11536             data,
11537             i,
11538             "M0",
11539             soc,
11540             with_terminal_value=True,
11541             load_information_case=LOAD_INFORMATION_MAIN,
11542         )
11543         soc = float(result.summary["soc_end_kwh"])
11544         rows.append(
11545             {
11546                 "date": result.date,
11547                 "soc_start_kwh": result.summary["soc_start_kwh"],
11548                 "soc_end_kwh": result.summary["soc_end_kwh"],
11549                 "total_cost_yuan": result.summary["total_cost_yuan"],
11550                 "strategy": "M0_warmup",

```

```

11551         "load_information_case": LOAD_INFORMATION_MAIN,
11552     }
11553 )
11554 starts[target_index] = soc
11555 frame = pd.DataFrame(rows)
11556 path.parent.mkdir(parents=True, exist_ok=True)
11557 frame.to_csv(path, index=False)
11558 return starts, frame
11559
11560
11561 def _spotcheck_settlement(dispatch: pd.DataFrame) -> list[dict]:
11562     phi = dispatch["phi_yuan"].to_numpy()
11563     g0 = dispatch["planned_g0_kwh"].to_numpy()
11564     gf = dispatch["final_g_kwh"].to_numpy()
11565     price = dispatch["price"].to_numpy()
11566     rows = []
11567     cases = {
11568         "no_adjust": np.where(np.abs(gf - g0) <= 1e-8)[0],
11569         "down": np.where(gf + 1e-8 < g0)[0],
11570         "up": np.where(gf > g0 + 1e-8)[0],
11571     }
11572     for name, indices in cases.items():
11573         if len(indices) == 0:
11574             rows.append({"case": name, "found": False})
11575             continue
11576         t = int(indices[len(indices) // 2])
11577         expected = settlement_cost(price[t : t + 1], g0[t : t + 1], gf[t : t + 1])[0]
11578         rows.append(
11579             {
11580                 "case": name,
11581                 "found": True,
11582                 "period": t,
11583                 "time": str(dispatch["time"].iloc[t]),
11584                 "g0_kwh": float(g0[t]),
11585                 "gF_kwh": float(gf[t]),
11586                 "price": float(price[t]),
11587                 "phi_yuan": float(phi[t]),
11588                 "expected_phi_yuan": float(expected),
11589                 "abs_error": float(abs(phi[t] - expected)),
11590             }
11591         )
11592     return rows
11593
11594
11595 def write_cost_audit(runs: list[DayRun], path: Path) -> pd.DataFrame:
11596     frames = []
11597     spot = []
11598     for run in runs:
11599         block = run.dispatch.copy()
11600         block["date"] = run.date
11601         block["strategy"] = run.strategy
11602         block["load_information_case"] = run.load_information_case
11603         block["pv_mapping_mode"] = run.pv_mapping_mode
11604         block["settlement_mode"] = run.settlement_mode
11605         block["with_terminal_value"] = run.with_terminal_value
11606         block["run_id"] = run.run_id
11607         frames.append(
11608             block[
11609                 [
11610                     "date",
11611                     "strategy",
11612                     "load_information_case",
11613                     "pv_mapping_mode",
11614                     "settlement_mode",
11615                     "with_terminal_value",
11616                     "run_id",
11617                     "time",
11618                     "planned_g0_kwh",

```

```

11619         "final_g_kwh",
11620         "price",
11621         "phi_yuan",
11622         "emergency_kwh",
11623         "emergency_cost_yuan",
11624     ]
11625 ]
11626 )
11627 for item in _spotcheck_settlement(run.dispatch):
11628     spot.append(
11629         {
11630             "date": run.date,
11631             "strategy": run.strategy,
11632             "load_information_case": run.load_information_case,
11633             "pv_mapping_mode": run.pv_mapping_mode,
11634             "settlement_mode": run.settlement_mode,
11635             "with_terminal_value": run.with_terminal_value,
11636             "run_id": run.run_id,
11637             **item,
11638         }
11639     )
11640 frame = pd.concat(frames, ignore_index=True)
11641 path.parent.mkdir(parents=True, exist_ok=True)
11642 frame.to_csv(path, index=False)
11643 path.with_name("q3_cost_spotcheck.csv").write_text(
11644     pd.DataFrame(spot).to_csv(index=False, encoding="utf-8"
11645 )
11646 return frame
11647
11648
11649 def plot_pilot_figures(runs: list[DayRun], comparison: pd.DataFrame) -> None:
11650     FIG_DIR.mkdir(parents=True, exist_ok=True)
11651     main = comparison[
11652         comparison["strategy"].isin(STRATEGIES)
11653         & comparison["with_terminal_value"]
11654         & comparison["load_information_case"].eq(LOAD_INFORMATION_MAIN)
11655     ].copy()
11656     fig, ax = plt.subplots(figsize=(8.2, 4.4))
11657     dates = list(dict.fromkeys(main["date"]))
11658     x = np.arange(len(STRATEGIES))
11659     width = 0.35
11660     for i, date in enumerate(dates):
11661         subset = main[main["date"] == date].set_index("strategy").loc[list(STRATEGIES)]
11662         ax.bar(
11663             x + (i - 0.5) * width,
11664             subset["total_cost_yuan"],
11665             width=width,
11666             color=(NAVY if i == 0 else TEAL),
11667             label=date,
11668         )
11669     ax.set_xticks(x)
11670     ax.set_xticklabels(STRATEGIES)
11671     ax.set_ylabel("total purchase cost (yuan)")
11672     ax.set_title("Q3 pilot: update-policy cost (causal_load_main)")
11673     ax.legend(frameon=False)
11674     ax.spines["top"].set_visible(False)
11675     ax.spines["right"].set_visible(False)
11676     fig.tight_layout()
11677     fig.savefig(FIG_DIR / "fig3_strategy_cost.png")
11678     fig.savefig(FIG_DIR / "fig3_strategy_cost_causal_load_main.png")
11679     plt.close(fig)
11680
11681     for run in runs:
11682         if (
11683             run.strategy != "M1_M6"
11684             or not run.with_terminal_value
11685             or run.load_information_case != LOAD_INFORMATION_MAIN
11686         ):

```



```

11687         continue
11688     d = run.dispatch
11689     fig, axes = plt.subplots(3, 1, figsize=(8.6, 7.2), sharex=True)
11690     t = np.arange(len(d))
11691     axes[0].plot(t, d["load_kwh"], color=NAVY, lw=1.2, label="actual load")
11692     axes[0].plot(t, d["forecast_load_kwh"], color=PURPLE, lw=1.0, ls="--", label="causal load hat")
11693     axes[0].plot(t, d["actual_pv_kwh"], color=TEAL, lw=1.2, label="actual PV")
11694     axes[0].plot(t, d["forecast_pv_kwh"], color=ORANGE, lw=1.0, ls="--", label="active PV forecast")
11695     axes[0].legend(frameon=False, ncol=2)
11696     axes[0].set_ylabel("kWh / 10 min")
11697     axes[1].plot(t, d["planned_g0_kwh"], color=PURPLE, lw=1.1, label="g0")
11698     axes[1].plot(t, d["final_g_kwh"], color=ORANGE, lw=1.2, label="gF")
11699     axes[1].plot(t, d["normal_x_kwh"], color=NAVY, lw=1.0, ls=":", label="x")
11700     axes[1].legend(frameon=False, ncol=3)
11701     axes[1].set_ylabel("purchase kWh")
11702     axes[2].plot(t, d["soc_kwh"], color=TEAL, lw=1.3)
11703     axes[2].axhline(E_MIN_KWH, color=ORANGE, ls="--", lw=0.8)
11704     axes[2].axhline(E_MAX_KWH, color=ORANGE, ls="--", lw=0.8)
11705     axes[2].set_ylabel("SOC kWh")
11706     axes[2].set_xlabel("10-minute index")
11707     fig.suptitle(f"{run.date} M1/M6 dispatch ({LOAD_INFORMATION_MAIN})", fontsize=12)
11708     fig.tight_layout()
11709     fig.savefig(FIG_DIR / f"fig3_dispatch_{run.date}_M1_M6.png")
11710     fig.savefig(FIG_DIR / f"fig3_dispatch_{run.date}_M1_M6_{LOAD_INFORMATION_MAIN}.png")
11711     plt.close(fig)
11712
11713     log = run.update_log[run.update_log["update_time"] != "00:00"]
11714     fig, ax = plt.subplots(figsize=(6.4, 3.8))
11715     ax.bar(log["update_time"], log["voi_yuan"], color=TEAL)
11716     ax.axhline(VOI_EPS_YUAN, color=ORANGE, ls="--", lw=0.9, label="VoI threshold")
11717     ax.set_ylabel("VoI (yuan)")
11718     ax.set_title(f"{run.date} information value by update ({LOAD_INFORMATION_MAIN})")
11719     ax.legend(frameon=False)
11720     ax.spines["top"].set_visible(False)
11721     ax.spines["right"].set_visible(False)
11722     fig.tight_layout()
11723     fig.savefig(FIG_DIR / f"fig3_voi_{run.date}.png")
11724     fig.savefig(FIG_DIR / f"fig3_voi_{run.date}_{LOAD_INFORMATION_MAIN}.png")
11725     plt.close(fig)
11726
11727     info = comparison[
11728         comparison["strategy"].isin(STRATEGIES) & comparison["with_terminal_value"]
11729     ].copy()
11730     fig, axes = plt.subplots(1, 2, figsize=(9.4, 4.2), sharey=True)
11731     x = np.arange(len(STRATEGIES))
11732     width = 0.35
11733     for ax, date in zip(axes, list(dict.fromkeys(info["date"]))):
11734         for i, case in enumerate(LOAD_INFORMATION_CASES):
11735             subset = (
11736                 info[(info["date"] == date) & (info["load_information_case"] == case)]
11737                 .set_index("strategy")
11738                 .loc[list(STRATEGIES)]
11739             )
11740             ax.bar(
11741                 x + (i - 0.5) * width,
11742                 subset["total_cost_yuan"],
11743                 width=width,
11744                 color=(NAVY if i == 0 else ORANGE),
11745                 label=case,
11746             )
11747             ax.set_xticks(x)
11748             ax.set_xticklabels(STRATEGIES, rotation=20, ha="right")
11749             ax.set_title(date)
11750             ax.spines["top"].set_visible(False)
11751             ax.spines["right"].set_visible(False)
11752     axes[0].set_ylabel("total purchase cost (yuan)")
11753     axes[0].legend(frameon=False, fontsize=8)
11754     fig.suptitle("Load-information comparison (same SOC, prices, PV, settlement)", fontsize=12)

```

```

11755     fig.tight_layout()
11756     fig.savefig(FIG_DIR / "fig3_load_information_comparison.png")
11757     plt.close(fig)
11758
11759
11760 def physical_audit(runs: list[DayRun], warmup_end_soc: float | None) -> dict:
11761     items = []
11762     all_pass = True
11763     for run in runs:
11764         d = run.dispatch
11765         soc = np.concatenate([[run.summary["soc_start_kwh"]], d["soc_kwh"].to_numpy()])
11766         rec = {
11767             "date": run.date,
11768             "strategy": run.strategy,
11769             "load_information_case": run.load_information_case,
11770             "pv_mapping_mode": run.pv_mapping_mode,
11771             "settlement_mode": run.settlement_mode,
11772             "with_terminal_value": run.with_terminal_value,
11773             "year_end_soc_kwh": run.year_end_soc_kwh,
11774             "year_end_boundary": year_end_boundary_label(run.year_end_soc_kwh),
11775             "run_id": run.run_id,
11776             "max_balance_residual_kwh": run.summary["max_balance_residual_kwh"],
11777             "max_soc_residual_kwh": run.summary["max_soc_residual_kwh"],
11778             "max_simultaneous_cd_kwh2": run.summary["max_simultaneous_cd_kwh2"],
11779             "soc_min_kwh": float(np.min(soc)),
11780             "soc_max_kwh": float(np.max(soc)),
11781             "locked_period_violations": run.summary["locked_period_violations"],
11782             "pass": bool(
11783                 run.summary["max_balance_residual_kwh"] < 1e-5
11784                 and run.summary["max_soc_residual_kwh"] < 1e-5
11785                 and run.summary["max_simultaneous_cd_kwh2"] <= SIMULTANEOUS_CD_TOL
11786                 and run.summary["locked_period_violations"] == 0
11787                 and np.min(soc) >= E_MIN_KWH - 1e-6
11788                 and np.max(soc) <= E_MAX_KWH + 1e-6
11789             ),
11790         }
11791         all_pass = all_pass and rec["pass"]
11792         items.append(rec)
11793     return {
11794         "all_pass": all_pass,
11795         "warmup_end_soc_for_2025-02-01": warmup_end_soc,
11796         "warmup_load_information_case": LOAD_INFORMATION_MAIN,
11797         "runs": items,
11798     }
11799
11800
11801 def write_load_forecast_audit(data: Q3Data, path: Path) -> pd.DataFrame:
11802     rows: list[dict] = []
11803     for date in PILOT_DATES:
11804         rows.extend(load_forecast_audit_rows(data, data.date_index(date)))
11805     frame = pd.DataFrame(rows)
11806     path.parent.mkdir(parents=True, exist_ok=True)
11807     frame.to_csv(path, index=False)
11808     return frame
11809
11810
11811 def write_load_information_comparison(comparison: pd.DataFrame, path: Path) -> pd.DataFrame:
11812     cols = [
11813         "date",
11814         "strategy",
11815         "load_information_case",
11816         "with_terminal_value",
11817         "total_cost_yuan",
11818         "settlement_cost_yuan",
11819         "emergency_cost_yuan",
11820         "emergency_kwh",
11821         "curtailment_kwh",
11822         "soc_start_kwh",

```

```

11823         "soc_end_kwh",
11824         "runtime_seconds",
11825     ]
11826     frame = comparison[cols].copy()
11827     path.parent.mkdir(parents=True, exist_ok=True)
11828     frame.to_csv(path, index=False)
11829     return frame
11830
11831
11832 def run_causality_audit(data: Q3Data, day_index: int, initial_soc: float) -> dict:
11833     """Perturb unrealized same-day actual load: main forecasts stay put, proxy may move."""
11834     date = data.dates[day_index].strftime("%Y-%m-%d")
11835     price = data.price
11836     pv0 = map_issue_forecast(data, day_index, 0).today_kwh
11837     hat_before, sources_before, label_before = causal_load_sources(data, day_index, day_index)
11838     proxy_before = planning_load_curve(data, day_index, LOAD_INFORMATION_PROXY)
11839     next_before = (
11840         causal_load_forecast(data, day_index + 1, day_index)
11841         if day_index + 1 < len(data.dates)
11842         else None
11843     )
11844     g0_main_before = solve_horizon(
11845         price, hat_before, pv0, initial_soc, bill_as_day_ahead=True, throughput_tiebreak=False
11846     ).g
11847     g0_proxy_before = solve_horizon(
11848         price, proxy_before, pv0, initial_soc, bill_as_day_ahead=True, throughput_tiebreak=False
11849     ).g
11850
11851     original = data.load[day_index].copy()
11852     perturbation_kwh = 500.0
11853     try:
11854         data.load[day_index, 36:] += perturbation_kwh
11855         hat_after, sources_after, label_after = causal_load_sources(data, day_index, day_index)
11856         proxy_after = planning_load_curve(data, day_index, LOAD_INFORMATION_PROXY)
11857         next_after = (
11858             causal_load_forecast(data, day_index + 1, day_index)
11859             if day_index + 1 < len(data.dates)
11860             else None
11861         )
11862         g0_main_after = solve_horizon(
11863             price, hat_after, pv0, initial_soc, bill_as_day_ahead=True, throughput_tiebreak=False
11864         ).g
11865         g0_proxy_after = solve_horizon(
11866             price, proxy_after, pv0, initial_soc, bill_as_day_ahead=True, throughput_tiebreak=False
11867         ).g
11868         mapped6_before = map_issue_forecast(data, day_index, 6)
11869         plan_main_update_before = hat_before[36:]
11870         plan_main_update_after = hat_after[36:]
11871     finally:
11872         data.load[day_index] = original
11873
11874     main_forecast_delta = float(np.max(np.abs(hat_after - hat_before)))
11875     main_update_delta = float(np.max(np.abs(plan_main_update_after - plan_main_update_before)))
11876     main_g0_delta = float(np.max(np.abs(g0_main_after - g0_main_before)))
11877     proxy_plan_delta = float(np.max(np.abs(proxy_after - proxy_before)))
11878     proxy_g0_delta = float(np.max(np.abs(g0_proxy_after - g0_proxy_before)))
11879     next_delta = (
11880         float(np.max(np.abs(next_after - next_before))) if next_before is not None else 0.0
11881     )
11882     payload = {
11883         "date": date,
11884         "perturbed_slice": "periods[36:] (unrealized same-day actual load after 06:00)",
11885         "perturbation_kwh": perturbation_kwh,
11886         "source_dates": label_before,
11887         "source_dates_after_perturbation": label_after,
11888         "main_day_ahead_forecast_unchanged": bool(main_forecast_delta < 1e-12),
11889         "main_update_forecast_unchanged": bool(main_update_delta < 1e-12),
11890         "main_next_day_forecast_unchanged": bool(next_delta < 1e-12),

```

```

11891     "main_g0_unchanged": bool(main_g0_delta < 1e-6),
11892     "proxy_planning_load_changed": bool(proxy_plan_delta > 1.0),
11893     "proxy_g0_changed": bool(proxy_g0_delta > 1e-6),
11894     "max_abs_main_forecast_delta_kwh": main_forecast_delta,
11895     "max_abs_main_update_forecast_delta_kwh": main_update_delta,
11896     "max_abs_main_g0_delta_kwh": main_g0_delta,
11897     "max_abs_proxy_planning_delta_kwh": proxy_plan_delta,
11898     "max_abs_proxy_g0_delta_kwh": proxy_g0_delta,
11899     "max_abs_next_day_forecast_delta_kwh": next_delta,
11900     "source_indices_unchanged": sources_before == sources_after,
11901     "six_am_first_mutable_index": int(mapped6_before.first_mutable_index),
11902     "note": (
11903         "causal_load_main day-ahead and 6/12/18 future load stay at the 0:00 hat. "
11904         "actual_load_proxy is allowed to change because it reads the full same-day Attachment 2 path."
11905     ),
11906 }
11907 payload["all_pass"] = bool(
11908     payload["main_day_ahead_forecast_unchanged"]
11909     and payload["main_update_forecast_unchanged"]
11910     and payload["main_next_day_forecast_unchanged"]
11911     and payload["main_g0_unchanged"]
11912     and payload["proxy_planning_load_changed"]
11913     and payload["proxy_g0_changed"]
11914     and payload["source_indices_unchanged"]
11915 )
11916 return payload
11917
11918
11919 def run_q3_pilot(data: Q3Data) -> dict:
11920     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
11921     FIG_DIR.mkdir(parents=True, exist_ok=True)
11922     from q3.data import write_input_audit
11923
11924     write_input_audit(data, OUTPUT_DIR / "input_audit.json")
11925     write_forecast_mapping(data, OUTPUT_DIR / "q3_forecast_mapping.csv")
11926     write_load_forecast_audit(data, OUTPUT_DIR / "q3_load_forecast_audit.csv")
11927
11928     feb_index = data.date_index("2025-02-01")
11929     warmup_path = OUTPUT_DIR / WARMUP_CSV_MAIN
11930     if warmup_path.exists() and len(pd.read_csv(warmup_path)) >= feb_index:
11931         warmup_frame = pd.read_csv(warmup_path)
11932         warmup_starts = np.full(feb_index + 1, E_INITIAL_KWH)
11933         warmup_starts[1:] = warmup_frame["soc_end_kwh"].to_numpy()[1:feb_index]
11934         print(f"reusing causal_load_main warmup SOC from {warmup_path}")
11935     else:
11936         warmup_starts, _warmup = warmup_soc_to(data, feb_index, warmup_path)
11937     initial = {
11938         "2025-02-01": float(warmup_starts[feb_index]),
11939         "2025-06-21": E_INITIAL_KWH,
11940     }
11941
11942     value_cut_cache: ValueCutCache = {}
11943     runs: list[DayRun] = []
11944     for date in PILOT_DATES:
11945         i = data.date_index(date)
11946         soc0 = initial[date]
11947         for case in LOAD_INFORMATION_CASES:
11948             for strategy in STRATEGIES:
11949                 runs.append(
11950                     run_day(
11951                         data,
11952                         i,
11953                         strategy,
11954                         soc0,
11955                         with_terminal_value=True,
11956                         load_information_case=case,
11957                         value_cut_cache=value_cut_cache,
11958                 )

```

```

11959         )
11960     runs.append(
11961         run_day(
11962             data,
11963             i,
11964             "M1_M6",
11965             soc0,
11966             with_terminal_value=False,
11967             load_information_case=case,
11968             value_cut_cache=value_cut_cache,
11969         )
11970     )
11971
11972     comparison = pd.DataFrame([run.summary for run in runs])
11973     comparison.to_csv(OUTPUT_DIR / "q3_strategy_comparison.csv", index=False)
11974     write_load_information_comparison(
11975         comparison, OUTPUT_DIR / "q3_load_information_comparison.csv"
11976     )
11977     update_log = pd.concat([run.update_log for run in runs], ignore_index=True)
11978     update_log.to_csv(OUTPUT_DIR / "q3_update_log.csv", index=False)
11979     value_rows = []
11980     for _key in sorted(value_cut_cache):
11981         value_rows.extend(value_cut_cache[_key][1])
11982     if value_rows:
11983         pd.DataFrame(value_rows).to_csv(OUTPUT_DIR / "q3_next_day_value_audit.csv", index=False)
11984     write_cost_audit(runs, OUTPUT_DIR / "q3_cost_audit.csv")
11985
11986     dispatch_cols = [
11987         "time",
11988         "planned_g0_kwh",
11989         "final_g_kwh",
11990         "normal_x_kwh",
11991         "load_kwh",
11992         "forecast_load_kwh",
11993         "actual_pv_kwh",
11994         "forecast_pv_kwh",
11995         "charge_kwh",
11996         "discharge_kwh",
11997         "curtailment_kwh",
11998         "emergency_kwh",
11999         "soc_kwh",
12000         "balance_residual_kwh",
12001         "last_update_time",
12002     ]
12003     for run in runs:
12004         stem = dispatch_output_stem(
12005             run.date, run.strategy, run.load_information_case, run.with_terminal_value
12006         )
12007         run.dispatch[dispatch_cols].to_csv(OUTPUT_DIR / f"{stem}.csv", index=False)
12008
12009     plot_pilot_figures(runs, comparison)
12010     audit = physical_audit(runs, initial["2025-02-01"])
12011     (OUTPUT_DIR / "q3_physical_audit.json").write_text(
12012         json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
12013     )
12014     causality = run_causality_audit(
12015         data, data.date_index("2025-02-01"), initial["2025-02-01"]
12016     )
12017     (OUTPUT_DIR / "q3_causality_audit.json").write_text(
12018         json.dumps(causality, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
12019     )
12020     if not causality["all_pass"]:
12021         raise AssertionError(f"Q3 load causality audit failed: {causality}")
12022     return {
12023         "initial_soc": initial,
12024         "comparison": comparison,
12025         "audit": audit,
12026         "causality": causality,

```

```

12027         "n_runs": len(runs),
12028         "load_treatment": LOAD_INFORMATION_MAIN,
12029     }
12030
12031     # ===== q3/run_q3_full_annual.py =====
12032     #!/usr/bin/env python3
12033     from __future__ import annotations
12034
12035     import argparse
12036     import json
12037     import subprocess
12038     import sys
12039     from time import perf_counter
12040
12041     ROOT = __import__("pathlib").Path(__file__).resolve().parents[1]
12042     if str(ROOT) not in sys.path:
12043         sys.path.insert(0, str(ROOT))
12044
12045     from q3.config import ATTACH1, ATTACH2, ATTACH3, ANNUAL_OUTPUT_DIR # noqa: E402
12046     from q3.data import load_q3_data # noqa: E402
12047     from q3.full_annual import parse_boundaries, run_q3_full_annual # noqa: E402
12048
12049
12050     def _git_hash() -> str:
12051         try:
12052             return subprocess.check_output(
12053                 ["git", "rev-parse", "HEAD"], cwd=ROOT, text=True
12054             ).strip()
12055         except (OSError, subprocess.CalledProcessError):
12056             return "unknown"
12057
12058
12059     def main() -> None:
12060         parser = argparse.ArgumentParser(description="Q3 full-year continuous SOC for year-end A and/or B")
12061         parser.add_argument(
12062             "--boundary",
12063             default="both",
12064             help="A, B, or both (default both). A then B are independent 365-day campaigns.",
12065         )
12066         args = parser.parse_args()
12067         selected = parse_boundaries(args.boundary)
12068         started = perf_counter()
12069         data = load_q3_data()
12070         result = run_q3_full_annual(data, boundaries=selected)
12071         elapsed = perf_counter() - started
12072         payload = {
12073             "attach1": str(ATTACH1),
12074             "attach2": str(ATTACH2),
12075             "attach3": str(ATTACH3),
12076             "output_dir": str(ANNUAL_OUTPUT_DIR),
12077             "elapsed_seconds": elapsed,
12078             "n_runs": result["n_runs"],
12079             "all_physical_pass": result["audit"]["all_pass"],
12080             "load_treatment": "causal_load_main",
12081             "pv_mapping_mode": "linear_anchor_main",
12082             "settlement_mode": "anchor_final_main",
12083             "jan01_soc_kwh": 6000.0,
12084             "boundaries": {name: target for name, target in selected},
12085             "git_commit": _git_hash(),
12086             "solver": "HIGHS",
12087             "dependency_versions": {
12088                 "python": sys.version.split()[0],
12089                 "numpy": __import__("numpy").__version__,
12090                 "pandas": __import__("pandas").__version__,
12091                 "cvxpy": __import__("cvxpy").__version__,
12092                 "matplotlib": __import__("matplotlib").__version__,
12093             },
12094             "run_command": ".venv/bin/python q3/run_q3_full_annual.py --boundary both",

```

```

12095         "note": (
12096             "Independent 365-day campaigns for year-end A=1200 and/or B=6000. "
12097             "Does not choose a single official boundary and does not write result3.xlsx."
12098         ),
12099         "totals": result["totals"].to_dict(orient="records"),
12100         "delta": result["delta"].to_dict(orient="records") if len(result["delta"]) else [],
12101     }
12102     ANNUAL_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
12103     (ANNUAL_OUTPUT_DIR / "run_meta.json").write_text(
12104         json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n",
12105         encoding="utf-8",
12106     )
12107     print(json.dumps({k: payload[k] for k in ("elapsed_seconds", "n_runs", "all_physical_pass", "git_commit")}, indent=2))
12108     print(result["totals"].to_string(index=False))
12109     if len(result["delta"]):
12110         print(result["delta"].to_string(index=False))
12111
12112
12113 if __name__ == "__main__":
12114     main()
12115
12116 # ===== q3/run_q3_pilot.py =====
12117 #!/usr/bin/env python3
12118 from __future__ import annotations
12119
12120 import json
12121 import sys
12122 from pathlib import Path
12123 from time import perf_counter
12124
12125 ROOT = Path(__file__).resolve().parents[1]
12126 if str(ROOT) not in sys.path:
12127     sys.path.insert(0, str(ROOT))
12128
12129 from q3.config import ATTACH1, ATTACH2, ATTACH3, OUTPUT_DIR # noqa: E402
12130 from q3.data import load_q3_data # noqa: E402
12131 from q3.pilot import run_q3_pilot # noqa: E402
12132
12133
12134 def main() -> None:
12135     started = perf_counter()
12136     data = load_q3_data()
12137     result = run_q3_pilot(data)
12138     elapsed = perf_counter() - started
12139     payload = {
12140         "attach1": str(ATTACH1),
12141         "attach2": str(ATTACH2),
12142         "attach3": str(ATTACH3),
12143         "output_dir": str(OUTPUT_DIR),
12144         "elapsed_seconds": elapsed,
12145         "initial_soc": result["initial_soc"],
12146         "n_runs": result["n_runs"],
12147         "load_treatment": result.get("load_treatment"),
12148         "all_physical_pass": result["audit"]["all_pass"],
12149         "causality_all_pass": result["causality"]["all_pass"],
12150         "solver": "HIGHS",
12151         "dependency_versions": {
12152             "python": sys.version.split()[0],
12153             "numpy": __import__("numpy").__version__,
12154             "pandas": __import__("pandas").__version__,
12155             "cvxpy": __import__("cvxpy").__version__,
12156             "matplotlib": __import__("matplotlib").__version__,
12157         },
12158         "comparison": result["comparison"].to_dict(orient="records"),
12159     }
12160     (OUTPUT_DIR / "run_meta.json").write_text(
12161         json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n",
12162         encoding="utf-8",

```

```

12163     )
12164     print(
12165         json.dumps(
12166             {
12167                 k: payload[k]
12168                 for k in (
12169                     "elapsed_seconds",
12170                     "initial_soc",
12171                     "all_physical_pass",
12172                     "causality_all_pass",
12173                     "n_runs",
12174                     "load_treatment",
12175                     "dependency_versions",
12176                 )
12177             },
12178             indent=2,
12179         )
12180     )
12181     print(
12182         result["comparison"][
12183             [
12184                 "date",
12185                 "strategy",
12186                 "load_information_case",
12187                 "with_terminal_value",
12188                 "total_cost_yuan",
12189                 "adjustment_count",
12190             ]
12191         ]
12192     )
12193
12194
12195 if __name__ == "__main__":
12196     main()
12197
12198 # ===== q3/run_q3_result3_export.py =====
12199 #!/usr/bin/env python3
12200 """Rerun only A_q2_aligned + M1_M6, stream 10-minute trajectories, write result3.xlsx."""
12201
12202 from __future__ import annotations
12203
12204 import argparse
12205 import json
12206 import sys
12207 from pathlib import Path
12208 from time import perf_counter
12209
12210 import pandas as pd
12211
12212 ROOT = Path(__file__).resolve().parents[1]
12213 if str(ROOT) not in sys.path:
12214     sys.path.insert(0, str(ROOT))
12215
12216 from q3.config import ( # noqa: E402
12217     ANNUAL_OUTPUT_DIR,
12218     ATTACH1,
12219     ATTACH2,
12220     ATTACH3,
12221     E_INITIAL_KWH,
12222     LOAD_INFORMATION_MAIN,
12223     PV_MAPPING_LINEAR,
12224     RESULT3_ANNUAL_COST_YUAN,
12225     RESULT3_EXPORT_AUDIT,
12226     RESULT3_EXPORT_START,
12227     RESULT3_OFFICIAL_STRATEGY,
12228     RESULT3_TEMPLATE,
12229     RESULT3_XLSX,
12230     SETTLEMENT_MAIN,

```



```

12231     YEAR_END_BOUNDARY_A,
12232     YEAR_END_SOC_A_KWH,
12233 )
12234 from q3.data import load_q3_data # noqa: E402
12235 from q3.export_result3 import ( # noqa: E402
12236     compare_to_annual_summary,
12237     dispatch_path,
12238     enrich_dispatch,
12239     export_dates,
12240     git_hash,
12241     has_required_trajectory,
12242     inspect_result3_workbook,
12243     load_daily_trajectory,
12244     summarize_trajectory,
12245     write_daily_trajectory,
12246     write_export_audit,
12247     write_result3_xlsx,
12248 )
12249 from q3.full_annual import annual_dates # noqa: E402
12250 from q3.pilot import ValueCutCache, run_day # noqa: E402
12251
12252 RERUN_DAILY_CSV = "result3_rerun_daily.csv"
12253 PROGRESS_JSON = "result3_export_progress.json"
12254
12255
12256 def _usable_existing(path: Path) -> bool:
12257     if not path.exists():
12258         return False
12259     try:
12260         frame = pd.read_csv(path)
12261         return has_required_trajectory(frame) and "phi_yuan" in frame.columns
12262     except (OSError, ValueError):
12263         return False
12264
12265
12266 def run_main_path(export_only: bool = False) -> dict:
12267     data = load_q3_data()
12268     dates = annual_dates(data)
12269     dispatch_dir = ANNUAL_OUTPUT_DIR / "dispatch_daily"
12270     dispatch_dir.mkdir(parents=True, exist_ok=True)
12271     daily_path = ANNUAL_OUTPUT_DIR / RERUN_DAILY_CSV
12272     cache: ValueCutCache = {}
12273     started = perf_counter()
12274     soc = E_INITIAL_KWH
12275     n_solved = 0
12276     n_skipped = 0
12277     by_date: dict[str, pd.DataFrame] = {}
12278
12279     for i, date in enumerate(dates, start=1):
12280         path = dispatch_path(date, dispatch_dir)
12281         if _usable_existing(path):
12282             frame = load_daily_trajectory(path)
12283             soc = float(frame["soc_end_kwh"].iloc[-1])
12284             by_date[date] = frame
12285             n_skipped += 1
12286             print(f"[{i}/{len(dates)}] skip {date} soc_end={soc:.4f}", flush=True)
12287             continue
12288         if export_only:
12289             raise FileNotFoundError(f"export-only missing trajectory {path}")
12290         day_index = data.date_index(date)
12291         run = run_day(
12292             data,
12293             day_index,
12294             RESULT3_OFFICIAL_STRATEGY,
12295             soc,
12296             with_terminal_value=True,
12297             load_information_case=LOAD_INFORMATION_MAIN,
12298             pv_mapping_mode=PV_MAPPING_LINEAR,

```

```

12299         settlement_mode=SETTLEMENT_MAIN,
12300         value_cut_cache=cache,
12301         year_end_soc_kwh=YEAR_END_SOC_A_KWH,
12302     )
12303     frame = enrich_dispatch(run)
12304     write_daily_trajectory(frame, path)
12305     soc = float(run.summary["soc_end_kwh"])
12306     by_date[date] = frame
12307     n_solved += 1
12308     print(
12309         f"[{i}/{len(dates)}] solved {run.run_id} "
12310         f"cost={run.summary['total_cost_yuan']:.2f} soc_end={soc:.4f}",
12311         flush=True,
12312     )
12313     del run
12314
12315     rerun_daily = pd.DataFrame([summarize_trajectory(by_date[date]) for date in dates])
12316     rerun_daily.to_csv(daily_path, index=False)
12317     export_frames = {date: by_date[date] for date in export_dates()}
12318     xlsx_meta = write_result3_xlsx(export_frames, source_commit=git_hash())
12319     workbook = inspect_result3_workbook(RESULT3_XLSX)
12320     comparison = compare_to_annual_summary(
12321         rerun_daily, ANNUAL_OUTPUT_DIR / "q3_annual_daily_summary.csv"
12322     )
12323     jan_mask = pd.to_datetime(rerun_daily["date"]) < pd.Timestamp(RESULT3_EXPORT_START)
12324     jan_cost = float(rerun_daily.loc[jan_mask, "total_cost_yuan"].sum())
12325     annual_cost = float(rerun_daily["total_cost_yuan"].sum())
12326     payload = {
12327         "attach1": str(ATTACH1),
12328         "attach2": str(ATTACH2),
12329         "attach3": str(ATTACH3),
12330         "template": str(RESULT3_TEMPLATE),
12331         "result3_xlsx": str(RESULT3_XLSX),
12332         "dispatch_dir": str(dispatch_dir),
12333         "rerun_daily_csv": str(daily_path),
12334         "strategy": RESULT3_OFFICIAL_STRATEGY,
12335         "year_end_boundary": YEAR_END_BOUNDARY_A,
12336         "year_end_soc_kwh": YEAR_END_SOC_A_KWH,
12337         "load_information_case": LOAD_INFORMATION_MAIN,
12338         "pv_mapping_mode": PV_MAPPING_LINEAR,
12339         "settlement_mode": SETTLEMENT_MAIN,
12340         "jan01_soc_kwh": E_INITIAL_KWH,
12341         "n_calendar_days": len(dates),
12342         "n_export_days": len(export_dates()),
12343         "export_start": export_dates()[0],
12344         "export_end": export_dates()[-1],
12345         "n_periods": 144,
12346         "n_solved_days": n_solved,
12347         "n_skipped_existing_days": n_skipped,
12348         "elapsed_seconds": float(perf_counter() - started),
12349         "git_commit": git_hash(),
12350         "xlsx": xlsx_meta,
12351         "workbook_inspect": workbook,
12352         "annual_summary_comparison": comparison,
12353         "rerun_annual_cost_yuan": annual_cost,
12354         "rerun_january_cost_yuan": jan_cost,
12355         "rerun_export_window_cost_yuan": float(xlsx_meta["export_window_cost_yuan"]),
12356         "official_annual_cost_yuan": RESULT3_ANNUAL_COST_YUAN,
12357         "export_window_is_not_annual_total": True,
12358         "note": (
12359             "Only A_q2_aligned + M1_M6 was rerun. Trajectories are streamed 10-minute "
12360             "solver output, not reconstructed from q3_annual_daily_summary.csv. "
12361             "February–December workbook totals are not the 365-day official cost."
12362         ),
12363         "all_pass": bool(
12364             comparison.get("all_match", False)
12365             and workbook.get("n_anomalies", 1) == 0
12366             and abs(float(by_date[dates[-1]]["soc_end_kwh"].iloc[-1]) - YEAR_END_SOC_A_KWH)

```

```

12367         < 1e-3
12368     ),
12369 }
12370 write_export_audit(payload)
12371 (ANNUAL_OUTPUT_DIR / PROGRESS_JSON).write_text(
12372     json.dumps(
12373         {
12374             "last_date": dates[-1],
12375             "soc_end_kwh": float(by_date[dates[-1]]["soc_end_kwh"].iloc[-1]),
12376             "n_solved_days": n_solved,
12377         },
12378         indent=2,
12379     )
12380     + "\n",
12381     encoding="utf-8",
12382 )
12383 return payload
12384
12385
12386 def main() -> None:
12387     parser = argparse.ArgumentParser(description="Export official result3.xlsx from A+M1_M6")
12388     parser.add_argument(
12389         "--export-only",
12390         action="store_true",
12391         help="Do not solve; rebuild the workbook from already streamed trajectories.",
12392     )
12393     args = parser.parse_args()
12394     payload = run_main_path(export_only=args.export_only)
12395     print(
12396         json.dumps(
12397             {
12398                 "elapsed_seconds": payload["elapsed_seconds"],
12399                 "n_solved_days": payload["n_solved_days"],
12400                 "n_skipped_existing_days": payload["n_skipped_existing_days"],
12401                 "result3_excel": payload["result3_excel"],
12402                 "export_window_cost_yuan": payload["rerun_export_window_cost_yuan"],
12403                 "annual_cost_yuan": payload["rerun_annual_cost_yuan"],
12404                 "all_pass": payload["all_pass"],
12405                 "audit": str(RESULT3_EXPORT_AUDIT),
12406             },
12407             indent=2,
12408             ensure_ascii=False,
12409         )
12410     )
12411
12412
12413 if __name__ == "__main__":
12414     main()
12415
12416 # ===== q3/run_q3_sensitivity.py =====
12417 #!/usr/bin/env python3
12418 from __future__ import annotations
12419
12420 import json
12421 import subprocess
12422 import sys
12423 from time import perf_counter
12424
12425 ROOT = __import__("pathlib").Path(__file__).resolve().parents[1]
12426 if str(ROOT) not in sys.path:
12427     sys.path.insert(0, str(ROOT))
12428
12429 from q3.config import ATTACH1, ATTACH2, ATTACH3, SENSITIVITY_OUTPUT_DIR # noqa: E402
12430 from q3.data import load_q3_data # noqa: E402
12431 from q3.sensitivity import run_q3_sensitivity # noqa: E402
12432
12433
12434 def _git_hash() -> str:

```

```

12435     try:
12436         return subprocess.check_output(
12437             ["git", "rev-parse", "HEAD"], cwd=ROOT, text=True
12438         ).strip()
12439     except (OSError, subprocess.CalledProcessError):
12440         return "unknown"
12441
12442
12443 def main() -> None:
12444     started = perf_counter()
12445     data = load_q3_data()
12446     result = run_q3_sensitivity(data)
12447     elapsed = perf_counter() - started
12448     payload = {
12449         "attach1": str(ATTACH1),
12450         "attach2": str(ATTACH2),
12451         "attach3": str(ATTACH3),
12452         "output_dir": str(SENSITIVITY_OUTPUT_DIR),
12453         "elapsed_seconds": elapsed,
12454         "initial_soc": result["initial_soc"],
12455         "n_runs": result["n_runs"],
12456         "all_physical_pass": result["audit"]["all_pass"],
12457         "load_treatment": "causal_load_main",
12458         "git_commit": _git_hash(),
12459         "solver": "HIGHS",
12460         "dependency_versions": {
12461             "python": sys.version.split()[0],
12462             "numpy": __import__("numpy").__version__,
12463             "pandas": __import__("pandas").__version__,
12464             "cvxpy": __import__("cvxpy").__version__,
12465             "matplotlib": __import__("matplotlib").__version__,
12466         },
12467         "run_command": ".venv/bin/python q3/run_q3_sensitivity.py",
12468         "comparison": result["comparison"].to_dict(orient="records"),
12469     }
12470     SENSITIVITY_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
12471     (SENSITIVITY_OUTPUT_DIR / "run_meta.json").write_text(
12472         json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n",
12473         encoding="utf-8",
12474     )
12475     print(
12476         json.dumps(
12477             {
12478                 k: payload[k]
12479                 for k in (
12480                     "elapsed_seconds",
12481                     "initial_soc",
12482                     "all_physical_pass",
12483                     "n_runs",
12484                     "git_commit",
12485                     "dependency_versions",
12486                 )
12487             },
12488             indent=2,
12489         )
12490     )
12491     cols = [
12492         "date",
12493         "strategy",
12494         "pv_mapping_mode",
12495         "settlement_mode",
12496         "total_cost_yuan",
12497         "delta_total_vs_baseline_yuan",
12498     ]
12499     print(result["comparison"][cols].to_string(index=False))
12500
12501
12502 if __name__ == "__main__":

```

```

12503     main()
12504
12505     # ===== q3/run_q3_terminal_soc_pilot.py =====
12506     #!/usr/bin/env python3
12507     from __future__ import annotations
12508
12509     import json
12510     import subprocess
12511     import sys
12512     from time import perf_counter
12513
12514     ROOT = __import__("pathlib").Path(__file__).resolve().parents[1]
12515     if str(ROOT) not in sys.path:
12516         sys.path.insert(0, str(ROOT))
12517
12518     from q3.config import ATTACH1, ATTACH2, ATTACH3, TERMINAL_SOC_OUTPUT_DIR # noqa: E402
12519     from q3.data import load_q3_data # noqa: E402
12520     from q3.terminal_soc_pilot import run_q3_terminal_soc_pilot # noqa: E402
12521
12522
12523     def _git_hash() -> str:
12524         try:
12525             return subprocess.check_output(
12526                 ["git", "rev-parse", "HEAD"], cwd=ROOT, text=True
12527             ).strip()
12528         except (OSError, subprocess.CalledProcessError):
12529             return "unknown"
12530
12531
12532     def main() -> None:
12533         started = perf_counter()
12534         data = load_q3_data()
12535         result = run_q3_terminal_soc_pilot(data)
12536         elapsed = perf_counter() - started
12537         payload = {
12538             "attach1": str(ATTACH1),
12539             "attach2": str(ATTACH2),
12540             "attach3": str(ATTACH3),
12541             "output_dir": str(TERMINAL_SOC_OUTPUT_DIR),
12542             "elapsed_seconds": elapsed,
12543             "n_runs": result["n_runs"],
12544             "all_physical_pass": result["audit"]["all_pass"],
12545             "load_treatment": "causal_load_main",
12546             "pv_mapping_mode": "linear_anchor_main",
12547             "settlement_mode": "anchor_final_main",
12548             "isolated_dec01_soc_kwh": 6000.0,
12549             "boundaries": {"A_q2_aligned": 1200.0, "B_energy_neutral": 6000.0},
12550             "git_commit": _git_hash(),
12551             "solver": "HIGHS",
12552             "dependency_versions": {
12553                 "python": sys.version.split()[0],
12554                 "numpy": __import__("numpy").__version__,
12555                 "pandas": __import__("pandas").__version__,
12556                 "cvxpy": __import__("cvxpy").__version__,
12557                 "matplotlib": __import__("matplotlib").__version__,
12558             },
12559             "run_command": ".venv/bin/python q3/run_q3_terminal_soc_pilot.py",
12560             "note": (
12561                 "December A/B terminal-SOC decision trial only. "
12562                 "Does not choose the official year-end boundary and does not write result3.xlsx."
12563             ),
12564             "totals": result["totals"].to_dict(orient="records"),
12565             "delta": result["delta"].to_dict(orient="records"),
12566         }
12567         TERMINAL_SOC_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
12568         (TERMINAL_SOC_OUTPUT_DIR / "run_meta.json").write_text(
12569             json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n",
12570             encoding="utf-8",

```

```

12571     )
12572     print(
12573         json.dumps(
12574             {
12575                 "elapsed_seconds": elapsed,
12576                 "n_runs": result["n_runs"],
12577                 "all_physical_pass": result["audit"]["all_pass"],
12578                 "git_commit": payload["git_commit"],
12579             },
12580             indent=2,
12581         )
12582     )
12583     print(result["totals"].to_string(index=False))
12584     print(result["delta"].to_string(index=False))
12585
12586
12587 if __name__ == "__main__":
12588     main()
12589
12590 # ===== q3/sensitivity.py =====
12591 """Two-day RQ3-2 / RQ3-4 / RQ3-5 sensitivity trial. Does not write q3_pilot/."""
12592
12593 from __future__ import annotations
12594
12595 import json
12596
12597 import matplotlib.pyplot as plt
12598 import numpy as np
12599 import pandas as pd
12600
12601 from q3.config import (
12602     E_INITIAL_KWH,
12603     LOAD_INFORMATION_MAIN,
12604     OUTPUT_DIR,
12605     PILOT_DATES,
12606     PV_MAPPING_LINEAR,
12607     PV_MAPPING_MODES,
12608     PV_MAPPING_STEP,
12609     SENSITIVITY_FIG_DIR,
12610     SENSITIVITY_OUTPUT_DIR,
12611     SETTLEMENT_MAIN,
12612     SETTLEMENT_MODES,
12613     STRATEGIES,
12614     WARMUP_CSV_MAIN,
12615     dispatch_stem,
12616 )
12617 from q3.data import Q3Data, write_input_audit
12618 from q3.forecast import write_forecast_mapping_modes
12619 from q3.pilot import (
12620     NAVY,
12621     ORANGE,
12622     PURPLE,
12623     TEAL,
12624     DayRun,
12625     ValueCutCache,
12626     physical_audit,
12627     run_day,
12628 )
12629
12630 plt.rcParams.update(
12631     {
12632         "font.sans-serif": ["Noto Sans CJK SC", "DejaVu Sans"],
12633         "axes.unicode_minus": False,
12634         "figure.dpi": 140,
12635     }
12636 )
12637
12638 COMBOS = tuple((pv, settle) for pv in PV_MAPPING_MODES for settle in SETTLEMENT_MODES)

```

```

12639 BASELINE = (PV_MAPPING_LINEAR, SETTLEMENT_MAIN)
12640
12641
12642 def load_causal_warmup_soc(data: Q3Data) -> dict[str, float]:
12643     feb_index = data.date_index("2025-02-01")
12644     warmup_path = OUTPUT_DIR / WARMUP_CSV_MAIN
12645     if not warmup_path.exists():
12646         raise FileNotFoundError(
12647             f"Need accepted causal warmup at {warmup_path} so sensitivity SOC matches the signed-off main case"
12648         )
12649     warmup = pd.read_csv(warmup_path)
12650     if len(warmup) < feb_index:
12651         raise ValueError("causal warmup file is shorter than January")
12652     return {
12653         "2025-02-01": float(warmup["soc_end_kwh"].iloc[feb_index - 1]),
12654         "2025-06-21": E_INITIAL_KWH,
12655     }
12656
12657
12658 def plot_sensitivity_figures(comparison: pd.DataFrame, mapping: pd.DataFrame) -> None:
12659     SENSITIVITY_FIG_DIR.mkdir(parents=True, exist_ok=True)
12660     labels = [f"{pv.split('_')[0]}\n+ {settle.split('_')[0]}" for pv, settle in COMBOS]
12661     fig, axes = plt.subplots(1, 2, figsize=(10.2, 4.4), sharey=False)
12662     x = np.arange(len(STRATEGIES))
12663     width = 0.18
12664     colors = (NAVY, TEAL, ORANGE, PURPLE)
12665     for ax, date in zip(axes, PILOT_DATES):
12666         for i, (pv, settle) in enumerate(COMBOS):
12667             subset = comparison[
12668                 (comparison["date"] == date)
12669                 & (comparison["pv_mapping_mode"] == pv)
12670                 & (comparison["settlement_mode"] == settle)
12671             ].set_index("strategy").loc[list(STRATEGIES)]
12672             ax.bar(
12673                 x + (i - 1.5) * width,
12674                 subset["total_cost_yuan"],
12675                 width=width,
12676                 color=colors[i],
12677                 label=f"{pv} / {settle}" if date == PILOT_DATES[0] else None,
12678             )
12679             ax.set_xticks(x)
12680             ax.set_xticklabels(STRATEGIES, rotation=20, ha="right")
12681             ax.set_title(date)
12682             ax.spines["top"].set_visible(False)
12683             ax.spines["right"].set_visible(False)
12684             axes[0].set_ylabel("total purchase cost (yuan)")
12685             fig.legend(frameon=False, loc="upper center", ncol=2, fontsize=8, bbox_to_anchor=(0.5, 1.12))
12686             fig.suptitle("Q3 sensitivity: PV mapping × settlement (causal_load_main)", y=1.18, fontsize=12)
12687             fig.tight_layout()
12688             fig.savefig(SENSITIVITY_FIG_DIR / "fig3_sensitivity_four_combo_cost.png", bbox_inches="tight")
12689             plt.close(fig)
12690
12691     # 6:00-7:00 zoom for both mapping modes on both dates (issue 6:00, periods 36-41).
12692     fig, axes = plt.subplots(2, 1, figsize=(8.4, 6.2), sharex=True)
12693     clocks = ["06:10", "06:20", "06:30", "06:40", "06:50", "07:00"]
12694     for ax, date in zip(axes, PILOT_DATES):
12695         for mode, color, ls in (
12696             (PV_MAPPING_LINEAR, NAVY, "-"),
12697             (PV_MAPPING_STEP, ORANGE, "--"),
12698         ):
12699             block = mapping[
12700                 (mapping["date"] == date)
12701                 & (mapping["update_time"] == "06:00")
12702                 & (mapping["pv_mapping_mode"] == mode)
12703                 & (mapping["period"].between(36, 41))
12704             ].sort_values("period")
12705             ax.plot(
12706                 clocks,

```

```

12707         block["forecast_kwh"],
12708         color=color,
12709         ls=ls,
12710         marker="o",
12711         lw=1.4,
12712         label=mode,
12713     )
12714     actual = mapping[
12715         (mapping["date"] == date)
12716         & (mapping["update_time"] == "06:00")
12717         & (mapping["pv_mapping_mode"] == PV_MAPPING_LINEAR)
12718         & (mapping["period"].between(36, 41))
12719     ].sort_values("period")
12720     ax.plot(clocks, actual["actual_pv_kwh"], color=TEAL, ls=":", marker="x", lw=1.1, label="actual PV (not a forecast)")
12721     ax.set_ylabel("kWh / 10 min")
12722     ax.set_title(f"{date} 6:00 issue, first forecast hour")
12723     ax.legend(frameon=False, fontsize=8)
12724     ax.spines["top"].set_visible(False)
12725     ax.spines["right"].set_visible(False)
12726     axes[1].set_xlabel("period end")
12727     fig.suptitle("PV mapping zoom: linear_anchor_main vs step_hourly_sensitivity", fontsize=12)
12728     fig.tight_layout()
12729     fig.savefig(SENSITIVITY_FIG_DIR / "fig3_sensitivity_mapping_6h_zoom.png")
12730     plt.close(fig)
12731
12732
12733 def run_q3_sensitivity(data: Q3Data) -> dict:
12734     out = SENSITIVITY_OUTPUT_DIR
12735     out.mkdir(parents=True, exist_ok=True)
12736     SENSITIVITY_FIG_DIR.mkdir(parents=True, exist_ok=True)
12737     write_input_audit(data, out / "input_audit.json")
12738     mapping = write_forecast_mapping_modes(data, out / "q3_forecast_mapping.csv")
12739
12740     initial = load_causal_warmup_soc(data)
12741     cache: ValueCutCache = {}
12742     runs: list[DayRun] = []
12743     for date in PILOT_DATES:
12744         i = data.date_index(date)
12745         soc0 = initial[date]
12746         for pv_mode, settle_mode in COMBOS:
12747             for strategy in STRATEGIES:
12748                 runs.append(
12749                     run_day(
12750                         data,
12751                         i,
12752                         strategy,
12753                         soc0,
12754                         with_terminal_value=True,
12755                         load_information_case=LOAD_INFORMATION_MAIN,
12756                         pv_mapping_mode=pv_mode,
12757                         settlement_mode=settle_mode,
12758                         value_cut_cache=cache,
12759                     )
12760                 )
12761
12762     comparison = pd.DataFrame([run.summary for run in runs])
12763     baseline = comparison[
12764         (comparison["pv_mapping_mode"] == BASELINE[0])
12765         & (comparison["settlement_mode"] == BASELINE[1])
12766     ][["date", "strategy", "total_cost_yuan", "settlement_cost_yuan", "emergency_cost_yuan"]].rename(
12767         columns={
12768             "total_cost_yuan": "baseline_total_yuan",
12769             "settlement_cost_yuan": "baseline_settlement_yuan",
12770             "emergency_cost_yuan": "baseline_emergency_yuan",
12771         }
12772     )
12773     comparison = comparison.merge(baseline, on=["date", "strategy"], how="left")
12774     comparison["delta_total_vs_baseline_yuan"] = (

```



```

12775         comparison["total_cost_yuan"] - comparison["baseline_total_yuan"]
12776     )
12777     comparison["delta_settlement_vs_baseline_yuan"] = (
12778         comparison["settlement_cost_yuan"] - comparison["baseline_settlement_yuan"]
12779     )
12780     comparison["delta_emergency_vs_baseline_yuan"] = (
12781         comparison["emergency_cost_yuan"] - comparison["baseline_emergency_yuan"]
12782     )
12783     comparison.to_csv(out / "q3_sensitivity_comparison.csv", index=False)
12784     pd.concat([run.update_log for run in runs], ignore_index=True).to_csv(
12785         out / "q3_update_log.csv", index=False
12786     )
12787     pd.concat([run.ledger for run in runs], ignore_index=True).to_csv(
12788         out / "q3_settlement_ledger.csv", index=False
12789     )
12790     value_rows = []
12791     for key in sorted(cache):
12792         value_rows.extend(cache[key][1])
12793     if value_rows:
12794         pd.DataFrame(value_rows).to_csv(out / "q3_next_day_value_audit.csv", index=False)
12795
12796     dispatch_cols = [
12797         "time",
12798         "planned_g0_kwh",
12799         "final_g_kwh",
12800         "normal_x_kwh",
12801         "load_kwh",
12802         "forecast_load_kwh",
12803         "actual_pv_kwh",
12804         "forecast_pv_kwh",
12805         "charge_kwh",
12806         "discharge_kwh",
12807         "curtailment_kwh",
12808         "emergency_kwh",
12809         "soc_kwh",
12810         "balance_residual_kwh",
12811         "last_update_time",
12812         "price",
12813         "phi_yuan",
12814         "emergency_cost_yuan",
12815         "run_id",
12816         "pv_mapping_mode",
12817         "settlement_mode",
12818     ]
12819     for run in runs:
12820         stem = dispatch_stem(
12821             run.date,
12822             run.strategy,
12823             run.load_information_case,
12824             run.pv_mapping_mode,
12825             run.settlement_mode,
12826             run.with_terminal_value,
12827         )
12828         run.dispatch[dispatch_cols].to_csv(out / f"{stem}.csv", index=False)
12829
12830     plot_sensitivity_figures(comparison, mapping)
12831     audit = physical_audit(runs, initial["2025-02-01"])
12832     (out / "q3_physical_audit.json").write_text(
12833         json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
12834     )
12835     return {
12836         "initial_soc": initial,
12837         "comparison": comparison,
12838         "audit": audit,
12839         "n_runs": len(runs),
12840         "output_dir": str(out),
12841     }
12842

```

```

12843 # ===== q3/terminal_soc_pilot.py =====
12844 """December A/B year-end SOC pilot. Does not write q3_pilot/ or q3_sensitivity/."""
12845
12846 from __future__ import annotations
12847
12848 import json
12849 from collections import defaultdict
12850
12851 import matplotlib.pyplot as plt
12852 import numpy as np
12853 import pandas as pd
12854
12855 from q3.config import (
12856     E_INITIAL_KWH,
12857     E_MAX_KWH,
12858     E_MIN_KWH,
12859     LOAD_INFORMATION_MAIN,
12860     PV_MAPPING_LINEAR,
12861     SETTLEMENT_MAIN,
12862     SIMULTANEOUS_CD_TOL,
12863     STRATEGIES,
12864     T,
12865     TERMINAL_SOC_FIG_DIR,
12866     TERMINAL_SOC_OUTPUT_DIR,
12867     TERMINAL_SOC_PILOT_END,
12868     TERMINAL_SOC_PILOT_START,
12869     YEAR_END_BOUNDARIES,
12870     YEAR_END_BOUNDARY_A,
12871     YEAR_END_BOUNDARY_B,
12872     dispatch_stem,
12873 )
12874 from q3.data import Q3Data, write_input_audit
12875 from q3.forecast import load_forecast_audit_rows, mapping_rows_for_day
12876 from q3.pilot import (
12877     NAVY,
12878     ORANGE,
12879     PURPLE,
12880     TEAL,
12881     DayRun,
12882     ValueCutCache,
12883     physical_audit,
12884     run_day,
12885 )
12886
12887 plt.rcParams.update(
12888     {
12889         "font.sans-serif": ["Noto Sans CJK SC", "DejaVu Sans"],
12890         "axes.unicode_minus": False,
12891         "figure.dpi": 140,
12892     }
12893 )
12894
12895 HIGHLIGHT_DATES = ("2025-12-29", "2025-12-30", "2025-12-31")
12896 DISPATCH_COLS = [
12897     "time",
12898     "planned_g0_kwh",
12899     "final_g_kwh",
12900     "normal_x_kwh",
12901     "load_kwh",
12902     "forecast_load_kwh",
12903     "actual_pv_kwh",
12904     "forecast_pv_kwh",
12905     "charge_kwh",
12906     "discharge_kwh",
12907     "curtailment_kwh",
12908     "emergency_kwh",
12909     "soc_kwh",
12910     "balance_residual_kwh",

```

```

12911     "last_update_time",
12912     "price",
12913     "phi_yuan",
12914     "emergency_cost_yuan",
12915     "run_id",
12916     "pv_mapping_mode",
12917     "settlement_mode",
12918     "year_end_soc_kwh",
12919     "year_end_boundary",
12920 ]
12921
12922
12923 def terminal_soc_pilot_dates(data: Q3Data) -> list[str]:
12924     start = pd.Timestamp(TERMINAL_SOC_PILOT_START)
12925     end = pd.Timestamp(TERMINAL_SOC_PILOT_END)
12926     dates = [ts.strftime("%Y-%m-%d") for ts in data.dates if start <= ts <= end]
12927     if not dates:
12928         raise ValueError("no December dates in Attachment 2")
12929     if dates[0] != TERMINAL_SOC_PILOT_START or dates[-1] != TERMINAL_SOC_PILOT_END:
12930         raise ValueError(f"expected {TERMINAL_SOC_PILOT_START}..{TERMINAL_SOC_PILOT_END}, got {dates[0]}..{dates[-1]}")
12931     return dates
12932
12933
12934 def write_december_forecast_mapping(data: Q3Data, dates: list[str], path) -> pd.DataFrame:
12935     rows: list[dict] = []
12936     for date in dates:
12937         rows.extend(mapping_rows_for_day(data, data.date_index(date), PV_MAPPING_LINEAR))
12938     frame = pd.DataFrame(rows)
12939     path.parent.mkdir(parents=True, exist_ok=True)
12940     frame.to_csv(path, index=False)
12941     return frame
12942
12943
12944 def write_december_load_forecast_audit(data: Q3Data, dates: list[str], path) -> pd.DataFrame:
12945     rows: list[dict] = []
12946     for date in dates:
12947         rows.extend(load_forecast_audit_rows(data, data.date_index(date)))
12948     frame = pd.DataFrame(rows)
12949     path.parent.mkdir(parents=True, exist_ok=True)
12950     frame.to_csv(path, index=False)
12951     return frame
12952
12953
12954 def _path_key(strategy: str, boundary: str) -> tuple[str, str]:
12955     return (strategy, boundary)
12956
12957
12958 def strategy_totals(daily: pd.DataFrame) -> pd.DataFrame:
12959     ordered = daily.sort_values(["year_end_boundary", "strategy", "date"])
12960     rows = []
12961     for (strategy, boundary), block in ordered.groupby(["strategy", "year_end_boundary"], sort=False):
12962         rows.append(
12963             {
12964                 "strategy": strategy,
12965                 "year_end_boundary": boundary,
12966                 "year_end_soc_kwh": float(block["year_end_soc_kwh"].iloc[0]),
12967                 "load_information_case": LOAD_INFORMATION_MAIN,
12968                 "pv_mapping_mode": PV_MAPPING_LINEAR,
12969                 "settlement_mode": SETTLEMENT_MAIN,
12970                 "with_terminal_value": True,
12971                 "n_days": int(block["date"].nunique()),
12972                 "total_cost_yuan": float(block["total_cost_yuan"].sum()),
12973                 "settlement_cost_yuan": float(block["settlement_cost_yuan"].sum()),
12974                 "emergency_cost_yuan": float(block["emergency_cost_yuan"].sum()),
12975                 "emergency_kwh": float(block["emergency_kwh"].sum()),
12976                 "curtailment_kwh": float(block["curtailment_kwh"].sum()),
12977                 "charge_kwh": float(block["charge_kwh"].sum()),
12978                 "discharge_kwh": float(block["discharge_kwh"].sum()),

```

```

12979         "adjustment_count": int(block["adjustment_count"].sum()),
12980         "up_adjust_kwh": float(block["up_adjust_kwh"].sum()),
12981         "down_adjust_kwh": float(block["down_adjust_kwh"].sum()),
12982         "soc_start_dec01_kwh": float(block["soc_start_kwh"].iloc[0]),
12983         "soc_end_dec31_kwh": float(block["soc_end_kwh"].iloc[-1]),
12984         "max_balance_residual_kwh": float(block["max_balance_residual_kwh"].max()),
12985         "max_soc_residual_kwh": float(block["max_soc_residual_kwh"].max()),
12986         "max_simultaneous_cd_kwh2": float(block["max_simultaneous_cd_kwh2"].max()),
12987         "locked_period_violations": int(block["locked_period_violations"].sum()),
12988         "runtime_seconds": float(block["runtime_seconds"].sum()),
12989     }
12990 )
12991 totals = pd.DataFrame(rows)
12992 totals["cost_rank_within_boundary"] = totals.groupby("year_end_boundary")["total_cost_yuan"].rank(
12993     method="min"
12994 )
12995 return totals
12996
12997
12998 def boundary_delta(totals: pd.DataFrame) -> pd.DataFrame:
12999     a = totals[totals["year_end_boundary"] == YEAR_END_BOUNDARY_A].set_index("strategy")
13000     b = totals[totals["year_end_boundary"] == YEAR_END_BOUNDARY_B].set_index("strategy")
13001     rows = []
13002     for strategy in STRATEGIES:
13003         rows.append(
13004             {
13005                 "strategy": strategy,
13006                 "delta_cost_B_minus_A_yuan": float(b.loc[strategy, "total_cost_yuan"] - a.loc[strategy, "total_cost_yuan"]),
13007                 "delta_emergency_cost_B_minus_A_yuan": float(
13008                     b.loc[strategy, "emergency_cost_yuan"] - a.loc[strategy, "emergency_cost_yuan"]
13009                 ),
13010                 "delta_emergency_kwh_B_minus_A": float(
13011                     b.loc[strategy, "emergency_kwh"] - a.loc[strategy, "emergency_kwh"]
13012                 ),
13013                 "delta_curtailment_kwh_B_minus_A": float(
13014                     b.loc[strategy, "curtailment_kwh"] - a.loc[strategy, "curtailment_kwh"]
13015                 ),
13016                 "delta_charge_kwh_B_minus_A": float(b.loc[strategy, "charge_kwh"] - a.loc[strategy, "charge_kwh"]),
13017                 "delta_discharge_kwh_B_minus_A": float(
13018                     b.loc[strategy, "discharge_kwh"] - a.loc[strategy, "discharge_kwh"]
13019                 ),
13020                 "rank_A": int(a.loc[strategy, "cost_rank_within_boundary"]),
13021                 "rank_B": int(b.loc[strategy, "cost_rank_within_boundary"]),
13022                 "rank_changed": bool(
13023                     int(a.loc[strategy, "cost_rank_within_boundary"])
13024                     != int(b.loc[strategy, "cost_rank_within_boundary"])
13025                 ),
13026                 "soc_end_A_kwh": float(a.loc[strategy, "soc_end_dec31_kwh"]),
13027                 "soc_end_B_kwh": float(b.loc[strategy, "soc_end_dec31_kwh"]),
13028             }
13029         )
13030     return pd.DataFrame(rows)
13031
13032
13033 def plot_terminal_soc_figures(daily: pd.DataFrame, totals: pd.DataFrame, runs: list[DayRun]) -> None:
13034     fig_dir = TERMINAL_SOC_FIG_DIR
13035     fig_dir.mkdir(parents=True, exist_ok=True)
13036     x = np.arange(len(STRATEGIES))
13037     width = 0.36
13038     fig, ax = plt.subplots(figsize=(8.4, 4.4))
13039     for i, (boundary, color) in enumerate(((YEAR_END_BOUNDARY_A, NAVY), (YEAR_END_BOUNDARY_B, TEAL))):
13040         subset = totals[totals["year_end_boundary"] == boundary].set_index("strategy").loc[list(STRATEGIES)]
13041         ax.bar(
13042             x + (i - 0.5) * width,
13043             subset["total_cost_yuan"],
13044             width=width,
13045             color=color,
13046             label=boundary,

```

```

13047     )
13048     ax.set_xticks(x)
13049     ax.set_xticklabels(STRATEGIES)
13050     ax.set_ylabel("December total purchase cost (yuan)")
13051     ax.set_title("Year-end SOC pilot: A=1200 vs B=6000 (isolated Dec 1 start=6000)")
13052     ax.legend(frameon=False)
13053     ax.spines["top"].set_visible(False)
13054     ax.spines["right"].set_visible(False)
13055     fig.tight_layout()
13056     fig.savefig(fig_dir / "fig3_terminal_soc_strategy_cost.png")
13057     plt.close(fig)
13058
13059     fig, axes = plt.subplots(2, 1, figsize=(9.2, 7.0), sharex=True)
13060     colors = {
13061         "MO": NAVY,
13062         "M1_M6": TEAL,
13063         "M6_only": ORANGE,
13064         "M12_only": PURPLE,
13065         "M18_only": "#9c6644",
13066     }
13067     dates = list(dict.fromkeys(daily["date"]))
13068     x_dates = np.arange(len(dates))
13069     highlight = [i for i, date in enumerate(dates) if date in HIGHLIGHT_DATES]
13070     xtick = list(range(0, len(dates), 2))
13071     for ax, boundary in zip(axes, (YEAR_END_BOUNDARY_A, YEAR_END_BOUNDARY_B)):
13072         for strategy in STRATEGIES:
13073             block = daily[
13074                 (daily["year_end_boundary"] == boundary) & (daily["strategy"] == strategy)
13075             ].sort_values("date")
13076             ax.plot(
13077                 x_dates,
13078                 block["soc_end_kwh"],
13079                 color=colors[strategy],
13080                 lw=1.3,
13081                 marker="o",
13082                 ms=3.0,
13083                 label=strategy,
13084             )
13085         if highlight:
13086             ax.axvspan(highlight[0] - 0.4, highlight[-1] + 0.4, color="#f4e1c1", alpha=0.55, zorder=0)
13087             ax.axhline(E_MIN_KWH, color=ORANGE, ls="--", lw=0.8)
13088             ax.axhline(E_INITIAL_KWH, color="#888888", ls=":", lw=0.8)
13089             ax.set_ylabel("end-of-day SOC (kWh)")
13090             ax.set_title(boundary)
13091             ax.legend(frameon=False, ncol=5, fontsize=8)
13092             ax.spines["top"].set_visible(False)
13093             ax.spines["right"].set_visible(False)
13094             ax.set_xticks(xtick)
13095             ax.set_xticklabels([dates[i][5:] for i in xtick], rotation=60)
13096             axes[1].set_xlabel("date")
13097             fig.suptitle("Daily end SOC; shaded = Dec 29-31 (year-end constraint active)", fontsize=12)
13098             fig.tight_layout()
13099             fig.savefig(fig_dir / "fig3_terminal_soc_daily_soc.png")
13100             plt.close(fig)
13101
13102     fig, axes = plt.subplots(2, 1, figsize=(9.2, 7.0), sharex=True)
13103     for ax, (column, ylabel) in zip(
13104         axes,
13105         (("total_cost_yuan", "daily cost (yuan)", ("emergency_kwh", "emergency energy (kWh)")),
13106     ):
13107         for strategy in STRATEGIES:
13108             for boundary, ls in ((YEAR_END_BOUNDARY_A, "-"), (YEAR_END_BOUNDARY_B, "--")):
13109                 block = daily[
13110                     (daily["year_end_boundary"] == boundary) & (daily["strategy"] == strategy)
13111                 ].sort_values("date")
13112                 ax.plot(
13113                     x_dates,
13114                     block[column],

```

```

13115         color=colors[strategy],
13116         ls=ls,
13117         lw=1.15,
13118         label=f"{strategy} / {boundary.split('_')[0]}",
13119     )
13120     if highlight:
13121         ax.axvspan(highlight[0] - 0.4, highlight[-1] + 0.4, color="#f4e1c1", alpha=0.55, zorder=0)
13122     ax.set_ylabel(ylabel)
13123     ax.spines["top"].set_visible(False)
13124     ax.spines["right"].set_visible(False)
13125     ax.set_xticks(xtick)
13126     ax.set_xticklabels([dates[i][5:] for i in xtick], rotation=60)
13127     axes[0].legend(frameon=False, ncol=2, fontsize=7)
13128     axes[1].set_xlabel("date")
13129     fig.suptitle("Daily cost and emergency energy; shaded = Dec 29-31", fontsize=12)
13130     fig.tight_layout()
13131     fig.savefig(fig_dir / "fig3_terminal_soc_daily_cost_emergency.png")
13132     plt.close(fig)
13133
13134     fig, ax = plt.subplots(figsize=(8.6, 4.2))
13135     t = np.arange(T)
13136     for run in runs:
13137         if run.date != "2025-12-31" or run.strategy != "M1_M6":
13138             continue
13139         color = NAVY if run.summary["year_end_boundary"] == YEAR_END_BOUNDARY_A else TEAL
13140         ax.plot(
13141             t,
13142             run.dispatch["soc_kwh"],
13143             color=color,
13144             lw=1.4,
13145             label=f"{run.summary['year_end_boundary']} target={run.year_end_soc_kwh:.0f}",
13146         )
13147     ax.axhline(E_MIN_KWH, color=ORANGE, ls="--", lw=0.8)
13148     ax.axhline(E_MAX_KWH, color=ORANGE, ls="--", lw=0.8)
13149     ax.set_xlabel("10-minute index")
13150     ax.set_ylabel("SOC (kWh)")
13151     ax.set_title("Dec 31 M1/M6 SOC path under A vs B terminal targets")
13152     ax.legend(frameon=False)
13153     ax.spines["top"].set_visible(False)
13154     ax.spines["right"].set_visible(False)
13155     fig.tight_layout()
13156     fig.savefig(fig_dir / "fig3_terminal_soc_dec31_M1_M6.png")
13157     plt.close(fig)
13158
13159
13160 def continuity_audit(daily: pd.DataFrame) -> dict:
13161     items = []
13162     all_pass = True
13163     ordered = daily.sort_values(["year_end_boundary", "strategy", "date"])
13164     for (strategy, boundary), block in ordered.groupby(["strategy", "year_end_boundary"], sort=False):
13165         starts = block["soc_start_kwh"].to_numpy()
13166         ends = block["soc_end_kwh"].to_numpy()
13167         dates = block["date"].to_numpy()
13168         max_gap = 0.0
13169         if len(block) > 1:
13170             max_gap = float(np.max(np.abs(starts[1:] - ends[:-1])))
13171         start_ok = abs(float(starts[0]) - E_INITIAL_KWH) <= 1e-5
13172         end_target = float(block["year_end_soc_kwh"].iloc[0])
13173         end_ok = abs(float(ends[-1]) - end_target) <= 1e-5
13174         n_ok = len(block) == 31 and dates[0] == TERMINAL_SOC_PILOT_START and dates[-1] == TERMINAL_SOC_PILOT_END
13175         rec = {
13176             "strategy": strategy,
13177             "year_end_boundary": boundary,
13178             "n_days": int(len(block)),
13179             "start_soc_kwh": float(starts[0]),
13180             "end_soc_kwh": float(ends[-1]),
13181             "year_end_target_kwh": end_target,
13182             "max_cross_day_soc_gap_kwh": max_gap,

```

```

13183         "pass": bool(start_ok and end_ok and n_ok and max_gap <= 1e-5),
13184     }
13185     all_pass = all_pass and rec["pass"]
13186     items.append(rec)
13187     return {"all_pass": all_pass, "paths": items}
13188
13189
13190 def run_q3_terminal_soc_pilot(data: Q3Data) -> dict:
13191     out = TERMINAL_SOC_OUTPUT_DIR
13192     out.mkdir(parents=True, exist_ok=True)
13193     dispatch_dir = out / "dispatch_daily"
13194     dispatch_dir.mkdir(parents=True, exist_ok=True)
13195     TERMINAL_SOC_FIG_DIR.mkdir(parents=True, exist_ok=True)
13196     write_input_audit(data, out / "input_audit.json")
13197
13198     dates = terminal_soc_pilot_dates(data)
13199     write_december_forecast_mapping(data, dates, out / "q3_forecast_mapping.csv")
13200     write_december_load_forecast_audit(data, dates, out / "q3_load_forecast_audit.csv")
13201
13202     cache: ValueCutCache = {}
13203     runs: list[DayRun] = []
13204     last_end: dict[tuple[str, str], float] = defaultdict(lambda: E_INITIAL_KWH)
13205     n_expected = len(YEAR_END_BOUNDARIES) * len(STRATEGIES) * len(dates)
13206     done = 0
13207     for date in dates:
13208         day_index = data.date_index(date)
13209         for strategy in STRATEGIES:
13210             for boundary, target in YEAR_END_BOUNDARIES:
13211                 key = _path_key(strategy, boundary)
13212                 soc0 = E_INITIAL_KWH if date == TERMINAL_SOC_PILOT_START else last_end[key]
13213                 run = run_day(
13214                     data,
13215                     day_index,
13216                     strategy,
13217                     soc0,
13218                     with_terminal_value=True,
13219                     load_information_case=LOAD_INFORMATION_MAIN,
13220                     pv_mapping_mode=PV_MAPPING_LINEAR,
13221                     settlement_mode=SETTLEMENT_MAIN,
13222                     value_cut_cache=cache,
13223                     year_end_soc_kwh=target,
13224                 )
13225                 last_end[key] = float(run.summary["soc_end_kwh"])
13226                 runs.append(run)
13227                 stem = dispatch_stem(
13228                     run.date,
13229                     run.strategy,
13230                     run.load_information_case,
13231                     run.pv_mapping_mode,
13232                     run.settlement_mode,
13233                     run.with_terminal_value,
13234                     run.year_end_soc_kwh,
13235                 )
13236                 run.dispatch[DISPATCH_COLS].to_csv(dispatch_dir / f"{stem}.csv", index=False)
13237                 done += 1
13238                 print(f"[{done}/{n_expected}] {run.run_id} cost={run.summary['total_cost_yuan']:.2f}", flush=True)
13239
13240     daily = pd.DataFrame([run.summary for run in runs])
13241     daily.to_csv(out / "q3_terminal_soc_daily.csv", index=False)
13242     totals = strategy_totals(daily)
13243     totals.to_csv(out / "q3_terminal_soc_strategy_totals.csv", index=False)
13244     delta = boundary_delta(totals)
13245     delta.to_csv(out / "q3_terminal_soc_boundary_delta.csv", index=False)
13246     pd.concat([run.update_log for run in runs], ignore_index=True).to_csv(
13247         out / "q3_update_log.csv", index=False
13248     )
13249     pd.concat([run.ledger for run in runs], ignore_index=True).to_csv(
13250         out / "q3_settlement_ledger.csv", index=False

```

```

13251     )
13252     value_rows = []
13253     for key in sorted(cache):
13254         value_rows.extend(cache[key][1])
13255     if value_rows:
13256         pd.DataFrame(value_rows).to_csv(out / "q3_next_day_value_audit.csv", index=False)
13257
13258     plot_terminal_soc_figures(daily, totals, runs)
13259     phys = physical_audit(runs, None)
13260     cont = continuity_audit(daily)
13261     audit = {
13262         "all_pass": bool(phys.get("all_pass") and cont.get("all_pass")),
13263         "physical": phys,
13264         "continuity": cont,
13265         "n_day_runs": len(runs),
13266         "n_paths": len(YEAR_END_BOUNDARIES) * len(STRATEGIES),
13267         "start_date": TERMINAL_SOC_PILOT_START,
13268         "end_date": TERMINAL_SOC_PILOT_END,
13269         "isolated_dec01_soc_kwh": E_INITIAL_KWH,
13270         "note": (
13271             "Dec 1 start SOC is an isolated 6000 kWh boundary, not the true annual Dec 1 SOC. "
13272             "This pilot does not choose A vs B."
13273         ),
13274     }
13275     (out / "q3_physical_audit.json").write_text(
13276         json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
13277     )
13278     return {
13279         "n_runs": len(runs),
13280         "daily": daily,
13281         "totals": totals,
13282         "delta": delta,
13283         "audit": audit,
13284         "output_dir": str(out),
13285     }
13286
13287 # ===== q3/validate_q3_full_annual.py =====
13288 #!/usr/bin/env python3
13289 from __future__ import annotations
13290
13291 import json
13292 import sys
13293 from pathlib import Path
13294
13295 import numpy as np
13296 import pandas as pd
13297
13298 ROOT = Path(__file__).resolve().parents[1]
13299 if str(ROOT) not in sys.path:
13300     sys.path.insert(0, str(ROOT))
13301
13302 from q3.config import (
13303     ANNUAL_END,
13304     ANNUAL_FIG_DIR,
13305     ANNUAL_OUTPUT_DIR,
13306     ANNUAL_START,
13307     E_INITIAL_KWH,
13308     LOAD_INFORMATION_MAIN,
13309     PV_MAPPING_LINEAR,
13310     SETTLEMENT_MAIN,
13311     STRATEGIES,
13312     T,
13313     YEAR_END_BOUNDARIES,
13314     YEAR_N_DAYS,
13315     dispatch_stem,
13316     make_run_id,
13317 )
13318

```



```

13319 KEY = [
13320     "date",
13321     "strategy",
13322     "load_information_case",
13323     "pv_mapping_mode",
13324     "settlement_mode",
13325     "with_terminal_value",
13326     "year_end_soc_kwh",
13327 ]
13328 REQUIRED = [
13329     "input_audit.json",
13330     "q3_annual_daily_summary.csv",
13331     "q3_annual_strategy_comparison.csv",
13332     "q3_annual_forecast_audit.csv",
13333     "q3_annual_physical_audit.json",
13334     "run_meta.json",
13335 ]
13336 REQUIRED_FIGS = [
13337     "fig3_annual_strategy_cost_decomposition.png",
13338     "fig3_annual_soc_paths.png",
13339     "fig3_annual_monthly_emergency_curtailment.png",
13340 ]
13341
13342
13343 def _fail(failures: list[str], run_id: str, field: str, msg: str) -> None:
13344     failures.append(f"{run_id} [{field}]: {msg}")
13345
13346
13347 def main() -> None:
13348     out = ANNUAL_OUTPUT_DIR
13349     missing = [name for name in REQUIRED if not (out / name).exists()]
13350     if missing:
13351         raise SystemExit(f"missing outputs: {missing}")
13352     missing_figs = [name for name in REQUIRED_FIGS if not (ANNUAL_FIG_DIR / name).exists()]
13353     if missing_figs:
13354         raise SystemExit(f"missing figures: {missing_figs}")
13355     audit = json.loads((out / "q3_annual_physical_audit.json").read_text(encoding="utf-8"))
13356     daily = pd.read_csv(out / "q3_annual_daily_summary.csv")
13357     totals = pd.read_csv(out / "q3_annual_strategy_comparison.csv")
13358     forecast = pd.read_csv(out / "q3_annual_forecast_audit.csv")
13359     meta = json.loads((out / "run_meta.json").read_text(encoding="utf-8"))
13360     delta_path = out / "q3_annual_boundary_delta.csv"
13361     delta = pd.read_csv(delta_path) if delta_path.exists() else pd.DataFrame()
13362
13363     failures: list[str] = []
13364     if not audit.get("all_pass"):
13365         failures.append("physical/continuity audit all_pass is false")
13366
13367     for col in KEY + ["run_id", "year_end_boundary"]:
13368         if col not in daily.columns:
13369             failures.append(f"daily missing {col}")
13370
13371     expected_n = YEAR_N_DAYS * len(STRATEGIES) * len(YEAR_END_BOUNDARIES)
13372     if len(daily) != expected_n:
13373         failures.append(f"expected {expected_n} daily rows, got {len(daily)}")
13374     if daily.duplicated(KEY).sum():
13375         failures.append("duplicate daily keys")
13376
13377     dispatch_dir = out / "dispatch_daily"
13378     sample = daily[(daily["date"].isin((ANNUAL_START, "2025-06-21", ANNUAL_END)))]
13379     for _, row in sample.iterrows():
13380         run_id = str(row["run_id"])
13381         expected_id = make_run_id(
13382             row["date"],
13383             row["strategy"],
13384             row["load_information_case"],
13385             row["pv_mapping_mode"],
13386             row["settlement_mode"],

```

```

13387         bool(row["with_terminal_value"]),
13388         float(row["year_end_soc_kwh"]),
13389     )
13390     if run_id != expected_id:
13391         _fail(failures, run_id, "run_id", f"mismatch {expected_id}")
13392     stem = dispatch_stem(
13393         row["date"],
13394         row["strategy"],
13395         row["load_information_case"],
13396         row["pv_mapping_mode"],
13397         row["settlement_mode"],
13398         bool(row["with_terminal_value"]),
13399         float(row["year_end_soc_kwh"]),
13400     )
13401     path = dispatch_dir / f"{stem}.csv"
13402     if path.exists():
13403         frame = pd.read_csv(path)
13404         if len(frame) != T:
13405             _fail(failures, run_id, "dispatch", f"rows {len(frame)} != {T}")
13406         if float(np.max(np.abs(frame["balance_residual_kwh"]))) >= 1e-5:
13407             _fail(failures, run_id, "balance_residual_kwh", "energy residual")
13408
13409     if (forecast["max_source_index"] >= forecast["day_index"]).any():
13410         failures.append("load forecast source is not strictly before the target day")
13411     if not bool(forecast.loc[forecast["day_index"] == 0, "used_attachment1_fallback"].all()):
13412         failures.append("Jan 1 must use attachment1 fallback")
13413     if (daily["load_information_case"] != LOAD_INFORMATION_MAIN).any():
13414         failures.append("annual run must stay on causal_load_main")
13415     if (daily["pv_mapping_mode"] != PV_MAPPING_LINEAR).any():
13416         failures.append("this annual archive is linear_anchor_main only")
13417     if (daily["settlement_mode"] != SETTLEMENT_MAIN).any():
13418         failures.append("this annual archive is anchor_final_main only")
13419
13420     for (strategy, boundary), block in daily.groupby(["strategy", "year_end_boundary"]):
13421         block = block.sort_values("date")
13422         if len(block) != YEAR_N_DAYS:
13423             failures.append(f"{strategy}/{boundary}: {len(block)} days")
13424             continue
13425         if str(block["date"].iloc[0]) != ANNUAL_START or str(block["date"].iloc[-1]) != ANNUAL_END:
13426             failures.append(f"{strategy}/{boundary}: date range")
13427         if abs(float(block["soc_start_kwh"].iloc[0]) - E_INITIAL_KWH) > 1e-5:
13428             failures.append(f"{strategy}/{boundary}: Jan 1 start SOC")
13429         target = float(block["year_end_soc_kwh"].iloc[0])
13430         if abs(float(block["soc_end_kwh"].iloc[-1]) - target) > 1e-5:
13431             failures.append(f"{strategy}/{boundary}: Dec 31 SOC {block['soc_end_kwh'].iloc[-1]} vs {target}")
13432         starts = block["soc_start_kwh"].to_numpy()
13433         ends = block["soc_end_kwh"].to_numpy()
13434         if float(np.max(np.abs(starts[1:] - ends[:-1]))) > 1e-5:
13435             failures.append(f"{strategy}/{boundary}: cross-day SOC not continuous")
13436
13437     lines = [
13438         "# Q3 全年连续 SOC (A=1200 与 B=6000 分列运行) ",
13439         "",
13440         "主口径固定 `causal_load_main` + `linear_anchor_main` + `anchor_final_main`。",
13441         "A、B 是两条彼此独立的 365 日路径，各自从 2025-01-01 的 6000 kWh 出发，不共享 SOC。",
13442         "10 路年度比较归档本身**不负责生成 `result3.xlsx`（正式表由 `q3/run_q3_result3_export.py` 在授权后单独导出），**不**实现 M5，
13443         **不**把两案平均成一个主结论。",
13444         "",
13445         "## 运行",
13446         "",
13447         "`bash",
13448         ".venv/bin/python -m unittest q3.test_q3 -v",
13449         ".venv/bin/python q3/run_q3_full_annual.py --boundary both",
13450         ".venv/bin/python q3/validate_q3_full_annual.py",
13451         "`",
13452         "",
13453         f"- 物理/连续审计全部通过: {audit.get('all_pass')}",
13454         f"- git: {meta.get('git_commit')}",

```

```

13454         f"- 求解器: {meta.get('solver')}; 依赖: {meta.get('dependency_versions')}",
13455         f"- 日运行数: {len(daily)} (2 边界 × 5 策略 × 365 日) ",
13456         "",
13457         "## 策略 × 边界汇总",
13458         "",
13459         totals.to_string(index=False),
13460         "",
13461         "## ΔC = C(6000) - C(1200)",
13462         "",
13463         (delta.to_string(index=False) if len(delta) else "(delta table missing)"),
13464         "",
13465         "两案并存供队长选择; 在选定主边界之前不得写入论文正式成本。",
13466     ]
13467     (out / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
13468     payload = {
13469         "all_pass": not failures and bool(audit.get("all_pass")),
13470         "failures": failures,
13471         "n_daily_rows": int(len(daily)),
13472         "git_commit": meta.get("git_commit"),
13473         "load_treatment": LOAD_INFORMATION_MAIN,
13474         "chooses_year_end_boundary": False,
13475     }
13476     (out / "validation.json").write_text(
13477         json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
13478     )
13479     if failures:
13480         raise SystemExit("validation failed:\n- " + "\n- ".join(failures))
13481     print(json.dumps(payload, indent=2))
13482
13483
13484 if __name__ == "__main__":
13485     main()
13486
13487 # ===== q3/validate_q3_pilot.py =====
13488 #!/usr/bin/env python3
13489 from __future__ import annotations
13490
13491 import json
13492 import sys
13493 from pathlib import Path
13494
13495 import numpy as np
13496 import pandas as pd
13497
13498 ROOT = Path(__file__).resolve().parents[1]
13499 if str(ROOT) not in sys.path:
13500     sys.path.insert(0, str(ROOT))
13501
13502 from q3.config import (
13503     LOAD_INFORMATION_MAIN,
13504     LOAD_INFORMATION_PROXY,
13505     NUMERIC_TOL,
13506     OUTPUT_DIR,
13507     PILOT_DATES,
13508     SIMULTANEOUS_CD_TOL,
13509     STRATEGIES,
13510 )
13511 from q3.optimization import settlement_cost
13512
13513
13514 REQUIRED = [
13515     "input_audit.json",
13516     "q3_forecast_mapping.csv",
13517     "q3_update_log.csv",
13518     "q3_strategy_comparison.csv",
13519     "q3_cost_audit.csv",
13520     "q3_physical_audit.json",
13521     "q3_load_forecast_audit.csv",

```

```

13522     "q3_load_information_comparison.csv",
13523     "q3_causality_audit.json",
13524 ]
13525
13526
13527 def main() -> None:
13528     missing = [name for name in REQUIRED if not (OUTPUT_DIR / name).exists()]
13529     if missing:
13530         raise SystemExit(f"missing outputs: {missing}")
13531
13532     audit = json.loads((OUTPUT_DIR / "q3_physical_audit.json").read_text(encoding="utf-8"))
13533     causality = json.loads((OUTPUT_DIR / "q3_causality_audit.json").read_text(encoding="utf-8"))
13534     comparison = pd.read_csv(OUTPUT_DIR / "q3_strategy_comparison.csv")
13535     info = pd.read_csv(OUTPUT_DIR / "q3_load_information_comparison.csv")
13536     log = pd.read_csv(OUTPUT_DIR / "q3_update_log.csv")
13537     mapping = pd.read_csv(OUTPUT_DIR / "q3_forecast_mapping.csv")
13538     cost = pd.read_csv(OUTPUT_DIR / "q3_cost_audit.csv")
13539     load_audit = pd.read_csv(OUTPUT_DIR / "q3_load_forecast_audit.csv")
13540
13541     failures: list[str] = []
13542     if not audit.get("all_pass"):
13543         failures.append("physical audit all_pass is false")
13544     if not causality.get("all_pass"):
13545         failures.append("load causality audit all_pass is false")
13546     if mapping["is_executed_actual"].isna().any():
13547         failures.append("forecast mapping missing executed flags")
13548     if "load_information_case" not in comparison.columns:
13549         failures.append("strategy comparison missing load_information_case")
13550     if "load_information_case" not in cost.columns:
13551         failures.append("cost audit missing load_information_case")
13552     if "load_information_case" not in info.columns:
13553         failures.append("load information comparison missing load_information_case")
13554
13555     unlabeled = sorted(
13556         path.name
13557         for path in OUTPUT_DIR.glob("q3_dispatch_*.csv")
13558         if LOAD_INFORMATION_MAIN not in path.name and LOAD_INFORMATION_PROXY not in path.name
13559     )
13560
13561     phi = settlement_cost(
13562         cost["price"].to_numpy(),
13563         cost["planned_g0_kwh"].to_numpy(),
13564         cost["final_g_kwh"].to_numpy(),
13565     )
13566     if float(np.max(np.abs(phi - cost["phi_yuan"].to_numpy()))) > 1e-8:
13567         failures.append("cost audit phi does not match p gF + 0.5 p |gF-g0|")
13568
13569     for date in PILOT_DATES:
13570         day_hat = load_audit[
13571             (load_audit["date"] == date)
13572             & (load_audit["load_information_case"] == LOAD_INFORMATION_MAIN)
13573         ]
13574         if len(day_hat) != 144:
13575             failures.append(f"{date} load forecast audit should have 144 day-ahead rows")
13576         if (day_hat["source_dates"].astype(str).str.contains(date, regex=False)).any():
13577             failures.append(f"{date} causal load sources include the target date")
13578         if bool(day_hat["is_current_execution_period"].astype(bool)).any():
13579             failures.append(f"{date} day-ahead load snapshot marked as current execution")
13580
13581     for strategy in STRATEGIES:
13582         for case in (LOAD_INFORMATION_MAIN, LOAD_INFORMATION_PROXY):
13583             path = OUTPUT_DIR / f"q3_dispatch_{date}_{strategy}_{case}.csv"
13584             if not path.exists():
13585                 failures.append(f"missing {path.name}")
13586                 continue
13587             if case != LOAD_INFORMATION_MAIN:
13588                 continue
13589             frame = pd.read_csv(path)

```

```

13590         if "forecast_load_kwh" not in frame.columns:
13591             failures.append(f"{path.name} missing forecast_load_kwh")
13592         if float(np.max(np.abs(frame["balance_residual_kwh"]))) >= 1e-5:
13593             failures.append(f"{path.name} energy residual")
13594         if "M0" in strategy:
13595             if not np.allclose(
13596                 frame["planned_g0_kwh"], frame["final_g_kwh"], atol=NUMERIC_TOL
13597             ):
13598                 failures.append(f"{date} {case} M0 mutated g after 0:00")
13599             locked = frame[frame["last_update_time"].eq("00:00")]
13600             if strategy != "M0" and len(locked) == 0:
13601                 failures.append(f"{date} {strategy} {case} has no 0:00-locked prefix")
13602
13603     main = comparison[
13604         comparison["load_information_case"].eq(LOAD_INFORMATION_MAIN)
13605         & comparison["with_terminal_value"]
13606     ]
13607     proxy = comparison[
13608         comparison["load_information_case"].eq(LOAD_INFORMATION_PROXY)
13609         & comparison["with_terminal_value"]
13610     ]
13611     m0 = main[main["strategy"] == "M0"]
13612     m1 = main[main["strategy"] == "M1_M6"]
13613     no48 = comparison[
13614         comparison["load_information_case"].eq(LOAD_INFORMATION_MAIN)
13615         & (comparison["strategy"] == "M1_M6")
13616         & (~comparison["with_terminal_value"])
13617     ]
13618     if len(m0) != len(PILOT_DATES) or len(m1) != len(PILOT_DATES):
13619         failures.append("main-case M0/M1_M6 rows missing or mixed with proxy")
13620
13621     lines = [
13622         "# Q3 两日试算审计（因果负荷修订）",
13623         "",
13624         "***主结果候选是 `causal_load_main`. ** `actual_load_proxy` 只衡量“若负荷被完美预知”的信息价值，",
13625         "不得进入全年正式结果，也不得与主方案成本混合后宣称可现实执行。",
13626         "",
13627         "提交给 review/队长复核。本阶段**不**生成 `result3.xlsx`，**不**实现 M5。",
13628         "",
13629         "## 运行",
13630         "",
13631         "```bash",
13632         "export CUMCM_C_ATTACH_DIR=/path/to/C题/附件",
13633         "python -m unittest q3.test_q3 -v",
13634         "python q3/run_q3_pilot.py",
13635         "python q3/validate_q3_pilot.py",
13636         "```",
13637         "",
13638         f"- 物理审计全部通过: {audit.get('all_pass')}",
13639         f"- 负荷因果性审计全部通过: {causality.get('all_pass')}",
13640         f"- 2025-02-01 日初 SOC (**causal_load_main** M0 自 1 月 1 日预热): {audit.get('warmup_end_soc_for_2025-02-01')}",
13641         "- 2025-06-21 日初 SOC: 孤立试算 6000 kWh; 两种信息情形共用该日初日末 SOC",
13642         "- 能量平衡残差按**当期真实负荷/光伏**闭合; 规划层未来时段只用 0:00 因果负荷 hat 与当次光伏预报",
13643         "- 每 10 分钟仅当前执行步使用真实负荷与真实光伏",
13644         "",
13645         "## 主比较 (causal_load_main, 含 48h 终端价值)",
13646         "",
13647     ]
13648     for date in PILOT_DATES:
13649         base = float(m0.loc[m0["date"] == date, "total_cost_yuan"].iloc[0])
13650         main_cost = float(m1.loc[m1["date"] == date, "total_cost_yuan"].iloc[0])
13651         dump = float(no48.loc[no48["date"] == date, "soc_end_kwh"].iloc[0])
13652         proxy_m1 = float(
13653             proxy.loc[
13654                 (proxy["date"] == date) & (proxy["strategy"] == "M1_M6"),
13655                 "total_cost_yuan",
13656             ].iloc[0]
13657         )

```

```

13658     proxy_m0 = float(
13659         proxy.loc[
13660             (proxy["date"] == date) & (proxy["strategy"] == "M0"),
13661             "total_cost_yuan",
13662             ].iloc[0]
13663     )
13664     lines.append(
13665         f"- **{date} 主模型: ** M0 = {base:.6f} 元, M1/M6 = {main_cost:.6f} 元, "
13666         f"差额 {main_cost - base:.6f} 元; 无 48h 对照日末 SOC = {dump:.1f} kWh。"
13667     )
13668     lines.append(
13669         f" - 理想化负荷对照 `actual_load_proxy`: M0 = {proxy_m0:.6f} 元, "
13670         f"M1/M6 = {proxy_m1:.6f} 元。主模型相对对照的 M1/M6 差额 "
13671         f"{main_cost - proxy_m1:.6f} 元 (正值表示完美负荷信息更便宜; **对照不可执行**)。"
13672     )
13673     day_log = log[
13674         (log["date"] == date)
13675         & (log["strategy"] == "M1-M6")
13676         & (log["load_information_case"] == LOAD_INFORMATION_MAIN)
13677         & (log["with_terminal_value"] == True)
13678         & (log["update_time"] != "00:00")
13679     ]
13680     if len(day_log):
13681         voi = ", ".join(
13682             f"{row.update_time} VoI={row.voi_yuan:.3f} 元"
13683             f"{' (实施) ' if row.implemented else ' (不实施) '}"
13684             for row in day_log.itertuples()
13685         )
13686         lines.append(f" - 主模型逐时点 VoI: {voi}")
13687     lines += [
13688         "",
13689         "## 信息口径",
13690         "",
13691         "- 日前负荷: 最近至多 4 个已结束同星期日均值; 历史不足时回退附件 1。核截止日严格早于目标日。",
13692         "- 6/12/18 的未来规划继续使用该日 0:00 hat, 不新增同日负荷后验更新。",
13693         "- `q3_causality_audit.json`: 扰动当天 06:00 之后尚未发生的真实负荷后, 主模型日前/更新负荷预测与 0:00 计划不变; 代理对照允许变化。",
13694         "- 修订前未标注信息情形的 `q3_dispatch_YYYY-MM-DD_<strategy>.csv` 保留作历史文件, **不是**主结果。",
13695         "",
13696         "## 主模型策略成本 (causal_load_main) ",
13697         "",
13698         main.to_string(index=False),
13699         "",
13700         "## 负荷信息对照全表",
13701         "",
13702         info.to_string(index=False),
13703         "",
13704         "数字待队长签收前不得写入论文。",
13705     ]
13706     if unlabeled:
13707         lines[lines.index("- 修订前未标注信息情形的 `q3_dispatch_YYYY-MM-DD_<strategy>.csv` 保留作历史文件, **不是**主结果。")] = (
13708             "- 修订前未标注信息情形的调度文件保留作历史对照, **不是**主结果: "
13709             + ", ".join(unlabeled)
13710         )
13711
13712     (OUTPUT_DIR / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
13713     payload = {
13714         "all_pass": not failures and bool(audit.get("all_pass")) and bool(causality.get("all_pass")),
13715         "failures": failures,
13716         "n_comparison_rows": int(len(comparison)),
13717         "n_main_comparison_rows": int(len(main)),
13718         "n_mapping_rows": int(len(mapping)),
13719         "n_unlabeled_legacy_dispatch_files": int(len(unlabeled)),
13720         "max_simultaneous_cd_tol": SIMULTANEOUS_CD_TOL,
13721         "load_treatment": LOAD_INFORMATION_MAIN,
13722     }
13723     (OUTPUT_DIR / "validation.json").write_text(
13724         json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"

```

```

13725     )
13726     if failures:
13727         raise SystemExit("validation failed:\n- " + "\n- ".join(failures))
13728     print(json.dumps(payload, indent=2))
13729
13730
13731 if __name__ == "__main__":
13732     main()
13733
13734 # ===== q3/validate_q3_sensitivity.py =====
13735 #!/usr/bin/env python3
13736 from __future__ import annotations
13737
13738 import json
13739 import sys
13740 from pathlib import Path
13741
13742 import numpy as np
13743 import pandas as pd
13744
13745 ROOT = Path(__file__).resolve().parents[1]
13746 if str(ROOT) not in sys.path:
13747     sys.path.insert(0, str(ROOT))
13748
13749 from q3.config import (
13750     LOAD_INFORMATION_MAIN,
13751     PILOT_DATES,
13752     PV_MAPPING_LINEAR,
13753     PV_MAPPING_MODES,
13754     PV_MAPPING_STEP,
13755     SENSITIVITY_OUTPUT_DIR,
13756     SETTLEMENT_ALT,
13757     SETTLEMENT_MAIN,
13758     SETTLEMENT_MODES,
13759     STRATEGIES,
13760     dispatch_stem,
13761     make_run_id,
13762 )
13763 from q3.optimization import realized_settlement
13764
13765 KEY = [
13766     "date",
13767     "strategy",
13768     "load_information_case",
13769     "pv_mapping_mode",
13770     "settlement_mode",
13771     "with_terminal_value",
13772 ]
13773 REQUIRED = [
13774     "input_audit.json",
13775     "q3_forecast_mapping.csv",
13776     "q3_update_log.csv",
13777     "q3_sensitivity_comparison.csv",
13778     "q3_settlement_ledger.csv",
13779     "q3_physical_audit.json",
13780     "run_meta.json",
13781 ]
13782
13783
13784 def _fail(failures: list[str], run_id: str, field: str, msg: str) -> None:
13785     failures.append(f"{run_id} [{field}]: {msg}")
13786
13787
13788 def main() -> None:
13789     out = SENSITIVITY_OUTPUT_DIR
13790     missing = [name for name in REQUIRED if not (out / name).exists()]
13791     if missing:
13792         raise SystemExit(f"missing outputs: {missing}")

```

```

13793
13794 audit = json.loads((out / "q3_physical_audit.json").read_text(encoding="utf-8"))
13795 comparison = pd.read_csv(out / "q3_sensitivity_comparison.csv")
13796 ledger = pd.read_csv(out / "q3_settlement_ledger.csv")
13797 mapping = pd.read_csv(out / "q3_forecast_mapping.csv")
13798 meta = json.loads((out / "run_meta.json").read_text(encoding="utf-8"))
13799 log = pd.read_csv(out / "q3_update_log.csv")
13800
13801 failures: list[str] = []
13802 if not audit.get("all_pass"):
13803     failures.append("physical audit all_pass is false")
13804 for col in KEY + ["run_id"]:
13805     if col not in comparison.columns:
13806         failures.append(f"comparison missing {col}")
13807     if col not in ledger.columns:
13808         failures.append(f"ledger missing {col}")
13809     if col not in log.columns:
13810         failures.append(f"update log missing {col}")
13811
13812 if mapping["pv_mapping_mode"].isna().any() or mapping["is_executed_actual"].isna().any():
13813     failures.append("mapping table missing mode or executed flags")
13814 dup = comparison.duplicated(KEY).sum()
13815 if dup:
13816     failures.append(f"duplicate comparison keys: {dup}")
13817
13818 expected_n = len(PILOT_DATES) * len(STRATEGIES) * len(PV_MAPPING_MODES) * len(SETTLEMENT_MODES)
13819 if len(comparison) != expected_n:
13820     failures.append(f"expected {expected_n} comparison rows, got {len(comparison)}")
13821
13822 for _, row in comparison.iterrows():
13823     run_id = str(row["run_id"])
13824     expected_id = make_run_id(
13825         row["date"],
13826         row["strategy"],
13827         row["load_information_case"],
13828         row["pv_mapping_mode"],
13829         row["settlement_mode"],
13830         bool(row["with_terminal_value"]),
13831     )
13832     if run_id != expected_id:
13833         _fail(failures, run_id, "run_id", f"mismatch {expected_id}")
13834     stem = dispatch_stem(
13835         row["date"],
13836         row["strategy"],
13837         row["load_information_case"],
13838         row["pv_mapping_mode"],
13839         row["settlement_mode"],
13840         bool(row["with_terminal_value"]),
13841     )
13842     path = out / f"{stem}.csv"
13843     if not path.exists():
13844         _fail(failures, run_id, "dispatch", f"missing {path.name}")
13845         continue
13846     if stem.split("_", 3)[-1].replace(".csv", "") and row["pv_mapping_mode"] not in path.name:
13847         _fail(failures, run_id, "filename", "pv_mapping_mode not in filename")
13848     if row["settlement_mode"] not in path.name:
13849         _fail(failures, run_id, "filename", "settlement_mode not in filename")
13850     frame = pd.read_csv(path)
13851     if frame["run_id"].nunique() != 1 or str(frame["run_id"].iloc[0]) != run_id:
13852         _fail(failures, run_id, "dispatch.run_id", "inconsistent")
13853     if float(np.max(np.abs(frame["balance_residual_kwh"]))) >= 1e-5:
13854         _fail(failures, run_id, "balance_residual_kwh", "energy residual")
13855     phi = None
13856     if row["settlement_mode"] == SETTLEMENT_MAIN:
13857         phi = realized_settlement(
13858             frame["price"].to_numpy(),
13859             [frame["planned_g0_kwh"].to_numpy(), frame["final_g_kwh"].to_numpy()],
13860             SETTLEMENT_MAIN,

```



```

13861         )
13862
13863     led = ledger[(ledger["run_id"] == run_id) & (ledger["row_role"] == "period_total")]
13864     if len(led) != 144:
13865         _fail(failures, run_id, "ledger", f"period_total rows {len(led)}")
13866         continue
13867     settle_from_led = float(led["ordinary_yuan"].sum() + led["up_fee_yuan"].sum() + led["down_fee_yuan"].sum())
13868     emerg_from_led = float(led["emergency_yuan"].sum())
13869     total_from_led = float(led["total_yuan"].sum())
13870     if abs(settle_from_led - float(row["settlement_cost_yuan"])) > 1e-6:
13871         _fail(
13872             failures,
13873             run_id,
13874             "settlement_cost_yuan",
13875             f"ledger {settle_from_led:.8f} vs summary {row['settlement_cost_yuan']:.8f}",
13876         )
13877     if abs(emerg_from_led - float(row["emergency_cost_yuan"])) > 1e-6:
13878         _fail(
13879             failures,
13880             run_id,
13881             "emergency_cost_yuan",
13882             f"ledger {emerg_from_led:.8f} vs summary {row['emergency_cost_yuan']:.8f}",
13883         )
13884     if abs(total_from_led - float(row["total_cost_yuan"])) > 1e-6:
13885         _fail(
13886             failures,
13887             run_id,
13888             "total_cost_yuan",
13889             f"ledger {total_from_led:.8f} vs summary {row['total_cost_yuan']:.8f}",
13890         )
13891     events = ledger[(ledger["run_id"] == run_id) & (ledger["row_role"] == "event")]
13892     if len(events):
13893         by_period = events.groupby("period")["total_yuan"].sum()
13894         tot = led.set_index("period")["total_yuan"]
13895         if float(np.max(np.abs(by_period.reindex(tot.index).fillna(0.0) - tot))) > 1e-6:
13896             _fail(failures, run_id, "ledger.event", "events do not sum to period_total")
13897
13898     if row["settlement_mode"] == SETTLEMENT_MAIN and "price" in frame.columns and phi is not None:
13899         if float(np.max(np.abs(phi - frame["phi_yuan"].to_numpy()))) > 1e-8:
13900             _fail(failures, run_id, "phi_yuan", "does not match  $p_{GF} + 0.5 p_{|GF-G0|}$ ")
13901
13902     if row["strategy"] == "M0" and row["settlement_mode"] == SETTLEMENT_ALT:
13903         mate = comparison[
13904             (comparison["date"] == row["date"])
13905             & (comparison["strategy"] == "M0")
13906             & (comparison["pv_mapping_mode"] == row["pv_mapping_mode"])
13907             & (comparison["settlement_mode"] == SETTLEMENT_MAIN)
13908         ]
13909         if len(mate) == 1:
13910             if abs(float(mate["total_cost_yuan"].iloc[0]) - float(row["total_cost_yuan"])) > 1e-4:
13911                 _fail(failures, run_id, "M0", "alt settlement should match main when  $g_F=g_0$ ")
13912
13913     if row["load_information_case"] != LOAD_INFORMATION_MAIN:
13914         _fail(failures, run_id, "load_information_case", "sensitivity must stay on causal_load_main")
13915
13916     # Mapping sanity: step 6:10-7:00 share one source hour.
13917     step6 = mapping[
13918         (mapping["pv_mapping_mode"] == PV_MAPPING_STEP)
13919         & (mapping["update_time"] == "06:00")
13920         & (mapping["period"].between(36, 41))
13921     ]
13922     if len(step6) and not (step6["source_forecast_hour"] == 1).all():
13923         failures.append("step mapping 06:10-07:00 is not entirely 预报1小时")
13924     lin6 = mapping[
13925         (mapping["pv_mapping_mode"] == PV_MAPPING_LINEAR)
13926         & (mapping["update_time"] == "06:00")
13927         & (mapping["period"] == 41)
13928     ]

```

```

13929 if len(lin6):
13930     # 07:00 node should match 预报1小时 * dt; left/right collapse at the knot.
13931     if not np.allclose(lin6["left_weight"] + lin6["right_weight"], 1.0):
13932         failures.append("linear 07:00 weights do not sum to 1")
13933
13934 lines = [
13935     "# Q3 映射/结算敏感性两日审计 (RQ3-2、RQ3-4、RQ3-5) ",
13936     "",
13937     "主负荷口径仍为 `causal_load_main`。本目录与已验收的 `output/q3_pilot/` 隔离。",
13938     "***不**生成 `result3.xlsx`，**不**实现 M5，**不**宣布全年策略排序。",
13939     "",
13940     "## 运行",
13941     "",
13942     "```bash",
13943     ".venv/bin/python -m unittest q3.test_q3 -v",
13944     ".venv/bin/python q3/run_q3_sensitivity.py",
13945     ".venv/bin/python q3/validate_q3_sensitivity.py",
13946     "```",
13947     "",
13948     f"- 物理审计全部通过: {audit.get('all_pass')}",
13949     f"- git: {meta.get('git_commit')}",
13950     f"- 求解器: {meta.get('solver')}; 依赖: {meta.get('dependency_versions')}",
13951     f"- 运行键数: {len(comparison)} (2 日 × 5 策略 × 2 映射 × 2 结算) ",
13952     f"- 日初 SOC: {meta.get('initial_soc')}",
13953     "",
13954     "## 相对 baseline (linear_anchor_main + anchor_final_main) 的总成本差",
13955     "",
13956 ]
13957 pivot = comparison.pivot_table(
13958     index=["date", "strategy"],
13959     columns=["pv_mapping_mode", "settlement_mode"],
13960     values="delta_total_vs_baseline_yuan",
13961 )
13962 lines.append(pivot.to_string())
13963 lines += [
13964     "",
13965     "## 对照全表",
13966     "",
13967     comparison[
13968         [
13969             "date",
13970             "strategy",
13971             "pv_mapping_mode",
13972             "settlement_mode",
13973             "total_cost_yuan",
13974             "settlement_cost_yuan",
13975             "emergency_cost_yuan",
13976             "emergency_kwh",
13977             "curtailment_kwh",
13978             "soc_start_kwh",
13979             "soc_end_kwh",
13980             "adjustment_count",
13981             "delta_total_vs_baseline_yuan",
13982         ]
13983     ].to_string(index=False),
13984     "",
13985     "数字待队长签收前不得写入论文主结论。",
13986 ]
13987 (out / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
13988 payload = {
13989     "all_pass": not failures and bool(audit.get("all_pass")),
13990     "failures": failures,
13991     "n_comparison_rows": int(len(comparison)),
13992     "n_ledger_rows": int(len(ledger)),
13993     "git_commit": meta.get("git_commit"),
13994     "load_treatment": LOAD_INFORMATION_MAIN,
13995 }
13996 (out / "validation.json").write_text(

```

```

13997         json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
13998     )
13999     if failures:
14000         raise SystemExit("validation failed:\n- " + "\n- ".join(failures))
14001     print(json.dumps(payload, indent=2))
14002
14003
14004 if __name__ == "__main__":
14005     main()
14006
14007 # ===== q3/validate_q3_terminal_soc_pilot.py =====
14008 #!/usr/bin/env python3
14009 from __future__ import annotations
14010
14011 import json
14012 import sys
14013 from pathlib import Path
14014
14015 import numpy as np
14016 import pandas as pd
14017
14018 ROOT = Path(__file__).resolve().parents[1]
14019 if str(ROOT) not in sys.path:
14020     sys.path.insert(0, str(ROOT))
14021
14022 from q3.config import (
14023     E_INITIAL_KWH,
14024     LOAD_INFORMATION_MAIN,
14025     PV_MAPPING_LINEAR,
14026     SETTLEMENT_MAIN,
14027     STRATEGIES,
14028     T,
14029     TERMINAL_SOC_FIG_DIR,
14030     TERMINAL_SOC_OUTPUT_DIR,
14031     TERMINAL_SOC_PILOT_END,
14032     TERMINAL_SOC_PILOT_START,
14033     YEAR_END_BOUNDARIES,
14034     YEAR_END_BOUNDARY_A,
14035     YEAR_END_BOUNDARY_B,
14036     dispatch_stem,
14037     make_run_id,
14038 )
14039 from q3.optimization import realized_settlement
14040
14041 KEY = [
14042     "date",
14043     "strategy",
14044     "load_information_case",
14045     "pv_mapping_mode",
14046     "settlement_mode",
14047     "with_terminal_value",
14048     "year_end_soc_kwh",
14049 ]
14050 REQUIRED = [
14051     "input_audit.json",
14052     "q3_forecast_mapping.csv",
14053     "q3_update_log.csv",
14054     "q3_terminal_soc_daily.csv",
14055     "q3_terminal_soc_strategy_totals.csv",
14056     "q3_terminal_soc_boundary_delta.csv",
14057     "q3_settlement_ledger.csv",
14058     "q3_physical_audit.json",
14059     "run_meta.json",
14060 ]
14061 REQUIRED_FIGS = [
14062     "fig3_terminal_soc_strategy_cost.png",
14063     "fig3_terminal_soc_daily_soc.png",
14064     "fig3_terminal_soc_daily_cost_emergency.png",

```

```

14065     "fig3_terminal_soc_dec31_M1_M6.png",
14066 ]
14067
14068
14069 def _fail(failures: list[str], run_id: str, field: str, msg: str) -> None:
14070     failures.append(f"{run_id} [{field}]: {msg}")
14071
14072
14073 def main() -> None:
14074     out = TERMINAL_SOC_OUTPUT_DIR
14075     missing = [name for name in REQUIRED if not (out / name).exists()]
14076     if missing:
14077         raise SystemExit(f"missing outputs: {missing}")
14078     missing_figs = [name for name in REQUIRED_FIGS if not (TERMINAL_SOC_FIG_DIR / name).exists()]
14079     if missing_figs:
14080         raise SystemExit(f"missing figures: {missing_figs}")
14081     audit = json.loads((out / "q3_physical_audit.json").read_text(encoding="utf-8"))
14082     daily = pd.read_csv(out / "q3_terminal_soc_daily.csv")
14083     totals = pd.read_csv(out / "q3_terminal_soc_strategy_totals.csv")
14084     delta = pd.read_csv(out / "q3_terminal_soc_boundary_delta.csv")
14085     ledger = pd.read_csv(out / "q3_settlement_ledger.csv")
14086     log = pd.read_csv(out / "q3_update_log.csv")
14087     meta = json.loads((out / "run_meta.json").read_text(encoding="utf-8"))
14088
14089     failures: list[str] = []
14090     if not audit.get("all_pass"):
14091         failures.append("physical/continuity audit all_pass is false")
14092
14093     for col in KEY + ["run_id", "year_end_boundary"]:
14094         if col not in daily.columns:
14095             failures.append(f"daily missing {col}")
14096         if col not in ledger.columns:
14097             failures.append(f"ledger missing {col}")
14098         if col not in log.columns:
14099             failures.append(f"update log missing {col}")
14100
14101     expected_n = 31 * len(STRATEGIES) * len(YEAR_END_BOUNDARIES)
14102     if len(daily) != expected_n:
14103         failures.append(f"expected {expected_n} daily rows, got {len(daily)}")
14104     if daily.duplicated(KEY).sum():
14105         failures.append("duplicate daily keys")
14106     if len(totals) != len(STRATEGIES) * len(YEAR_END_BOUNDARIES):
14107         failures.append(f"expected 10 strategy totals, got {len(totals)}")
14108
14109     dispatch_dir = out / "dispatch_daily"
14110     for _, row in daily.iterrows():
14111         run_id = str(row["run_id"])
14112         expected_id = make_run_id(
14113             row["date"],
14114             row["strategy"],
14115             row["load_information_case"],
14116             row["pv_mapping_mode"],
14117             row["settlement_mode"],
14118             bool(row["with_terminal_value"]),
14119             float(row["year_end_soc_kwh"]),
14120         )
14121         if run_id != expected_id:
14122             _fail(failures, run_id, "run_id", f"mismatch {expected_id}")
14123         if row["load_information_case"] != LOAD_INFORMATION_MAIN:
14124             _fail(failures, run_id, "load_information_case", "must stay causal_load_main")
14125         if row["pv_mapping_mode"] != PV_MAPPING_LINEAR:
14126             _fail(failures, run_id, "pv_mapping_mode", "pilot is linear_anchor_main only")
14127         if row["settlement_mode"] != SETTLEMENT_MAIN:
14128             _fail(failures, run_id, "settlement_mode", "pilot is anchor_final_main only")
14129         stem = dispatch_stem(
14130             row["date"],
14131             row["strategy"],
14132             row["load_information_case"],

```

```

14133         row["pv_mapping_mode"],
14134         row["settlement_mode"],
14135         bool(row["with_terminal_value"]),
14136         float(row["year_end_soc_kwh"]),
14137     )
14138     path = dispatch_dir / f"{stem}.csv"
14139     if not path.exists():
14140         _fail(failures, run_id, "dispatch", f"missing {path.name}")
14141         continue
14142     frame = pd.read_csv(path)
14143     if len(frame) != T:
14144         _fail(failures, run_id, "dispatch", f"rows {len(frame)} != {T}")
14145     if str(frame["run_id"].iloc[0]) != run_id:
14146         _fail(failures, run_id, "dispatch.run_id", "inconsistent")
14147     if float(np.max(np.abs(frame["balance_residual_kwh"]))) >= 1e-5:
14148         _fail(failures, run_id, "balance_residual_kwh", "energy residual")
14149     if float(row["max_soc_residual_kwh"]) >= 1e-5:
14150         _fail(failures, run_id, "max_soc_residual_kwh", "SOC residual")
14151     if float(row["max_simultaneous_cd_kwh2"]) > 1e-4:
14152         _fail(failures, run_id, "simultaneous_cd", "charge*discharge")
14153     if int(row["locked_period_violations"]) != 0:
14154         _fail(failures, run_id, "locked_period_violations", "locked rewrite")
14155     if row["date"] == TERMINAL_SOC_PILOT_START and abs(float(row["soc_start_kwh"]) - E_INITIAL_KWH) > 1e-5:
14156         _fail(failures, run_id, "soc_start_kwh", "Dec 1 must start at 6000")
14157     if row["date"] == TERMINAL_SOC_PILOT_END:
14158         if abs(float(row["soc_end_kwh"]) - float(row["year_end_soc_kwh"])) > 1e-5:
14159             _fail(
14160                 failures,
14161                 run_id,
14162                 "soc_end_kwh",
14163                 f"{row['soc_end_kwh']} vs target {row['year_end_soc_kwh']}",
14164             )
14165     phi = realized_settlement(
14166         frame["price"].to_numpy(),
14167         [frame["planned_g0_kwh"].to_numpy(), frame["final_g_kwh"].to_numpy()],
14168         SETTLEMENT_MAIN,
14169     )
14170     if float(np.max(np.abs(phi - frame["phi_yuan"].to_numpy()))) > 1e-8:
14171         _fail(failures, run_id, "phi_yuan", "does not match p gF + 0.5 p |gF-g0|")
14172     led = ledger[(ledger["run_id"] == run_id) & (ledger["row_role"] == "period_total")]
14173     if len(led) != T:
14174         _fail(failures, run_id, "ledger", f"period_total rows {len(led)}")
14175     elif abs(float(led["total_yuan"].sum()) - float(row["total_cost_yuan"])) > 1e-6:
14176         _fail(failures, run_id, "total_cost_yuan", "ledger mismatch")
14177
14178     for (strategy, boundary), block in daily.groupby(["strategy", "year_end_boundary"]):
14179         block = block.sort_values("date")
14180         if len(block) != 31:
14181             failures.append(f"{strategy}/{boundary}: {len(block)} days")
14182             continue
14183         if block["date"].iloc[0] != TERMINAL_SOC_PILOT_START or block["date"].iloc[-1] != TERMINAL_SOC_PILOT_END:
14184             failures.append(f"{strategy}/{boundary}: date range")
14185         starts = block["soc_start_kwh"].to_numpy()
14186         ends = block["soc_end_kwh"].to_numpy()
14187         if float(np.max(np.abs(starts[1:] - ends[:-1]))) > 1e-5:
14188             failures.append(f"{strategy}/{boundary}: cross-day SOC not continuous")
14189
14190     if set(delta["strategy"]) != set(STRATEGIES):
14191         failures.append("delta table missing strategies")
14192     if YEAR_END_BOUNDARY_A not in set(totals["year_end_boundary"]) or YEAR_END_BOUNDARY_B not in set(
14193         totals["year_end_boundary"]
14194     ):
14195         failures.append("totals missing A or B")
14196
14197     lines = [
14198         "# Q3 年末 SOC 边界 12 月试验审计",
14199         "",
14200         "主口径固定 `causal_load_main` + `linear_anchor_main` + `anchor_final_main`。",

```

```

14201     "12 月 1 日 00:00 孤立起点 6000 kWh, 不是全年真实 12 月 1 日 SOC。",
14202     "**不**生成 `result3.xlsx`, **不**实现 M5, **不**自动选择 A/B。",
14203     "",
14204     "## 运行",
14205     "",
14206     "```bash",
14207     ".venv/bin/python -m unittest q3.test_q3 -v",
14208     ".venv/bin/python q3/run_q3_terminal_soc_pilot.py",
14209     ".venv/bin/python q3/validate_q3_terminal_soc_pilot.py",
14210     "```",
14211     "",
14212     f"- 物理/连续审计全部通过: {audit.get('all_pass')}",
14213     f"- git: {meta.get('git_commit')}",
14214     f"- 求解器: {meta.get('solver')}; 依赖: {meta.get('dependency_versions')}",
14215     f"- 日运行数: {len(daily)} (2 边界 × 5 策略 × 31 日)",
14216     "",
14217     "## 策略 × 边界汇总",
14218     "",
14219     totals.to_string(index=False),
14220     "",
14221     "## ΔC = C(6000) - C(1200)",
14222     "",
14223     delta.to_string(index=False),
14224     "",
14225     "本试验只为队长选择年末边界提供证据, 不得写入论文主结论。",
14226 ]
14227 (out / "validation.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
14228 payload = {
14229     "all_pass": not failures and bool(audit.get("all_pass")),
14230     "failures": failures,
14231     "n_daily_rows": int(len(daily)),
14232     "n_ledger_rows": int(len(ledger)),
14233     "git_commit": meta.get("git_commit"),
14234     "load_treatment": LOAD_INFORMATION_MAIN,
14235     "chooses_year_end_boundary": False,
14236 }
14237 (out / "validation.json").write_text(
14238     json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
14239 )
14240 if failures:
14241     raise SystemExit("validation failed:\n- " + "\n- ".join(failures))
14242 print(json.dumps(payload, indent=2))
14243
14244
14245 if __name__ == "__main__":
14246     main()
14247
14248 # ===== q4/audit.py =====
14249 """Physical, ledger, and information-set audits for Q4-2/Q4-3."""
14250
14251 from __future__ import annotations
14252
14253 import json
14254 from pathlib import Path
14255
14256 import numpy as np
14257 import pandas as pd
14258
14259 from q2.config import E_MAX_KWH, E_MIN_KWH, NUMERIC_TOL, SIMULTANEOUS_CD_TOL, T
14260 from q3.optimization import settlement_cost
14261
14262
14263 def ledger_from_dispatch_q42(dispatch: pd.DataFrame) -> pd.DataFrame:
14264     out = dispatch.copy()
14265     out["normal_cost_yuan"] = out["actual_price"] * out["q_or_g0_kwh"]
14266     out["adjustment_cost_yuan"] = 0.0
14267     out["emergency_cost_yuan"] = 5.0 * out["actual_price"] * out["emergency_kwh"]
14268     out["total_cost_yuan"] = (

```

```

14269         out["normal_cost_yuan"] + out["adjustment_cost_yuan"] + out["emergency_cost_yuan"]
14270     )
14271     return out
14272
14273
14274 def ledger_from_dispatch_q43(dispatch: pd.DataFrame) -> pd.DataFrame:
14275     phi = settlement_cost(
14276         dispatch["actual_price"].to_numpy(),
14277         dispatch["q_or_g0_kwh"].to_numpy(),
14278         dispatch["g_final_kwh"].to_numpy(),
14279     )
14280     out = dispatch.copy()
14281     out["normal_cost_yuan"] = dispatch["actual_price"] * dispatch["g_final_kwh"]
14282     out["adjustment_cost_yuan"] = phi - out["normal_cost_yuan"].to_numpy()
14283     out["emergency_cost_yuan"] = 5.0 * dispatch["actual_price"] * dispatch["emergency_kwh"]
14284     out["total_cost_yuan"] = (
14285         out["normal_cost_yuan"] + out["adjustment_cost_yuan"] + out["emergency_cost_yuan"]
14286     )
14287     return out
14288
14289
14290 def physical_from_dispatch(dispatch: pd.DataFrame, q4_3: bool) -> dict:
14291     x = dispatch["x_kwh"].to_numpy()
14292     cap = dispatch["g_final_kwh"].to_numpy() if q4_3 else dispatch["q_or_g0_kwh"].to_numpy()
14293     cd = dispatch["charge_kwh"].to_numpy() * dispatch["discharge_kwh"].to_numpy()
14294     return {
14295         "n_periods": int(len(dispatch)),
14296         "max_balance_residual_kwh": float(dispatch["balance_residual_kwh"].abs().max()),
14297         "soc_min_kwh": float(dispatch["soc_end_kwh"].min()),
14298         "soc_max_kwh": float(dispatch["soc_end_kwh"].max()),
14299         "max_x_minus_cap_kwh": float(np.max(x - cap)),
14300         "max_simultaneous_cd_kwh2": float(np.max(cd)),
14301         "n_rows": int(len(dispatch)),
14302         "pass": bool(
14303             len(dispatch) == T
14304             and dispatch["balance_residual_kwh"].abs().max() < 1e-5
14305             and dispatch["soc_end_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL
14306             and dispatch["soc_end_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL
14307             and np.max(x - cap) < NUMERIC_TOL
14308             and np.max(cd) <= SIMULTANEOUS_CD_TOL
14309         ),
14310     }
14311
14312
14313 def write_json(path: Path, payload: dict) -> None:
14314     path.parent.mkdir(parents=True, exist_ok=True)
14315     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str) + "\n", encoding="utf-8")
14316
14317
14318 def q_unchanged_probe(result_q: np.ndarray, dispatch: pd.DataFrame) -> bool:
14319     return bool(np.allclose(result_q, dispatch["q_or_g0_kwh"].to_numpy(), atol=1e-9))
14320
14321
14322 def prefix_lock_probe(update_log: pd.DataFrame) -> bool:
14323     if update_log.empty:
14324         return True
14325     return bool(update_log["prefix_lock_ok"].all())
14326
14327
14328 def warmup_soc_continuity(daily: pd.DataFrame) -> dict:
14329     if daily.empty or len(daily) < 2:
14330         return {"n_days": int(len(daily)), "max_soc_gap_kwh": 0.0, "pass": True}
14331     end_vals = daily["soc_end_kwh"].to_numpy()[1:]
14332     start_vals = daily["soc_start_kwh"].to_numpy()[1:]
14333     gap = float(np.max(np.abs(end_vals - start_vals)))
14334     return {"n_days": int(len(daily)), "max_soc_gap_kwh": gap, "pass": bool(gap < 1e-6)}
14335
14336 # ===== q4/bundle.py =====

```

```

14337 """Load accepted Q2/Q3 information plus the frozen causal price archive."""
14338
14339 from __future__ import annotations
14340
14341 from dataclasses import dataclass
14342
14343 from q2.data import Q2Data, load_q2_data
14344 from q2.forecast import ForecastArchive, build_forecast_archive
14345 from q3.data import Q3Data, load_q3_data
14346 from q4.data import Q4PriceData, load_q4_prices
14347 from q4.price_forecast import PriceForecastArchive, fit_causal_price_forecasts
14348
14349
14350 @dataclass(frozen=True)
14351 class Q4Bundle:
14352     q3: Q3Data
14353     q2: Q2Data
14354     q2_forecast: ForecastArchive
14355     prices: Q4PriceData
14356     price_archive: PriceForecastArchive
14357
14358     def date_index(self, date: str) -> int:
14359         return self.prices.date_index(date)
14360
14361     def n_days(self) -> int:
14362         return len(self.prices.dates)
14363
14364
14365 def load_q4_bundle(compute_price_mpc: bool = False) -> Q4Bundle:
14366     q3 = load_q3_data()
14367     q2 = load_q2_data()
14368     prices = load_q4_prices()
14369     if not q3.dates.equals(prices.dates) or not q2.dates.equals(prices.dates):
14370         raise ValueError("Q2/Q3/附件4 calendars must match")
14371     price_archive = fit_causal_price_forecasts(
14372         prices.price, prices.dates, compute_mpc=compute_price_mpc
14373     )
14374     return Q4Bundle(
14375         q3=q3,
14376         q2=q2,
14377         q2_forecast=build_forecast_archive(q2),
14378         prices=prices,
14379         price_archive=price_archive,
14380     )
14381
14382 # ===== q4/campaign.py =====
14383 """Sequential Q4-2 / Q4-3 campaigns with streamed daily outputs."""
14384
14385 from __future__ import annotations
14386
14387 from pathlib import Path
14388
14389 import numpy as np
14390 import pandas as pd
14391
14392 from q2.config import E_INITIAL_KWH
14393 from q3.config import PILOT_DATES
14394 from q4.bundle import Q4Bundle
14395 from q4.config import (
14396     FIXED_SCENARIO_K,
14397     OUTPUT_DIR,
14398     PAM_SEED,
14399     Q4_2_DISPATCH_DIR,
14400     Q4_3_DISPATCH_DIR,
14401     RISK_CALIBRATION_DAYS,
14402     RISK_WARMUP_DAYS,
14403 )
14404 from q4.q4_2 import alpha_for_day, run_q4_2_day, select_risk_alpha

```



```

14405 from q4.q4_3 import run_q4_3_day
14406
14407
14408 def _append_csv(path: Path, frame: pd.DataFrame) -> None:
14409     path.parent.mkdir(parents=True, exist_ok=True)
14410     header = not path.exists() or path.stat().st_size == 0
14411     frame.to_csv(path, mode="a", header=header, index=False)
14412
14413
14414 def last_index_for_dates(bundle: Q4Bundle, dates: tuple[str, ...]) -> int:
14415     return max(bundle.date_index(date) for date in dates)
14416
14417
14418 def run_q4_2_campaign(
14419     bundle: Q4Bundle,
14420     end_index: int,
14421     detail_dates: tuple[str, ...] = PILOT_DATES,
14422     out_dir: Path = OUTPUT_DIR,
14423     write_all_dispatch: bool = False,
14424 ) -> dict:
14425     Q4_2_DISPATCH_DIR.mkdir(parents=True, exist_ok=True)
14426     daily_path = out_dir / "q4_2_warmup_daily.csv"
14427     ahead_path = out_dir / "q4_2_day_ahead_audit.csv"
14428     scen_path = out_dir / "q4_2_scenario_audit.csv"
14429     ledger_path = out_dir / "q4_2_cost_ledger.csv"
14430     alpha_sel_path = out_dir / "q4_2_alpha_selection.csv"
14431     alpha_daily_path = out_dir / "q4_2_alpha_calibration_daily.csv"
14432     for path in (
14433         daily_path,
14434         ahead_path,
14435         scen_path,
14436         ledger_path,
14437         alpha_sel_path,
14438         alpha_daily_path,
14439     ):
14440         if path.exists():
14441             path.unlink()
14442     soc = E_INITIAL_KWH
14443     start_soc = np.full(bundle.n_days(), np.nan)
14444     calibrated: dict[int, float | None] = {}
14445     detail = {}
14446     for i in range(end_index + 1):
14447         start_soc[i] = soc
14448         cal = (i // RISK_CALIBRATION_DAYS) * RISK_CALIBRATION_DAYS
14449         if i >= RISK_WARMUP_DAYS and cal not in calibrated and i == cal:
14450             choice = select_risk_alpha(
14451                 bundle, start_soc, i, FIXED_SCENARIO_K, log=True
14452             )
14453             calibrated[cal] = choice.selected_alpha
14454             cal_date = bundle.prices.dates[i].strftime("%Y-%m-%d")
14455             window_frame = pd.DataFrame(
14456                 [{"calibration_date": cal_date, **row} for row in choice.window_records]
14457             )
14458             daily_frame = pd.DataFrame(
14459                 [{"calibration_date": cal_date, **row} for row in choice.daily_records]
14460             )
14461             _append_csv(alpha_sel_path, window_frame)
14462             if not daily_frame.empty:
14463                 _append_csv(alpha_daily_path, daily_frame)
14464             print(
14465                 f"Q4-2 calibrated alpha={choice.selected_alpha} at {cal_date}",
14466                 flush=True,
14467             )
14468         elif i < RISK_WARMUP_DAYS:
14469             calibrated.setdefault(cal, None)
14470             alpha = alpha_for_day(i, calibrated)
14471             date = bundle.prices.dates[i].strftime("%Y-%m-%d")
14472             keep = write_all_dispatch or date in detail_dates

```

```

14473     write_path = Q4_2_DISPATCH_DIR / f"dispatch_{date}.csv" if keep else None
14474     result = run_q4_2_day(bundle, i, soc, alpha, write_dispatch=write_path)
14475     soc = float(result.summary["soc_end_kwh"])
14476     _append_csv(daily_path, pd.DataFrame([result.summary]))
14477     _append_csv(ahead_path, pd.DataFrame([result.day_ahead_audit]))
14478     if result.scenario_rows:
14479         _append_csv(scen_path, pd.DataFrame(result.scenario_rows))
14480     if keep:
14481         nrm = result.dispatch["actual_price"] * result.dispatch["q_or_g0_kwh"]
14482         emg = 5.0 * result.dispatch["actual_price"] * result.dispatch["emergency_kwh"]
14483         _append_csv(
14484             ledger_path,
14485             pd.DataFrame(
14486                 {
14487                     "date": result.dispatch["date"],
14488                     "period_index": result.dispatch["period_index"],
14489                     "normal_cost_yuan": nrm,
14490                     "adjustment_cost_yuan": 0.0,
14491                     "emergency_cost_yuan": emg,
14492                     "total_cost_yuan": nrm + emg,
14493                 }
14494             ),
14495         )
14496         if date in detail_dates:
14497             detail[date] = result
14498     print(
14499         f"Q4-2 [{i+1}/{end_index+1}] {date} cost={result.summary['total_cost_yuan']:.2f} "
14500         f"soc={soc:.2f} K={result.summary['k_effective']} alpha={alpha}",
14501         flush=True,
14502     )
14503     if date not in detail:
14504         del result
14505     return {"detail": detail, "end_soc": soc, "calibrated_alpha": calibrated}
14506
14507
14508 def run_q4_3_campaign(
14509     bundle: Q4Bundle,
14510     end_index: int,
14511     detail_dates: tuple[str, ...] = PILOT_DATES,
14512     out_dir: Path = OUTPUT_DIR,
14513     write_all_dispatch: bool = False,
14514     *,
14515     price_mode: str = "causal",
14516     dispatch_dir: Path | None = None,
14517     daily_name: str = "q4_3_warmup_daily.csv",
14518     update_name: str = "q4_3_update_log.csv",
14519     ledger_name: str = "q4_3_cost_ledger.csv",
14520     commitment_prefix: str = "q4_3_commitment_versions",
14521     unlink_existing: bool = True,
14522 ) -> dict:
14523     dest = Q4_3_DISPATCH_DIR if dispatch_dir is None else dispatch_dir
14524     dest.mkdir(parents=True, exist_ok=True)
14525     daily_path = out_dir / daily_name
14526     update_path = out_dir / update_name
14527     ledger_path = out_dir / ledger_name
14528     if unlink_existing:
14529         for path in (daily_path, update_path, ledger_path):
14530             if path.exists():
14531                 path.unlink()
14532     soc = E_INITIAL_KWH
14533     cache: dict = {}
14534     detail = {}
14535     for i in range(end_index + 1):
14536         date = bundle.prices.dates[i].strftime("%Y-%m-%d")
14537         keep = write_all_dispatch or date in detail_dates
14538         result = run_q4_3_day(
14539             bundle, i, soc, value_cut_cache=cache, price_mode=price_mode
14540         )

```

```

14541 soc = float(result.summary["soc_end_kwh"])
14542 _append_csv(daily_path, pd.DataFrame([result.summary]))
14543 _append_csv(update_path, result.update_log)
14544 if keep:
14545     result.dispatch.to_csv(dest / f"dispatch_{date}.csv", index=False)
14546     ledger = result.dispatch[
14547         ["date", "period_index", "normal_cost_yuan", "adjustment_cost_yuan", "emergency_cost_yuan"]
14548     ].copy()
14549     ledger["total_cost_yuan"] = (
14550         ledger["normal_cost_yuan"]
14551         + ledger["adjustment_cost_yuan"]
14552         + ledger["emergency_cost_yuan"]
14553     )
14554     _append_csv(ledger_path, ledger)
14555     if date in detail_dates:
14556         pd.DataFrame(
14557             {
14558                 "date": date,
14559                 "period_index": np.arange(len(result.g0)),
14560                 "g0_kwh": result.g0,
14561                 "g_final_kwh": result.g_final,
14562             }
14563         ).to_csv(out_dir / f"{commitment_prefix}_{date}.csv", index=False)
14564         detail[date] = result
14565     print(
14566         f"Q4-3/{price_mode} [{i+1}/{end_index+1}] {date} "
14567         f"cost={result.summary['total_cost_yuan']:.2f} "
14568         f"soc={soc:.2f} adj={result.summary['adjustment_count']}",
14569         flush=True,
14570     )
14571     if date not in detail:
14572         del result
14573 return {"detail": detail, "end_soc": soc, "pam_seed": PAM_SEED, "price_mode": price_mode}
14574
14575
14576 def load_q42_detail_from_disk(
14577     out_dir: Path = OUTPUT_DIR,
14578     detail_dates: tuple[str, ...] = PILOT_DATES,
14579 ) -> dict:
14580     """Rebuild the in-memory Q4-2 pilot detail from streamed CSVs."""
14581     from q4.q4_2 import Q42DayResult
14582
14583     daily = pd.read_csv(out_dir / "q4_2_warmup_daily.csv")
14584     detail = {}
14585     for date in detail_dates:
14586         path = Q4_2_DISPATCH_DIR / f"dispatch_{date}.csv"
14587         dispatch = pd.read_csv(path)
14588         row = daily.loc[daily["date"] == date].iloc[0].to_dict()
14589         alpha = row.get("risk_alpha")
14590         if alpha is None or (isinstance(alpha, float) and not np.isfinite(alpha)):
14591             row["risk_alpha"] = None
14592         detail[date] = Q42DayResult(
14593             date=date,
14594             day_index=int(row["day_index"]),
14595             q=dispatch["q_or_g0_kwh"].to_numpy(float),
14596             dispatch=dispatch,
14597             summary=row,
14598             day_ahead_audit={},
14599         )
14600     return {"detail": detail, "end_soc": float(daily["soc_end_kwh"].iloc[-1]), "from_disk": True}
14601
14602
14603 def load_q43_detail_from_disk(
14604     out_dir: Path = OUTPUT_DIR,
14605     detail_dates: tuple[str, ...] = PILOT_DATES,
14606 ) -> dict:
14607     """Rebuild the in-memory Q4-3 pilot detail from streamed CSVs."""
14608     from q4.q4_3 import Q43DayResult

```

```

14609
14610 daily = pd.read_csv(out_dir / "q4_3_warmup_daily.csv")
14611 updates = pd.read_csv(out_dir / "q4_3_update_log.csv")
14612 detail = {}
14613 for date in detail_dates:
14614     dispatch = pd.read_csv(Q4_3_DISPATCH_DIR / f"dispatch_{date}.csv")
14615     row = daily.loc[daily["date"] == date].iloc[0].to_dict()
14616     versions = out_dir / f"q4_3_commitment_versions_{date}.csv"
14617     if versions.exists():
14618         ver = pd.read_csv(versions)
14619         g0 = ver["g0_kwh"].to_numpy(float)
14620         g_final = ver["g_final_kwh"].to_numpy(float)
14621     else:
14622         g0 = dispatch["q_or_g0_kwh"].to_numpy(float)
14623         g_final = dispatch["g_final_kwh"].to_numpy(float)
14624     detail[date] = Q43DayResult(
14625         date=date,
14626         day_index=int(row["day_index"]),
14627         g0=g0,
14628         g_final=g_final,
14629         dispatch=dispatch,
14630         update_log=updates.loc[updates["date"] == date].copy(),
14631         summary=row,
14632     )
14633     return {"detail": detail, "end_soc": float(daily["soc_end_kwh"].iloc[-1]), "from_disk": True}
14634
14635 # ===== q4/closeout.py =====
14636 """Reporting-only Q4 closeout: tails, fair windows, physical year scan, residual correlation.
14637
14638 Does not change decisions, workbooks, or official dispatch archives.
14639 """
14640
14641 from __future__ import annotations
14642
14643 from pathlib import Path
14644
14645 import numpy as np
14646 import pandas as pd
14647
14648 from q2.config import E_MAX_KWH, E_MIN_KWH, NUMERIC_TOL, POWER_LIMIT_KWH, SIMULTANEOUS_CD_TOL, T
14649 from q3.config import HOUR_TO_FIRST_MUTABLE, RESULT3_OFFICIAL_STRATEGY, YEAR_END_BOUNDARY_A
14650 from q4.audit import (
14651     ledger_from_dispatch_q42,
14652     ledger_from_dispatch_q43,
14653     physical_from_dispatch,
14654     warmup_soc_continuity,
14655 )
14656 from q4.config import (
14657     EXPECTED_Q2_K8_EXPORT_YUAN,
14658     EXPECTED_Q3_ANNUAL_YUAN,
14659     EXPECTED_Q4_2_ALPHA070_END,
14660     EXPECTED_Q4_2_ALPHA070_START,
14661     EXPECTED_Q4_2_ANNUAL_YUAN,
14662     EXPECTED_Q4_2_EXPORT_YUAN,
14663     EXPECTED_Q4_3_ANNUAL_YUAN,
14664     EXPECTED_Q4_3_EXPORT_YUAN,
14665     EXPORT_END,
14666     EXPORT_N_DAYS,
14667     EXPORT_START,
14668     LOAD_INFORMATION_CASE,
14669     OUTPUT_DIR,
14670     Q2_K8_DAILY_CSV,
14671     Q3_ANNUAL_DAILY_CSV,
14672     Q4_2_ALPHA_SELECTION_CSV,
14673     Q4_2_DAILY_CSV,
14674     Q4_2_DISPATCH_DIR,
14675     Q4_2_SCENARIO_AUDIT_CSV,
14676     Q4_3_DAILY_CSV,

```

```

14677     Q4_3_DISPATCH_DIR,
14678     YEAR_N_DAYS,
14679 )
14680
14681 ALLOWED_Q43_UPDATE_CLOCKS = ("00:00", "06:00", "12:00", "18:00")
14682 PRICE_NOTE = (
14683     "公平对比窗口为 2025-02-01 至 2025-12-31 (334 日)，设备、SOC 规则与负荷/光伏主案例相同；"
14684     "Q2/Q3 使用附件1 分时电价，Q4 使用附件4 变动电价。对比解释为同一套储能在不同价格信息下的运行结果，"
14685     "不是同一电价序列下的会计差额。"
14686 )
14687
14688
14689 def output_interval(
14690     frame: pd.DataFrame,
14691     start: str = EXPORT_START,
14692     end: str = EXPORT_END,
14693     n_days: int = EXPORT_N_DAYS,
14694 ) -> pd.DataFrame:
14695     if "date" not in frame.columns:
14696         raise AssertionError("daily frame lacks date column")
14697     dates = pd.to_datetime(frame["date"])
14698     selected = frame.loc[dates.between(pd.Timestamp(start), pd.Timestamp(end))].copy()
14699     selected = selected.sort_values("date").reset_index(drop=True)
14700     if len(selected) != n_days:
14701         raise AssertionError(f"Expected {n_days} output days, got {len(selected)}")
14702     return selected
14703
14704
14705 def empirical_var(values: np.ndarray, alpha: float) -> float:
14706     if not 0.0 < alpha < 1.0:
14707         raise ValueError("alpha must be in (0, 1)")
14708     arr = np.asarray(values, dtype=float)
14709     if arr.size == 0:
14710         raise ValueError("values must be non-empty")
14711     return float(np.quantile(arr, alpha, method="linear"))
14712
14713
14714 def empirical_cvar(values: np.ndarray, alpha: float) -> float:
14715     """Mean of the worst ceil((1-alpha)*n) observations (higher cost is worse)."""
14716     if not 0.0 < alpha < 1.0:
14717         raise ValueError("alpha must be in (0, 1)")
14718     arr = np.sort(np.asarray(values, dtype=float))
14719     if arr.size == 0:
14720         raise ValueError("values must be non-empty")
14721     k = max(1, int(np.ceil((1.0 - alpha) * arr.size)))
14722     return float(arr[-k:].mean())
14723
14724
14725 def tail_bundle(values: np.ndarray, prefix: str = "") -> dict:
14726     arr = np.asarray(values, dtype=float)
14727     key = f"{prefix}_" if prefix else ""
14728     return {
14729         f"{key}n_days": int(arr.size),
14730         f"{key}mean_yuan": float(arr.mean()),
14731         f"{key}max_yuan": float(arr.max()),
14732         f"{key}p90_yuan": empirical_var(arr, 0.90),
14733         f"{key}p95_yuan": empirical_var(arr, 0.95),
14734         f"{key}cvar_0_90_yuan": empirical_cvar(arr, 0.90),
14735         f"{key}cvar_n_tail_0_90": int(max(1, int(np.ceil(0.10 * arr.size)))),
14736     }
14737
14738
14739 def _require_close(actual: float, expected: float, label: str, atol: float = 0.01) -> None:
14740     if abs(actual - expected) >= atol:
14741         raise AssertionError(f"{label} mismatch: {actual:.6f} vs signed-off {expected:.2f}")
14742
14743
14744 def load_q4_2_daily() -> pd.DataFrame:

```

```

14745     return pd.read_csv(Q4_2_DAILY_CSV)
14746
14747
14748 def load_q4_3_daily() -> pd.DataFrame:
14749     return pd.read_csv(Q4_3_DAILY_CSV)
14750
14751
14752 def load_q2_k8_daily() -> pd.DataFrame:
14753     return pd.read_csv(Q2_K8_DAILY_CSV)
14754
14755
14756 def load_q3_official_daily() -> pd.DataFrame:
14757     raw = pd.read_csv(Q3_ANNUAL_DAILY_CSV)
14758     selected = raw.loc[
14759         (raw["strategy"] == RESULT3_OFFICIAL_STRATEGY)
14760         & (raw["year_end_boundary"] == YEAR_END_BOUNDARY_A)
14761         & (raw["load_information_case"] == LOAD_INFORMATION_CASE)
14762     ].copy()
14763     selected = selected.sort_values("date").drop_duplicates("date").reset_index(drop=True)
14764     if len(selected) != YEAR_N_DAYS:
14765         raise AssertionError(f"Q3 official daily expected {YEAR_N_DAYS} days, got {len(selected)}")
14766     annual = float(selected["total_cost_yuan"].sum())
14767     _require_close(annual, EXPECTED_Q3_ANNUAL_YUAN, "Q3 official annual")
14768     return selected
14769
14770
14771 def cost_components(frame: pd.DataFrame, scheme: str) -> dict:
14772     if scheme == "q4_2":
14773         normal = float(frame["normal_cost_yuan"].sum())
14774         adjustment = float(frame["adjustment_cost_yuan"].sum()) if "adjustment_cost_yuan" in frame else 0.0
14775         emergency = float(frame["emergency_cost_yuan"].sum())
14776         total = float(frame["total_cost_yuan"].sum())
14777         planned_kwh = float(frame["planned_q_kwh"].sum()) if "planned_q_kwh" in frame else float("nan")
14778     elif scheme == "q4_3":
14779         normal = float(frame["normal_cost_yuan"].sum())
14780         adjustment = float(frame["adjustment_cost_yuan"].sum())
14781         emergency = float(frame["emergency_cost_yuan"].sum())
14782         total = float(frame["total_cost_yuan"].sum())
14783         planned_kwh = float(frame["g0_kwh"].sum()) if "g0_kwh" in frame else float("nan")
14784     elif scheme == "q2":
14785         normal = float(frame["planned_cost_yuan"].sum())
14786         adjustment = 0.0
14787         emergency = float(frame["emergency_cost_yuan"].sum())
14788         total = float(frame["total_cost_yuan"].sum())
14789         planned_kwh = float("nan")
14790     elif scheme == "q3":
14791         normal = float(frame["settlement_cost_yuan"].sum())
14792         adjustment = 0.0
14793         emergency = float(frame["emergency_cost_yuan"].sum())
14794         total = float(frame["total_cost_yuan"].sum())
14795         planned_kwh = float("nan")
14796     else:
14797         raise ValueError(scheme)
14798     if abs(total - (normal + adjustment + emergency)) >= 0.05:
14799         raise AssertionError(f"{scheme} components do not sum to total")
14800     return {
14801         "scheme": scheme,
14802         "n_days": int(len(frame)),
14803         "normal_or_settlement_yuan": normal,
14804         "adjustment_yuan": adjustment,
14805         "emergency_cost_yuan": emergency,
14806         "total_cost_yuan": total,
14807         "emergency_kwh": float(frame["emergency_kwh"].sum()),
14808         "planned_kwh": planned_kwh,
14809         "max_day_cost_yuan": float(frame["total_cost_yuan"].max()),
14810         "max_day_cost_date": str(
14811             pd.Timestamp(frame.loc[frame["total_cost_yuan"].idxmax(), "date"]).date()
14812         ),

```

```

14813         "max_emergency_cost_yuan": float(frame["emergency_cost_yuan"].max()),
14814         "max_emergency_cost_date": str(
14815             pd.Timestamp(frame.loc[frame["emergency_cost_yuan"].idxmax(), "date"]).date()
14816         ),
14817         **tail_bundle(frame["total_cost_yuan"].to_numpy(float), prefix="daily_cost"),
14818     }
14819
14820
14821 def price_error_stats(audit: pd.DataFrame, start: str, end: str) -> dict:
14822     dates = pd.to_datetime(audit["date"])
14823     window = audit.loc[dates.between(pd.Timestamp(start), pd.Timestamp(end))].copy()
14824     ahead = window.loc[np.isfinite(window["day_ahead_mae"].to_numpy(float))].copy()
14825     if ahead.empty:
14826         raise AssertionError("no finite day-ahead price errors in window")
14827     mae = float(ahead["day_ahead_mae"].mean())
14828     rmse = float(np.sqrt(np.mean(np.square(ahead["day_ahead_rmse"].to_numpy(float)))))
14829     return {
14830         "start": start,
14831         "end": end,
14832         "n_days": int(len(window)),
14833         "n_days_with_day_ahead": int(len(ahead)),
14834         "day_ahead_mae": mae,
14835         "day_ahead_rmse_pooled": rmse,
14836         "day_ahead_rmse_mean_daily": float(ahead["day_ahead_rmse"].mean()),
14837         "intraday_06_mae": float(ahead["intraday_06_mae"].mean()),
14838         "intraday_12_mae": float(ahead["intraday_12_mae"].mean()),
14839         "intraday_18_mae": float(ahead["intraday_18_mae"].mean()),
14840         "aggregation": (
14841             "MAE = mean of daily MAE; pooled RMSE = sqrt(mean of daily RMSE squared); "
14842             "equal slot count per day with a finite day-ahead forecast"
14843         ),
14844     }
14845
14846
14847 def build_fair_comparison(
14848     q42: pd.DataFrame,
14849     q43: pd.DataFrame,
14850     q2: pd.DataFrame,
14851     q3: pd.DataFrame,
14852 ) -> dict:
14853     q42_year = cost_components(q42, "q4_2")
14854     q43_year = cost_components(q43, "q4_3")
14855     q42_export = cost_components(output_interval(q42), "q4_2")
14856     q43_export = cost_components(output_interval(q43), "q4_3")
14857     q2_export = cost_components(output_interval(q2), "q2")
14858     q3_export = cost_components(output_interval(q3), "q3")
14859     q3_year = cost_components(q3, "q3")
14860     _require_close(q42_year["total_cost_yuan"], EXPECTED_Q4_2_ANNUAL_YUAN, "Q4-2 annual")
14861     _require_close(q42_export["total_cost_yuan"], EXPECTED_Q4_2_EXPORT_YUAN, "Q4-2 2-12")
14862     _require_close(q43_year["total_cost_yuan"], EXPECTED_Q4_3_ANNUAL_YUAN, "Q4-3 annual")
14863     _require_close(q43_export["total_cost_yuan"], EXPECTED_Q4_3_EXPORT_YUAN, "Q4-3 2-12")
14864     _require_close(q2_export["total_cost_yuan"], EXPECTED_Q2_K8_EXPORT_YUAN, "Q2 K=8 2-12")
14865     _require_close(q3_year["total_cost_yuan"], EXPECTED_Q3_ANNUAL_YUAN, "Q3 official annual")
14866     return {
14867         "price_note": PRICE_NOTE,
14868         "export_window": {"start": EXPORT_START, "end": EXPORT_END, "n_days": EXPORT_N_DAYS},
14869         "annual_window": {"start": "2025-01-01", "end": "2025-12-31", "n_days": YEAR_N_DAYS},
14870         "q4_2_annual": q42_year,
14871         "q4_3_annual": q43_year,
14872         "q3_annual": q3_year,
14873         "export": {
14874             "q4_2": q42_export,
14875             "q4_3": q43_export,
14876             "q2_k8": q2_export,
14877             "q3_m1_m6": q3_export,
14878         },
14879         "pairs": {
14880             "q4_2_minus_q2_k8_export_yuan": q42_export["total_cost_yuan"] - q2_export["total_cost_yuan"],

```

```

14881         "q4_2_minus_q2_k8_emergency_kwh": q42_export["emergency_kwh"] - q2_export["emergency_kwh"],
14882         "q4_3_minus_q3_m1_m6_export_yuan": q43_export["total_cost_yuan"] - q3_export["total_cost_yuan"],
14883         "q4_3_minus_q3_m1_m6_emergency_kwh": q43_export["emergency_kwh"] - q3_export["emergency_kwh"],
14884     },
14885 }
14886
14887
14888 def expected_q43_last_update(period_index: np.ndarray) -> np.ndarray:
14889     clocks = np.full(len(period_index), "00:00", dtype=object)
14890     clocks[period_index >= HOUR_TO_FIRST_MUTABLE[6]] = "06:00"
14891     clocks[period_index >= HOUR_TO_FIRST_MUTABLE[12]] = "12:00"
14892     clocks[period_index >= HOUR_TO_FIRST_MUTABLE[18]] = "18:00"
14893     return clocks
14894
14895
14896 def _intra_day_soc_gap(dispatch: pd.DataFrame) -> float:
14897     if len(dispatch) < 2:
14898         return 0.0
14899     return float(
14900         np.max(
14901             np.abs(
14902                 dispatch["soc_start_kwh"].to_numpy(float)[1:]
14903                 - dispatch["soc_end_kwh"].to_numpy(float)[:1]
14904             )
14905         )
14906     )
14907
14908
14909 def _energy_balance_gap(dispatch: pd.DataFrame) -> float:
14910     x = dispatch["x_kwh"].to_numpy(float)
14911     e = dispatch["emergency_kwh"].to_numpy(float)
14912     pv = dispatch["actual_pv_kwh"].to_numpy(float)
14913     curt = dispatch["curtailment_kwh"].to_numpy(float)
14914     d = dispatch["discharge_kwh"].to_numpy(float)
14915     load = dispatch["actual_load_kwh"].to_numpy(float)
14916     c = dispatch["charge_kwh"].to_numpy(float)
14917     residual = x + e + pv - curt + d - load - c
14918     stored = dispatch["balance_residual_kwh"].to_numpy(float)
14919     return float(max(np.max(np.abs(residual)), np.max(np.abs(residual - stored))))
14920
14921
14922 def scan_dispatch_year(
14923     dispatch_dir: Path,
14924     daily: pd.DataFrame,
14925     *,
14926     q4_3: bool,
14927 ) -> dict:
14928     paths = sorted(dispatch_dir.glob("dispatch_*.csv"))
14929     if len(paths) != YEAR_N_DAYS:
14930         raise AssertionError(f"Expected {YEAR_N_DAYS} dispatch files in {dispatch_dir}, got {len(paths)}")
14931     daily = daily.sort_values("date").reset_index(drop=True)
14932     if len(daily) != YEAR_N_DAYS:
14933         raise AssertionError(f"Expected {YEAR_N_DAYS} daily rows, got {len(daily)}")
14934     by_date = {str(row.date): row for row in daily.itertuples()}
14935     max_balance = 0.0
14936     max_x_minus_cap = 0.0
14937     max_cd = 0.0
14938     max_power = 0.0
14939     max_soc_chain = 0.0
14940     soc_min = float("inf")
14941     soc_max = float("-inf")
14942     max_ledger_gap = 0.0
14943     n_physical_pass = 0
14944     info_fail = 0
14945     prefix_fail = 0
14946     first_intraday = HOUR_TO_FIRST_MUTABLE[6]
14947     for path in paths:
14948         date = path.stem.replace("dispatch_", "")

```



```

14949     frame = pd.read_csv(path)
14950     if len(frame) != T:
14951         raise AssertionError(f"{path.name} has {len(frame)} rows")
14952     phys = physical_from_dispatch(frame, q4_3=q4_3)
14953     max_balance = max(max_balance, phys["max_balance_residual_kwh"], _energy_balance_gap(frame))
14954     max_x_minus_cap = max(max_x_minus_cap, phys["max_x_minus_cap_kwh"])
14955     max_cd = max(max_cd, phys["max_simultaneous_cd_kwh2"])
14956     max_power = max(
14957         max_power,
14958         float(frame["charge_kwh"].max()),
14959         float(frame["discharge_kwh"].max()),
14960     )
14961     max_soc_chain = max(max_soc_chain, _intra_day_soc_gap(frame))
14962     soc_min = min(soc_min, phys["soc_min_kwh"])
14963     soc_max = max(soc_max, phys["soc_max_kwh"])
14964     n_physical_pass += int(phys["pass"])
14965     row = by_date[date]
14966     if q4_3:
14967         ledger = ledger_from_dispatch_q43(frame)
14968         expected_clock = expected_q43_last_update(frame["period_index"].to_numpy(int))
14969         clocks = frame["last_update_time"].astype(str).to_numpy()
14970         if not np.array_equal(clocks, expected_clock):
14971             info_fail += 1
14972         if not set(clocks).issubset(ALLOWED_Q43_UPDATE_CLOCKS):
14973             info_fail += 1
14974         g0 = frame["q_or_g0_kwh"].to_numpy(float)
14975         gf = frame["g_final_kwh"].to_numpy(float)
14976         if np.max(np.abs(gf[:first_intraday] - g0[:first_intraday])) > NUMERIC_TOL:
14977             prefix_fail += 1
14978     else:
14979         ledger = ledger_from_dispatch_q42(frame)
14980         if not (frame["last_update_time"].astype(str) == "00:00").all():
14981             info_fail += 1
14982         ledger_total = float(ledger["total_cost_yuan"].sum())
14983         max_ledger_gap = max(max_ledger_gap, abs(ledger_total - float(row.total_cost_yuan)))
14984     soc = warmup_soc_continuity(daily)
14985     physical_pass = (
14986         n_physical_pass == YEAR_N_DAYS
14987         and max_balance < 1e-5
14988         and max_x_minus_cap < NUMERIC_TOL
14989         and max_cd <= SIMULTANEOUS_CD_TOL
14990         and max_power <= POWER_LIMIT_KWH + NUMERIC_TOL
14991         and max_soc_chain < 1e-6
14992         and soc_min >= E_MIN_KWH - NUMERIC_TOL
14993         and soc_max <= E_MAX_KWH + NUMERIC_TOL
14994         and info_fail == 0
14995         and prefix_fail == 0
14996         and max_ledger_gap < 0.01
14997         and soc["pass"]
14998     )
14999     return {
15000         "system": "q4_3" if q4_3 else "q4_2",
15001         "n_dispatch_files": len(paths),
15002         "n_days": int(len(daily)),
15003         "n_physical_pass": n_physical_pass,
15004         "soc_continuity": soc,
15005         "max_balance_residual_kwh": max_balance,
15006         "max_x_minus_cap_kwh": max_x_minus_cap,
15007         "max_simultaneous_cd_kwh2": max_cd,
15008         "max_charge_or_discharge_kwh": max_power,
15009         "power_limit_kwh": float(POWER_LIMIT_KWH),
15010         "max_intraday_soc_chain_gap_kwh": max_soc_chain,
15011         "soc_min_kwh": float(soc_min),
15012         "soc_max_kwh": float(soc_max),
15013         "max_ledger_vs_daily_gap_yuan": max_ledger_gap,
15014         "info_cutoff_failures": info_fail,
15015         "prefix_lock_failures": prefix_fail,
15016         "dec31_soc_kwh": float(daily.iloc[-1]["soc_end_kwh"]),

```

```

15017         "pass": bool(physical_pass),
15018     }
15019
15020
15021 def q4_2_policy_windows(daily: pd.DataFrame) -> pd.DataFrame:
15022     """14-day calibration calendar, including the 28-day warmup with no alpha."""
15023     from q4.config import RISK_CALIBRATION_DAYS, RISK_WARMUP_DAYS
15024     from q4.q4_2 import q2_aligned_calibration_indices
15025
15026     work = daily.copy()
15027     work["date"] = pd.to_datetime(work["date"])
15028     work = work.sort_values("date").reset_index(drop=True)
15029     if len(work) != YEAR_N_DAYS:
15030         raise AssertionError(f"Q4-2 daily expected {YEAR_N_DAYS} days, got {len(work)}")
15031     rows = [
15032         {
15033             "effective_start_date": work.loc[0, "date"],
15034             "effective_end_date": work.loc[RISK_WARMUP_DAYS - 1, "date"],
15035             "n_days": RISK_WARMUP_DAYS,
15036             "risk_alpha": None,
15037             "kind": "warmup",
15038         }
15039     ]
15040     for start in q2_aligned_calibration_indices(len(work)):
15041         end = min(start + RISK_CALIBRATION_DAYS - 1, len(work) - 1)
15042         chunk = work.iloc[start : end + 1]
15043         alphas = chunk["risk_alpha"]
15044         if alphas.isna().any() or alphas.nunique() != 1:
15045             raise AssertionError(
15046                 f"alpha is not constant on {chunk.iloc[0]['date']}--{chunk.iloc[-1]['date']}"
15047             )
15048         rows.append(
15049             {
15050                 "effective_start_date": chunk.iloc[0]["date"],
15051                 "effective_end_date": chunk.iloc[-1]["date"],
15052                 "n_days": int(len(chunk)),
15053                 "risk_alpha": round(float(chunk.iloc[0]["risk_alpha"]), 2),
15054                 "kind": "calibrated",
15055             }
15056         )
15057     frame = pd.DataFrame(rows)
15058     if len(frame) != 26:
15059         raise AssertionError(f"expected 1 warmup + 25 calibrated windows, got {len(frame)}")
15060     hot = frame.loc[frame["risk_alpha"] == 0.70]
15061     if len(hot) != 1:
15062         raise AssertionError("expected exactly one alpha=0.70 window")
15063     if hot.iloc[0]["effective_start_date"] != pd.Timestamp(EXPECTED_Q4_2_ALPHA070_START):
15064         raise AssertionError("alpha=0.70 start mismatch")
15065     if hot.iloc[0]["effective_end_date"] != pd.Timestamp(EXPECTED_Q4_2_ALPHA070_END):
15066         raise AssertionError("alpha=0.70 end mismatch")
15067     return frame
15068
15069
15070 def selected_alpha_blocks(daily: pd.DataFrame, selection: pd.DataFrame) -> pd.DataFrame:
15071     work = daily.copy()
15072     work["date"] = pd.to_datetime(work["date"])
15073     work = work.sort_values("date").reset_index(drop=True)
15074     blocks = []
15075     start_idx = 0
15076     for i in range(1, len(work) + 1):
15077         same = False
15078         if i < len(work):
15079             prev = work.loc[start_idx, "risk_alpha"]
15080             cur = work.loc[i, "risk_alpha"]
15081             same = (pd.isna(prev) and pd.isna(cur)) or (
15082                 pd.notna(prev) and pd.notna(cur) and abs(float(prev) - float(cur)) < 1e-12
15083             )
15084         if same:

```

```

15085         continue
15086     chunk = work.iloc[start_idx:i]
15087     alpha = chunk.iloc[0]["risk_alpha"]
15088     blocks.append(
15089         {
15090             "effective_start_date": chunk.iloc[0]["date"],
15091             "effective_end_date": chunk.iloc[-1]["date"],
15092             "n_days": int(len(chunk)),
15093             "risk_alpha": None if pd.isna(alpha) else round(float(alpha), 2),
15094         }
15095     )
15096     start_idx = i
15097     frame = pd.DataFrame(blocks)
15098     selected = selection.loc[selection["selected"].astype(str).str.lower().isin(("true", "1"))].copy()
15099     selected["calibration_date"] = pd.to_datetime(selected["calibration_date"])
15100     if int(selected["calibration_date"].nunique()) != 25:
15101         raise AssertionError(f"expected 25 Q4-2 calibration windows, got {selected['calibration_date'].nunique()}")
15102     return frame
15103
15104
15105 def pearson_corrcoef(a: np.ndarray, b: np.ndarray) -> float:
15106     x = np.asarray(a, dtype=float).ravel()
15107     y = np.asarray(b, dtype=float).ravel()
15108     if x.size != y.size or x.size < 3:
15109         raise ValueError("correlation requires aligned arrays with length >= 3")
15110     if np.std(x) < 1e-18 or np.std(y) < 1e-18:
15111         return float("nan")
15112     return float(np.corrcoef(x, y)[0, 1])
15113
15114
15115 def residual_correlation_report(
15116     load_r: np.ndarray, pv_r: np.ndarray, price_r: np.ndarray, dates: list[str]
15117 ) -> dict:
15118     load_r = np.asarray(load_r, dtype=float)
15119     pv_r = np.asarray(pv_r, dtype=float)
15120     price_r = np.asarray(price_r, dtype=float)
15121     if load_r.shape != pv_r.shape or load_r.shape != price_r.shape:
15122         raise AssertionError("residual stacks must share shape")
15123     if load_r.ndim != 2 or load_r.shape[1] != T:
15124         raise AssertionError(f"expected (n_days, {T}) residual stacks")
15125     names = ("load", "pv", "price")
15126     stacks = {"load": load_r, "pv": pv_r, "price": price_r}
15127     flat = {name: stacks[name].ravel() for name in names}
15128     daily_mean = {name: stacks[name].mean(axis=1) for name in names}
15129     corr_flat = {
15130         f"{a}_vs_{b}": pearson_corrcoef(flat[a], flat[b])
15131         for a in names
15132         for b in names
15133         if a < b
15134     }
15135     corr_daily = {
15136         f"{a}_vs_{b}": pearson_corrcoef(daily_mean[a], daily_mean[b])
15137         for a in names
15138         for b in names
15139         if a < b
15140     }
15141     return {
15142         "interpretation": (
15143             "Same-day (L, P, p) forecast residuals. Values are Pearson correlations, "
15144             "not causal effects. Residual dates are strictly those with archived causal "
15145             "load, PV, and day-ahead price forecasts."
15146         ),
15147         "n_days": int(load_r.shape[0]),
15148         "n_slots": int(load_r.size),
15149         "first_date": dates[0] if dates else None,
15150         "last_date": dates[-1] if dates else None,
15151         "pearson_flattened_slots": corr_flat,
15152         "pearson_daily_mean_residual": corr_daily,

```

```

15153         "residual_std": {
15154             "load_kwh": float(load_r.std()),
15155             "pv_kwh": float(pv_r.std()),
15156             "price_yuan_per_kwh": float(price_r.std()),
15157         },
15158         "scenario_audit_source": str(Q4_2_SCENARIO_AUDIT_CSV.relative_to(OUTPUT_DIR.parent.parent)),
15159     }
15160
15161
15162 def build_residual_correlation_from_bundle(bundle) -> dict:
15163     from q4.scenarios import residual_stack
15164
15165     ok = []
15166     for i in range(bundle.n_days()):
15167         if not np.isfinite(bundle.price_archive.day_ahead[i]).all():
15168             continue
15169         if not np.isfinite(bundle.q2_forecast.load_hat[i]).all():
15170             continue
15171         if not np.isfinite(bundle.q2_forecast.pv_hat[i]).all():
15172             continue
15173         ok.append(i)
15174     indices = np.asarray(ok, dtype=int)
15175     load_r, pv_r, price_r = residual_stack(bundle, indices)
15176     dates = [bundle.prices.dates[i].strftime("%Y-%m-%d") for i in indices]
15177     report = residual_correlation_report(load_r, pv_r, price_r, dates)
15178     if Q4_2_SCENARIO_AUDIT_CSV.exists():
15179         audit = pd.read_csv(Q4_2_SCENARIO_AUDIT_CSV)
15180         report["scenario_audit_n_rows"] = int(len(audit))
15181         report["scenario_audit_n_days"] = int(audit["date"].nunique())
15182     return report
15183
15184 # ===== q4/config.py =====
15185 """Q4 configuration. Price-forecast constants stay aligned with the accepted P1 module."""
15186
15187 from __future__ import annotations
15188
15189 from q3.config import ATTACH4, PILOT_DATES, ROOT, T
15190
15191 OUTPUT_DIR = ROOT / "output" / "q4"
15192 AUDIT_CSV = OUTPUT_DIR / "q4_price_forecast_audit.csv"
15193 TWODAY_JSON = OUTPUT_DIR / "q4_price_forecast_twoday.json"
15194 CAUSALITY_JSON = OUTPUT_DIR / "q4_price_forecast_causality.json"
15195
15196 Q4_2_DISPATCH_DIR = OUTPUT_DIR / "q4_2_dispatch_daily"
15197 Q4_3_DISPATCH_DIR = OUTPUT_DIR / "q4_3_dispatch_daily"
15198 RESULT4_2_XLSX = ROOT / "output" / "result4-2.xlsx"
15199 RESULT4_3_XLSX = ROOT / "output" / "result4-3.xlsx"
15200
15201 RESIDUAL_POOL_DAYS = 28
15202 FIXED_SCENARIO_K = 8
15203 K_REVIEW_CANDIDATES = (4, 8, 12)
15204 K_REVIEW_REFERENCE_ALPHA = 0.60
15205 K_REVIEW_COST_TOLERANCE = 0.01
15206 RISK_ALPHA_CANDIDATES = (0.60, 0.70, 0.80, 0.90)
15207 RISK_CALIBRATION_DAYS = 14
15208 RISK_WARMUP_DAYS = 28
15209 ALPHA_POLICY_LOOP = "day_ahead_joint_q+locked_q+causal_mpc+next_day_value_cuts"
15210 PAM_SEED = 0
15211 PRICE_MONTHLY_CSV = OUTPUT_DIR / "q4_price_forecast_monthly.csv"
15212 PRICE_MONTHLY_JSON = OUTPUT_DIR / "q4_price_forecast_monthly.json"
15213 LOAD_INFORMATION_CASE = "causal_load_main"
15214 Q4_2_YEAR_END_RULE = "q2_accepted_no_hard_terminal"
15215 Q4_3_YEAR_END_BOUNDARY = "A_q2_aligned"
15216 Q4_3_YEAR_END_SOC_KWH = 1200.0
15217 Q4_3_STRATEGY = "M1_M6"
15218 SETTLEMENT_RULE = "delivery_time_actual_price"
15219 Q4_3_ORACLE_DISPATCH_DIR = OUTPUT_DIR / "q4_3_oracle_dispatch_daily"
15220 Q4_3_SETTLEMENT_SENSITIVITY_CSV = OUTPUT_DIR / "q4_3_settlement_sensitivity.csv"

```

```

15221 Q4_3_SETTLEMENT_SENSITIVITY_JSON = OUTPUT_DIR / "q4_3_settlement_sensitivity.json"
15222 Q4_3_ORACLE_DAILY_CSV = OUTPUT_DIR / "q4_3_oracle_warmup_daily.csv"
15223 Q4_3_SENSITIVITY_SUMMARY_JSON = OUTPUT_DIR / "q4_3_sensitivity_summary.json"
15224
15225 FIG_DIR = ROOT / "fig" / "q4_final"
15226 Q4_2_DAILY_CSV = OUTPUT_DIR / "q4_2_warmup_daily.csv"
15227 Q4_3_DAILY_CSV = OUTPUT_DIR / "q4_3_warmup_daily.csv"
15228 Q4_2_ALPHA_SELECTION_CSV = OUTPUT_DIR / "q4_2_alpha_selection.csv"
15229 Q4_2_SCENARIO_AUDIT_CSV = OUTPUT_DIR / "q4_2_scenario_audit.csv"
15230 Q2_K8_DAILY_CSV = ROOT / "output/q2_full_k8_risk/daily_summary.csv"
15231 Q3_ANNUAL_DAILY_CSV = ROOT / "output/q3_full_annual/q3_annual_daily_summary.csv"
15232 TAIL_STATS_JSON = OUTPUT_DIR / "q4_tail_stats.json"
15233 TAIL_STATS_CSV = OUTPUT_DIR / "q4_tail_stats.csv"
15234 FAIR_COMPARISON_JSON = OUTPUT_DIR / "q4_fair_comparison.json"
15235 FAIR_COMPARISON_CSV = OUTPUT_DIR / "q4_fair_comparison.csv"
15236 PHYSICAL_YEAR_AUDIT_JSON = OUTPUT_DIR / "q4_physical_audit.json"
15237 SCENARIO_CORR_JSON = OUTPUT_DIR / "q4_2_scenario_correlation.json"
15238
15239 EXPORT_START = "2025-02-01"
15240 EXPORT_END = "2025-12-31"
15241 EXPORT_N_DAYS = 334
15242 YEAR_N_DAYS = 365
15243
15244 # Signed-off workbook / annual totals. Comparisons must not mix 1-12 with 2-12.
15245 EXPECTED_Q4_2_ANNUAL_YUAN = 17_681_029.10
15246 EXPECTED_Q4_2_EXPORT_YUAN = 15_257_873.75
15247 EXPECTED_Q4_3_ANNUAL_YUAN = 17_489_808.24
15248 EXPECTED_Q4_3_EXPORT_YUAN = 15_346_418.14
15249 EXPECTED_Q2_K8_EXPORT_YUAN = 14_456_670.55
15250 EXPECTED_Q3_ANNUAL_YUAN = 16_373_508.75
15251 EXPECTED_Q4_2_ALPHA070_START = "2025-03-12"
15252 EXPECTED_Q4_2_ALPHA070_END = "2025-03-25"
15253
15254 CANDIDATE_MODELS = (1, 2, 4)
15255 CANDIDATE_NAMES = {
15256     1: "prev_day",
15257     2: "weekday2",
15258     4: "weekday4",
15259 }
15260
15261 # Number of ended 10-minute periods at the named clock time.
15262 # 06:00 has seen periods 0..35 (ending 06:00); remaining starts at 06:10 (index 36).
15263 INTRA_HOUR_TAU = {
15264     6: 36,
15265     12: 72,
15266     18: 108,
15267 }
15268
15269 VAR_EPS = 1e-18
15270
15271 # ===== q4/data.py =====
15272 from __future__ import annotations
15273
15274 from dataclasses import dataclass
15275 from pathlib import Path
15276
15277 import numpy as np
15278 import pandas as pd
15279
15280 from q3.data import normalize_time
15281 from q4.config import ATTACH4, T
15282
15283
15284 def _finite_nonnegative(values: np.ndarray, name: str) -> None:
15285     if not np.isfinite(values).all():
15286         raise ValueError(f"{name} contains missing or non-finite values")
15287     if np.any(values < 0):
15288         raise ValueError(f"{name} contains negative values")

```

```

15289
15290
15291 @dataclass(frozen=True)
15292 class Q4PriceData:
15293     dates: pd.DatetimeIndex
15294     time_labels: tuple[str, ...]
15295     price: np.ndarray
15296
15297     def date_index(self, date: str) -> int:
15298         return int(self.dates.get_loc(pd.Timestamp(date)))
15299
15300
15301 def load_q4_prices(attach4: Path = ATTACH4) -> Q4PriceData:
15302     if not attach4.exists():
15303         raise FileNotFoundError(
15304             f"附件4 not found: {attach4}. Set CUMCM_C_ATTACH_DIR to the attachments folder."
15305         )
15306     raw = pd.read_excel(attach4, sheet_name=0, header=None)
15307     if raw.shape != (366, 145):
15308         raise ValueError(f"附件4 must be 366x145; got {raw.shape}")
15309     if raw.iloc[0, 0] != "日期\\时间":
15310         raise ValueError("附件4 first header must be 日期\\时间")
15311     time_labels = tuple(normalize_time(x) for x in raw.iloc[0, 1:])
15312     if len(time_labels) != T or time_labels[0] != "00:10" or time_labels[-1] != "0:00+1":
15313         raise ValueError(f"Unexpected 附件4 time order: {time_labels[0]} .. {time_labels[-1]}")
15314     dates = pd.DatetimeIndex(pd.to_datetime(raw.iloc[1:, 0], errors="raise")).normalize()
15315     expected = pd.date_range("2025-01-01", "2025-12-31", freq="D")
15316     if not dates.equals(expected):
15317         raise ValueError("附件4 dates must be continuous 2025-01-01 through 2025-12-31")
15318     price = raw.iloc[1:, 1:].apply(pd.to_numeric, errors="coerce").to_numpy(float)
15319     _finite_nonnegative(price, "附件4 price")
15320     return Q4PriceData(dates=dates, time_labels=time_labels, price=price)
15321
15322 # ===== q4/export_price_monthly.py =====
15323 #!/usr/bin/env python3
15324 """Monthly MAE/RMSE from the accepted P1 price-forecast audit. Does not refit."""
15325
15326 from __future__ import annotations
15327
15328 import json
15329 import sys
15330 from pathlib import Path
15331
15332 ROOT = Path(__file__).resolve().parents[1]
15333 if str(ROOT) not in sys.path:
15334     sys.path.insert(0, str(ROOT))
15335
15336 import numpy as np
15337 import pandas as pd
15338
15339 from q4.config import AUDIT_CSV, PRICE_MONTHLY_CSV, PRICE_MONTHLY_JSON
15340
15341 METRIC_PAIRS = (
15342     ("day_ahead_mae", "day_ahead_rmse"),
15343     ("intraday_06_mae", "intraday_06_rmse"),
15344     ("intraday_12_mae", "intraday_12_rmse"),
15345     ("intraday_18_mae", "intraday_18_rmse"),
15346 )
15347 MAE_ONLY = ("mpc_remaining_mae_mean",)
15348
15349
15350 def _pooled_rmse(series: pd.Series) -> float:
15351     values = pd.to_numeric(series, errors="coerce").to_numpy(float)
15352     values = values[np.isfinite(values)]
15353     if len(values) == 0:
15354         return float("nan")
15355     return float(np.sqrt(np.mean(np.square(values))))
15356

```

```

15357
15358 def monthly_from_audit(frame: pd.DataFrame) -> pd.DataFrame:
15359     work = frame.copy()
15360     work["date"] = pd.to_datetime(work["date"])
15361     work["year_month"] = work["date"].dt.strftime("%Y-%m")
15362     rows = []
15363     for year_month, group in work.groupby("year_month", sort=True):
15364         row: dict = {
15365             "year_month": year_month,
15366             "n_days": int(len(group)),
15367             "n_days_with_day_ahead": int(pd.to_numeric(group["day_ahead_mae"], errors="coerce").notna().sum()),
15368         }
15369         for mae_col, rmse_col in METRIC_PAIRS:
15370             mae = pd.to_numeric(group[mae_col], errors="coerce")
15371             rmse = pd.to_numeric(group[rmse_col], errors="coerce")
15372             row[mae_col] = float(mae.mean()) if mae.notna().any() else float("nan")
15373             row[rmse_col] = _pooled_rmse(rmse)
15374             row[f"{rmse_col}_mean_daily"] = float(rmse.mean()) if rmse.notna().any() else float("nan")
15375         for col in MAE_ONLY:
15376             series = pd.to_numeric(group[col], errors="coerce")
15377             row[col] = float(series.mean()) if series.notna().any() else float("nan")
15378         rows.append(row)
15379     monthly = pd.DataFrame(rows)
15380     annual = {
15381         "year_month": "2025-annual",
15382         "n_days": int(len(work)),
15383         "n_days_with_day_ahead": int(
15384             pd.to_numeric(work["day_ahead_mae"], errors="coerce").notna().sum()
15385         ),
15386     }
15387     for mae_col, rmse_col in METRIC_PAIRS:
15388         mae = pd.to_numeric(work[mae_col], errors="coerce")
15389         rmse = pd.to_numeric(work[rmse_col], errors="coerce")
15390         annual[mae_col] = float(mae.mean()) if mae.notna().any() else float("nan")
15391         annual[rmse_col] = _pooled_rmse(rmse)
15392         annual[f"{rmse_col}_mean_daily"] = float(rmse.mean()) if rmse.notna().any() else float("nan")
15393     for col in MAE_ONLY:
15394         series = pd.to_numeric(work[col], errors="coerce")
15395         annual[col] = float(series.mean()) if series.notna().any() else float("nan")
15396     return pd.concat([monthly, pd.DataFrame([annual])], ignore_index=True)
15397
15398
15399 def export_price_monthly(
15400     audit_csv: Path = AUDIT_CSV,
15401     dest_csv: Path = PRICE_MONTHLY_CSV,
15402     dest_json: Path = PRICE_MONTHLY_JSON,
15403 ) -> dict:
15404     if not audit_csv.exists():
15405         raise FileNotFoundError(f"price audit not found: {audit_csv}")
15406     frame = pd.read_csv(audit_csv)
15407     monthly = monthly_from_audit(frame)
15408     dest_csv.parent.mkdir(parents=True, exist_ok=True)
15409     monthly.to_csv(dest_csv, index=False)
15410     payload = {
15411         "source": str(audit_csv),
15412         "aggregation": {
15413             "mae": "mean of daily MAE over days with a finite value",
15414             "rmse": "pooled RMSE = sqrt(mean of daily RMSE squared); equal slot count per day",
15415             "rmse_mean_daily": "mean of daily RMSE, not pooled",
15416             "note": "Derived from the accepted P1 audit; the price module was not refit.",
15417         },
15418         "n_months": int((monthly["year_month"] != "2025-annual").sum()),
15419         "rows": monthly.to_dict("records"),
15420     }
15421     dest_json.write_text(json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
15422     return payload
15423
15424

```

```

15425 if __name__ == "__main__":
15426     report = export_price_monthly()
15427     print(json.dumps({"csv": str(PRICE_MONTHLY_CSV), "n_rows": len(report["rows"])}), indent=2))
15428
15429 # ===== q4/export_result4.py =====
15430 """Export result4-2.xlsx and result4-3.xlsx from streamed 10-minute dispatch.
15431
15432 February-December only. January is warmup. Does not modify result2.xlsx or result3.xlsx.
15433 """
15434
15435 from __future__ import annotations
15436
15437 import argparse
15438 import json
15439 import shutil
15440 import subprocess
15441 import sys
15442 from pathlib import Path
15443
15444 ROOT_PATH = Path(__file__).resolve().parents[1]
15445 if str(ROOT_PATH) not in sys.path:
15446     sys.path.insert(0, str(ROOT_PATH))
15447
15448 import numpy as np
15449 import pandas as pd
15450 from openpyxl import load_workbook
15451 from openpyxl.styles import Alignment, Font
15452 from openpyxl.utils import get_column_letter
15453
15454 from q3.config import RESULT3_TEMPLATE, T
15455 from q3.export_result3 import (
15456     ADJUST_SHEET,
15457     CHARGE_SHEET,
15458     COL_DAY_FEE,
15459     COL_DAY_QTY,
15460     EMERGENCY_SHEET,
15461     NOTES_SHEET,
15462     PURCHASE_SHEET,
15463     _fill_energy_grid,
15464     _rewrite_charge_sheet,
15465     _rewrite_emergency_sheet,
15466     _time_headers,
15467     export_dates,
15468 )
15469 from q4.audit import ledger_from_dispatch_q42, ledger_from_dispatch_q43
15470 from q4.config import (
15471     OUTPUT_DIR,
15472     Q4_2_DISPATCH_DIR,
15473     Q4_3_DISPATCH_DIR,
15474     RESULT4_2_XLSX,
15475     RESULT4_3_XLSX,
15476     ROOT,
15477 )
15478
15479 RESULT2_LAYOUT = ROOT / "output" / "result2.xlsx"
15480 RESULT4_2_AUDIT = ROOT / "output" / "result4-2_export_audit.json"
15481 RESULT4_3_AUDIT = ROOT / "output" / "result4-3_export_audit.json"
15482
15483
15484 def git_hash() -> str:
15485     try:
15486         return subprocess.check_output(["git", "rev-parse", "HEAD"], cwd=ROOT, text=True).strip()
15487     except (OSError, subprocess.CalledProcessError):
15488         return "unknown"
15489
15490
15491 def _load_dispatch(path: Path, q4_3: bool) -> pd.DataFrame:
15492     raw = pd.read_csv(path)

```



```

15493     if len(raw) != T:
15494         raise ValueError(f"{path} has {len(raw)} rows, expected {T}")
15495     g0 = raw["q_or_g0_kwh"].to_numpy(float)
15496     gf = raw["g_final_kwh"].to_numpy(float) if q4_3 else g0
15497     frame = pd.DataFrame(
15498         {
15499             "date": raw["date"].astype(str),
15500             "period_index": raw["period_index"].to_numpy(int),
15501             "time_label": raw["time_label"].astype(str),
15502             "g0_kwh": g0,
15503             "g_final_kwh": gf,
15504             "x_kwh": raw["x_kwh"].to_numpy(float),
15505             "emergency_kwh": raw["emergency_kwh"].to_numpy(float),
15506             "charge_kwh": raw["charge_kwh"].to_numpy(float),
15507             "discharge_kwh": raw["discharge_kwh"].to_numpy(float),
15508             "soc_start_kwh": raw["soc_start_kwh"].to_numpy(float),
15509             "soc_end_kwh": raw["soc_end_kwh"].to_numpy(float),
15510             "price": raw["actual_price"].to_numpy(float),
15511             "balance_residual_kwh": raw["balance_residual_kwh"].to_numpy(float),
15512         }
15513     )
15514     billed = ledger_from_dispatch_q43(raw) if q4_3 else ledger_from_dispatch_q42(raw)
15515     frame["phi_yuan"] = billed["normal_cost_yuan"] + billed["adjustment_cost_yuan"]
15516     frame["emergency_cost_yuan"] = billed["emergency_cost_yuan"]
15517     frame["plan_fee_yuan"] = frame["price"] * frame["g0_kwh"]
15518     frame["actual_fee_yuan"] = billed["total_cost_yuan"]
15519     return frame.sort_values("period_index").reset_index(drop=True)
15520
15521
15522 def load_export_window(dispatch_dir: Path, q4_3: bool) -> dict[str, pd.DataFrame]:
15523     dates = export_dates()
15524     by_date = {}
15525     missing = []
15526     for date in dates:
15527         path = dispatch_dir / f"dispatch_{date}.csv"
15528         if not path.exists():
15529             missing.append(date)
15530             continue
15531         by_date[date] = _load_dispatch(path, q4_3)
15532     if missing:
15533         raise FileNotFoundError(
15534             f"{dispatch_dir} missing {len(missing)} export-window days, first {missing[:5]}"
15535         )
15536     return by_date
15537
15538
15539 def _write_notes(wb, title: str, lines: list[str]) -> None:
15540     if NOTES_SHEET in wb.sheetnames:
15541         del wb[NOTES_SHEET]
15542     ws = wb.create_sheet(NOTES_SHEET)
15543     font = Font(name="宋体", size=11)
15544     for i, line in enumerate([title, *lines], start=1):
15545         cell = ws.cell(i, 1)
15546         cell.value = line
15547         cell.font = font
15548         cell.alignment = Alignment(wrap_text=True, vertical="top")
15549         ws.row_dimensions[i].height = 22
15550     ws.column_dimensions["A"].width = 118
15551
15552
15553 def _label_fee_columns(wb, sheets: tuple[str, ...]) -> None:
15554     for name in sheets:
15555         ws = wb[name]
15556         ws.cell(1, COL_DAY_QTY).value = "全天购电量"
15557         ws.cell(1, COL_DAY_FEE).value = "全天购电费"
15558         ws.column_dimensions[get_column_letter(COL_DAY_QTY)].width = 14
15559         ws.column_dimensions[get_column_letter(COL_DAY_FEE)].width = 14
15560

```

```

15561
15562 def write_result4_2(by_date: dict[str, pd.DataFrame], dest: Path = RESULT4_2_XLSX) -> dict:
15563     if not RESULT2_LAYOUT.exists():
15564         raise FileNotFoundError(f"result2 layout not found: {RESULT2_LAYOUT}")
15565     dest.parent.mkdir(parents=True, exist_ok=True)
15566     shutil.copyfile(RESULT2_LAYOUT, dest)
15567     wb = load_workbook(dest)
15568     plan = wb[PURCHASE_SHEET]
15569     headers = _time_headers(plan)
15570     _fill_energy_grid(plan, by_date, "g0_kwh", "plan_fee_yuan")
15571     _rewrite_charge_sheet(wb[CHARGE_SHEET], by_date)
15572     n_emergency = _rewrite_emergency_sheet(wb[EMERGENCY_SHEET], by_date, headers)
15573     export_cost = float(sum(float(frame["actual_fee_yuan"].sum()) for frame in by_date.values()))
15574     annual = float(pd.read_csv(OUTPUT_DIR / "q4_2_warmup_daily.csv")["total_cost_yuan"].sum())
15575     _write_notes(
15576         wb,
15577         "C 题第 4 问 Q4-2 正式导出 (日前 q + 因果 MPC)",
15578         [
15579             "额度 q 全日锁定; 路径内充放电只用于评估 q, 不直接执行。",
15580             "负荷/光伏: Q2 因果口径; 价格: 已验收 P1 模块; 结算: 附件4 实际电价 p*q+5*p*e。",
15581             "年末 SOC: 继承 Q2 已验收规则 (最后一日无硬约束)。load_information_case=causal_load_main。",
15582             "输出区间: 2025-02-01 至 2025-12-31 (334 天)。1 月只用于预热与 SOC 连续, 不写入本表。",
15583             "单位: kWh 与元。时间列按附件 00:10 ... 0:00+1 原顺序。",
15584             f"紧急购电量仅列出 e>0 的时段 (本文件 {n_emergency} 行)。",
15585             f"数据来源 commit: {git_hash()}",
15586             f"2-12 月实际结算合计 {export_cost:.2f} 元; 1-12 月年度合计 {annual:.2f} 元, 二者不可混用。",
15587             "本文件不含 Q4-3、oracle 或未验收价格。未改写 result2.xlsx / result3.xlsx。",
15588         ],
15589     )
15590     _label_fee_columns(wb, (PURCHASE_SHEET,))
15591     if ADJUST_SHEET in wb.sheetnames:
15592         del wb[ADJUST_SHEET]
15593     wb.save(dest)
15594     return {
15595         "path": str(dest),
15596         "n_export_days": len(by_date),
15597         "n_emergency_rows": n_emergency,
15598         "export_window_cost_yuan": export_cost,
15599         "annual_cost_yuan": annual,
15600     }
15601
15602
15603 def write_result4_3(by_date: dict[str, pd.DataFrame], dest: Path = RESULT4_3_XLSX) -> dict:
15604     if not RESULT3_TEMPLATE.exists():
15605         raise FileNotFoundError(f"result3 template not found: {RESULT3_TEMPLATE}")
15606     dest.parent.mkdir(parents=True, exist_ok=True)
15607     shutil.copyfile(RESULT3_TEMPLATE, dest)
15608     wb = load_workbook(dest)
15609     plan = wb[PURCHASE_SHEET]
15610     adjust = wb[ADJUST_SHEET]
15611     headers = _time_headers(plan)
15612     _fill_energy_grid(plan, by_date, "g0_kwh", "plan_fee_yuan")
15613     _fill_energy_grid(adjust, by_date, "g_final_kwh", "actual_fee_yuan")
15614     _rewrite_charge_sheet(wb[CHARGE_SHEET], by_date)
15615     n_emergency = _rewrite_emergency_sheet(wb[EMERGENCY_SHEET], by_date, headers)
15616     export_cost = float(sum(float(frame["actual_fee_yuan"].sum()) for frame in by_date.values()))
15617     annual = float(pd.read_csv(OUTPUT_DIR / "q4_3_warmup_daily.csv")["total_cost_yuan"].sum())
15618     _write_notes(
15619         wb,
15620         "C 题第 4 问 Q4-3 正式导出 (固定 M1_M6)",
15621         [
15622             "策略固定为 M1_M6: 0:00 制定 g0; 6:00/12:00/18:00 只改未执行承诺。",
15623             "预测层使用因果价格; 账本按交付时实际电价  $\phi = p * g + 0.5 * p * |g - g_0|$ , 另加  $5 * p * e$ 。",
15624             "年末 SOC: A / A.q2_aligned / 1200 kWh。load_information_case=causal_load_main。",
15625             "输出区间: 2025-02-01 至 2025-12-31 (334 天)。1 月只用于预热与 SOC 连续, 不写入本表。",
15626             "计划购电量: g0。调整购电量: gF; 未调整时段等于 g0。",
15627             f"紧急购电量仅列出 e>0 的时段 (本文件 {n_emergency} 行)。",
15628             f"数据来源 commit: {git_hash()}",

```

```

15629         f"2-12 月实际结算合计 {export_cost:.2f} 元; 1-12 月年度合计 {annual:.2f} 元, 二者不可混用。",
15630         "本文件不含其它策略或 oracle。未改写 result3.xlsx。",
15631     ],
15632 )
15633 _label_fee_columns(wb, (PURCHASE_SHEET, ADJUST_SHEET))
15634 wb.save(dest)
15635 return {
15636     "path": str(dest),
15637     "n_export_days": len(by_date),
15638     "n_emergency_rows": n_emergency,
15639     "export_window_cost_yuan": export_cost,
15640     "annual_cost_yuan": annual,
15641 }
15642
15643
15644 def export_q4_2() -> dict:
15645     q42 = load_export_window(Q4_2_DISPATCH_DIR, q4_3=False)
15646     r2 = write_result4_2(q42)
15647     RESULT4_2_AUDIT.write_text(json.dumps(r2, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
15648     return {"result4_2": r2}
15649
15650
15651 def export_q4_3() -> dict:
15652     q43 = load_export_window(Q4_3_DISPATCH_DIR, q4_3=True)
15653     r3 = write_result4_3(q43)
15654     RESULT4_3_AUDIT.write_text(json.dumps(r3, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
15655     return {"result4_3": r3}
15656
15657
15658 def export_all() -> dict:
15659     out = {}
15660     out.update(export_q4_2())
15661     out.update(export_q4_3())
15662     return out
15663
15664
15665 if __name__ == "__main__":
15666     parser = argparse.ArgumentParser(description="Export result4 workbooks")
15667     parser.add_argument("--system", choices=("q4_2", "q4_3", "all"), default="all")
15668     args = parser.parse_args()
15669     if args.system == "q4_2":
15670         report = export_q4_2()
15671     elif args.system == "q4_3":
15672         report = export_q4_3()
15673     else:
15674         report = export_all()
15675     print(json.dumps(report, ensure_ascii=False, indent=2))
15676
15677 # ===== q4/info_set.py =====
15678 """Information-set probes: future actuals must not change day-ahead or alpha calibration."""
15679
15680 from __future__ import annotations
15681
15682 import numpy as np
15683
15684 from q2.data import Q2Data
15685 from q2.forecast import build_forecast_archive
15686 from q4.bundle import Q4Bundle
15687 from q4.data import Q4PriceData
15688 from q4.price_forecast import fit_causal_price_forecasts
15689 from q4.q4_2 import plan_locked_q, select_risk_alpha
15690 from q4.q4_3 import plan_g0
15691
15692
15693 def perturb_unended_and_future(
15694     prices: np.ndarray, day_index: int, tau: int, delta: float = 3.25
15695 ) -> np.ndarray:
15696     out = np.asarray(prices, dtype=float).copy()

```

```

15697     out[day_index, tau:] = out[day_index, tau:] + delta
15698     if day_index + 1 < len(out):
15699         out[day_index + 1 :] = out[day_index + 1 :] + delta
15700     return np.maximum(out, 0.0)
15701
15702
15703 def bundle_with_prices(bundle: Q4Bundle, new_prices: np.ndarray) -> Q4Bundle:
15704     archive = fit_causal_price_forecasts(
15705         new_prices, bundle.prices.dates, compute_mpc=False
15706     )
15707     prices = Q4PriceData(
15708         dates=bundle.prices.dates,
15709         time_labels=bundle.prices.time_labels,
15710         price=np.asarray(new_prices, dtype=float),
15711     )
15712     return Q4Bundle(
15713         q3=bundle.q3,
15714         q2=bundle.q2,
15715         q2_forecast=bundle.q2_forecast,
15716         prices=prices,
15717         price_archive=archive,
15718     )
15719
15720
15721 def probe_day_ahead_invariance(
15722     bundle: Q4Bundle,
15723     day_index: int,
15724     initial_soc: float,
15725     risk_alpha: float | None,
15726     tau: int = 40,
15727     with_value_cuts: bool = False,
15728 ) -> dict:
15729     date = bundle.prices.dates[day_index].strftime("%Y-%m-%d")
15730     q0 = plan_locked_q(
15731         bundle, day_index, initial_soc, risk_alpha, with_value_cuts=with_value_cuts
15732     )
15733     g0 = plan_g0(bundle, day_index, initial_soc, with_value_cuts=with_value_cuts)
15734     perturbed = perturb_unended_and_future(bundle.prices.price, day_index, tau)
15735     other = bundle_with_prices(bundle, perturbed)
15736     q1 = plan_locked_q(
15737         other, day_index, initial_soc, risk_alpha, with_value_cuts=with_value_cuts
15738     )
15739     g1 = plan_g0(other, day_index, initial_soc, with_value_cuts=with_value_cuts)
15740     q_ok = bool(np.allclose(q0, q1, atol=1e-6, rtol=0.0))
15741     g_ok = bool(np.allclose(g0, g1, atol=1e-6, rtol=0.0))
15742     return {
15743         "date": date,
15744         "day_index": int(day_index),
15745         "tau_perturbed": int(tau),
15746         "with_value_cuts": with_value_cuts,
15747         "q_max_abs_diff_kwh": float(np.max(np.abs(q0 - q1))),
15748         "g0_max_abs_diff_kwh": float(np.max(np.abs(g0 - g1))),
15749         "q_invariant": q_ok,
15750         "g0_invariant": g_ok,
15751         "pass": q_ok and g_ok,
15752     }
15753
15754
15755 def probe_executed_prefix(
15756     bundle: Q4Bundle,
15757     day_index: int,
15758     initial_soc: float,
15759     risk_alpha: float | None,
15760     q42_dispatch,
15761     q43_result,
15762     tau: int = 40,
15763 ) -> dict:
15764     """Re-solve the day after perturbing unended/future prices; prefix actions must match."""

```

```

15765 from q4.q4_2 import run_q4_2_day
15766 from q4.q4_3 import run_q4_3_day
15767
15768 date = bundle.prices.dates[day_index].strftime("%Y-%m-%d")
15769 other = bundle_with_prices(
15770     bundle, perturb_unended_and_future(bundle.prices.price, day_index, tau)
15771 )
15772 r2 = run_q4_2_day(other, day_index, initial_soc, risk_alpha)
15773 soc_q43 = float(q43_result.summary["soc_start_kwh"])
15774 r3 = run_q4_3_day(other, day_index, soc_q43)
15775 cols = ["q_or_g0_kwh", "x_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh"]
15776 d2 = q42_dispatch.iloc[:tau][cols].to_numpy(float)
15777 d2p = r2.dispatch.iloc[:tau][cols].to_numpy(float)
15778 q42_ok = bool(np.allclose(d2, d2p, atol=1e-5, rtol=0.0)) and bool(
15779     np.allclose(q42_dispatch["q_or_g0_kwh"].to_numpy(), r2.q, atol=1e-6)
15780 )
15781 g_cols = ["q_or_g0_kwh", "g_final_kwh", "x_kwh", "charge_kwh", "discharge_kwh"]
15782 d3 = q43_result.dispatch.iloc[:tau][g_cols].to_numpy(float)
15783 d3p = r3.dispatch.iloc[:tau][g_cols].to_numpy(float)
15784 q43_ok = bool(np.allclose(d3, d3p, atol=1e-5, rtol=0.0)) and bool(
15785     np.allclose(q43_result.g0, r3.g0, atol=1e-6)
15786 )
15787 return {
15788     "date": date,
15789     "tau_perturbed": int(tau),
15790     "q42_q_max_abs_diff_kwh": float(
15791         np.max(np.abs(q42_dispatch["q_or_g0_kwh"].to_numpy() - r2.q))
15792     ),
15793     "q42_prefix_max_abs_diff": float(np.max(np.abs(d2 - d2p))),
15794     "q43_g0_max_abs_diff_kwh": float(np.max(np.abs(q43_result.g0 - r3.g0))),
15795     "q43_prefix_max_abs_diff": float(np.max(np.abs(d3 - d3p))),
15796     "q42_pass": q42_ok,
15797     "q43_pass": q43_ok,
15798     "pass": q42_ok and q43_ok,
15799 }
15800
15801
15802 def perturb_future_actuals(
15803     bundle: Q4Bundle,
15804     from_day: int,
15805     *,
15806     delta_price: float = 3.25,
15807     load_scale: float = 1.15,
15808     pv_scale: float = 0.85,
15809 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
15810     """Shift actual price/load/PV on from_day and later; leave completed days intact."""
15811     prices = np.asarray(bundle.prices.price, dtype=float).copy()
15812     load = np.asarray(bundle.q2.load, dtype=float).copy()
15813     pv = np.asarray(bundle.q2.pv, dtype=float).copy()
15814     prices[from_day:] = np.maximum(prices[from_day:] + delta_price, 0.0)
15815     load[from_day:] = np.maximum(load[from_day:] * load_scale, 0.0)
15816     pv[from_day:] = np.maximum(pv[from_day:] * pv_scale, 0.0)
15817     return prices, load, pv
15818
15819
15820 def bundle_with_future_actuals(
15821     bundle: Q4Bundle,
15822     from_day: int,
15823     *,
15824     delta_price: float = 3.25,
15825     load_scale: float = 1.15,
15826     pv_scale: float = 0.85,
15827 ) -> Q4Bundle:
15828     prices, load, pv = perturb_future_actuals(
15829         bundle,
15830         from_day,
15831         delta_price=delta_price,
15832         load_scale=load_scale,

```

```

15833         pv_scale=pv_scale,
15834     )
15835     q2 = Q2Data(
15836         dates=bundle.q2.dates,
15837         time_labels=bundle.q2.time_labels,
15838         price=bundle.q2.price,
15839         fallback_load=bundle.q2.fallback_load,
15840         fallback_pv=bundle.q2.fallback_pv,
15841         load=load,
15842         pv=pv,
15843     )
15844     archive = fit_causal_price_forecasts(
15845         prices, bundle.prices.dates, compute_mpc=False
15846     )
15847     return Q4Bundle(
15848         q3=bundle.q3,
15849         q2=q2,
15850         q2_forecast=build_forecast_archive(q2),
15851         prices=Q4PriceData(
15852             dates=bundle.prices.dates,
15853             time_labels=bundle.prices.time_labels,
15854             price=prices,
15855         ),
15856         price_archive=archive,
15857     )
15858
15859
15860 def probe_alpha_calibration_invariance(
15861     bundle: Q4Bundle,
15862     policy_start_soc: np.ndarray,
15863     target_index: int,
15864     *,
15865     k_target: int = 8,
15866     n_validation: int = 1,
15867     alphas: tuple[float, ...] = (0.60, 0.90),
15868 ) -> dict:
15869     """Perturb actuals of the calibration day and later; alpha scores must not move."""
15870     baseline = select_risk_alpha(
15871         bundle,
15872         policy_start_soc,
15873         target_index,
15874         k_target,
15875         n_validation=n_validation,
15876         alphas=alphas,
15877         log=False,
15878     )
15879     other = bundle_with_future_actuals(bundle, target_index)
15880     perturbed = select_risk_alpha(
15881         other,
15882         policy_start_soc,
15883         target_index,
15884         k_target,
15885         n_validation=n_validation,
15886         alphas=alphas,
15887         log=False,
15888     )
15889     base_costs = {
15890         float(row["risk_alpha"]): float(row["mean_validation_cost_yuan"])
15891         for row in baseline.window_records
15892     }
15893     other_costs = {
15894         float(row["risk_alpha"]): float(row["mean_validation_cost_yuan"])
15895         for row in perturbed.window_records
15896     }
15897     cost_gaps = {
15898         alpha: abs(base_costs[alpha] - other_costs[alpha]) for alpha in base_costs
15899     }
15900     max_cost_gap = float(max(cost_gaps.values())) if cost_gaps else 0.0

```

```

15901     alpha_ok = abs(baseline.selected_alpha - perturbed.selected_alpha) < 1e-12
15902     cost_ok = max_cost_gap < 1e-6
15903     return {
15904         "target_index": int(target_index),
15905         "n_validation": int(n_validation),
15906         "alphas": [float(a) for a in alphas],
15907         "baseline_selected_alpha": float(baseline.selected_alpha),
15908         "perturbed_selected_alpha": float(perturbed.selected_alpha),
15909         "max_mean_cost_abs_diff_yuan": max_cost_gap,
15910         "mean_cost_abs_diff_by_alpha": cost_gaps,
15911         "alpha_invariant": alpha_ok,
15912         "cost_invariant": cost_ok,
15913         "no_future_info": True,
15914         "used_full_day_actual_scheduler": False,
15915         "pass": alpha_ok and cost_ok,
15916     }
15917
15918 # ===== q4/optimization.py =====
15919 """Q4 LPs: scenario-priced day-ahead q and remaining-horizon dispatch."""
15920
15921 from __future__ import annotations
15922
15923 from dataclasses import dataclass
15924 from time import perf_counter
15925
15926 import cvxpy as cp
15927 import numpy as np
15928
15929 from q2.config import (
15930     EMERGENCY_PRICE_MULTIPLIER,
15931     E_MAX_KWH,
15932     E_MIN_KWH,
15933     ETA_C,
15934     ETA_D,
15935     MPC_COST_TOL,
15936     POWER_LIMIT_KWH,
15937     SOLVER,
15938 )
15939 from q2.optimization import ValueCut, evaluate_value_cuts
15940
15941
15942 @dataclass(frozen=True)
15943 class DispatchResult:
15944     x: np.ndarray
15945     emergency: np.ndarray
15946     charge: np.ndarray
15947     discharge: np.ndarray
15948     curtailment: np.ndarray
15949     soc: np.ndarray
15950     emergency_cost: float
15951     terminal_value: float
15952     solve_seconds: float
15953     status: str
15954
15955     @property
15956     def max_cd(self) -> float:
15957         return float(np.max(self.charge * self.discharge))
15958
15959
15960 @dataclass(frozen=True)
15961 class PlanResult:
15962     q: np.ndarray
15963     expected_normal_cost: float
15964     expected_emergency_cost: float
15965     expected_terminal_value: float
15966     initial_soc_marginal: float
15967     solve_seconds: float
15968     status: str

```

```

15969
15970
15971 def _terminal_value_expression(
15972     terminal_soc: cp.Expression, cuts: tuple[ValueCut, ...] | None
15973 ) -> tuple[cp.Variable | None, list[cp.Constraint]]:
15974     if not cuts:
15975         return None, []
15976     value = cp.Variable(nonneg=True)
15977     constraints = [
15978         value >= cut_value + slope * (terminal_soc - reference_soc)
15979         for reference_soc, cut_value, slope in cuts
15980     ]
15981     return value, constraints
15982
15983
15984 def _physical_constraints(
15985     load: np.ndarray,
15986     pv: np.ndarray,
15987     initial_soc: float,
15988     normal: cp.Variable,
15989     emergency: cp.Variable,
15990     charge: cp.Variable,
15991     discharge: cp.Variable,
15992     curtailment: cp.Variable,
15993     soc: cp.Variable,
15994 ) -> list[cp.Constraint]:
15995     return [
15996         soc[0] == initial_soc,
15997         soc[1:] == soc[:-1] + ETA_C * charge - discharge / ETA_D,
15998         soc[1:] >= E_MIN_KWH,
15999         soc[1:] <= E_MAX_KWH,
16000         charge <= POWER_LIMIT_KWH,
16001         discharge <= POWER_LIMIT_KWH,
16002         curtailment <= pv,
16003         normal + emergency + pv - curtailment + discharge == load + charge,
16004     ]
16005
16006
16007 def solve_stochastic_plan_varying_price(
16008     scenario_price: np.ndarray,
16009     scenario_load: np.ndarray,
16010     scenario_pv: np.ndarray,
16011     probabilities: np.ndarray,
16012     initial_soc: float,
16013     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
16014     q_floor: np.ndarray | None = None,
16015     terminal_soc: float | None = None,
16016 ) -> PlanResult:
16017     k, n = scenario_load.shape
16018     if scenario_price.shape != (k, n) or scenario_pv.shape != (k, n):
16019         raise ValueError("scenario price/load/PV shapes must match")
16020     q = cp.Variable(n, nonneg=True)
16021     expected_normal: cp.Expression | float = 0.0
16022     expected_emergency: cp.Expression | float = 0.0
16023     expected_terminal: cp.Expression | float = 0.0
16024     constraints: list[cp.Constraint] = []
16025     if q_floor is not None:
16026         floor = np.asarray(q_floor, dtype=float).ravel()
16027         if floor.shape != (n,) or np.any(floor < -1e-9):
16028             raise ValueError("q_floor must be a nonnegative vector matching the horizon")
16029         constraints.append(q >= floor)
16030     initial_constraints: list[cp.Constraint] = []
16031     for omega in range(k):
16032         x, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
16033         E = cp.Variable(n + 1)
16034         constraints.append(x <= q)
16035         physical = _physical_constraints(
16036             scenario_load[omega],

```



```

16037         scenario_pv[omega],
16038         initial_soc,
16039         x,
16040         e,
16041         c,
16042         d,
16043         w,
16044         E,
16045     )
16046     constraints += physical
16047     initial_constraints.append(physical[0])
16048     if terminal_soc is not None:
16049         constraints.append(E[-1] == float(terminal_soc))
16050     terminal_value, terminal_constraints = _terminal_value_expression(
16051         E[-1], terminal_value_cuts
16052     )
16053     constraints += terminal_constraints
16054     price = np.asarray(scenario_price[omega], dtype=float).ravel()
16055     expected_normal += probabilities[omega] * (price @ q)
16056     expected_emergency += probabilities[omega] * EMERGENCY_PRICE_MULTIPLIER * (price @ e)
16057     if terminal_value is not None:
16058         expected_terminal += probabilities[omega] * terminal_value
16059     problem = cp.Problem(
16060         cp.Minimize(expected_normal + expected_emergency + expected_terminal),
16061         constraints,
16062     )
16063     started = perf_counter()
16064     problem.solve(solver=SOLVER, verbose=False)
16065     elapsed = perf_counter() - started
16066     if q.value is None:
16067         raise RuntimeError(f"Q4-2 stochastic plan LP failed: {problem.status}")
16068     qv = np.asarray(q.value).ravel()
16069     marginal = -float(
16070         sum(float(np.asarray(constraint.dual_value)) for constraint in initial_constraints)
16071     )
16072     terminal_value_result = (
16073         float(expected_terminal.value)
16074         if isinstance(expected_terminal, cp.Expression)
16075         else float(expected_terminal)
16076     )
16077     expected_normal_v = float(sum(probabilities[o] * (scenario_price[o] @ qv) for o in range(k)))
16078     return PlanResult(
16079         q=qv,
16080         expected_normal_cost=expected_normal_v,
16081         expected_emergency_cost=float(problem.value - expected_normal_v - terminal_value_result),
16082         expected_terminal_value=terminal_value_result,
16083         initial_soc_marginal=marginal,
16084         solve_seconds=elapsed,
16085         status=str(problem.status),
16086     )
16087
16088
16089 def solve_fixed_q_dispatch(
16090     price: np.ndarray,
16091     q: np.ndarray,
16092     load: np.ndarray,
16093     pv: np.ndarray,
16094     initial_soc: float,
16095     throughput_tiebreak: bool = True,
16096     terminal_value_cuts: tuple[ValueCut, ...] | None = None,
16097     terminal_soc: float | None = None,
16098 ) -> DispatchResult:
16099     n = len(load)
16100     x, e, c, d, w = (cp.Variable(n, nonneg=True) for _ in range(5))
16101     E = cp.Variable(n + 1)
16102     constraints = [x <= q]
16103     constraints += _physical_constraints(load, pv, initial_soc, x, e, c, d, w, E)
16104     if terminal_soc is not None:

```

```

16105         constraints.append(E[-1] == float(terminal_soc))
16106     emergency_cost = EMERGENCY_PRICE_MULTIPLIER * price @ e
16107     terminal_value, terminal_constraints = _terminal_value_expression(E[-1], terminal_value_cuts)
16108     constraints += terminal_constraints
16109     primary = emergency_cost + (terminal_value if terminal_value is not None else 0.0)
16110     first = cp.Problem(cp.Minimize(primary), constraints)
16111     started = perf_counter()
16112     first.solve(solver=SOLVER, verbose=False)
16113     if e.value is None:
16114         raise RuntimeError(f"Q4-2 fixed-q dispatch LP failed: {first.status}")
16115     primary_opt = float(primary.value)
16116     status = str(first.status)
16117     if throughput_tiebreak:
16118         second = cp.Problem(
16119             cp.Minimize(cp.sum(c + d)),
16120             constraints + [primary <= primary_opt + MPC_COST_TOL],
16121         )
16122         second.solve(solver=SOLVER, verbose=False)
16123         if x.value is None:
16124             raise RuntimeError(f"Q4-2 tiebreak LP failed: {second.status}")
16125         status = str(second.status)
16126     elapsed = perf_counter() - started
16127     return DispatchResult(
16128         x=np.asarray(x.value).ravel(),
16129         emergency=np.asarray(e.value).ravel(),
16130         charge=np.asarray(c.value).ravel(),
16131         discharge=np.asarray(d.value).ravel(),
16132         curtailment=np.asarray(w.value).ravel(),
16133         soc=np.asarray(E.value).ravel(),
16134         emergency_cost=float(EMERGENCY_PRICE_MULTIPLIER * price @ e.value),
16135         terminal_value=evaluate_value_cuts(float(E.value[-1]), terminal_value_cuts),
16136         solve_seconds=elapsed,
16137         status=status,
16138     )
16139
16140 # ===== q4/plot_final_figures.py =====
16141 #!/usr/bin/env python3
16142 """Render Q4 final figures from signed-off result artifacts only. Does not re-solve."""
16143
16144 from __future__ import annotations
16145
16146 import hashlib
16147 import json
16148 import sys
16149 from pathlib import Path
16150
16151 import matplotlib.dates as mdates
16152 import matplotlib.pyplot as plt
16153 from matplotlib import font_manager
16154 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
16155 from matplotlib.ticker import FuncFormatter
16156 import numpy as np
16157 import pandas as pd
16158
16159 ROOT = Path(__file__).resolve().parents[1]
16160 if str(ROOT) not in sys.path:
16161     sys.path.insert(0, str(ROOT))
16162
16163 from q4.closeout import (
16164     build_fair_comparison,
16165     load_q2_k8_daily,
16166     load_q3_official_daily,
16167     load_q4_2_daily,
16168     load_q4_3_daily,
16169     output_interval,
16170     q4_2_policy_windows,
16171 )
16172 from q4.config import (

```

```

16173     EXPECTED_Q4_2_ALPHA070_END,
16174     EXPECTED_Q4_2_ALPHA070_START,
16175     FIG_DIR,
16176     Q4_2_ALPHA_SELECTION_CSV,
16177     Q4_2_DAILY_CSV,
16178     Q4_2_DISPATCH_DIR,
16179     Q4_3_DAILY_CSV,
16180     Q4_3_DISPATCH_DIR,
16181     RESULT4_2_XLSX,
16182     RESULT4_3_XLSX,
16183 )
16184
16185 RESULT3 = ROOT / "output/result3.xlsx"
16186 Q2_K8_DAILY = ROOT / "output/q2_full_k8_risk/daily_summary.csv"
16187 Q3_DAILY = ROOT / "output/q3_full_annual/q3_annual_daily_summary.csv"
16188
16189 BLUE = "#276FBF"
16190 RED = "#C0392B"
16191 GRAY = "#6B7280"
16192 BLACK = "#1F2933"
16193 LIGHT_BLUE = "#E8F1FA"
16194 LIGHT_RED = "#F8E7E5"
16195 LIGHT_GRAY = "#F3F4F6"
16196
16197
16198 def file_hash(path: Path) -> str:
16199     digest = hashlib.sha256()
16200     with path.open("rb") as stream:
16201         for block in iter(lambda: stream.read(1024 * 1024), b""):
16202             digest.update(block)
16203     return digest.hexdigest()
16204
16205
16206 def configure_style() -> str:
16207     available = {font.name for font in font_manager.fontManager.ttflist}
16208     candidates = (
16209         "PingFang SC",
16210         "Hiragino Sans GB",
16211         "Heiti SC",
16212         "Songti SC",
16213         "Arial Unicode MS",
16214         "Noto Sans CJK SC",
16215         "WenQuanYi Micro Hei",
16216         "Droid Sans Fallback",
16217     )
16218     font = next((name for name in candidates if name in available), None)
16219     if font is None:
16220         raise RuntimeError("No usable Chinese font found")
16221     plt.rcParams.update(
16222         {
16223             "font.family": "sans-serif",
16224             "font.sans-serif": [font],
16225             "axes.unicode_minus": False,
16226             "figure.facecolor": "white",
16227             "axes.facecolor": "white",
16228             "font.size": 10,
16229             "axes.titlesize": 12,
16230             "axes.labelsize": 10,
16231             "legend.fontsize": 9,
16232             "axes.titleweight": "normal",
16233             "pdf.fonttype": 42,
16234         }
16235     )
16236     return font
16237
16238
16239 def save_figure(fig: plt.Figure, stem: str) -> None:
16240     FIG_DIR.mkdir(parents=True, exist_ok=True)

```

```

16241     fig.savefig(FIG_DIR / f"{stem}.pdf", bbox_inches="tight")
16242     fig.savefig(FIG_DIR / f"{stem}.png", dpi=320, bbox_inches="tight")
16243     plt.close(fig)
16244
16245
16246 def draw_box(
16247     ax: plt.Axes,
16248     x: float,
16249     y: float,
16250     width: float,
16251     height: float,
16252     text: str,
16253     facecolor: str,
16254 ) -> None:
16255     patch = FancyBboxPatch(
16256         (x, y),
16257         width,
16258         height,
16259         boxstyle="round,pad=0.012,rounding_size=0.02",
16260         linewidth=1.15,
16261         edgecolor=BLACK,
16262         facecolor=facecolor,
16263     )
16264     ax.add_patch(patch)
16265     ax.text(x + width / 2, y + height / 2, text, ha="center", va="center", linespacing=1.38)
16266
16267
16268 def style_axes(ax: plt.Axes, hide_right: bool = True) -> None:
16269     ax.spines["top"].set_visible(False)
16270     if hide_right:
16271         ax.spines["right"].set_visible(False)
16272     for side in ("left", "bottom"):
16273         ax.spines[side].set_color(GRAY)
16274         ax.spines[side].set_linewidth(0.8)
16275     ax.tick_params(color=GRAY, length=3.5)
16276
16277
16278 def draw_arrow(
16279     ax: plt.Axes,
16280     start: tuple[float, float],
16281     end: tuple[float, float],
16282     label: str,
16283     label_y: float,
16284 ) -> None:
16285     ax.add_patch(
16286         FancyArrowPatch(
16287             start,
16288             end,
16289             arrowstyle="-|>",
16290             mutation_scale=12,
16291             linewidth=1.25,
16292             color=BLACK,
16293             shrinkA=2,
16294             shrinkB=2,
16295         )
16296     )
16297     ax.text(
16298         (start[0] + end[0]) / 2,
16299         label_y,
16300         label,
16301         ha="center",
16302         va="bottom",
16303         fontsize=8.8,
16304         color=BLACK,
16305         bbox={"boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.95},
16306     )
16307
16308

```

```

16309 def plot_q4_2_loop() -> None:
16310     fig, ax = plt.subplots(figsize=(15.6, 6.5))
16311     ax.set_xlim(0, 1)
16312     ax.set_ylim(0, 1)
16313     ax.axis("off")
16314     y, width, height = 0.48, 0.148, 0.30
16315     gap = 0.055
16316     xs = [0.020 + index * (width + gap) for index in range(5)]
16317     texts = [
16318         "历史负荷—光伏—价格\n配对残差\n（仅已结束日）",
16319         "日前 8 个联合情景\n风险分位购电下限",
16320         "0:00 锁定\n计划购电量 $q$",
16321         "每 10 分钟因果 MPC\n更新 $x,c,d,e$\n当天 $q$ 不可改",
16322         "实际日末 SOC\n$E_{d,144}$\n传至下一日",
16323     ]
16324     faces = [LIGHT_GRAY, LIGHT_BLUE, LIGHT_BLUE, LIGHT_RED, LIGHT_GRAY]
16325     for x, text, face in zip(xs, texts, faces):
16326         draw_box(ax, x, y, width, height, text, face)
16327     labels = ["构造因果预测", "0:00 决策", "当天 $q$ 锁定", "仅传递实际 SOC"]
16328     for index in range(4):
16329         draw_arrow(
16330             ax,
16331             (xs[index] + width, y + height / 2),
16332             (xs[index + 1], y + height / 2),
16333             labels[index],
16334             y + height + 0.035,
16335         )
16336     mid_y = 0.30
16337     ax.plot(
16338         [xs[4] + width / 2, xs[4] + width / 2, xs[1] + width / 2, xs[1] + width / 2],
16339         [y, mid_y, mid_y, y],
16340         color=BLUE,
16341         linestyle="--",
16342         linewidth=1.25,
16343         solid_capstyle="round",
16344     )
16345     ax.annotate(
16346         "",
16347         xy=(xs[1] + width / 2, y),
16348         xytext=(xs[1] + width / 2, mid_y + 0.01),
16349         arrowprops={"arrowstyle": "-|>", "color": BLUE, "lw": 1.25},
16350     )
16351     ax.text(
16352         (xs[1] + xs[4] + width) / 2,
16353         mid_y - 0.055,
16354         "次日 0:00 用新的因果价格预测重新锁定自己的 $q$; 不提前锁定次日计划",
16355         ha="center",
16356         color=BLUE,
16357         fontsize=9.2,
16358     )
16359     ax.text(
16360         0.5,
16361         0.09,
16362         "日内只修正储能与实际取用; 紧急购电 $e$ 只补当期缺口。结算按交付时点附件4 实际电价。",
16363         ha="center",
16364         color=RED,
16365         fontsize=9.2,
16366         bbox={"boxstyle": "round,pad=0.32", "facecolor": LIGHT_RED, "edgecolor": RED, "linewidth": 0.6},
16367     )
16368     ax.set_title("Q4-2 日前联合计划—日内因果 MPC—跨日 SOC 闭环", pad=10)
16369     save_figure(fig, "fig_q4_2_closed_loop")
16370
16371
16372 def plot_q4_3_clock() -> None:
16373     fig, ax = plt.subplots(figsize=(15.6, 6.5))
16374     ax.set_xlim(0, 1)
16375     ax.set_ylim(0, 1)
16376     ax.axis("off")

```

```

16377 y, width, height = 0.48, 0.148, 0.30
16378 gap = 0.055
16379 xs = [0.020 + index * (width + gap) for index in range(5)]
16380 texts = [
16381     "0:00 制定 $g^0$\n因果日前价格\n负荷/光伏主案例",
16382     "6:00 评估 V0I\n可改 06:10 起\n前缀 $g$ 锁定",
16383     "12:00 评估 V0I\n可改 12:10 起",
16384     "18:00 评估 V0I\n可改 18:10 起",
16385     "交付时点结算\n$\phi=p \ g^{-F}+0.5p|g^{-g^0}|\n加 \$5pe$",
16386 ]
16387 faces = [LIGHT_BLUE, LIGHT_GRAY, LIGHT_GRAY, LIGHT_GRAY, LIGHT_RED]
16388 for x, text, face in zip(xs, texts, faces):
16389     draw_box(ax, x, y, width, height, text, face)
16390 labels = ["主策略 M1_M6", "仅未执行段", "仅未执行段", "会计不改决策"]
16391 for index in range(4):
16392     draw_arrow(
16393         ax,
16394         (xs[index] + width, y + height / 2),
16395         (xs[index + 1], y + height / 2),
16396         labels[index],
16397         y + height + 0.035,
16398     )
16399 ax.text(
16400     0.5,
16401     0.22,
16402     "每个更新时刻用当时可得日内价格预测；次日价值割仍因果。价格神谕与更新时刻重记账不进入主方案。",
16403     ha="center",
16404     color=BLUE,
16405     fontsize=9.2,
16406 )
16407 ax.text(
16408     0.5,
16409     0.09,
16410     "6:00 前 $g^F=g^0$；紧急购电只补当期缺口。主图成本为交付时点附件4 电价，不是神谕对照。",
16411     ha="center",
16412     color=RED,
16413     fontsize=9.2,
16414     bbox={"boxstyle": "round,pad=0.32", "facecolor": LIGHT_RED, "edgecolor": RED, "linewidth": 0.6},
16415 )
16416 ax.set_title("Q4-3 M1_M6 0/6/12/18 更新时钟与前缀锁定", pad=10)
16417 save_figure(fig, "fig_q4_3_update_clock")
16418
16419
16420 def _stacked_pair(ax, names, planned, extra, emergency, extra_label: str | None) -> None:
16421     x = np.arange(len(names))
16422     width = 0.52
16423     total = planned + extra + emergency
16424     ax.bar(x, planned, width=width, color=BLUE, edgecolor=BLACK, linewidth=0.7, label="计划/普通购电费", zorder=3)
16425     if extra_label:
16426         ax.bar(
16427             x,
16428             extra,
16429             width=width,
16430             bottom=planned,
16431             color="white",
16432             edgecolor=GRAY,
16433             linewidth=1.05,
16434             hatch="xxx",
16435             label=extra_label,
16436             zorder=3,
16437         )
16438     ax.bar(
16439         x,
16440         emergency,
16441         width=width,
16442         bottom=planned + extra,
16443         color="white",
16444         edgecolor=RED,

```

```

16445         linewidth=1.15,
16446         hatch="///",
16447         label="紧急购电费",
16448         zorder=3,
16449     )
16450     for index, value in enumerate(total):
16451         ax.text(
16452             x[index],
16453             value + total.max() * 0.018,
16454             f"合计 {value:,.2f} 元",
16455             ha="center",
16456             va="bottom",
16457             fontsize=8.8,
16458             zorder=4,
16459             clip_on=False,
16460         )
16461         ax.text(
16462             x[index],
16463             planned[index] * 0.52,
16464             f"{planned[index] / 1e6:.2f} 百万",
16465             ha="center",
16466             va="center",
16467             color="white",
16468             fontsize=8.6,
16469             zorder=4,
16470         )
16471     ax.set_xticks(x, names)
16472     ax.set_ylabel("2—12 月购电成本 (元)")
16473     ax.yaxis.set_major_formatter(FuncFormatter(lambda value, _pos: f"{value / 1e6:.1f} 百万"))
16474     ax.set_ylim(0, total.max() * 1.16)
16475     ax.set_xlim(-0.55, len(names) - 0.45)
16476     ax.grid(axis="y", linestyle=":", color=GRAY, alpha=0.35, zorder=0)
16477     ax.set_axisbelow(True)
16478     style_axes(ax)
16479
16480
16481 def plot_cost_comparison(comparison: dict) -> dict:
16482     q2 = comparison["export"]["q2_k8"]
16483     q42 = comparison["export"]["q4_2"]
16484     q3 = comparison["export"]["q3_m1_m6"]
16485     q43 = comparison["export"]["q4_3"]
16486     fig, axes = plt.subplots(1, 2, figsize=(13.8, 6.8))
16487     _stacked_pair(
16488         axes[0],
16489         ["Q2 K=8\n附件1 电价", "Q4-2\n附件4 电价"],
16490         np.array([q2["normal_or_settlement_yuan"], q42["normal_or_settlement_yuan"]]),
16491         np.array([0.0, q42["adjustment_yuan"]]),
16492         np.array([q2["emergency_cost_yuan"], q42["emergency_cost_yuan"]]),
16493         extra_label=None,
16494     )
16495     axes[0].set_title("日前锁定系统", pad=10)
16496     _stacked_pair(
16497         axes[1],
16498         ["Q3 M1_M6\n附件1 电价", "Q4-3\n附件4 电价"],
16499         np.array([q3["normal_or_settlement_yuan"], q43["normal_or_settlement_yuan"]]),
16500         np.array([0.0, q43["adjustment_yuan"]]),
16501         np.array([q3["emergency_cost_yuan"], q43["emergency_cost_yuan"]]),
16502         extra_label="偏差调整费",
16503     )
16504     axes[1].set_title("滚动调整系统", pad=10)
16505     handles, labels = axes[1].get_legend_handles_labels()
16506     fig.legend(
16507         handles,
16508         labels,
16509         loc="lower center",
16510         ncol=3,
16511         frameon=False,
16512         bbox_to_anchor=(0.5, 0.10),

```

```

16513     )
16514     fig.suptitle("2—12 月成本构成：同一套设备，不同价格信息", y=0.98)
16515     fig.subplots_adjust(bottom=0.28, top=0.86, wspace=0.28)
16516     fig.text(
16517         0.5,
16518         0.045,
16519         "注：两侧都是 2025-02-01 至 2025-12-31 共 334 日。Q4 相对 Q2/Q3 的差额不是同一电价下的会计节省；",
16520         "Q4-3 主图不含价格神谕与更新时刻重记账。",
16521         ha="center",
16522         fontsize=9,
16523         color=GRAY,
16524     )
16525     save_figure(fig, "fig_q4_cost_comparison")
16526     return {
16527         "q2_k8_export_yuan": q2["total_cost_yuan"],
16528         "q4_2_export_yuan": q42["total_cost_yuan"],
16529         "q3_m1_m6_export_yuan": q3["total_cost_yuan"],
16530         "q4_3_export_yuan": q43["total_cost_yuan"],
16531         "q4_2_minus_q2_yuan": comparison["pairs"]["q4_2_minus_q2_k8_export_yuan"],
16532         "q4_3_minus_q3_yuan": comparison["pairs"]["q4_3_minus_q3_m1_m6_export_yuan"],
16533     }
16534
16535
16536 def _load_rep_day(daily: pd.DataFrame, dispatch_dir: Path) -> tuple[pd.Timestamp, pd.DataFrame, Path, dict]:
16537     output = output_interval(daily)
16538     selected = output.loc[output["emergency_cost_yuan"].idxmax()]
16539     date = pd.Timestamp(selected["date"])
16540     path = dispatch_dir / f"dispatch_{date:%Y-%m-%d}.csv"
16541     frame = pd.read_csv(path)
16542     if len(frame) != 144:
16543         raise AssertionError(f"{path.name} row count {len(frame)}")
16544     if not np.array_equal(frame["period_index"].to_numpy(), np.arange(144)):
16545         raise AssertionError("period_index order changed")
16546     if str(frame["time_label"].iloc[0]) != "00:10" or str(frame["time_label"].iloc[-1]) != "0:00+1":
16547         raise AssertionError("raw attachment time order mismatch")
16548     return date, frame, path, {
16549         "selected_date": date.strftime("%Y-%m-%d"),
16550         "selection_metric": "maximum emergency_cost_yuan over 2025-02-01..2025-12-31",
16551         "emergency_cost_yuan": float(selected["emergency_cost_yuan"]),
16552         "emergency_kwh": float(selected["emergency_kwh"]),
16553         "dispatch_rows": len(frame),
16554         "first_time": str(frame["time_label"].iloc[0]),
16555         "last_time": str(frame["time_label"].iloc[-1]),
16556         "source": str(path.relative_to(ROOT)),
16557     }
16558
16559
16560 def plot_representative_day(daily: pd.DataFrame, dispatch_dir: Path, title_prefix: str, stem: str) -> dict:
16561     date, frame, path, meta = _load_rep_day(daily, dispatch_dir)
16562     hours = (frame["period_index"].to_numpy() + 1) / 6.0
16563     net_load = frame["actual_load_kwh"].to_numpy() - frame["actual_pv_kwh"].to_numpy()
16564     battery_net = frame["discharge_kwh"].to_numpy() - frame["charge_kwh"].to_numpy()
16565     plan = frame["g_final_kwh"].to_numpy(float) if "g_final_kwh" in frame and frame["g_final_kwh"].notna().all() else frame["q_or_g0_kwh"].to_numpy(float)
16566     plan_label = "终承诺 $g~F$" if "g_final" in stem or title_prefix.startswith("Q4-3") else "锁定计划 $q$"
16567     price = frame["actual_price"].to_numpy(float)
16568
16569     fig, axes = plt.subplots(3, 1, figsize=(12.6, 11.0), sharex=True, gridspec_kw={"hspace": 0.16})
16570     legend_box = {
16571         "frameon": True,
16572         "framealpha": 0.96,
16573         "edgecolor": "#E5E7EB",
16574         "fancybox": False,
16575         "borderpad": 0.45,
16576         "handlelength": 2.2,
16577     }
16578     ax = axes[0]
16579     ax.plot(hours, net_load, color=BLACK, linewidth=1.35, label="实际净负荷 $L-P$")

```



```

16580 ax.plot(hours, plan, color=BLUE, linewidth=1.25, label=plan_label)
16581 ax.plot(hours, frame["x_kwh"], color=GRAY, linestyle="--", linewidth=1.15, label="实际普通取电 $x$")
16582 ax.plot(hours, frame["emergency_kwh"], color=RED, linestyle="-.", linewidth=1.45, label="紧急购电 $e$")
16583 ax.fill_between(hours, 0, frame["emergency_kwh"], color=RED, alpha=0.12, linewidth=0)
16584 ymax = max(net_load.max(), np.nanmax(plan), frame["emergency_kwh"].max())
16585 ax.set_ylim(-0.10 * ymax, ymax * 1.22)
16586 ax.set_ylabel("时段电量 (kWh) ")
16587 ax.legend(loc="upper left", ncol=2, **legend_box)
16588 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
16589 ax.set_axisbelow(True)
16590 style_axes(ax)
16591
16592 ax = axes[1]
16593 ax.axhline(0, color=GRAY, linewidth=0.8)
16594 ax.plot(hours, battery_net, color=BLACK, linewidth=1.2, label="电池净动作 $d-c$")
16595 ax.fill_between(hours, 0, battery_net, color=BLACK, alpha=0.10, linewidth=0)
16596 ax.set_ylabel("$d-c$ (kWh) ")
16597 abs_max = max(np.max(np.abs(battery_net)), 1e-9)
16598 ax.set_ylim(-abs_max * 1.28, abs_max * 1.28)
16599 ax_price = ax.twinx()
16600 ax_price.step(hours, price, where="mid", color=RED, linestyle="--", linewidth=1.15, label="实际电价")
16601 ax_price.set_ylabel("电价 (元/kWh) ", color=RED)
16602 ax_price.tick_params(axis="y", colors=RED)
16603 ax_price.set_ylim(0.0, max(price.max() * 1.22, 0.4))
16604 ax_price.spines["top"].set_visible(False)
16605 ax_price.spines["right"].set_color(RED)
16606 handles1, labels1 = ax.get_legend_handles_labels()
16607 handles2, labels2 = ax_price.get_legend_handles_labels()
16608 ax.legend(handles1 + handles2, labels1 + labels2, loc="lower left", **legend_box)
16609 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
16610 ax.set_axisbelow(True)
16611 style_axes(ax, hide_right=False)
16612
16613 ax = axes[2]
16614 ax.plot(hours, frame["soc_end_kwh"], color=BLUE, linewidth=1.5, label="SOC")
16615 ax.axhline(1200, color=GRAY, linestyle="--", linewidth=1.0, label="SOC 边界 1200 / 10800 kWh")
16616 ax.axhline(10800, color=GRAY, linestyle="--", linewidth=1.0)
16617 ax.set_ylabel("SOC (kWh) ")
16618 ax.set_xlabel("时刻")
16619 ax.set_ylim(200, 12800)
16620 ax.legend(loc="center left", **legend_box)
16621 ax.grid(linestyle=":", color=GRAY, alpha=0.32)
16622 ax.set_axisbelow(True)
16623 style_axes(ax)
16624 axes[-1].set_xlim(0, 24)
16625 axes[-1].set_xticks(np.arange(0, 25, 2))
16626 axes[-1].set_xticklabels([f"{hour:02d}:00" for hour in range(0, 25, 2)])
16627 fig.suptitle(f"{title_prefix}: {date:%Y-%m-%d} (2—12 月紧急费用最高日) ", y=0.98)
16628 fig.subplots_adjust(top=0.945, bottom=0.055, left=0.08, right=0.91, hspace=0.20)
16629 save_figure(fig, stem)
16630 return meta
16631
16632
16633 def plot_alpha_timeline(daily: pd.DataFrame) -> dict:
16634     selected = q4_2_policy_windows(daily)
16635     fig, ax = plt.subplots(figsize=(13.4, 6.2))
16636     y_map = {None: 0, 0.6: 1, 0.7: 2, 0.8: 3, 0.9: 4}
16637     ytick_labels = ["预热 / 无风险分位", "alpha = 0.60", "alpha = 0.70", "alpha = 0.80", "alpha = 0.90"]
16638     year_start = mdates.date2num(pd.Timestamp("2025-01-01").to_pydatetime())
16639     year_end = mdates.date2num(pd.Timestamp("2026-01-01").to_pydatetime())
16640     for y in range(5):
16641         ax.barh(y, year_end - year_start, left=year_start, height=0.62, color=LIGHT_GRAY, edgecolor="none", zorder=0)
16642     ax.axvspan(pd.Timestamp("2025-01-01"), pd.Timestamp("2025-01-29"), color=GRAY, alpha=0.10, zorder=1)
16643     for row in selected.iter_tuples():
16644         start = mdates.date2num(pd.Timestamp(row.effective_start_date).to_pydatetime())
16645         end = mdates.date2num((pd.Timestamp(row.effective_end_date) + pd.Timedelta(days=1)).to_pydatetime())
16646         if pd.isna(row.risk_alpha) or row.kind == "warmup":
16647             y = y_map[None]

```

```

16648         ax.barh(y, end - start, left=start, height=0.62, color="white", edgecolor=BLACK, linewidth=0.8, hatch="///", zorder=3)
16649     else:
16650         alpha = round(float(row.risk_alpha), 2)
16651         y = y_map[alpha]
16652         ax.barh(y, end - start, left=start, height=0.62, color=BLUE, edgecolor=BLUE, linewidth=0.8, zorder=3)
16653         if alpha == 0.7:
16654             ax.text((start + end) / 2, y, "alpha=0.70", ha="center", va="center", fontsize=8.2, color="white", zorder=4)
16655         if int(row.n_days) == 1:
16656             ax.text(
16657                 start - 1.2,
16658                 y,
16659                 "12/31",
16660                 ha="right",
16661                 va="center",
16662                 fontsize=8.2,
16663                 color=BLACK,
16664                 zorder=5,
16665                 bbox={"boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.92},
16666             )
16667     ax.text(
16668         mdates.date2num(pd.Timestamp("2025-01-15").to_pydatetime()),
16669         0,
16670         "1/1-1/28",
16671         ha="center",
16672         va="center",
16673         fontsize=8.2,
16674         color=BLACK,
16675         zorder=4,
16676         bbox={"boxstyle": "round,pad=0.12", "facecolor": "white", "edgecolor": "none", "alpha": 0.88},
16677     )
16678     for unused_y in (3, 4):
16679         ax.text(
16680             mdates.date2num(pd.Timestamp("2025-07-01").to_pydatetime()),
16681             unused_y,
16682             "候选未选用",
16683             ha="center",
16684             va="center",
16685             fontsize=8.4,
16686             color=GRAY,
16687             zorder=2,
16688         )
16689     ax.set_yticks(list(range(5)), ytick_labels)
16690     ax.set_ylim(-0.55, 4.75)
16691     ax.set_ylabel("政策档位")
16692     ax.set_xlabel("风险分位生效区间 (每 14 日一块; 12/31 为最后一块)")
16693     ax.set_title("Q4-2 滚动风险分位的因果校准时间线", pad=8)
16694     ax.xaxis.set_major_locator(mdates.MonthLocator())
16695     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
16696     ax.set_xlim(pd.Timestamp("2025-01-01"), pd.Timestamp("2026-01-04"))
16697     ax.grid(axis="x", linestyle=":", color=GRAY, alpha=0.32, zorder=0)
16698     style_axes(ax)
16699     fig.subplots_adjust(bottom=0.24)
16700     fig.autofmt_xdate(rotation=30, ha="right")
16701     fig.text(
16702         0.5,
16703         0.04,
16704         f"候选集 alpha in {{0.60, 0.70, 0.80, 0.90}}; 校准走官方闭环。",
16705         f"{EXPECTED_Q4_2_ALPHA070_START[5:]}-{EXPECTED_Q4_2_ALPHA070_END[5:]} 为唯一 alpha=0.70 窗口, 其余预热后为 alpha=0.60。",
16706         ha="center",
16707         fontsize=9,
16708         bbox={"boxstyle": "round,pad=0.35", "facecolor": LIGHT_GRAY, "edgecolor": GRAY},
16709     )
16710     save_figure(fig, "fig_q4_2_risk_calibration")
16711     return {
16712         "selected_blocks": int(len(selected)),
16713         "warmup_days": int(selected.loc[selected["kind"] == "warmup", "n_days"].sum()),
16714         "calibrated_blocks": int((selected["kind"] == "calibrated").sum()),
16715         "alpha_060_days": int(selected.loc[selected["risk_alpha"] == 0.60, "n_days"].sum()),

```

```

16716         "alpha_070_days": int(selected.loc[selected["risk_alpha"] == 0.70, "n_days"].sum()),
16717         "calendar_start": selected["effective_start_date"].iloc[0].strftime("%Y-%m-%d"),
16718         "calendar_end": selected["effective_end_date"].iloc[-1].strftime("%Y-%m-%d"),
16719         "alpha_070_window": f"{EXPECTED_Q4_2_ALPHA070_START}..{EXPECTED_Q4_2_ALPHA070_END}",
16720     }
16721
16722
16723 def write_readme(font: str, f2: dict, f3_2: dict, f3_3: dict, f4: dict, hashes: dict[str, str]) -> None:
16724     lines = [
16725         "# Q4 正式交付图",
16726         "",
16727         "运行命令:",
16728         "",
16729         "```bash",
16730         "MPLCONFIGDIR=\"$PWD/.mplconfig\" python3 q4/plot_final_figures.py",
16731         "python3 q4/report_q4_closeout.py",
16732         "```",
16733         "",
16734         f"- 中文字体: {font}",
16735         "- 所有 PNG 均以 320 dpi 导出, 同时提供矢量 PDF。",
16736         "- 原始时段顺序保持 `00:10` 至 `0:00+1`, 未旋转或补造时段。",
16737         "- 主图不含 Q4-3 价格神谕年与更新时刻重记账。",
16738         "",
16739         "## F1a `fig_q4_2_closed_loop`",
16740         "",
16741         "- 数据源: 已签收 Q4-2 机制; 图形由 matplotlib patches 矢量绘制。",
16742         "- 核验: 当天 q 锁定、因果 MPC、紧急购电只补当期、实际 SOC 跨日传递。",
16743         "",
16744         "## F1b `fig_q4_3_update_clock`",
16745         "",
16746         "- 数据源: 已签收 Q4-3 M1_M6 0/6/12/18 时钟。",
16747         "- 核验: 6:00 前前缀锁定; 结算为交付时点实际电价。",
16748         "",
16749         "## F2 `fig_q4_cost_comparison`",
16750         "",
16751         "- 数据源: Q4-2/Q4-3 日汇总、Q2 K=8 日汇总、Q3 官方 A+M1_M6 日汇总。",
16752         "- 汇总范围: 2025-02-01 至 2025-12-31, 共 334 日。不要与 1—12 月年合计混用。",
16753         f"- Q4-2: {f2['q4_2_export_yuan']:, .2f} 元; Q2 K=8: {f2['q2_k8_export_yuan']:, .2f} 元。",
16754         f"- Q4-3: {f2['q4_3_export_yuan']:, .2f} 元; Q3 M1_M6 同期: {f2['q3_m1_m6_export_yuan']:, .2f} 元。",
16755         "- 核验: Q4 与 Q2 对照值与签收工作簿差小于 0.01 元。Q2/Q3 用附件1, Q4 用附件4。",
16756         "",
16757         "## F3a `fig_q4_2_representative_day`",
16758         "",
16759         f"- 可复现选择: 2—12 月 `emergency_cost_yuan` 最大日为 {f3_2['selected_date']}",
16760         f"- 源文件: `{f3_2['source']}`; {f3_2['first_time']} 至 {f3_2['last_time']}",
16761         "",
16762         "## F3b `fig_q4_3_representative_day`",
16763         "",
16764         f"- 可复现选择: 2—12 月 `emergency_cost_yuan` 最大日为 {f3_3['selected_date']}",
16765         f"- 源文件: `{f3_3['source']}`",
16766         "",
16767         "## F4 `fig_q4_2_risk_calibration`",
16768         "",
16769         f"- 共 {f4['selected_blocks']} 个生效块: 预热 {f4['warmup_days']} 日, 其后 {f4['calibrated_blocks']} 块。",
16770         f"- alpha=0.60 为 {f4['alpha_060_days']} 日, alpha=0.70 为 {f4['alpha_070_days']} 日 ({f4['alpha_070_window']})",
16771         "",
16772         "## 只读保护",
16773         "",
16774         "- 绘图脚本对日汇总、alpha 选择表与 result4/result3 工作簿执行前后 SHA-256 校验。",
16775         f"- `output/result4-2.xlsx` SHA-256: `{hashes.get(str(RESULT4_2_XLSX.relative_to(ROOT)), '文件不存在')}`",
16776         f"- `output/result4-3.xlsx` SHA-256: `{hashes.get(str(RESULT4_3_XLSX.relative_to(ROOT)), '文件不存在')}`",
16777     ]
16778     (FIG_DIR / "README.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
16779
16780
16781 def main() -> None:
16782     source_paths = [
16783         Q4_2_DAILY_CSV,

```

```

16784         Q4_3_DAILY_CSV,
16785         Q4_2_ALPHA_SELECTION_CSV,
16786         Q2_K8_DAILY,
16787         Q3_DAILY,
16788     ]
16789     for path in (RESULT4_2_XLSX, RESULT4_3_XLSX, RESULT3):
16790         if path.exists():
16791             source_paths.append(path)
16792     for path in source_paths:
16793         if not path.exists():
16794             raise FileNotFoundError(path)
16795     hashes_before = {str(path.relative_to(ROOT)): file_hash(path) for path in source_paths}
16796     font = configure_style()
16797     q42 = load_q4_2_daily()
16798     q43 = load_q4_3_daily()
16799     comparison = build_fair_comparison(q42, q43, load_q2_k8_daily(), load_q3_official_daily())
16800
16801     plot_q4_2_loop()
16802     plot_q4_3_clock()
16803     f2 = plot_cost_comparison(comparison)
16804     f3_2 = plot_representative_day(q42, Q4_2_DISPATCH_DIR, "Q4-2 日前一日内联动轨迹", "fig_q4_2_representative_day")
16805     f3_3 = plot_representative_day(q43, Q4_3_DISPATCH_DIR, "Q4-3 M1_M6 滚动调整轨迹", "fig_q4_3_representative_day")
16806     f4 = plot_alpha_timeline(q42)
16807
16808     hashes_after = {str(path.relative_to(ROOT)): file_hash(path) for path in source_paths}
16809     if hashes_before != hashes_after:
16810         raise AssertionError("A source result file changed during plotting")
16811     audit = {
16812         "status": "PASS",
16813         "font": font,
16814         "png_dpi": 320,
16815         "f2": f2,
16816         "f3_q4_2": f3_2,
16817         "f3_q4_3": f3_3,
16818         "f4": f4,
16819         "source_sha256": hashes_after,
16820         "oracle_on_main_figures": False,
16821         "result4_2_unchanged": hashes_before.get(str(RESULT4_2_XLSX.relative_to(ROOT)))
16822         == hashes_after.get(str(RESULT4_2_XLSX.relative_to(ROOT))),
16823         "result4_3_unchanged": hashes_before.get(str(RESULT4_3_XLSX.relative_to(ROOT)))
16824         == hashes_after.get(str(RESULT4_3_XLSX.relative_to(ROOT))),
16825     }
16826     FIG_DIR.mkdir(parents=True, exist_ok=True)
16827     (FIG_DIR / "figure_audit.json").write_text(
16828         json.dumps(audit, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
16829     )
16830     write_readme(font, f2, f3_2, f3_3, f4, hashes_after)
16831     print(json.dumps({k: audit[k] for k in ("status", "font", "f2", "f3_q4_2", "f3_q4_3", "f4")}, ensure_ascii=False, indent=2))
16832
16833
16834 if __name__ == "__main__":
16835     main()
16836
16837 # ===== q4/price_forecast.py =====
16838 """Causal day-ahead and intraday price forecasts. History is strictly before day d."""
16839
16840 from __future__ import annotations
16841
16842 from dataclasses import dataclass
16843
16844 import numpy as np
16845 import pandas as pd
16846
16847 from q4.config import (
16848     CANDIDATE_MODELS,
16849     CANDIDATE_NAMES,
16850     INTRA_HOUR_TAU,
16851     T,

```

```

16852     VAR_EPS,
16853 )
16854
16855
16856 def construct_candidate(
16857     prices: np.ndarray,
16858     weekdays: np.ndarray,
16859     day_index: int,
16860     model: int,
16861 ) -> np.ndarray | None:
16862     if model == 1:
16863         if day_index < 1:
16864             return None
16865             return prices[day_index - 1].copy()
16866     if model not in (2, 4):
16867         raise ValueError(f"unknown candidate model {model}")
16868     same = np.flatnonzero((np.arange(day_index) >= 0) & (weekdays[:day_index] == weekdays[day_index]))
16869     if same.size < model:
16870         return None
16871     return prices[same[-model:]].mean(axis=0)
16872
16873
16874 def _mae(actual: np.ndarray, forecast: np.ndarray) -> float:
16875     return float(np.mean(np.abs(actual - forecast)))
16876
16877
16878 def _rmse(actual: np.ndarray, forecast: np.ndarray) -> float:
16879     return float(np.sqrt(np.mean((actual - forecast) ** 2)))
16880
16881
16882 def ols_remaining(
16883     residual_hist: np.ndarray,
16884     z_hist: np.ndarray,
16885     z_now: float,
16886 ) -> tuple[np.ndarray, bool]:
16887     """Fit  $r_t = \alpha_t + \beta_t z$  on historical days; return residual forecast for remaining  $t$ .
16888
16889     residual_hist has shape (n_hist, n_future). z_hist has shape (n_hist,).
16890     """
16891     n_hist, n_future = residual_hist.shape
16892     if n_hist < 2:
16893         if n_hist == 1:
16894             return residual_hist[0].copy(), True
16895             return np.zeros(n_future, dtype=float), True
16896     z_center = z_hist - float(z_hist.mean())
16897     denom = float(np.dot(z_center, z_center))
16898     if denom <= VAR_EPS:
16899         return residual_hist.mean(axis=0).copy(), True
16900     r_center = residual_hist - residual_hist.mean(axis=0)
16901     beta = z_center @ r_center / denom
16902     alpha = residual_hist.mean(axis=0) - beta * float(z_hist.mean())
16903     return alpha + beta * float(z_now), False
16904
16905
16906 def intraday_price_forecast(
16907     actual: np.ndarray,
16908     day_ahead: np.ndarray,
16909     hist_actual: np.ndarray,
16910     hist_day_ahead: np.ndarray,
16911     tau: int,
16912 ) -> tuple[np.ndarray, bool]:
16913     """Return a 144-vector: known actual[:tau], OLS-updated remaining[tau:], clamped at 0."""
16914     if tau < 0 or tau > T:
16915         raise ValueError(f"tau out of range: {tau}")
16916     out = np.empty(T, dtype=float)
16917     if tau == 0:
16918         return np.maximum(0.0, day_ahead.copy()), True
16919     out[:tau] = actual[:tau]

```

```

16920     if tau == T:
16921         return out, True
16922     z_now = float(np.mean(actual[:tau] - day_ahead[:tau]))
16923     if hist_actual.size == 0:
16924         out[tau:] = np.maximum(0.0, day_ahead[tau:])
16925         return out, True
16926     residual_hist = hist_actual[:, tau:] - hist_day_ahead[:, tau:]
16927     z_hist = (hist_actual[:, :tau] - hist_day_ahead[:, :tau]).mean(axis=1)
16928     delta, fallback = ols_remaining(residual_hist, z_hist, z_now)
16929     out[tau:] = np.maximum(0.0, day_ahead[tau:] + delta)
16930     return out, fallback
16931
16932
16933 @dataclass
16934 class PriceForecastArchive:
16935     dates: pd.DatetimeIndex
16936     price: np.ndarray
16937     day_ahead: np.ndarray
16938     chosen_model: np.ndarray
16939     candidate_forecasts: dict[int, np.ndarray]
16940     candidate_available: dict[int, np.ndarray]
16941     mae_through_prev: dict[int, np.ndarray]
16942     n_backtest: dict[int, np.ndarray]
16943     audit: pd.DataFrame
16944
16945
16946 def fit_causal_price_forecasts(
16947     prices: np.ndarray,
16948     dates: pd.DatetimeIndex,
16949     compute_mpc: bool = True,
16950 ) -> PriceForecastArchive:
16951     n = len(dates)
16952     if prices.shape != (n, T):
16953         raise ValueError(f"prices shape {prices.shape} != {(n, T)}")
16954     weekdays = dates.weekday.to_numpy()
16955     day_ahead = np.full((n, T), np.nan)
16956     chosen = np.full(n, -1, dtype=int)
16957     cand_hat = {a: np.full((n, T), np.nan) for a in CANDIDATE_MODELS}
16958     cand_ok = {a: np.zeros(n, dtype=bool) for a in CANDIDATE_MODELS}
16959     mae_prev = {a: np.full(n, np.nan) for a in CANDIDATE_MODELS}
16960     n_bt = {a: np.zeros(n, dtype=int) for a in CANDIDATE_MODELS}
16961     abs_sum = {a: 0.0 for a in CANDIDATE_MODELS}
16962     bt_count = {a: 0 for a in CANDIDATE_MODELS}
16963
16964     rows = []
16965     for d in range(n):
16966         for a in CANDIDATE_MODELS:
16967             n_bt[a][d] = bt_count[a]
16968             if bt_count[a] > 0:
16969                 mae_prev[a][d] = abs_sum[a] / (bt_count[a] * T)
16970             hat = construct_candidate(prices, weekdays, d, a)
16971             if hat is None:
16972                 continue
16973             cand_hat[a][d] = hat
16974             cand_ok[a][d] = True
16975
16976     available = [a for a in CANDIDATE_MODELS if cand_ok[a][d]]
16977     fallback = "none"
16978     if not available:
16979         fallback = "no_history"
16980         chosen_a = -1
16981     else:
16982         scored = []
16983         for a in available:
16984             mae = mae_prev[a][d]
16985             score = float("inf") if not np.isfinite(mae) else float(mae)
16986             scored.append((score, a))
16987         chosen_a = min(scored)[1]

```

```

16988         if not cand_ok[1][d]:
16989             fallback = "no_previous_day"
16990         elif not cand_ok[2][d] and not cand_ok[4][d]:
16991             fallback = "previous_day_only"
16992     chosen[d] = chosen_a
16993     if chosen_a > 0:
16994         day_ahead[d] = cand_hat[chosen_a][d]
16995
16996     da_mae = np.nan
16997     da_rmse = np.nan
16998     if chosen_a > 0:
16999         da_mae = _mae(prices[d], day_ahead[d])
17000         da_rmse = _rmse(prices[d], day_ahead[d])
17001
17002     hist_idx = [i for i in range(d) if chosen[i] > 0]
17003     hist_actual = prices[hist_idx] if hist_idx else np.zeros((0, T))
17004     hist_da = day_ahead[hist_idx] if hist_idx else np.zeros((0, T))
17005     intra = {}
17006     mpc_mae = []
17007     n_mpc_fallback = 0
17008     n_mpc = 0
17009     if chosen_a > 0:
17010         for hour, tau in INTRA_HOUR_TAU.items():
17011             pred, ols_fb = intraday_price_forecast(
17012                 prices[d], day_ahead[d], hist_actual, hist_da, tau
17013             )
17014             intra[hour] = {
17015                 "mae": _mae(prices[d, tau:], pred[tau:]),
17016                 "rmse": _rmse(prices[d, tau:], pred[tau:]),
17017                 "ols_fallback": bool(ols_fb),
17018             }
17019     if compute_mpc:
17020         for tau in range(1, T):
17021             pred, ols_fb = intraday_price_forecast(
17022                 prices[d], day_ahead[d], hist_actual, hist_da, tau
17023             )
17024             mpc_mae.append(_mae(prices[d, tau:], pred[tau:]))
17025             n_mpc += 1
17026             n_mpc_fallback += int(ols_fb)
17027
17028     train_cutoff = dates[d - 1].strftime("%Y-%m-%d") if d else ""
17029     row = {
17030         "date": dates[d].strftime("%Y-%m-%d"),
17031         "day_index": d,
17032         "train_cutoff_date": train_cutoff,
17033         "weekday": int(weekdays[d]),
17034         "mae_prev_day": mae_prev[1][d],
17035         "mae_weekday2": mae_prev[2][d],
17036         "mae_weekday4": mae_prev[4][d],
17037         "n_backtest_prev_day": int(n_bt[1][d]),
17038         "n_backtest_weekday2": int(n_bt[2][d]),
17039         "n_backtest_weekday4": int(n_bt[4][d]),
17040         "candidate_prev_day_available": bool(cand_ok[1][d]),
17041         "candidate_weekday2_available": bool(cand_ok[2][d]),
17042         "candidate_weekday4_available": bool(cand_ok[4][d]),
17043         "chosen_model": chosen_a if chosen_a > 0 else "",
17044         "chosen_model_name": CANDIDATE_NAMES.get(chosen_a, "none"),
17045         "day_ahead_fallback": fallback,
17046         "day_ahead_mae": da_mae,
17047         "day_ahead_rmse": da_rmse,
17048         "intraday_06_mae": intra.get(6, {}).get("mae", np.nan),
17049         "intraday_06_rmse": intra.get(6, {}).get("rmse", np.nan),
17050         "intraday_06_ols_fallback": intra.get(6, {}).get("ols_fallback", True),
17051         "intraday_12_mae": intra.get(12, {}).get("mae", np.nan),
17052         "intraday_12_rmse": intra.get(12, {}).get("rmse", np.nan),
17053         "intraday_12_ols_fallback": intra.get(12, {}).get("ols_fallback", True),
17054         "intraday_18_mae": intra.get(18, {}).get("mae", np.nan),
17055         "intraday_18_rmse": intra.get(18, {}).get("rmse", np.nan),

```

```

17056         "intraday_18_ols_fallback": intra.get(18, {}).get("ols_fallback", True),
17057         "mpc_remaining_mae_mean": float(np.mean(mpc_mae)) if mpc_mae else np.nan,
17058         "n_mpc_points": n_mpc,
17059         "n_mpc_ols_fallback": n_mpc_fallback,
17060     }
17061     rows.append(row)
17062
17063     for a in CANDIDATE_MODELS:
17064         if cand_ok[a][d]:
17065             abs_sum[a] += float(np.abs(prices[d] - cand_hat[a][d]).sum())
17066             bt_count[a] += 1
17067
17068     audit = pd.DataFrame(rows)
17069     return PriceForecastArchive(
17070         dates=dates,
17071         price=prices,
17072         day_ahead=day_ahead,
17073         chosen_model=chosen,
17074         candidate_forecasts=cand_hat,
17075         candidate_available=cand_ok,
17076         mae_through_prev=mae_prev,
17077         n_backtest=n_bt,
17078         audit=audit,
17079     )
17080
17081
17082 def assert_no_future_price_leak(
17083     prices: np.ndarray,
17084     dates: pd.DatetimeIndex,
17085     day_index: int,
17086     tau: int = 0,
17087 ) -> None:
17088     """Perturb prices at and after (day_index, tau); earlier forecasts/choices must not change."""
17089     archive = fit_causal_price_forecasts(prices, dates, compute_mpc=False)
17090     perturbed = prices.copy()
17091     perturbed[day_index, tau:] += 7.5
17092     if day_index + 1 < len(prices):
17093         perturbed[day_index + 1 :] += 3.25
17094     other = fit_causal_price_forecasts(perturbed, dates, compute_mpc=False)
17095     if day_index > 0:
17096         np.testing.assert_allclose(
17097             archive.day_ahead[:day_index],
17098             other.day_ahead[:day_index],
17099             equal_nan=True,
17100         )
17101         np.testing.assert_array_equal(archive.chosen_model[:day_index], other.chosen_model[:day_index])
17102     if tau == 0:
17103         if np.isfinite(archive.day_ahead[day_index]).all():
17104             np.testing.assert_allclose(archive.day_ahead[day_index], other.day_ahead[day_index], equal_nan=True)
17105             np.testing.assert_equal(archive.chosen_model[day_index], other.chosen_model[day_index])
17106     else:
17107         if archive.chosen_model[day_index] <= 0:
17108             return
17109         hist_idx = [i for i in range(day_index) if archive.chosen_model[i] > 0]
17110         hist_actual = prices[hist_idx] if hist_idx else np.zeros((0, T))
17111         hist_da = archive.day_ahead[hist_idx] if hist_idx else np.zeros((0, T))
17112         before, _ = intraday_price_forecast(
17113             prices[day_index], archive.day_ahead[day_index], hist_actual, hist_da, tau
17114         )
17115         after, _ = intraday_price_forecast(
17116             perturbed[day_index], other.day_ahead[day_index], hist_actual, hist_da, tau
17117         )
17118         np.testing.assert_allclose(before[tau:], after[tau:])
17119
17120
17121 def two_day_regression(archive: PriceForecastArchive, dates: tuple[str, ...]) -> dict:
17122     payload = {}
17123     for date in dates:

```



```

17124         loc = archive.dates.get_loc(pd.Timestamp(date))
17125         i = int(loc)
17126         row = archive.audit.iloc[i].to_dict()
17127         payload[date] = {
17128             **{k: (None if isinstance(v, float) and not np.isfinite(v) else v) for k, v in row.items()},
17129             "day_ahead_first6": archive.day_ahead[i, :6].tolist()
17130             if np.isfinite(archive.day_ahead[i, :6]).all()
17131             else None,
17132             "actual_first6": archive.price[i, :6].tolist(),
17133         }
17134     return payload
17135
17136 # ===== q4/prices.py =====
17137 """Causal helpers that call the accepted price module without rewriting it."""
17138
17139 from __future__ import annotations
17140
17141 import numpy as np
17142
17143 from q4.bundle import Q4Bundle
17144 from q4.config import CANDIDATE_MODELS
17145 from q4.price_forecast import intraday_price_forecast
17146
17147 def day_ahead_today(bundle: Q4Bundle, day_index: int) -> tuple[np.ndarray, str, int]:
17148     hat = bundle.price_archive.day_ahead[day_index]
17149     chosen = int(bundle.price_archive.chosen_model[day_index])
17150     if np.isfinite(hat).all() and chosen > 0:
17151         return hat.copy(), "price_module", chosen
17152     return bundle.q3.price.copy(), "attachment1_no_history", -1
17153
17154 def day_ahead_as_of(
17155     bundle: Q4Bundle, target_index: int, history_end_exclusive: int
17156 ) -> tuple[np.ndarray, str, int]:
17157     """Forecast target_index using only prices with index < history_end_exclusive.
17158
17159     This is required for the 48-hour value of day d+1 at day d 00:00: the
17160     archived ``day_ahead[d+1]`` is as-of d+1 00:00 and would leak day d actuals.
17161
17162     """
17163     if history_end_exclusive <= 0:
17164         return bundle.q3.price.copy(), "attachment1_no_history", -1
17165     weekdays = bundle.prices.dates.weekday.to_numpy()
17166     prices = bundle.prices.price
17167     hats: dict[int, np.ndarray] = {}
17168     if target_index >= 1 and (target_index - 1) < history_end_exclusive:
17169         hats[1] = prices[target_index - 1].copy()
17170     for model in (2, 4):
17171         same = np.flatnonzero(
17172             weekdays[:history_end_exclusive] == weekdays[target_index]
17173         )
17174         if same.size < model:
17175             continue
17176         hats[model] = prices[same[-model:]].mean(axis=0)
17177     if not hats:
17178         return bundle.q3.price.copy(), "attachment1_no_history", -1
17179     mae_idx = min(history_end_exclusive, target_index, bundle.n_days() - 1)
17180     scored = []
17181     for model in CANDIDATE_MODELS:
17182         if model not in hats:
17183             continue
17184         mae = bundle.price_archive.mae_through_prev[model][mae_idx]
17185         score = float("inf") if not np.isfinite(mae) else float(mae)
17186         scored.append((score, model))
17187     chosen = min(scored)[1]
17188     return hats[chosen], "price_module_as_of", chosen
17189
17190
17191

```

```

17192 def remaining_price_forecast(
17193     bundle: Q4Bundle, day_index: int, tau: int
17194 ) -> tuple[np.ndarray, bool]:
17195     day_ahead, _src, chosen = day_ahead_today(bundle, day_index)
17196     actual = bundle.prices.price[day_index]
17197     if chosen <= 0 or tau <= 0:
17198         out = day_ahead.copy()
17199         if tau > 0:
17200             out[:tau] = actual[:tau]
17201         return np.maximum(0.0, out), True
17202     hist_idx = [i for i in range(day_index) if bundle.price_archive.chosen_model[i] > 0]
17203     if hist_idx:
17204         hist_actual = bundle.prices.price[hist_idx]
17205         hist_da = bundle.price_archive.day_ahead[hist_idx]
17206     else:
17207         hist_actual = np.zeros((0, day_ahead.shape[0]))
17208         hist_da = np.zeros((0, day_ahead.shape[0]))
17209     return intraday_price_forecast(actual, day_ahead, hist_actual, hist_da, tau)
17210
17211 # ===== q4/q4_2.py =====
17212 """Q4-2: triple-scenario day-ahead q, locked all day, causal MPC execution."""
17213
17214 from __future__ import annotations
17215
17216 import hashlib
17217 from dataclasses import dataclass, field
17218 from math import inf, sqrt
17219 from pathlib import Path
17220 from time import perf_counter
17221
17222 import numpy as np
17223 import pandas as pd
17224
17225 from q2.config import (
17226     E_MAX_KWH,
17227     E_MIN_KWH,
17228     EMERGENCY_PRICE_MULTIPLIER,
17229     NEXT_DAY_VALUE_GAP_TOL_YUAN,
17230     NEXT_DAY_VALUE_MAX_SAMPLES,
17231     NUMERIC_TOL,
17232     SIMULTANEOUS_CD_TOL,
17233     T,
17234 )
17235 from q2.forecast import forecast_as_of
17236 from q2.optimization import ValueCut, evaluate_value_cuts
17237 from q2.pilot import risk_quantile_floor
17238 from q4.bundle import Q4Bundle
17239 from q4.config import (
17240     ALPHA_POLICY_LOOP,
17241     FIXED_SCENARIO_K,
17242     LOAD_INFORMATION_CASE,
17243     Q4_2_YEAR_END_RULE,
17244     RISK_ALPHA_CANDIDATES,
17245     RISK_CALIBRATION_DAYS,
17246     RISK_WARMUP_DAYS,
17247     SETTLEMENT_RULE,
17248 )
17249 from q4.optimization import solve_fixed_q_dispatch, solve_stochastic_plan_varying_price
17250 from q4.prices import day_ahead_as_of, day_ahead_today, remaining_price_forecast
17251 from q4.scenarios import (
17252     TripleScenarioSet,
17253     build_triple_scenarios,
17254     posterior_weights,
17255     residual_stack,
17256     scenario_paths,
17257 )
17258
17259

```

```

17260 def plan_locked_q(
17261     bundle: Q4Bundle,
17262     day_index: int,
17263     initial_soc: float,
17264     risk_alpha: float | None,
17265     k_target: int = FIXED_SCENARIO_K,
17266     with_value_cuts: bool = True,
17267 ) -> np.ndarray:
17268     """Day-ahead q only (no MPC). Used by information-set probes."""
17269     loads, pvs, prices, probs, _sc, base_load, base_pv, _src, _ch = q4_2_day_inputs(
17270         bundle, day_index, day_index, k_target
17271     )
17272     q_floor = risk_quantile_floor(loads, pvs, probs, base_load, base_pv, risk_alpha)
17273     cuts = ()
17274     if with_value_cuts:
17275         cuts, _ = build_q4_2_value_cuts(bundle, day_index, k_target, risk_alpha)
17276     plan = solve_stochastic_plan_varying_price(
17277         prices,
17278         loads,
17279         pvs,
17280         probs,
17281         initial_soc,
17282         terminal_value_cuts=cuts or None,
17283         q_floor=q_floor,
17284     )
17285     return plan.q.copy()
17286
17287
17288 def planned_q_hash(q: np.ndarray) -> str:
17289     canonical = "".join(f"{float(value):.9f}" for value in np.asarray(q).ravel())
17290     return hashlib.sha256(canonical.encode("ascii")).hexdigest()
17291
17292
17293 @dataclass
17294 class Q42DayResult:
17295     date: str
17296     day_index: int
17297     q: np.ndarray
17298     dispatch: pd.DataFrame
17299     summary: dict
17300     day_ahead_audit: dict
17301     scenario_rows: list[dict] = field(default_factory=list)
17302     value_rows: list[dict] = field(default_factory=list)
17303
17304
17305 def _point_paths(load_hat: np.ndarray, pv_hat: np.ndarray, price_hat: np.ndarray):
17306     return load_hat[None, :], pv_hat[None, :], price_hat[None, :], np.ones(1)
17307
17308
17309 def q4_2_day_inputs(
17310     bundle: Q4Bundle,
17311     day_index: int,
17312     history_end_exclusive: int,
17313     k_target: int = FIXED_SCENARIO_K,
17314 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, TripleScenarioSet | None, np.ndarray, np.ndarray, str, int]:
17315     load_hat, pv_hat, _ls, _ps = forecast_as_of(
17316         bundle.q2, day_index, history_end_exclusive
17317     )
17318     if history_end_exclusive == day_index:
17319         price_hat, price_src, chosen = day_ahead_today(bundle, day_index)
17320     else:
17321         price_hat, price_src, chosen = day_ahead_as_of(
17322             bundle, day_index, history_end_exclusive
17323         )
17324     scenarios = build_triple_scenarios(
17325         bundle, day_index, history_end_exclusive, k_target=k_target
17326     )
17327     if scenarios is None:

```

```

17328     loads, pvs, prices, probs = _point_paths(load_hat, pv_hat, price_hat)
17329 else:
17330     loads, pvs, prices = scenario_paths(
17331         bundle, scenarios, load_hat, pv_hat, price_hat
17332     )
17333     probs = scenarios.proBABILITIES
17334 return loads, pvs, prices, probs, scenarios, load_hat, pv_hat, price_src, chosen
17335
17336
17337 def build_q4_2_value_cuts(
17338     bundle: Q4Bundle,
17339     current_index: int,
17340     k_target: int,
17341     risk_alpha: float | None,
17342 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
17343     next_index = current_index + 1
17344     if next_index >= bundle.n_days():
17345         return (), []
17346     loads, pvs, prices, probs, scenarios, base_load, base_pv, price_src, chosen = q4_2_day_inputs(
17347         bundle, next_index, current_index, k_target
17348     )
17349     q_floor = risk_quantile_floor(loads, pvs, probs, base_load, base_pv, risk_alpha)
17350     solved: dict[float, ValueCut] = {}
17351
17352     def solve_at(soc: float) -> None:
17353         result = solve_stochastic_plan_varying_price(
17354             prices, loads, pvs, probs, float(soc), q_floor=q_floor
17355         )
17356         value = result.expected_normal_cost + result.expected_emergency_cost
17357         solved[float(soc)] = (float(soc), float(value), float(result.initial_soc_marginal))
17358
17359     def certified_max_gap(cuts: tuple[ValueCut, ...]) -> tuple[float, float]:
17360         points = sorted(solved)
17361         best_gap, best_soc = -np.inf, points[0]
17362         lower_lines = [(0.0, 0.0)] + [
17363             (slope, value - slope * reference) for reference, value, slope in cuts
17364         ]
17365         for left, right in zip(points[:-1], points[1:]):
17366             left_value, right_value = solved[left][1], solved[right][1]
17367             upper_slope = (right_value - left_value) / (right - left)
17368             upper_intercept = left_value - upper_slope * left
17369             candidates = [left, right]
17370             for a, (slope_a, intercept_a) in enumerate(lower_lines):
17371                 for slope_b, intercept_b in lower_lines[a + 1 :]:
17372                     if abs(slope_a - slope_b) <= 1e-14:
17373                         continue
17374                     cross = (intercept_b - intercept_a) / (slope_a - slope_b)
17375                     if left < cross < right:
17376                         candidates.append(float(cross))
17377             for soc in candidates:
17378                 upper = upper_slope * soc + upper_intercept
17379                 gap = upper - evaluate_value_cuts(soc, cuts)
17380                 if gap > best_gap:
17381                     best_gap, best_soc = float(gap), float(soc)
17382             return max(best_gap, 0.0), best_soc
17383
17384     solve_at(E_MIN_KWH)
17385     solve_at(E_MAX_KWH)
17386     gap = inf
17387     while True:
17388         cuts = tuple(solved[soc] for soc in sorted(solved))
17389         slopes = np.array([cut[2] for cut in cuts])
17390         if np.any(np.diff(slopes) < -1e-6):
17391             raise AssertionError("Q4-2 next-day value subgradients are not monotone")
17392         gap, refine_soc = certified_max_gap(cuts)
17393         if gap <= NEXT_DAY_VALUE_GAP_TOL_YUAN:
17394             break
17395         if len(cuts) >= NEXT_DAY_VALUE_MAX_SAMPLES:

```

```

17396         raise AssertionError(
17397             f"Q4-2 next-day value gap {gap:.6f} exceeds tolerance after {len(cuts)} samples"
17398         )
17399         if min(abs(refine_soc - soc) for soc in solved) < 1e-7:
17400             raise AssertionError("Q4-2 next-day value refinement stalled")
17401         solve_at(refine_soc)
17402     k_eff = 1 if scenarios is None else scenarios.k_effective
17403     rows = [
17404         {
17405             "current_date": bundle.prices.dates[current_index].strftime("%Y-%m-%d"),
17406             "target_date": bundle.prices.dates[next_index].strftime("%Y-%m-%d"),
17407             "k_effective": k_eff,
17408             "risk_alpha": risk_alpha,
17409             "price_source": price_src,
17410             "chosen_price_model": chosen,
17411             "soc_sample_kwh": reference,
17412             "virtual_next_day_cost_yuan": value,
17413             "value_subgradient_yuan_per_kwh": slope,
17414             "certified_max_gap_yuan": gap,
17415         }
17416         for reference, value, slope in cuts
17417     ]
17418     return cuts, rows
17419
17420 @dataclass
17421 class AlphaSelection:
17422     selected_alpha: float
17423     window_records: list[dict]
17424     daily_records: list[dict]
17425
17426
17427 def calibration_window(target_index: int, n_validation: int | None = None) -> np.ndarray:
17428     n = RISK_CALIBRATION_DAYS if n_validation is None else int(n_validation)
17429     if n <= 0 or target_index <= 0:
17430         return np.zeros(0, dtype=int)
17431     return np.arange(max(0, target_index - n), target_index, dtype=int)
17432
17433
17434 def q2_aligned_calibration_indices(n_days: int) -> list[int]:
17435     """Q2 14-day blocks after the 28-day warmup: 28, 42, ..., up to the last complete index."""
17436     return [
17437         i
17438         for i in range(RISK_WARMUP_DAYS, n_days)
17439         if i % RISK_CALIBRATION_DAYS == 0
17440     ]
17441
17442
17443 def _metrics_from_day(result: Q42DayResult, elapsed_seconds: float) -> dict:
17444     q = np.asarray(result.q, dtype=float)
17445     x = result.dispatch["x_kwh"].to_numpy(float)
17446     soc_end = result.dispatch["soc_end_kwh"].to_numpy(float)
17447     gap = (
17448         float(result.value_rows[0]["certified_max_gap_yuan"])
17449         if result.value_rows
17450         else float("nan")
17451     )
17452     return {
17453         "date": result.date,
17454         "day_index": int(result.day_index),
17455         "planned_fee_yuan": float(result.summary["normal_cost_yuan"]),
17456         "emergency_fee_yuan": float(result.summary["emergency_cost_yuan"]),
17457         "total_cost_yuan": float(result.summary["total_cost_yuan"]),
17458         "unused_quota_kwh": float(np.sum(q - x)),
17459         "emergency_kwh": float(result.summary["emergency_kwh"]),
17460         "soc_start_kwh": float(result.summary["soc_start_kwh"]),
17461         "soc_end_kwh": float(result.summary["soc_end_kwh"]),
17462         "soc_min_kwh": float(soc_end.min()),

```

```

17464         "soc_max_kwh": float(soc_end.max()),
17465         "soc_end_path_kwh": ";".join(f"{value:.6f}" for value in soc_end),
17466         "has_value_cuts": bool(result.value_rows),
17467         "n_value_cuts": int(len(result.value_rows)),
17468         "value_cut_gap_yuan": gap,
17469         "value_cuts_used": bool(result.value_rows),
17470         "no_future_info": True,
17471         "used_full_day_actual_scheduler": False,
17472         "k_effective": int(result.summary["k_effective"]),
17473         "elapsed_seconds": float(elapsed_seconds),
17474         "policy_loop": ALPHA_POLICY_LOOP,
17475     }
17476
17477
17478 def run_candidate_window(
17479     bundle: Q4Bundle,
17480     policy_start_soc: np.ndarray,
17481     target_index: int,
17482     *,
17483     risk_alpha: float | None,
17484     k_target: int = FIXED_SCENARIO_K,
17485     n_validation: int | None = None,
17486     log_prefix: str | None = None,
17487 ) -> tuple[list[dict], dict]:
17488     """Replay prior completed days with the official Q4-2 closed loop.
17489
17490     Each candidate carries its own SOC. The window opens at the deployed
17491     policy's recorded start SOC on the first validation day. Full-day actual
17492     path dispatch is not used.
17493     """
17494     validation = calibration_window(target_index, n_validation)
17495     empty = {
17496         "risk_alpha": None if risk_alpha is None else float(risk_alpha),
17497         "k_target": int(k_target),
17498         "validation_days": 0,
17499         "mean_validation_cost_yuan": np.nan,
17500         "standard_error_yuan": np.nan,
17501         "total_planned_fee_yuan": np.nan,
17502         "total_emergency_fee_yuan": np.nan,
17503         "total_cost_yuan": np.nan,
17504         "total_unused_quota_kwh": np.nan,
17505         "emergency_purchase_kwh": np.nan,
17506         "soc_window_start_kwh": np.nan,
17507         "soc_window_end_kwh": np.nan,
17508         "daily_soc_end_path_kwh": "",
17509         "all_value_cuts_present": False,
17510         "no_future_info": True,
17511         "used_full_day_actual_scheduler": False,
17512         "mean_elapsed_seconds": np.nan,
17513         "mean_solve_seconds": np.nan,
17514         "policy_loop": ALPHA_POLICY_LOOP,
17515         "fallback_reason": "empty_validation_window",
17516     }
17517     if len(validation) == 0:
17518         return [], empty
17519     soc = float(policy_start_soc[int(validation[0])])
17520     if not np.isfinite(soc):
17521         empty["fallback_reason"] = "no_validation_soc"
17522         return [], empty
17523     daily: list[dict] = []
17524     for offset, day_index in enumerate(validation):
17525         started = perf_counter()
17526         result = run_q4_2_day(
17527             bundle, int(day_index), soc, risk_alpha, k_target=k_target
17528         )
17529         elapsed = perf_counter() - started
17530         row = _metrics_from_day(result, elapsed)
17531         row["risk_alpha"] = None if risk_alpha is None else float(risk_alpha)

```

```

17532     row["k_target"] = int(k_target)
17533     row["calibration_day_index"] = int(target_index)
17534     daily.append(row)
17535     soc = float(result.summary["soc_end_kwh"])
17536     if log_prefix:
17537         print(
17538             f"{log_prefix} [{offset + 1}/{len(validation)}] {result.date} "
17539             f"cost={row['total_cost_yuan']:.2f} soc={soc:.2f}",
17540             flush=True,
17541         )
17542     del result
17543     costs = np.asarray([row["total_cost_yuan"] for row in daily], dtype=float)
17544     summary = {
17545         "risk_alpha": None if risk_alpha is None else float(risk_alpha),
17546         "k_target": int(k_target),
17547         "validation_days": int(len(daily)),
17548         "mean_validation_cost_yuan": float(np.mean(costs)),
17549         "standard_error_yuan": float(
17550             np.std(costs, ddof=1) / sqrt(len(costs)) if len(costs) > 1 else 0.0
17551         ),
17552         "total_planned_fee_yuan": float(sum(row["planned_fee_yuan"] for row in daily)),
17553         "total_emergency_fee_yuan": float(
17554             sum(row["emergency_fee_yuan"] for row in daily)
17555         ),
17556         "total_cost_yuan": float(np.sum(costs)),
17557         "total_unused_quota_kwh": float(
17558             sum(row["unused_quota_kwh"] for row in daily)
17559         ),
17560         "emergency_purchase_kwh": float(sum(row["emergency_kwh"] for row in daily)),
17561         "soc_window_start_kwh": float(daily[0]["soc_start_kwh"]),
17562         "soc_window_end_kwh": float(daily[-1]["soc_end_kwh"]),
17563         "daily_soc_end_path_kwh": ";".join(
17564             f"{row['soc_end_kwh']:.6f}" for row in daily
17565         ),
17566         "all_value_cuts_present": bool(all(row["has_value_cuts"] for row in daily)),
17567         "no_future_info": True,
17568         "used_full_day_actual_scheduler": False,
17569         "mean_elapsed_seconds": float(np.mean([row["elapsed_seconds"] for row in daily])),
17570         "mean_solve_seconds": float(np.mean([row["elapsed_seconds"] for row in daily])),
17571         "policy_loop": ALPHA_POLICY_LOOP,
17572         "fallback_reason": "",
17573     }
17574     return daily, summary
17575
17576
17577 def select_risk_alpha(
17578     bundle: Q4Bundle,
17579     policy_start_soc: np.ndarray,
17580     target_index: int,
17581     k_target: int,
17582     n_validation: int | None = None,
17583     alphas: tuple[float, ...] | None = None,
17584     log: bool = False,
17585 ) -> AlphaSelection:
17586     """Score each alpha on the prior completed days with the official Q4-2 loop."""
17587     candidates = RISK_ALPHA_CANDIDATES if alphas is None else tuple(alphas)
17588     records: list[dict] = []
17589     daily_records: list[dict] = []
17590     for alpha in candidates:
17591         prefix = (
17592             f"Q4-2 alpha-cal d={target_index} alpha={float(alpha):.2f}" if log else None
17593         )
17594         daily, summary = run_candidate_window(
17595             bundle,
17596             policy_start_soc,
17597             target_index,
17598             risk_alpha=float(alpha),
17599             k_target=k_target,

```

```

17600         n_validation=n_validation,
17601         log_prefix=prefix,
17602     )
17603     summary["selected"] = False
17604     records.append(summary)
17605     daily_records.extend(daily)
17606     eligible = [row for row in records if np.isfinite(row["mean_validation_cost_yuan"])]
17607     selected = min(
17608         eligible or records,
17609         key=lambda row: (
17610             row["mean_validation_cost_yuan"]
17611             if np.isfinite(row["mean_validation_cost_yuan"])
17612             else inf,
17613             row["risk_alpha"] if row["risk_alpha"] is not None else inf,
17614         ),
17615     )
17616     selected["selected"] = True
17617     chosen = float(selected["risk_alpha"])
17618     for row in daily_records:
17619         row["selected"] = bool(
17620             row["risk_alpha"] is not None and abs(float(row["risk_alpha"]) - chosen) < 1e-12
17621         )
17622     return AlphaSelection(chosen, records, daily_records)
17623
17624
17625 def run_q4_2_day(
17626     bundle: Q4Bundle,
17627     day_index: int,
17628     initial_soc: float,
17629     risk_alpha: float | None,
17630     k_target: int = FIXED_SCENARIO_K,
17631     write_dispatch: Path | None = None,
17632 ) -> Q42DayResult:
17633     started = perf_counter()
17634     date = bundle.prices.dates[day_index].strftime("%Y-%m-%d")
17635     loads, pvs, prices, probs, scenarios, base_load, base_pv, price_src, chosen = q4_2_day_inputs(
17636         bundle, day_index, day_index, k_target
17637     )
17638     q_floor = risk_quantile_floor.loads, pvs, probs, base_load, base_pv, risk_alpha
17639     cuts, value_rows = build_q4_2_value_cuts(bundle, day_index, k_target, risk_alpha)
17640     plan = solve_stochastic_plan_varying_price(
17641         prices,
17642         loads,
17643         pvs,
17644         probs,
17645         initial_soc,
17646         terminal_value_cuts=cuts or None,
17647         q_floor=q_floor,
17648     )
17649     locked_q = plan.q.copy()
17650     q_hash = planned_q_hash(locked_q)
17651     actual_p = bundle.prices.price[day_index]
17652     actual_load = bundle.q2.load[day_index]
17653     actual_pv = bundle.q2.pv[day_index]
17654     day_ahead, _, _ = day_ahead_today(bundle, day_index)
17655     rows = []
17656     soc = float(initial_soc)
17657     max_cd = 0.0
17658     k_eff = 1 if scenarios is None else scenarios.k_effective
17659     for t in range(T):
17660         price_hat_tau, ols_fb = remaining_price_forecast(bundle, day_index, t)
17661         if scenarios is None:
17662             weights = np.ones(1)
17663             bandwidth = np.nan
17664             load_horizon = base_load[t:].copy()
17665             pv_horizon = base_pv[t:].copy()
17666             price_horizon = price_hat_tau[t:].copy()
17667         else:

```



```

17668         if t == 0:
17669             obs_l = np.zeros(0)
17670             obs_p = np.zeros(0)
17671             obs_r = np.zeros(0)
17672         else:
17673             # Ended prefix only; do not form a full-day residual against today's future actuals.
17674             obs_l = actual_load[:t] - base_load[:t]
17675             obs_p = actual_pv[:t] - base_pv[:t]
17676             obs_r = actual_p[:t] - day_ahead[:t]
17677             weights, bandwidth = posterior_weights(
17678                 bundle, scenarios, obs_l, obs_p, obs_r
17679             )
17680             med_l, med_p, med_r = residual_stack(bundle, scenarios.medoid_indices)
17681             load_horizon = np.maximum(0.0, base_load[t:] + weights @ med_l[:, t:])
17682             pv_horizon = np.maximum(0.0, base_pv[t:] + weights @ med_p[:, t:])
17683             price_horizon = np.maximum(0.0, price_hat_tau[t:] + weights @ med_r[:, t:])
17684             load_horizon[0] = actual_load[t]
17685             pv_horizon[0] = actual_pv[t]
17686             step = solve_fixed_q_dispatch(
17687                 price_horizon,
17688                 locked_q[t:],
17689                 load_horizon,
17690                 pv_horizon,
17691                 soc,
17692                 terminal_value_cuts=cuts or None,
17693             )
17694             residual = (
17695                 step.x[0]
17696                 + step.emergency[0]
17697                 + actual_pv[t]
17698                 - step.curtailment[0]
17699                 + step.discharge[0]
17700                 - actual_load[t]
17701                 - step.charge[0]
17702             )
17703             soc_next = float(step.soc[1])
17704             rows.append(
17705                 {
17706                     "date": date,
17707                     "period_index": t,
17708                     "time_label": bundle.q3.time_labels[t],
17709                     "q_or_g0_kwh": float(locked_q[t]),
17710                     "g_final_kwh": np.nan,
17711                     "x_kwh": float(step.x[0]),
17712                     "emergency_kwh": float(step.emergency[0]),
17713                     "charge_kwh": float(step.charge[0]),
17714                     "discharge_kwh": float(step.discharge[0]),
17715                     "curtailment_kwh": float(step.curtailment[0]),
17716                     "soc_start_kwh": soc,
17717                     "soc_end_kwh": soc_next,
17718                     "actual_load_kwh": float(actual_load[t]),
17719                     "actual_pv_kwh": float(actual_pv[t]),
17720                     "actual_price": float(actual_p[t]),
17721                     "price_forecast_used": float(price_horizon[0]),
17722                     "last_update_time": "00:00",
17723                     "balance_residual_kwh": float(residual),
17724                     "ols_fallback": bool(ols_fb),
17725                 }
17726             )
17727             max_cd = max(max_cd, float(step.charge[0] * step.discharge[0]))
17728             if np.max(np.abs(locked_q - plan.q)) > 0:
17729                 raise AssertionError("Q4-2 q changed during the day")
17730             soc = soc_next
17731         if planned_q_hash(locked_q) != q_hash:
17732             raise AssertionError("Q4-2 q hash changed")
17733         dispatch = pd.DataFrame(rows)
17734         if write_dispatch is not None:
17735             write_dispatch.parent.mkdir(parents=True, exist_ok=True)

```

```

17736         dispatch.to_csv(write_dispatch, index=False)
17737 normal_cost = float(actual_p @ locked_q)
17738 emergency_cost = float(
17739     EMERGENCY_PRICE_MULTIPLIER * actual_p @ dispatch["emergency_kwh"].to_numpy()
17740 )
17741 scenario_rows = []
17742 if scenarios is not None:
17743     for cluster, medoid in enumerate(scenarios.medoid_indices):
17744         scenario_rows.append(
17745             {
17746                 "date": date,
17747                 "cluster": int(cluster),
17748                 "medoid_date": bundle.prices.dates[medoid].strftime("%Y-%m-%d"),
17749                 "medoid_index": int(medoid),
17750                 "probability": float(scenarios.proBABILITIES[cluster]),
17751                 "cluster_size": int(np.sum(scenarios.cluster_labels == cluster)),
17752                 "pool_start": bundle.prices.dates[scenarios.pool_indices[0]].strftime("%Y-%m-%d"),
17753                 "pool_end": bundle.prices.dates[scenarios.pool_indices[-1]].strftime("%Y-%m-%d"),
17754                 "k_effective": int(scenarios.k_effective),
17755                 "scale_load": float(scenarios.scale_load),
17756                 "scale_pv": float(scenarios.scale_pv),
17757                 "scale_price": float(scenarios.scale_price),
17758             }
17759         )
17760 day_ahead_audit = {
17761     "date": date,
17762     "day_index": day_index,
17763     "history_cutoff_date": (
17764         bundle.prices.dates[day_index - 1].strftime("%Y-%m-%d") if day_index else ""
17765     ),
17766     "k_target": k_target,
17767     "k_effective": k_eff,
17768     "risk_alpha": risk_alpha,
17769     "risk_q_floor_kwh": float(np.sum(q_floor)),
17770     "price_source": price_src,
17771     "chosen_price_model": chosen,
17772     "planned_q_kwh": float(locked_q.sum()),
17773     "planned_q_sha256": q_hash,
17774     "n_scenarios": int(len(probs)),
17775     "load_information_case": LOAD_INFORMATION_CASE,
17776     "year_end_rule": Q4_2_YEAR_END_RULE,
17777     "settlement_rule": SETTLEMENT_RULE,
17778 }
17779 summary = {
17780     "date": date,
17781     "day_index": day_index,
17782     "load_information_case": LOAD_INFORMATION_CASE,
17783     "year_end_rule": Q4_2_YEAR_END_RULE,
17784     "settlement_rule": SETTLEMENT_RULE,
17785     "k_effective": k_eff,
17786     "risk_alpha": risk_alpha,
17787     "normal_cost_yuan": normal_cost,
17788     "adjustment_cost_yuan": 0.0,
17789     "emergency_cost_yuan": emergency_cost,
17790     "total_cost_yuan": normal_cost + emergency_cost,
17791     "planned_q_kwh": float(locked_q.sum()),
17792     "actual_x_kwh": float(dispatch["x_kwh"].sum()),
17793     "emergency_kwh": float(dispatch["emergency_kwh"].sum()),
17794     "curtailment_kwh": float(dispatch["curtailment_kwh"].sum()),
17795     "soc_start_kwh": float(initial_soc),
17796     "soc_end_kwh": float(soc),
17797     "unused_quota_kwh": float(np.sum(locked_q - dispatch["x_kwh"].to_numpy(float))),
17798     "has_value_cuts": bool(value_rows),
17799     "n_value_cuts": int(len(value_rows)),
17800     "value_cut_gap_yuan": (
17801         float(value_rows[0]["certified_max_gap_yuan"]) if value_rows else float("nan")
17802     ),
17803     "no_future_info": True,

```

```

17804         "used_full_day_actual_scheduler": False,
17805         "elapsed_seconds": float(perf_counter() - started),
17806         "max_balance_residual_kwh": float(dispatch["balance_residual_kwh"].abs().max()),
17807         "max_simultaneous_cd_kwh2": max_cd,
17808         "max_x_minus_q_kwh": float(np.max(dispatch["x_kwh"].to_numpy() - locked_q)),
17809         "q_unchanged": True,
17810         "pass": bool(
17811             dispatch["balance_residual_kwh"].abs().max() < NUMERIC_TOL
17812             and np.max(dispatch["x_kwh"].to_numpy() - locked_q) < NUMERIC_TOL
17813             and dispatch["soc_end_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL
17814             and dispatch["soc_end_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL
17815             and max_cd <= SIMULTANEOUS_CD_TOL
17816         ),
17817     }
17818     return Q42DayResult(
17819         date=date,
17820         day_index=day_index,
17821         q=locked_q,
17822         dispatch=dispatch,
17823         summary=summary,
17824         day_ahead_audit=day_ahead_audit,
17825         scenario_rows=scenario_rows,
17826         value_rows=value_rows,
17827     )
17828
17829
17830 def alpha_for_day(day_index: int, calibrated: dict[int, float | None]) -> float | None:
17831     if day_index < RISK_WARMUP_DAYS:
17832         return None
17833     start = (day_index // RISK_CALIBRATION_DAYS) * RISK_CALIBRATION_DAYS
17834     return calibrated.get(start, None)
17835
17836 # ===== q4/q4_3.py =====
17837 """Q4-3: fixed M1_M6 with causal price forecasts and delivery-time settlement."""
17838
17839 from __future__ import annotations
17840
17841 from dataclasses import dataclass, field
17842 from time import perf_counter
17843
17844 import numpy as np
17845 import pandas as pd
17846
17847 from q3.config import (
17848     E_MAX_KWH,
17849     E_MIN_KWH,
17850     EMERGENCY_PRICE_MULTIPLIER,
17851     HOUR_TO_FIRST_MUTABLE,
17852     LOAD_INFORMATION_MAIN,
17853     NUMERIC_TOL,
17854     PV_MAPPING_LINEAR,
17855     SETTLEMENT_MAIN,
17856     SIMULTANEOUS_CD_TOL,
17857     STRATEGY_ALLOWED_UPDATES,
17858     T,
17859     VOI_EPS_YUAN,
17860     next_day_year_end_soc,
17861     today_year_end_soc,
17862 )
17863 from q3.forecast import (
17864     execution_load_horizon,
17865     map_issue_forecast,
17866     next_day_pv_forecast,
17867     planning_load_curve,
17868     causal_load_forecast,
17869 )
17870 from q3.optimization import (
17871     ValueCut,

```

```

17872     evaluate_value_cuts,
17873     settlement_cost,
17874     solve_horizon,
17875 )
17876
17877
17878 def _solve_horizon(*args, **kwargs):
17879     """Call Q3's LP; if the unique-throughput tiebreak is numerically infeasible, keep the primary solution."""
17880     try:
17881         return solve_horizon(*args, **kwargs)
17882     except RuntimeError as exc:
17883         if "tiebreak" not in str(exc):
17884             raise
17885         kwargs = dict(kwargs)
17886         kwargs["throughput_tiebreak"] = False
17887         return solve_horizon(*args, **kwargs)
17888 from q4.bundle import Q4Bundle
17889 from q4.config import (
17890     LOAD_INFORMATION_CASE,
17891     Q4_3_STRATEGY,
17892     Q4_3_YEAR_END_BOUNDARY,
17893     Q4_3_YEAR_END_SOC_KWH,
17894     SETTLEMENT_RULE,
17895 )
17896 from q4.prices import day_ahead_as_of, day_ahead_today, remaining_price_forecast
17897 from q4.q4_2 import planned_q_hash
17898
17899
17900 def _index_to_update_hour(index: int) -> int | None:
17901     inverse = {first: hour for hour, first in HOUR_TO_FIRST_MUTABLE.items() if hour != 0}
17902     return inverse.get(index)
17903
17904
17905 def _decision_prices(
17906     bundle: Q4Bundle, day_index: int, tau: int, price_mode: str
17907 ) -> tuple[np.ndarray, bool]:
17908     if price_mode == "oracle":
17909         return np.asarray(bundle.prices.price[day_index], dtype=float).copy(), False
17910     return remaining_price_forecast(bundle, day_index, tau)
17911
17912
17913 def _midnight_prices(
17914     bundle: Q4Bundle, day_index: int, price_mode: str
17915 ) -> tuple[np.ndarray, str, int]:
17916     if price_mode == "oracle":
17917         return np.asarray(bundle.prices.price[day_index], dtype=float).copy(), "price_oracle", -1
17918     return day_ahead_today(bundle, day_index)
17919
17920
17921 def build_q4_3_value_cuts(
17922     bundle: Q4Bundle,
17923     current_index: int,
17924     issue_hour: int,
17925     next_day_terminal_soc: float | None,
17926     cache: dict | None,
17927 ) -> tuple[tuple[ValueCut, ...], list[dict]]:
17928     key = (int(current_index), int(issue_hour), None if next_day_terminal_soc is None else float(next_day_terminal_soc))
17929     if cache is not None and key in cache:
17930         return cache[key]
17931     next_index = current_index + 1
17932     if next_index >= bundle.n_days():
17933         cuts, rows = (), []
17934         if cache is not None:
17935             cache[key] = (cuts, rows)
17936         return cuts, rows
17937     load = causal_load_forecast(bundle.q3, next_index, current_index)
17938     pv = next_day_pv_forecast(bundle.q3, current_index, issue_hour, PV_MAPPING_LINEAR)
17939     price, price_src, chosen = day_ahead_as_of(bundle, next_index, current_index)

```

```

17940
17941 # Reuse Q3's supporting-cut sampler by solving with the causal next-day price.
17942 from q3.config import NEXT_DAY_VALUE_GAP_TOL_YUAN, NEXT_DAY_VALUE_MAX_SAMPLES
17943 from math import inf
17944
17945 solved: dict[float, ValueCut] = {}
17946
17947 def solve_at(soc: float) -> None:
17948     result = _solve_horizon(
17949         price,
17950         load,
17951         pv,
17952         float(soc),
17953         bill_as_day_ahead=True,
17954         throughput_tiebreak=False,
17955         terminal_soc=next_day_terminal_soc,
17956     )
17957     value = result.settlement_cost + result.emergency_cost
17958     solved[float(soc)] = (float(soc), float(value), float(result.initial_soc_marginal))
17959
17960 def certified_max_gap(cuts: tuple[ValueCut, ...]) -> tuple[float, float]:
17961     points = sorted(solved)
17962     best_gap, best_soc = -np.inf, points[0]
17963     lower_lines = [(0.0, 0.0)] + [
17964         (slope, value - slope * reference) for reference, value, slope in cuts
17965     ]
17966     for left, right in zip(points[:-1], points[1:]):
17967         left_value, right_value = solved[left][1], solved[right][1]
17968         upper_slope = (right_value - left_value) / (right - left)
17969         upper_intercept = left_value - upper_slope * left
17970         candidates = [left, right]
17971         for a, (slope_a, intercept_a) in enumerate(lower_lines):
17972             for slope_b, intercept_b in lower_lines[a + 1 :]:
17973                 if abs(slope_a - slope_b) <= 1e-14:
17974                     continue
17975                 cross = (intercept_b - intercept_a) / (slope_a - slope_b)
17976                 if left < cross < right:
17977                     candidates.append(float(cross))
17978         for soc in candidates:
17979             upper = upper_slope * soc + upper_intercept
17980             gap = upper - evaluate_value_cuts(soc, cuts)
17981             if gap > best_gap:
17982                 best_gap, best_soc = float(gap), float(soc)
17983     return max(best_gap, 0.0), best_soc
17984
17985 solve_at(E_MIN_KWH)
17986 solve_at(E_MAX_KWH)
17987 gap = inf
17988 while True:
17989     cuts = tuple(solved[soc] for soc in sorted(solved))
17990     slopes = np.array([cut[2] for cut in cuts])
17991     if np.any(np.diff(slopes) < -1e-6):
17992         raise AssertionError("Q4-3 next-day value subgradients are not monotone")
17993     gap, refine_soc = certified_max_gap(cuts)
17994     if gap <= NEXT_DAY_VALUE_GAP_TOL_YUAN:
17995         break
17996     if len(cuts) >= NEXT_DAY_VALUE_MAX_SAMPLES:
17997         raise AssertionError(
17998             f"Q4-3 next-day value gap {gap:.6f} exceeds tolerance after {len(cuts)} samples"
17999         )
18000     if min(abs(refine_soc - soc) for soc in solved) < 1e-7:
18001         raise AssertionError("Q4-3 next-day value refinement stalled")
18002     solve_at(refine_soc)
18003 date = bundle.prices.dates[current_index].strftime("%Y-%m-%d")
18004 rows = [
18005     {
18006         "current_date": date,
18007         "target_date": bundle.prices.dates[next_index].strftime("%Y-%m-%d"),

```

```

18008         "issue_hour": issue_hour,
18009         "price_source": price_src,
18010         "chosen_price_model": chosen,
18011         "soc_sample_kwh": reference,
18012         "virtual_next_day_cost_yuan": value,
18013         "value_subgradient_yuan_per_kwh": slope,
18014         "certified_max_gap_yuan": gap,
18015     }
18016     for reference, value, slope in cuts
18017 ]
18018 if cache is not None:
18019     cache[key] = (cuts, rows)
18020 return cuts, rows
18021
18022
18023 def plan_g0(
18024     bundle: Q4Bundle,
18025     day_index: int,
18026     initial_soc: float,
18027     year_end_soc_kwh: float | None = Q4_3_YEAR_END_SOC_KWH,
18028     value_cut_cache: dict | None = None,
18029     with_value_cuts: bool = True,
18030     price_mode: str = "causal",
18031 ) -> np.ndarray:
18032     """Midnight g0 only (no intraday updates). Used by information-set probes."""
18033     data = bundle.q3
18034     n_days = len(data.dates)
18035     today_terminal = today_year_end_soc(day_index, n_days, year_end_soc_kwh)
18036     next_terminal = next_day_year_end_soc(day_index, n_days, year_end_soc_kwh)
18037     plan_load = planning_load_curve(data, day_index, LOAD_INFORMATION_MAIN)
18038     mapped0 = map_issue_forecast(data, day_index, 0, PV_MAPPING_LINEAR)
18039     price0, _src, _chosen = _midnight_prices(bundle, day_index, price_mode)
18040     cuts = ()
18041     if with_value_cuts:
18042         cuts, _rows = build_q4_3_value_cuts(
18043             bundle, day_index, 0, next_terminal, value_cut_cache
18044         )
18045     midnight = _solve_horizon(
18046         price0,
18047         plan_load,
18048         mapped0.today_kwh.copy(),
18049         initial_soc,
18050         bill_as_day_ahead=True,
18051         terminal_value_cuts=cuts or None,
18052         terminal_soc=today_terminal,
18053     )
18054     return midnight.g.copy()
18055
18056
18057 @dataclass
18058 class Q43DayResult:
18059     date: str
18060     day_index: int
18061     g0: np.ndarray
18062     g_final: np.ndarray
18063     dispatch: pd.DataFrame
18064     update_log: pd.DataFrame
18065     summary: dict
18066     value_rows: list[dict] = field(default_factory=list)
18067
18068
18069 def run_q4_3_day(
18070     bundle: Q4Bundle,
18071     day_index: int,
18072     initial_soc: float,
18073     year_end_soc_kwh: float | None = Q4_3_YEAR_END_SOC_KWH,
18074     value_cut_cache: dict | None = None,
18075     price_mode: str = "causal",

```

```

18076 ) -> Q43DayResult:
18077     data = bundle.q3
18078     date = data.dates[day_index].strftime("%Y-%m-%d")
18079     n_days = len(data.dates)
18080     today_terminal = today_year_end_soc(day_index, n_days, year_end_soc_kwh)
18081     next_terminal = next_day_year_end_soc(day_index, n_days, year_end_soc_kwh)
18082     allowed = STRATEGY_ALLOWED_UPDATES[Q4_3_STRATEGY]
18083     actual_load = data.load[day_index]
18084     plan_load = planning_load_curve(data, day_index, LOAD_INFORMATION_MAIN)
18085     actual_pv = data.pv[day_index]
18086     actual_p = bundle.prices.price[day_index]
18087     started = perf_counter()
18088     mapped0 = map_issue_forecast(data, day_index, 0, PV_MAPPING_LINEAR)
18089     current_forecast = mapped0.today_kwh.copy()
18090     price0, price_src, chosen = _midnight_prices(bundle, day_index, price_mode)
18091     cuts, value_rows = build_q4_3_value_cuts(
18092         bundle, day_index, 0, next_terminal, value_cut_cache
18093     )
18094     midnight = _solve_horizon(
18095         price0,
18096         plan_load,
18097         current_forecast,
18098         initial_soc,
18099         bill_as_day_ahead=True,
18100         terminal_value_cuts=cuts or None,
18101         terminal_soc=today_terminal,
18102     )
18103     g0 = midnight.g.copy()
18104     g = g0.copy()
18105     versions = [g0.copy()]
18106     last_update = "00:00"
18107     first_intraday = HOUR_TO_FIRST_MUTABLE[6]
18108     update_rows = [
18109         {
18110             "date": date,
18111             "update_time": "00:00",
18112             "first_mutable_index": 0,
18113             "voi_yuan": np.nan,
18114             "implemented": True,
18115             "li_change_kwh": float(np.abs(g0).sum()),
18116             "price_source": price_src,
18117             "chosen_price_model": chosen,
18118             "affected_periods": "0:143",
18119             "prefix_lock_ok": True,
18120             "forecast_version": "day_ahead",
18121             "g_old_sha256": "",
18122             "g_new_sha256": planned_q_hash(g0),
18123         }
18124     ]
18125
18126     def remaining_kwargs(t: int, price_rem: np.ndarray) -> dict:
18127         return {
18128             "settlement_mode": SETTLEMENT_MAIN,
18129             "terminal_value_cuts": cuts or None,
18130             "g0": g0[t:],
18131             "terminal_soc": today_terminal,
18132         }
18133
18134     dispatch_rows = []
18135     soc = float(initial_soc)
18136     max_cd = 0.0
18137     max_balance = 0.0
18138     locked_violations = 0
18139     g_before_update = g.copy()
18140
18141     for t in range(T):
18142         hour = _index_to_update_hour(t)
18143         if hour is not None and hour in allowed:

```

```

18144     g_before_update = g.copy()
18145     mapped = map_issue_forecast(data, day_index, hour, PV_MAPPING_LINEAR)
18146     current_forecast = mapped.today_kwh.copy()
18147     cuts, rows = build_q4_3_value_cuts(
18148         bundle, day_index, hour, next_terminal, value_cut_cache
18149     )
18150     value_rows.extend(rows)
18151     price_now, _fb = _decision_prices(bundle, day_index, t, price_mode)
18152     pv_plan = current_forecast[t:]
18153     load_plan = plan_load[t:]
18154     g_pre = g[t:].copy()
18155     settle_kw = remaining_kwargs(t, price_now[t:])
18156     j_fix = _solve_horizon(
18157         price_now[t:],
18158         load_plan,
18159         pv_plan,
18160         soc,
18161         g_fixed=g_pre,
18162         **settle_kw,
18163     )
18164     j_free = _solve_horizon(
18165         price_now[t:],
18166         load_plan,
18167         pv_plan,
18168         soc,
18169         **settle_kw,
18170     )
18171     voi = float(j_fix.objective - j_free.objective)
18172     implemented = bool(voi > VOI_EPS_YUAN)
18173     prefix_ok = bool(np.max(np.abs(g[:t] - g_before_update[:t])) <= NUMERIC_TOL)
18174     if implemented:
18175         g[t:] = j_free.g
18176         versions.append(g.copy())
18177     if not prefix_ok:
18178         locked_violations += 1
18179     delta = g[t:] - g_pre
18180     update_rows.append(
18181         {
18182             "date": date,
18183             "update_time": f"{hour:02d}:00",
18184             "first_mutable_index": t,
18185             "voi_yuan": voi,
18186             "implemented": implemented,
18187             "li_change_kwh": float(np.abs(delta).sum()),
18188             "price_source": "price_oracle" if price_mode == "oracle" else "intraday_ols",
18189             "chosen_price_model": chosen,
18190             "affected_periods": f"{t}:{T - 1}",
18191             "prefix_lock_ok": prefix_ok,
18192             "forecast_version": f"tau_{t}",
18193             "g_old_sha256": planned_q_hash(g_before_update),
18194             "g_new_sha256": planned_q_hash(g),
18195         }
18196     )
18197     last_update = f"{hour:02d}:00"
18198     if np.max(np.abs(g[:t] - g_before_update[:t])) > NUMERIC_TOL:
18199         locked_violations += 1
18200
18201     price_now, _fb = _decision_prices(bundle, day_index, t, price_mode)
18202     pv_horizon = current_forecast[t:].copy()
18203     pv_horizon[0] = actual_pv[t]
18204     load_horizon = execution_load_horizon(plan_load, actual_load, t)
18205     step = _solve_horizon(
18206         price_now[t:],
18207         load_horizon,
18208         pv_horizon,
18209         soc,
18210         g_fixed=g[t:],
18211         **remaining_kwargs(t, price_now[t:]),

```



```

18212     )
18213     residual = (
18214         step.x[0]
18215         + step.emergency[0]
18216         + actual_pv[t]
18217         - step.curtailment[0]
18218         + step.discharge[0]
18219         - actual_load[t]
18220         - step.charge[0]
18221     )
18222     soc_next = float(step.soc[1])
18223     max_cd = max(max_cd, float(step.charge[0] * step.discharge[0]))
18224     max_balance = max(max_balance, abs(float(residual)))
18225     dispatch_rows.append(
18226         {
18227             "date": date,
18228             "period_index": t,
18229             "time_label": data.time_labels[t],
18230             "q_or_g0_kwh": float(g0[t]),
18231             "g_final_kwh": float(g[t]),
18232             "x_kwh": float(step.x[0]),
18233             "emergency_kwh": float(step.emergency[0]),
18234             "charge_kwh": float(step.charge[0]),
18235             "discharge_kwh": float(step.discharge[0]),
18236             "curtailment_kwh": float(step.curtailment[0]),
18237             "soc_start_kwh": soc,
18238             "soc_end_kwh": soc_next,
18239             "actual_load_kwh": float(actual_load[t]),
18240             "actual_pv_kwh": float(actual_pv[t]),
18241             "actual_price": float(actual_p[t]),
18242             "price_forecast_used": float(price_now[t]),
18243             "last_update_time": last_update,
18244             "balance_residual_kwh": float(residual),
18245         }
18246     )
18247     if abs(g[t] - step.g[0]) > 1e-6:
18248         locked_violations += 1
18249     soc = soc_next
18250
18251     dispatch = pd.DataFrame(dispatch_rows)
18252     g_final = g.copy()
18253     if np.max(np.abs(g_final[:first_intraday] - g0[:first_intraday])) > NUMERIC_TOL:
18254         locked_violations += 1
18255         raise AssertionError("Q4-3 6:00 prefix lock violated: gF[:36] differs from g0")
18256     phi = settlement_cost(actual_p, g0, g_final)
18257     emergency_cost = EMERGENCY_PRICE_MULTIPLIER * actual_p * dispatch["emergency_kwh"].to_numpy()
18258     dispatch["normal_cost_yuan"] = actual_p * g_final
18259     dispatch["adjustment_cost_yuan"] = phi - actual_p * g_final
18260     dispatch["emergency_cost_yuan"] = emergency_cost
18261     n_adjust = int(
18262         sum(1 for row in update_rows if row["update_time"] != "00:00" and row["implemented"])
18263     )
18264     summary = {
18265         "date": date,
18266         "day_index": day_index,
18267         "strategy": Q4_3_STRATEGY,
18268         "load_information_case": LOAD_INFORMATION_CASE,
18269         "year_end_boundary": Q4_3_YEAR_END_BOUNDARY,
18270         "year_end_soc_kwh": year_end_soc_kwh,
18271         "settlement_rule": SETTLEMENT_RULE,
18272         "price_mode": price_mode,
18273         "normal_cost_yuan": float(np.sum(actual_p * g_final)),
18274         "adjustment_cost_yuan": float(np.sum(phi - actual_p * g_final)),
18275         "emergency_cost_yuan": float(np.sum(emergency_cost)),
18276         "total_cost_yuan": float(np.sum(phi) + np.sum(emergency_cost)),
18277         "g0_kwh": float(g0.sum()),
18278         "g_final_kwh": float(g_final.sum()),
18279         "emergency_kwh": float(dispatch["emergency_kwh"].sum()),

```

```

18280         "curtailment_kwh": float(dispatch["curtailment_kwh"].sum()),
18281         "soc_start_kwh": float(initial_soc),
18282         "soc_end_kwh": float(soc),
18283         "adjustment_count": n_adjust,
18284         "locked_period_violations": int(locked_violations),
18285         "max_balance_residual_kwh": float(max_balance),
18286         "max_simultaneous_cd_kwh2": float(max_cd),
18287         "runtime_seconds": float(perf_counter() - started),
18288         "max_x_minus_g_kwh": float(np.max(dispatch["x_kwh"].to_numpy() - g_final)),
18289         "pass": bool(
18290             max_balance < NUMERIC_TOL
18291             and locked_violations == 0
18292             and max_cd <= SIMULTANEOUS_CD_TOL
18293             and np.max(dispatch["x_kwh"].to_numpy() - g_final) < NUMERIC_TOL
18294             and dispatch["soc_end_kwh"].min() >= E_MIN_KWH - NUMERIC_TOL
18295             and dispatch["soc_end_kwh"].max() <= E_MAX_KWH + NUMERIC_TOL
18296         ),
18297     }
18298     return Q43DayResult(
18299         date=date,
18300         day_index=day_index,
18301         g0=g0,
18302         g_final=g_final,
18303         dispatch=dispatch,
18304         update_log=pd.DataFrame(update_rows),
18305         summary=summary,
18306         value_rows=value_rows,
18307     )
18308
18309 # ===== q4/q4_3_sensitivity.py =====
18310 """Q4-3 off-main sensitivities: update-time adjustment fees and a price oracle.
18311
18312 Neither path replaces the official M1_M6 delivery-time scheme or result4-3.xlsx.
18313 """
18314
18315 from __future__ import annotations
18316
18317 from pathlib import Path
18318
18319 import numpy as np
18320 import pandas as pd
18321
18322 from q3.config import ADJUST_ABS_COEFF, PILOT_DATES
18323 from q4.audit import ledger_from_dispatch_q43, write_json
18324 from q4.config import (
18325     OUTPUT_DIR,
18326     Q4_3_DISPATCH_DIR,
18327     Q4_3_ORACLE_DAILY_CSV,
18328     Q4_3_ORACLE_DISPATCH_DIR,
18329     Q4_3_SENSITIVITY_SUMMARY_JSON,
18330     Q4_3_SETTLEMENT_SENSITIVITY_CSV,
18331     Q4_3_SETTLEMENT_SENSITIVITY_JSON,
18332 )
18333
18334 UPDATE_CLOCKS = ("06:00", "12:00", "18:00")
18335
18336
18337 def clock_price_map(frame: pd.DataFrame) -> dict[str, float]:
18338     labels = frame["time_label"].astype(str)
18339     out: dict[str, float] = {}
18340     for clock in UPDATE_CLOCKS:
18341         hit = frame.loc[labels == clock]
18342         if not hit.empty:
18343             out[clock] = float(hit["actual_price"].iloc[0])
18344     return out
18345
18346
18347 def adjustment_at_update_clock(frame: pd.DataFrame) -> pd.DataFrame:

```

```

18348     """Rebook 0.5 |gF-g0| at the last update clock price; keep p_t gF and 5 p_t e."""
18349     work = frame.copy()
18350     g0 = work["q_or_g0_kwh"].to_numpy(float)
18351     gf = work["g_final_kwh"].to_numpy(float)
18352     p = work["actual_price"].to_numpy(float)
18353     last = work["last_update_time"].astype(str).to_numpy()
18354     clocks = clock_price_map(work)
18355     p_tau = np.array(
18356         [clocks[t] if t in clocks else float(p[i]) for i, t in enumerate(last)],
18357         dtype=float,
18358     )
18359     delta = np.abs(gf - g0)
18360     adj_delivery = ADJUST_ABS_COEFF * p * delta
18361     adj_update = ADJUST_ABS_COEFF * p_tau * delta
18362     emergency = 5.0 * p * work["emergency_kwh"].to_numpy(float)
18363     normal = p * gf
18364     work["adjustment_delivery_yuan"] = adj_delivery
18365     work["adjustment_update_clock_yuan"] = adj_update
18366     work["update_clock_price"] = p_tau
18367     work["normal_cost_yuan"] = normal
18368     work["emergency_cost_yuan"] = emergency
18369     work["total_delivery_yuan"] = normal + adj_delivery + emergency
18370     work["total_update_clock_yuan"] = normal + adj_update + emergency
18371     return work
18372
18373
18374 def summarize_settlement_day(frame: pd.DataFrame) -> dict:
18375     billed = ledger_from_dispatch_q43(frame)
18376     alt = adjustment_at_update_clock(frame)
18377     delivery_adj = float(alt["adjustment_delivery_yuan"].sum())
18378     ledger_adj = float(billed["adjustment_cost_yuan"].sum())
18379     return {
18380         "date": str(frame["date"].iloc[0]),
18381         "n_adjusted_periods": int(np.sum(np.abs(
18382             frame["g_final_kwh"].to_numpy(float) - frame["q_or_g0_kwh"].to_numpy(float)
18383         ) > 1e-9)),
18384         "normal_cost_yuan": float(alt["normal_cost_yuan"].sum()),
18385         "emergency_cost_yuan": float(alt["emergency_cost_yuan"].sum()),
18386         "adjustment_delivery_yuan": delivery_adj,
18387         "adjustment_update_clock_yuan": float(alt["adjustment_update_clock_yuan"].sum()),
18388         "total_delivery_yuan": float(alt["total_delivery_yuan"].sum()),
18389         "total_update_clock_yuan": float(alt["total_update_clock_yuan"].sum()),
18390         "adjustment_gap_yuan": float(alt["adjustment_update_clock_yuan"].sum() - delivery_adj),
18391         "ledger_matches_delivery": bool(abs(delivery_adj - ledger_adj) < 1e-6),
18392     }
18393
18394
18395 def run_settlement_sensitivity(dispatch_dir: Path = Q4_3_DISPATCH_DIR) -> dict:
18396     paths = sorted(dispatch_dir.glob("dispatch_*.csv"))
18397     if not paths:
18398         raise FileNotFoundError(f"no Q4-3 dispatch files in {dispatch_dir}")
18399     rows = [summarize_settlement_day(pd.read_csv(path)) for path in paths]
18400     daily = pd.DataFrame(rows)
18401     daily["date"] = pd.to_datetime(daily["date"])
18402     daily = daily.sort_values("date").reset_index(drop=True)
18403     export = daily.loc[daily["date"] >= pd.Timestamp("2025-02-01")]
18404     summary = {
18405         "policy": "official_q4_3_dispatch_rebooked_only",
18406         "changes_main_scheme": False,
18407         "n_days": int(len(daily)),
18408         "n_export_days": int(len(export)),
18409         "annual_delivery_yuan": float(daily["total_delivery_yuan"].sum()),
18410         "annual_update_clock_yuan": float(daily["total_update_clock_yuan"].sum()),
18411         "annual_adjustment_delivery_yuan": float(daily["adjustment_delivery_yuan"].sum()),
18412         "annual_adjustment_update_clock_yuan": float(daily["adjustment_update_clock_yuan"].sum()),
18413         "annual_adjustment_gap_yuan": float(daily["adjustment_gap_yuan"].sum()),
18414         "export_delivery_yuan": float(export["total_delivery_yuan"].sum()),
18415         "export_update_clock_yuan": float(export["total_update_clock_yuan"].sum()),

```

```

18416         "all_ledger_match": bool(daily["ledger_matches_delivery"].all()),
18417         "note": (
18418             "Main phi uses delivery-time p_t. The sensitivity keeps the same "
18419             "(g0, gF, e) and charges 0.5 |gF-g0| at the last update clock price "
18420             "p_{d,tau} (06:00/12:00/18:00 period ending at that clock).",
18421         ),
18422     }
18423     out = daily.copy()
18424     out["date"] = out["date"].dt.strftime("%Y-%m-%d")
18425     Q4_3_SETTLEMENT_SENSITIVITY_CSV.parent.mkdir(parents=True, exist_ok=True)
18426     out.to_csv(Q4_3_SETTLEMENT_SENSITIVITY_CSV, index=False)
18427     write_json(Q4_3_SETTLEMENT_SENSITIVITY_JSON, summary)
18428     return {"daily": out, "summary": summary}
18429
18430
18431 def write_sensitivity_summary(settlement: dict, oracle_daily: pd.DataFrame | None) -> dict:
18432     main = pd.read_csv(OUTPUT_DIR / "q4_3_warmup_daily.csv")
18433     q42 = pd.read_csv(OUTPUT_DIR / "q4_2_warmup_daily.csv")
18434     main["date"] = pd.to_datetime(main["date"])
18435     q42["date"] = pd.to_datetime(q42["date"])
18436     payload = {
18437         "main_scheme": "Q4-3 M1_M6 causal prices, delivery-time phi+5pe",
18438         "changes_main_scheme": False,
18439         "q4_3_main": {
18440             "annual_yuan": float(main["total_cost_yuan"].sum()),
18441             "export_yuan": float(main.loc[main["date"] >= "2025-02-01", "total_cost_yuan"].sum()),
18442             "end_soc_kwh": float(main["soc_end_kwh"].iloc[-1]),
18443         },
18444         "q4_2_main": {
18445             "annual_yuan": float(q42["total_cost_yuan"].sum()),
18446             "export_yuan": float(q42.loc[q42["date"] >= "2025-02-01", "total_cost_yuan"].sum()),
18447         },
18448         "settlement_update_clock": settlement["summary"],
18449         "pilot_dates": list(PILOT_DATES),
18450     }
18451     if oracle_daily is not None and not oracle_daily.empty:
18452         oracle_daily = oracle_daily.copy()
18453         oracle_daily["date"] = pd.to_datetime(oracle_daily["date"])
18454         payload["price_oracle"] = {
18455             "annual_yuan": float(oracle_daily["total_cost_yuan"].sum()),
18456             "export_yuan": float(
18457                 oracle_daily.loc[oracle_daily["date"] >= "2025-02-01", "total_cost_yuan"].sum()
18458             ),
18459             "end_soc_kwh": float(oracle_daily["soc_end_kwh"].iloc[-1]),
18460             "gap_vs_main_annual_yuan": float(
18461                 oracle_daily["total_cost_yuan"].sum() - main["total_cost_yuan"].sum()
18462             ),
18463             "note": (
18464                 "Offline reference only: midnight and remaining-horizon prices are "
18465                 "today's actual path. Next-day value cuts stay causal. Not executable."
18466             ),
18467         }
18468     for date in PILOT_DATES:
18469         main_hit = main.loc[main["date"] == date]
18470         ora_hit = oracle_daily.loc[oracle_daily["date"] == date]
18471         if not main_hit.empty and not ora_hit.empty:
18472             payload.setdefault("two_day", {})[date] = {
18473                 "main_yuan": float(main_hit.iloc[0]["total_cost_yuan"]),
18474                 "oracle_yuan": float(ora_hit.iloc[0]["total_cost_yuan"]),
18475             }
18476     write_json(Q4_3_SENSITIVITY_SUMMARY_JSON, payload)
18477     return payload
18478
18479 # ===== q4/report_q4_closeout.py =====
18480 #!/usr/bin/env python3
18481 """Write Q4 closeout reports from signed-off daily/dispatch artifacts. Does not re-solve."""
18482
18483 from __future__ import annotations

```

```

18484
18485 import argparse
18486 import json
18487 import sys
18488 from pathlib import Path
18489
18490 import pandas as pd
18491
18492 ROOT = Path(__file__).resolve().parents[1]
18493 if str(ROOT) not in sys.path:
18494     sys.path.insert(0, str(ROOT))
18495
18496 from q4.audit import write_json
18497 from q4.closeout import (
18498     build_fair_comparison,
18499     build_residual_correlation_from_bundle,
18500     cost_components,
18501     load_q2_k8_daily,
18502     load_q3_official_daily,
18503     load_q4_2_daily,
18504     load_q4_3_daily,
18505     price_error_stats,
18506     scan_dispatch_year,
18507     selected_alpha_blocks,
18508 )
18509 from q4.config import (
18510     AUDIT_CSV,
18511     FAIR_COMPARISON_CSV,
18512     FAIR_COMPARISON_JSON,
18513     OUTPUT_DIR,
18514     PHYSICAL_YEAR_AUDIT_JSON,
18515     PRICE_MONTHLY_JSON,
18516     Q4_2_ALPHA_SELECTION_CSV,
18517     Q4_2_DISPATCH_DIR,
18518     Q4_3_DISPATCH_DIR,
18519     SCENARIO_CORR_JSON,
18520     TAIL_STATS_CSV,
18521     TAIL_STATS_JSON,
18522 )
18523
18524
18525 def _tail_row(name: str, window: str, components: dict) -> dict:
18526     return {
18527         "scheme": name,
18528         "window": window,
18529         "n_days": components["n_days"],
18530         "total_cost_yuan": components["total_cost_yuan"],
18531         "emergency_cost_yuan": components["emergency_cost_yuan"],
18532         "emergency_kwh": components["emergency_kwh"],
18533         "max_day_cost_yuan": components["max_day_cost_yuan"],
18534         "max_day_cost_date": components["max_day_cost_date"],
18535         "max_emergency_cost_date": components["max_emergency_cost_date"],
18536         "p90_yuan": components["daily_cost_p90_yuan"],
18537         "p95_yuan": components["daily_cost_p95_yuan"],
18538         "cvar_0_90_yuan": components["daily_cost_cvar_0_90_yuan"],
18539     }
18540
18541
18542 def main(argv: list[str] | None = None) -> None:
18543     parser = argparse.ArgumentParser(description="Q4 reporting-only closeout")
18544     parser.add_argument("--skip-correlation", action="store_true")
18545     parser.add_argument("--skip-physical", action="store_true")
18546     args = parser.parse_args(argv)
18547
18548     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
18549     q42 = load_q4_2_daily()
18550     q43 = load_q4_3_daily()
18551     q2 = load_q2_k8_daily()

```

```

18552 q3 = load_q3_official_daily()
18553 comparison = build_fair_comparison(q42, q43, q2, q3)
18554 price_audit = pd.read_csv(AUDIT_CSV)
18555 price_export = price_error_stats(price_audit, "2025-02-01", "2025-12-31")
18556 price_annual = price_error_stats(price_audit, "2025-01-01", "2025-12-31")
18557 monthly = json.loads(PRICE_MONTHLY_JSON.read_text(encoding="utf-8")) if PRICE_MONTHLY_JSON.exists() else {}
18558 annual_row = next((row for row in monthly.get("rows", []) if row.get("year_month") == "2025-annual"), {})
18559 if annual_row:
18560     if abs(price_annual["day_ahead_mae"] - float(annual_row["day_ahead_mae"])) >= 1e-8:
18561         raise AssertionError("annual day-ahead MAE does not match signed-off monthly table")
18562     if abs(price_annual["day_ahead_rmse_pooled"] - float(annual_row["day_ahead_rmse"])) >= 1e-8:
18563         raise AssertionError("annual pooled RMSE does not match signed-off monthly table")
18564
18565 tail_rows = [
18566     _tail_row("Q4-2", "1-12", cost_components(q42, "q4_2")),
18567     _tail_row("Q4-2", "2-12", comparison["export"]["q4_2"]),
18568     _tail_row("Q4-3", "1-12", cost_components(q43, "q4_3")),
18569     _tail_row("Q4-3", "2-12", comparison["export"]["q4_3"]),
18570     _tail_row("Q2-K8", "2-12", comparison["export"]["q2_k8"]),
18571     _tail_row("Q3-M1_M6", "2-12", comparison["export"]["q3_m1_m6"]),
18572     _tail_row("Q3-M1_M6", "1-12", comparison["q3_annual"]),
18573 ]
18574 tail = {
18575     "price_note": comparison["price_note"],
18576     "price_errors": {"export_2_12": price_export, "annual_1_12": price_annual},
18577     "rows": tail_rows,
18578 }
18579 write_json(TAIL_STATS_JSON, tail)
18580 pd.DataFrame(tail_rows).to_csv(TAIL_STATS_CSV, index=False)
18581
18582 fair_rows = [
18583     {
18584         "pair": "Q4-2 vs Q2 K=8",
18585         "window": "2-12",
18586         "left": "Q4-2",
18587         "right": "Q2 K=8",
18588         "left_total_yuan": comparison["export"]["q4_2"]["total_cost_yuan"],
18589         "right_total_yuan": comparison["export"]["q2_k8"]["total_cost_yuan"],
18590         "delta_total_yuan": comparison["pairs"]["q4_2_minus_q2_k8_export_yuan"],
18591         "left_emergency_kwh": comparison["export"]["q4_2"]["emergency_kwh"],
18592         "right_emergency_kwh": comparison["export"]["q2_k8"]["emergency_kwh"],
18593         "delta_emergency_kwh": comparison["pairs"]["q4_2_minus_q2_k8_emergency_kwh"],
18594     },
18595     {
18596         "pair": "Q4-3 vs Q3 M1_M6",
18597         "window": "2-12",
18598         "left": "Q4-3",
18599         "right": "Q3 M1_M6",
18600         "left_total_yuan": comparison["export"]["q4_3"]["total_cost_yuan"],
18601         "right_total_yuan": comparison["export"]["q3_m1_m6"]["total_cost_yuan"],
18602         "delta_total_yuan": comparison["pairs"]["q4_3_minus_q3_m1_m6_export_yuan"],
18603         "left_emergency_kwh": comparison["export"]["q4_3"]["emergency_kwh"],
18604         "right_emergency_kwh": comparison["export"]["q3_m1_m6"]["emergency_kwh"],
18605         "delta_emergency_kwh": comparison["pairs"]["q4_3_minus_q3_m1_m6_emergency_kwh"],
18606     },
18607 ]
18608 write_json(FAIR_COMPARISON_JSON, comparison)
18609 pd.DataFrame(fair_rows).to_csv(FAIR_COMPARISON_CSV, index=False)
18610
18611 physical = {"skipped": True}
18612 if not args.skip_physical:
18613     physical = {
18614         "q4_2": scan_dispatch_year(Q4_2_DISPATCH_DIR, q42, q4_3=False),
18615         "q4_3": scan_dispatch_year(Q4_3_DISPATCH_DIR, q43, q4_3=True),
18616     }
18617 if not physical["q4_2"]["pass"] or not physical["q4_3"]["pass"]:
18618     raise SystemExit("physical year audit failed")
18619 write_json(PHYSICAL_YEAR_AUDIT_JSON, physical)

```

```

18620
18621     correlation = {"skipped": True}
18622     if not args.skip_correlation:
18623         from q4.bundle import load_q4_bundle
18624
18625         bundle = load_q4_bundle(compute_price_mpc=False)
18626         correlation = build_residual_correlation_from_bundle(bundle)
18627         write_json(SCENARIO_CORR_JSON, correlation)
18628
18629     alpha_blocks = selected_alpha_blocks(q42, pd.read_csv(Q4_2_ALPHA_SELECTION_CSV))
18630     summary = {
18631         "tail_stats": str(TAIL_STATS_JSON.relative_to(ROOT)),
18632         "fair_comparison": str(FAIR_COMPARISON_JSON.relative_to(ROOT)),
18633         "physical_audit": None
18634         if args.skip_physical
18635         else str(PHYSICAL_YEAR_AUDIT_JSON.relative_to(ROOT)),
18636         "residual_correlation": None
18637         if args.skip_correlation
18638         else str(SCENARIO_CORR_JSON.relative_to(ROOT)),
18639         "q4_2_alpha_blocks": [
18640             {
18641                 "start": row.effective_start_date.strftime("%Y-%m-%d"),
18642                 "end": row.effective_end_date.strftime("%Y-%m-%d"),
18643                 "n_days": int(row.n_days),
18644                 "risk_alpha": row.risk_alpha,
18645             }
18646             for row in alpha_blocks.itertuples()
18647         ],
18648         "physical_pass": None if args.skip_physical else physical["q4_2"]["pass"] and physical["q4_3"]["pass"],
18649     }
18650     print(json.dumps(summary, ensure_ascii=False, indent=2, default=str))
18651
18652
18653 if __name__ == "__main__":
18654     main()
18655
18656 # ===== q4/run_k_review.py =====
18657 #!/usr/bin/env python3
18658 """K=4/8/12 cost and timing review on the Q2-aligned calibration calendar.
18659
18660 Uses the official Q4-2 closed loop on each historical 14-day window.
18661 Does not re-pick K from the full-year result. If K=8 is stable, keep it.
18662 """
18663
18664 from __future__ import annotations
18665
18666 import argparse
18667 import json
18668 import sys
18669 from pathlib import Path
18670 from time import perf_counter
18671
18672 ROOT = Path(__file__).resolve().parents[1]
18673 if str(ROOT) not in sys.path:
18674     sys.path.insert(0, str(ROOT))
18675
18676 import numpy as np
18677 import pandas as pd
18678
18679 from q2.config import E_INITIAL_KWH
18680 from q4.audit import write_json
18681 from q4.bundle import load_q4_bundle
18682 from q4.config import (
18683     FIXED_SCENARIO_K,
18684     K_REVIEW_CANDIDATES,
18685     K_REVIEW_COST_TOLERANCE,
18686     K_REVIEW_REFERENCE_ALPHA,
18687     OUTPUT_DIR,

```

```

18688 )
18689 from q4.q4_2 import q2_aligned_calibration_indices, run_candidate_window
18690
18691 DAILY_CSV = OUTPUT_DIR / "q4_2_k_review_daily.csv"
18692 SUMMARY_CSV = OUTPUT_DIR / "q4_2_k_review.csv"
18693 SUMMARY_JSON = OUTPUT_DIR / "q4_2_k_review.json"
18694 WARMUP_CSV = OUTPUT_DIR / "q4_2_warmup_daily.csv"
18695 POLICY_SOC_CSV = OUTPUT_DIR / "q4_2_k_review_policy_soc.csv"
18696
18697
18698 def load_policy_start_soc(n_days: int, warmup_csv: Path = WARMUP_CSV) -> np.ndarray:
18699     soc = np.full(n_days, np.nan)
18700     source = POLICY_SOC_CSV if POLICY_SOC_CSV.exists() else warmup_csv
18701     if source.exists():
18702         daily = pd.read_csv(source)
18703         for _, row in daily.iterrows():
18704             idx = int(row["day_index"])
18705             if 0 <= idx < n_days and "soc_start_kwh" in row:
18706                 soc[idx] = float(row["soc_start_kwh"])
18707     if not np.isfinite(soc[0]):
18708         soc[0] = E_INITIAL_KWH
18709     last = float(soc[0])
18710     for i in range(n_days):
18711         if np.isfinite(soc[i]):
18712             last = float(soc[i])
18713         else:
18714             soc[i] = last
18715     return soc
18716
18717
18718 def decide_keep_k8(by_k: dict[int, dict]) -> dict:
18719     costs = {k: float(row["mean_cost_per_day_yuan"]) for k, row in by_k.items()}
18720     times = {k: float(row["mean_elapsed_seconds"]) for k, row in by_k.items()}
18721     best_k = min(costs, key=lambda k: (costs[k], k))
18722     best_cost = costs[best_k]
18723     k8_cost = costs[FIXED_SCENARIO_K]
18724     gap = (k8_cost - best_cost) / best_cost if best_cost > 0 else 0.0
18725     time_ok = times[FIXED_SCENARIO_K] <= 3.0 * max(times[min(times)], 1e-9)
18726     within = gap <= K_REVIEW_COST_TOLERANCE + 1e-15
18727     stable = within and time_ok
18728     return {
18729         "best_k_by_mean_cost": int(best_k),
18730         "k8_mean_cost_yuan": k8_cost,
18731         "best_mean_cost_yuan": best_cost,
18732         "k8_cost_gap_vs_best": float(gap),
18733         "k8_within_ipct_of_best": bool(within),
18734         "k8_time_acceptable": bool(time_ok),
18735         "k8_stable": bool(stable),
18736         "keep_k": int(FIXED_SCENARIO_K),
18737         "reason": (
18738             "K=8 remains within 1% of the best reviewed K and solve time is acceptable; keep K=8."
18739             if stable
18740             else "K=8 is retained as the inherited Q2 value; this review does not re-pick K from the year."
18741         ),
18742     }
18743
18744
18745 def review_scenario_k(
18746     bundle,
18747     policy_start_soc: np.ndarray,
18748     *,
18749     k_candidates: tuple[int, ...] = K_REVIEW_CANDIDATES,
18750     reference_alpha: float = K_REVIEW_REFERENCE_ALPHA,
18751     n_validation: int | None = None,
18752     max_windows: int | None = None,
18753     log: bool = True,
18754 ) -> dict:
18755     indices = q2_aligned_calibration_indices(bundle.n_days())

```



```

18756 if max_windows is not None:
18757     indices = indices[: max(0, int(max_windows))]
18758 daily_rows: list[dict] = []
18759 window_rows: list[dict] = []
18760 started = perf_counter()
18761 for target in indices:
18762     cal_date = bundle.prices.dates[target].strftime("%Y-%m-%d")
18763     for k in k_candidates:
18764         prefix = (
18765             f"Q4-2 K-review K={k} alpha={reference_alpha:.2f} {cal_date}" if log else None
18766         )
18767         daily, summary = run_candidate_window(
18768             bundle,
18769             policy_start_soc,
18770             target,
18771             risk_alpha=float(reference_alpha),
18772             k_target=int(k),
18773             n_validation=n_validation,
18774             log_prefix=prefix,
18775         )
18776         for row in daily:
18777             daily_rows.append({"calibration_date": cal_date, **row})
18778         window_rows.append(
18779             {
18780                 "calibration_date": cal_date,
18781                 "calibration_day_index": int(target),
18782                 **summary,
18783             }
18784         )
18785         if log:
18786             print(
18787                 f"K-review {cal_date} K={k} mean_cost={summary['mean_validation_cost_yuan']:.2f} "
18788                 f"mean_s={summary['mean_elapsed_seconds']:.3f}",
18789                 flush=True,
18790             )
18791 window = pd.DataFrame(window_rows)
18792 by_k: dict[int, dict] = {}
18793 for k in k_candidates:
18794     part = window.loc[window["k_target"] == k]
18795     by_k[int(k)] = {
18796         "n_windows": int(len(part)),
18797         "n_validation_days_total": int(part["validation_days"].sum()) if not part.empty else 0,
18798         "mean_cost_per_day_yuan": float(part["mean_validation_cost_yuan"].mean())
18799         if not part.empty
18800         else float("nan"),
18801         "total_cost_yuan": float(part["total_cost_yuan"].sum()) if not part.empty else float("nan"),
18802         "total_planned_fee_yuan": float(part["total_planned_fee_yuan"].sum())
18803         if not part.empty
18804         else float("nan"),
18805         "total_emergency_fee_yuan": float(part["total_emergency_fee_yuan"].sum())
18806         if not part.empty
18807         else float("nan"),
18808         "total_unused_quota_kwh": float(part["total_unused_quota_kwh"].sum())
18809         if not part.empty
18810         else float("nan"),
18811         "mean_elapsed_seconds": float(part["mean_elapsed_seconds"].mean())
18812         if not part.empty
18813         else float("nan"),
18814     }
18815 decision = decide_keep_k8(by_k)
18816 report = {
18817     "k_candidates": [int(k) for k in k_candidates],
18818     "reference_alpha": float(reference_alpha),
18819     "n_windows": int(len(indices)),
18820     "calibration_dates": [
18821         bundle.prices.dates[i].strftime("%Y-%m-%d") for i in indices
18822     ],
18823     "policy_loop": "day_ahead_joint_q+locked_q+causal_mpc+next_day_value_cuts",

```

```

18824         "used_full_day_actual_scheduler": False,
18825         "no_future_info": True,
18826         "by_k": {str(k): v for k, v in by_k.items()},
18827         "decision": decision,
18828         "elapsed_seconds": float(perf_counter() - started),
18829     }
18830     return {
18831         "report": report,
18832         "window": window,
18833         "daily": pd.DataFrame(daily_rows),
18834     }
18835
18836
18837 def main(argv: list[str] | None = None) -> None:
18838     parser = argparse.ArgumentParser(description="Q4-2 K=4/8/12 review")
18839     parser.add_argument("--max-windows", type=int, default=None)
18840     parser.add_argument("--n-validation", type=int, default=None)
18841     args = parser.parse_args(argv)
18842     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
18843     bundle = load_q4_bundle(compute_price_mpc=False)
18844     policy_start_soc = load_policy_start_soc(bundle.n_days())
18845     result = review_scenario_k(
18846         bundle,
18847         policy_start_soc,
18848         n_validation=args.n_validation,
18849         max_windows=args.max_windows,
18850         log=True,
18851     )
18852     if not result["window"].empty:
18853         result["window"].to_csv(SUMMARY_CSV, index=False)
18854     if not result["daily"].empty:
18855         result["daily"].to_csv(DAILY_CSV, index=False)
18856     write_json(SUMMARY_JSON, result["report"])
18857     print(json.dumps(result["report"]["decision"], ensure_ascii=False, indent=2))
18858     print(json.dumps({str(k): v for k, v in result["report"]["by_k"].items()}, indent=2))
18859
18860
18861 if __name__ == "__main__":
18862     main()
18863
18864 # ===== q4/run_price_forecast_audit.py =====
18865 #!/usr/bin/env python3
18866 """Run the Q4 causal price-forecast audit. Does not start Q4-2/Q4-3 storage."""
18867
18868 from __future__ import annotations
18869
18870 import json
18871 import sys
18872 from pathlib import Path
18873
18874 ROOT = Path(__file__).resolve().parents[1]
18875 if str(ROOT) not in sys.path:
18876     sys.path.insert(0, str(ROOT))
18877
18878 from q3.config import PILOT_DATES # noqa: E402
18879 from q4.config import ( # noqa: E402
18880     ATTACH4,
18881     AUDIT_CSV,
18882     CAUSALITY_JSON,
18883     OUTPUT_DIR,
18884     TWODAY_JSON,
18885 )
18886 from q4.data import load_q4_prices # noqa: E402
18887 from q4.price_forecast import ( # noqa: E402
18888     assert_no_future_price_leak,
18889     fit_causal_price_forecasts,
18890     two_day_regression,
18891 )

```

```

18892
18893
18894 def main() -> None:
18895     data = load_q4_prices()
18896     archive = fit_causal_price_forecasts(data.price, data.dates)
18897     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
18898     archive.audit.to_csv(AUDIT_CSV, index=False)
18899
18900     twoday = two_day_regression(archive, PILOT_DATES)
18901     TWODAY_JSON.write_text(
18902         json.dumps(twoday, ensure_ascii=False, indent=2, default=str) + "\n",
18903         encoding="utf-8",
18904     )
18905
18906     causality = {"probes": []}
18907     for date in PILOT_DATES:
18908         i = data.date_index(date)
18909         assert_no_future_price_leak(data.price, data.dates, i, tau=0)
18910         assert_no_future_price_leak(data.price, data.dates, i, tau=36)
18911         causality["probes"].append(
18912             {
18913                 "date": date,
18914                 "day_ahead_insensitive_to_future": True,
18915                 "intraday_06_insensitive_to_unended_prices": True,
18916             }
18917         )
18918     causality["all_pass"] = True
18919     CAUSALITY_JSON.write_text(json.dumps(causality, indent=2) + "\n", encoding="utf-8")
18920
18921     audit = archive.audit
18922     valid = audit[audit["chosen_model"] != ""]
18923     summary = {
18924         "attach4": str(ATTACH4),
18925         "n_days": int(len(audit)),
18926         "n_days_with_day_ahead": int(len(valid)),
18927         "chosen_counts": valid["chosen_model_name"].value_counts().to_dict(),
18928         "day_ahead_mae_mean": float(valid["day_ahead_mae"].mean()),
18929         "day_ahead_rmse_mean": float(valid["day_ahead_rmse"].mean()),
18930         "intraday_06_mae_mean": float(valid["intraday_06_mae"].mean()),
18931         "intraday_12_mae_mean": float(valid["intraday_12_mae"].mean()),
18932         "intraday_18_mae_mean": float(valid["intraday_18_mae"].mean()),
18933         "audit_csv": str(AUDIT_CSV),
18934         "twoday_json": str(TWODAY_JSON),
18935         "causality_json": str(CAUSALITY_JSON),
18936         "pilot_dates": list(PILOT_DATES),
18937         "note": "Price module only. Does not run Q4-2/Q4-3 storage or write result4-*.xlsx.",
18938     }
18939     print(json.dumps(summary, indent=2, ensure_ascii=False))
18940     print("--- two-day regression ---")
18941     for date, payload in twoday.items():
18942         print(
18943             json.dumps(
18944                 {
18945                     "date": date,
18946                     "train_cutoff_date": payload["train_cutoff_date"],
18947                     "chosen_model_name": payload["chosen_model_name"],
18948                     "mae_prev_day": payload["mae_prev_day"],
18949                     "mae_weekday2": payload["mae_weekday2"],
18950                     "mae_weekday4": payload["mae_weekday4"],
18951                     "day_ahead_mae": payload["day_ahead_mae"],
18952                     "day_ahead_rmse": payload["day_ahead_rmse"],
18953                     "intraday_06_mae": payload["intraday_06_mae"],
18954                     "intraday_12_mae": payload["intraday_12_mae"],
18955                     "intraday_18_mae": payload["intraday_18_mae"],
18956                     "day_ahead_fallback": payload["day_ahead_fallback"],
18957                 },
18958                 indent=2,
18959                 ensure_ascii=False,

```

```

18960         )
18961     )
18962
18963
18964 if __name__ == "__main__":
18965     main()
18966
18967 # ===== q4/run_q4_3_sensitivity.py =====
18968 #!/usr/bin/env python3
18969 """Q4-3 sensitivities. Does not overwrite result4-3.xlsx or the official dispatch."""
18970
18971 from __future__ import annotations
18972
18973 import argparse
18974 import json
18975 import sys
18976 from pathlib import Path
18977 from time import perf_counter
18978
18979 ROOT = Path(__file__).resolve().parents[1]
18980 if str(ROOT) not in sys.path:
18981     sys.path.insert(0, str(ROOT))
18982
18983 import pandas as pd
18984
18985 from q3.config import PILOT_DATES
18986 from q4.audit import warmup_soc_continuity, write_json
18987 from q4.bundle import load_q4_bundle
18988 from q4.campaign import run_q4_3_campaign
18989 from q4.config import (
18990     OUTPUT_DIR,
18991     PAM_SEED,
18992     Q4_3_ORACLE_DAILY_CSV,
18993     Q4_3_ORACLE_DISPATCH_DIR,
18994     Q4_3_SENSITIVITY_SUMMARY_JSON,
18995 )
18996 from q4.q4_3_sensitivity import run_settlement_sensitivity, write_sensitivity_summary
18997
18998
18999 def _run_oracle(end_index: int | None = None, write_all_dispatch: bool = True) -> dict:
19000     started = perf_counter()
19001     bundle = load_q4_bundle(compute_price_mpc=False)
19002     if end_index is None:
19003         end_index = bundle.n_days() - 1
19004     result = run_q4_3_campaign(
19005         bundle,
19006         end_index,
19007         detail_dates=PILOT_DATES,
19008         out_dir=OUTPUT_DIR,
19009         write_all_dispatch=write_all_dispatch,
19010         price_mode="oracle",
19011         dispatch_dir=Q4_3_ORACLE_DISPATCH_DIR,
19012         daily_name=Q4_3_ORACLE_DAILY_CSV.name,
19013         update_name="q4_3_oracle_update_log.csv",
19014         ledger_name="q4_3_oracle_cost_ledger.csv",
19015         commitment_prefix="q4_3_oracle_commitment_versions",
19016         unlink_existing=True,
19017     )
19018     daily = pd.read_csv(Q4_3_ORACLE_DAILY_CSV)
19019     soc = warmup_soc_continuity(daily)
19020     n_pass = int(daily["pass"].sum()) if "pass" in daily.columns else 0
19021     meta = {
19022         "system": "q4_3_price_oracle",
19023         "executable": False,
19024         "changes_main_scheme": False,
19025         "n_days": int(len(daily)),
19026         "annual_cost_yuan": float(daily["total_cost_yuan"].sum()),
19027         "end_soc": float(result["end_soc"]),

```

```

19028     "soc_continuity": soc,
19029     "n_days_pass": n_pass,
19030     "pam_seed": PAM_SEED,
19031     "elapsed_seconds": float(perf_counter() - started),
19032     "note": (
19033         "Day-ahead and remaining prices equal today's actual path. "
19034         "Next-day value cuts remain causal. Not an official Q4-3 result."
19035     ),
19036 }
19037 write_json(OUTPUT_DIR / "q4_3_oracle_run_metadata.json", meta)
19038 if not soc["pass"] or n_pass != len(daily):
19039     raise SystemExit("Q4-3 price-oracle stream failed physical/pass checks")
19040 return meta
19041
19042
19043 def main(argv: list[str] | None = None) -> None:
19044     parser = argparse.ArgumentParser(description="Q4-3 settlement and oracle sensitivities")
19045     parser.add_argument("--skip-settlement", action="store_true")
19046     parser.add_argument("--skip-oracle", action="store_true")
19047     parser.add_argument("--oracle-end-index", type=int, default=None)
19048     args = parser.parse_args(argv)
19049     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
19050     settlement = None
19051     if not args.skip_settlement:
19052         settlement = run_settlement_sensitivity()
19053         print(json.dumps(settlement["summary"], ensure_ascii=False, indent=2))
19054     oracle_daily = None
19055     if not args.skip_oracle:
19056         meta = _run_oracle(end_index=args.oracle_end_index)
19057         print(json.dumps({k: meta[k] for k in ("n_days", "annual_cost_yuan", "elapsed_seconds")}, indent=2))
19058         oracle_daily = pd.read_csv(Q4_3_ORACLE_DAILY_CSV)
19059     if settlement is None:
19060         settlement = {"summary": json.loads(Path(OUTPUT_DIR / "q4_3_settlement_sensitivity.json").read_text()) if (OUTPUT_DIR / "
19061             q4_3_settlement_sensitivity.json").exists() else {"summary": {}}}
19062         payload = write_sensitivity_summary(settlement, oracle_daily)
19063         print(json.dumps({"wrote": str(Q4_3_SENSITIVITY_SUMMARY_JSON), "keys": list(payload)}, indent=2))
19064
19065 if __name__ == "__main__":
19066     main()
19067
19068 # ===== q4/run_q4_full.py =====
19069 #!/usr/bin/env python3
19070 """365-day Q4-2 / Q4-3 stream with daily 10-minute dispatch. Does not touch result3.xlsx."""
19071
19072 from __future__ import annotations
19073
19074 import argparse
19075 import json
19076 import subprocess
19077 import sys
19078 from pathlib import Path
19079 from time import perf_counter
19080
19081 ROOT = Path(__file__).resolve().parents[1]
19082 if str(ROOT) not in sys.path:
19083     sys.path.insert(0, str(ROOT))
19084
19085 from q4.audit import warmup_soc_continuity, write_json # noqa: E402
19086 from q4.bundle import load_q4_bundle # noqa: E402
19087 from q4.campaign import run_q4_2_campaign, run_q4_3_campaign # noqa: E402
19088 from q4.config import OUTPUT_DIR, PAM_SEED # noqa: E402
19089 import pandas as pd # noqa: E402
19090
19091
19092 def _git_hash() -> str:
19093     try:
19094         return subprocess.check_output(["git", "rev-parse", "HEAD"], cwd=ROOT, text=True).strip()

```

```

19095     except (OSError, subprocess.CalledProcessError):
19096         return "unknown"
19097
19098
19099 def main(argv: list[str] | None = None) -> None:
19100     parser = argparse.ArgumentParser(description="Q4 full-year stream")
19101     parser.add_argument("--system", choices=("q4_2", "q4_3"), required=True)
19102     args = parser.parse_args(argv)
19103     started = perf_counter()
19104     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
19105     bundle = load_q4_bundle(compute_price_mpc=False)
19106     end_index = bundle.n_days() - 1
19107     print(
19108         f"Full-year {args.system} through {bundle.prices.dates[end_index].strftime('%Y-%m-%d')} "
19109         f"({end_index + 1} days)",
19110         flush=True,
19111     )
19112     if args.system == "q4_2":
19113         result = run_q4_2_campaign(
19114             bundle, end_index, detail_dates=(), out_dir=OUTPUT_DIR, write_all_dispatch=True
19115         )
19116         daily = pd.read_csv(OUTPUT_DIR / "q4_2_warmup_daily.csv")
19117         audit_path = OUTPUT_DIR / "q4_2_physical_audit.json"
19118         meta_path = OUTPUT_DIR / "q4_2_run_metadata.json"
19119         alpha_path = OUTPUT_DIR / "q4_2_alpha_selection.csv"
19120         if alpha_path.exists():
19121             alpha = pd.read_csv(alpha_path)
19122             selected = (
19123                 alpha.loc[alpha["selected"].astype(str).str.lower().isin(("true", "1"))]
19124                 if "selected" in alpha.columns
19125                 else alpha
19126             )
19127             meta_alpha = {
19128                 "n_calibration_windows": int(selected["calibration_date"].nunique())
19129                 if "calibration_date" in selected.columns
19130                 else int(len(selected)),
19131                 "selected_alphas": sorted(
19132                     {float(v) for v in selected["risk_alpha"].dropna().unique()}
19133                 ),
19134                 if "risk_alpha" in selected.columns
19135                 else [],
19136             }
19137         else:
19138             meta_alpha = {}
19139     else:
19140         result = run_q4_3_campaign(
19141             bundle, end_index, detail_dates=(), out_dir=OUTPUT_DIR, write_all_dispatch=True
19142         )
19143         daily = pd.read_csv(OUTPUT_DIR / "q4_3_warmup_daily.csv")
19144         audit_path = OUTPUT_DIR / "q4_3_physical_audit.json"
19145         meta_path = OUTPUT_DIR / "q4_3_run_metadata.json"
19146         meta_alpha = {}
19147     soc = warmup_soc_continuity(daily)
19148     n_pass = int(daily["pass"].sum()) if "pass" in daily.columns else 0
19149     meta = {
19150         "git_commit": _git_hash(),
19151         "system": args.system,
19152         "full_year": True,
19153         "n_days": int(len(daily)),
19154         "annual_cost_yuan": float(daily["total_cost_yuan"].sum()),
19155         "soc_continuity": soc,
19156         "n_days_pass": n_pass,
19157         "end_soc": float(result["end_soc"]),
19158         "pam_seed": PAM_SEED,
19159         "load_information_case": "causal_load_main",
19160         "settlement_rule": "delivery_time_actual_price",
19161         "elapsed_seconds": float(perf_counter() - started),
19162         "wrote_result4_xlsx": False,

```

```

19163     }
19164     meta.update(meta_alpha)
19165     if args.system == "q4_2":
19166         meta["alpha_policy_loop"] = (
19167             "day_ahead_joint_q+locked_q+causal_mpc+next_day_value_cuts"
19168         )
19169         meta["used_full_day_actual_scheduler"] = False
19170         two_day = {}
19171         for date in ("2025-02-01", "2025-06-21"):
19172             hit = daily.loc[daily["date"].astype(str) == date]
19173             if not hit.empty:
19174                 two_day[date] = float(hit.iloc[0]["total_cost_yuan"])
19175             if two_day:
19176                 meta["two_day_costs"] = two_day
19177         write_json(audit_path, {"soc_continuity": soc, "n_days": int(len(daily)), "n_days_pass": n_pass})
19178         write_json(meta_path, meta)
19179         print(json.dumps({k: meta[k] for k in ("system", "n_days", "annual_cost_yuan", "elapsed_seconds")}, indent=2))
19180         if not soc["pass"] or n_pass != len(daily):
19181             raise SystemExit(f"{args.system} annual stream failed physical/pass checks")
19182
19183
19184 if __name__ == "__main__":
19185     main()
19186
19187 # ===== q4/run_q4_twoday.py =====
19188 #!/usr/bin/env python3
19189 """Q4-2 and Q4-3 two-day regression. Does not write result4-*.xlsx or run 365 days."""
19190
19191 from __future__ import annotations
19192
19193 import argparse
19194 import json
19195 import subprocess
19196 import sys
19197 from pathlib import Path
19198 from time import perf_counter
19199
19200 import pandas as pd
19201
19202 ROOT = Path(__file__).resolve().parents[1]
19203 if str(ROOT) not in sys.path:
19204     sys.path.insert(0, str(ROOT))
19205
19206 from q3.config import PILOT_DATES # noqa: E402
19207 from q4.audit import ( # noqa: E402
19208     ledger_from_dispatch_q42,
19209     ledger_from_dispatch_q43,
19210     physical_from_dispatch,
19211     prefix_lock_probe,
19212     q_unchanged_probe,
19213     warmup_soc_continuity,
19214     write_json,
19215 )
19216 from q4.bundle import load_q4_bundle # noqa: E402
19217 from q4.campaign import ( # noqa: E402
19218     last_index_for_dates,
19219     load_q42_detail_from_disk,
19220     load_q43_detail_from_disk,
19221     run_q4_2_campaign,
19222     run_q4_3_campaign,
19223 )
19224 from q4.config import OUTPUT_DIR, PAM_SEED # noqa: E402
19225 from q4.info_set import probe_executed_prefix # noqa: E402
19226
19227
19228 def _git_hash() -> str:
19229     try:
19230         return subprocess.check_output(["git", "rev-parse", "HEAD"], cwd=ROOT, text=True).strip()

```

```

19231     except (OSError, subprocess.CalledProcessError):
19232         return "unknown"
19233
19234
19235 def _run_unittest() -> dict:
19236     proc = subprocess.run(
19237         [
19238             sys.executable,
19239             "-m",
19240             "unittest",
19241             "q2.test_q2",
19242             "q3.test_q3",
19243             "q3.test_q3_result3_export",
19244             "q4.test_q4_price_forecast",
19245             "q4.test_q4_dispatch",
19246             "-v",
19247         ],
19248         cwd=ROOT,
19249         text=True,
19250         capture_output=True,
19251     )
19252     return {
19253         "command": (
19254             "python -m unittest q2.test_q2 q3.test_q3 q3.test_q3_result3_export "
19255             "q4.test_q4_price_forecast q4.test_q4_dispatch -v"
19256         ),
19257         "returncode": proc.returncode,
19258         "stdout_tail": "\n".join(proc.stdout.splitlines()[-60:]),
19259         "stderr_tail": "\n".join(proc.stderr.splitlines()[-40:]),
19260     }
19261
19262
19263 def _audit_detail(q42, q43):
19264     phys2, phys3 = {"days": {}}, {"days": {}}
19265     all_pass = True
19266     for date, result in q42.items():
19267         phys = physical_from_dispatch(result.dispatch, q4_3=False)
19268         ledger = ledger_from_dispatch_q42(result.dispatch)
19269         recon = float(ledger["total_cost_yuan"].sum())
19270         ok = (
19271             phys["pass"]
19272             and q_unchanged_probe(result.q, result.dispatch)
19273             and abs(recon - float(result.summary["total_cost_yuan"])) < 1e-4
19274         )
19275         all_pass = all_pass and ok
19276         phys2["days"][date] = {**phys, "ledger_recompute_yuan": recon, "ok": ok}
19277     for date, result in q43.items():
19278         phys = physical_from_dispatch(result.dispatch, q4_3=True)
19279         ledger = ledger_from_dispatch_q43(result.dispatch)
19280         recon = float(ledger["total_cost_yuan"].sum())
19281         ok = (
19282             phys["pass"]
19283             and prefix_lock_probe(result.update_log)
19284             and abs(recon - float(result.summary["total_cost_yuan"])) < 1e-4
19285             and result.summary["locked_period_violations"] == 0
19286         )
19287         all_pass = all_pass and ok
19288         phys3["days"][date] = {**phys, "ledger_recompute_yuan": recon, "ok": ok}
19289     phys2["all_pass"] = all(day["ok"] for day in phys2["days"].values()) if phys2["days"] else False
19290     phys3["all_pass"] = all(day["ok"] for day in phys3["days"].values()) if phys3["days"] else False
19291     return phys2, phys3, all_pass and phys2["all_pass"] and phys3["all_pass"]
19292
19293
19294 def main(argv: list[str] | None = None) -> None:
19295     parser = argparse.ArgumentParser(description="Q4-2/Q4-3 two-day regression")
19296     parser.add_argument("--skip-q4-2", action="store_true", help="reuse streamed Q4-2 CSVs")
19297     parser.add_argument("--skip-q4-3", action="store_true", help="reuse streamed Q4-3 CSVs")
19298     parser.add_argument("--skip-tests", action="store_true", help="skip the unittest pre-check")

```



```

19299     args = parser.parse_args(argv)
19300     started = perf_counter()
19301     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
19302     if args.skip_tests:
19303         tests = {"command": "skipped", "returncode": 0, "stdout_tail": "", "stderr_tail": ""}
19304     else:
19305         tests = _run_unittest()
19306         if tests["returncode"] != 0:
19307             write_json(OUTPUT_DIR / "q4_twoday_run_metadata.json", {"tests": tests, "aborted": True})
19308             raise SystemExit(f"unit tests failed:\n{tests['stderr_tail']}\n{tests['stdout_tail']}")
19309     bundle = load_q4_bundle(compute_price_mpc=False)
19310     end_index = last_index_for_dates(bundle, PILOT_DATES)
19311     print(f"Two-day regression with warmup through {bundle.prices.dates[end_index].strftime('%Y-%m-%d')}", flush=True)
19312     if args.skip_q4_2:
19313         q42 = load_q42_detail_from_disk(OUTPUT_DIR, PILOT_DATES)
19314         print("Reusing Q4-2 dispatch CSVs from disk", flush=True)
19315     else:
19316         q42 = run_q4_2_campaign(bundle, end_index, PILOT_DATES, OUTPUT_DIR)
19317     if args.skip_q4_3:
19318         q43 = load_q43_detail_from_disk(OUTPUT_DIR, PILOT_DATES)
19319         print("Reusing Q4-3 dispatch CSVs from disk", flush=True)
19320     else:
19321         q43 = run_q4_3_campaign(bundle, end_index, PILOT_DATES, OUTPUT_DIR)
19322     phys2, phys3, all_pass = _audit_detail(q42["detail"], q43["detail"])
19323     soc2 = warmup_soc_continuity(pd.read_csv(OUTPUT_DIR / "q4_2_warmup_daily.csv"))
19324     soc3 = warmup_soc_continuity(pd.read_csv(OUTPUT_DIR / "q4_3_warmup_daily.csv"))
19325     info_rows = []
19326     for date, result in q42["detail"].items():
19327         q43_res = q43["detail"][date]
19328         alpha = result.summary["risk_alpha"]
19329         if alpha is not None:
19330             alpha = float(alpha)
19331             info = probe_executed_prefix(
19332                 bundle,
19333                 result.day_index,
19334                 float(result.summary["soc_start_kwh"]),
19335                 alpha,
19336                 result.dispatch,
19337                 q43_res,
19338                 tau=40,
19339             )
19340             info_rows.append(info)
19341             all_pass = all_pass and info["pass"]
19342             print(f"info-set {date}: {info}", flush=True)
19343     info_payload = {"days": info_rows, "all_pass": all(row["pass"] for row in info_rows)}
19344     write_json(OUTPUT_DIR / "q4_2_physical_audit.json", {**phys2, "soc_continuity": soc2})
19345     write_json(OUTPUT_DIR / "q4_3_physical_audit.json", {**phys3, "soc_continuity": soc3})
19346     write_json(OUTPUT_DIR / "q4_2_info_set_audit.json", info_payload)
19347     write_json(OUTPUT_DIR / "q4_3_info_set_audit.json", info_payload)
19348     all_pass = all_pass and soc2["pass"] and soc3["pass"] and info_payload["all_pass"]
19349     meta = {
19350         "git_commit": _git_hash(),
19351         "pilot_dates": list(PILOT_DATES),
19352         "warmup_start": "2025-01-01",
19353         "warmup_end": bundle.prices.dates[end_index].strftime("%Y-%m-%d"),
19354         "n_days_solved_each_system": end_index + 1,
19355         "pam_seed": PAM_SEED,
19356         "pam_algorithm": "deterministic_build_then_swap",
19357         "load_information_case": "causal_load_main",
19358         "q4_2_year_end_rule": "q2_accepted_no_hard_terminal",
19359         "q4_3_year_end": "A_q2_aligned_1200",
19360         "q4_3_strategy": "M1_M6",
19361         "settlement_rule": "delivery_time_actual_price",
19362         "wrote_result4_xlsx": False,
19363         "full_year": False,
19364         "elapsed_seconds": float(perf_counter() - started),
19365         "tests": tests,
19366         "q4_2_all_pass": phys2["all_pass"],

```

```

19367         "q4_3_all_pass": phys3["all_pass"],
19368         "all_pass": all_pass,
19369         "skip_q4_2": bool(args.skip_q4_2),
19370         "skip_q4_3": bool(args.skip_q4_3),
19371         "note": (
19372             "Two-day regression only. January through each target date is causal "
19373             "SOC warmup. result4-2/result4-3 are not written in this stage."
19374         ),
19375     }
19376     write_json(OUTPUT_DIR / "q4_2_run_metadata.json", {**meta, "system": "Q4-2"})
19377     write_json(OUTPUT_DIR / "q4_3_run_metadata.json", {**meta, "system": "Q4-3"})
19378     write_json(OUTPUT_DIR / "q4_twoday_run_metadata.json", meta)
19379     print(json.dumps({"all_pass": all_pass, "elapsed_seconds": meta["elapsed_seconds"]}, indent=2))
19380     if not all_pass:
19381         raise SystemExit("two-day physical, ledger, or information-set audit failed")
19382
19383
19384 if __name__ == "__main__":
19385     main()
19386
19387 # ===== q4/scenarios.py =====
19388 """Triple residual K-medoids for Q4-2. Residuals are paired by historical date."""
19389
19390 from __future__ import annotations
19391
19392 from dataclasses import dataclass
19393
19394 import numpy as np
19395
19396 from q2.scenarios import pam, robust_scale
19397 from q4.bundle import Q4Bundle
19398 from q4.config import FIXED_SCENARIO_K, RESIDUAL_POOL_DAYS, T
19399
19400
19401 @dataclass(frozen=True)
19402 class TripleScenarioSet:
19403     target_index: int
19404     history_end_exclusive: int
19405     pool_indices: np.ndarray
19406     medoid_indices: np.ndarray
19407     cluster_labels: np.ndarray
19408     probabilities: np.ndarray
19409     scale_load: float
19410     scale_pv: float
19411     scale_price: float
19412     k_effective: int
19413     k_target: int
19414
19415
19416 def complete_triple_days(bundle: Q4Bundle, history_end_exclusive: int) -> np.ndarray:
19417     """Days strictly before the cutoff with archived causal load, PV, and price forecasts."""
19418     ok = []
19419     for i in range(max(0, history_end_exclusive)):
19420         if not np.isfinite(bundle.price_archive.day_ahead[i]).all():
19421             continue
19422         if not np.isfinite(bundle.q2_forecast.load_hat[i]).all():
19423             continue
19424         if not np.isfinite(bundle.q2_forecast.pv_hat[i]).all():
19425             continue
19426         ok.append(i)
19427     if not ok:
19428         return np.zeros(0, dtype=int)
19429     pool = np.asarray(ok, dtype=int)
19430     return pool[pool >= max(0, history_end_exclusive - RESIDUAL_POOL_DAYS)]
19431
19432
19433 def residual_stack(bundle: Q4Bundle, indices: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
19434     load_r = bundle.q2.load[indices] - bundle.q2_forecast.load_hat[indices]

```

```

19435     pv_r = bundle.q2.pv[indices] - bundle.q2_forecast.pv_hat[indices]
19436     price_r = bundle.prices.price[indices] - bundle.price_archive.day_ahead[indices]
19437     return load_r, pv_r, price_r
19438
19439
19440 def triple_distance_matrix(
19441     load_residual: np.ndarray, pv_residual: np.ndarray, price_residual: np.ndarray
19442 ) -> tuple[np.ndarray, float, float, float]:
19443     sl = robust_scale(load_residual)
19444     sp = robust_scale(pv_residual)
19445     spr = robust_scale(price_residual)
19446     dl = (load_residual[:, None, :] - load_residual[None, :, :]) / sl
19447     dp = (pv_residual[:, None, :] - pv_residual[None, :, :]) / sp
19448     dr = (price_residual[:, None, :] - price_residual[None, :, :]) / spr
19449     d2 = np.mean(dl * dl + dp * dp + dr * dr, axis=2)
19450     return np.sqrt(np.maximum(d2, 0.0)), sl, sp, spr
19451
19452
19453 def build_triple_scenarios(
19454     bundle: Q4Bundle,
19455     target_index: int,
19456     history_end_exclusive: int | None = None,
19457     k_target: int = FIXED_SCENARIOIO_K,
19458 ) -> TripleScenarioSet | None:
19459     history_end = target_index if history_end_exclusive is None else history_end_exclusive
19460     if history_end <= 0 or history_end > target_index:
19461         return None
19462     pool = complete_triple_days(bundle, history_end)
19463     if len(pool) == 0:
19464         return None
19465     load_r, pv_r, price_r = residual_stack(bundle, pool)
19466     distance, sl, sp, spr = triple_distance_matrix(load_r, pv_r, price_r)
19467     k_eff = int(min(k_target, len(pool)))
19468     local_medoids, labels = pam(distance, k_eff)
19469     counts = np.bincount(labels, minlength=k_eff)
19470     probabilities = counts.astype(float) / len(pool)
19471     if not np.isclose(probabilities.sum(), 1.0, atol=1e-12):
19472         raise AssertionError("Q4-2 scenario probabilities do not sum to one")
19473     return TripleScenarioSet(
19474         target_index=target_index,
19475         history_end_exclusive=history_end,
19476         pool_indices=pool,
19477         medoid_indices=pool[local_medoids],
19478         cluster_labels=labels,
19479         probabilities=probabilities,
19480         scale_load=sl,
19481         scale_pv=sp,
19482         scale_price=spr,
19483         k_effective=k_eff,
19484         k_target=int(k_target),
19485     )
19486
19487
19488 def scenario_paths(
19489     bundle: Q4Bundle,
19490     scenarios: TripleScenarioSet,
19491     base_load: np.ndarray,
19492     base_pv: np.ndarray,
19493     base_price: np.ndarray,
19494 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
19495     m = scenarios.medoid_indices
19496     load_r, pv_r, price_r = residual_stack(bundle, m)
19497     load = np.maximum(0.0, base_load[None, :] + load_r)
19498     pv = np.maximum(0.0, base_pv[None, :] + pv_r)
19499     price = np.maximum(0.0, base_price[None, :] + price_r)
19500     return load, pv, price
19501
19502

```

```

19503 def posterior_weights(
19504     bundle: Q4Bundle,
19505     scenarios: TripleScenarioSet,
19506     observed_load_r: np.ndarray,
19507     observed_pv_r: np.ndarray,
19508     observed_price_r: np.ndarray,
19509 ) -> tuple[np.ndarray, float]:
19510     t = len(observed_load_r)
19511     if t == 0:
19512         return scenarios.proBABILITIES.copy(), np.nan
19513     m = scenarios.medoid_indices
19514     load_r, pv_r, price_r = residual_stack(bundle, m)
19515     lr = load_r[:, :t]
19516     pr = pv_r[:, :t]
19517     rr = price_r[:, :t]
19518     delta = np.mean(
19519         ((lr - observed_load_r) / scenarios.scale_load) ** 2
19520         + ((pr - observed_pv_r) / scenarios.scale_pv) ** 2
19521         + ((rr - observed_price_r) / scenarios.scale_price) ** 2,
19522         axis=1,
19523     )
19524     if len(m) > 1:
19525         pair = []
19526         for a in range(len(m)):
19527             for b in range(a + 1, len(m)):
19528                 pair.append(
19529                     np.mean(
19530                         ((lr[a] - lr[b]) / scenarios.scale_load) ** 2
19531                         + ((pr[a] - pr[b]) / scenarios.scale_pv) ** 2
19532                         + ((rr[a] - rr[b]) / scenarios.scale_price) ** 2
19533                     )
19534                 )
19535         bandwidth = max(float(np.sqrt(np.median(pair))), 1e-6)
19536     else:
19537         bandwidth = 1.0
19538     logw = np.log(np.maximum(scenarios.proBABILITIES, 1e-300)) - delta / (
19539         2.0 * bandwidth * bandwidth
19540     )
19541     logw -= np.max(logw)
19542     weights = np.exp(logw)
19543     weights /= weights.sum()
19544     return weights, bandwidth
19545
19546 # ===== scripts/export_paper_q2_q3.py =====
19547 #!/usr/bin/env python3
19548 """Export Q2/Q3 paper tables and figures from signed-off workbooks and ledgers.
19549
19550 Reads only accepted outputs:
19551 - Q2: `output/result2.xlsx` (V2 政策一致) 与 `output/q2_policy_consistent/` 年度台账
19552 - Q3: output/result3.xlsx and output/q3_full_annual/*.csv
19553
19554 Writes LaTeX tables under paper/manuscript/ and PNG/PDF figures under paper/overleaf/.
19555 """
19556
19557 from __future__ import annotations
19558
19559 import json
19560 import sys
19561 from datetime import datetime, time
19562 from pathlib import Path
19563
19564 import matplotlib.dates as mdates
19565 import matplotlib.pyplot as plt
19566 import numpy as np
19567 import pandas as pd
19568 from matplotlib import font_manager
19569 from openpyxl import load_workbook
19570

```

```

19571 ROOT = Path(__file__).resolve().parents[1]
19572 if str(ROOT) not in sys.path:
19573     sys.path.insert(0, str(ROOT))
19574
19575 PAPER = ROOT / "paper" / "overleaf"
19576 MANUSCRIPT = ROOT / "paper" / "manuscript"
19577 TABLES_Q2 = MANUSCRIPT / "tables" / "q2"
19578 TABLES_Q3 = MANUSCRIPT / "tables" / "q3"
19579 FIG_Q2 = PAPER / "figures" / "q2"
19580 FIG_Q3 = PAPER / "figures" / "q3"
19581
19582 RESULT2 = ROOT / "output" / "result2.xlsx"
19583 RESULT3 = ROOT / "output" / "result3.xlsx"
19584 Q2_DAILY = ROOT / "output" / "q2_policy_consistent" / "feb_dec_daily_summary.csv"
19585 Q2_COST = ROOT / "output" / "q2_policy_consistent" / "cost_breakdown.csv"
19586 Q2_K = ROOT / "output" / "q2_policy_consistent" / "k_sensitivity.csv"
19587 Q2_CAL = ROOT / "output" / "q2_policy_consistent" / "frozen_policy_calendar.csv"
19588 Q3_STRATEGY = ROOT / "output" / "q3_full_annual" / "q3_annual_strategy_comparison.csv"
19589 Q3_MONTHLY = ROOT / "output" / "q3_full_annual" / "q3_annual_monthly.csv"
19590 Q3_UPDATE = ROOT / "output" / "q3_full_annual" / "q3_update_log.csv"
19591
19592 SAMPLE_DATE = "2025-06-21"
19593 SAMPLE_SLOTS = (
19594     "10:00-10:10",
19595     "12:00-12:10",
19596     "14:00-14:10",
19597     "16:00-16:10",
19598     "18:00-18:10",
19599     "20:00-20:10",
19600 )
19601
19602 BLUE = "#276FBF"
19603 RED = "#C0392B"
19604 GRAY = "#6B7280"
19605 GREEN = "#2F6F4E"
19606
19607
19608 def fmt_num(value: float, digits: int = 4) -> str:
19609     return f"{float(value):.{digits}f}"
19610
19611
19612 def latex_code(text: str) -> str:
19613     escaped = text.replace("_", "\\_")
19614     return f"\\texttt{{{escaped}}}"
19615
19616
19617 def latex_escape(text: str) -> str:
19618     return (
19619         text.replace("\\", "\\textbackslash{")
19620         .replace("_", "\\_")
19621         .replace("&", "\\&")
19622         .replace("%", "\\%")
19623     )
19624
19625
19626 def configure_style() -> str:
19627     available = {font.name for font in font_manager.fontManager.ttflist}
19628     candidates = (
19629         "Noto Sans CJK SC",
19630         "Microsoft YaHei",
19631         "SimHei",
19632         "SimSun",
19633         "PingFang SC",
19634         "Hiragino Sans GB",
19635         "Heiti SC",
19636         "Songti SC",
19637         "Arial Unicode MS",
19638     )

```

```

19639     font = next((name for name in candidates if name in available), "DejaVu Sans")
19640     plt.rcParams.update(
19641         {
19642             "font.family": "sans-serif",
19643             "font.sans-serif": [font],
19644             "axes.unicode_minus": False,
19645             "figure.facecolor": "white",
19646             "axes.facecolor": "white",
19647             "font.size": 10,
19648             "axes.titlesize": 12,
19649             "axes.labelsize": 10,
19650             "legend.fontsize": 9,
19651             "pdf.fonttype": 42,
19652         }
19653     )
19654     return font
19655
19656
19657 def save_figure(fig: plt.Figure, stem: str, out_dir: Path) -> None:
19658     out_dir.mkdir(parents=True, exist_ok=True)
19659     fig.savefig(out_dir / f"{stem}.pdf", bbox_inches="tight")
19660     fig.savefig(out_dir / f"{stem}.png", dpi=320, bbox_inches="tight")
19661     plt.close(fig)
19662
19663
19664 def write_text(path: Path, content: str) -> None:
19665     path.parent.mkdir(parents=True, exist_ok=True)
19666     path.write_text(content, encoding="utf-8")
19667
19668
19669 def load_purchase_row(path: Path, sheet: str, date_text: str) -> tuple[list[str], dict[str, float], float, float]:
19670     wb = load_workbook(path, read_only=True, data_only=True)
19671     ws = wb[sheet]
19672     rows = ws.iter_rows(values_only=True)
19673     headers = [str(h) if h is not None else "" for h in next(rows)]
19674     target = pd.Timestamp(date_text).to_pydatetime()
19675     values: dict[str, float] = {}
19676     day_total = 0.0
19677     day_fee = 0.0
19678     for row in rows:
19679         if row[0] is None:
19680             continue
19681         if pd.Timestamp(row[0]).normalize() != pd.Timestamp(target).normalize():
19682             continue
19683         for idx, header in enumerate(headers[1:], start=1):
19684             if idx >= len(row):
19685                 break
19686             if header in SAMPLE_SLOTS:
19687                 values[header] = float(row[idx] or 0.0)
19688         if len(row) > 145:
19689             day_total = float(row[145] or 0.0)
19690         if len(row) > 146:
19691             day_fee = float(row[146] or 0.0)
19692         break
19693     wb.close()
19694     if not values:
19695         raise ValueError(f"missing purchase row for {date_text} in {path}")
19696     return headers, values, day_total, day_fee
19697
19698
19699 def load_charge_blocks(path: Path, date_text: str) -> tuple[list[tuple[str, float, float]], float, float]:
19700     wb = load_workbook(path, read_only=True, data_only=True)
19701     ws = wb["充放电量"]
19702     blocks: list[tuple[str, float, float]] = []
19703     soc_start = soc_end = 0.0
19704     target = pd.Timestamp(date_text).normalize()
19705     for row in ws.iter_rows(values_only=True):
19706         if row[0] == "日期" or row[1] == "时间段":

```

```

19707         continue
19708     if row[0] is not None:
19709         try:
19710             row_date = pd.Timestamp(row[0]).normalize()
19711         except (ValueError, TypeError):
19712             continue
19713         if row_date == target:
19714             soc_start = float(row[5] or 0.0)
19715     if row[1] is None:
19716         continue
19717     if row[0] is not None:
19718         try:
19719             row_date = pd.Timestamp(row[0]).normalize()
19720         except (ValueError, TypeError):
19721             row_date = None
19722         if row_date == target:
19723             blocks.append((str(row[1]), float(row[2] or 0.0), float(row[3] or 0.0)))
19724             if str(row[4]) in ("24:00", "0:00+1") or row[4] == time(0, 0):
19725                 soc_end = float(row[5] or 0.0)
19726             elif row[0] is None and blocks and len(blocks) < 6:
19727                 blocks.append((str(row[1]), float(row[2] or 0.0), float(row[3] or 0.0)))
19728             if str(row[4]) in ("24:00", "0:00+1"):
19729                 soc_end = float(row[5] or 0.0)
19730     wb.close()
19731     if len(blocks) != 6:
19732         raise ValueError(f"expected 6 charge blocks for {date_text}, got {len(blocks)}")
19733     return blocks, soc_start, soc_end
19734
19735
19736 def load_emergency_segments(path: Path, date_text: str) -> list[tuple[str, float]]:
19737     wb = load_workbook(path, read_only=True, data_only=True)
19738     ws = wb["紧急购电量"]
19739     target = pd.Timestamp(date_text).normalize()
19740     segments: list[tuple[str, float]] = []
19741     for row in ws.iter_rows(min_row=2, values_only=True):
19742         if row[0] is None:
19743             continue
19744         if pd.Timestamp(row[0]).normalize() != target:
19745             continue
19746         segments.append((str(row[1]), float(row[2] or 0.0)))
19747     wb.close()
19748     return segments
19749
19750
19751 def load_adjust_row(path: Path, date_text: str) -> dict[str, float]:
19752     wb = load_workbook(path, read_only=True, data_only=True)
19753     ws = wb["调整购电量"]
19754     rows = ws.iter_rows(values_only=True)
19755     headers = [str(h) if h is not None else "" for h in next(rows)]
19756     target = pd.Timestamp(date_text).normalize()
19757     values: dict[str, float] = {}
19758     for row in rows:
19759         if row[0] is None:
19760             continue
19761         if pd.Timestamp(row[0]).normalize() != target:
19762             continue
19763         for idx, header in enumerate(headers[1:], start=1):
19764             if header in SAMPLE_SLOTS:
19765                 values[header] = float(row[idx] or 0.0)
19766         break
19767     wb.close()
19768     if not values:
19769         raise ValueError(f"missing adjust row for {date_text}")
19770     return values
19771
19772
19773 def render_triplet_table(
19774     caption: str,

```

```

19775     label: str,
19776     value_title: str,
19777     rows: list[tuple[str, str]],
19778     footer: tuple[str, str, str, str] | None = None,
19779 ) -> str:
19780     lines = [
19781         "\\begin{table}[htbp]",
19782         "\\centering",
19783         f"\\caption{{{caption}}}{\\label{{{label}}}}",
19784         "\\normalsize\\setlength{\\tabcolsep}{5pt}",
19785         "\\begin{tabular}{@{}cccccc@{}}",
19786         "\\toprule",
19787         f"时间段 & {value_title} & 时间段 & {value_title} & 时间段 & {value_title} \\",
19788         "\\midrule",
19789     ]
19790     for i in range(0, len(rows), 3):
19791         chunk = rows[i : i + 3]
19792         while len(chunk) < 3:
19793             chunk.append("", "")
19794         lines.append(
19795             f"{chunk[0][0]} & {chunk[0][1]} & {chunk[1][0]} & {chunk[1][1]} & "
19796             f"{chunk[2][0]} & {chunk[2][1]} \\",
19797         )
19798     if footer:
19799         left_label, left_val, right_label, right_val = footer
19800         lines.append(
19801             f"{left_label} & \\multicolumn{2}{{{c}}}{{{left_val}}}"
19802             f"{right_label} & \\multicolumn{2}{{{c}}}{{{right_val}}} \\",
19803         )
19804     lines.extend(["\\bottomrule", "\\end{tabular}", "\\end{table}", ""])
19805     return "\\n".join(lines)
19806
19807
19808 def export_q2_tables() -> dict:
19809     _, purchase_vals, day_q, day_fee = load_purchase_row(RESULT2, "计划购电量", SAMPLE_DATE)
19810     blocks, soc_start, soc_end = load_charge_blocks(RESULT2, SAMPLE_DATE)
19811     emergency = load_emergency_segments(RESULT2, SAMPLE_DATE)
19812     cost = pd.read_csv(Q2_COST)
19813     k_df = pd.read_csv(Q2_K)
19814     feb = cost[cost["label"] == "feb_dec_inherited_warmup_soc"].iloc[0]
19815     jan = cost[cost["label"] == "january_warmup_main"].iloc[0]
19816
19817     purchase_rows = [(slot, fmt_num(purchase_vals[slot])) for slot in SAMPLE_SLOTS]
19818     purchase_tex = render_triplet_table(
19819         f"问题二 {SAMPLE_DATE} 计划购电量及全天汇总 (政策一致 $K=8$ 主方案)",
19820         "tab:q2-purchase",
19821         "购电量",
19822         purchase_rows,
19823         footer=("全天购电量", fmt_num(day_q), "全天购电费", fmt_num(day_fee)),
19824     )
19825
19826     storage_lines = [
19827         "\\begin{table}[htbp]",
19828         "\\centering",
19829         f"\\caption{{{问题二 {SAMPLE_DATE} 充放电量及日初/日末储电量}}}{\\label{{{tab:q2-storage}}}}",
19830         "\\normalsize\\setlength{\\tabcolsep}{5pt}",
19831         "\\begin{tabular}{@{}cccccc@{}}",
19832         "\\toprule",
19833         "时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 \\",
19834         "\\midrule",
19835     ]
19836     for i in range(0, 6, 2):
19837         b0, b1 = blocks[i], blocks[i + 1]
19838         storage_lines.append(
19839             f"{b0[0]} & {fmt_num(b0[1])} & {fmt_num(b0[2])} & "
19840             f"{b1[0]} & {fmt_num(b1[1])} & {fmt_num(b1[2])} \\",
19841         )
19842     storage_lines.extend(

```



```

19843     [
19844         f"0:00 储电量 & \\multicolumn{{2}}{{c}}{{fmt_num(soc_start)}}} & "
19845         f"24:00 储电量 & \\multicolumn{{2}}{{c}}{{fmt_num(soc_end)}}} \\",
19846         "\\bottomrule",
19847         "\\end{tabular}",
19848         "\\end{table}",
19849         "",
19850     ]
19851 )
19852 storage_tex = "\\n".join(storage_lines)
19853
19854 emergency_lines = [
19855     "\\begin{table}[htbp]",
19856     "\\centering",
19857     f"\\caption{{问题二 {SAMPLE_DATE} 紧急购电记录 (2--12 月输出区间内典型日) }}\\label{{tab:q2-emergency}}",
19858     "\\normalsize\\setlength{{\\tabcolsep}}{{6pt}}",
19859     "\\begin{tabular}{{@{{ccc@{{}}}}",
19860     "\\toprule",
19861     "购电时间段 & 购电量 & 备注 \\",
19862     "\\midrule",
19863 ]
19864 total_e = 0.0
19865 for interval, amount in emergency[:8]:
19866     total_e += amount
19867     emergency_lines.append(f"{{interval}} & {{fmt_num(amount)}} & 紧急购电 \\")
19868 if len(emergency) > 8:
19869     emergency_lines.append(
19870         f"\\multicolumn{{3}}{{c}}{{其余 {len(emergency) - 8} 条记录见随附正式工作簿}} \\")
19871 )
19872 emergency_lines.extend(
19873     [
19874         f"\\midrule",
19875         f"当日合计 & {{fmt_num(sum(x for _, x in emergency))}} & kWh \\",
19876         "\\bottomrule",
19877         "\\end{tabular}",
19878         "\\end{table}",
19879         "",
19880     ]
19881 )
19882 emergency_tex = "\\n".join(emergency_lines)
19883
19884 cost_tex = f"\\begin{{table}}[htbp]
19885 \\centering
19886 \\caption{{问题二年度成本分解 (政策一致 $K=8$; 2--12 月为题设输出区间) }}\\label{{tab:q2-cost-summary}}
19887 \\normalsize\\setlength{{\\tabcolsep}}{{6pt}}
19888 \\begin{{tabular}}>{{@{{}}lrrrr@{{}}}}
19889 \\toprule
19890 统计口径 & 天数 & 计划购电费/元 & 紧急购电费/元 & 总成本/元 \\
19891 \\midrule
19892 1 月预热 (不计入题设输出) & 31 & {{fmt_num(jan.planned_cost_yuan, 2)}} & {{fmt_num(jan.emergency_cost_yuan, 2)}} & {{fmt_num(jan.
19893     total_cost_yuan, 2)}} \\
19894 2--12 月正式输出 & 334 & {{fmt_num(feb.planned_cost_yuan, 2)}} & {{fmt_num(feb.emergency_cost_yuan, 2)}} & {{fmt_num(feb.total_cost_yuan
19895     , 2)}} \\
19896 \\midrule
19897 2--12 月紧急购电量/kWh & \\multicolumn{{4}}{{c}}{{fmt_num(feb.emergency_kwh, 2)}} \\
19898 2 月 1 日继承 SOC/kWh & \\multicolumn{{4}}{{c}}{{fmt_num(feb.soc_start_kwh, 4)}} \\
19899 \\bottomrule
19900 \\end{{tabular}}
19901 \\end{{table}}
19902 ""
19903
19904 k_rows = []
19905 for _, row in k_df.iterrows():
19906     conclusion = "主方案" if int(row.scenario_k) == 8 else "对照"
19907     k_rows.append(
19908         f"${K}={{int(row.scenario_k)}}$ & {{int(row.n_days)}} & "
19909         f"{{fmt_num(row.total_cost_yuan, 2)}} & {{fmt_num(row.emergency_kwh, 2)}} & "
19910         f"{{fmt_num(row.elapsed_seconds, 1)}} & {{conclusion}} \\")

```

```

19909         )
19910         param_tex = f"""\begin{{table}}[htbp]
19911         \\centering
19912         \\caption{{问题二情景数 $K$ 的闭环复核 (2--12 月输出区间; 正式主方案取 $K=8$) }}\\label{{tab:q2-parameter}}
19913         \\small\\setlength{{\\tabcolsep}}{{3pt}}
19914         \\begin{{tabular}}{{@{{}}crrrrl@{{}}}}
19915         \\toprule
19916         $K$ & 天数 & 总成本/元 & 紧急购电/kWh & 耗时/s & 结论 \\\\
19917         \\midrule
19918         {chr(10).join(k_rows)}\\bottomrule
19919         \\end{{tabular}}
19920         \\end{{table}}
19921         """
19922
19923         write_text(TABLES_Q2 / "purchase.tex", purchase_tex)
19924         write_text(TABLES_Q2 / "storage.tex", storage_tex)
19925         write_text(TABLES_Q2 / "emergency.tex", emergency_tex)
19926         write_text(TABLES_Q2 / "cost_summary.tex", cost_tex)
19927         write_text(TABLES_Q2 / "parameter_selection.tex", param_tex)
19928         return {
19929             "sample_date": SAMPLE_DATE,
19930             "feb_dec_total_yuan": float(feb.total_cost_yuan),
19931             "emergency_segments": len(emergency),
19932         }
19933
19934
19935     def export_q3_tables() -> dict:
19936         _, g0_vals, day_g0, _ = load_purchase_row(RESULT3, "计划购电量", SAMPLE_DATE)
19937         g_vals = load_adjust_row(RESULT3, SAMPLE_DATE)
19938         blocks, soc_start, soc_end = load_charge_blocks(RESULT3, SAMPLE_DATE)
19939         emergency = load_emergency_segments(RESULT3, SAMPLE_DATE)
19940         strategy = pd.read_csv(Q3_STRATEGY)
19941         a = strategy[
19942             (strategy["strategy"].isin(["M0", "M1_M6"]))
19943             & (strategy["year_end_boundary"] == "A_q2_aligned")
19944         ].set_index("strategy")
19945
19946     def build_value_table(caption: str, label: str, col_title: str, mapping: dict[str, float]) -> str:
19947         rows = [(slot, fmt_num(mapping[slot])) for slot in SAMPLE_SLOTS]
19948         lines = [
19949             "\\begin{{table}}[htbp]",
19950             "\\centering",
19951             f"\\caption{{{{caption}}}}\\label{{{{label}}}}",
19952             "\\normalsize\\setlength{{\\tabcolsep}}{{5pt}}",
19953             "\\begin{{tabular}}{{@{{}}cccc@{{}}}}",
19954             "\\toprule",
19955             f"时间段 & {{col_title}} & 时间段 & {{col_title}} & 时间段 & {{col_title}} \\\\",
19956             "\\midrule",
19957         ]
19958         for i in range(0, 6, 3):
19959             chunk = rows[i : i + 3]
19960             lines.append(
19961                 f"{{chunk[0][0]}} & {{chunk[0][1]}} & {{chunk[1][0]}} & {{chunk[1][1]}} & "
19962                 f"{{chunk[2][0]}} & {{chunk[2][1]}} \\\\\\"
19963             )
19964         lines.extend(["\\bottomrule", "\\end{{tabular}}", "\\end{{table}}", ""])
19965         return "\n".join(lines)
19966
19967     purchase_tex = build_value_table(
19968         f"问题三 {{SAMPLE_DATE}} 计划购电量 $g^0$ (A 边界; 全时点价值触发策略)",
19969         "tab:q3-purchase",
19970         "计划购电量",
19971         g0_vals,
19972     )
19973     adjust_tex = build_value_table(
19974         f"问题三 {{SAMPLE_DATE}} 调整购电量 $g^F$ (A 边界; 全时点价值触发策略)",
19975         "tab:q3-adjust",
19976         "调整购电量",

```

```

19977     g_vals,
19978 )
19979
19980 storage_lines = [
19981     "\\begin{table}[htbp]",
19982     "\\centering",
19983     f"\\caption{{问题三 {SAMPLE_DATE} 充放电量及日初/日末储电量}}\\label{{tab:q3-storage}}",
19984     "\\normalsize\\setlength{{\\tabcolsep}}{{5pt}}",
19985     "\\begin{tabular}{{@{{cccccc@{}}}}",
19986     "\\toprule",
19987     "时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 \\\\",
19988     "\\midrule",
19989 ]
19990 for i in range(0, 6, 2):
19991     b0, b1 = blocks[i], blocks[i + 1]
19992     storage_lines.append(
19993         f"{{b0[0]}} & {{fmt_num(b0[1])}} & {{fmt_num(b0[2])}} & "
19994         f"{{b1[0]}} & {{fmt_num(b1[1])}} & {{fmt_num(b1[2])}} \\\\\\"
19995     )
19996     storage_lines.extend(
19997         [
19998             f"0:00 储电量 & \\multicolumn{{2}}{{c}}{{{fmt_num(soc_start)}}} & "
19999             f"24:00 储电量 & \\multicolumn{{2}}{{c}}{{{fmt_num(soc_end)}}} \\\\",
20000             "\\bottomrule",
20001             "\\end{tabular}",
20002             "\\end{table}",
20003             "",
20004         ]
20005     )
20006
20007 emergency_lines = [
20008     "\\begin{table}[htbp]",
20009     "\\centering",
20010     f"\\caption{{问题三 {SAMPLE_DATE} 紧急购电记录}}\\label{{tab:q3-emergency}}",
20011     "\\normalsize\\setlength{{\\tabcolsep}}{{6pt}}",
20012     "\\begin{tabular}{{@{{ccc@{}}}}",
20013     "\\toprule",
20014     "购电时间段 & 购电量 & 备注 \\\\",
20015     "\\midrule",
20016 ]
20017 for interval, amount in emergency[:6]:
20018     emergency_lines.append(f"interval & {{fmt_num(amount)}} & 紧急购电 \\\\")
20019 if len(emergency) > 6:
20020     emergency_lines.append(
20021         f"\\multicolumn{{3}}{{c}}{{{其余 {len(emergency) - 6} 条记录见随附正式工作簿}} \\\\\\"
20022     )
20023 emergency_lines.extend(
20024     [
20025         "\\midrule",
20026         f"当日合计 & {{fmt_num(sum(x for _, x in emergency))}} & kWh \\\\",
20027         "\\bottomrule",
20028         "\\end{tabular}",
20029         "\\end{table}",
20030         "",
20031     ]
20032 )
20033
20034 m0 = a.loc["M0"]
20035 m16 = a.loc["M1_M6"]
20036 saving = float(m0.total_cost_yuan - m16.total_cost_yuan)
20037 saving_pct = 100.0 * saving / float(m0.total_cost_yuan)
20038 compare_tex = f"""\begin{{table}}[htbp]
20039 \\centering
20040 \\caption{{问题三全年策略比较 (A 边界; 1--12 月连续运行口径)}}\\label{{tab:q3-strategy-compare}}
20041 \\small\\setlength{{\\tabcolsep}}{{3pt}}
20042 \\begin{{tabular}}{{@{{}}lrrrrr@{{}}}
20043 \\toprule
20044 策略 & 总成本/元 & 结算成本/元 & 紧急购电/元 & 紧急购电/kWh & 调整次数 \\\\

```

```

20045 \\midrule
20046 不调整 (M0) & {fmt_num(m0.total_cost_yuan, 2)} & {fmt_num(m0.settlement_cost_yuan, 2)} & {fmt_num(m0.emergency_cost_yuan, 2)} & {
      fmt_num(m0.emergency_kwh, 2)} & {int(m0.adjustment_count)} \\
20047 全时点调整 (M1/M6) & {fmt_num(m16.total_cost_yuan, 2)} & {fmt_num(m16.settlement_cost_yuan, 2)} & {fmt_num(m16.emergency_cost_yuan,
      2)} & {fmt_num(m16.emergency_kwh, 2)} & {int(m16.adjustment_count)} \\
20048 \\midrule
20049 相对 M0 节省 & \\multicolumn{5}{c}{\\{\\{fmt_num(saving, 2)} 元 (\\{fmt_num(saving_pct, 2)}\\}\\)} \\
20050 2--12 月输出区间成本 & \\multicolumn{5}{c}{\\{\\{14512748.53 元 (334 日, 非全年口径)} \\} \\
20051 \\bottomrule
20052 \\end{\\tabular}\\}
20053 \\end{\\table}\\}
20054 ""
20055
20056 write_text(TABLES_Q3 / "purchase.tex", purchase_tex)
20057 write_text(TABLES_Q3 / "adjust.tex", adjust_tex)
20058 write_text(TABLES_Q3 / "storage.tex", "\\n".join(storage_lines))
20059 write_text(TABLES_Q3 / "emergency.tex", "\\n".join(emergency_lines))
20060 write_text(TABLES_Q3 / "strategy_comparison.tex", compare_tex)
20061 return {
20062     "annual_m1m6_yuan": float(m16.total_cost_yuan),
20063     "annual_saving_yuan": saving,
20064     "export_window_yuan": 14512748.527966088,
20065 }
20066
20067
20068 def export_q2_figures() -> None:
20069     configure_style()
20070     daily = pd.read_csv(Q2_DAILY, parse_dates=["date"])
20071     daily = daily[(daily["date"] >= "2025-02-01") & (daily["date"] <= "2025-12-31")]
20072     daily["month"] = daily["date"].dt.to_period("M").astype(str)
20073     monthly = daily.groupby("month", as_index=False).agg(
20074         planned_cost_yuan=("planned_cost_yuan", "sum"),
20075         emergency_cost_yuan=("emergency_cost_yuan", "sum"),
20076     )
20077     x = np.arange(len(monthly))
20078     fig, ax = plt.subplots(figsize=(11.5, 5.2))
20079     ax.bar(x, monthly["planned_cost_yuan"] / 1e6, label="计划购电费", color=BLUE)
20080     ax.bar(
20081         x,
20082         monthly["emergency_cost_yuan"] / 1e6,
20083         bottom=monthly["planned_cost_yuan"] / 1e6,
20084         label="紧急购电费",
20085         color=RED,
20086         alpha=0.85,
20087     )
20088     ax.set_xticks(x, [m.replace("2025-", "") + "月" for m in monthly["month"]], rotation=0)
20089     ax.set_ylabel("成本 / 百万元")
20090     ax.set_title("问题二 2025 年 2--12 月月度购电成本构成 (政策一致 $K=8$)")
20091     ax.legend(frameon=False)
20092     ax.spines["top"].set_visible(False)
20093     ax.spines["right"].set_visible(False)
20094     save_figure(fig, "fig_q2_monthly_cost", FIG_Q2)
20095
20096     fig, ax = plt.subplots(figsize=(11.5, 4.8))
20097     ax.plot(daily["date"], daily["soc_end_kwh"], color=BLUE, linewidth=0.9)
20098     ax.axhline(1200, color=GRAY, linestyle="--", linewidth=0.8, label="SOC 下界")
20099     ax.axhline(10800, color=GRAY, linestyle=":", linewidth=0.8, label="SOC 上界")
20100     ax.set_ylabel("日末 SOC / kWh")
20101     ax.set_title("问题二 2025 年 2--12 月日末储电量轨迹")
20102     ax.xaxis.set_major_locator(mdates.MonthLocator())
20103     ax.xaxis.set_major_formatter(mdates.DateFormatter("%m月"))
20104     ax.legend(frameon=False, loc="upper right")
20105     ax.spines["top"].set_visible(False)
20106     ax.spines["right"].set_visible(False)
20107     save_figure(fig, "fig_q2_soc_daily", FIG_Q2)
20108
20109     cal = pd.read_csv(Q2_CAL, parse_dates=["date"])
20110     selected = (

```

```

20111         cal.groupby(["effective_start_date", "effective_end_date", "forecast_mode", "risk_alpha"], dropna=False)
20112         .size()
20113         .reset_index(name="n_days")
20114         .sort_values("effective_start_date")
20115     )
20116     fig, ax = plt.subplots(figsize=(11.5, 3.8))
20117     forecast_labels = {
20118         "m1": "历史均值",
20119         "m2": "历史中位数",
20120         "m3": "近期加权均值",
20121     }
20122     for _, row in selected.iterrows():
20123         start = pd.Timestamp(row.effective_start_date)
20124         end = pd.Timestamp(row.effective_end_date) + pd.Timedelta(days=1)
20125         alpha = "" if pd.isna(row.risk_alpha) else f", alpha={row.risk_alpha:.2f}"
20126         label = f"{forecast_labels.get(row.forecast_mode, row.forecast_mode)}{alpha}"
20127         ax.barh(label, (end - start).days, left=start, height=0.55, color=BLUE, alpha=0.75)
20128     ax.axvspan(pd.Timestamp("2025-01-01"), pd.Timestamp("2025-02-01"), color=GRAY, alpha=0.12)
20129     ax.set_xlim(pd.Timestamp("2025-01-01"), pd.Timestamp("2026-01-01"))
20130     ax.xaxis.set_major_locator(mdates.MonthLocator())
20131     ax.xaxis.set_major_formatter(mdates.DateFormatter("%m月"))
20132     ax.set_title("问题二预测结构与风险分位滚动选择日历 ($K=8$)")
20133     ax.spines["top"].set_visible(False)
20134     ax.spines["right"].set_visible(False)
20135     save_figure(fig, "fig_q2_policy_calendar", FIG_Q2)
20136
20137
20138 def export_q3_figures() -> None:
20139     configure_style()
20140     strategy = pd.read_csv(Q3_STRATEGY)
20141     subset = strategy[
20142         (strategy["year_end_boundary"] == "A_q2_aligned")
20143         & (strategy["strategy"].isin(["M0", "M1_M6", "M6_only", "M12_only", "M18_only"]))
20144     ].copy()
20145     subset = subset.sort_values("total_cost_yuan")
20146     label_map = {
20147         "M0": "不调整",
20148         "M1_M6": "全时点调整",
20149         "M6_only": "仅 6:00 更新",
20150         "M12_only": "仅 12:00 更新",
20151         "M18_only": "仅 18:00 更新",
20152     }
20153     labels = [label_map[name] for name in subset["strategy"]]
20154     planned = subset["settlement_cost_yuan"].to_numpy()
20155     emergency = subset["emergency_cost_yuan"].to_numpy()
20156     x = np.arange(len(labels))
20157     fig, ax = plt.subplots(figsize=(8.8, 5.0))
20158     ax.bar(x, planned / 1e6, label="结算成本", color=BLUE)
20159     ax.bar(x, emergency / 1e6, bottom=planned / 1e6, label="紧急购电成本", color=RED, alpha=0.85)
20160     ax.set_xticks(x, labels, rotation=15)
20161     ax.set_ylabel("成本 / 百万元")
20162     ax.set_title("问题三 A 边界下各策略总成本分解 (2025 年 1--12 月)")
20163     ax.legend(frameon=False)
20164     ax.spines["top"].set_visible(False)
20165     ax.spines["right"].set_visible(False)
20166     save_figure(fig, "fig_q3_strategy_cost", FIG_Q3)
20167
20168     monthly = pd.read_csv(Q3_MONTHLY)
20169     monthly = monthly[monthly["year_end_boundary"] == "A_q2_aligned"]
20170     months = sorted(m for m in monthly["month"].unique() if m >= "2025-02")
20171     m0 = monthly[(monthly["strategy"] == "M0").set_index("month").loc[months]["emergency_cost_yuan"]]
20172     m16 = monthly[(monthly["strategy"] == "M1_M6").set_index("month").loc[months]["emergency_cost_yuan"]]
20173     x = np.arange(len(months))
20174     width = 0.36
20175     fig, ax = plt.subplots(figsize=(11.5, 5.0))
20176     ax.bar(x - width / 2, m0 / 1e4, width=width, label="不调整 (M0)", color=GRAY)
20177     ax.bar(x + width / 2, m16 / 1e4, width=width, label="全时点调整 (M1/M6)", color=BLUE)
20178     ax.set_xticks(x, [m.replace("2025-", "") + "月" for m in months])

```

```

20179     ax.set_ylabel("紧急购电成本 / 万元")
20180     ax.set_title("问题三 2025 年 2--12 月紧急购电成本月度对比")
20181     ax.legend(frameon=False)
20182     ax.spines["top"].set_visible(False)
20183     ax.spines["right"].set_visible(False)
20184     save_figure(fig, "fig_q3_monthly_emergency", FIG_Q3)
20185
20186     updates = pd.read_csv(Q3_UPDATE, parse_dates=["date"])
20187     updates = updates[
20188         (updates["strategy"] == "M1_M6")
20189         & (updates["year_end_boundary"] == "A_q2_aligned")
20190         & (updates["update_time"].isin(["06:00", "12:00", "18:00"]))
20191         & (updates["implemented"] == True)
20192     ].copy()
20193     updates["month"] = updates["date"].dt.to_period("M").astype(str)
20194     monthly_voi = updates.groupby("month")["voi_yuan"].sum()
20195     months_voi = [m for m in sorted(monthly_voi.index) if m >= "2025-02"]
20196     fig, ax = plt.subplots(figsize=(11.5, 4.6))
20197     ax.bar(
20198         np.arange(len(months_voi)),
20199         [monthly_voi[m] / 1e4 for m in months_voi],
20200         color=GREEN,
20201         alpha=0.85,
20202     )
20203     ax.set_xticks(np.arange(len(months_voi)), [m.replace("2025-", "") + "月" for m in months_voi])
20204     ax.set_ylabel("信息价值 / 万元")
20205     ax.set_title("问题三全时点价值触发策略的月度信息价值 (2025 年 2--12 月)")
20206     ax.spines["top"].set_visible(False)
20207     ax.spines["right"].set_visible(False)
20208     save_figure(fig, "fig_q3_update_value", FIG_Q3)
20209
20210
20211 def main() -> None:
20212     for path in (RESULT2, RESULT3, Q2_DAILY, Q2_COST, Q3_STRATEGY):
20213         if not path.exists():
20214             raise FileNotFoundError(path)
20215     q2_meta = export_q2_tables()
20216     q3_meta = export_q3_tables()
20217     export_q2_figures()
20218     export_q3_figures()
20219     audit = {"q2": q2_meta, "q3": q3_meta}
20220     write_text(PAPER / "paper_assets_audit.json", json.dumps(audit, indent=2, ensure_ascii=False) + "\n")
20221     print(json.dumps(audit, indent=2, ensure_ascii=False))
20222
20223
20224 if __name__ == "__main__":
20225     main()
20226
20227 # ===== scripts/export_paper_q4_2.py =====
20228 #!/usr/bin/env python3
20229 """Export Q4-2 paper tables and figures from signed-off outputs."""
20230
20231 from __future__ import annotations
20232
20233 import json
20234 import sys
20235 from pathlib import Path
20236
20237 import matplotlib.pyplot as plt
20238 import pandas as pd
20239
20240 ROOT = Path(__file__).resolve().parents[1]
20241 if str(ROOT) not in sys.path:
20242     sys.path.insert(0, str(ROOT))
20243
20244 from scripts.export_paper_q2_q3 import ( # noqa: E402
20245     SAMPLE_DATE,
20246     SAMPLE_SLOTS,

```

```

20247     configure_style,
20248     fmt_num,
20249     load_charge_blocks,
20250     load_purchase_row,
20251     render_triplet_table,
20252     save_figure,
20253     write_text,
20254 )
20255
20256 RESULT42 = ROOT / "output" / "result4-2.xlsx"
20257 WARMUP = ROOT / "output" / "q4" / "q4_2_warmup_daily.csv"
20258 K_REVIEW = ROOT / "output" / "q4" / "q4_2_k_review.json"
20259 PRICE_MONTHLY = ROOT / "output" / "q4" / "q4_price_forecast_monthly.csv"
20260 META = ROOT / "output" / "q4" / "q4_2_run_metadata.json"
20261 TABLES = ROOT / "paper" / "manuscript" / "tables" / "q4"
20262 FIGURES = ROOT / "paper" / "overleaf" / "figures" / "q4"
20263
20264
20265 def load_cost_slices() -> tuple[pd.Series, pd.Series, float]:
20266     daily = pd.read_csv(WARMUP)
20267     daily["date"] = pd.to_datetime(daily["date"])
20268     jan = daily[daily["date"].dt.month == 1]
20269     feb_dec = daily[(daily["date"] >= "2025-02-01") & (daily["date"] <= "2025-12-31")]
20270     feb1_soc = float(feb_dec.iloc[0]["soc_start_kwh"])
20271     return jan.sum(numeric_only=True), feb_dec.sum(numeric_only=True), feb1_soc
20272
20273
20274 def export_tables() -> dict:
20275     TABLES.mkdir(parents=True, exist_ok=True)
20276     jan, feb, feb1_soc = load_cost_slices()
20277     meta = json.loads(META.read_text(encoding="utf-8"))
20278     k_meta = json.loads(K_REVIEW.read_text(encoding="utf-8"))
20279     k_rows = []
20280     for k in k_meta["k_candidates"]:
20281         row = k_meta["by_k"][str(k)]
20282         if k == 4:
20283             conclusion = "低场景数对照"
20284         elif k == k_meta["decision"]["keep_k"]:
20285             conclusion = "主方案（稳定折中）"
20286         else:
20287             conclusion = "高场景数对照"
20288         k_rows.append(
20289             f"${k}={k} & {int(row['n_windows'])} & "
20290             f"{fmt_num(row['total_cost_yuan'], 2)} & "
20291             f"{fmt_num(row['total_emergency_fee_yuan'], 2)} & "
20292             f"{fmt_num(row['mean_elapsed_seconds'], 1)} & {conclusion} \\\\"
20293         )
20294     _, purchase_vals, day_q, day_fee = load_purchase_row(RERESULT42, "计划购电量", SAMPLE_DATE)
20295     blocks, soc_start, soc_end = load_charge_blocks(RERESULT42, SAMPLE_DATE)
20296
20297     purchase_rows = [(slot, fmt_num(purchase_vals[slot])) for slot in SAMPLE_SLOTS]
20298     purchase_tex = render_triplet_table(
20299         f"问题四日前额度策略 (Q4-2) {SAMPLE_DATE} 计划购电量及全天汇总 ($K=8$) ",
20300         "tab:q4-purchase",
20301         "购电量",
20302         purchase_rows,
20303         footer=("全天购电量", fmt_num(day_q), "全天购电费", fmt_num(day_fee)),
20304     )
20305
20306     storage_lines = [
20307         "\\begin{table}[htbp]",
20308         "\\centering",
20309         f"\\caption{{问题四日前额度策略 (Q4-2) {SAMPLE_DATE} 充放电量及日初/日末储电量}}\\label{{tab:q4-storage}}",
20310         "\\normalsize\\setlength{{\\tabcolsep}}{{5pt}}",
20311         "\\begin{tabular}{{@{}cccccc@{}}}",
20312         "\\toprule",
20313         "时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 \\\\",
20314         "\\midrule",

```

```

20315 ]
20316 for i in range(0, 6, 2):
20317     b0, b1 = blocks[i], blocks[i + 1]
20318     storage_lines.append(
20319         f"{b0[0]} & {fmt_num(b0[1])} & {fmt_num(b0[2])} & "
20320         f"{b1[0]} & {fmt_num(b1[1])} & {fmt_num(b1[2])} \\\\"
20321     )
20322     storage_lines.extend(
20323         [
20324             f"0:00 储电量 & \multicolumn{{{c}}}{c}{{{fmt_num(soc_start)}}} & "
20325             f"24:00 储电量 & \multicolumn{{{c}}}{c}{{{fmt_num(soc_end)}}} \\\\",
20326             "\\bottomrule",
20327             "\\end{tabular}",
20328             "\\end{table}",
20329             "",
20330         ]
20331     )
20332
20333     cost_tex = f"""\begin{table}[htbp]
20334 \\\centering
20335 \\\caption{{问题四日前额度策略 (Q4-2) 成本分解 (交付时段实际价; 2--12 月为题设输出区间)}}\\label{{tab:q4-cost-summary}}
20336 \\\normalsize\\setlength{{\\tabcolsep}}{{6pt}}
20337 \\\begin{tabular}{{@{{}}lrrrr@{{}}}}
20338 \\\toprule
20339 统计口径 & 天数 & 计划购电费/元 & 紧急购电费/元 & 总成本/元 \\\\"
20340 \\\midrule
20341 1 月预热 (不计入题设输出) & 31 & {fmt_num(jan.normal_cost_yuan, 2)} & {fmt_num(jan.emergency_cost_yuan, 2)} & {fmt_num(jan.
20342     total_cost_yuan, 2)} \\\\"
20343 2--12 月正式输出 & 334 & {fmt_num(feb.normal_cost_yuan, 2)} & {fmt_num(feb.emergency_cost_yuan, 2)} & {fmt_num(feb.total_cost_yuan,
20344     2)} \\\\"
20345 \\\midrule
20346 2--12 月紧急购电量/kWh & \multicolumn{{{c}}}{c}{{{fmt_num(feb.emergency_kwh, 2)}}} \\\\"
20347 2 月 1 日继承 SOC/kWh & \multicolumn{{{c}}}{c}{{{fmt_num(feb1_soc, 4)}}} \\\\"
20348 \\\bottomrule
20349 \\\end{tabular}}
20350 \\\end{table}}
20351 ""
20352
20353     param_tex = f"""\begin{table}[htbp]
20354 \\\centering
20355 \\\caption{{问题四日前额度策略 (Q4-2) 情景数 $K$ 的闭环复核}}\\label{{tab:q4-parameter}}
20356 \\\small\\setlength{{\\tabcolsep}}{{3pt}}
20357 \\\begin{tabular}{{@{{}}crrrrl@{{}}}}
20358 \\\toprule
20359 $K$ & 窗口数 & 验证总成本/元 & 验证紧急费/元 & 平均耗时/s & 角色 \\\\"
20360 \\\midrule
20361 {chr(10).join(k_rows)}\\bottomrule
20362 \\\end{tabular}}
20363 \\\end{table}}
20364 ""
20365
20366     price_all = pd.read_csv(PRICE_MONTHLY)
20367     annual = price_all[price_all["year_month"] == "2025-annual"].iloc[0]
20368     monthly = price_all[price_all["year_month"] != "2025-annual"]
20369     price_rows = []
20370     for _, row in monthly.iterrows():
20371         price_rows.append(
20372             f"{row.year_month} & {fmt_num(row.day_ahead_mae, 4)} & "
20373             f"{fmt_num(row.day_ahead_rmse, 4)} & {fmt_num(row.intraday_06_mae, 4)} & "
20374             f"{fmt_num(row.intraday_18_mae, 4)} & {fmt_num(row.mpc_remaining_mae_mean, 4)} \\\\"
20375         )
20376     price_tex = f"""\begin{table}[htbp]
20377 \\\centering
20378 \\\caption{{问题四因果价格预测误差 (2025 年 1--12 月; 单位: 元/kWh)}}\\label{{tab:q4-price-error}}
20379 \\\small\\setlength{{\\tabcolsep}}{{3pt}}
20380 \\\begin{tabular}{{@{{}}lrrrrr@{{}}}}
20381 \\\toprule
20382 月份 & 日初 MAE & 日初 RMSE & 6:00 更新 MAE & 18:00 更新 MAE & 日内剩余 MAE \\\\"

```



```

20381 \\midrule
20382 {chr(10).join(price_rows)}\\midrule
20383 全年 & {fmt_num(annual.day_ahead_mae, 4)} & {fmt_num(annual.day_ahead_rmse, 4)} & {fmt_num(annual.intraday_06_mae, 4)} & {fmt_num(
    annual.intraday_18_mae, 4)} & {fmt_num(annual.mpc_remaining_mae_mean, 4)} \\\
20384 \\bottomrule
20385 \\end{{tabular}}
20386 \\end{{table}}
20387 ""
20388
20389 write_text(TABLES / "purchase.tex", purchase_tex)
20390 write_text(TABLES / "storage.tex", "\n".join(storage_lines))
20391 write_text(TABLES / "cost_summary.tex", cost_tex)
20392 write_text(TABLES / "parameter_selection.tex", param_tex)
20393 write_text(TABLES / "price_forecast_error.tex", price_tex)
20394
20395 return {
20396     "annual_cost_yuan": float(meta["annual_cost_yuan"]),
20397     "export_window_yuan": float(meta["export_window_cost_yuan"]),
20398     "feb_dec_emergency_kwh": float(feb.emergency_kwh),
20399 }
20400
20401
20402 def export_figures() -> None:
20403     FIGURES.mkdir(parents=True, exist_ok=True)
20404     configure_style()
20405     daily = pd.read_csv(WARMUP)
20406     daily["date"] = pd.to_datetime(daily["date"])
20407     feb = daily[daily["date"] >= "2025-02-01"].copy()
20408     feb["month"] = feb["date"].dt.to_period("M").astype(str)
20409     monthly = feb.groupby("month", sort=True)[["normal_cost_yuan", "emergency_cost_yuan"]].sum()
20410
20411     fig, ax = plt.subplots(figsize=(8.5, 4.2))
20412     x = range(len(monthly))
20413     ax.bar(x, monthly["normal_cost_yuan"] / 1e6, label="计划购电费")
20414     ax.bar(
20415         x,
20416         monthly["emergency_cost_yuan"] / 1e6,
20417         bottom=monthly["normal_cost_yuan"] / 1e6,
20418         label="紧急购电费",
20419     )
20420     ax.set_xticks(list(x), [m.replace("2025-", "") + "月" for m in monthly.index])
20421     ax.set_ylabel("成本 / 百万元")
20422     ax.set_title("日前额度策略月度购电成本构成 (2025 年 2--12 月)")
20423     ax.legend()
20424     fig.tight_layout()
20425     save_figure(fig, "fig_q4_monthly_cost", FIGURES)
20426
20427     price = pd.read_csv(PRICE_MONTHLY)
20428     price = price[price["year_month"] != "2025-annual"]
20429     fig, ax = plt.subplots(figsize=(8.5, 4.0))
20430     month_labels = [m.replace("2025-", "") + "月" for m in price["year_month"]]
20431     ax.plot(month_labels, price["day_ahead_mae"], marker="o", label="日初 MAE")
20432     ax.plot(month_labels, price["mpc_remaining_mae_mean"], marker="s", label="日内剩余 MAE 均值")
20433     ax.set_ylabel("误差 / 元 · kWh${-1}$")
20434     ax.set_title("因果价格预测误差 (2025 年 1--12 月)")
20435     ax.tick_params(axis="x", rotation=0)
20436     ax.legend()
20437     fig.tight_layout()
20438     save_figure(fig, "fig_q4_price_error", FIGURES)
20439
20440
20441 def main() -> None:
20442     summary = export_tables()
20443     export_figures()
20444     audit_path = ROOT / "paper" / "overleaf" / "paper_assets_audit.json"
20445     payload = {}
20446     if audit_path.exists():
20447         payload = json.loads(audit_path.read_text(encoding="utf-8"))

```

```

20448     payload["q4_2"] = summary
20449     audit_path.write_text(json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
20450     print(json.dumps(summary, indent=2))
20451
20452
20453 if __name__ == "__main__":
20454     main()
20455
20456 # ===== scripts/export_paper_q4_3.py =====
20457 #!/usr/bin/env python3
20458 """Export Q4-3 paper tables and figures from signed-off outputs."""
20459
20460 from __future__ import annotations
20461
20462 import json
20463 import sys
20464 from pathlib import Path
20465
20466 import matplotlib.pyplot as plt
20467 import numpy as np
20468 import pandas as pd
20469
20470 ROOT = Path(__file__).resolve().parents[1]
20471 if str(ROOT) not in sys.path:
20472     sys.path.insert(0, str(ROOT))
20473
20474 from scripts.export_paper_q2_q3 import ( # noqa: E402
20475     SAMPLE_DATE,
20476     SAMPLE_SLOTS,
20477     configure_style,
20478     fmt_num,
20479     load_adjust_row,
20480     load_charge_blocks,
20481     load_emergency_segments,
20482     load_purchase_row,
20483     save_figure,
20484     write_text,
20485 )
20486
20487 RESULT43 = ROOT / "output" / "result4-3.xlsx"
20488 WARMUP = ROOT / "output" / "q4" / "q4_3_warmup_daily.csv"
20489 META = ROOT / "output" / "q4" / "q4_3_run_metadata.json"
20490 Q3_STRATEGY = ROOT / "output" / "q3_full_annual" / "q3_annual_strategy_comparison.csv"
20491 TABLES = ROOT / "paper" / "manuscript" / "tables" / "q4"
20492 FIGURES = ROOT / "paper" / "overleaf" / "figures" / "q4"
20493
20494 Q3_ANNUAL_FIXED = 16_373_508.75
20495 Q3_EXPORT_FIXED = 14_512_748.53
20496
20497
20498 def load_cost_slices() -> tuple[pd.Series, pd.Series, pd.Series, float]:
20499     daily = pd.read_csv(WARMUP, parse_dates=["date"])
20500     jan = daily[daily["date"].dt.month == 1].sum(numeric_only=True)
20501     feb_frame = daily[(daily["date"] >= "2025-02-01") & (daily["date"] <= "2025-12-31")]
20502     feb = feb_frame.sum(numeric_only=True)
20503     full = daily.sum(numeric_only=True)
20504     feb1_soc = float(february_frame.iloc[0]["soc_start_kwh"])
20505     return jan, feb, full, feb1_soc
20506
20507
20508 def build_value_table(caption: str, label: str, col_title: str, mapping: dict[str, float]) -> str:
20509     rows = [(slot, fmt_num(mapping[slot])) for slot in SAMPLE_SLOTS]
20510     lines = [
20511         "\\begin{table}[htbp]",
20512         "\\centering",
20513         f"\\caption{{{caption}}}\n\\label{{{label}}}",
20514         "\\normalsize\\setlength{\\tabcolsep}{5pt}",
20515         "\\begin{tabular}{@{}ccccc@{}}",

```

```

20516         "\\toprule",
20517         f"时间段 & {col_title} & 时间段 & {col_title} & 时间段 & {col_title} \\\\",
20518         "\\midrule",
20519     ]
20520     for i in range(0, 6, 3):
20521         chunk = rows[i : i + 3]
20522         lines.append(
20523             f"{chunk[0][0]} & {chunk[0][1]} & {chunk[1][0]} & {chunk[1][1]} & "
20524             f"{chunk[2][0]} & {chunk[2][1]} \\\\"
20525         )
20526     lines.extend(["\\bottomrule", "\\end{tabular}", "\\end{table}", ""])
20527     return "\n".join(lines)
20528
20529
20530 def export_tables() -> dict:
20531     TABLES.mkdir(parents=True, exist_ok=True)
20532     jan, feb, full, feb1_soc = load_cost_slices()
20533     meta = json.loads(META.read_text(encoding="utf-8"))
20534     m16 = pd.read_csv(Q3_STRATEGY)
20535     m16 = m16[m16["strategy"] == "M1_M6"] & (m16["year_end_boundary"] == "A_q2_aligned").iloc[0]
20536
20537     _, g0_vals, _, _ = load_purchase_row(RESULT43, "计划购电量", SAMPLE_DATE)
20538     gf_vals = load_adjust_row(RESULT43, SAMPLE_DATE)
20539     blocks, soc_start, soc_end = load_charge_blocks(RESULT43, SAMPLE_DATE)
20540     emergency = load_emergency_segments(RESULT43, SAMPLE_DATE)
20541
20542     purchase_tex = build_value_table(
20543         f"问题四多时段调整策略 (Q4-3) {SAMPLE_DATE} 计划购电量 $g^0$ (交付时段实际价)",
20544         "tab:q4-3-purchase",
20545         "计划购电量",
20546         g0_vals,
20547     )
20548     adjust_tex = build_value_table(
20549         f"问题四多时段调整策略 (Q4-3) {SAMPLE_DATE} 调整购电量 $g^F$ (交付时段实际价)",
20550         "tab:q4-3-adjust",
20551         "调整购电量",
20552         gf_vals,
20553     )
20554
20555     storage_lines = [
20556         "\\begin{table}[htbp]",
20557         "\\centering",
20558         f"\\caption{{问题四多时段调整策略 (Q4-3) {SAMPLE_DATE} 充放电量及日初/日末储电量}}\\label{{tab:q4-3-storage}}",
20559         "\\normalsize\\setlength{{\\tabcolsep}}{{5pt}}",
20560         "\\begin{tabular}{{@{{cccccc@{}}}}",
20561         "\\toprule",
20562         "时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 \\\\",
20563         "\\midrule",
20564     ]
20565     for i in range(0, 6, 2):
20566         b0, b1 = blocks[i], blocks[i + 1]
20567         storage_lines.append(
20568             f"{{b0[0]}} & {{fmt_num(b0[1])}} & {{fmt_num(b0[2])}} & "
20569             f"{{b1[0]}} & {{fmt_num(b1[1])}} & {{fmt_num(b1[2])}} \\\\"
20570         )
20571     storage_lines.extend(
20572         [
20573             f"0:00 储电量 & \\multicolumn{{2}}{{{c}}}{{{{fmt_num(soc_start)}}}} & "
20574             f"24:00 储电量 & \\multicolumn{{2}}{{{c}}}{{{{fmt_num(soc_end)}}}} \\\\",
20575             "\\bottomrule",
20576             "\\end{tabular}",
20577             "\\end{table}",
20578             "",
20579         ]
20580     )
20581
20582     emergency_lines = [
20583         "\\begin{table}[htbp]",

```

```

20584         "\\centering",
20585         f"\\caption{{问题四多时点调整策略 (Q4-3) {SAMPLE_DATE} 紧急购电记录}}\\label{{tab:q4-3-emergency}}",
20586         "\\normalsize\\setlength{{\\tabcolsep}}{{6pt}}",
20587         "\\begin{{tabular}}{{@{{}}ccc@{{}}}}",
20588         "\\toprule",
20589         "购电时间段 & 购电量 & 备注 \\\\",
20590         "\\midrule",
20591     ]
20592     for interval, amount in emergency[:6]:
20593         emergency_lines.append(f"interval & {fmt_num(amount)} & 紧急购电 \\\\")
20594     if len(emergency) > 6:
20595         emergency_lines.append(
20596             f"\\multicolumn{{3}}{{c}}{{{其余 {len(emergency) - 6} 条记录见随附正式工作簿}} \\\\")
20597         )
20598     emergency_lines.extend(
20599         [
20600             "\\midrule",
20601             f"当日合计 & {fmt_num(sum(x for _, x in emergency))} & kWh \\\\",
20602             "\\bottomrule",
20603             "\\end{{tabular}}",
20604             "\\end{{table}}",
20605             "",
20606         ]
20607     )
20608
20609     cost_tex = f"\\begin{{table}}[htbp]
20610 \\centering
20611 \\caption{{问题四多时点调整策略 (Q4-3) 成本分解 (交付时段实际价; 2--12 月为题设输出区间)}}\\label{{tab:q4-3-cost-summary}}
20612 \\small\\setlength{{\\tabcolsep}}{{3pt}}
20613 \\begin{{tabular}}{{@{{}}lrrrrr@{{}}}}
20614 \\toprule
20615 统计口径 & 天数 & 普通购电/元 & 调整费/元 & 紧急购电/元 & 总成本/元 \\\\
20616 \\midrule
20617 1 月预热 (不计入题设输出) & 31 & {fmt_num(jan.normal_cost_yuan, 2)} & {fmt_num(jan.adjustment_cost_yuan, 2)} & {fmt_num(jan.
20618     emergency_cost_yuan, 2)} & {fmt_num(jan.total_cost_yuan, 2)} \\\\
20619 2--12 月正式输出 & 334 & {fmt_num(feb.normal_cost_yuan, 2)} & {fmt_num(feb.adjustment_cost_yuan, 2)} & {fmt_num(feb.
20620     emergency_cost_yuan, 2)} & {fmt_num(feb.total_cost_yuan, 2)} \\\\
20621 \\midrule
20622 2--12 月紧急购电量/kWh & \\multicolumn{{5}}{{c}}{{{fmt_num(feb.emergency_kwh, 2)}}} \\\\
20623 2--12 月调整次数 & \\multicolumn{{5}}{{c}}{{{int(feb.adjustment_count)}}} \\\\
20624 2 月 1 日继承 SOC/kWh & \\multicolumn{{5}}{{c}}{{{fmt_num(feb1_soc, 4)}}} \\\\
20625 \\bottomrule
20626 \\end{{tabular}}
20627 \\end{{table}}
20628 ""
20629
20630     full_delta = float(full.total_cost_yuan) - Q3_ANNUAL_FIXED
20631     full_pct = 100.0 * full_delta / Q3_ANNUAL_FIXED
20632     export_delta = float(feb.total_cost_yuan) - Q3_EXPORT_FIXED
20633     export_pct = 100.0 * export_delta / Q3_EXPORT_FIXED
20634     adj_share = 100.0 * float(full.adjustment_cost_yuan) / float(full.total_cost_yuan)
20635
20636     compare_tex = f"\\begin{{table}}[htbp]
20637 \\centering
20638 \\caption{{多时点调整策略在固定价与交付价机制下的成本对照}}\\label{{tab:q4-3-mechanism-compare}}
20639 \\footnotesize\\setlength{{\\tabcolsep}}{{2.5pt}}
20640 \\begin{{tabular}}{{@{{}}lrrrrr@{{}}}}
20641 \\toprule
20642 方案 & 价格机制 & 1--12 月/元 & 2--12 月输出/元 & 紧急购电/元 & 调整次数 \\\\
20643 \\midrule
20644 问题三 & 固定价附件 1 & {fmt_num(Q3_ANNUAL_FIXED, 2)} & {fmt_num(Q3_EXPORT_FIXED, 2)} & {fmt_num(m16.emergency_cost_yuan, 2)} & {int
20645     (m16.adjustment_count)} \\\\
20646 问题四调整策略 & 交付时段实际价 & {fmt_num(full.total_cost_yuan, 2)} & {fmt_num(feb.total_cost_yuan, 2)} & {fmt_num(full.
20647     emergency_cost_yuan, 2)} & {int(full.adjustment_count)} \\\\
20648 \\midrule
20649 1--12 月机制差 & \\multicolumn{{5}}{{c}}{{{fmt_num(full_delta, 2)} 元 ({fmt_num(full_pct, 2)}\\%)}; 反映价格波动权重, 非策略重选收益}}
20650     \\\\
20651 2--12 月输出区价差 & \\multicolumn{{5}}{{c}}{{{fmt_num(export_delta, 2)} 元 ({fmt_num(export_pct, 2)}\\%)} \\\\

```

```

20647 问题四调整费占全年 & \\multicolumn{{5}}{{c}}{{{fmt_num(full.adjustment_cost_yuan, 2)} 元 ({fmt_num(adj_share, 2)}\\%) }} \\\\
20648 \\bottomrule
20649 \\end{{tabular}}
20650 \\end{{table}}
20651 ""
20652
20653 write_text(TABLES / "purchase_q43.tex", purchase_tex)
20654 write_text(TABLES / "adjust_q43.tex", adjust_tex)
20655 write_text(TABLES / "storage_q43.tex", "\\n".join(storage_lines))
20656 write_text(TABLES / "emergency_q43.tex", "\\n".join(emergency_lines))
20657 write_text(TABLES / "cost_summary_q43.tex", cost_tex)
20658 write_text(TABLES / "mechanism_compare_q43.tex", compare_tex)
20659
20660 return {
20661     "annual_cost_yuan": float(meta["annual_cost_yuan"]),
20662     "export_window_yuan": float(feb.total_cost_yuan),
20663     "adjustment_count": int(full.adjustment_count),
20664     "adjustment_share_pct": adj_share,
20665 }
20666
20667
20668 def export_figures() -> None:
20669     FIGURES.mkdir(parents=True, exist_ok=True)
20670     configure_style()
20671     daily = pd.read_csv(WARMUP, parse_dates=["date"])
20672     feb = daily[daily["date"] >= "2025-02-01"].copy()
20673     feb["month"] = feb["date"].dt.to_period("M").astype(str)
20674     monthly = feb.groupby("month", sort=True)[
20675         ["normal_cost_yuan", "adjustment_cost_yuan", "emergency_cost_yuan"]
20676     ].sum()
20677
20678     fig, ax = plt.subplots(figsize=(8.8, 4.2))
20679     x = range(len(monthly))
20680     bottom = monthly["normal_cost_yuan"] / 1e6
20681     ax.bar(x, bottom, label="普通购电费")
20682     ax.bar(
20683         x,
20684         monthly["adjustment_cost_yuan"] / 1e6,
20685         bottom=bottom,
20686         label="调整费",
20687     )
20688     ax.bar(
20689         x,
20690         monthly["emergency_cost_yuan"] / 1e6,
20691         bottom=bottom + monthly["adjustment_cost_yuan"] / 1e6,
20692         label="紧急购电费",
20693     )
20694     ax.set_xticks(list(x), [m.replace("2025-", "") + "月" for m in monthly.index])
20695     ax.set_ylabel("成本 / 百万元")
20696     ax.set_title("多时点调整策略月度购电成本构成 (2025 年 2--12 月)")
20697     ax.legend(fontsize=8)
20698     fig.tight_layout()
20699     save_figure(fig, "fig_q4_3_monthly_cost", FIGURES)
20700
20701     _jan, feb_sum, full, _feb1_soc = load_cost_slices()
20702     labels = ["1--12 月", "2--12 月输出"]
20703     q3_vals = [Q3_ANNUAL_FIXED, Q3_EXPORT_FIXED]
20704     q43_vals = [float(full.total_cost_yuan), float(feb_sum.total_cost_yuan)]
20705     width = 0.35
20706     fig, ax = plt.subplots(figsize=(7.2, 4.0))
20707     ax.bar([i - width / 2 for i in range(2)], np.asarray(q3_vals) / 1e6, width, label="问题三固定价")
20708     ax.bar([i + width / 2 for i in range(2)], np.asarray(q43_vals) / 1e6, width, label="问题四交付价")
20709     ax.set_xticks([0, 1], labels)
20710     ax.set_ylabel("总成本 / 百万元")
20711     ax.set_title("多时点调整策略: 同一策略、不同价格机制")
20712     ax.legend()
20713     fig.tight_layout()
20714     save_figure(fig, "fig_q4_3_mechanism_compare", FIGURES)

```

```

20715
20716
20717 def main() -> None:
20718     summary = export_tables()
20719     export_figures()
20720     audit_path = ROOT / "paper" / "overleaf" / "paper_assets_audit.json"
20721     payload = {}
20722     if audit_path.exists():
20723         payload = json.loads(audit_path.read_text(encoding="utf-8"))
20724     payload["q4_3"] = summary
20725     audit_path.write_text(json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
20726     print(json.dumps(summary, indent=2))
20727
20728
20729 if __name__ == "__main__":
20730     main()
20731
20732 # ===== scripts/reconcile_paper_numbers.py =====
20733 #!/usr/bin/env python3
20734 """Cross-check paper/manuscript numbers against signed output/ ledgers.
20735
20736 Writes:
20737 - paper/reconciliation.md
20738 - paper/reconciliation_audit.json
20739 """
20740
20741 from __future__ import annotations
20742
20743 import json
20744 import re
20745 import sys
20746 from dataclasses import asdict, dataclass
20747 from datetime import UTC, datetime
20748 from pathlib import Path
20749
20750 import pandas as pd
20751 from openpyxl import load_workbook
20752
20753 ROOT = Path(__file__).resolve().parents[1]
20754 MANUSCRIPT = ROOT / "paper" / "manuscript"
20755 PAPER = ROOT / "paper"
20756
20757
20758 @dataclass
20759 class Row:
20760     question: str
20761     paper_location: str
20762     metric: str
20763     unit: str
20764     display_precision: str
20765     paper_value: str
20766     source_file: str
20767     source_column: str
20768     source_value: str
20769     status: str # 已核 | 不一致 | 未核
20770     note: str = ""
20771
20772
20773 def fmt(value: float, digits: int) -> str:
20774     return f"{float(value):.{digits}f}"
20775
20776
20777 def near(a: float, b: float, tol: float) -> bool:
20778     return abs(float(a) - float(b)) <= tol
20779
20780
20781 def tol_for_digits(digits: int) -> float:
20782     return 0.5 * 10 ** (-digits) + 1e-9

```

```

20783
20784
20785 def add(rows: list[Row], **kwargs) -> None:
20786     rows.append(Row(**kwargs))
20787
20788
20789 def check_numeric(
20790     rows: list[Row],
20791     *,
20792     question: str,
20793     paper_location: str,
20794     metric: str,
20795     unit: str,
20796     digits: int,
20797     paper_value: float,
20798     source_file: Path,
20799     source_column: str,
20800     source_value: float,
20801     note: str = "",
20802 ) -> None:
20803     tol = tol_for_digits(digits)
20804     status = "已核" if near(paper_value, source_value, tol) else "不一致"
20805     add(
20806         rows,
20807         question=question,
20808         paper_location=paper_location,
20809         metric=metric,
20810         unit=unit,
20811         display_precision=str(digits),
20812         paper_value=fmt(paper_value, digits),
20813         source_file=str(source_file.relative_to(ROOT)),
20814         source_column=source_column,
20815         source_value=f"{source_value:.12g}",
20816         status=status,
20817         note=note,
20818     )
20819
20820
20821 def load_q1(rows: list[Row]) -> None:
20822     summary = pd.read_csv(ROOT / "output" / "q1_summary.csv").set_index("metric")["value"]
20823     baseline = pd.read_csv(ROOT / "output" / "q1_baseline.csv").set_index("metric")["value"]
20824
20825     plan = pd.read_csv(ROOT / "output" / "q1_plan.csv")
20826     charge_sum = plan["charge_kwh"].sum()
20827     discharge_sum = plan["discharge_kwh"].sum()
20828
20829     check_numeric(
20830         rows,
20831         question="Q1",
20832         paper_location="sections/q1/05_results.tex",
20833         metric="无储能基准购电费",
20834         unit="元",
20835         digits=4,
20836         paper_value=48052.0466,
20837         source_file=ROOT / "output" / "q1_baseline.csv",
20838         source_column="baseline_purchase_cost_yuan",
20839         source_value=float(baseline["baseline_purchase_cost_yuan"]),
20840     )
20841
20842     check_numeric(
20843         rows,
20844         question="Q1",
20845         paper_location="sections/q1/05_results.tex",
20846         metric="成本节省",
20847         unit="元",
20848         digits=4,
20849         paper_value=14250.5510,
20850         source_file=ROOT / "output" / "q1_baseline.csv",
20851         source_column="absolute_cost_saving_yuan",

```

```

20851         source_value=float(baseline["absolute_cost_saving_yuan"]),
20852     )
20853     check_numeric(
20854         rows,
20855         question="Q1",
20856         paper_location="sections/q1/05_results.tex",
20857         metric="节省比例",
20858         unit="1",
20859         digits=4,
20860         paper_value=0.296565,
20861         source_file=ROOT / "output" / "q1_baseline.csv",
20862         source_column="cost_saving_ratio",
20863         source_value=float(baseline["cost_saving_ratio"]),
20864     )
20865     check_numeric(
20866         rows,
20867         question="Q1",
20868         paper_location="sections/q1/05_results.tex",
20869         metric="全天购电量",
20870         unit="kWh",
20871         digits=4,
20872         paper_value=57526.2435,
20873         source_file=ROOT / "output" / "q1_summary.csv",
20874         source_column="grid_purchase_kwh",
20875         source_value=float(summary["grid_purchase_kwh"]),
20876     )
20877     check_numeric(
20878         rows,
20879         question="Q1",
20880         paper_location="sections/q1/05_results.tex",
20881         metric="全天购电费",
20882         unit="元",
20883         digits=4,
20884         paper_value=33801.4955,
20885         source_file=ROOT / "output" / "q1_summary.csv",
20886         source_column="purchase_cost_yuan",
20887         source_value=float(summary["purchase_cost_yuan"]),
20888     )
20889     check_numeric(
20890         rows,
20891         question="Q1",
20892         paper_location="sections/q1/05_results.tex",
20893         metric="总充电量",
20894         unit="kWh",
20895         digits=4,
20896         paper_value=19842.7105,
20897         source_file=ROOT / "output" / "q1_plan.csv",
20898         source_column="sum(charge_kwh)",
20899         source_value=charge_sum,
20900     )
20901     check_numeric(
20902         rows,
20903         question="Q1",
20904         paper_location="sections/q1/05_results.tex",
20905         metric="总放电量",
20906         unit="kWh",
20907         digits=4,
20908         paper_value=17858.4394,
20909         source_file=ROOT / "output" / "q1_plan.csv",
20910         source_column="sum(discharge_kwh)",
20911         source_value=discharge_sum,
20912     )
20913     check_numeric(
20914         rows,
20915         question="Q1",
20916         paper_location="sections/q1/05_results.tex",
20917         metric="峰值购电功率",
20918         unit="kW",

```



```

20919         digits=4,
20920         paper_value=8458.8273,
20921         source_file=ROOT / "output" / "q1_summary.csv",
20922         source_column="peak_grid_kw",
20923         source_value=float(summary["peak_grid_kw"]),
20924     )
20925
20926     purchase_tex = (MANUSCRIPT / "tables/q1/purchase.tex").read_text(encoding="utf-8")
20927     slot_map = {
20928         "10:00-10:10": (3, 2),
20929         "12:00-12:10": (3, 4),
20930         "14:00-14:10": (3, 6),
20931         "16:00-16:10": (5, 2),
20932         "18:00-18:10": (5, 4),
20933         "20:00-20:10": (5, 6),
20934     }
20935     for slot in slot_map:
20936         m = re.search(rf"{re.escape(slot)} & ([0-9.]*)", purchase_tex)
20937         if not m:
20938             continue
20939         paper_val = float(m.group(1))
20940         src_row = plan[plan["time"] == slot]
20941         if src_row.empty:
20942             add(
20943                 rows,
20944                 question="Q1",
20945                 paper_location="tables/q1/purchase.tex",
20946                 metric=f"购电量 {slot}",
20947                 unit="kWh",
20948                 display_precision="4",
20949                 paper_value=fmt(paper_val, 4),
20950                 source_file="output/q1_plan.csv",
20951                 source_column=f"time={slot}/grid_purchase_kwh",
20952                 source_value="MISSING",
20953                 status="不一致",
20954                 note="计划行未找到",
20955             )
20956             continue
20957         check_numeric(
20958             rows,
20959             question="Q1",
20960             paper_location="tables/q1/purchase.tex",
20961             metric=f"购电量 {slot}",
20962             unit="kWh",
20963             digits=4,
20964             paper_value=paper_val,
20965             source_file=ROOT / "output" / "q1_plan.csv",
20966             source_column=f"time={slot}/grid_purchase_kwh",
20967             source_value=float(src_row.iloc[0]["grid_purchase_kwh"]),
20968         )
20969
20970
20971     def load_q2(rows: list[Row]) -> None:
20972         cost = pd.read_csv(ROOT / "output" / "q2_policy_consistent" / "cost_breakdown.csv")
20973         kdf = pd.read_csv(ROOT / "output" / "q2_policy_consistent" / "k_sensitivity.csv")
20974         jan = cost[cost["label"] == "january_warmup_main"].iloc[0]
20975         feb = cost[cost["label"] == "feb_dec_inherited_warmup_soc"].iloc[0]
20976         k8 = kdf[kdf["label"] == "feb_dec_k8"].iloc[0]
20977
20978         pairs = [
20979             ("sections/q2/05_results.tex", "2-12月总成本", 2, float(feb["total_cost_yuan"]), 15166538.46),
20980             ("sections/q2/05_results.tex", "2-12月计划成本", 2, float(feb["planned_cost_yuan"]), 11883687.24),
20981             ("sections/q2/05_results.tex", "2-12月紧急成本", 2, float(feb["emergency_cost_yuan"]), 3282851.22),
20982             ("tables/q2/cost_summary.tex", "1月预热总成本", 2, float(jan["total_cost_yuan"]), 1977568.32),
20983             ("tables/q2/cost_summary.tex", "2-12月紧急购电量", 2, float(feb["emergency_kwh"]), 924502.93),
20984             ("tables/q2/cost_summary.tex", "2月1日继承SOC", 4, float(feb["soc_start_kwh"]), 8390.6808),
20985             ("tables/q2/parameter_selection.tex", "K=8总成本", 2, float(k8["total_cost_yuan"]), 15166538.46),
20986             ("tables/q2/parameter_selection.tex", "K=8紧急购电量", 2, float(k8["emergency_kwh"]), 924502.93),

```

```

20987 ]
20988 for loc, metric, digits, src, paper in pairs:
20989     check_numeric(
20990         rows,
20991         question="Q2",
20992         paper_location=loc,
20993         metric=metric,
20994         unit="元或kWh",
20995         digits=digits,
20996         paper_value=paper,
20997         source_file=ROOT / "output" / "q2_policy_consistent/cost_breakdown.csv",
20998         source_column=metric,
20999         source_value=src,
21000         note="V2 政策一致主方案",
21001     )
21002
21003 audit = json.loads((ROOT / "output" / "q2_policy_consistent" / "result2_cell_audit.json").read_text())
21004 add(
21005     rows,
21006     question="Q2",
21007     paper_location="output/result2.xlsx",
21008     metric="逐格对账",
21009     unit="-",
21010     display_precision="-",
21011     paper_value="PASS" if audit.get("pass") else "FAIL",
21012     source_file="output/q2_policy_consistent/result2_cell_audit.json",
21013     source_column="pass",
21014     source_value=str(audit.get("pass")),
21015     status="已核" if audit.get("pass") else "不一致",
21016     note=f"334日模板; SHA与台账一致",
21017 )
21018
21019
21020 def load_q3(rows: list[Row]) -> None:
21021     strat = pd.read_csv(ROOT / "output" / "q3_full_annual" / "q3_annual_strategy_comparison.csv")
21022     m0 = strat[(strat["strategy"] == "M0") & (strat["year_end_boundary"] == "A_q2_aligned")].iloc[0]
21023     m1 = strat[(strat["strategy"] == "M1_M6") & (strat["year_end_boundary"] == "A_q2_aligned")].iloc[0]
21024     mb = strat[(strat["strategy"] == "M1_M6") & (strat["year_end_boundary"] == "B_energy_neutral")].iloc[0]
21025     export_audit = json.loads((ROOT / "output" / "result3_export_audit.json").read_text())
21026     export_cost = float(export_audit["xlsx"]["export_window_cost_yuan"])
21027
21028     saving = float(m0["total_cost_yuan"]) - float(m1["total_cost_yuan"])
21029     ratio = saving / float(m0["total_cost_yuan"]) * 100
21030
21031     checks = [
21032         ("sections/q3/05_results.tex", "M1_M6全年总成本", 2, float(m1["total_cost_yuan"]), 16373508.75),
21033         ("sections/q3/05_results.tex", "M0全年总成本", 2, float(m0["total_cost_yuan"]), 17109568.25),
21034         ("sections/q3/05_results.tex", "相对M0节省", 2, saving, 736059.50),
21035         ("sections/q3/05_results.tex", "节省比例", 2, ratio, 4.30),
21036         ("sections/q3/05_results.tex", "B边界成本增量", 2, float(mb["total_cost_yuan"]) - float(m1["total_cost_yuan"]), 2262.88),
21037         ("tables/q3/strategy_comparison.tex", "result3输出区间成本", 2, export_cost, 14512748.53),
21038         ("tables/q3/strategy_comparison.tex", "M1_M6结算成本", 2, float(m1["settlement_cost_yuan"]), 13602473.20),
21039         ("tables/q3/strategy_comparison.tex", "M1_M6紧急购电/kWh", 2, float(m1["emergency_kwh"]), 742322.81),
21040         ("tables/q3/strategy_comparison.tex", "调整次数", 0, float(m1["adjustment_count"]), 947),
21041     ]
21042 for loc, metric, digits, src, paper in checks:
21043     check_numeric(
21044         rows,
21045         question="Q3",
21046         paper_location=loc,
21047         metric=metric,
21048         unit="元/%/次",
21049         digits=digits,
21050         paper_value=paper,
21051         source_file=ROOT / "output/q3_full_annual/q3_annual_strategy_comparison.csv",
21052         source_column=metric,
21053         source_value=src,
21054     )

```

```

21055
21056
21057 def load_q4_2(rows: list[Row]) -> None:
21058     meta = json.loads((ROOT / "output" / "q4" / "q4_2_run_metadata.json").read_text())
21059     daily = pd.read_csv(ROOT / "output" / "q4" / "q4_2_warmup_daily.csv")
21060     daily["date"] = pd.to_datetime(daily["date"])
21061     feb = daily[(daily["date"] >= "2025-02-01") & (daily["date"] <= "2025-12-31")]
21062     q2_feb = 15166538.46
21063
21064     checks = [
21065         ("sections/q4/05_results.tex", "2-12月总成本", 2, float(meta["export_window_cost_yuan"]), 15257873.75),
21066         ("sections/q4/05_results.tex", "1-12月总成本", 2, float(meta["annual_cost_yuan"]), 17681029.10),
21067         ("sections/q4/05_results.tex", "相对Q2增加", 2, float(meta["export_window_cost_yuan"]) - q2_feb, 91335.29),
21068         ("tables/q4/cost_summary.tex", "2-12月计划成本", 2, float(feb["normal_cost_yuan"].sum()), 13213373.44),
21069         ("tables/q4/cost_summary.tex", "2-12月紧急成本", 2, float(feb["emergency_cost_yuan"].sum()), 2044500.32),
21070     ]
21071     for loc, metric, digits, src, paper in checks:
21072         check_numeric(
21073             rows,
21074             question="Q4-2",
21075             paper_location=loc,
21076             metric=metric,
21077             unit="元",
21078             digits=digits,
21079             paper_value=paper,
21080             source_file=ROOT / "output/q4/q4_2_run_metadata.json",
21081             source_column=metric,
21082             source_value=src,
21083         )
21084
21085     phys = json.loads((ROOT / "output" / "q4" / "q4_2_physical_audit.json").read_text())
21086     add(
21087         rows,
21088         question="Q4-2",
21089         paper_location="sections/q4/06_validation.tex",
21090         metric="365日物理审计",
21091         unit="-",
21092         display_precision="-",
21093         paper_value="PASS",
21094         source_file="output/q4/q4_2_physical_audit.json",
21095         source_column="n_days_pass",
21096         source_value=str(phys.get("n_days_pass")),
21097         status="已核" if phys.get("n_days_pass") == 365 else "不一致",
21098     )
21099
21100
21101 def load_q4_3(rows: list[Row]) -> None:
21102     meta_path = ROOT / "output" / "q4" / "q4_3_run_metadata.json"
21103     if not meta_path.exists():
21104         return
21105     meta = json.loads(meta_path.read_text())
21106     daily = pd.read_csv(ROOT / "output" / "q4" / "q4_3_warmup_daily.csv")
21107     daily["date"] = pd.to_datetime(daily["date"])
21108     jan = daily[daily["date"].dt.month == 1]
21109     feb = daily[(daily["date"] >= "2025-02-01") & (daily["date"] <= "2025-12-31")]
21110     full = daily.sum(numeric_only=True)
21111     q3_annual = 16373508.75
21112     q3_export = 14512748.53
21113     q42_feb = 15257873.75
21114
21115     checks = [
21116         ("sections/q4/05_results.tex", "2-12月总成本", 2, float(feb["total_cost_yuan"].sum()), 15346418.14),
21117         ("sections/q4/05_results.tex", "1-12月总成本", 2, float(meta["annual_cost_yuan"]), 17489808.24),
21118         ("sections/q4/05_results.tex", "相对Q3增加(1-12)", 2, float(meta["annual_cost_yuan"]) - q3_annual, 1116299.49),
21119         ("sections/q4/05_results.tex", "相对Q3增加(2-12)", 2, float(feb["total_cost_yuan"].sum()) - q3_export, 833669.61),
21120         ("sections/q4/05_results.tex", "相对Q4-2增加(2-12)", 2, float(feb["total_cost_yuan"].sum()) - q42_feb, 88544.39),
21121         ("tables/q4/cost_summary_q43.tex", "2-12月普通购电", 2, float(feb["normal_cost_yuan"].sum()), 12447748.75),
21122         ("tables/q4/cost_summary_q43.tex", "2-12月调整费", 2, float(feb["adjustment_cost_yuan"].sum()), 321297.70),

```

```

21123         ("tables/q4/cost_summary_q43.tex", "2-12月紧急成本", 2, float(feb["emergency_cost_yuan"].sum()), 2577371.68),
21124         ("tables/q4/cost_summary_q43.tex", "1月预热成本", 2, float(jan["total_cost_yuan"].sum()), 2143390.10),
21125         ("tables/q4/mechanism_compare_q43.tex", "全年调整次数", 0, float(full["adjustment_count"]), 1004),
21126     ]
21127     for loc, metric, digits, src, paper in checks:
21128         check_numeric(
21129             rows,
21130             question="Q4-3",
21131             paper_location=loc,
21132             metric=metric,
21133             unit="元/次",
21134             digits=digits,
21135             paper_value=paper,
21136             source_file=meta_path,
21137             source_column=metric,
21138             source_value=src,
21139         )
21140
21141     phys = json.loads((ROOT / "output" / "q4" / "q4_3_physical_audit.json").read_text())
21142     add(
21143         rows,
21144         question="Q4-3",
21145         paper_location="sections/q4/06_validation.tex",
21146         metric="365日物理审计",
21147         unit="-",
21148         display_precision="-",
21149         paper_value="PASS",
21150         source_file="output/q4/q4_3_physical_audit.json",
21151         source_column="n_days_pass",
21152         source_value=str(phys.get("n_days_pass")),
21153         status="已核" if phys.get("n_days_pass") == 365 else "不一致",
21154     )
21155
21156     info = json.loads((ROOT / "output" / "q4" / "q4_3_info_set_audit.json").read_text())
21157     add(
21158         rows,
21159         question="Q4-3",
21160         paper_location="sections/q4/06_validation.tex",
21161         metric="信息集探针",
21162         unit="-",
21163         display_precision="-",
21164         paper_value="PASS",
21165         source_file="output/q4/q4_3_info_set_audit.json",
21166         source_column="all_pass",
21167         source_value=str(info.get("all_pass")),
21168         status="已核" if info.get("all_pass") else "不一致",
21169     )
21170
21171
21172     def render_markdown(rows: list[Row]) -> str:
21173         now = datetime.now(UTC).strftime("%Y-%m-%d %H:%M UTC")
21174         passed = sum(1 for r in rows if r.status == "已核")
21175         failed = [r for r in rows if r.status == "不一致"]
21176         lines = [
21177             "# 论文数字对账单 (ACCT) ",
21178             "",
21179             f"> 生成时间: {now}; 脚本: `scripts/reconcile_paper_numbers.py`",
21180             "",
21181             "## 签收摘要",
21182             "",
21183             f"- 核对条目: **{len(rows)}**",
21184             f"- **已核: {passed}**",
21185             f"- **不一致: {len(failed)}**",
21186             f"- **未核: {sum(1 for r in rows if r.status == '未核')}**",
21187             "",
21188             "| 小问 | 正式口径 | 台账/工作簿 | ACCT 状态 |",
21189             "| --- | --- | --- | --- |",
21190             "| Q1 | M1, 表2公共母线侧 | `output/result1.xlsx`、`q1_summary.csv` | 已核 |",

```

```

21191         "| Q2 | **V2 政策一致 K=8** | `output/result2.xlsx`、`q2_policy_consistent/` | "
21192         + ("已核" if not any(r.question == "Q2" and r.status == "不一致" for r in rows) else "存在不一致")
21193         + " |",
21194         "| Q3 | A边界 + `M1_M6` | `q3_annual_strategy_comparison.csv`、`result3.xlsx` | "
21195         + ("已核" if not any(r.question == "Q3" and r.status == "不一致" for r in rows) else "存在不一致")
21196         + " |",
21197         "| Q4-2 | 波动电价 Q2 链 | `result4-2.xlsx`、`q4_2_` | "
21198         + ("已核" if not any(r.question == "Q4-2" and r.status == "不一致" for r in rows) else "存在不一致")
21199         + " |",
21200         "| Q4-3 | 固定 \\texttt{M1\\M6} + 交付价 | `result4-3.xlsx`、`q4_3_` | "
21201         + ("已核" if not any(r.question == "Q4-3" and r.status == "不一致" for r in rows) else "存在不一致")
21202         + " |",
21203         "",
21204         "## 口径提醒（写入摘要前必读）",
21205         "",
21206         "- Q2: 2-12 月 **15,166,538.46 元** 为题设输出区间；1 月预热 **1,977,568.32 元** 须分列，不得相加混报。",
21207         "- Q3: 全年 **16,373,508.75 元** 为 1-12 月主结论；**14,512,748.53 元** 仅为 `result3.xlsx` 的 2-12 月核对区间。",
21208         "- Q4-2: 2-12 月 **15,257,873.75 元** 为题设输出区间；与 Q2 V2 同区间差 **91,335.29 元**。",
21209         "- Q4-3: 1-12 月 **17,489,808.24 元**；2-12 月 **15,346,418.14 元**。与 Q3 固定价比较须注明价格机制差异，不得混报口径。",
21210         "",
21211         "## 逐条对账",
21212         "",
21213         "| 小问 | 论文位置 | 指标 | 单位 | 展示精度 | 论文值 | 源文件 | 源列/字段 | 源值 | 状态 | 备注 |",
21214         "| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |",
21215     ]
21216     for r in rows:
21217         lines.append(
21218             f"| {r.question} | `{r.paper_location}` | {r.metric} | {r.unit} | {r.display_precision} | "
21219             f"{r.paper_value} | `{r.source_file}` | {r.source_column} | {r.source_value} | **{r.status}** | {r.note} |"
21220         )
21221     if failed:
21222         lines.extend(["", "## 不一致项（须修正后再签收）", ""])
21223     for r in failed:
21224         lines.append(
21225             f"- **{r.question}** `{r.paper_location}` · {r.metric}: 论文 {r.paper_value} != 源 {r.source_value}"
21226         )
21227     else:
21228         lines.extend(
21229             [
21230                 "",
21231                 "## 结论",
21232                 "",
21233                 "Q1-Q4 正文与表格数字均可在展示精度内回链至正式 `output/` 台账。",
21234                 "",
21235                 "## ACCT 签收清单",
21236                 "",
21237                 "- [x] Q1-Q3 逐条对账（本文件 + `reconciliation_audit.json`）",
21238                 "- [x] Q1 专用视图: `paper/q1_reconciliation.md`",
21239                 "- [x] Q4-2 逐条对账",
21240                 "- [x] Q4-3 逐条对账",
21241                 "- [ ] 摘要/评价写作时再次引用本对账单",
21242                 "",
21243                 "**复跑: ** `python3 scripts/reconcile_paper_numbers.py`（退出码 0 = 全部已核）",
21244             ]
21245         )
21246     return "\n".join(lines) + "\n"
21247
21248
21249 def main() -> int:
21250     rows: list[Row] = []
21251     load_q1(rows)
21252     load_q2(rows)
21253     load_q3(rows)
21254     load_q4_2(rows)
21255     load_q4_3(rows)
21256
21257     audit_path = PAPER / "reconciliation_audit.json"
21258     md_path = PAPER / "reconciliation.md"

```

```

21259     payload = {
21260         "generated_at": datetime.now(UTC).isoformat().replace("+00:00", "Z"),
21261         "rows": [asdict(r) for r in rows],
21262         "summary": {
21263             "total": len(rows),
21264             "passed": sum(1 for r in rows if r.status == "已核"),
21265             "failed": sum(1 for r in rows if r.status == "不一致"),
21266         },
21267     }
21268     audit_path.write_text(json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
21269     md_path.write_text(render_markdown(rows), encoding="utf-8")
21270
21271     print(json.dumps(payload["summary"], indent=2))
21272     failed = [r for r in rows if r.status == "不一致"]
21273     if failed:
21274         for r in failed:
21275             print(f"FAIL {r.question} {r.metric}: paper={r.paper_value} source={r.source_value}", file=sys.stderr)
21276         return 1
21277     return 0
21278
21279
21280 if __name__ == "__main__":
21281     raise SystemExit(main())

```